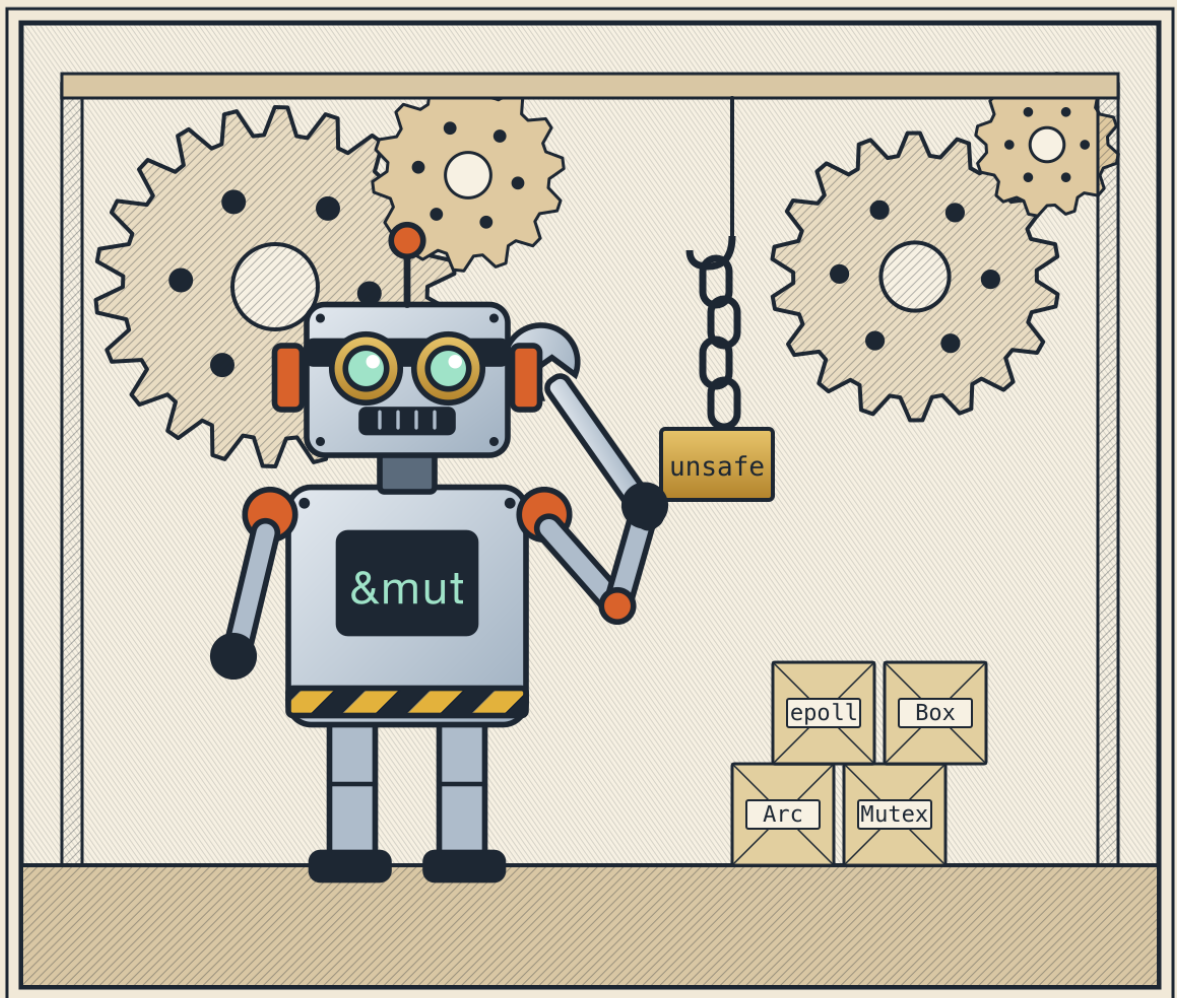


Cracking the Systems Programming Interview

Rust from data structures to the kernel boundary



Chief Mechanic of the Ferrous Works, Borrow-Checker Division

Arpan Pathak

IN RUST

Cracking the Systems Programming Interview

Rust from data structures to the kernel boundary

Arpan Pathak

github.com/arpanpathak/cracking-the-systems-programming-interview

Contents

<i>About this book</i>	5
<i>About the cover</i>	9
PART 1: FIRST STEPS	10
1 Bindings, functions, and loops	11
2 Files, paths, and the command line	27
PART 2: COLLECTIONS AND ALGORITHMS	48
3 Vectors, strings, and hash maps	49
4 Iterators and references	91
5 Heaps	104
6 Recursion and dynamic programming	123
PART 3: MODELING DATA AND OWNERSHIP	139
7 Types that refuse bad input	140
8 Ownership and the pointer types	163
9 Linked lists	178
10 Merging k sorted lists	212
11 Trees and tries	245
12 Graphs	280
PART 4: MEMORY AND THE MACHINE	327
13 An LRU cache	328
14 Shards and consistent hashing	368
15 Memory, CPU caches, and the operating system	388
PART 5: CONCURRENCY	416
16 Threads, atomics, and locks	417
17 Queues and bounded buffers	457
18 Thread pools	495

19	Rate limits, retries, and idempotency	525
PART 6: THE NETWORK		548
20	Addresses, windows, and echo servers	549
21	Parsing and serving HTTP/1.1	567
22	Async Rust from the executor up	587
PART 7: COMPACT IMPLEMENTATIONS		608
23	Compact implementations	609
<i>Appendix A. The Rust cheat sheet.....</i>		<i>623</i>
<i>Appendix B. The benchmark report.....</i>		<i>635</i>

About this book

This book is about systems programming in Rust.

By systems programming I mean programs that work close to the computer's hardware and operating system. Data structures that decide how bytes are laid out in memory are one example. Threads that share work are another, and so are servers that read requests from a network connection. For programs like these, you need to know what happens underneath your code. That includes how memory is allocated and how the processor reads it. It also includes how the operating system runs threads and moves data across a network.

I use Rust because it lets you control these details. Its compiler also checks how your program uses memory and threads before the program runs. When the compiler rejects a program in this book, I explain what it found.

The book starts with small things and builds up. The first chapters cover bindings, files, and collections. The middle chapters build data structures and look at how they use memory. The later chapters add threads, and then the network. In each chapter I introduce an idea in plain terms and work through a small example by hand. Only then do I show the code, a few lines at a time. Each chapter ends with the complete program.

Some programs appear in more than one version. When an early version has a problem, I show how the problem was found and what the next version changes.

Who should read this book

This book is for a programmer who has written some Rust and wants to use it for systems work.

You should know `fn`, `let`, `if`, `match`, `struct`, `enum`, `Vec`, and `String`. You should have met ownership and borrowing, for example in *The Rust Programming Language*. You do not need to know lifetimes in signatures, trait implementations such as `TryFrom`, `Rc`, `Arc`, atomics, `unsafe`, or `async`. Each of these is introduced in the chapter that first needs it.

No knowledge of operating systems or networking is assumed. When a chapter relies on a hardware or kernel detail, such as a cache line or a page fault, it explains that detail first.

What you will learn

After working through the book, you will be able to

- use Rust's collections and iterators to solve problems on arrays, strings, and maps
- design types that reject invalid values where input enters a program
- implement linked lists, trees, graphs, and caches in safe Rust
- choose between `Box`, `Rc`, and a `Vec` of nodes to store linked data
- measure a data structure with a benchmark you can trust
- explain how CPU caches, false sharing, and page faults affect a running program
- build locks, queues, and thread pools from atomics and condition variables
- parse a network protocol safely and write a server with threads and with `epoll`
- explain how an `async` executor runs a `Future` to completion

How this book is organized

The book has seven parts. Each part builds on the ones before it.

Part 1, First steps

- Chapter 1 covers bindings, shadowing, functions, recursion, and loops, and times three ways of computing Fibonacci numbers.
- Chapter 2 covers files, directories, and command-line arguments, and introduces `Result` and `?`.

Part 2, Collections and algorithms

- Chapter 3 solves problems with vectors, strings, and hash maps.
- Chapter 4 explains iterators and references, and introduces lifetimes.
- Chapter 5 uses heaps and sorted collections.
- Chapter 6 covers recursion and dynamic programming.

Part 3, Modeling data and ownership

- Chapter 7 builds types that reject invalid values, with structs, enums, and error types.
- Chapter 8 covers the pointer types `Box`, `Rc`, `RefCell`, `Weak`, `Arc`, and `Cow`.

- Chapter 9 builds linked lists.
- Chapter 10 merges many sorted lists into one.
- Chapter 11 covers trees and tries.
- Chapter 12 covers graphs.

Part 4, Memory and the machine

- Chapter 13 builds an LRU cache and measures two ways of storing its nodes.
- Chapter 14 splits a cache across locks and across machines.
- Chapter 15 covers allocation, CPU caches, paging, scheduling, and system calls.

Part 5, Concurrency

- Chapter 16 covers threads, atomics, and locks.
- Chapter 17 builds queues that threads use to pass work.
- Chapter 18 builds thread pools.
- Chapter 19 covers rate limiting, retries, and operations that are safe to repeat.

Part 6, The network

- Chapter 20 covers addresses, TCP, and two designs for a server.
- Chapter 21 parses and serves HTTP/1.1.
- Chapter 22 builds an async executor and two HTTP clients.

Part 7, Compact implementations

- Chapter 23 compares short versions of earlier programs with their full versions.

Appendix A lists common Rust compile errors and their fixes. Appendix B is the report on the benchmarks in chapters 9 and 13.

Conventions

Code in the text looks like `this`. Programs are introduced in numbered listings, usually a few lines at a time. Each caption names the source file and the lines it shows. At the end of each chapter, the complete program appears as one listing.

Output from a program follows the command that produced it. Timings name the machine they were measured on. They will differ on yours.

A term is printed in **bold** where it is defined.

Three kinds of callouts appear in the text:

NOTE: Background or detail you do not need to follow the main text.

TIP: A technique you can apply directly.

WARNING: A mistake that compiles, or behavior that differs from what you might expect.

Each chapter ends with a summary and a few exercises.

About the code

The programs are in the `rust-interview-lab` directory of the repository at <https://github.com/arpnpathak/cracking-the-systems-programming-interview>. They use Rust edition 2024 and were built with `rustc 1.96.0`. The listings are taken from the source files without changes.

Library code with unit tests is in `src/problems/`. Runnable programs are in `src/bin/`. The data structures measured in chapters 9 and 13 are in `benchmarking_examples/`.

Run the programs from the `rust-interview-lab` directory:

```
cargo test --lib           # the library's unit tests
cargo test --bin http_server # the tests inside one program
cargo run --bin tcp_echo_server # run one program
cargo run --release --bin benchmark # benchmarks, always in release mode
```

Building the book

The book's source is in the `cracking-the-systems-programming-interview` directory:

```
make art           # the chapter plates and the cover (needs rsvg-convert)
make figures       # the diagrams (needs Graphviz)
make html          # the browser edition, in book/
make pdf           # the print edition, in build/
make serve         # the browser edition at http://localhost:3000
```

About the cover

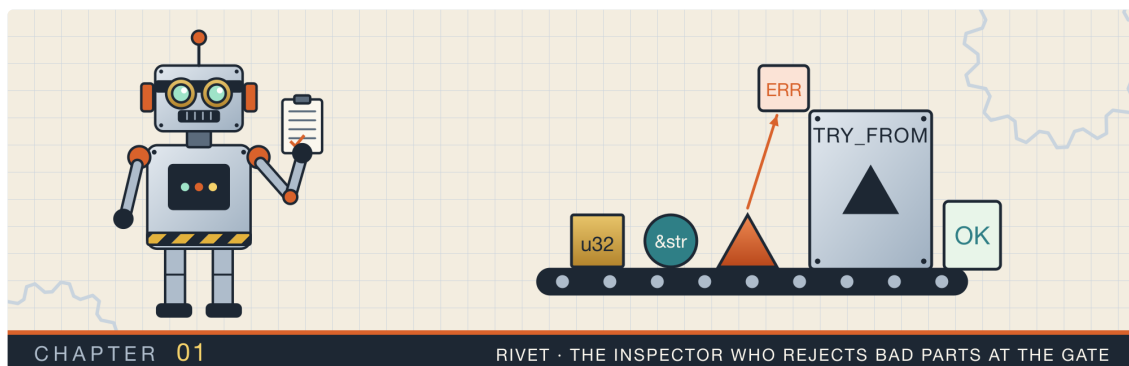
The cover shows the Chief Mechanic of the Ferrous Works in a workshop of gears. The crates are labelled `Arc`, `Mutex`, `epoll`, and `Box`, and the chain ends in a crate marked `unsafe`. Each chapter opens with a member of the same crew at a machine that represents the chapter's subject.

Chapter	Mechanic	Machine
1	Rivet	a gate marked <code>TRY_FROM</code> that rejects parts of the wrong shape
2	Ledger	a filing cabinet, a stack of pages, and an offset counter
3	Bins	numbered bins fed by a hashing hopper
4	Shackle	a chain on a crane, one link opened with <code>take()</code>
5	Conveyor	three belts feeding a heap and one output belt
6	Branch	a manifold of pipes laid out as a binary tree
7	Valve	pipes and valves wired into a weighted graph
8	Cog	a train of gears, each half the size of the last
9	Drawer	a rack of crates, the oldest leaving by an eviction chute
10	Turntable	a ring of shards around a hub gear
11	Gauge	a board of page frames and a cache-miss gauge
12	Spindle	thread spools on one axle, and a padlock
13	Carousel	a conveyor queue and a ring-buffer carousel
14	Foreman	a job hopper feeding four worker robots
15	Throttle	a token bucket under a refill valve
16	Signal	a radio mast and a socket panel
17	Courier	a pneumatic tube delivering a request to status-code slots
18	Flywheel	a flywheel driven by <code>poll</code> , with futures on a rail
19	Drill	a drill press and a checklist

The plates are generated by `tools/art.py`.

PART 1

First steps



Bindings, functions, and loops

THIS CHAPTER COVERS

- What a `let` binding is and where its value lives
- Shadowing a name, and how it differs from changing a value with `mut`
- Writing a function that calls itself, and what happens on the stack when it does
- Replacing that recursion with a loop, and with a table of saved answers
- Timing code with `Instant`, and passing a function to another function

This chapter begins Part 1 of the book. Part 1 covers the pieces of Rust that every later chapter uses. This first chapter works with the smallest pieces: names for values, functions, and loops.

You will read and run two short programs. The first gives one name to three different values in turn. It shows what a `let` statement does, and how that differs from changing a value in place. The second program computes Fibonacci numbers in three ways and times each one. It shows how a function that calls itself uses memory, and it compares the running times of the three versions.

A few terms first:

- A **binding** is a name attached to a value. `let x = 5;` binds the name `x` to the value `5`.
- A **scope** is the region of code where a binding can be used. In Rust, a scope is usually a block between `{` and `}`.
- The **stack** is the memory where a function keeps its local values while it runs.
- A **stack frame** is the part of the stack that belongs to one running call of a function.

1.1 A binding is a name for a value

When a function starts, Rust reserves a stack frame for it. Each `let` in the function gets a place in that frame, sized for its type. A `u32` takes 4 bytes, and a `usize` takes 8 bytes on a 64-bit machine.

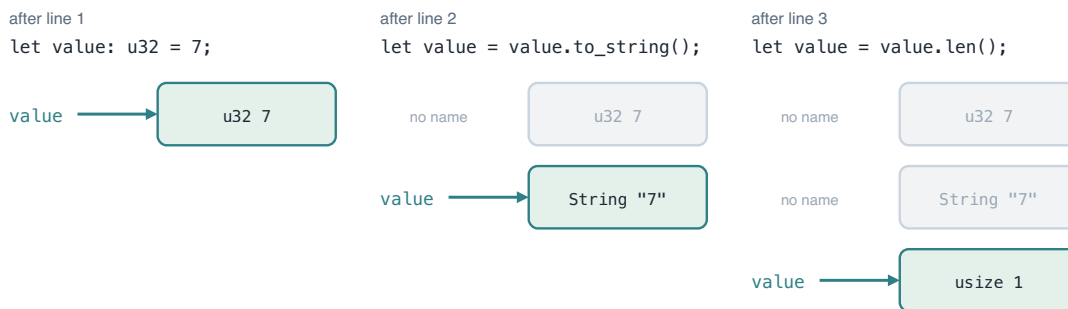
A `String` is different. Its text can grow, so the text is kept in heap memory. The **heap** is memory the program requests while it runs and gives back when it is done. The `String` value on the stack holds three numbers. They are the address of the text on the heap, the length of the text, and the size of the heap block.

When a function returns, its frame is removed. Every value in the frame is dropped. For a `String`, dropping it also frees its heap block.

1.2 Shadowing: one name, several values

You can write a second `let` with a name that is already in use. This does not change the first value. It creates a second binding with the same name, and the name now refers to the new value. Rust calls this **shadowing**. The first value still exists until the end of its scope, but you can no longer reach it by that name.

Because the second binding is a new variable, it can have a different type. Figure 1.1 follows the name `value` through three bindings of three types.



Earlier values are not modified. They lose their name and are dropped at the end of the scope.

Figure 1.1 Each `let` creates a new value and moves the name to it. The earlier values are not changed; they lose their name.

Here is the function that performs those three steps:

Listing 1.1 One name, three types. `src/bin/shadowing.rs`, lines 27 to 41

```
fn demonstrate_shadowing() -> String {
    // First binding is an integer.
    let value: u32 = 7;
    let mut log = format!("integer: {value}");

    // Shadow the integer with a String. The old integer is gone.
    let value = value.to_string();
    log.push_str(&format!("\nstring: {value}"));

    // Shadow the String with a usize computed from it.
    let value = value.len();
    log.push_str(&format!("\nusize: {value}"));

    log
}
```

Read the three `let value` lines in order:

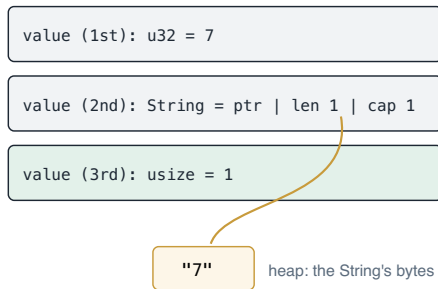
1. `let value: u32 = 7;` creates a `u32`.
2. `let value = value.to_string();` reads the `u32`, builds a `String` from it, and binds the name to the `String`. On the right-hand side, `value` still means the `u32`, because the new binding does not exist until the statement finishes.
3. `let value = value.len();` reads the `String` and binds the name to its length, a `usize`.

None of these lines needs `mut`, because no value is changed after it is created. The `log` string does change, so it is declared `let mut log`. Each `push_str` appends one line to it, so the test can check all three stages.

Figure 1.2 (left) shows the stack frame after the third line. There are three values, and only the last one has a usable name.

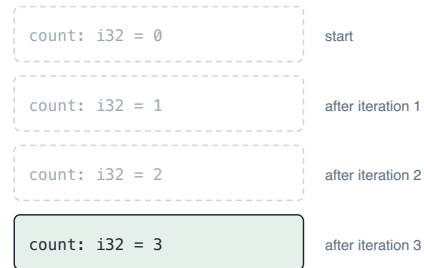
Shadowing: three bindings, three slots

stack frame of demonstrate_shadowing



Mutation: one slot, shown at four moments

stack frame of demonstrate_mutation (one slot)



Only the 3rd slot has a usable name. All three are freed when the function returns. The same slot is overwritten each time; the type never changes.

Figure 1.2 Shadowing creates a new slot for each binding. Mutation writes new values into the same slot. (The compiler may reuse slots it can prove are no longer used; the picture shows what the program means.)

1.2.1 When shadowing helps

Programs often receive a value in one form and need it in another. A number typed by a user arrives as text. After you parse it, you want the number and never the text again. Shadowing lets the number take the text's name:

```
let count = read_line_from_user(); // a String, for example "42"
let count: u32 = count.trim().parse().expect("a number");
```

After the second line, `count` is a `u32`. Code below it cannot use the unparsed text by mistake, because no name refers to it any more.

1.2.2 A value captured earlier keeps its old value

A closure or a reference created before a shadowing `let` still refers to the first value:

```
let x = 1;
let show = || println!("{x}");
let x = x + 1; // a new x
show();       // prints 1
```

The closure captured the first `x`. The second `let` created a new variable, and the closure never sees it.

WARNING: Inside a loop, `let x = x + 1;` creates a new `x` that lives only until the end of that pass through the loop. On the next pass, the outer `x` has its old value again. To count across passes, declare `let mut x` before the loop and write `x += 1` inside it.

1.3 Mutation: one value that changes

When you want one variable whose value changes over time, declare it with `mut`. The type stays the same, and each assignment writes into the same place in the frame.

Listing 1.2 A counter that changes in place. `src/bin/shadowing.rs`, lines 43 to 53

```
fn demonstrate_mutation() -> String {
    // If you need to mutate the same binding's value (same type) without
    // creating a new binding, mark it `mut`.
    let mut count = 0;
    let mut log = String::from("mut count:");
    for _ in 0..3 {
        count += 1;
        log.push_str(&format!("{}", count));
    }
    log
}
```

`for _ in 0..3` runs the loop body three times. The `_` means the loop does not use the counter it is given. Each pass adds 1 to `count` and appends the new value to `log`. Figure 1.2 (right) shows the one slot for `count` at four moments.

Table 1.1 compares the two ways of changing what a name refers to.

	Shadowing (<code>let x = ...</code> again)	Mutation (<code>let mut x, then x = ...</code>)
Creates a new variable	yes	no
Can change the type	yes	no
Needs <code>mut</code>	no	yes
A closure that captured <code>x</code> earlier sees	the old value	a compile error if it is still in use

1.4 The first complete program

The rest of `shadowing.rs` runs both functions and tests them. The file starts with a comment that explains the program:

Listing 1.3 The complete program. `src/bin/shadowing.rs`

```
// =====
// Variable shadowing vs mutation
//
// Ported from the Rust playground repo (`shadowing.rs`).
//
// What this demonstrates:
// - Rust allows a new `let` binding with the same name as an existing
//   binding. This is called shadowing.
// - Shadowing can change the value and the type of a variable without
//   declaring it `mut`.
// - The old binding is no longer reachable after the shadowing `let`.
//
// Where this appears in practice:
// - Rust SDK code often shadows parsed/deserialized values to convert them
//   from raw input to typed domain values:
//
//     let raw = request.header("X-GPU-Count");
//     let raw: u64 = raw.parse()?; // shadow with a typed value
//
// - Shadowing is also useful after unwrapping an `Option`/`Result` when the
//   guard value is no longer needed.
//
// Run:
// cargo run --bin shadowing
// =====

fn demonstrate_shadowing() -> String {
    // First binding is an integer.
    let value: u32 = 7;
    let mut log = format!("integer: {value}");

    // Shadow the integer with a String. The old integer is gone.
    let value = value.to_string();
    log.push_str(&format!("\nstring: {value}"));

    // Shadow the String with a usize computed from it.
    let value = value.len();
    log.push_str(&format!("\nusize: {value}"));

    log
}

fn demonstrate_mutation() -> String {
    // If you need to mutate the same binding's value (same type) without
    // creating a new binding, mark it `mut`.
    let mut count = 0;
    let mut log = String::from("mut count:");
    for _ in 0..3 {
        count += 1;
    }
}
```

```

        log.push_str(&format!("{}", count));
    }
    log
}

fn main() {
    println!("=== Shadowing ===");
    println!("{}", demonstrate_shadowing());

    println!("\n=== Mutation ===");
    println!("{}", demonstrate_mutation());
}

// =====
// Tests (cargo test --bin shadowing)
// =====
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn shadowing_allows_type_changes() {
        let log = demonstrate_shadowing();
        assert!(log.contains("integer: 7"));
        assert!(log.contains("string: 7"));
        assert!(log.contains("usize: 1"));
    }

    #[test]
    fn mutation_requires_mut_binding() {
        let log = demonstrate_mutation();
        assert!(log.contains("mut count: 1 2 3"));
    }
}

```

The tests at the end are in a module marked `#[cfg(test)]`. The compiler builds that module only when you run `cargo test`. `use super::*`; brings the functions from the rest of the file into the test module. Each test calls a function and uses `assert!` to check that the log contains the expected text.

Run the program and its tests from the `rust-interview-lab` directory:

```

$ cargo run --bin shadowing
=== Shadowing ===
integer: 7
string: 7
usize: 1

=== Mutation ===
mut count: 1 2 3

$ cargo test --bin shadowing
running 2 tests

```

```
test tests::mutation_requires_mut_binding ... ok
test tests::shadowing_allows_type_changes ... ok
```

Now that you know where a function keeps its values, you can look at what happens when a function calls itself many times.

1.5 A function that calls itself

The Fibonacci numbers start with 0 and 1. Each later number is the sum of the two before it:

n	0	1	2	3	4	5	6	7	8
fib(n)	0	1	1	2	3	5	8	13	21

The definition can be written directly as a Rust function. A function that calls itself is **recursive**.

Listing 1.4 Fibonacci by recursion. `src/bin/cs_fib.rs`, lines 8 to 15

```
/// Exponential: fib(n) recomputes fib(n - 1) and fib(n - 2) from scratch.
fn recursive(n: u64) -> u64 {
    if n < 2 {
        n
    } else {
        recursive(n - 1) + recursive(n - 2)
    }
}
```

For `n` of 0 or 1, the function returns `n`. This is the **base case**, the input that needs no further calls. For larger `n`, it calls itself twice and adds the results. The `if` is an expression, so its value is the function's return value, and no `return` keyword is needed.

1.5.1 Counting the calls

Work out `recursive(5)` by hand. It calls `recursive(4)` and `recursive(3)`. `recursive(4)` then calls `recursive(3)` again. Figure 1.3 draws every call.

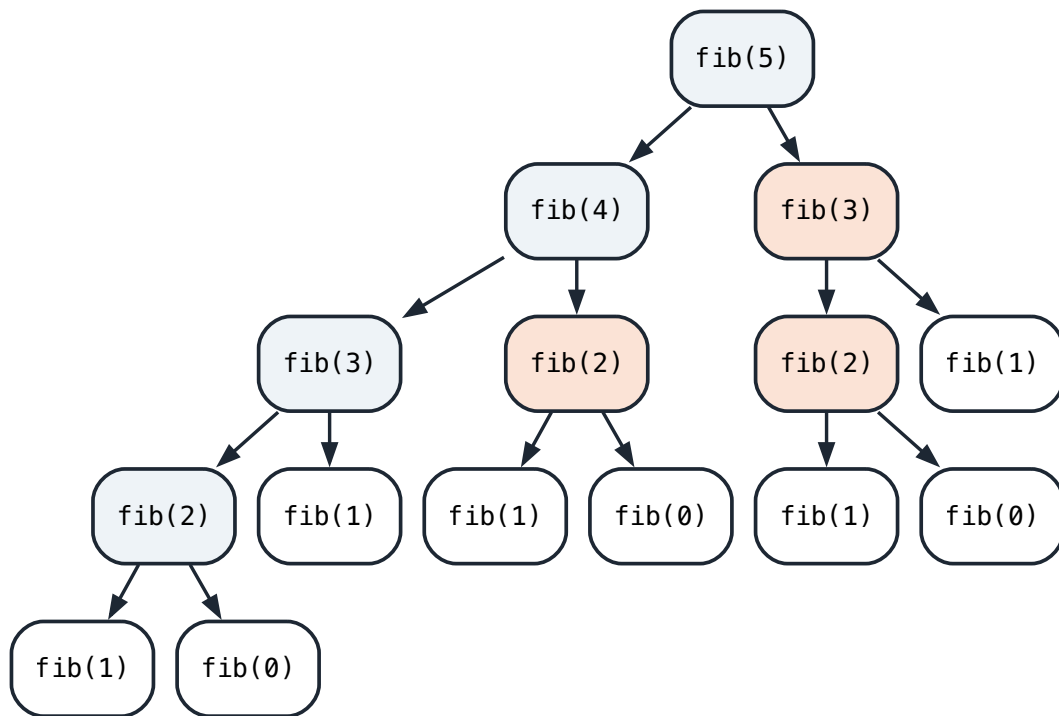


Figure 1.3 The calls made by `recursive(5)`. Shaded calls repeat work that another branch already did: `fib(3)` is computed twice and `fib(2)` three times.

The tree has 15 calls for $n = 5$. The tree for n contains the trees for $n - 1$ and $n - 2$, so the count grows by about 1.6 times per step. For $n = 30$, the function makes about 2.7 million calls. For $n = 90$, the count would be about 10^{19} .

1.5.2 What each call uses on the stack

Every call gets its own stack frame, holding its `n` and the address to return to. A frame is removed when its call returns. So the stack does not hold all the calls at once, only the chain of calls that are waiting on each other. Figure 1.4 shows the deepest moment of `recursive(4)`.

The stack while `recursive(4)` computes `recursive(1)` at its deepest point

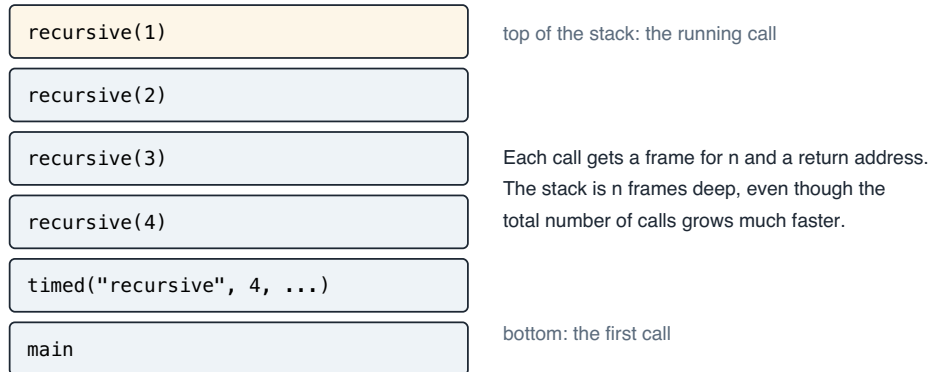


Figure 1.4 The stack at the deepest point of `recursive(4)`. The depth is about `n` frames, while the number of calls grows much faster.

So the recursive version has two costs. Its time grows exponentially with `n`, because it repeats work. Its stack depth grows with `n`, which is small here but limits recursion on very deep inputs. Chapter 9 shows a list that crashes because of this second cost.

1.6 The same numbers with a loop

You only ever need the last two numbers to compute the next one. A loop can keep those two and move them forward one step at a time.

Listing 1.5 Fibonacci with a loop. `src/bin/cs_fib.rs`, lines 17 to 24

```
/// Linear time, constant space.
fn iterative(n: u64) -> u64 {
    let (mut previous, mut current) = (0u64, 1u64);
    for _ in 0..n {
        (previous, current) = (current, previous + current);
    }
    previous
}
```

`let (mut previous, mut current) = (0u64, 1u64);` declares two variables in one statement by matching a pair. The suffix `u64` on each literal sets its type.

`(previous, current) = (current, previous + current);` assigns both variables at once. Rust evaluates the whole right-hand side first, using the old values, and only then writes both.

Without the pair, you would need a temporary variable to avoid overwriting `previous` before it is used.

Trace `iterative(5)`:

after pass	0 (start)	1	2	3	4	5
<code>previous</code>	0	1	1	2	3	5
<code>current</code>	1	1	2	3	5	8

After five passes, `previous` is 5, which is `fib(5)`. The loop runs `n` times and keeps two numbers, so its time grows in proportion to `n` and its memory stays constant.

1.7 Recursion with a memory

The loop is fast, but the recursive version matches the definition more closely. You can keep the recursion and remove the repeated work by saving each answer the first time you compute it. Saving answers this way is called **memoization**, and the table of saved answers is a **cache**.

Listing 1.6 Fibonacci with a cache. `src/bin/cs_fib.rs`, lines 26 to 41

```

// Linear time, linear space: recursion plus a cache of subproblem answers.
fn memoized(n: u64) -> u64 {
    fn go(n: u64, cache: &mut [u64]) -> u64 {
        if n < 2 {
            return n;
        }
        let index = n as usize;
        if cache[index] == 0 {
            cache[index] = go(n - 1, cache) + go(n - 2, cache);
        }
        cache[index]
    }

    let mut cache = vec![0u64; n as usize + 1];
    go(n, &mut cache)
}

```

This listing uses three pieces of syntax you may not have used before:

- **A function inside a function.** `go` is declared inside `memoized`. Only `memoized` can call it. It is an ordinary function; it does not see `memoized`'s variables.

- `vec![0u64; n as usize + 1]`. This creates a `Vec` of `n + 1` zeros. `n as usize` converts the `u64` to `usize`, the type Rust uses for lengths and indexes.
- `&mut [u64]`. This is a mutable slice: a borrowed view of the vector's elements that allows writing. `go` receives the cache this way, so every call shares one table instead of copying it.

`go` checks the base case first. Then it looks in the cache. A zero means "not computed yet". In that case it computes the value with two recursive calls and stores it. Either way, it returns the saved value.

Zero can mark an empty slot here because `fib(n)` is never zero for `n` of 2 or more. Smaller values return before the cache is read. Figure 1.5 shows the cache after `memoized(6)`.

cache after memoized(6)



Slots 0 and 1 are never written: `go()` returns `n` for `n < 2` before it reads the cache.

Every other slot is written once, the first time `go()` reaches it; later calls read it.

Figure 1.5 The cache after `memoized(6)`. Each slot from 2 to 6 is computed once and then read.

With the cache, each value from 2 to `n` is computed once, so the time grows in proportion to `n`. The cache uses memory in proportion to `n`, and the recursion still uses `n` stack frames.

1.8 Timing the three versions

To compare the versions, the program measures how long each call takes.

Listing 1.7 Timing a function and printing the result. `src/bin/cs_fib.rs`, lines 43 to 51

```
fn timed(label: &str, n: u64, operation: impl Fn(u64) -> u64) {
    let start = Instant::now();
    let value = operation(n);
    report(label, n, value, start.elapsed());
}

fn report(label: &str, n: u64, value: u64, elapsed: Duration) {
    println!(" {label:<10} fib({n:>2}) = {value:<20} in {elapsed:?}");
}
```

`timed` takes a parameter `operation: impl Fn(u64) -> u64`. This means "any function or closure that takes a `u64` and returns a `u64`". So `main` can pass `recursive`, `iterative`, or `memoized` by name.

`Instant::now()` reads a clock that only moves forward. `start.elapsed()` returns the time since then as a `Duration`.

`report` uses format specifiers inside the braces. `{label:<10}` pads the label to 10 characters, aligned left. `{n:>2}` right-aligns `n` in 2 characters. `{elapsed:?}` prints the `Duration` with its debug format, which chooses a unit such as `µs` or `ms`.

1.9 The complete Fibonacci program

Here is the whole program, including `main`. `main` times each version, then checks that all three agree.

Listing 1.8 The complete program. `src/bin/cs_fib.rs`

```

///! Three ways to compute Fibonacci: naive recursion, iteration, and memoization.
///! The first shows why exponential blowup happens; the other two fix it.
///!
///! Run with: cargo run --bin cs_fib

use std::time::{Duration, Instant};

/// Exponential: fib(n) recomputes fib(n - 1) and fib(n - 2) from scratch.
fn recursive(n: u64) -> u64 {
    if n < 2 {
        n
    } else {
        recursive(n - 1) + recursive(n - 2)
    }
}

/// Linear time, constant space.
fn iterative(n: u64) -> u64 {
    let (mut previous, mut current) = (0u64, 1u64);
    for _ in 0..n {
        (previous, current) = (current, previous + current);
    }
    previous
}

/// Linear time, linear space: recursion plus a cache of subproblem answers.
fn memoized(n: u64) -> u64 {
    fn go(n: u64, cache: &mut [u64]) -> u64 {
        if n < 2 {
            return n;
        }
        let index = n as usize;

```

```

        if cache[index] == 0 {
            cache[index] = go(n - 1, cache) + go(n - 2, cache);
        }
        cache[index]
    }

    let mut cache = vec![0u64; n as usize + 1];
    go(n, &mut cache)
}

fn timed(label: &str, n: u64, operation: impl Fn(u64) -> u64) {
    let start = Instant::now();
    let value = operation(n);
    report(label, n, value, start.elapsed());
}

fn report(label: &str, n: u64, value: u64, elapsed: Duration) {
    println!(" {label:<10} fib({n:>2}) = {value:<20} in {elapsed:?}");
}

fn main() {
    println!("small input, all three should agree");
    timed("recursive", 20, recursive);
    timed("iterative", 20, iterative);
    timed("memoized", 20, memoized);

    println!("\nrecursive cost grows quickly");
    timed("recursive", 30, recursive);
    timed("iterative", 30, iterative);

    println!("\nlarge input: recursion alone would not finish");
    timed("iterative", 90, iterative);
    timed("memoized", 90, memoized);

    for n in 0..20 {
        assert_eq!(recursive(n), iterative(n));
        assert_eq!(memoized(n), iterative(n));
    }
    assert_eq!(iterative(30), 832_040);
    assert_eq!(iterative(90), 2_880_067_194_370_816_120);

    println!("\nall checks passed");
}

```

The loop near the end of `main` compares the three versions for every `n` below 20. If any pair disagrees, `assert_eq!` stops the program with both values. The two final assertions check known values of `fib(30)` and `fib(90)`.

This output comes from a debug build, the default for `cargo run`:

```

$ cargo run --bin cs_fib
small input, all three should agree
recursive  fib(20) = 6765                in 90.908µs
iterative  fib(20) = 6765                in 567ns
memoized   fib(20) = 6765                in 10.826µs

```

```

recursive cost grows quickly
  recursive  fib(30) = 832040           in 11.124774ms
  iterative  fib(30) = 832040           in 542ns

large input: recursion alone would not finish
  iterative  fib(90) = 2880067194370816120 in 937ns
  memoized   fib(90) = 2880067194370816120 in 5.749µs

all checks passed

```

Compare the two recursive timings. Going from `n = 20` to `n = 30`, the time grows about 120 times. That matches the growth rate from section 1.5.1: 1.618 multiplied by itself 10 times is about 123. At that rate, `recursive(90)` would run for over a thousand years, so `main` does not call it.

The memoized version is slower than the loop, although both grow in proportion to `n`. It allocates a `Vec` and makes function calls, and the loop does neither.

NOTE: `fib(93)` is the largest Fibonacci number that fits in a `u64`. `iterative(94)` would overflow. In a debug build, an overflow stops the program with a panic. In a release build, the value silently wraps around.

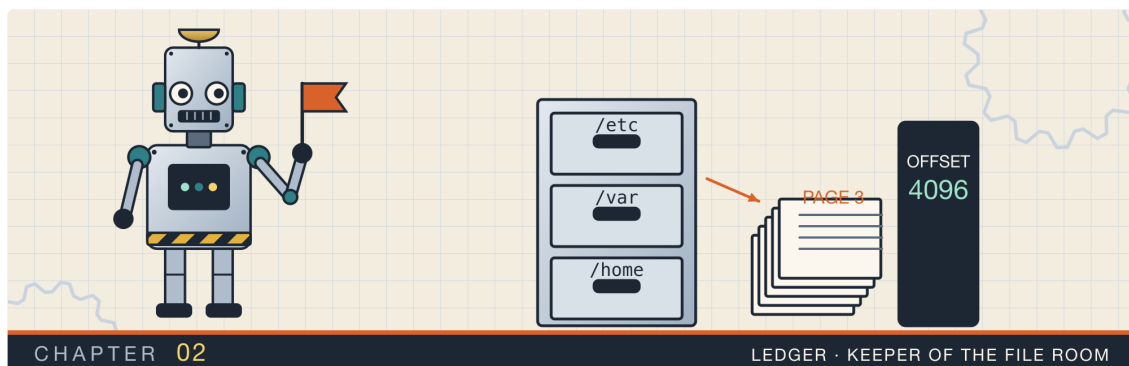
Summary

- A `let` binding attaches a name to a value in the current stack frame. A `String` keeps its text on the heap and a small header on the stack.
- Shadowing creates a new binding, which may have a new type. Earlier values are unchanged and lose their name. Closures that captured the old binding keep the old value.
- `mut` lets one binding change its value. The type stays the same.
- A recursive function uses one stack frame per active call. Naive recursive Fibonacci repeats work, so its time grows exponentially.
- A loop that keeps only the last two values computes the same result in linear time and constant memory.
- Memoization keeps the recursion and saves each answer in a cache, making the time linear.
- `impl Fn(u64) -> u64` accepts any function or closure with that signature, and `Instant` measures elapsed time.

Chapter 2 moves from computation to the operating system. You will read and write files, walk directories, and parse command-line arguments, and meet `Result` and `?`, which every later chapter uses.

Exercises

1. Add a counter to `recursive` that records how many calls it makes. Print the count for `n` from 1 to 25, and check that each count is close to 1.6 times the previous one.
2. Rewrite `memoized` so the cache holds `Option<u64>` instead of using 0 as "empty". Explain why the 0 marker is safe for Fibonacci but would not be safe for a function that can return 0.
3. Change `iterative` to return `Option<u64>` and use `checked_add`, so that `iterative(94)` returns `None` instead of overflowing.
4. Run `cargo run --release --bin cs_fib` and compare the timings with the debug build above.



Files, paths, and the command line

THIS CHAPTER COVERS

- What happens between your program and the disk when you read a file
- `Result` and the `?` operator, which every later chapter uses
- `Path` and `PathBuf`, and reading a file whole or line by line
- Appending, copying, renaming, and deleting files, and telling one error from another
- Listing and walking directories
- Reading a large file one page at a time, and parsing command-line arguments

In chapter 1 your programs only computed values. In this chapter they talk to the operating system. They read and write files, look inside directories, and read the arguments you type after a program's name.

Every one of these operations can fail. A file may not exist, or you may not have permission to read it. Rust reports such failures through a type called `Result`, so this chapter starts there. After that come paths, reading, writing, and directories. The chapter ends with two complete tools. One reads a file a page at a time. The other turns command-line arguments into typed options.

A few terms:

- The **kernel** is the core of the operating system. It controls the disk, memory, and devices.
- A **system call** is a request from a program to the kernel, such as "open this file" or "read 100 bytes".

- A **file descriptor** is a small number the kernel gives your program for each open file. Later system calls use it to name that file.

2.1 What happens when you read a file

Your program cannot read the disk directly. It asks the kernel with system calls, and the kernel does the work. Figure 2.1 shows the steps for reading a small file.

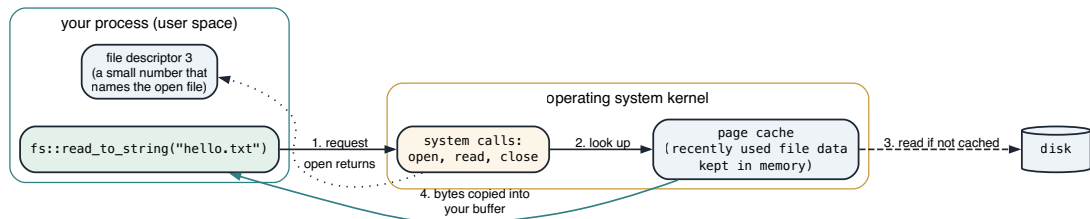


Figure 2.1 A read goes through the kernel. The kernel keeps recently used file data in memory, in its page cache, so a second read of the same file usually does not touch the disk.

The steps are:

1. `open` asks the kernel to open the file. The kernel checks permissions and returns a file descriptor.
2. `read` asks for bytes from that descriptor. The kernel copies them into a buffer in your program.
3. `close` tells the kernel you are done, so it can release the descriptor.

Rust's standard library makes these calls for you. Each call can fail, and the library reports that with `Result`.

2.2 `Result` and the `?` operator

`Result` is an enum with two variants:

```
enum Result<T, E> {
    Ok(T),    // the operation succeeded and produced a T
    Err(E),   // the operation failed with an error of type E
}
```


For file operations, the error type is `std::io::Error`, and `io::Result<T>` is a shorter name for `Result<T, io::Error>`. So `fs::read_to_string(path)` returns `io::Result<String>`: either the file's text, or the reason it could not be read.

You can handle a `Result` with `match`. That gets long when a function makes several calls, each of which can fail. The `?` operator shortens it. Put `?` after an expression of type `Result`, and it does one of two things (figure 2.2):

- If the value is `Ok(x)`, the expression evaluates to `x` and the function continues.
- If the value is `Err(e)`, the function returns `Err(e)` immediately.

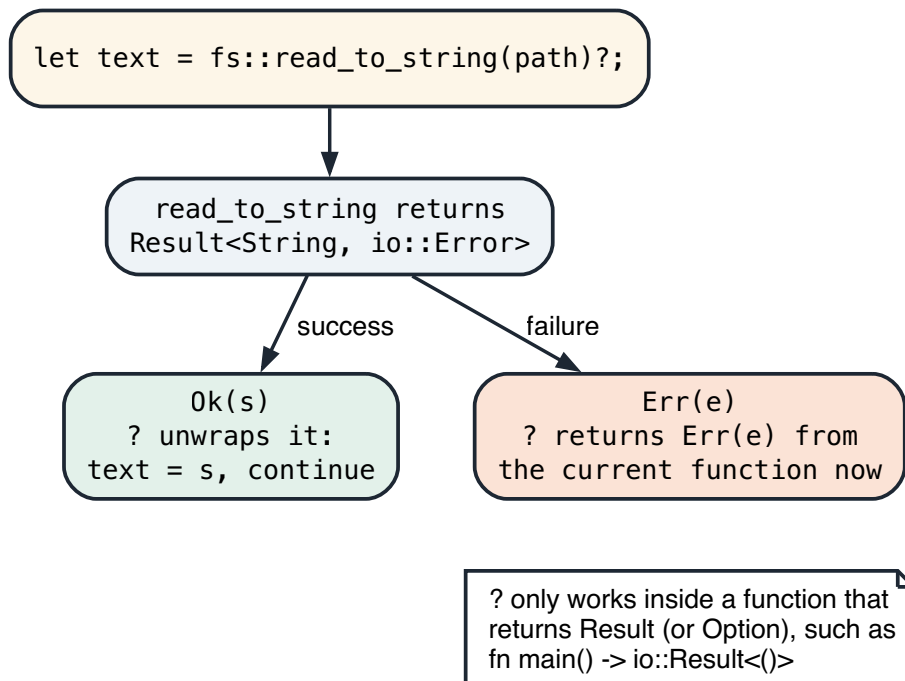


Figure 2.2 The `?` operator. It can only be used inside a function that itself returns `Result`.

Because `?` returns from the function, the function must return a `Result` too. That is why the programs in this chapter declare `fn main() -> io::Result<>`. If `main` returns an `Err`, Rust prints the error and the program exits with status 1.

2.3 Paths

A **path** names a file or directory, such as `/tmp/data.txt`. Rust has two path types. `&Path` is a borrowed view of a path, and `PathBuf` is an owned path that can grow. They relate the same way `&str` relates to `String` (figure 2.3).

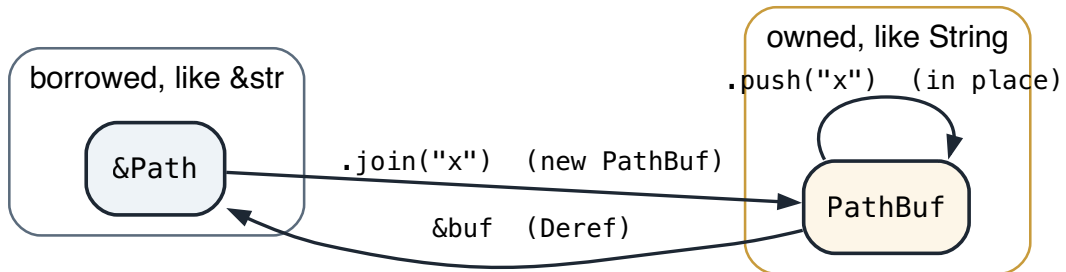


Figure 2.3 `join` returns a new owned path. `push` extends a `PathBuf` in place. A `&PathBuf` converts to `&Path` automatically when a function asks for one.

Listing 2.1 Building paths and taking them apart. `src/bin/path_buf.rs`

```

use std::path::{Path, PathBuf};

fn main() {
    let dir = Path::new("/tmp");
    let file = dir.join("data.txt");

    let mut buf = PathBuf::from(dir);
    buf.push("nested");
    buf.push("file.txt");

    println!("file: {}", file.display());
    println!("buf: {}", buf.display());
    println!("parent: {:?}", buf.parent());
    println!("file_name: {:?}", buf.file_name());
    println!("ext: {:?}", buf.extension());
}
  
```

`Path::new("/tmp")` wraps a string as a path without copying it. `dir.join("data.txt")` builds a new `PathBuf`. `PathBuf::from(dir)` copies the path into an owned buffer, and each `push` adds one component.

```
$ cargo run --bin path_buff
file: /tmp/data.txt
buf: /tmp/nested/file.txt
parent: Some("/tmp/nested")
file_name: Some("file.txt")
ext: Some("txt")
```

`parent`, `file_name`, and `extension` return `Option`, because some paths lack them: `/` has no parent. To print a path you call `display()`. On Linux and macOS a path is a sequence of bytes that need not be valid UTF-8. `display()` replaces any invalid bytes when it prints.

WARNING: If the argument to `join` or `push` is an absolute path, it replaces the whole path instead of extending it. `Path::new("/srv").join("/etc/passwd")` is `/etc/passwd`. If part of a path comes from a user, reject absolute paths and `..` before you join it.

2.4 Reading a file

2.4.1 The whole file at once

The simplest read loads the whole file into one `String`.

Listing 2.2 Write a file, then read it back. `src/bin/read_write_file.rs`

```
use std::fs;
use std::io;
use std::path::{Path, PathBuf};

// `&Path` accepts both a `&Path` and a `&PathBuf` argument; `&PathBuf` would not.
fn read_content_of_file(path: &Path) -> io::Result<String> {
    fs::read_to_string(path)
}

fn main() -> io::Result<> {
    fs::write("hello.txt", "Hello, Rust!\n");

    // `join` builds a new PathBuf and leaves `base_dir` unchanged.
    let base_dir = PathBuf::from(".");
    let joined_path = base_dir.join("hello.txt");

    // `push` mutates the PathBuf in place instead of returning a new one.
    let mut mut_base_dir = PathBuf::from(".");
    mut_base_dir.push("hello.txt");

    let text = read_content_of_file(&joined_path)?;
    println!("File contents (via .join()):\n{}", text);

    let text_inplace = read_content_of_file(&mut_base_dir)?;
```

```
println!("File contents (via .push()):\n{}", text_inplace);
Ok(())
}
```

`fs::write` creates the file (or empties an existing one) and writes the text. `fs::read_to_string` opens, reads, and closes the file in one call. Both calls end with `?`, so a failure in either one returns from `main`.

`read_content_of_file` takes `&Path`. The program calls it twice, with a path built by `join` and a path built by `push`. Both are `PathBuf` values that convert to `&Path`. The two calls print the same text.

`read_to_string` keeps the whole file in memory. That suits a small configuration file. For a log of several gigabytes, read one line at a time instead.

2.4.2 One line at a time

To read line by line, wrap the file in a `BufReader`. Figure 2.4 shows why the buffer helps.

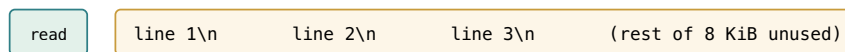
Reading a 3-line file, line by line

Without a buffer: one system call per small read



Each read crosses into the kernel and returns a few bytes.

With `BufReader`: one system call fills an 8 KiB buffer



`read_line` then copies each line out of the buffer, in your process, with no system call.

Figure 2.4 A `BufReader` asks the kernel for a large block at once and hands out lines from it. Fewer system calls means less time spent crossing into the kernel.

Listing 2.3 Numbered lines through a buffered reader. `src/bin/readfile_line_by_line.rs`

```
use std::fs::File;
use std::io::{BufRead, BufReader}; // BufRead brings `lines()` into scope.

fn main() -> std::io::Result<()> {
    let file = File::open("log.txt")?;
    let reader = BufReader::new(file);

    for (i, line) in reader.lines().enumerate() {
        let line = line?;
```

```

        println!("{}", i + 1, line);
    }

    Ok(())
}

```

`BufReader::new(file)` takes ownership of the file and adds an 8 KiB buffer. The `lines` method comes from the `BufRead` trait, so the program imports that trait. Without the import, the method is not visible.

`lines()` yields one `io::Result<String>` per line, with the newline removed. Reading can fail partway through the file, so each line is a `Result`. `let line = line?;` replaces the `Result` with the `String` inside it, using the shadowing you saw in section 1.2. `enumerate()` pairs each line with its index, starting at 0, which is why the program prints `i + 1`.

2.4.3 Reusing one buffer, and skipping bad lines

`lines()` allocates a new `String` for every line. For a very large file, you can read every line into the same `String` instead.

Listing 2.4 One reused buffer, with invalid lines skipped. `src/bin/read_large_file.rs`

```

use std::fs::File;
use std::io::{self, BufRead, BufReader};

fn main() -> io::Result<()> {
    let file = File::open("large.txt")?;
    let mut reader = BufReader::new(file);
    let mut line = String::new();

    while match reader.read_line(&mut line) {
        Ok(0) => false, // EOF, stop
        Ok(_) => {
            process(line.trim_end());
            line.clear();
            true // keep reading
        }
        Err(e) if e.kind() == io::ErrorKind::InvalidData => {
            eprintln!("skipping bad line: {e}");
            line.clear();
            true
        }
        Err(e) => return Err(e), // return works inside the condition
    } {}

    Ok(())
}

fn process(line: &str) {

```

```
println!("{line}");
}
```

`reader.read_line(&mut line)` appends the next line, including its newline, to `line`. It returns the number of bytes read. The loop is written in an unusual way: `while match ... { ... } {}`. The whole `match` is the loop's condition, and the loop's body, `{}`, is empty. Each arm of the `match` evaluates to `true` to continue or `false` to stop.

Table 2.1 lists the four outcomes.

read_line returns	Meaning	What the loop does
<code>Ok(0)</code>	end of the file	stops
<code>Ok(n)</code>	a line of <code>n</code> bytes	processes it, clears the buffer, continues
<code>Err(e)</code> with kind <code>InvalidData</code>	the line is not valid UTF-8 text	prints a message, clears, continues
any other <code>Err</code>	a real read failure	returns the error from <code>main</code>

Because `read_line` appends, the loop must call `line.clear()` after each line. Otherwise the buffer would grow to hold the whole file.

The `InvalidData` arm can continue safely because `read_line` has already consumed the invalid bytes. You can check this with a file whose second line holds two bytes that are not UTF-8:

```
$ printf 'ok\n\xff\xfebad\nfine\n' > large.txt
$ cargo run --bin read_large_file
ok
skipping bad line: stream did not contain valid UTF-8
fine
```

2.5 Writing and changing files

2.5.1 Appending

`fs::write` replaces a file's contents. To add lines to the end of a log instead, open the file with `OpenOptions`.

Listing 2.5 Appending lines to a file. `src/bin/append_to_file_open_options.rs`

```

use std::fs::OpenOptions;
use std::io::Write;

fn main() -> std::io::Result<()> {
    let mut file = OpenOptions::new()
        .create(true)    // create if missing
        .append(true)    // always write at the end
        .open("log.txt")?;

    writeln!(file, "New log line")?;
    writeln!(file, "Another line")?;

    Ok(())
}

```

`OpenOptions` collects settings, and `open` applies them. `create(true)` creates the file if it is missing. `append(true)` asks the kernel to move to the end of the file before every write. The kernel does that move and the write as one step. So two programs appending to the same log do not overwrite each other's lines.

`writeln!` works like `println!`, but writes to any value that implements the `std::io::Write` trait, such as a `File`. The trait must be imported, as the second `use` line does.

2.5.2 Copying, renaming, and deleting

Listing 2.6 Copy, rename, remove. `src/bin/copy_rename_delete_file.rs`

```

use std::fs;
use std::io;

fn main() -> io::Result<()> {
    fs::write("a.txt", "hello")?;

    fs::copy("a.txt", "b.txt")?;
    fs::rename("b.txt", "c.txt")?;
    fs::remove_file("a.txt")?;

    println!("Done");
    Ok(())
}

```

After the program runs, `a.txt` is gone and `c.txt` holds `hello`. Each call is one or two system calls.

`fs::rename` has a property you can build on. When both names are on the same disk, the rename is **atomic**. Any other program sees either the old name or the new one, never a half-

renamed file. To update a file safely, write the new contents to a temporary file, then rename it over the old one. A crash in the middle leaves either the old file or the new file, never a partial one.

2.5.3 Telling one error from another

An `io::Error` carries a **kind**, an enum value such as `NotFound` or `PermissionDenied`. You can match on it to handle one failure and report the others.

Listing 2.7 Matching on the kind of an error. `src/bin/file_error_handling.rs`

```
use std::fs;
use std::io;

fn read_file(path: &str) -> Result<String, io::Error> {
    fs::read_to_string(path)
}

fn main() {
    match read_file("missing.txt") {
        Ok(text) => println!("{}", text),
        Err(e) if e.kind() == io::ErrorKind::NotFound => {
            eprintln!("File not found");
        }
        Err(e) => eprintln!("Error: {}", e),
    }
}
```

The second arm has a **match guard**, the `if` after the pattern. It matches only errors whose kind is `NotFound`. Every other error falls through to the third arm. This `main` does not return a `Result`, so it cannot use `?`; it handles every case itself. Run it where `missing.txt` does not exist, and it prints `File not found`.

2.6 Directories

A directory is a list of entries. Each entry has a name and a type: a file, a directory, or a symbolic link. A **symbolic link** is an entry that points to another path.

2.6.1 One level

Listing 2.8 Listing the current directory. `src/bin/list_directory.rs`

```
use std::fs;
use std::io;
```



```
fn main() -> io::Result<()> {
    for entry in fs::read_dir(".")? {
        let entry = entry?;
        let path = entry.path();

        if path.is_dir() {
            println!("[DIR] {}", path.display());
        } else {
            println!("[FILE] {}", path.display());
        }
    }
    Ok(())
}
```

`fs::read_dir(".")` returns an iterator over the entries of the current directory. Each item is a `Result`, because reading an entry can fail. `path.is_dir()` asks the kernel for the entry's details. If the entry is a symbolic link, `is_dir` follows it and reports on the target.

2.6.2 Every level

To visit subdirectories, the function calls itself on each directory it finds, the recursion you met in section 1.5.

Listing 2.9 Walking a directory tree. `src/bin/recusive_directory_walk.rs`

```
use std::fs;
use std::io;
use std::path::Path;

fn walk(dir: &Path) -> io::Result<()> {
    for entry in fs::read_dir(dir)? {
        let entry = entry?;
        let path = entry.path();
        let file_type = entry.file_type()?;

        if file_type.is_dir() {
            println!("[DIR] {}", path.display());
            walk(&path)?;
        } else {
            println!("[FILE] {}", path.display());
        }
    }
    Ok(())
}

fn main() -> io::Result<()> {
    walk(Path::new(".")).ok();
    Ok(())
}
```

`walk` uses `entry.file_type()` instead of `path.is_dir()`. The difference is how each treats symbolic links. `file_type()` does not follow them, so a link to a directory is reported as a link, and `walk` does not descend into it. That keeps the walk from looping forever through a link that points back to one of its own parent directories. On Linux, `file_type()` is usually also faster, because the kernel returns the type together with the name.

The recursion goes one level deeper per directory level. Real directory trees are shallow, so the stack is not a concern here.

2.7 Project: reading a large file one page at a time

Suppose a service stores millions of job records, one per line, in a file. A client asks for them a page at a time. With each page, the service returns a **cursor**, a value the client sends back to get the next page.

If the cursor is the byte position where the next page starts, the service needs no memory of past requests. Any copy of the service can read the next page from the cursor alone.

2.7.1 The page type

Listing 2.10 The `Page` type (lines 1 to 7). `src/bin/file_pagination.rs`

```
use std::fs::File;
use std::io::{self, BufRead, BufReader, Seek, SeekFrom};

struct Page {
    jobs: Vec<String>,
    next_cursor: Option<u64> // Byte offset where the next page starts
}
```

A `Page` holds the lines and an `Option<u64>` for the next cursor. `None` means there are no more pages.

2.7.2 Counting bytes to find the next cursor

The next cursor is the byte where this page ends. There is a trap in finding it. You might ask the file for its current position after reading three lines. That answer is wrong, because the `BufReader` read 8 KiB ahead on its first read, as figure 2.4 showed. The file's position is past the buffered data, not after the third line (figure 2.5).

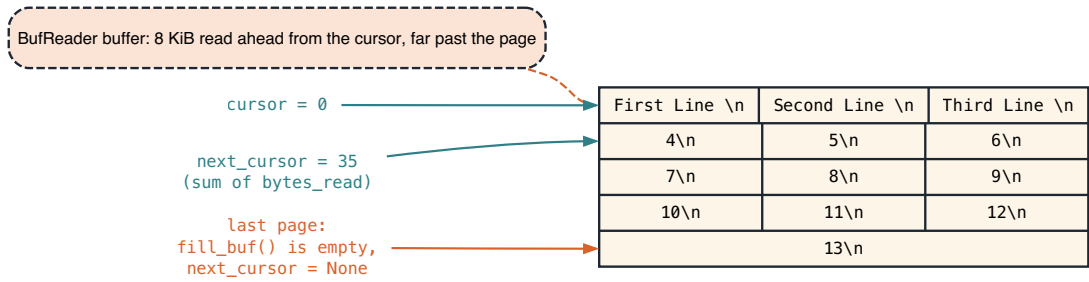


Figure 2.5 Each row is one page. The next cursor is the sum of the bytes each `read_line` returned: $12 + 12 + 11 = 35$ for the first page. The file's own position is far past that, at the end of what the buffer read.

So `read_page` counts the bytes itself:

Listing 2.11 `read_page` (lines 9 to 36).

```
fn read_page(path: &str, cursor: u64, page_size: usize) -> io::Result<Page> {
    let mut file = File::open(path)?;
    file.seek(SeekFrom::Start(cursor))?;
    let mut reader = BufReader::new(file);

    // Optimization which preserves as large as the max page size
    let mut jobs: Vec<String> = Vec::with_capacity(page_size);
    let mut offset = cursor;
    let mut line = String::new();

    while jobs.len() < page_size {
        // Clear the previously read line
        line.clear();

        match reader.read_line(&mut line)? {
            0 => break,
            bytes_read => {
                offset += bytes_read as u64;
                jobs.push(line.trim_end().to_string());
            },
        }
    }

    let at_end = reader.fill_buf()?.is_empty();
    let next_cursor = if at_end { None } else { Some(offset) };

    Ok(Page { jobs, next_cursor })
}
```

`read_page` opens the file and calls `seek(SeekFrom::Start(cursor))`, which moves the file's read position to byte `cursor`. Then it wraps the file in a `BufReader`.

`Vec::with_capacity(page_size)` reserves room for a full page, so pushing lines does not reallocate.

`offset` starts at `cursor`. Each `read_line` returns how many bytes it consumed, newline included, and `offset` grows by that amount. `line.trim_end()` removes the newline (and any trailing spaces) before the line is stored, but the count still includes them, so `offset` stays exact.

The last page needs care. If the file ends exactly after this page's last line, the page should report no next cursor. `reader.fill_buf()` returns the bytes waiting in the buffer, reading more if the buffer is empty, without consuming them. If it returns nothing, the file has ended, and the cursor is `None`.

2.7.3 The complete program

`main` acts as a client. It asks for pages of three lines until the cursor is `None`.

Listing 2.12 The complete program. `src/bin/file_pagination.rs`

```
use std::fs::File;
use std::io::{self, BufRead, BufReader, Seek, SeekFrom};

struct Page {
    jobs: Vec<String>,
    next_cursor: Option<u64> // Byte offset where the next page starts
}

fn read_page(path: &str, cursor: u64, page_size: usize) -> io::Result<Page> {
    let mut file = File::open(path)?;
    file.seek(SeekFrom::Start(cursor))?;
    let mut reader = BufReader::new(file);

    // Optimization which preserves as large as the max page size
    let mut jobs: Vec<String> = Vec::with_capacity(page_size);
    let mut offset = cursor;
    let mut line = String::new();

    while jobs.len() < page_size {
        // Clear the previously read line
        line.clear();

        match reader.read_line(&mut line)? {
            0 => break,
            bytes_read => {
                offset += bytes_read as u64;
                jobs.push(line.trim_end().to_string());
            },
        }
    }

    let at_end = reader.fill_buf()?.is_empty();
```

```

    let next_cursor = if at_end { None } else {Some(offset)};

    Ok(Page { jobs, next_cursor})
}

fn main() -> io::Result<()> {
    let mut cursor = 0;

    loop {
        let page = read_page("src/bin/large_file.txt", cursor, 3)?;

        println!("--- page ---");
        for job in &page.jobs {
            println!("{job}");
        }

        match page.next_cursor {
            Some(next) => cursor = next,
            None => break,
        }
    }

    Ok(())
}

```

The input file has thirteen lines, so the program prints four full pages and one page of one line:

```

$ cargo run --bin file_pagination
--- page ---
First Line
Second Line
Thrid Line
--- page ---
4
5
6
--- page ---
7
8
9
--- page ---
10
11
12
--- page ---
13

```

TIP: If the file can change between requests, an old cursor may point into the middle of a line. You can include a version number with the offset in the cursor, and reject cursors whose version is out of date.

2.8 Project: command-line arguments

When you run `command_line_args --name ada --count 2 --verbose`, the operating system passes the program a list of strings (figure 2.6). The first string is the program's own path. The rest are the arguments, in the order you typed them.

The arguments the operating system passes to the program



`args()` yields these `Strings` in order. `args().skip(1)` drops [0], the program's own path.

`--name` and `--count` each consume the next argument as their value; `--verbose` stands alone.

Figure 2.6 The arguments as the program receives them. `--name` and `--count` each take the next string as their value.

This program turns that list into a struct of typed options, or an error message.

2.8.1 The options and their defaults

Listing 2.13 The options (lines 1 to 17). `src/bin/command_line_args.rs`

```
use std::env::args;
use std::process::ExitCode;

/// The options this program understands.
#[derive(Debug, PartialEq)]
struct Options {
    name: String,
    count: u32,
    verbose: bool,
}

impl Default for Options {
    fn default() -> Self {
        Self { name: String::from("world"), count: 1, verbose: false }
    }
}
```

`Options` has one field per option. The `Default` trait gives the values used when an option is not given. `#[derive(Debug, PartialEq)]` lets the tests print and compare `Options` values.

2.8.2 Parsing

Listing 2.14 `parse` (lines 18 to 40).

```

/// Turn the arguments after the program name into `Options`.
fn parse(arguments: impl IntoIterator<Item = String>) -> Result<Options, String> {
    let mut options = Options::default();
    let mut arguments = arguments.into_iter();

    while let Some(argument) = arguments.next() {
        match argument.as_str() {
            "--name" => {
                options.name = arguments.next().ok_or("--name needs a value")?;
            }
            "--count" => {
                let value = arguments.next().ok_or("--count needs a value")?;
                options.count = value
                    .parse()
                    .map_err(|_| format!("--count needs a number, got {value:?}"))?;
            }
            "--verbose" => options.verbose = true,
            other => return Err(format!("unknown argument {other:?}")),
        }
    }

    Ok(options)
}

```

`parse` takes `impl IntoIterator<Item = String>`, which means "anything that can produce Strings one at a time". The program passes the real arguments. The tests pass short lists written in the test code.

The loop is `while let Some(argument) = arguments.next()`. It calls `next()` itself, rather than using `for`, because `--name` must take the next string from the same iterator as its value. A `for` loop holds the iterator for the whole loop, so the body could not call `next()` on it.

`arguments.next().ok_or("--name needs a value")?` does two steps. `ok_or` turns `None` into an `Err` with the message. Then `?` returns that error from `parse`. The `--count` arm adds a third step. `parse()` converts the text to a number, and `map_err` replaces a parse error with a message that includes the text.

An argument the program does not know is an error, so a misspelled option is reported instead of ignored.

2.8.3 The exit code

Listing 2.15 `main` (lines 42 to 61).

```
fn main() -> ExitCode {
    let options = match parse(args().skip(1)) {
        Ok(options) => options,
        Err(message) => {
            eprintln!("error: {message}");
            eprintln!("usage: command_line_args [--name NAME] [--count N] [--verbose]");
            return ExitCode::FAILURE;
        }
    };

    for index in 0..options.count {
        if options.verbose {
            println!("[{index}] hello, {}", options.name);
        } else {
            println!("hello, {}", options.name);
        }
    }

    ExitCode::SUCCESS
}
```

A program's **exit code** is a number it returns to whoever started it. Zero means success. `main` returns `ExitCode`, so it can report failure after printing its own message. `args().skip(1)` drops the program's path. On an error, `main` prints the message and a usage line to standard error with `eprintln!`, then returns `ExitCode::FAILURE`, which is 1.

2.8.4 The complete program

Listing 2.16 The complete program, with its tests. `src/bin/command_line_args.rs`

```
use std::env::args;
use std::process::ExitCode;

/// The options this program understands.
#[derive(Debug, PartialEq)]
struct Options {
    name: String,
    count: u32,
    verbose: bool,
}

impl Default for Options {
    fn default() -> Self {
        Self { name: String::from("world"), count: 1, verbose: false }
    }
}
```



```

/// Turn the arguments after the program name into `Options`.
fn parse(arguments: impl IntoIterator<Item = String>) -> Result<Options, String> {
    let mut options = Options::default();
    let mut arguments = arguments.into_iter();

    while let Some(argument) = arguments.next() {
        match argument.as_str() {
            "--name" => {
                options.name = arguments.next().ok_or("--name needs a value")?;
            }
            "--count" => {
                let value = arguments.next().ok_or("--count needs a value")?;
                options.count = value
                    .parse()
                    .map_err(|_| format!("--count needs a number, got {value:?}"))?;
            }
            "--verbose" => options.verbose = true,
            other => return Err(format!("unknown argument {other:?}")),
        }
    }

    Ok(options)
}

fn main() -> ExitCode {
    let options = match parse(args().skip(1)) {
        Ok(options) => options,
        Err(message) => {
            eprintln!("error: {message}");
            eprintln!("usage: command_line_args [--name NAME] [--count N] [--verbose]");
            return ExitCode::FAILURE;
        }
    };

    for index in 0..options.count {
        if options.verbose {
            println!("[{index}] hello, {}", options.name);
        } else {
            println!("hello, {}", options.name);
        }
    }

    ExitCode::SUCCESS
}

#[cfg(test)]
mod tests {
    use super::*;

    fn parse_args(arguments: &[&str]) -> Result<Options, String> {
        parse(arguments.iter().map(|value| value.to_string()))
    }

    #[test]
    fn defaults_apply_when_there_are_no_arguments() {
        assert_eq!(parse_args(&[]).unwrap(), Options::default());
    }

    #[test]

```

```

fn a_value_option_consumes_the_next_argument() {
    let options = parse_args(&["--name", "ada", "--count", "3"]).unwrap();
    assert_eq!(options.name, "ada");
    assert_eq!(options.count, 3);
}

#[test]
fn verbose_is_a_flag() {
    assert!(parse_args(&["--verbose"]).unwrap().verbose);
    assert!(!parse_args(&[]).unwrap().verbose);
}

#[test]
fn a_value_option_without_a_value_is_an_error() {
    assert!(parse_args(&["--name"]).is_err());
    assert!(parse_args(&["--count"]).is_err());
}

#[test]
fn a_non_numeric_count_is_an_error() {
    assert!(parse_args(&["--count", "many"]).is_err());
}

#[test]
fn an_unknown_argument_is_an_error() {
    assert!(parse_args(&["--colour", "red"]).is_err());
    assert!(parse_args(&["extra"]).is_err());
}
}

```

The helper `parse_args` in the test module converts a list of `&str` into `Strings`, so each test states its input in one line. The tests cover the defaults, each option, a missing value, a bad number, and an unknown argument.

```

$ cargo run --bin command_line_args -- --name ada --count 2 --verbose
[0] hello, ada
[1] hello, ada
$ cargo run --bin command_line_args -- --count many
error: --count needs a number, got "many"
usage: command_line_args [--name NAME] [--count N] [--verbose]
$ echo $?
1

```

The `--` after `cargo run --bin command_line_args` tells Cargo that the rest of the line belongs to your program, not to Cargo.

Summary

- A program reads files through system calls. The kernel keeps recent file data in its page cache.

- `Result<T, E>` holds either a value or an error. `?` returns the error from the current function, so that function must return `Result`.
- `&Path` and `PathBuf` are the borrowed and owned path types. Functions should take `&Path`.
- `fs::read_to_string` reads a whole file. A `BufReader` reads large files a line at a time with few system calls. `read_line` into one reused `String` avoids an allocation per line.
- `OpenOptions` with `append(true)` adds to a file safely. A rename within one disk is atomic.
- `io::Error::kind()` lets you handle one kind of failure and report the others.
- `file_type()` does not follow symbolic links, which keeps a recursive walk from looping.
- A byte offset works as a page cursor if you count the bytes you read, not the file's position.
- Command-line arguments arrive as strings. Parse them in a function that takes an iterator, and return an exit code.

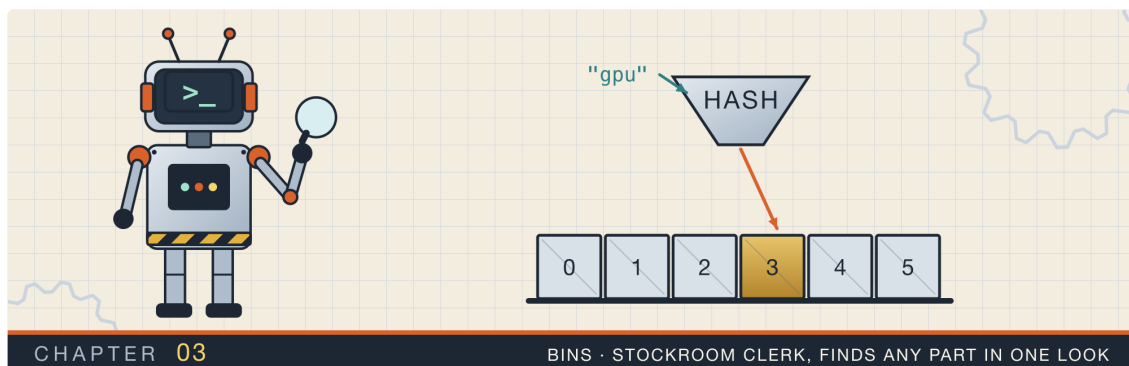
Chapter 3 begins Part 2. It works with Rust's main collections, `Vec`, `String`, and `HashMap`, on a series of problems about arrays and text.

Exercises

1. Rewrite `walk` without recursion. Keep a `Vec<PathBuf>` of directories still to visit, and count the files and total bytes for each file extension.
2. Write `atomic_write(path, bytes)`. Write to a temporary file in the same directory, call `sync_all()` on it, and rename it over the target.
3. Add a version number to the pagination cursor, and return an error when the file changed between pages.
4. Let the argument parser accept `--name=ada` as well as `--name ada`.

PART 2

Collections and algorithms



Vectors, strings, and hash maps

THIS CHAPTER COVERS

- Counting how much work an algorithm does, and the $O(n)$ notation for it
- How a `Vec`, a `String`, and a `HashMap` are laid out in memory
- Finding a pair of numbers with a hash map
- Using a vector as a stack to check brackets and to track a minimum
- Walking an array with two indexes: reversing, sliding windows, and merging intervals
- Binary search, rotating a grid, and searching text

This chapter begins Part 2. Part 2 is about data structures and the algorithms that use them. In this chapter you work with the three collections you will use most in Rust: `Vec`, `String`, and `HashMap`.

Before any problem, I spend a few sections on groundwork. First, a way to compare two solutions by how much work they do. Then a look at how each collection stores its data. With that in place, each problem in the rest of the chapter is short, and you can see why each solution is fast.

The problems are grouped by technique, not by topic. A technique is a pattern you can reuse. Two examples are "keep a stack of things not yet matched" and "move two indexes toward each other". Table 3.1 lists them.

Section	Technique	Problems
3.5	look things up in a hash map	find two numbers that add up to a target
3.6	a vector used as a stack	check brackets; a stack that knows its minimum
3.7	two indexes moving toward each other	reverse a string; find three numbers that add to zero
3.8	a window that slides along a string	longest piece of text with no repeated letter
3.9	sort, then walk once	merge overlapping ranges
3.10	halve the search each step	binary search in a rotated array
3.11	transform in place	rotate a square grid
3.12	reuse what you already matched	find text inside other text

3.1 Counting the work an algorithm does

Two programs can give the same answer and take very different amounts of time. You saw this in chapter 1: the recursive Fibonacci took milliseconds where the loop took nanoseconds. To choose between solutions, you need a way to describe how their running time grows as the input grows.

The usual way is to count the basic steps a program takes for an input of size n . Here n is the length of the array or string. A basic step is something that takes a fixed amount of time, such as comparing two numbers or reading one element.

Consider three ways to answer questions about an array of n numbers:

- Reading the first element takes one step, whatever n is.
- Adding up all the elements takes n steps, one per element.
- Comparing every element with every other element takes about $n \times n$ steps.

Big-O notation is a short way to write how the count grows. You keep only the part that grows fastest and drop constant factors. So the three examples above are written $O(1)$, $O(n)$, and $O(n^2)$. You read $O(n)$ as "order n ": the time grows in proportion to n .

Table 3.2 shows how different these are once n is large.

n	O(1)	O(log n)	O(n)	O(n log n)	O(n ²)
10	1	about 3	10	about 33	100
1,000	1	about 10	1,000	about 10,000	1,000,000
1,000,000	1	about 20	1,000,000	about 20,000,000	1,000,000,000,000

A modern processor does roughly a billion simple steps per second. At a million elements, an $O(n)$ solution finishes in about a millisecond. An $O(n^2)$ solution takes about a quarter of an hour.

The same notation describes memory. A solution that uses $O(n)$ extra memory stores something for each input element. A solution that uses $O(1)$ extra memory uses a fixed amount, however large the input.

NOTE: $O(\log n)$ appears when each step cuts the remaining work in half. Halving a million items takes about 20 steps to get down to one, because 2 multiplied by itself 20 times is about a million. Section 3.10 uses this.

3.2 How a `Vec` stores its elements

A `Vec<T>` is a growable array. Its elements sit next to each other in one block of heap memory. The `Vec` value itself is small. It holds three numbers. The first is a pointer to the block. The second is the number of elements in use, its **length**. The third is how many elements the block has room for, its **capacity**. Figure 3.1 shows a vector of four `i32` values with room for six.

```
let mut v: Vec<i32> = Vec::with_capacity(6); then push 10, 20, 30, 40
```

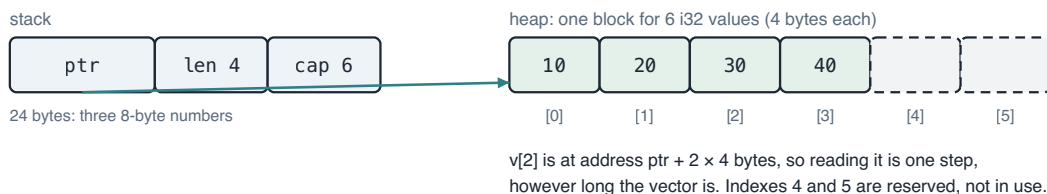


Figure 3.1 A `Vec<i32>` with length 4 and capacity 6. The elements are next to each other, so the address of any element can be computed from its index.

Because the elements are next to each other, reading `v[i]` takes one step. The address of element `i` is the block's address plus `i` times the size of one element. The length of the vector does not matter. So indexing a `Vec` is $O(1)$.

When you push an element and the block is full, the vector needs a bigger block. Figure 3.2 shows what happens.

`v.push(50)` when `len == cap == 4`



1. Allocate a block twice as large. 2. Copy the old elements. 3. Write 50. 4. Free the old block.

Most pushes find room and cost one write. A push that grows copies everything, but growth doubles the capacity, so it happens rarely. On average a push costs a constant amount.

Figure 3.2 Pushing into a full vector. The vector allocates a block twice as large, copies the old elements, writes the new one, and frees the old block.

That copy costs $O(n)$, but it happens rarely, because each growth doubles the capacity. Spread over all the pushes, each push costs a constant amount on average. This is called **amortized $O(1)$** .

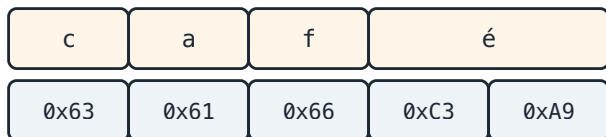
If you know how many elements you will push, `Vec::with_capacity(n)` allocates the whole block at once, and no copy happens at all. You will see this in several listings below.

WARNING: A push can move every element to a new block. So Rust does not let you keep a reference to an element while you push. The compiler rejects code that holds `&v[0]` across a `v.push(...)`, since the reference could point to freed memory afterward.

3.3 How a `String` stores text

A `String` is a `Vec<u8>` with one extra rule. Its bytes must always be valid UTF-8. **UTF-8** is the standard way of turning characters into bytes. An English letter takes one byte. Other characters take two, three, or four bytes. Figure 3.3 shows the four characters of `"café"`.

The String "café" in memory



4 characters (`chars().count() == 4`), stored in 5 bytes (`len() == 5`).

é needs two bytes, so byte index 4 is in the middle of a character.

Figure 3.3 "café" has four characters and five bytes. The letter é takes two bytes.

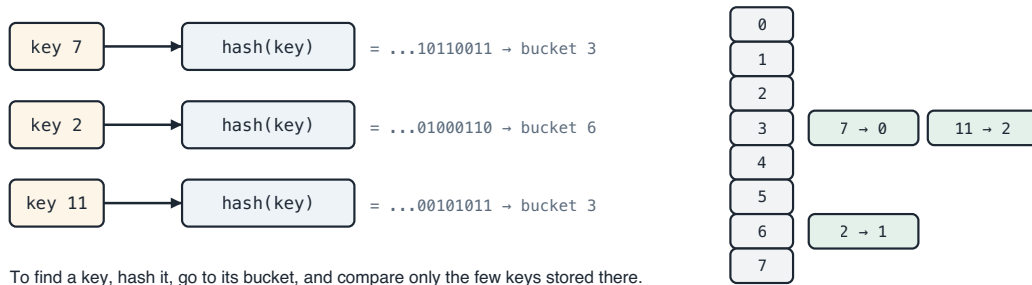
This has three consequences you will meet in this chapter:

- `s.len()` counts bytes, not characters. `"café".len()` is 5.
- `s.chars()` walks the characters one at a time, decoding the bytes as it goes.
- You cannot index a `String` with `s[3]`. A byte index might fall in the middle of a character, as index 4 does in figure 3.3, so Rust does not allow it.

3.4 How a `HashMap` finds a key

A `HashMap<K, V>` stores pairs of a key and a value, and finds the value for a key quickly. It works in three steps (figure 3.4):

1. It runs the key through a **hash function**, which turns any key into a large number. The same key always gives the same number.
2. It uses some of the bits of that number to pick a **bucket**, one slot in an array of buckets.
3. It stores the pair in that bucket, or looks for it there.

A hash map with 8 buckets holding three keys

To find a key, hash it, go to its bucket, and compare only the few keys stored there.

Figure 3.4 A map with 8 buckets. Keys 7 and 11 happen to land in the same bucket, so a lookup there compares both.

Two keys can land in the same bucket, as 7 and 11 do in the figure. Then a lookup compares the keys in that bucket one by one. The map keeps the number of keys per bucket small by growing its bucket array as it fills. So on average, `insert` and `get` are $O(1)$, however many keys the map holds.

Rust's `HashMap` uses a hash function with a random starting value, chosen when the program starts. This stops an attacker from sending keys that all land in one bucket. It also means the order in which you iterate over a map changes from one run to the next.

Now you have the tools for the problems. Each one starts with the simplest correct idea, then shows how a collection makes it faster.

3.5 Finding two numbers that add up to a target

The problem. Given an array of numbers and a target, find two positions whose numbers add up to the target. For `[2, 7, 11, 15]` and target 9, the answer is positions 0 and 1, because $2 + 7 = 9$.

The simple idea. Try every pair. For each position `i`, check every later position `j`. That is about $n \times n / 2$ pairs, so $O(n^2)$.

A faster idea. Walk the array once. For each number, you know which partner it needs: `target - num`. If you have already seen that partner, you are done. To answer "have I seen it?" in one step, keep a `HashMap` from each number seen so far to its position.

Trace it on `[2, 7, 11, 15]` with target 9:

Position	Number	Partner needed	In the map?	Map afterward
0	2	7	no	{2: 0}
1	7	2	yes, at position 0	answer: (0, 1)

Listing 3.1 Two Sum with one pass and a hash map. `src/problems/two_sum.rs`

```

    //! Two Sum: find the pair of indices whose values add up to a target.
    //!
    //! Use a `HashMap` for one pass: O(n) time, O(n) space.

    use std::collections::HashMap;

    pub fn two_sum(nums: &[i32], target: i32) -> Option<(usize, usize)> {
        let mut seen = HashMap::with_capacity(nums.len());
        for (idx, &num) in nums.iter().enumerate() {
            match seen.get(&(target - num)) {
                Some(&prev) => return Some((prev, idx)),
                None => {
                    seen.insert(num, idx);
                }
            }
        }
        None
    }

    #[cfg(test)]
    mod tests {
        use super::*;

        #[test]
        fn finds_pair() {
            let nums = [2, 7, 11, 15];
            assert_eq!(two_sum(&nums, 9), Some((0, 1)));
        }

        #[test]
        fn returns_none_when_missing() {
            assert_eq!(two_sum(&[1, 2, 3], 100), None);
        }
    }

```

Walk through the function:

- The parameter `nums: &[i32]` is a **slice**, a borrowed view of a sequence of `i32` values. You can pass a `Vec`, an array, or part of either.
- `HashMap::with_capacity(nums.len())` sizes the map for the worst case, so it never grows during the loop.
- `nums.iter().enumerate()` gives pairs of (position, reference to number). The pattern `(idx, &num)` copies the number out of the reference.

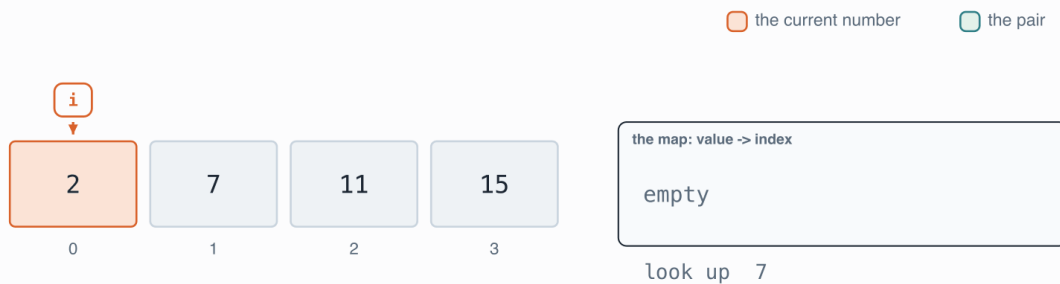
- `seen.get(&(target - num))` looks up the partner. `get` takes a reference to the key, hence the `&`.
- If the partner is found, the function returns both positions. Otherwise it records this number.

The lookup comes before the insert. Think of target 4 with a single 2 in the array. If the insert came first, the 2 would find itself and pair with itself.

The function returns `Option<(usize, usize)>`: `Some` with the two positions, or `None` if no pair exists. It runs in $O(n)$ time and uses $O(n)$ extra memory for the map.

Two Sum: remember each number so a later one can find its partner

One pass. For each x , ask the map for $\text{target} - x$.



$i = 0$ holds 2. The partner it needs is $9 - 2 = 7$, and the map is empty, so 2 is stored instead.

The map turns a second scan into a single lookup.

$i = 0$ $x = 2$ need = 7

Animation 3.1 The same trace as the table, one frame per number. The map on the right is what makes the partner lookup one step: 7 does not search the array again, it asks the map for $9 - 7$. Two of the four numbers are read before the pair is complete.

WARNING: `target - num` is computed in `i32`. For numbers near the ends of the `i32` range, such as `i32::MIN`, the subtraction overflows, and a debug build stops with a panic. Using `i64` for the arithmetic, or `target.checked_sub(num)`, avoids it.

3.6 A vector used as a stack

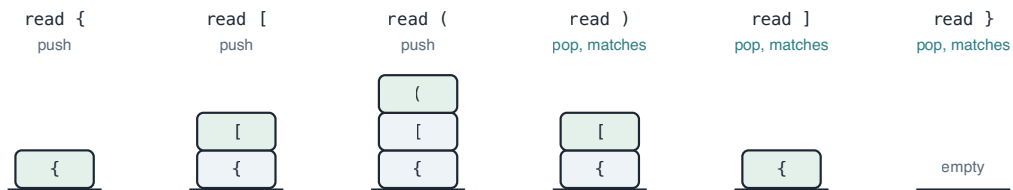
A **stack** is a collection where you add and remove items at the same end, called the top. The last item you added is the first one you remove. A `Vec` works as a stack. `push` adds to the end, and `pop` removes the last item and returns it. `last` looks at the end without removing it. All three are $O(1)$.

3.6.1 Checking brackets

The problem. A string of brackets is valid if every opening bracket is closed by the matching kind, in the right order. `"{[(())]}"` is valid. `"([)]"` is not, because `]` arrives while `(` is still open.

The idea. Read the string from left to right. Push each opening bracket onto a stack. When a closing bracket arrives, the top of the stack must be its matching opener; pop it and continue. At the end, the stack must be empty. Figure 3.5 traces `"{[(())]}"`.

Checking "{[(())]}" with a stack



The top of the stack is always the most recent unclosed opener, the one the next closer must match.

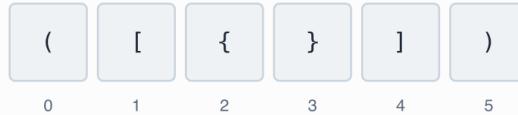
Figure 3.5 Checking `"{[(())]}"`. Each closing bracket pops the opener on top, which must be its partner.

Valid brackets: a closer must match the most recent unmatched opener

Counting openers is not enough. The order decides the answer.

□ the stack □ the character now

input



stack



Start before the first character. The stack is empty.

The stack is the reason a single counter cannot answer this.

i = -1 char = - stack = 0

Animation 3.5 The same six characters, with the stack drawn below. Watch the two directions the same character can move: an opener is always pushed, and a closer always pops. That is why a count of openers cannot work, and why the last opener is the first one to be closed.

Listing 3.2 Valid Parentheses. src/problems/valid_parentheses.rs

```

//! Valid Parentheses: classic stack problem with idiomatic pattern matching.
//!
pub fn is_valid(s: &str) -> bool {
    use std::collections::HashMap;

    let mut stack = Vec::with_capacity(s.len());

    // Maps each closing delimiter to the opening delimiter it must match.
    let expected = HashMap::from([(')', '('), (']', '['), ('}', '{')]);

    for ch in s.chars() {
        match ch {
            '(' | '[' | '{' => stack.push(ch),
            // Every other character, including one outside the six delimiters,
            // must close the top of the stack.
            _ => {
                if stack.pop().as_ref() != expected.get(&ch) {
                    return false;
                }
            }
        }
    }
}

```

```

        }
    }
}

stack.is_empty()
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn valid_cases() {
        assert!(is_valid("()"));
        assert!(is_valid("()[{}]"));
        assert!(is_valid("{}([)]"));
    }

    #[test]
    fn invalid_cases() {
        assert!(!is_valid("[)"]);
        assert!(!is_valid("([)]"));
        assert!(!is_valid("("));
        assert!(!is_valid(")"));
        assert!(!is_valid("(("));
    }
}
}

```

The map `expected` records, for each closing bracket, the opener it needs. `HashMap::from(...)` builds a map from an array of pairs.

The `match` has two arms. The first pushes any of the three openers. The second handles every other character and compares two `Option` values:

- `stack.pop()` removes the top and returns `Some(opener)`, or `None` if the stack is empty. `.as_ref()` turns `Option<char>` into `Option<&char>`, so it can be compared with the next value.
- `expected.get(&ch)` returns `Some(&opener)` for a closing bracket.

If they differ, the string is invalid. An empty stack gives `None`, which differs from `Some`, so an extra closer is caught. After the loop, `stack.is_empty()` catches openers that were never closed.

The tests list valid and invalid examples, including `"[()]"`, a lone `"")"`, and `"(("`.

WARNING: Suppose a character that is not a bracket arrives while the stack is empty. It gives `None` on both sides of the comparison, so it passes. `is_valid("a")` returns `true`. If your input can contain other characters, skip or reject them in their own arm.

3.6.2 A stack that knows its minimum

The problem. Build a stack that supports `push`, `pop`, `top`, and `get_min`, the smallest value currently in it, all in $O(1)$.

The idea. Scanning the stack for the minimum takes $O(n)$. Instead, keep a second stack beside the first. Each time you push a value, also push the minimum so far. Then the minimum of the whole stack is always on top of the second stack. When you pop, pop both.

Trace pushes of -2, 0, and -3:

Operation	values	mins	get_min()
push -2	[-2]	[-2]	-2
push 0	[-2, 0]	[-2, -2]	-2
push -3	[-2, 0, -3]	[-2, -2, -3]	-3
pop	[-2, 0]	[-2, -2]	-2

Listing 3.3 Min Stack with a parallel stack of minimums. `src/problems/min_stack.rs`

```

///! Min Stack: O(1) push, pop, top, and get_min.
///!
///! We keep a parallel `mins` stack. Each push stores the current minimum at that
///! point in history, so pop is symmetric.

#[derive(Debug, Default)]
pub struct MinStack {
    values: Vec<i32>,
    mins: Vec<i32>,
}

impl MinStack {
    pub fn new() -> Self {
        Self::default()
    }

    pub fn push(&mut self, val: i32) {
        match self.mins.last() {
            Some(&min) if min <= val => self.mins.push(min),
            _ => self.mins.push(val),
        }
    }
}

```



```

        self.values.push(val);
    }

    pub fn pop(&mut self) -> Option<i32> {
        self.mins.pop();
        self.values.pop()
    }

    pub fn top(&self) -> Option<i32> {
        self.values.last().copied()
    }

    pub fn get_min(&self) -> Option<i32> {
        self.mins.last().copied()
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn stack_tracks_minimum() {
        let mut stack = MinStack::new();
        stack.push(-2);
        stack.push(0);
        stack.push(-3);
        assert_eq!(stack.get_min(), Some(-3));
        assert_eq!(stack.pop(), Some(-3));
        assert_eq!(stack.top(), Some(0));
        assert_eq!(stack.get_min(), Some(-2));
    }

    #[test]
    fn handles_duplicates() {
        let mut stack = MinStack::new();
        stack.push(1);
        stack.push(1);
        stack.pop();
        assert_eq!(stack.get_min(), Some(1));
    }
}

```

`#[derive(Default)]` gives `MinStack` a `default()` function that creates two empty vectors, and `new` calls it.

In `push`, `self.mins.last()` returns the current minimum, if any. The arm `Some(&min)` if `min <= val` has a **guard**, the `if` after the pattern. It pushes the old minimum again when the new value is not smaller. The `_` arm covers both an empty stack and a new smaller value, and pushes `val`.

The guard uses `<=`, not `<`. When you push a value equal to the current minimum, the minimum is recorded again. So popping one copy leaves the other, and the minimum stays correct. The test `handles_duplicates` checks this case.

`top` and `get_min` use `.copied()` to turn `Option<i32>` into `Option<i32>`, so the caller gets a plain number.

3.7 Two indexes moving toward each other

Some problems need you to look at both ends of an array at once. Keep one index at the start and one at the end, and move them toward each other. Each step moves at least one index, so the whole walk is $O(n)$.

3.7.1 Reversing a string in place

The idea. Swap the first and last bytes, then the second and second-to-last, and stop when the indexes meet (figure 3.6).

Reversing "abcd" with two pointers

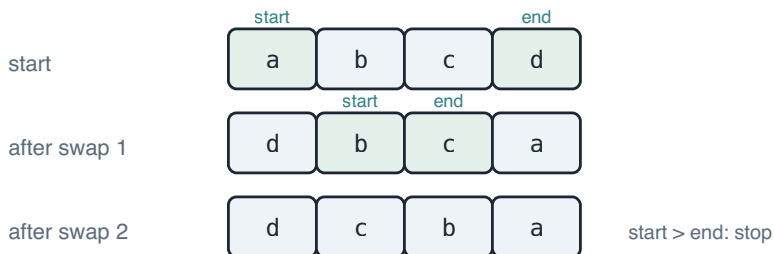


Figure 3.6 Reversing "abcd". Two swaps, then the indexes cross.

There is a complication. As section 3.3 explained, a `String` must stay valid UTF-8, and swapping the bytes of a two-byte character would break it. So Rust does not let you change a `String`'s bytes directly in safe code. `String::as_mut_vec` gives you the bytes, but it is marked `unsafe`. `unsafe` marks code where the compiler cannot check a rule, so the programmer must. Here the rule is "the bytes are valid UTF-8 when you are done".

Listing 3.4 The reversing function (lines 2 to 14). `src/bin/reverse_string.rs`

```
fn reverse_str(s: &mut String) {
    // SAFETY: s._as_mut_vec() is unsafe because it exposes the inner mutability to it's
    // character buffer, and we must ensure
    // to leave valid UTF-8. We enforce ASCII only characters.
    assert!(s.is_ascii());
    let bytes = unsafe { s.as_mut_vec() };

    let (mut start, mut end) = (0, bytes.len() - 1);

    while start < end {
        bytes.swap(start, end);
        start+=1; end-=1;
    }
}
```

The function keeps the rule by checking first: `assert!(s.is_ascii())` stops the program unless every byte is a plain ASCII character. Reversing ASCII bytes gives ASCII, which is valid UTF-8. The `unsafe` block is then as small as possible, one call.

`bytes.swap(start, end)` exchanges two elements of the vector. The loop runs while `start < end`, so the middle element of an odd-length string stays where it is.

Listing 3.5 The complete program, with its tests. `src/bin/reverse_string.rs`

```
fn reverse_str(s: &mut String) {
    // SAFETY: s._as_mut_vec() is unsafe because it exposes the inner mutability to it's
    // character buffer, and we must ensure
    // to leave valid UTF-8. We enforce ASCII only characters.
    assert!(s.is_ascii());
    let bytes = unsafe { s.as_mut_vec() };

    let (mut start, mut end) = (0, bytes.len() - 1);

    while start < end {
        bytes.swap(start, end);
        start+=1; end-=1;
    }
}

fn main() {
    for value in ["abcd", "abc", "a", "rust"] {
        let mut s = String::from(value);
        reverse_str(&mut s);
        println!("{value:>6} -> {s}");
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
```

```

fn an_even_length_string_is_reversed() {
    let mut s = String::from("abcd");
    reverse_str(&mut s);
    assert_eq!(s, "dcba");
}

#[test]
fn an_odd_length_string_keeps_its_middle_byte() {
    let mut s = String::from("abc");
    reverse_str(&mut s);
    assert_eq!(s, "cba");
}

#[test]
fn a_single_byte_is_unchanged() {
    let mut s = String::from("a");
    reverse_str(&mut s);
    assert_eq!(s, "a");
}

#[test]
fn the_buffer_is_reused() {
    let mut s = String::from("abcd");
    let capacity = s.capacity();
    let address = s.as_ptr();
    reverse_str(&mut s);
    assert_eq!(s.capacity(), capacity);
    assert_eq!(s.as_ptr(), address);
}

#[test]
fn reversing_twice_restores_the_string() {
    let mut s = String::from("interview");
    reverse_str(&mut s);
    reverse_str(&mut s);
    assert_eq!(s, "interview");
}

#[test]
#[should_panic]
fn non_ascii_input_panics() {
    let mut s = String::from("café");
    reverse_str(&mut s);
}
}

```

The test `the_buffer_is_reused` records the string's capacity and the address of its bytes before the call, then checks both afterward. They are unchanged, so the reversal used no new memory. `non_ascii_input_panics` is marked `#[should_panic]`: it passes only if the call panics, which the `assert!` does for "café".

```

$ cargo run --bin reverse_string
abcd -> dcba
abc -> cba

```

```
a -> a
rust -> tsur
```

WARNING: For an empty string, `bytes.len() - 1` subtracts 1 from 0 in `usize`, which cannot go below zero. A debug build panics. Return early when the string is empty.

3.7.2 Three numbers that add up to a target

The problem. Find every set of three numbers in an array that adds up to a target, without listing the same set twice.

The idea. Sort the array first. Then fix the first number, and look for the other two in the rest of the array. Use two indexes: `left` starts right after the fixed number, and `right` starts at the end. Because the array is sorted, you know which index to move:

- If the sum is too small, move `left` right. That can only increase the sum.
- If the sum is too large, move `right` left. That can only decrease it.
- If the sum is right, record the three numbers, then move both.

Each fixed number costs one $O(n)$ walk, and there are n fixed numbers, so the whole search is $O(n^2)$. Sorting costs $O(n \log n)$, which is smaller.

Listing 3.6 Three Sum. `src/problems/three_sum.rs`

```
/// Three Sum: every unique triplet of values that adds up to a target.
///
/// Sort, fix the first value, then close in on the other two with two pointers.
/// Sorts the caller's slice in place:  $O(n^2)$  time, no extra space beyond the result.

// Import the variants directly into scope
use std::cmp::Ordering::{Equal, Greater, Less};

pub fn three_sum(numbers: &mut [i32], target: i32) -> Vec<(i32, i32, i32)> {
    numbers.sort_unstable();

    let mut results = Vec::new();
    let target = i64::from(target);

    for i in 0..numbers.len() {
        // Skip a first value that was already used, so triplets stay unique.
        if i > 0 && numbers[i] == numbers[i - 1] {
            continue;
        }

        let (mut left, mut right) = (i + 1, numbers.len().saturating_sub(1));
        while left < right {
            // Widen to i64 so the sum of three i32 values cannot overflow.
```

```

    let current_sum = numbers[i] as i64 + numbers[left] as i64 + numbers[right]
    as i64;

    match current_sum.cmp(&target) {
        Less => left += 1,
        Greater => right -= 1,
        Equal => {
            results.push((numbers[i], numbers[left], numbers[right]));

            while left < right && numbers[left] == numbers[left + 1] {
                left += 1;
            }
            while left < right && numbers[right] == numbers[right - 1] {
                right -= 1;
            }

            left += 1;
            right -= 1;
        }
    }
}

results
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn finds_all_unique_triplets() {
        let result = three_sum(&mut [-1, 0, 1, 2, -1, -4], 0);
        assert_eq!(result, vec![(-1, -1, 2), (-1, 0, 1)]);
    }

    #[test]
    fn skips_duplicates_on_every_position() {
        assert_eq!(three_sum(&mut [0, 0, 0, 0], 0), vec![(0, 0, 0)]);
        assert_eq!(three_sum(&mut [-2, 0, 0, 2, 2], 0), vec![(-2, 0, 2)]);
    }

    #[test]
    fn supports_a_non_zero_target() {
        assert_eq!(
            three_sum(&mut [1, 2, 3, 4, 5], 9),
            vec![(1, 3, 5), (2, 3, 4)]
        );
    }

    #[test]
    fn returns_nothing_when_no_triplet_exists() {
        assert!(three_sum(&mut [1, 2, 3], 100).is_empty());
        assert!(three_sum(&mut [1, 2], 3).is_empty());
        assert!(three_sum(&mut [], 0).is_empty());
    }

    #[test]
    fn does_not_overflow_on_extreme_values() {

```

```

        assert_eq!(
            three_sum(&mut [i32::MAX, i32::MAX, i32::MIN], i32::MAX - 1),
            vec![(i32::MIN, i32::MAX, i32::MAX)]
        );
    }
}

```

A few details:

- use `std::cmp::Ordering::{Equal, Greater, Less}`; lets the code write `Less` instead of `Ordering::Less`. `current_sum.cmp(&target)` returns one of the three.
- Duplicate sets are skipped at each position. For the fixed number, the `continue` skips a value equal to the previous one. After a match, the two inner `while` loops step `left` and `right` past equal values.
- The sum is computed in `i64`. Three `i32` values can add up to more than `i32` can hold. The test `does_not_overflow_on_extreme_values` uses `i32::MAX` twice to check this.
- `numbers.len().saturating_sub(1)` gives 0 instead of underflowing when the slice is empty.

Three Sum: fix one number, then close two pointers over the rest

Sorted input turns a pointer move into a decision instead of a guess.

 fixed the two pointers the triple



triples so far: none

The sum is -3, below 0. The left value must grow, so `lo` moves right.

Every move drops one index, so the pair of pointers costs one scan, not two.

`i = 0` `lo = 1` `hi = 5` `sum = -3`

Animation 3.6 Three Sum on `[-4, -1, -1, 0, 1, 2]`. Each frame is one comparison. `i` is the fixed number, and `lo` and `hi` are the two pointers. Read the sign of the sum in the counter row: a negative sum moves `lo` right and a positive sum moves `hi` left, so the two pointers never cross without the sum being checked. The frame where `i` repeats `-1` shows the duplicate skip.

3.8 A window that slides along a string

The problem. Find the length of the longest piece of a string in which no character repeats. In `"pwwkew"`, the answer is 3, for `"wke"` or `"kew"`.

The idea. Keep a **window**, a range `start..end` of the string with no repeated character. Move `end` forward one character at a time. When the new character already appears inside the window, move `start` to the position right after its earlier one. The window never contains a repeat, and the answer is the largest window seen.

To know where a character last appeared, keep a `HashMap` from each character to its last position. Figure 3.7 shows the moment in "pwwkew" when the second `w` arrives.



Figure 3.7 At `end = 5`, the `w` was last seen at index 2. `start` moves to 3, and the window is "kew".

Sliding window: the longest run with no repeated character

The window grows at the right edge and shrinks at the left edge.

☒ in the window ☒ the repeat ☐ outside



end reaches 0. The window is "p", and best becomes 1.

start only ever moves forward, so a character leaves the window once.

start = 0 end = 0 best = 1

Animation 3.7 The same window over all seven steps. **end** moves right, and **start** moves forward only when the character arriving is already inside the window. **end** visits each index once, and **start** only moves forward, so the scan is $O(n)$.

Listing 3.7 Longest substring without a repeated character. src/problems/sliding_window.rs

```
///! Longest substring without repeating characters.
///!
///! Sliding window with `HashMap<char, usize>`; the window start is monotonic.

use std::collections::HashMap;

pub fn length_of_longest_substring(s: &str) -> usize {
    let mut last_seen = HashMap::with_capacity(s.len());

    let (mut start, mut longest) = (0, 0);

    for (end, ch) in s.chars().enumerate() {
        if let Some(&previous) = last_seen.get(&ch) {
            start = start.max(previous + 1);
        }
        longest = longest.max(end - start + 1);
        last_seen.insert(ch, end);
    }

    longest
}
```

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn examples() {
        assert_eq!(length_of_longest_substring("abcabcbb"), 3);
        assert_eq!(length_of_longest_substring("bbbb"), 1);
        assert_eq!(length_of_longest_substring("pwwkew"), 3);
        assert_eq!(length_of_longest_substring(""), 0);
    }
}
```

`if let Some(&previous) = last_seen.get(&ch)` runs its block only when the character has been seen before. Then `start = start.max(previous + 1)` moves `start` past the earlier position. The `max` stops `start` from moving backward when the earlier position is already outside the window. For example, in `"abba"`, the last `a` was seen at 0, but `start` is already at 2.

`end - start + 1` is the current window's length. Both `start` and `end` only move forward, so the loop does $O(n)$ work.

The positions come from `chars().enumerate()`, so they count characters, not bytes. That is what the problem asks for.

3.9 Sort, then walk once: merging ranges

The problem. Given ranges such as `[1,3]` `[2,6]` `[8,10]` `[15,18]`, merge every group that overlaps. The answer here is `[1,6]` `[8,10]` `[15,18]`.

The idea. Sort the ranges by their start. Then walk them once. Each range either overlaps the last merged range, and extends it, or starts a new merged range (figure 3.8).

Merging [1,3] [2,6] [8,10] [15,18] after sorting by start

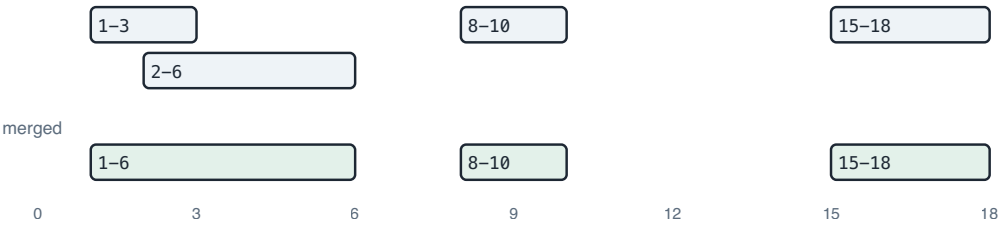
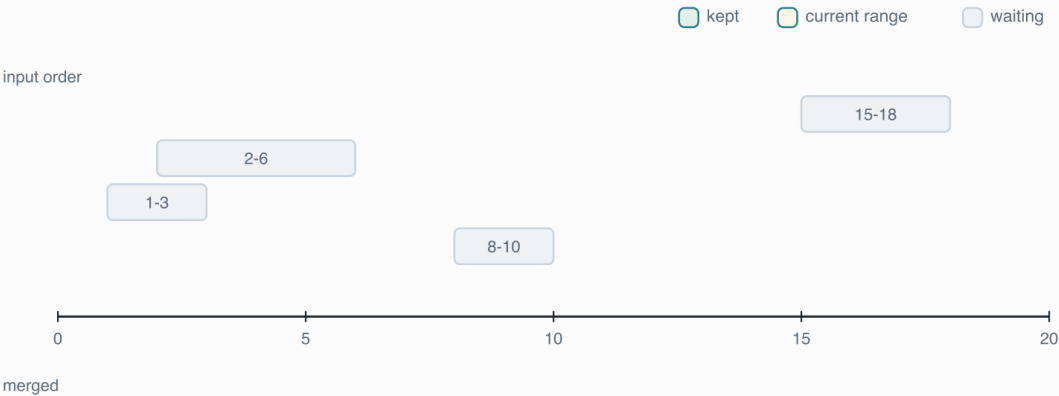


Figure 3.8 After sorting, a range overlaps the previous merged range exactly when its start is not after that range's end.

Merge intervals: sort by start, then extend or keep

Every range is visited once after the sort.



The input arrives unsorted, so no range can be trusted to arrive next.

Sorting is what makes the decision local: only the last merged range matters.

current = - merged = 0 ranges = 4

Animation 3.8 Every frame names the range being compared and the range kept so far. The teal-outlined bar is the range currently being merged; the green bars below the axis have already been closed. The first frame is the unsorted input, which is why the sort has to happen before the walk.

The file solves the problem three times: with a named struct, with plain tuples, and in place. First the struct and the first version:

Listing 3.8 Merging ranges held in a struct (lines 6 to 28). `src/problems/merge_intervals.rs`

```
#[derive(Debug, PartialEq, Clone, Copy)]
pub struct Interval {
    start: i32,
    end: i32
}

pub fn merge_intervals(mut intervals: Vec<Interval>) -> Vec<Interval> {
    if intervals.is_empty() {
        return intervals;
    }

    intervals.sort_unstable_by_key(|interval| interval.start);

    let mut merged: Vec<Interval> = Vec::with_capacity(intervals.len());
    for Interval { start, end } in intervals {
        match merged.last_mut() {
            Some(previous) if start <= previous.end => {
                previous.end = previous.end.max(end);
            }
            _ => merged.push(Interval{start, end}),
        }
    }
    merged
}
```

`sort_unstable_by_key(|interval| interval.start)` sorts by start. "Unstable" means equal starts may change order, which does not affect the result and is slightly faster.

The loop header `for Interval { start, end } in intervals` takes each range apart into two variables.

`merged.last_mut()` returns a mutable reference to the last merged range, or `None` if there is none yet. The guard `if start <= previous.end` checks for overlap. When it holds, the code extends the range in place through the reference: `previous.end = previous.end.max(end)`. The `max` handles a range that lies entirely inside the previous one. In every other case, the `_` arm starts a new merged range.

The second version does the same with `(i32, i32)` tuples, so the end is written `previous.1`:

Listing 3.9 The same merge over tuples (lines 30 to 48).

```
pub fn merge_intervals2(mut intervals: Vec<(i32, i32)>) -> Vec<(i32, i32)> {
    if intervals.is_empty() {
        return intervals;
    }
}
```

```

    intervals.sort_unstable_by_key(|(start, _)| *start);

    let mut merged: Vec<(i32, i32)> = Vec::with_capacity(intervals.len());

    for &(start, end) in &intervals {
        match merged.last_mut() {
            Some(previous) if start <= previous.1 => {
                previous.1 = previous.1.max(end);
            }
            _ => merged.push((start, end)),
        }
    }
    merged
}

```

It works the same way. The struct version is easier to read, because `previous.end` says what `previous.1` leaves to you to remember.

The third version needs no second vector. It writes the merged ranges into the front of the input vector:

Listing 3.10 Merging in place (lines 50 to 72).

```

/// In place: `last` is the last merged interval at the front of the vector.
/// O(1) extra space.
pub fn merge_intervals_in_place(intervals: &mut Vec<Interval>) {
    if intervals.is_empty() {
        return;
    }

    intervals.sort_unstable_by_key(|interval| interval.start);

    let mut last = 0;
    for i in 1..intervals.len() {
        match (intervals[last], intervals[i]) {
            (previous, current) if current.start <= previous.end => {
                intervals[last].end = previous.end.max(current.end);
            }
            (_, current) => {
                last += 1;
                intervals[last] = current;
            }
        }
    }
    intervals.truncate(last + 1);
}

```

`last` is the position of the last merged range. For each later range, the code either extends `intervals[last]`, or moves `last` forward one place and copies the range there. At the end, `truncate(last + 1)` drops the leftovers. This version uses $O(1)$ extra memory.

`match (intervals[last], intervals[i])` copies both ranges out of the vector into a pair. That works because `Interval` derives `Copy`. Because the arms work on copies, they are free to write to `intervals[last]` without a borrowing conflict.

Listing 3.11 The tests (lines 74 to 106).

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn merges_overlapping_and_adjacent() {
        let input = [(1, 3), (2, 6), (8, 10), (15, 18)].map(|(start, end)| Interval { start, end });
        let expected = [(1, 6), (8, 10), (15, 18)].map(|(start, end)| Interval { start, end });
        assert_eq!(merge_intervals(input.into()), expected);
    }

    #[test]
    fn handles_empty_and_disjoint() {
        assert_eq!(merge_intervals(vec![], vec![]));
        let input = [(1, 2), (3, 4)].map(|(start, end)| Interval { start, end });
        let expected = [(1, 2), (3, 4)].map(|(start, end)| Interval { start, end });
        assert_eq!(merge_intervals(input.into()), expected);
    }

    #[test]
    fn merges_in_place() {
        let mut intervals: Vec<Interval> = [(8, 10), (1, 3), (15, 18), (2, 6)]
            .map(|(start, end)| Interval { start, end })
            .into();
        merge_intervals_in_place(&mut intervals);
        let expected = [(1, 6), (8, 10), (15, 18)].map(|(start, end)| Interval { start, end });
        assert_eq!(intervals, expected);

        let mut empty = vec![];
        merge_intervals_in_place(&mut empty);
        assert!(empty.is_empty());
    }
}
```

The tests build ranges from tuples with `.map(|(start, end)| Interval { start, end })` on arrays. They cover overlapping ranges, an empty input, ranges that do not touch, and unsorted input for the in-place version.

All three versions merge ranges that only touch, such as `[1,3]` and `[3,5]`, because the check is `<=`.

3.10 Halving the search: binary search

In a sorted array, you can find a value without looking at every element. Compare the value with the middle element. If the value is smaller, it can only be in the left half; if larger, only in the right half. Discard the other half and repeat. Figure 3.9 finds 23 in eight values in three steps.

Binary search for 23 in a sorted array of 8 values



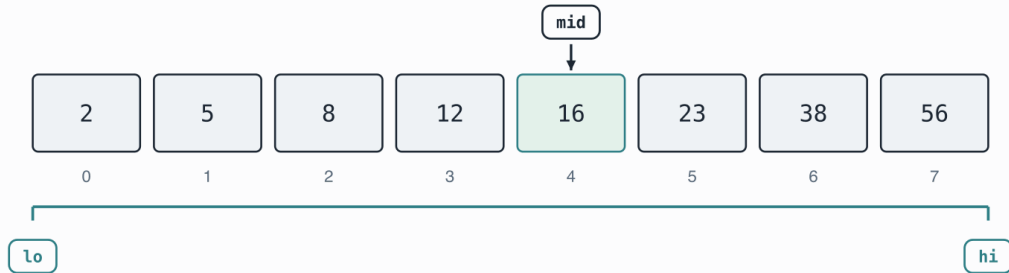
Each step compares the middle value (cream) with 23 and discards the half that cannot contain it.

Figure 3.9 Binary search. Each step halves the range, so a million elements need about 20 steps.

Binary search: each comparison halves the range

The array is sorted. The range is half open: lo is included and hi is not.

☒ in range ☐ ruled out



`lo = 0`, `hi = 8`, `mid = 4`. `a[4] = 16`, which is below 23, so 23 can only lie to the right. `lo = 5`.

The left half, indexes 0 to 4, is now out of the search.

`lo = 0` `hi = 8` `mid = 4` `a[mid] = 16`

Animation 3.9 The search for 23, one comparison per frame. The dashed box is the live range `lo..hi`. The green cell is `mid`, and the grey cells are already ruled out. Eight elements take three comparisons, and each extra comparison doubles the range that can be searched.

The code in this section uses a **half-open range** `low..high`. `low` is the first position still in the range, and `high` is one past the last. The range is empty when `low == high`.

3.10.1 Searching a rotated array

The file solves a harder version. A sorted array has been **rotated**: some elements from the front were moved to the back, as in `[4, 5, 6, 7, 0, 1, 2]`. You still want $O(\log n)$.

The idea rests on one observation. If you split a rotated array at any middle position, at least one of the two halves is still sorted. You can tell which by comparing its end values. Then:

- If the left half is sorted and the target lies between its first and middle values, search the left half.
- If the right half is sorted and the target lies between its middle and last values, search the right half.

- Otherwise, search the other half.

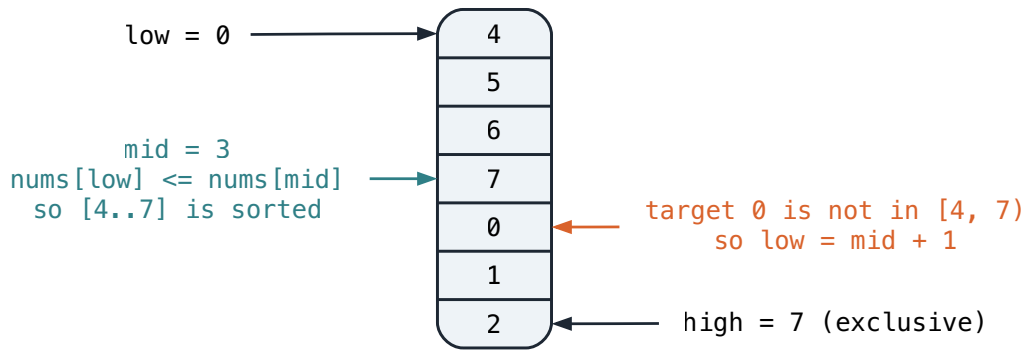


Figure 3.10 The first step of searching for 0. The left half 4 to 7 is sorted and cannot contain 0, so the search continues on the right.

Binary search on a rotated sorted array

At every split, at least one half is still sorted. That half decides the move.

☒ the sorted half

☐ ruled out

☐ still in range



The left half 4..7 is sorted and 0 is not inside it, so the search moves right: $lo = mid + 1$.

A sorted half can be tested with two comparisons, which is what keeps the cost logarithmic.

$lo = 0$

$hi = 7$

$mid = 3$

$nums[mid] = 7$

Animation 3.10 The whole search for 0, not just the first step. The dashed box is the live range and the green cell is `mid`. In the first frame the left half is sorted and does not hold the target, so the search moves right; in the second frame the left half 0..1 is sorted and does hold it, so the search moves left. Three comparisons again.

Listing 3.12 Search in a rotated sorted array. `src/problems/binary_search.rs`

```

/// Binary search on a rotated sorted array.
///
///  $O(\log n)$ . The code uses explicit range bounds and
/// sorted-half checks; no recursion or nested `if` chains are required.

pub fn search_rotated(nums: &[i32], target: i32) -> Option<usize> {
    let mut low = 0usize;
    let mut high = nums.len();

    while low < high {
        let mid = low + (high - low) / 2;
        if nums[mid] == target {
            return Some(mid);
        }
    }
}

```

```

    // 6,7, 1,2,3,4,5
    // start = 0 , end = 6, 0 + (6-0)/2 = 3

    // 4,5,6,1,2,3
    let left_is_sorted = nums[low] <= nums[mid];
    if left_is_sorted {
        if nums[low] <= target && target < nums[mid] {
            high = mid;
        } else {
            low = mid + 1;
        }
    } else if nums[mid] < target && target <= nums[high - 1] {
        low = mid + 1;
    } else {
        high = mid;
    }
}

None
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn finds_target_in_rotated_array() {
        let nums = [4, 5, 6, 7, 0, 1, 2];
        assert_eq!(search_rotated(&nums, 0), Some(4));
        assert_eq!(search_rotated(&nums, 3), None);
        assert_eq!(search_rotated(&nums, 5), Some(1));
    }

    #[test]
    fn handles_unrotated_and_single() {
        assert_eq!(search_rotated(&[1, 2, 3, 4], 3), Some(2));
        assert_eq!(search_rotated(&[1], 1), Some(0));
        assert_eq!(search_rotated(&[1], 2), None);
    }
}

```

`mid = low + (high - low) / 2` finds the middle. The shorter `(low + high) / 2` can overflow when both are very large; this form cannot.

`nums[low] <= nums[mid]` means the left half is sorted. The test that the target is in it is `nums[low] <= target && target < nums[mid]`. The right-half test uses `nums[high - 1]`, the last element, because `high` itself is one past the end.

The comments in the middle of the function are small worked examples. Working two or three by hand, as those comments do, is how you convince yourself that each branch is right.

3.11 Rotating a square grid in place

The problem. Rotate an $n \times n$ grid a quarter turn clockwise, without a second grid.

The idea. Do it in two simple steps (figure 3.11). First **transpose** the grid. That means swapping the element at row i , column j with the one at row j , column i . It flips the grid across its diagonal. Then reverse each row.



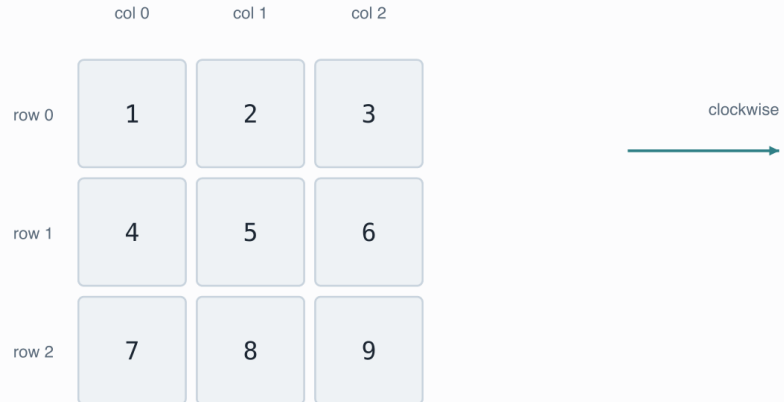
Transpose swaps $m[i][j]$ with $m[j][i]$. Reversing each row then gives a clockwise quarter turn.

Figure 3.11 Transpose, then reverse each row. The result is the grid turned a quarter turn clockwise.

Rotate a grid in place: transpose, then reverse every row

Two sweeps of simple swaps, with no second grid and no extra memory.

☐ swapping now ☒ in place



Start. Reading the grid row by row gives 1 2 3 / 4 5 6 / 7 8 9.

A transpose swaps across the diagonal. Reversing each row then turns it clockwise.

sweep = **start** cell = - grid = 3 x 3

Animation 3.11 Transpose then reverse, one swap at a time. Each frame names the two cells being exchanged, so you can see that the transpose visits only the cells above the diagonal. The last four frames reverse a whole row at once, which is what `row.reverse()` does.

Listing 3.13 The complete program. `src/bin/rotate_matrix.rs`

```
fn rotate(m: &mut Vec<Vec<i32>>) {
    let n = m.len();

    for i in 0..n {
        for j in i + 1..n {
            let tmp = m[i][j];
            m[i][j] = m[j][i];
            m[j][i] = tmp;
        }
    }

    for row in m.iter_mut() {
        row.reverse();
    }
}
```

```

}

fn main() {
    let mut m = vec![
        vec![1, 2, 3],
        vec![4, 5, 6],
        vec![7, 8, 9],
    ];

    rotate(&mut m);

    for row in &m {
        println!("{:?}", row);
    }
}

```

The grid is a `Vec<Vec<i32>>`, a vector of rows. The transpose loop starts `j` at `i + 1`, so it visits only the cells above the diagonal and swaps each pair once. Visiting every cell would swap each pair twice and undo the work.

The swap uses a temporary variable `tmp`. `std::mem::swap` needs two mutable references, and `m[i][j]` and `m[j][i]` are in different rows of the same outer vector. The borrow checker cannot see that they do not overlap, so it would reject taking both at once.

`for row in m.iter_mut()` gives a mutable reference to each row, and `row.reverse()` reverses it.

```

$ cargo run --bin rotate_matrix
[7, 4, 1]
[8, 5, 2]
[9, 6, 3]

```

3.12 Finding text inside text

The problem. Find the first position where a short string, the **needle**, appears in a longer one, the **haystack**.

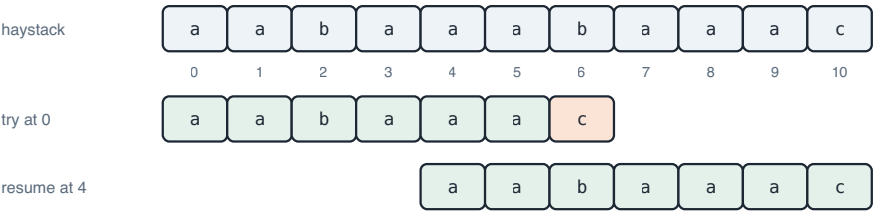
The simple idea. Try each position of the haystack, and compare the needle character by character. On a mismatch, move one position right and start over. In the worst case this compares every needle character at every position. For a haystack of length n and a needle of length m , that is $O(n \times m)$.

The better idea. Suppose a mismatch comes after several matching characters. You already know what those characters were, because they are the start of the needle. Often a later part

of them is also the start of the needle, so you can continue from there instead of starting over. The Knuth-Morris-Pratt algorithm (KMP) uses this. It never moves backward in the haystack, so it runs in $O(n + m)$.

Figure 3.12 shows one such jump.

Searching for "aabaaac" in "aabaaabaaac"



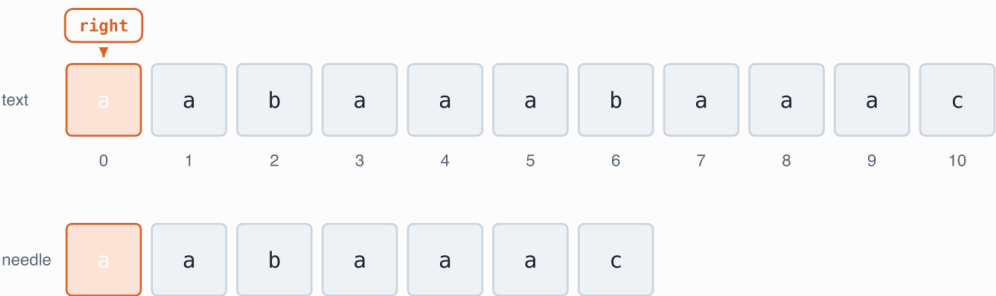
Six characters matched, then 'c' $\neq$ 'b'. $\text{lps}[5] = 2$ says the last two matched characters, "aa", are also the needle's start.

Figure 3.12 After six matching characters and a mismatch, the last two matched characters, "aa", are also the start of the needle. The search continues with them already matched, and finds the needle at position 4.

KMP search: reuse the characters that already matched

right only ever moves forward. On a mismatch, left falls back instead.

 matched so far the character under right



Start: left and right are both 0, so the needle is aligned with the first character.

Green is one run of characters that matched, reused rather than re-read.

left = 0 right = 0 start = 0

Animation 3.12 The whole search, one comparison per frame. The **right** pointer only ever moves right; the needle row shows how far the needle is shifted, which is **right** - **left** before a comparison. Watch the fallback frame: **right** stays at 6 while **left** drops from 6 to 2, so the characters "aa" are reused instead of re-read. That single move is the whole reason KMP is $O(n + m)$.

To know how far to jump, KMP first builds a table from the needle alone. For each position i , look at the piece `needle[0..=i]`. `lps[i]` is the length of the longest part of that piece that is both its start and its end. The whole piece does not count. The name "lps" stands for longest proper prefix that is also a suffix. For the needle in the figure:

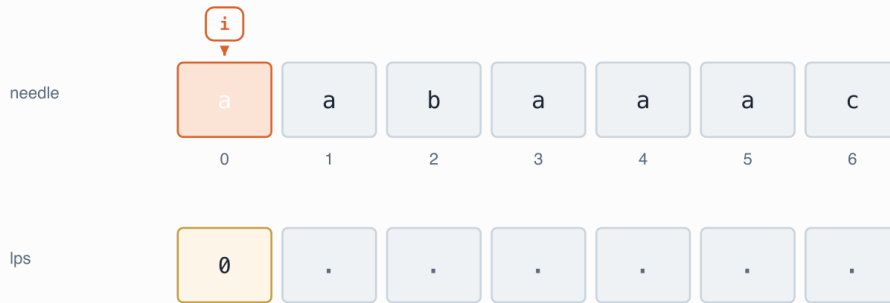
index	0	1	2	3	4	5	6
needle	a	a	b	a	a	a	c
lps	0	1	0	1	2	2	0

For example, `lps[4] = 2`, because "aabaa" starts with "aa" and ends with "aa".

KMP, first step: the longest proper prefix that is also a suffix

The table is built from the needle alone, before the haystack is read.

 matched prefix compared with i not written yet



`lps[0]` is 0. A one-character string has no proper prefix that is also a suffix.

The table is what the search falls back to, so it is built once, up front.

`i = 0` `length = 0` `needle[length] = a`

Animation 3.13 Building the table. The rust cell is `needle[i]` and the outlined cell is `needle[length]`, the character it is compared with. When they match, `length` grows and the green prefix is the longest one that is also a suffix. When they do not, the fallback frames show `length` dropping to `lps[length - 1]` while `i` stays put. A dot in the `lps` row is an entry the build has not written yet.

Listing 3.14 The search and the table (lines 1 to 35). `src/bin/kmp_pattern_matching.rs`

```
struct Solution { }
impl Solution {
    /// @param haystack the string to search in
    /// @param needle the pattern to find
    /// @return first index of needle in haystack, or -1
    pub fn str_str(haystack: String, needle: String) -> i32 {
        let (haystack, needle) = (haystack.as_bytes(), needle.as_bytes());
        if needle.is_empty() { return 0; }

        let lps = Self::build_lps(needle);
        let (mut left, mut right) = (0,
```

```

0); // left walks needle, right walks haystack

while right < haystack.len() {
    if haystack[right] == needle[left] { left += 1; right += 1; }
    else if left > 0 { left = lps[left - 1]; } // fall back
    else { right += 1; } // mismatch at the start

    if left == needle.len() { return (right - left) as i32; } // pattern found
}
-1
}

// LPS: longest proper prefix of needle[0..=right] that is also a suffix
fn build_lps(needle: &[u8]) -> Vec<usize> {
    let mut lps = vec![0; needle.len()];
    let (mut left, mut right) = (0, 1);

    while right < needle.len() {
        if needle[right] == needle[left] { left += 1; lps[right] = left; right += 1; }
        else if left > 0 { left = lps[left - 1]; }
        else { right += 1; } // lps[right] stays 0
    }
    lps
}
}

```

`str_str` works on bytes, from `as_bytes()`, so it can index directly. `left` counts how many needle characters currently match, and `right` walks the haystack. Each pass of the loop takes one of three branches:

1. The characters match: advance both.
2. They do not match, but some characters had matched: jump `left` back to `lps[left - 1]`, keeping `right` where it is.
3. They do not match and nothing had matched: advance `right`.

When `left` reaches the needle's length, the needle starts at `right - left`.

`build_lps` runs the same three-branch loop, comparing the needle against itself. Both loops only move forward, which gives the $O(n + m)$ total.

Listing 3.15 The complete program. `src/bin/kmp_pattern_matching.rs`

```

struct Solution { }
impl Solution {
    /// @param haystack the string to search in
    /// @param needle the pattern to find
    /// @return first index of needle in haystack, or -1
    pub fn str_str(haystack: String, needle: String) -> i32 {
        let (haystack, needle) = (haystack.as_bytes(), needle.as_bytes());
        if needle.is_empty() { return 0; }
    }
}

```

```

    let lps = Self::build_lps(needle);
    let (mut left, mut right) = (0,
0);           // left walks needle, right walks haystack

    while right < haystack.len() {
        if haystack[right] == needle[left] { left += 1; right += 1; }
        else if left > 0 { left = lps[left - 1]; } // fall back
        else { right += 1; } // mismatch at the start

        if left == needle.len() { return (right - left) as i32; } // pattern found
    }
    -1
}

// LPS: longest proper prefix of needle[0..=right] that is also a suffix
fn build_lps(needle: &[u8]) -> Vec<usize> {
    let mut lps = vec![0; needle.len()];
    let (mut left, mut right) = (0, 1);

    while right < needle.len() {
        if needle[right] == needle[left] { left += 1; lps[right] = left; right += 1; }
        else if left > 0 { left = lps[left - 1]; }
        else { right += 1; } // lps[right] stays 0
    }
    lps
}

fn main() {
    let cases = [
        ("sadbutsad", "sad"),
        ("leetcode", "leeto"),
        ("aabaabaabaac", "aabaac"),
        ("hello", ""),
    ];

    for (haystack, needle) in cases {
        let result = Solution::str_str(haystack.to_string(), needle.to_string());
        println!("str_str({haystack:?}, {needle:?}) = {result}\n");
    }
}

```

```

$ cargo run --bin kmp_pattern_matching
str_str("sadbutsad", "sad") = 0

str_str("leetcode", "leeto") = -1

str_str("aabaabaabaac", "aabaac") = 4

str_str("hello", "") = 0

```

The function returns -1 when the needle is absent, following the convention of the problem it solves. In your own code, `Option<usize>` states that more clearly. For everyday searching, the standard library's `str::find` is also linear time.

Summary

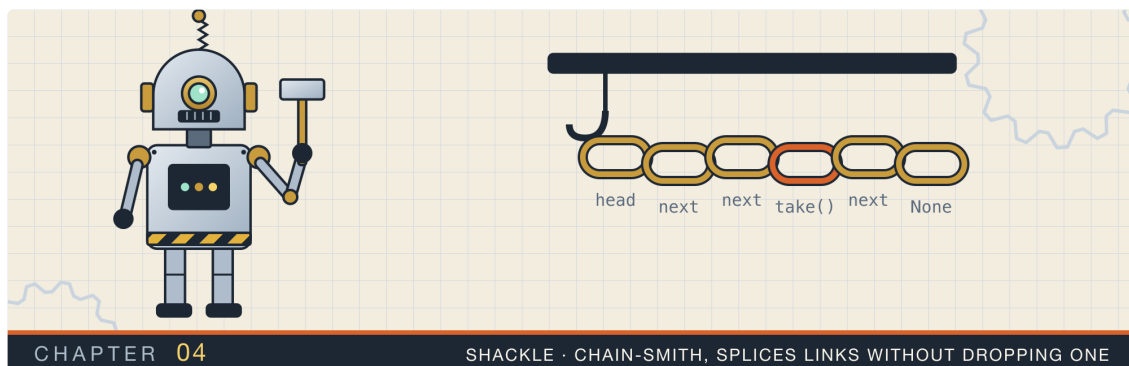
- Big-O notation describes how running time or memory grows with the input size, keeping only the fastest growing part.
- A `Vec` stores its elements next to each other on the heap, so indexing is $O(1)$. Pushing is amortized $O(1)$ because the capacity doubles when it runs out.
- A `String` is UTF-8 bytes. `len()` counts bytes, and `chars()` walks characters.
- A `HashMap` hashes each key to a bucket, so `get` and `insert` are $O(1)$ on average.
- A hash map turns "have I seen the partner?" into one step (Two Sum).
- A vector used as a stack tracks unmatched openers (brackets) and running minimums (Min Stack).
- Two indexes moving toward each other reverse a string or search a sorted array for sums in $O(n)$ per pass.
- A sliding window with a map of last positions finds the longest run without repeats in $O(n)$.
- Sorting first lets one pass merge overlapping ranges.
- Binary search halves the range each step, $O(\log n)$, and still works on a rotated sorted array.
- KMP reuses matched characters, so text search takes $O(n + m)$.

This chapter used iterators such as `iter()` and `enumerate()` without much comment. Chapter 4 looks at them closely, along with the references they hand you and the lifetimes that keep those references valid.

Exercises

1. Change `two_sum` to use `i64` arithmetic so that it cannot overflow, and add a test with `i32::MIN`.
2. Fix `is_valid` so that characters other than brackets are rejected, and add a test for `"a"`.
3. Make `reverse_str` handle an empty string, then write a version that reverses the characters of any UTF-8 string, including `"café"`.
4. Write a plain binary search over a sorted slice that returns `Option<usize>`, and test it on arrays of length 0, 1, and 2.

5. Rotate the grid counterclockwise instead, using a transpose and one other reversal.



Iterators and references

THIS CHAPTER COVERS

- What an iterator is, and what a `for` loop does with one
- The three ways to walk a collection: by reference, by mutable reference, and by value
- Chaining `filter`, `map`, and `collect`, and why nothing runs until `collect`
- Why a closure sometimes receives `&i32`, and how patterns like `|&&x|` remove the references
- Lifetimes: what they are, why the compiler needs them, and how to read `'a` in a signature
- Building a map that stores references instead of copies

Chapter 3 used iterators and references in almost every listing: `nums.iter().enumerate()`, `s.chars()`, `seen.get(&key)`. I explained each use briefly and moved on. This chapter slows down and looks at them properly, because every later chapter depends on them.

The chapter has two halves. The first half is about iterators: what they are, how a `for` loop uses them, and how to chain them. The second half is about references and lifetimes. A **life-time** is the part of a program during which a reference is guaranteed to point to valid data. You will see why Rust needs them, how to read them, and how to write a function whose signature uses one.

Both halves meet in one short program. It turns a list of users into a map from each user's ID to that user, without copying any user.

4.1 What an iterator is

An **iterator** is a value that hands out items one at a time. It has one required method, `next`, which returns `Some(item)` while there are items left and `None` when there are none.

You can think of an iterator over a slice as a cursor that starts before the first element. Each call to `next` returns the element under the cursor and moves the cursor one step (figure 4.1).

`let mut it = [10, 20, 30].iter();` each call to `it.next()` moves a cursor one element

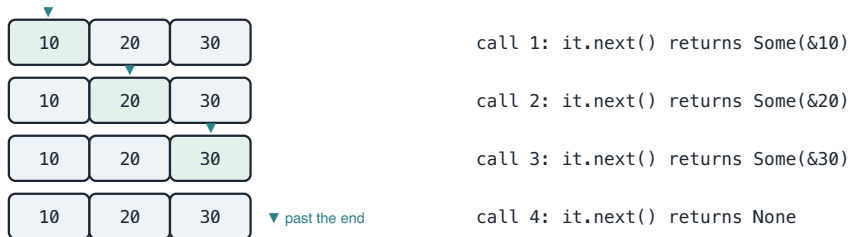


Figure 4.1 An iterator over `[10, 20, 30]`. Each call to `next` returns one item. After the last item, it returns `None`.

You rarely call `next` yourself. A `for` loop calls it for you. This loop:

```
for x in [10, 20, 30].iter() {
    println!("{x}");
}
```

does the same as this:

```
let mut it = [10, 20, 30].iter();
while let Some(x) = it.next() {
    println!("{x}");
}
```

`while let Some(x) = it.next()` keeps looping as long as `next` returns `Some`, and binds the item to `x`. Section 2.8 used this form directly, because it needed to call `next` inside the loop body.

4.2 Three ways to walk a collection

A `Vec<String>` owns its strings. When you walk it, you have to decide what you get for each element: a shared reference, a mutable reference, or the string itself. Each choice has its own method (figure 4.2).

Three ways to walk a `Vec<String>` named `names`

each item is a		
<code>names.iter()</code>	<code>&String</code>	borrows each element; <code>names</code> is unchanged and still usable
<code>names.iter_mut()</code>	<code>&mut String</code>	borrows each element mutably; you can change them in place
<code>names.into_iter()</code>	<code>String</code>	moves each element out; <code>names</code> is used up and cannot be used after

A `for` loop over `&names` calls `iter()`; over `&mut names`, `iter_mut()`; over `names`, `into_iter()`.

Figure 4.2 The three iteration methods and the item type each one yields.

- `iter()` gives `&T`, a shared reference to each element. You can read the elements, and the collection is unchanged afterward.
- `iter_mut()` gives `&mut T`. You can change each element in place.
- `into_iter()` gives `T`, the element itself. The collection is moved into the iterator, so you cannot use it afterward.

A `for` loop chooses for you based on what you write after `in`. `for x in &names` means `names.iter()`. `for x in &mut names` means `names.iter_mut()`. `for x in names` means `names.into_iter()`, and `names` is gone after the loop.

TIP: When you are not sure which one you have, look at the item type the compiler reports. A deliberate type error such as `let () = x;` inside the loop makes the compiler print the type of `x`.

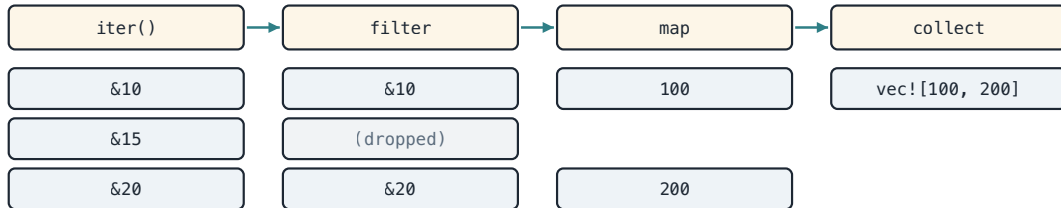
4.3 Chaining iterators

An **iterator adapter** is a method that takes one iterator and returns another. `filter` keeps only the items for which a closure returns `true`. `map` replaces each item with the closure's

result. `collect` is different: it runs the whole chain and gathers the items into a collection, such as a `Vec`.

Figure 4.3 follows three numbers through a chain.

```
slice.iter().filter(|&x| x % 2 == 0).map(|&x| x * 10).collect() on [10, 15, 20]
```



Nothing runs until `collect()` asks for items. Then each item flows through every stage before the next one starts.

Figure 4.3 Items flow through the chain one at a time. The odd number 15 stops at `filter`.

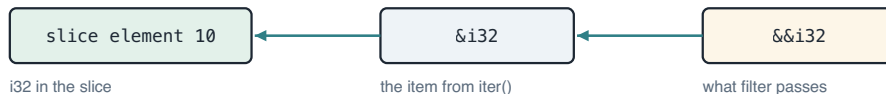
Adapters are **lazy**. `iter().filter(...).map(...)` only builds a description of the work. Nothing happens until something asks for items, as `collect` or a `for` loop does. Then each item goes through every stage before the next item starts. No temporary vector is created between stages.

WARNING: Because adapters are lazy, a chain that ends without `collect`, a `for` loop, or another method that consumes it does nothing at all. The compiler warns about such a chain: "unused Map that must be used".

4.4 References to references

Look closely at the chain in figure 4.3. `slice.iter()` yields items of type `&i32`. `filter` passes its closure a reference to each item, so that the closure can look at the item without taking it. So the closure receives `&&i32`: a reference to a reference to an `i32` (figure 4.4).

Inside filter's closure, the argument is a reference to an item, and the item is itself a reference



The pattern `|&&x|` removes both layers, so `x` is the `i32`. In `map`, `|&x|` removes the one layer.

Figure 4.4 The two layers of reference inside `filter`'s closure.

A closure's parameter can be a pattern, the same as the left side of a `let`. The pattern `&x` matches a reference and binds `x` to what it points to. So:

- `.filter(|&&x| x % 2 == 0)` removes both layers. Inside the closure, `x` is an `i32`.
- `.map(|&x| x * 10)` removes the one layer that `map` gives. `map` passes the item itself, not a reference to it.

This works for `i32` because `i32` is `Copy`: removing the reference copies the number. A `String` is not `Copy`. For it, a pattern like `|&s|` would try to move the value out of a reference, and the compiler rejects that. You would write `|s|` and use `s` as a reference instead.

4.5 The same work in two styles

The program for this chapter starts with a small `User` type and two functions that do similar work in two styles.

Listing 4.1 The `User` type and two ways to transform a slice (lines 1 to 22). `src/bin/lifetime_reference_drill.rs`

```
use std::collections::HashMap;

#[derive(Debug)]
struct User {
    id: u32,
    name: String,
}

fn functional_numbers_drill(slice: &[i32]) -> Vec<i32> {
    slice.iter()
        .filter(|&&x| x % 2 == 0)
        .map(|&x| x * 10)
        .collect()
}

fn imperative_numbers_map(slice: &[i32]) -> HashMap<i32, i32> {
```

```

    let mut map = HashMap::new();
    for &num in slice {
        map.insert(num, num * 100);
    }
    map
}

```

`functional_numbers_drill` is the chain from figure 4.3. `collect()` knows to build a `Vec<i32>` because that is the function's return type. When the type is not clear from context, you write it, as in `collect::<Vec<i32>>()`.

`imperative_numbers_map` uses a `for` loop and builds a `HashMap` from each number to that number times 100. `for &num in slice` removes the one reference that iterating a slice gives, so `num` is an `i32`.

Neither style is faster. Both compile to similar machine code. Iterator chains read well when the work is a sequence of steps. Loops read well when the body does several unrelated things.

With iterators covered, the rest of the chapter turns to the references they hand out, and to the rules that keep those references safe.

4.6 Lifetimes

A reference points to data owned by something else. If the owner is dropped while the reference is still in use, the reference points to freed memory. Reading through it would give garbage or crash the program. Figure 4.5 shows the smallest example.

Why a reference must not outlive its data

```

let r;
{
    let s = String::from("hi");
    r = &s;
}
// s is dropped here; its heap text is freed
println!("{}", r); // r would point to freed memory

```

error[E0597]: `s` does not live long enough
 borrowed value does not live long enough

The compiler compares how long `r` is used with how long `s` lives, and rejects the program before it runs.

Figure 4.5 A reference that would outlive its data. The compiler rejects this program.

To prevent this, the compiler tracks, for every reference, the part of the program where it is used. It also knows where each value is dropped. If a reference is used after its value is

dropped, the program does not compile. The part of the program where a reference is valid is its lifetime.

Inside one function, the compiler works out all lifetimes on its own. Across function calls it needs help. Consider a function that takes a reference and returns one:

```
fn first_word(s: &str) -> &str { ... }
```

The caller needs to know how long the returned reference is valid. Here the only possible answer is "as long as `s`", and the compiler assumes that without being told. These assumptions are called **lifetime elision rules**. For a function with one reference parameter, the returned reference gets that parameter's lifetime.

When the rules cannot decide, or when you want to state a relationship yourself, you name lifetimes. A lifetime name is an apostrophe and a short name, usually `'a`. You declare it in angle brackets after the function name, like a generic type, and then attach it to references:

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str { ... }
```

Read this as: "for some lifetime `'a`, both inputs are valid for `'a`, and so is the output". The compiler then checks every caller. The returned reference may not be used after either input's data is dropped.

A lifetime annotation does not change how long anything lives. It describes a relationship, and the compiler checks that the code keeps it.

4.7 A map that points into a slice

Now you have what you need to read the main functions of the program. The goal is to index a list of users by ID. You could copy each user into a `HashMap<u32, User>`. Or you could store only a reference to each user, in a `HashMap<u32, &User>`, and leave the users where they are. Figure 4.6 compares the two.

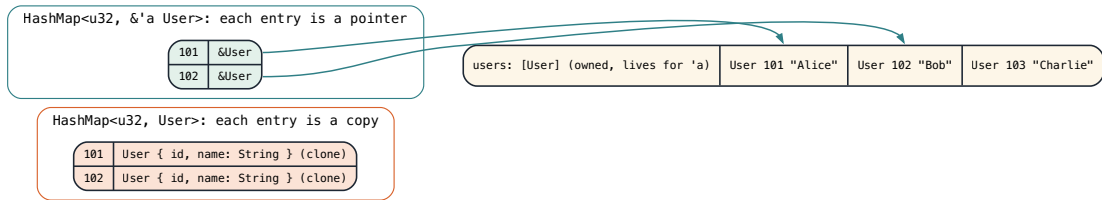


Figure 4.6 A map of copies duplicates every user and its name. A map of references stores one pointer per entry, and depends on the original array staying alive.

The map of references is cheaper. It also depends on the users staying alive while the map is used. The function's signature states that dependency with a lifetime.

Listing 4.2 A map whose values point into the slice (lines 24 to 30).

```
fn struct_slice_to_map<'a>(slice: &'a [User]) -> HashMap<u32, &'a User> {
    let mut map = HashMap::new();
    for user in slice {
        map.insert(user.id, user);
    }
    map
}
```

Read the signature the way section 4.6 showed. For some lifetime `'a`, the input is a slice of users valid for `'a`. The output is a map whose values are references valid for `'a`. Because both mention the same `'a`, the map may not be used after the users are dropped. If you tried, the compiler would reject the program with the same error as figure 4.5.

Inside the function, `for user in slice` walks the slice by reference, so `user` is a `&User`. The map stores that reference. No `User` is cloned.

4.8 Two lifetimes in one type

The second function takes a different input. Instead of a slice of users, it takes a slice of references to users.

Listing 4.3 A map built from a slice of references (lines 32 to 38).

```
fn ref_slice_to_map<'a>(slice: &[&'a User]) -> HashMap<u32, &'a User> {
    let mut map = HashMap::new();
    for &user in slice {
        map.insert(user.id, user);
    }
}
```

```

    }
    map
}

```

The type `&[&'a User]` has two references in it:

- The outer `&` is the slice itself, a short list of pointers. Its lifetime has no name here.
- The inner `&'a User` is each pointer in the list. Its lifetime `'a` belongs to the users those pointers point to.

The returned map mentions only `'a`. So the map depends on the users, not on the list of pointers. The list can be dropped right after the call, and the map stays valid (figure 4.7).

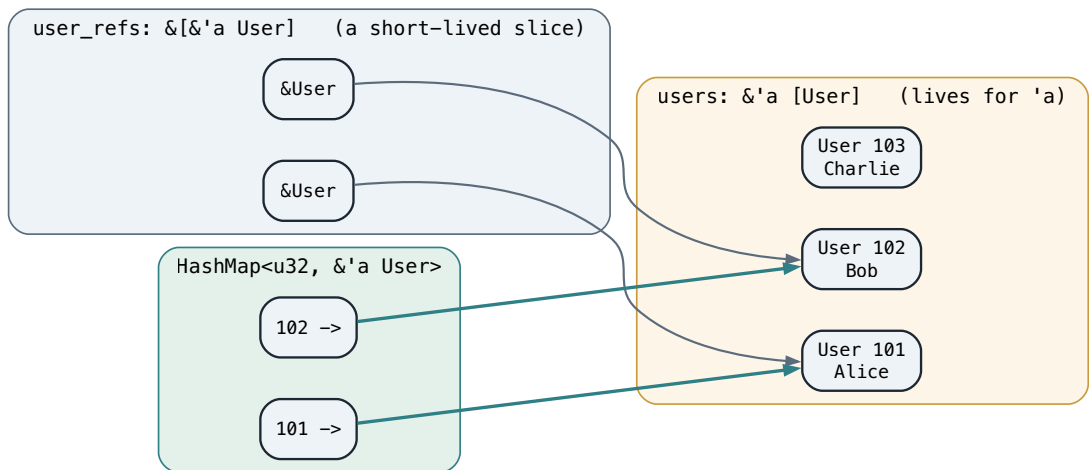


Figure 4.7 The map copies the pointers out of the short-lived slice. It points at the users directly, so only the users must stay alive.

`for &user in slice` copies each pointer out of the slice. A reference is a `Copy` value, one machine word, so this copy is cheap.

4.9 The complete program

Listing 4.4 The complete program. `src/bin/lifetime_reference_drill.rs`

```

use std::collections::HashMap;

```

```

#[derive(Debug)]
struct User {
    id: u32,
    name: String,
}

fn functional_numbers_drill(slice: &[i32]) -> Vec<i32> {
    slice.iter()
        .filter(|&x| x % 2 == 0)
        .map(|&x| x * 10)
        .collect()
}

fn imperative_numbers_map(slice: &[i32]) -> HashMap<i32, i32> {
    let mut map = HashMap::new();
    for &num in slice {
        map.insert(num, num * 100);
    }
    map
}

fn struct_slice_to_map<'a>(slice: &'a [User]) -> HashMap<u32, &'a User> {
    let mut map = HashMap::new();
    for user in slice {
        map.insert(user.id, user);
    }
    map
}

fn ref_slice_to_map<'a>(slice: &[&'a User]) -> HashMap<u32, &'a User> {
    let mut map = HashMap::new();
    for &user in slice {
        map.insert(user.id, user);
    }
    map
}

fn main() {
    let vv = vec![1,2,3,4,5,6];

    let filtered: Vec<i32> = vv.iter().filter(|x| *x % 2 == 0).copied().collect();
    println!("Filtered : {:?}", filtered);

    println!("--- Ultra-Lean Inline Slice Drill ---\n");

    let processed_nums = functional_numbers_drill(&[10, 15, 20, 25, 30]);
    println!("1. Functional Numbers (Even * 10): {:?}", processed_nums);

    let num_map = imperative_numbers_map(&[10, 15, 20, 25, 30]);
    println!("2. Imperative Numbers Map: {:?}", num_map);

    // explicit reference binding holding the inline array literal
    let users: &[User] = &[
        User { id: 101, name: "Alice".to_string() },
        User { id: 102, name: "Bob".to_string() },
        User { id: 103, name: "Charlie".to_string() },
    ];
    println!("\n3. Zero-Allocation Struct Map (From &[User]):");
    let struct_map = struct_slice_to_map(users);

```



```

    for (id, user_ref) in &struct_map {
        println!("    Key ID: {} -> Value Name: {}", id, user_ref.name);
    }

    // explicit reference binding holding the inline array of &User
    let user_refs: &[&User] = &[
        struct_map[&101],
        struct_map[&102],
    ];
    println!("\n4. Zero-Allocation Ref Slice Map (From &[&User]):");
    let ref_slice_map = ref_slice_to_map(user_refs);
    for (id, user_ref) in &ref_slice_map {
        println!("    Key ID: {} -> Value Name: {}", id, user_ref.name);
    }

    println!("\n--- Drill Completed Successfully! ---");
}

```

`main` starts with one more iterator chain. `vv.iter()` yields `&i32`, and `filter` receives `&&i32`. This closure writes `|x|` and then `*x % 2`. The `*` removes one layer of reference, and the `%` operator works through the other. `.copied()` turns each `&i32` into an `i32`, so `collect` builds a `Vec<i32>`.

Two lines in `main` use a rule you have not seen yet:

```
let users: &[User] = &[ User { ... }, User { ... }, User { ... } ];
```

`&[ ... ]` takes a reference to an array that was never given a name of its own. Such a value is a **temporary**, and a temporary is normally dropped at the end of the statement. That would leave `users` pointing to freed memory. Rust extends the temporary's life to the end of the enclosing block when it is the direct value of a `let`. This is called **temporary lifetime extension**. It is why `users` and `user_refs` stay valid for the rest of `main`.

Run the program:

```

$ cargo run --bin lifetime_reference_drill
Filtered : [2, 4, 6]
--- Ultra-Lean Inline Slice Drill ---

1. Functional Numbers (Even * 10): [100, 200, 300]
2. Imperative Numbers Map: {20: 2000, 10: 1000, 25: 2500, 15: 1500, 30: 3000}

3. Zero-Allocation Struct Map (From &[User]):
   Key ID: 101 -> Value Name: Alice
   Key ID: 102 -> Value Name: Bob
   Key ID: 103 -> Value Name: Charlie

4. Zero-Allocation Ref Slice Map (From &[&User]):
   Key ID: 102 -> Value Name: Bob
   Key ID: 101 -> Value Name: Alice

```

```
--- Drill Completed Successfully! ---
```

Run it again and the map lines may come out in a different order. As section 3.4 explained, `HashMap` uses a randomly seeded hash, so its iteration order is not fixed. When you need a stable order, use a `BTreeMap`, which keeps its keys sorted, or sort the keys before printing.

Summary

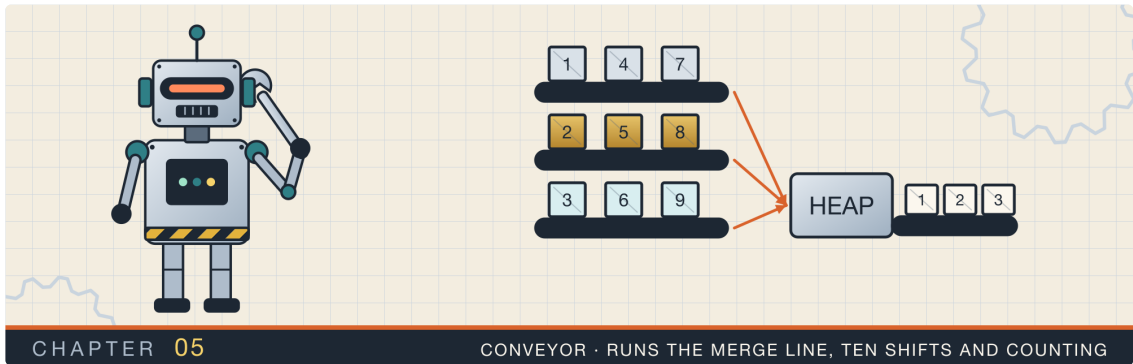
- An iterator hands out items through `next`, which returns `Some(item)` or `None`. A `for` loop calls `next` until it gets `None`.
- `iter()` yields `&T`, `iter_mut()` yields `&mut T`, and `into_iter()` yields `T` and uses up the collection.
- Adapters such as `filter` and `map` are lazy. `collect` or a loop drives them, one item at a time.
- `filter` passes a reference to each item, so over `iter()` its closure receives `&&T`. Patterns like `|&&x|` remove the references for `Copy` types.
- A lifetime is the part of the program where a reference is valid. The compiler rejects a reference used after its data is dropped.
- `'a` in a signature names a lifetime and ties inputs to outputs. It describes a relationship; it does not change how long anything lives.
- A `HashMap<u32, &'a User>` indexes data without copying it, and cannot outlive that data.

Chapter 5 uses iterators and collections to solve problems with heaps and sorted collections, such as finding the most frequent words in a text.

Exercises

1. Rewrite `imperative_numbers_map` as an iterator chain that ends in `collect::<HashMap<_, _>>()`.
2. Write `fn longest<'a>(x: &'a str, y: &'a str) -> &'a str`. Then write a `main` where calling it and using the result fails to compile, and read the error.
3. Change `struct_slice_to_map` to return a `BTreeMap`, and check that the output order is now stable.

4. Write a function that takes `&mut [User]` and uses `iter_mut()` to append " (admin)" to every name.



Heaps

THIS CHAPTER COVERS

- What a heap is, and how it stores a tree inside a `Vec`
- How `push` and `pop` keep the largest item on top in $O(\log n)$ steps
- Rust's `BinaryHeap`, and turning it into a min-heap with `Reverse`
- How tuples compare, and why `(count, value)` orders items by count
- Finding the k most frequent items, first with a full heap and then with a heap of size k
- Keeping the median of a stream of numbers with two heaps

Chapter 3 used vectors, strings, and hash maps. This chapter adds one more collection, the heap. A heap answers one question quickly: "what is the largest item right now?" It keeps answering it while items are added and removed.

That question comes up whenever you want the best few items from many. Two examples are the most frequent words in a text and the next job to run from a queue ordered by priority. You will build three programs. Two find the most frequent items. The third keeps the median of a stream of numbers.

As in chapter 3, I start with how the structure works, with pictures, and then show the code.

A few terms used throughout the chapter:

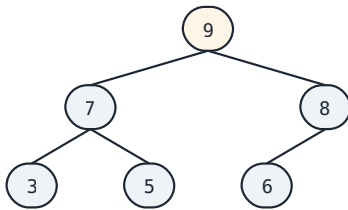
- A **binary tree** is a set of nodes where each node has at most two children, a left one and a right one. The node with no parent is the **root**.

- A **max-heap** is a binary tree where every parent is at least as large as each of its children. So the root holds the largest item.
- A **min-heap** is the same with "at least as large" replaced by "at most as large". The root holds the smallest item.
- The **median** of a list of numbers is the middle value once they are sorted. For an even count, it is the average of the two middle values.

5.1 A tree stored in a vector

A heap is a tree, but it does not store pointers between nodes. It stores the nodes in a **Vec**, level by level: the root first, then its two children, then their four children, and so on. Figure 5.1 shows a max-heap of six numbers and the vector that holds it.

A max-heap: every parent is at least as large as its children



stored in a Vec, level by level:



children of index i : $2i + 1$ and $2i + 2$

parent of index i : $(i - 1) / 2$

No pointers are stored; the positions come from arithmetic on the index.

Figure 5.1 A max-heap and its vector. The children of the node at index i are at $2i + 1$ and $2i + 2$.

Because the nodes are packed level by level, simple arithmetic finds a node's relatives. The children of index i are at $2i + 1$ and $2i + 2$. Its parent is at $(i - 1) / 2$, rounded down. In figure 5.1, the children of 7 (index 1) are at indexes 3 and 4, which hold 3 and 5.

The heap rule only compares parents with children. It says nothing about brothers or cousins. In figure 5.1, 7 is on the left and 8 on the right, and that is allowed. That weak rule keeps a heap cheap to maintain.

A tree with n nodes packed this way has about $\log_2 n$ levels. A heap of a million items is only about 20 levels deep.

5.2 Adding and removing

5.2.1 Push

To add an item, put it at the end of the vector, which is the next free spot on the bottom level. It may be larger than its parent, which breaks the rule. So swap it with its parent, and keep swapping upward until it is not larger than its parent, or it reaches the root. Figure 5.2 pushes 10.

push(10): add at the end, then swap upward while larger than the parent

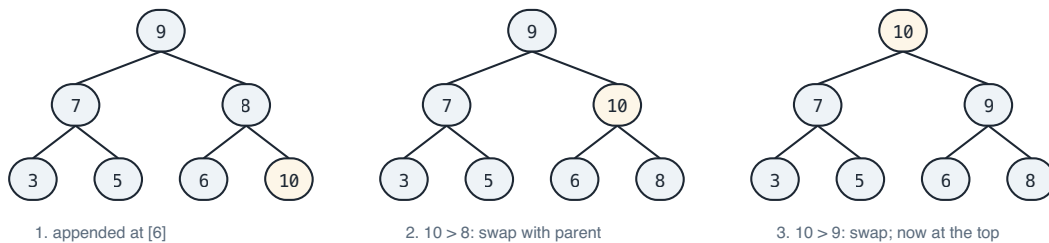


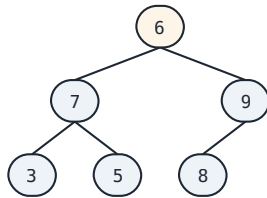
Figure 5.2 Pushing 10. It moves up one level per swap until the rule holds again.

Each swap moves the item one level up, and there are about $\log_2 n$ levels. So a push takes $O(\log n)$ steps.

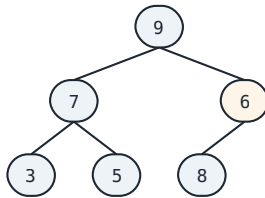
5.2.2 Pop

To remove the largest item, take the root. That leaves a hole at the top. Fill it with the last item in the vector, which shrinks the vector by one. The moved item is probably too small for the top. So swap it with its larger child, and keep swapping downward until both its children are smaller. Figure 5.3 pops the 10 that figure 5.2 added.

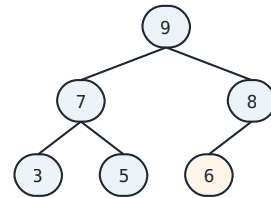
pop(): take the top, move the last element to the top, swap downward with the larger child



1. 10 removed; last element 6 moved up



2. larger child is 9: swap



3. larger child is 8: swap; done

Figure 5.3 Popping the largest item. The item moved to the top sinks one level per swap.

It swaps with the larger child for a reason. That child is about to become the parent of the other one, and a parent must be at least as large as its children. Pop is also $O(\log n)$.

Looking at the largest item without removing it is $O(1)$: it is always at index 0.

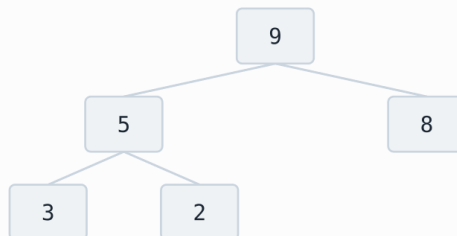
Heap push and pop: a new value climbs, the last value sinks

A max-heap in an array. The children of i are $2i+1$ and $2i+2$; the parent is $(i-1)/2$.

 on the move in place



the same values as a tree



A valid max-heap: index 0 holds 9, and every parent is at least as large as its children.

The tree is the array: no links and no allocation, and the parent of i is one division away.

phase = **climb**

index 0 = **9**

size = **5**

Animation 5.4 Both fixes on the same heap, one swap per frame. The upper row is the array and the lower row is the same values drawn as the tree `BinaryHeap` keeps inside that array. Watch the index arithmetic in the caption: the parent of index i is $(i - 1) / 2$, and the children are $2i + 1$ and $2i + 2$. A push walks up, a pop walks down, and each walk is one level per swap, which is why both are $O(\log n)$.

5.3 Heaps in Rust

Rust's standard library provides `std::collections::BinaryHeap<T>`, a max-heap. Its main methods match the operations above:

Method	What it does	Cost
<code>push(x)</code>	adds <code>x</code>	$O(\log n)$
<code>pop()</code>	removes and returns the largest item, as an <code>Option</code>	$O(\log n)$
<code>peek()</code>	returns a reference to the largest item, as an <code>Option</code>	$O(1)$
<code>len()</code>	the number of items	$O(1)$
<code>into_sorted_vec()</code>	consumes the heap and returns its items in ascending order	$O(n \log n)$

To decide which item is larger, `BinaryHeap` uses the `Ord` trait, which types such as `i32`, `String`, and tuples implement.

5.3.1 How tuples compare

Tuples compare element by element, from the left. Two tuples are compared on their first elements, and only if those are equal does the second element decide. So `(3, 1) > (2, 9)`, because `3 > 2`, and the 9 is never looked at. And `(3, 1) < (3, 2)`, because the first elements tie and `1 < 2`.

This gives a way to choose what a heap orders by: put that field first in a tuple. A heap of `(count, value)` pairs orders by count, and breaks ties by value.

5.3.2 A min-heap with `Reverse`

`BinaryHeap` has no min-heap mode. Instead, the standard library provides a wrapper, `std::cmp::Reverse<T>`, whose comparison is the opposite of `T`'s. `Reverse(3) > Reverse(5)`, because `3 < 5`. So a `BinaryHeap<Reverse<i32>>` keeps the smallest number on top.

To put a value in, wrap it: `heap.push(Reverse(x))`. To get it out, unwrap it with a pattern: `if let Some(Reverse(x)) = heap.pop()`.

Now you can read the three programs.

5.4 The k most frequent numbers

The problem. Given a list of numbers and a count k , return the k numbers that occur most often. For `[1, 1, 1, 2, 2, 3]` and $k = 2$, the answer is 1 and 2.

The idea. Count each number with a `HashMap`. Put every `(count, number)` pair into a max-heap. Pop `k` times.

Listing 5.1 Top `k` with a full heap. `src/problems/top_k_frequent.rs`

```

///! Top K Frequent Elements.
///!
///! Count occurrences with a HashMap, then use a max-heap (`BinaryHeap`) to pull
///! out the top k elements.  $O(n \log n)$  worst case; acceptable and readable.

use std::collections::{BinaryHeap, HashMap};

pub fn top_k_frequent(nums: &[i32], k: usize) -> Vec<i32> {
    let mut counts: HashMap<i32, usize> = HashMap::with_capacity(nums.len());
    for &num in nums.iter() {
        *counts.entry(num).or_insert(0) += 1;
    }

    // BinaryHeap is a max-heap in Rust, so (count, num) keeps highest counts first.
    let mut heap: BinaryHeap<(usize, i32)> = counts
        .into_iter()
        .map(|(num, count)| (count, num))
        .collect();

    let mut result = Vec::with_capacity(k);
    while result.len() < k {
        match heap.pop() {
            Some( (_, num)) => result.push(num),
            None => break,
        }
    }
    result
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn returns_top_two() {
        let nums = [1, 1, 1, 2, 2, 3];
        let mut result = top_k_frequent(&nums, 2);
        result.sort_unstable();
        assert_eq!(result, vec![1, 2]);
    }

    #[test]
    fn k_larger_than_unique_values_returns_all() {
        let result = top_k_frequent(&[1, 2, 3], 10);
        assert_eq!(result.len(), 3);
    }
}

```

Step through it:

1. `*counts.entry(num).or_insert(0) += 1`; counts one occurrence. `entry(num)` finds the map slot for `num`, and `or_insert(0)` creates it with 0 if it is missing. Both steps return a mutable reference to the count, and `*` followed by `+= 1` adds one to the value it points to.
2. `counts.into_iter().map(|(num, count)| (count, num)).collect()` turns each map entry around, so the count comes first, and collects the pairs straight into a `BinaryHeap`. A `BinaryHeap` can be built with `collect` like a `Vec`.
3. The `while` loop pops until it has `k` numbers. If there are fewer than `k` distinct numbers, `pop` returns `None` and the loop stops early. The second test checks that case.

Counting takes $O(n)$. Building the heap from u distinct numbers takes $O(u)$, and k pops take $O(k \log u)$.

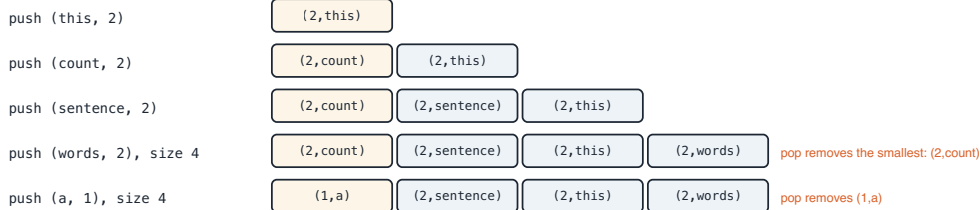
The first test sorts the result before comparing it. The order of numbers with equal counts depends on the map's iteration order. That order changes between runs, so a test must not depend on it.

5.5 The k most frequent words, with a heap of size k

The first version puts every distinct number in the heap. If the input has millions of distinct words and you only want the top 3, most of that heap is wasted. The second version never lets the heap grow past $k + 1$ entries.

The idea. Use a min-heap. Push each `(count, word)` pair. Whenever the heap has more than k entries, pop once. A min-heap pops its smallest entry, which is the least frequent word seen so far, so the k most frequent words survive (figure 5.4).

Keeping the 3 most frequent words with a min-heap of size 3



Contents listed smallest first. The smallest entry (cream) is at the top of the heap, so pop removes it. The heap never holds more than $k + 1$ entries. Words arrive in whatever order the `HashMap` yields them.

Figure 5.4 A min-heap bounded to $k = 3$ entries. Each time a fourth entry arrives, the smallest is removed.

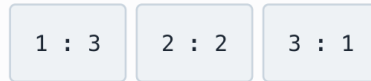
The program first turns the text into words, then counts them, then runs the bounded heap.

The k most frequent values: a heap of size k, never bigger

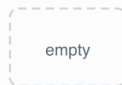
The heap stores k entries, so its cost does not grow with the input.

☒ kept ☐ just arrived ☐ ignored

counts



the best k, as a min-heap



Count each number first, then keep a min-heap of size k over the counts, so the root is the weakest entry still kept.

The root of a min-heap is the weakest entry kept, so a new count is compared with it alone.

arriving = — kept = 0 k = 2

Animation 5.5 Counting first, then a min-heap of size k, on the sample [1, 1, 1, 2, 2, 3] with k = 2. The root is always the weakest entry kept, so one comparison decides whether an arriving count is worth keeping. The heap never grows past k, which is what bounds the work to $O(u \log k)$ rather than sorting all u words.

Listing 5.2 The function (lines 1 to 36).src/bin/top_k_frequent_words.rs

```
use std::cmp::Reverse as Rev;
use std::collections::{BinaryHeap, HashMap};

/// Returns vec!(word, frequency)>, most frequent first.
fn top_k_frequent(sentence: &str, k: usize) -> Vec<(String, u32)> {

    // Split string by white spaces. Map it and trim any non alphanumeric characters.
    Finally,
    // filter all the empty strings and map everything to lowercase characters.
    let words: Vec<String> = sentence.split_whitespace()
        .map(|w| w.trim_matches(|c: char| !c.is_alphanumeric()))
```

```

        .filter(|w| !w.is_empty())
        .map(str::to_lowercase)
        .collect();

    // Build the frequency map...
    let mut frequencies = HashMap::<&str, u32>::new();
    for word in &words {
        *frequencies.entry(word.as_str()).or_default() += 1;
    }

    // Keep a min heap of size k. Reverse flips the max heap, so the least
    // frequent word sits on top and gets popped once we go over k.
    let mut min_heap = BinaryHeap::<Rev<(u32, &str)>>::new();
    for (&word, &frequency) in &frequencies {
        min_heap.push(Rev((frequency, word)));
        if min_heap.len() > k {
            min_heap.pop();
        }
    }

    // Ascending order of Reverse means most frequent first.
    min_heap.into_sorted_vec()
        .into_iter()
        .map(|Rev((frequency, word))| (word.to_string(), frequency))
        .collect()
}

```

Turning text into words. The chain splits on whitespace. Then it trims characters that are not letters or digits from both ends of each piece, so "sentence." becomes "sentence". It drops empty pieces, which a lone punctuation mark leaves behind, and lowercases each word. `trim_matches` takes a closure that decides which characters to trim. `map(str::to_lowercase)` passes a function by name instead of writing a closure.

Counting without copying. The map is `HashMap<&str, u32>`. Its keys are references into the `words` vector, so counting allocates nothing for each occurrence. `or_default()` inserts the type's default value, 0 for `u32`, when the key is missing.

The bounded heap. use `std::cmp::Reverse` as `Rev`; gives `Reverse` a shorter name. Each entry is `Rev((frequency, word))`, so the heap's top is the smallest `(frequency, word)` pair. After each push, if the heap has more than `k` entries, one `pop` removes that smallest pair.

The result. `into_sorted_vec()` returns the entries in ascending order of `Rev`, which is descending order of frequency. The final `map` unwraps each `Rev` with a pattern and copies the word into an owned `String`. The result must not borrow from `words`, because `words` is dropped when the function returns.

The heap holds at most $k + 1$ entries, so each push and pop costs $O(\log k)$. For u distinct words, the heap work is $O(u \log k)$ instead of $O(u \log u)$, and its memory is $O(k)$ instead of $O(u)$.

5.5.1 The complete program

Listing 5.3 The complete program, with its tests. `src/bin/top_k_frequent_words.rs`

```
use std::cmp::Reverse as Rev;
use std::collections::{BinaryHeap, HashMap};

// Returns vec!<(word, frequency)>, most frequent first.
fn top_k_frequent(sentence: &str, k: usize) -> Vec<(String, u32)> {
    // Split string by white spaces. Map it and trim any non alphanumeric characters.
    Finally,
    // filter all the empty strings and map everything to lowercase characters.
    let words: Vec<String> = sentence.split_whitespace()
        .map(|w| w.trim_matches(|c: char| !c.is_alphanumeric()))
        .filter(|w| !w.is_empty())
        .map(str::to_lowercase)
        .collect();

    // Build the frequency map...
    let mut frequencies = HashMap::<&str, u32>::new();
    for word in &words {
        *frequencies.entry(word.as_str()).or_default() += 1;
    }

    // Keep a min heap of size k. Reverse flips the max heap, so the least
    // frequent word sits on top and gets popped once we go over k.
    let mut min_heap = BinaryHeap::<Rev<(u32, &str)>>::new();
    for (&word, &frequency) in &frequencies {
        min_heap.push(Rev((frequency, word)));
        if min_heap.len() > k {
            min_heap.pop();
        }
    }

    // Ascending order of Reverse means most frequent first.
    min_heap.into_sorted_vec()
        .into_iter()
        .map(|Rev((frequency, word))| (word.to_string(), frequency))
        .collect()
}

fn main() {
    let sentence = r"This is a short sentence. We'll count words.
    Let us count words in this sentence
    ";

    let top_k = top_k_frequent(sentence, 3);
    println!("Top 3 frequency [(word, frequency)] : \n {:#?}", top_k);
}
```

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn picks_most_frequent() {
        let result = top_k_frequent("a a a b b c", 2);
        assert_eq!(result, vec!["a".to_string(), 3], ("b".to_string(), 2));
    }

    #[test]
    fn ignores_case_and_punctuation() {
        let result = top_k_frequent("Hello, hello! HELLO world.", 1);
        assert_eq!(result, vec!["hello".to_string(), 3]);
    }

    #[test]
    fn k_larger_than_word_count() {
        assert_eq!(top_k_frequent("x y", 10).len(), 2);
    }

    #[test]
    fn empty_input() {
        assert!(top_k_frequent("", 3).is_empty());
    }
}
```

The sentence in `main` is a **raw string literal**, written `r"..."`. A raw string treats backslashes as ordinary characters, and this one also spans several lines. The tests cover the basic case, uppercase and punctuation, a `k` larger than the number of words, and an empty input.

```
$ cargo run --bin top_k_frequent_words
Top 3 frequency [(word, frequency)] :
[
  (
    "words",
    2,
  ),
  (
    "this",
    2,
  ),
  (
    "sentence",
    2,
  ),
]
```

Four words occur twice in the sentence: "this", "sentence", "count", and "words". Only three fit, and "count" was evicted. When two entries have the same count, the tuple comparison falls to the word, and the min-heap evicts the alphabetically smallest one. So among equal counts, this version keeps the words that come last in alphabetical order.

TIP: To keep the alphabetically first words instead, push `Rev((frequency, Reverse(word)))`. The inner `Reverse` flips the order of words only. Among equal counts, the alphabetically last word then has the smallest key and is evicted first.

5.6 The median of a stream

The problem. Numbers arrive one at a time. After each one, report the median of all the numbers so far.

The simple idea. Keep every number in a sorted `Vec`, and read the middle. Inserting into the middle of a sorted vector shifts the elements after it, which costs $O(n)$ per number.

The idea with heaps. Split the numbers into two halves (figure 5.5):

- `low`, a max-heap holding the smaller half. Its top is the largest of the small numbers.
- `high`, a min-heap holding the larger half. Its top is the smallest of the large numbers.

Keep two rules. Every number in `low` is at most every number in `high`. And `low` has either the same number of items as `high`, or one more. Then the median is always at the tops. With an odd count, it is the top of `low`. With an even count, it is the average of the two tops.

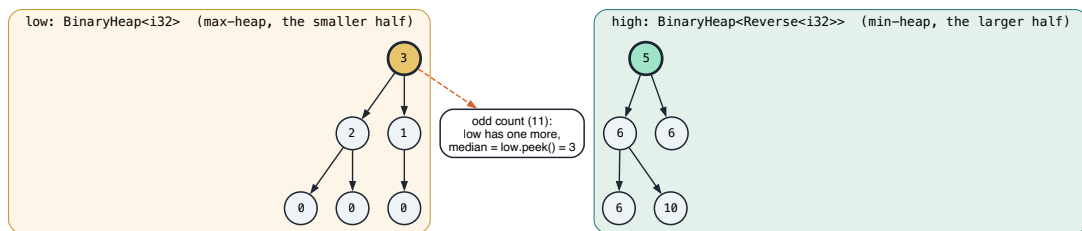


Figure 5.5 The two heaps after all eleven sample values. `low` has six values and `high` five, so the median is 3, the top of `low`.

5.6.1 Adding a number

Listing 5.4 The structure and `add_num` (lines 1 to 27). `src/bin/median_finder.rs`

```
use std::cmp::Reverse;
use std::collections::BinaryHeap;
```



```
#[derive(Default)]
pub struct MedianFinder {
    low: BinaryHeap<i32>,
    high: BinaryHeap<Reverse<i32>>,
}

pub fn new() -> Self {
    Self::default()
}

pub fn add_num(&mut self, num: i32) {
    self.low.push(num);

    if let Some(val) = self.low.pop() {
        self.high.push(Reverse(val));
    }

    if self.high.len() > self.low.len() {
        if let Some(Reverse(val)) = self.high.pop() {
            self.low.push(val);
        }
    }
}
```

`add_num` keeps both rules with three moves, and never compares numbers itself:

1. Push the new number into `low`.
2. Pop the largest number from `low` and push it into `high`. This keeps the first rule: whatever `high` receives is at least as large as everything left in `low`.
3. If `high` now has more items than `low`, pop its smallest and push it back into `low`. This keeps the second rule.

Trace the first three numbers of the sample, 6, 10, and 2:

Add	After step 1 (Low)	After step 2 (Low / high)	After step 3 (Low / high)	Median
6	{6}	{}/	{6} / {}	6
10	{6, 10}	{6} /	{6} /	$(6 + 10) / 2 = 8$
2	{2, 6}	{2} /	{2, 6} /	6

Each move is one heap operation, so adding a number is $O(\log n)$.

5.6.2 Reading the median

Listing 5.5 `find_median` (lines 29 to 38).

```
pub fn find_median(&self) -> Option<f64> {
    let max_low = *self.low.peek()?;

    if self.low.len() > self.high.len() {
        Some(max_low as f64)
    } else {
        let Reverse(min_high) = *self.high.peek()?;
        Some((max_low as f64 + min_high as f64) / 2.0)
    }
}
```

`*self.low.peek()?` reads the top of `low`. The `?` works on `Option` the same way it works on `Result`. If no number has been added, `peek` returns `None`, and so does the function. The function returns `Option<f64>`, a floating-point number, because the average of two integers can be a half.

`let Reverse(min_high) = *self.high.peek()?;` unwraps the `Reverse` in the `let` pattern itself.

5.6.3 The complete program

Listing 5.6 The complete program, with its tests. `src/bin/median_finder.rs`

```
use std::cmp::Reverse;
use std::collections::BinaryHeap;

#[derive(Default)]
pub struct MedianFinder {
    low: BinaryHeap<i32>,
    high: BinaryHeap<Reverse<i32>>,
}

impl MedianFinder {
    pub fn new() -> Self {
        Self::default()
    }

    pub fn add_num(&mut self, num: i32) {
        self.low.push(num);

        if let Some(val) = self.low.pop() {
            self.high.push(Reverse(val));
        }

        if self.high.len() > self.low.len() {
            if let Some(Reverse(val)) = self.high.pop() {

```

```

        self.low.push(val);
    }
}

pub fn find_median(&self) -> Option<f64> {
    let max_low = *self.low.peek()?;

    if self.low.len() > self.high.len() {
        Some(max_low as f64)
    } else {
        let Reverse(min_high) = *self.high.peek()?;
        Some((max_low as f64 + min_high as f64) / 2.0)
    }
}

const SAMPLE: [i32; 11] = [6, 10, 2, 6, 5, 0, 6, 3, 1, 0, 0];

fn main() {
    let mut finder = MedianFinder::new();

    for value in SAMPLE {
        finder.add_num(value);
        match finder.find_median() {
            Some(median) => println!("after {value:>2}: median {median}"),
            None => println!("after {value:>2}: no median"),
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    /// The median of a sorted slice, computed directly.
    fn median_of_sorted(values: &[i32]) -> f64 {
        let middle = values.len() / 2;
        if values.len() % 2 == 1 {
            values[middle] as f64
        } else {
            (values[middle - 1] as f64 + values[middle] as f64) / 2.0
        }
    }

    #[test]
    fn an_empty_finder_has_no_median() {
        assert_eq!(MedianFinder::new().find_median(), None);
    }

    #[test]
    fn one_value_is_the_median() {
        let mut finder = MedianFinder::new();
        finder.add_num(7);
        assert_eq!(finder.find_median(), Some(7.0));
    }

    #[test]
    fn an_even_count_averages_the_middle_two() {

```

```

        let mut finder = MedianFinder::new();
        finder.add_num(1);
        finder.add_num(2);
        assert_eq!(finder.find_median(), Some(1.5));
    }

    #[test]
    fn three_values_are_ordered() {
        let mut finder = MedianFinder::new();
        for (value, median) in [1, 2, 3].into_iter().zip([1.0, 1.5, 2.0]) {
            finder.add_num(value);
            assert_eq!(finder.find_median(), Some(median));
        }
    }

    #[test]
    fn duplicates_are_kept() {
        let mut finder = MedianFinder::new();
        for value in [4, 4, 4, 4] {
            finder.add_num(value);
        }
        assert_eq!(finder.find_median(), Some(4.0));
    }

    #[test]
    fn the_stream_matches_a_sort_on_every_step() {
        let mut finder = MedianFinder::new();
        let mut seen = Vec::new();

        for value in SAMPLE {
            finder.add_num(value);
            seen.push(value);
            seen.sort_unstable();

            assert_eq!(finder.find_median(), Some(median_of_sorted(&seen)));
        }
    }

    #[test]
    fn the_two_heaps_never_differ_by_more_than_one() {
        let mut finder = MedianFinder::new();

        for value in SAMPLE {
            finder.add_num(value);
            assert!(finder.low.len().abs_diff(finder.high.len()) <= 1);
        }
    }
}

```

`main` adds the eleven sample values and prints the median after each one.

```

$ cargo run --bin median_finder
after 6: median 6
after 10: median 8
after 2: median 6
after 6: median 6
after 5: median 6

```

```

after 0: median 5.5
after 6: median 6
after 3: median 5.5
after 1: median 5
after 0: median 4
after 0: median 3

```

Look at the test `the_stream_matches_a_sort_on_every_step`. After every added number, it sorts a copy of all the numbers so far. Then it computes the median the slow way, with the helper `median_of_sorted`. Then it checks that the heaps give the same answer. A test like this, which compares a fast method with a slow method you can trust, is called an **oracle test**. It catches mistakes that a few hand-picked examples miss.

The last test checks the second rule directly: after every number, the two heaps' sizes differ by at most one.

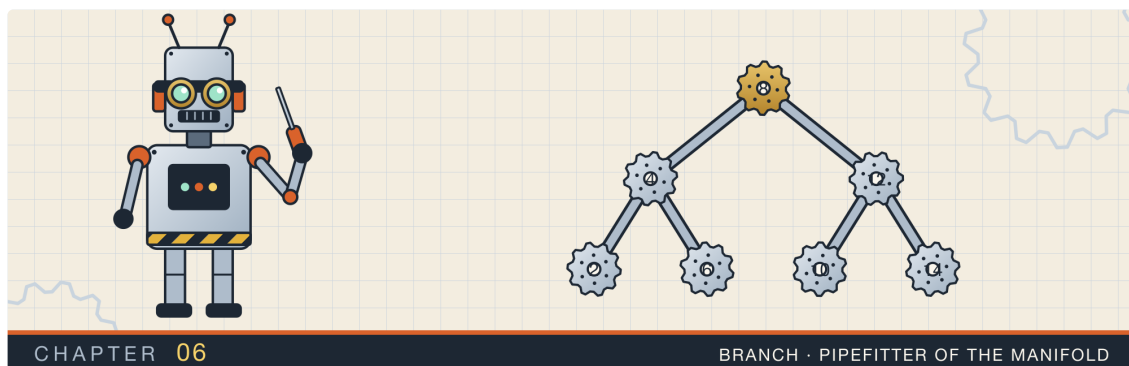
Summary

- A heap is a binary tree stored level by level in a `Vec`. The children of index i are at $2i + 1$ and $2i + 2$.
- In a max-heap every parent is at least as large as its children, so the largest item is at index 0.
- Push adds at the end and swaps upward. Pop moves the last item to the top and swaps downward with the larger child. Both are $O(\log n)$.
- `BinaryHeap` is a max-heap. Wrapping items in `Reverse` makes it a min-heap.
- Tuples compare element by element from the left, so `(count, value)` orders by count first.
- Top k with a full heap costs $O(u)$ memory. A min-heap bounded to k entries costs $O(k)$ memory and $O(u \log k)$ time.
- Two heaps, a max-heap for the small half and a min-heap for the large half, give the running median in $O(\log n)$ per number.

Chapter 6 covers recursion and dynamic programming. You will solve problems by combining answers to smaller versions of themselves, and save those answers so none is computed twice.

Exercises

1. Implement a max-heap yourself on a `Vec<i32>`, with `push`, `pop`, and `peek`, using the index formulas from section 5.1. Test it against `BinaryHeap` on random input.
2. Change `top_k_frequent_words` to keep the alphabetically first words among ties, as the tip in section 5.5 describes, and add a test.
3. Add a method `len` to `MedianFinder` that returns how many numbers it holds.
4. Extend the running median to remove numbers as well, so it can report the median of the last 100 numbers only. Hint: remove lazily, by remembering which numbers are waiting to be deleted.



Recursion and dynamic programming

THIS CHAPTER COVERS

- Recognizing a problem that can be built from answers to smaller versions of itself
- Filling a table of answers from the smallest case upward
- Listing every subset of a set by exploring a tree of choices
- The longest increasing run hidden inside a list, in $O(n \log n)$
- Choosing jobs that do not overlap, to earn the most

In chapter 1 you computed Fibonacci numbers three ways. The recursive version repeated work. The memoized version saved each answer in a table, and the loop version kept only the answers it still needed. That small example contains the whole idea of this chapter.

Dynamic programming is a way to solve a problem by answering smaller versions of the same problem first, saving those answers, and combining them. It applies when two things are true:

1. The answer for a large input can be built from the answers for smaller inputs.
2. The same smaller inputs come up again and again, so saving their answers avoids repeated work.

This chapter works through four problems. The first, making change with coins, is dynamic programming in its plainest form. The second, listing subsets, uses recursion without any saved answers. The last two combine dynamic programming with the sorting and searching you learned in chapter 3.

For each problem I first say, in one sentence, what one entry of the table means. That sentence is called the **subproblem**. Once it is written down, the code follows from it.

6.1 Making change with the fewest coins

The problem. You have coins of some values, as many of each as you like. What is the fewest number of coins that add up to a given amount? With coins 1, 2, and 5, the amount 11 needs three coins: $5 + 5 + 1$. Some amounts cannot be made at all. With only a 2 coin, the amount 3 is impossible.

6.1.1 Why the obvious approach fails

A natural first try is to take the largest coin that fits, again and again. This is called a **greedy** approach. It works for 1, 2, and 5. It does not work for every set of coins (figure 6.1).

Coins 1, 3, 4. Make 6 with as few coins as possible.

Greedy (always take the largest coin that fits):



Best:



Taking the largest coin first can lead to a worse answer, so the program must consider every coin.

Figure 6.1 Greedy picks 4 first and needs three coins. The best answer uses two.

So the program has to consider every coin at every step. Trying every combination directly takes exponential time. Dynamic programming makes it fast.

6.1.2 The subproblem

Write down what one entry of the table means:

$dp[t]$ is the fewest coins that add up to exactly t .

Now think about the last coin used to make t . If that coin is c , the coins before it add up to $t - c$, and the fewest coins for that is $dp[t - c]$. So using coin c last costs $dp[t - c] + 1$ coins. You do not know which coin is last in the best answer, so try every coin and keep the smallest:

$dp[t] = \text{the smallest of } dp[t - c] + 1, \text{ over every coin } c \text{ that is not larger than } t.$

The starting point is $dp[0] = 0$: zero coins make zero.

Fill the table by hand for the coins in figure 6.1, which are 1, 3, and 4, and the amount 6. Each row tries every coin that fits, and keeps the smallest count:

t	coins that fit	options from each coin	dp[t]
1	1	$dp[0] + 1 = 1$	1
2	1	$dp[1] + 1 = 2$	2
3	1, 3	$dp[2] + 1 = 3, dp[0] + 1 = 1$	1
4	1, 3, 4	$dp[3] + 1 = 2, dp[1] + 1 = 2, dp[0] + 1 = 1$	1
5	1, 3, 4	$dp[4] + 1 = 2, dp[2] + 1 = 3, dp[1] + 1 = 2$	2
6	1, 3, 4	$dp[5] + 1 = 3, dp[3] + 1 = 2, dp[2] + 1 = 3$	2

$dp[6]$ is 2, from $3 + 3$. Greedy needed three coins for the same amount, because it took the 4 first and never went back. The table records the best count for every smaller amount. Each new amount is then one lookup plus one.

Two properties must hold for that to work. The answers must **overlap**, so that saving them avoids work. $dp[6]$ and $dp[5]$ both read $dp[1]$, and a longer table reads each entry many times. The best answer must also be built from best answers to smaller amounts. Taking the smallest of $dp[t - c] + 1$ relies on that. If some way of making $t - c$ used fewer coins, $dp[t - c]$ would hold that smaller number. When either property fails, the table gives a wrong answer.

Because $dp[t]$ only uses entries for smaller amounts, you can fill the table from left to right. Figure 6.2 shows the table for coins 1, 2, and 5, and how $dp[11]$ is computed.

Filling dp for coins [1, 2, 5]: $dp[t]$ = fewest coins that make t

dp												
	0	1	1	2	2	1	2	2	3	3	2	3
	t=0	t=1	t=2	t=3	t=4	t=5	t=6	t=7	t=8	t=9	t=10	t=11

To fill $dp[11]$, try each coin c and look back at $dp[11 - c]$ (green cells):

coin 1: $dp[10] + 1 = 3$	coin 2: $dp[9] + 1 = 4$	coin 5: $dp[6] + 1 = 3$
--------------------------	-------------------------	-------------------------

The smallest option is 3, so $dp[11] = 3$ (cream).

Every cell depends only on cells to its left, so filling the table from left to right works.

Figure 6.2 The filled table. For $dp[11]$, the three coins lead back to $dp[10]$, $dp[9]$, and $dp[6]$.

Coin change: the fewest coins for every amount up to 11

$dp[t]$ is the smallest of $dp[t - \text{coin}] + 1$ over the coins 1, 2, and 5.

being filled read by it not reachable

t												
	0	.	.	.	.	.	.	.	.	.	.	
	0	1	2	3	4	5	6	7	8	9	10	11

dp

$dp[0] = 0$. Zero coins make zero, whatever the coins are.

A dot means the amount cannot be made from the coins, so no rule reaches it yet.

t = 0 dp[t] = 0 coins = 1 2 5

Animation 6.1 The table being filled left to right, one amount per frame. The rust cell is the entry being computed; the cream cell is the earlier entry chosen as the best source. Nothing looks to the right, which is why one pass suffices. On the last frame all three sources of $dp[11]$ are highlighted at once, so you can see the $\min$ over the coins.

6.1.3 The code

Listing 6.1 Coin change (lines 6 to 29). src/problems/dp.rs

```
pub fn coin_change(coins: &[i32], amount: i32) -> Option<i32> {
    if amount < 0 {
        return None;
    }

    let amount = amount as usize;
    let mut dp = vec![usize::MAX; amount + 1];
    dp[0] = 0;

    for total in 1..=amount {
        for &coin in coins {
            let coin = coin as usize;
            if coin > total || dp[total - coin] == usize::MAX {
                continue;
            }
            dp[total] = dp[total].min(dp[total - coin] + 1);
        }
    }

    match dp[amount] {
        usize::MAX => None,
        count => Some(count as i32),
    }
}
```

Read it against the recurrence:

- A negative amount is rejected first. After that check, `amount as usize` cannot produce a strange value.
- `vec![usize::MAX; amount + 1]` creates the table, one entry per amount from 0 to `amount`. `usize::MAX`, the largest `usize`, stands for "cannot be made yet".
- `dp[0] = 0` is the starting point.
- The outer loop fills amounts from 1 upward. The inner loop tries every coin.
- The `continue` skips a coin that is larger than the amount, or whose remainder cannot be made. The second check also prevents `usize::MAX + 1`, which would overflow.
- `dp[total].min(dp[total - coin] + 1)` keeps the smaller of the current best and the new option.

WARNING The second half of that `continue` is not an optimisation. Remove it and the function breaks. If `dp[total - coin]` holds the sentinel, then `dp[total - coin] + 1` adds one to the largest `usize`. A debug build panics with "attempt to add with overflow". A release build wraps around to 0, and the final `match` reads 0 as a real count of zero coins. The table then reports fewer coins than exist. Add the sentinel check before adding, or `usize::MAX` cannot stand for "impossible".

At the end, a `match` turns the table entry into the result. The pattern `usize::MAX` matches that exact value, meaning impossible, and returns `None`. Any other value is a count. So the function returns `Option<i32>`, and the caller cannot mistake "impossible" for a number of coins.

The table has `amount + 1` entries and each one tries every coin. The cost is $O(\text{amount} \times \text{coins})$ time and $O(\text{amount})$ memory.

Listing 6.2 The complete file. `src/problems/dp.rs`

```

///! Coin Change: minimum number of coins to make an amount.
///!
///! Classic dynamic programming. `Option<i32>` expresses "impossible" instead of
///! a magic `-1`.

pub fn coin_change(coins: &[i32], amount: i32) -> Option<i32> {
    if amount < 0 {
        return None;
    }

    let amount = amount as usize;
    let mut dp = vec![usize::MAX; amount + 1];
    dp[0] = 0;

    for total in 1..=amount {
        for &coin in coins {
            let coin = coin as usize;
            if coin > total || dp[total - coin] == usize::MAX {
                continue;
            }
            dp[total] = dp[total].min(dp[total - coin] + 1);
        }
    }

    match dp[amount] {
        usize::MAX => None,
        count => Some(count as i32),
    }
}

#[cfg(test)]
mod tests {
    use super::*;

```

```

#[test]
fn computes_minimum_coins() {
    assert_eq!(coin_change(&[1, 2, 5], 11), Some(3)); // 5 + 5 + 1
    assert_eq!(coin_change(&[2], 3), None);
    assert_eq!(coin_change(&[1], 0), Some(0));
}
}

```

Run the tests with `cargo test --lib coin_change`. The tests check the example from section 6.1, an impossible amount, and the amount 0.

6.2 Listing every subset

Not every recursive problem needs a table. Sometimes you need every possible answer, not the best one. Then recursion explores all the choices, and there is nothing to save.

The problem. List every subset of a set of distinct numbers. A **subset** is any selection of the elements, including none of them and all of them. `[1, 2, 3]` has eight subsets: `[]`, `[1]`, `[2]`, `[3]`, `[1, 2]`, `[1, 3]`, `[2, 3]`, and `[1, 2, 3]`.

The idea. Build subsets one element at a time. Start with the empty subset. From any subset, you can add one more element, as long as it comes later in the input than the elements already chosen. That rule is what stops `[2, 1]` from appearing as a copy of `[1, 2]`. Figure 6.3 draws the choices as a tree. Every node in the tree is one subset.

Every subset of `[1, 2, 3]` as a tree of choices

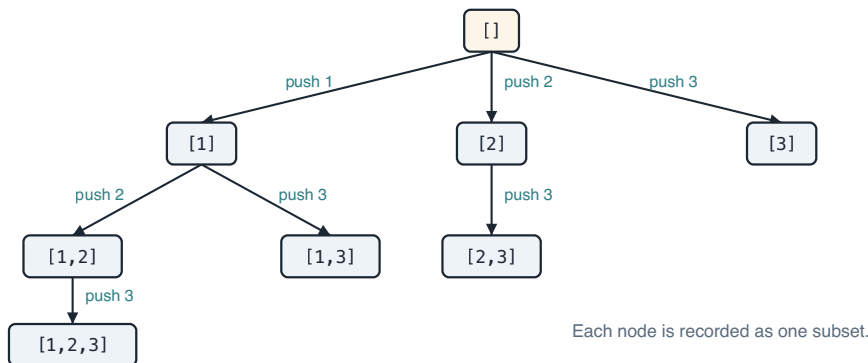


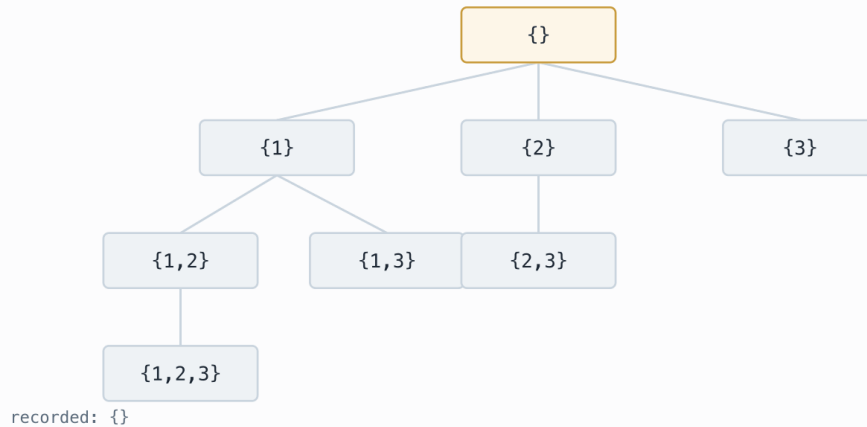
Figure 6.3 The tree of choices for `[1, 2, 3]`. Each edge adds an element that comes after the last one chosen.

To visit the whole tree, the program walks it depth first, going down one branch as far as it can before trying the next. Going down means adding an element; coming back up means removing it. This way of exploring choices is called **backtracking**.

Subsets: every path down the tree is one subset

Each call records the path so far, then tries every larger index.

 the path now its prefixes off the path



The call reaches {} and records it, then tries each index from 0 upward.

Backtracking pops the value it added, so one path buffer serves the whole tree.

path = {} start = 0 recorded = 1

Animation 6.2 One frame per subset the program records. The green path is the branch the recursion is on, and the printed list grows in the order the recursion reaches each node. Notice that the empty subset is recorded first, at the root, before any element is added. The order is depth first: the recursion finishes {1, 2, 3} before it ever tries {1, 3}.

Listing 6.3 Subsets by backtracking. `src/problems/backtracking.rs`

```

/// Subsets: enumerate every subset of a set of distinct integers.
///
/// Backtracking with clean `path.push(...)` + recursive exploration + `pop`.
/// `Vec<Vec<i32>>` is sorted in the tests to avoid order-dependent assertions.

pub fn subsets(nums: Vec<i32>) -> Vec<Vec<i32>> {
    let mut result = Vec::with_capacity(1 << nums.len());

```

```

    let mut path = Vec::new();

    fn backtrack(nums: &[i32], start: usize, path: &mut Vec<i32>, result: &mut
Vec<Vec<i32>>) {
        result.push(path.clone());
        for idx in start..nums.len() {
            path.push(nums[idx]);
            backtrack(nums, idx + 1, path, result);
            path.pop();
        }
    }

    backtrack(&nums, 0, &mut path, &mut result);
    result
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn generates_all_subsets() {
        let mut subsets = subsets(vec![1, 2, 3]);
        subsets.sort_unstable();
        assert_eq!(subsets.len(), 8);
        assert!(subsets.contains(&vec![]));
        assert!(subsets.contains(&vec![1, 2, 3]));
    }

    #[test]
    fn empty_input_has_one_subset() {
        assert_eq!(subsets(vec![]), vec![vec![]]);
    }
}

```

`backtrack` is a function defined inside `subsets`, as `go` was in chapter 1. It takes four things:

- `nums`, the input, borrowed.
- `start`, the first position it may choose from.
- `path`, the subset being built. Only one `path` vector exists for the whole search, passed as `&mut`.
- `result`, the list of finished subsets, also passed as `&mut`.

On entry, `backtrack` records `path.clone()` in the result. The current path is one subset, the node of the tree it has reached. Then, for each later position, it pushes that element, recurses to explore the subtree below, and pops the element. After the pop, `path` is exactly as it was before the push, ready for the next choice.

Trace the start of `subsets(vec![1, 2, 3])`:

Call	path on entry	Records	Then pushes
<code>backtrack(start 0)</code>	<code>[]</code>	<code>[]</code>	1
<code>backtrack(start 1)</code>	<code>[1]</code>	<code>[1]</code>	2
<code>backtrack(start 2)</code>	<code>[1, 2]</code>	<code>[1, 2]</code>	3
<code>backtrack(start 3)</code>	<code>[1, 2, 3]</code>	<code>[1, 2, 3]</code>	nothing: the loop is empty
back in <code>start 2</code>	<code>[1, 2]</code> after pop		no more positions
back in <code>start 1</code>	<code>[1]</code> after pop		3

`Vec::with_capacity(1 << nums.len())` reserves exactly enough room. $1 \ll n$ means 2 to the power n , because shifting 1 left by n bits doubles it n times. A set of n elements has 2^n subsets.

The work is $O(n \cdot 2^n)$: there are 2^n subsets and each clone copies up to n numbers. No algorithm can do better, because the output itself has that size.

6.3 The longest increasing run inside a list

The problem. In a list of numbers, pick some of them and keep their order. Each picked number must be larger than the one before. What is the most you can pick? The picked numbers are called an **increasing subsequence**. In `[10, 9, 2, 5, 3, 7, 101, 18]`, you can pick `2, 3, 7, 18`, so the answer is 4.

A subsequence may skip numbers. That is the difference from the substrings of chapter 3, which were contiguous.

The direct dynamic program. Let `dp[i]` be the length of the longest increasing subsequence that ends at position i . For each i , look at every earlier j with a smaller number, and take the best `dp[j] + 1`. That is $O(n^2)$, because each position looks at every earlier one.

A faster idea. Keep a different table, called `tails`. Its entries have this meaning:

The k -th smallest entry of `tails` is the smallest number that can end an increasing subsequence of length k .

Keeping the smallest possible ending number is useful because a smaller ending leaves more room for later numbers to extend it. When a new number arrives:

- If it is larger than every entry, it extends the longest subsequence found so far. Add it.
- Otherwise, find the smallest entry that is at least as large as it, and replace that entry with it. Some subsequence of that length can now end with a smaller number.

At the end, the number of entries is the answer. Figure 6.4 follows the example.

tails while reading [10, 9, 2, 5, 3, 7, 101, 18]

read 10	10	new length 1
read 9	9	9 replaces 10
read 2	2	2 replaces 9
read 5	2 5	5 extends
read 3	2 3	3 replaces 5
read 7	2 3 7	7 extends
read 101	2 3 7 101	101 extends
read 18	2 3 7 18	18 replaces 101

tails[k] is the smallest value that can end an increasing run of length k + 1. Its length, 4, is the answer.

Figure 6.4 tails after each number. The green cell is the one that changed.

Look at the last row. `tails` is `2, 3, 7, 18`, which happens to be a real increasing subsequence here. That is not true in general. `tails` can mix numbers from different subsequences. Only its length is meaningful.

The entries of `tails` are always in increasing order, so finding "the smallest entry at least as large" can use a sorted structure. The program uses a `BTreeSet`, a set that keeps its elements sorted.

Listing 6.4 The complete program. `src/bin/longest_increasing_subsequence.rs`

```
use std::collections::BTreeSet;

// TODO: Practice iterating over slice reference, BTreeSet. And also, BTreeMap...
fn lis_len(nums: &[i32]) -> usize {
    let mut tails = BTreeSet::new();

    for &num in nums {
```

```

        if let Some(&ceiling) = tails.range(num..).next() {
            tails.remove(&ceiling);
        }
        tails.insert(num);
    }

    tails.len()
}

fn main() {
    println!("{}", lis_len(&[10, 9, 2, 5, 3, 7, 101, 18])); // 4
    println!("{}", lis_len(&[0, 1, 0, 3, 2, 3]));           // 4
    println!("{}", lis_len(&[7, 7, 7, 7]));                 // 1
    println!("{}", lis_len(&[5, 4, 3, 2, 1]));               // 1
    println!("{}", lis_len(&[1, 2, 3, 4, 5]));               // 5
    println!("{}", lis_len(&[]));                           // 0
}

```

`tails.range(num..)` returns the elements from `num` upward, in sorted order. `.next()` takes the first of them, the smallest element that is at least `num`. If there is one, the program removes it. Then it inserts `num` in every case. Removing and inserting is how the set replaces one element with another.

The range `num..` includes `num` itself, which handles repeated numbers. In `[7, 7, 7, 7]`, the second 7 finds the first 7, removes it, and inserts itself again, so the answer stays 1. A run with a repeated number is not strictly increasing, so 1 is correct.

Each `range`, `remove`, and `insert` on a `BTreeSet` costs $O(\log n)$, so the whole program is $O(n \log n)$.

```

$ cargo run --bin longest_increasing_subsequence
4
4
1
1
5
0

```

The six outputs match the comments in `main`.

6.4 Choosing jobs that do not overlap

The problem. A machine can run one job at a time. Each job has a start time, an end time, and a profit. Choose jobs that do not overlap so that the total profit is as large as possible. Figure 6.5 shows the four sample jobs.

Four jobs on a timeline; A and D do not overlap and earn $50 + 70 = 120$

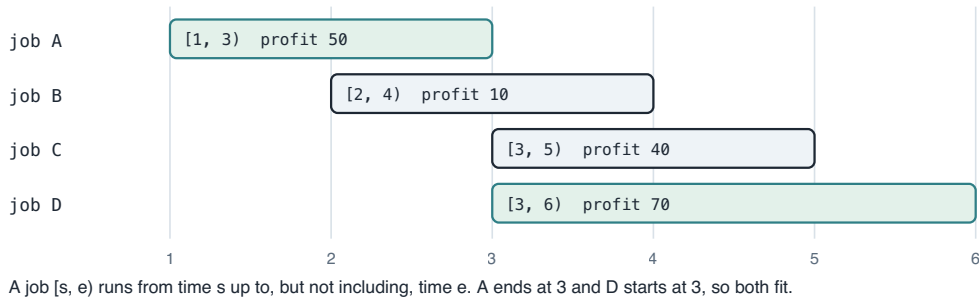


Figure 6.5 The sample jobs. A and D fit together and earn 120, the best possible.

A job written `[s, e)` runs from time s up to, but not including, time e. So a job ending at 3 and a job starting at 3 do not overlap.

6.4.1 Building the subproblem

First sort the jobs by end time. Then any job that can come before job i in a schedule ends earlier, so it appears earlier in the sorted list. The subproblem is:

`dp[i]` is the best profit using only the first i jobs of the sorted list.

For job i there are two choices:

- **Skip it.** The best profit stays `dp[i]`, the best without it.
- **Take it.** Then only jobs that end by its start time can come before it. Those jobs are a prefix of the sorted list, say the first `prev` jobs. The profit is `dp[prev] + profit of job i`.

So `dp[i + 1]` is the larger of the two. The last entry is the answer.

Finding `prev` is a search in a sorted list, so it can be a binary search, like chapter 3's. The standard library provides one as `partition_point`. Given a condition that is true for a prefix of the list and false after it, `partition_point` returns the length of that prefix.

6.4.2 The code

Listing 6.5 The complete program. `src/bin/job_scheduling_with_profit_and_deadline.rs`

```
#[derive(Debug, Default, Clone, Copy)]
struct Job {
```

```

    start: u32,
    end: u32,
    profit: u64,
}

fn max_profit(mut jobs: Vec<Job>) -> u64 {
    // Sort by end time so every job that could come before
    // job `i` in a schedule sits somewhere in jobs[..i].
    jobs.sort_unstable_by_key(|job| job.end);

    // dp[i] = best profit achievable using only the first `i` jobs.
    let mut dp = vec![0; jobs.len() + 1];

    for (i, job) in jobs.iter().enumerate() {
        // Binary search for how many earlier jobs finish by the time
        // this one starts. Those are exactly the jobs compatible with it,
        // and since they form a prefix, dp[prev] is their best profit.
        let prev = jobs[..i].partition_point(|j| j.end <= job.start);

        // Either skip this job (keep dp[i]),
        // or take it on top of the best compatible prefix.
        dp[i + 1] = dp[i].max(dp[prev] + job.profit);
    }

    // After considering every job, the last entry holds the answer.
    dp[jobs.len()]
}

fn main() {
    let jobs = vec![
        Job { start: 1, end: 3, profit: 50 },
        Job { start: 2, end: 4, profit: 10 },
        Job { start: 3, end: 5, profit: 40 },
        Job { start: 3, end: 6, profit: 70 },
    ];

    println!("{}", max_profit(jobs)); // 120
}

```

The comments inside `max_profit` walk through the same reasoning as section 6.4.1, step by step. `jobs[..i].partition_point(|j| j.end <= job.start)` counts the jobs among the first `i` that end by this job's start. Because the list is sorted by end time, those jobs come first, which is the condition `partition_point` needs.

Trace it on the sorted jobs A, B, C, D:

i	job	prev	skip: dp[i]	take: dp[prev] + profit	dp[i + 1]
0	A [1, 3) 50	0	0	0 + 50 = 50	50
1	B [2, 4) 10	0	50	0 + 10 = 10	50
2	C [3, 5) 40	1	50	50 + 40 = 90	90
3	D [3, 6) 70	1	90	50 + 70 = 120	120

For C and D, `prev` is 1: only A ends by time 3. Sorting costs $O(n \log n)$, and each job does one binary search, so the whole program is $O(n \log n)$.

```
$ cargo run --bin job_scheduling_with_profit_and_deadline
120
```

Summary

- Dynamic programming answers smaller versions of a problem first, saves the answers, and combines them.
- Start by writing, in one sentence, what one table entry means. The recurrence and the code follow from it.
- Coin change fills `dp[t]` from `dp[t - c] + 1` for each coin, left to right, in $O(\text{amount} \times \text{coins})$.
- Greedy choices can fail. Coins 1, 3, and 4 make 6 greedily with three coins, but two suffice.
- Backtracking explores a tree of choices with one shared `path`, pushing before each recursive call and popping after it. Listing all subsets is $O(n \cdot 2^n)$, the size of the output.
- The longest increasing subsequence keeps the smallest possible ending for each length, in a sorted set, for $O(n \log n)$.
- Weighted job scheduling sorts by end time, uses `partition_point` to find compatible jobs, and fills a table of best profits.

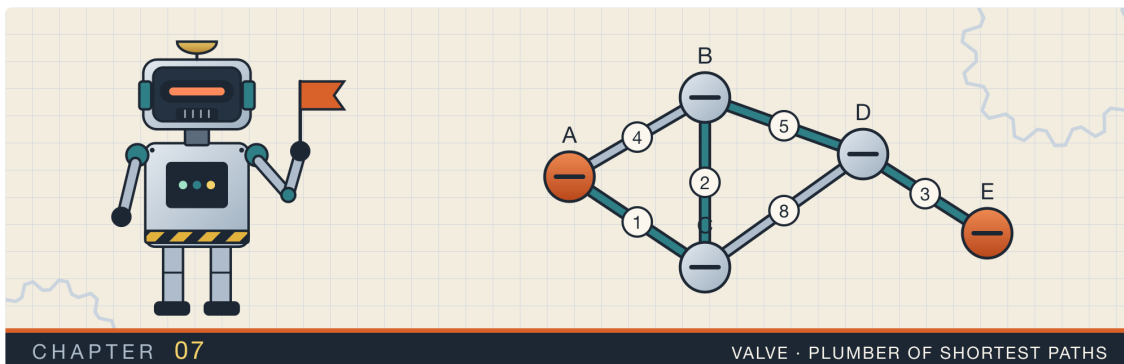
Chapter 7 begins Part 3, on modeling data. You will build types that can only hold valid values, using structs, enums, and error types.

Exercises

1. Change `coin_change` to also return the coins it used. Store the last coin chosen for each amount, then follow those choices back from `amount`.
2. Write the $O(n^2)$ version of the longest increasing subsequence described in section 6.3, and check that it agrees with `lis_len` on random lists.
3. Generate all subsets without recursion. Count a number `mask` from 0 to $2^n - 1$, and include element i when bit i of `mask` is set.
4. Extend `max_profit` to return which jobs it chose, not only the total.

PART 3

Modeling data and ownership



Types that refuse bad input

THIS CHAPTER COVERS

- Traits: what they are, how to implement one, and what `#[derive]` writes for you
- A newtype: a struct around one number that can only hold valid values
- Error types that say what went wrong, with `Display` and `std::error::Error`
- Converting between types with `From` and `TryFrom`
- Enums whose variants carry data, used for commands, for states, and for errors
- A parser that turns a line of text into a typed command

This chapter begins Part 3, on modeling data. So far your programs have used the types Rust provides: numbers, strings, vectors, and maps. In this chapter you design your own types, so that invalid values cannot be represented at all.

The examples come from a small command-line tool that manages jobs on a remote computer. The jobs are called **workloads**. Each workload asks for a number of GPUs, which must be between 1 and 64. A GPU is a graphics processor, and servers also use GPUs for general computation. A workload moves through states such as pending, running, and terminated. The tool reads commands typed by a user, such as `create triton 4`, which asks for a workload named `triton` with 4 GPUs. That is all you need to know about the tool.

Here is the problem this chapter solves. A GPU count arrives as text typed by a user. The program must reject `0`, `99`, and `many`, and it must do so before the count reaches the code that uses it. If each function checked the count itself, one forgotten check would let a bad value through. Instead, the program checks the count once, where the text enters, and stores

the result in a type that can only hold valid counts. Every function that receives that type can rely on it without checking again.

The chapter builds this in order:

1. Section 7.1 explains traits, which the rest of the chapter implements.
2. Section 7.2 builds the GPU count type.
3. Section 7.3 gives it an error type.
4. Section 7.4 adds the standard conversions.
5. Section 7.5 builds an enum for commands and a parser for them.
6. Section 7.6 models the workload's states and the moves between them.
7. Section 7.7 models the errors a server can return.

7.1 Traits

A **trait** is a set of methods that a type promises to provide. The standard library defines many traits. For example, the `Display` trait has one method, `fmt`, which writes a value as text for a person to read. `println!("{}", x)` works for any `x` whose type implements `Display`.

You implement a trait for your type with an `impl` block that names the trait:

```
impl fmt::Display for GpuCountError {
    fn fmt(&self, formatter: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(formatter, "...")
    }
}
```

Some traits are so regular that the compiler can write them for you. `#[derive(Debug, Clone, PartialEq)]` placed above a struct asks the compiler to generate three traits. `Debug` prints with `{:?}`, `Clone` copies with `.clone()`, and `PartialEq` compares with `==`. The derived versions work field by field.

Table 7.1 lists the traits this chapter uses.

Trait	What it gives you	How you get it
Debug	printing with <code>{:?}", for programmers</code>	<code>#[derive(Debug)]</code>
Display	printing with <code>{}</code> , for users	write <code>impl Display</code> yourself
Clone, Copy	<code>.clone()</code> , and for <code>Copy</code> , copying by assignment	derive
PartialEq, Eq	<code>==</code> and <code>!=</code>	derive
PartialOrd, Ord	<code><</code> , <code>></code> , and sorting	derive
Hash	use as a <code>HashMap</code> key	derive
<code>std::error::Error</code>	use as an error with <code>?</code> and <code>Box<dyn Error></code>	write an empty <code>impl</code> once <code>Debug</code> and <code>Display</code> exist
From, TryFrom	conversions between types	write <code>impl</code> yourself

7.2 A GPU count that is always valid

A **newtype** is a struct with one field, used to give a plain value a type of its own. Here the plain value is a `u32`, and the new type is `GpuCount`.

The trick is privacy. Fields in Rust are private by default. Code outside the module where the struct is defined cannot read or write them, and cannot build the struct with `GpuCount(0)`. The only way to get a `GpuCount` is through the functions the module offers. If those functions check the value, every `GpuCount` in the program is valid. Figure 7.1 shows the design.

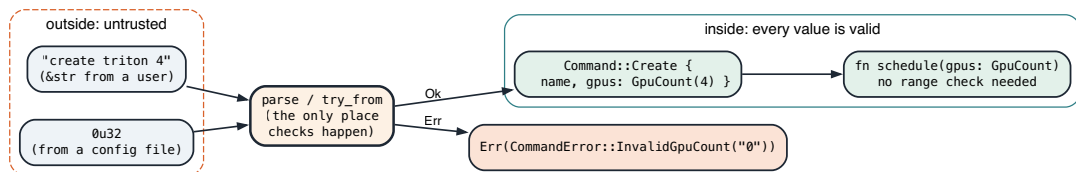


Figure 7.1 Checks happen once, at the gate. Inside, the types guarantee the values are valid.

The module starts with a comment that lists its ideas:

Listing 7.1 The module comment (lines 1 to 12). `src/problems/adt_idioms.rs`

```

/// Small algebraic-data-type idioms.
///
/// These patterns are short and cannot represent invalid states.
///
/// - A newtype gives a primitive a name and a single validating constructor, so
///   an out-of-range value cannot exist.
/// - An enum with data models a closed set of alternatives, and `match` forces
///   every alternative to be handled.
/// - Typed errors carry the reason instead of a string, so the caller can react.
/// - `TryFrom` puts the validation on the type boundary, where it belongs.
/// - An iterator chain states the intent in one pass and removes index
///   bookkeeping.

```

Here is the type:

Listing 7.2 `GpuCount` (lines 14 to 36).

```

use std::fmt;

/// A GPU count that is validated at construction.
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct GpuCount(u32);

impl GpuCount {
    pub const MIN: u32 = 1;
    pub const MAX: u32 = 64;

    /// Validate a raw count.
    pub fn new(value: u32) -> Result<Self, GpuCountError> {
        if (Self::MIN..=Self::MAX).contains(&value) {
            Ok(Self(value))
        } else {
            Err(GpuCountError(value))
        }
    }

    pub fn get(self) -> u32 {
        self.0
    }
}

```

- `pub struct GpuCount(u32);` is a **tuple struct**: a struct whose field has no name, only a position. The struct is public, but its field is not.
- `pub const MIN: u32 = 1;` and `MAX` are **associated constants**, constants that belong to the type. You refer to them as `GpuCount::MIN`, or `Self::MIN` inside the `impl`.
- `new` is the gate. `(Self::MIN..=Self::MAX)` is the range 1 to 64, including both ends, and `.contains(&value)` checks the value. If it is in range, `new` returns `Ok(Self(value))`. If not, it returns an error.

- `get` returns the number inside. It takes `self` by value, which is fine because `GpuCount` is `Copy`.

The long `#[derive(...)]` line gives `GpuCount` the same abilities as the `u32` inside it. You can print it, copy it, compare it, sort it, and use it as a map key.

7.3 An error that says what went wrong

When `new` rejects a value, it returns a `GpuCountError`. Here is that type:

Listing 7.3 The error type (lines 38 to 54).

```
/// Why a raw GPU count was rejected.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct GpuCountError(pub u32);

impl fmt::Display for GpuCountError {
    fn fmt(&self, formatter: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(
            formatter,
            "gpu count {} is outside {}.{}={}",
            self.0,
            GpuCount::MIN,
            GpuCount::MAX
        )
    }
}

impl std::error::Error for GpuCountError {}
```

`GpuCountError(pub u32)` keeps the rejected value. Its field is `pub`, so a caller can read which value was rejected.

The `Display` implementation writes the message a user sees. `write!` works like `format!`, but writes into the formatter. The message includes the value and the allowed range: `gpu count 0 is outside 1..=64`.

`impl std::error::Error for GpuCountError {}` is empty. The `Error` trait's methods all have defaults, and it requires `Debug` and `Display`, which the type already has. Implementing it lets this error work with other error-handling code, such as `Box<dyn std::error::Error>`, which you will see in section 7.5.

7.4 Converting with `From` and `TryFrom`

The standard library has two traits for converting a value of one type into another:

- `From<A>` for `B` is for conversions that always succeed. Implementing it gives you `B::from(a)`, and also `a.into()` wherever a `B` is expected.
- `TryFrom<A>` for `B` is for conversions that can fail. Its `try_from` returns a `Result`. Implementing it gives you `B::try_from(a)` and `a.try_into()`.

Listing 7.4 The two conversions (lines 56 to 68).

```
impl TryFrom<u32> for GpuCount {
    type Error = GpuCountError;

    fn try_from(value: u32) -> Result<Self, Self::Error> {
        Self::new(value)
    }
}

impl From<GpuCount> for u32 {
    fn from(count: GpuCount) -> u32 {
        count.0
    }
}
```

Going from `u32` to `GpuCount` can fail, so it is `TryFrom`. Its body calls `new`, so the check still lives in one place. `type Error = GpuCountError;` names the error type this conversion returns.

Going from `GpuCount` back to `u32` cannot fail, so it is `From`. The body returns the field, `count.0`.

TIP: Rust also has the `as` keyword, as in `x as u32`. `as` never reports failure: converting 300 to a `u8` with `as` quietly gives 44. Prefer `From` when a conversion cannot fail and `TryFrom` when it can, and keep `as` for cases where you want truncation.

7.5 Commands: an enum that carries data

7.5.1 Why an enum

The tool accepts four commands. `list` takes no arguments. `get` and `delete` each take an ID. `create` takes a name and a GPU count. A first design might put all the possible fields in one struct:

```
struct Command {
    verb: String,
    id: Option<String>,           // only for get and delete
    name: Option<String>,        // only for create
    gpus: Option<u32>,           // only for create
}
```

This struct can hold a `get` without an ID, or a `list` with a GPU count. Every function that receives it has to check which fields make sense for the verb.

An enum whose variants each carry their own data cannot hold those combinations:

Listing 7.5 The command enum (lines 70 to 77).

```
/// A closed set of commands, each carrying exactly the data it needs.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Command {
    List,
    Get { id: String },
    Create { name: String, gpus: GpuCount },
    Delete { id: String },
}
```

A `Create` always has a name and a valid `GpuCount`, because Rust will not build one without them. A `List` has no fields, because it needs none.

In memory, every `Command` takes the same space, enough for its largest variant, as figure 7.2 shows.

Every Command value takes 32 bytes: room for its largest variant

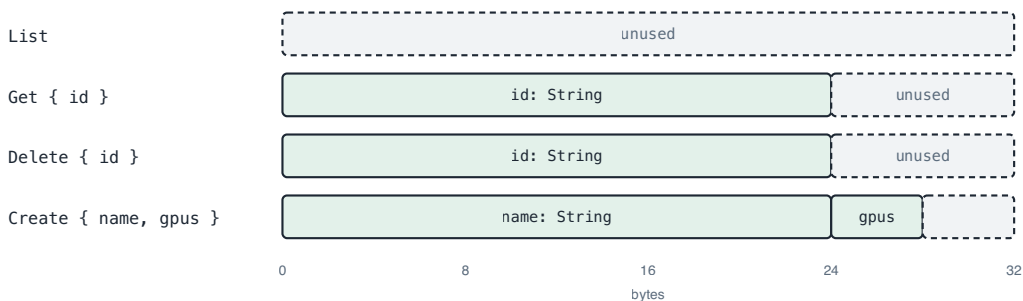


Figure 7.2 A Command is 32 bytes. Rust also records which variant a value is. Here it stores that in bit patterns the `String` never uses, so no extra byte is needed.

7.5.2 Parsing a line of text

Listing 7.6 `Command::parse` and `verb` (lines 79 to 115).

```
impl Command {
    /// Parse one command line.
    pub fn parse(line: &str) -> Result<Self, CommandError> {
        let mut parts = line.split_whitespace();
        let verb = parts.next().ok_or(CommandError::Empty)?;

        match verb {
            "list" => Ok(Self::List),
            "get" => Ok(Self::Get {
                id: argument(&mut parts, "id")?.to_string(),
            }),
            "delete" => Ok(Self::Delete {
                id: argument(&mut parts, "id")?.to_string(),
            }),
            "create" => {
                let name = argument(&mut parts, "name")?.to_string();
                let raw = argument(&mut parts, "gpu count")?;
                let value: u32 = raw
                    .parse()
                    .map_err(|_| CommandError::InvalidGpuCount(raw.to_string()))?;
                let gpus = GpuCount::new(value)
                    .map_err(|_| CommandError::InvalidGpuCount(raw.to_string()))?;
                Ok(Self::Create { name, gpus })
            },
            other => Err(CommandError::UnknownVerb(other.to_string())),
        }
    }

    pub fn verb(&self) -> &'static str {
        match self {
            Self::List => "list",
            Self::Get { .. } => "get",
            Self::Create { .. } => "create",
        }
    }
}
```

```

        Self::Delete { .. } => "delete",
    }
}

```

`parse` returns `Result<Self, CommandError>`: a command, or the reason the line could not be parsed. Trace `Command::parse("create triton 4")` step by step:

Step	Code	Result
1	<code>line.split_whitespace()</code>	an iterator over "create", "triton", "4"
2	<code>parts.next().ok_or(CommandError::Empty)?</code>	verb is "create"
3	<code>match verb</code>	runs the "create" arm
4	<code>argument(&mut parts, "name")?</code>	"triton", turned into a <code>String</code>
5	<code>argument(&mut parts, "gpu count")?</code>	raw is "4"
6	<code>raw.parse()</code>	the number 4
7	<code>GpuCount::new(value)</code>	<code>GpuCount(4)</code>
8	<code>Ok(Self::Create { name, gpus })</code>	the finished command

Now try some bad inputs. An empty line fails at step 2, because there is no first word. `ok_or` turns the `None` into `Err(CommandError::Empty)`, and `?` returns it. The line `create triton many` fails at step 6, because "many" is not a number. The line `create triton 0` passes step 6 and fails at step 7.

Steps 6 and 7 can fail for different reasons, but both failures are reported the same way. `.map_err(|_| CommandError::InvalidGpuCount(raw.to_string()))` replaces whatever error came back with `InvalidGpuCount`, carrying the text the user typed. The user then sees their own input in the message.

`verb` goes the other way, from a command to its name. The patterns `Self::Get { .. }` match a variant and ignore its fields.

7.5.3 The parser's errors and helper

Listing 7.7 `CommandError` and `argument` (lines 117 to 144).

```
/// A typed parse failure, so callers can distinguish the cases.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum CommandError {
    Empty,
    UnknownVerb(String),
    MissingArgument(&'static str),
    InvalidGpuCount(String),
}

impl fmt::Display for CommandError {
    fn fmt(&self, formatter: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            Self::Empty => write!(formatter, "empty command"),
            Self::UnknownVerb(verb) => write!(formatter, "unknown verb {verb:?}"),
            Self::MissingArgument(name) => write!(formatter, "missing argument: {name}"),
            Self::InvalidGpuCount(raw) => write!(formatter, "invalid gpu count: {raw:?}"),
        }
    }
}

impl std::error::Error for CommandError {}

fn argument<'a>(&mut impl Iterator<Item = &'a str>,
    name: &'static str,
) -> Result<&'a str, CommandError> {
    parts.next().ok_or(CommandError::MissingArgument(name))
}
```

`CommandError` lists every way parsing can fail, and its `Display` writes a message for each. Because the errors are enum variants, a caller can `match` on them and react differently to each. A CLI could print usage help for `MissingArgument` and the allowed range for `InvalidGpuCount`. With a plain `String` error, the caller would have to read the message text to tell the cases apart.

`argument` takes the next word from the iterator, or fails with the argument's name. Look at its first parameter, `&mut impl Iterator<Item = &'a str>`. It borrows the caller's iterator mutably, so that each call consumes one more word. The lifetime `'a`, which you met in chapter 4, says the returned `&str` points into the original line. So no text is copied until `parse` decides to keep it with `to_string()`.

7.5.4 Adding up without an index

The file has one more small function:

Listing 7.8 summarize (lines 146 to 151).

```

/// Count and sum in one pass, without managing an index.
pub fn summarize(values: &[i32]) -> (usize, i64) {
    values.iter().fold((0usize, 0i64), |(count, sum), value| {
        (count + 1, sum + i64::from(*value))
    })
}

```

`fold` walks the slice once and carries a value along. Here the value is a pair, `(count, sum)`, starting at `(0, 0)`. For each element, the closure returns the new pair. `i64::from(*value)` widens each `i32` to `i64`. That conversion is a `From`, so it can never lose information.

7.5.5 The complete file

Listing 7.9 The complete file, with its tests. `src/problems/adt_idioms.rs`

```

///! Small algebraic-data-type idioms.
///!
///! These patterns are short and cannot represent invalid states.
///!
///! - A newtype gives a primitive a name and a single validating constructor, so
///!   an out-of-range value cannot exist.
///! - An enum with data models a closed set of alternatives, and `match` forces
///!   every alternative to be handled.
///! - Typed errors carry the reason instead of a string, so the caller can react.
///! - `TryFrom` puts the validation on the type boundary, where it belongs.
///! - An iterator chain states the intent in one pass and removes index
///!   bookkeeping.

use std::fmt;

/// A GPU count that is validated at construction.
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct GpuCount(u32);

impl GpuCount {
    pub const MIN: u32 = 1;
    pub const MAX: u32 = 64;

    /// Validate a raw count.
    pub fn new(value: u32) -> Result<Self, GpuCountError> {
        if (Self::MIN..=Self::MAX).contains(&value) {
            Ok(Self(value))
        } else {
            Err(GpuCountError(value))
        }
    }

    pub fn get(self) -> u32 {
        self.0
    }
}

```

```

/// Why a raw GPU count was rejected.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct GpuCountError(pub u32);

impl fmt::Display for GpuCountError {
    fn fmt(&self, formatter: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(
            formatter,
            "gpu count {} is outside {}.{}={}",
            self.0,
            GpuCount::MIN,
            GpuCount::MAX
        )
    }
}

impl std::error::Error for GpuCountError {}

impl TryFrom<u32> for GpuCount {
    type Error = GpuCountError;

    fn try_from(value: u32) -> Result<Self, Self::Error> {
        Self::new(value)
    }
}

impl From<GpuCount> for u32 {
    fn from(count: GpuCount) -> u32 {
        count.0
    }
}

/// A closed set of commands, each carrying exactly the data it needs.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Command {
    List,
    Get { id: String },
    Create { name: String, gpus: GpuCount },
    Delete { id: String },
}

impl Command {
    /// Parse one command line.
    pub fn parse(line: &str) -> Result<Self, CommandError> {
        let mut parts = line.split_whitespace();
        let verb = parts.next().ok_or(CommandError::Empty)?;

        match verb {
            "list" => Ok(Self::List),
            "get" => Ok(Self::Get {
                id: argument(&mut parts, "id")?.to_string(),
            }),
            "delete" => Ok(Self::Delete {
                id: argument(&mut parts, "id")?.to_string(),
            }),
            "create" => {
                let name = argument(&mut parts, "name")?.to_string();
                let raw = argument(&mut parts, "gpu count"?);
            }
        }
    }
}

```

```

        let value: u32 = raw
            .parse()
            .map_err(|_| CommandError::InvalidGpuCount(raw.to_string()))?;
        let gpus = GpuCount::new(value)
            .map_err(|_| CommandError::InvalidGpuCount(raw.to_string()))?;
        Ok(Self::Create { name, gpus })
    }
    other => Err(CommandError::UnknownVerb(other.to_string())),
}

pub fn verb(&self) -> &'static str {
    match self {
        Self::List => "list",
        Self::Get { .. } => "get",
        Self::Create { .. } => "create",
        Self::Delete { .. } => "delete",
    }
}

/// A typed parse failure, so callers can distinguish the cases.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum CommandError {
    Empty,
    UnknownVerb(String),
    MissingArgument(&'static str),
    InvalidGpuCount(String),
}

impl fmt::Display for CommandError {
    fn fmt(&self, formatter: &mut fmt::Formatter<'_,>) -> fmt::Result {
        match self {
            Self::Empty => write!(formatter, "empty command"),
            Self::UnknownVerb(verb) => write!(formatter, "unknown verb {verb:?}"),
            Self::MissingArgument(name) => write!(formatter, "missing argument: {name:?}"),
            Self::InvalidGpuCount(raw) => write!(formatter, "invalid gpu count: {raw:?}"),
        }
    }
}

impl std::error::Error for CommandError {}

fn argument<'a>(&mut impl Iterator<Item = &'a str>,
    name: &'static str,
) -> Result<&'a str, CommandError> {
    parts.next().ok_or(CommandError::MissingArgument(name))
}

/// Count and sum in one pass, without managing an index.
pub fn summarize(values: &[i32]) -> (usize, i64) {
    values.iter().fold((0usize, 0i64), |(count, sum), value| {
        (count + 1, sum + i64::from(*value))
    })
}

#[cfg(test)]
mod tests {

```

```

use super::*;

#[test]
fn parses_each_command_variant() {
    assert_eq!(Command::parse("list"), Ok(Command::List));
    assert_eq!(
        Command::parse("get wl-7"),
        Ok(Command::Get { id: "wl-7".into() })
    );
    assert_eq!(
        Command::parse("create triton 4"),
        Ok(Command::Create {
            name: "triton".into(),
            gpus: GpuCount::new(4).expect("valid")
        })
    );
    assert_eq!(
        Command::parse("delete wl-7"),
        Ok(Command::Delete { id: "wl-7".into() })
    );
}

#[test]
fn rejects_bad_input_with_typed_errors() {
    assert_eq!(Command::parse(""), Err(CommandError::Empty));
    assert_eq!(
        Command::parse("froblicate"),
        Err(CommandError::UnknownVerb("froblicate".into()))
    );
    assert_eq!(
        Command::parse("get"),
        Err(CommandError::MissingArgument("id"))
    );
    assert_eq!(
        Command::parse("create triton 0"),
        Err(CommandError::InvalidGpuCount("0".into()))
    );
    assert_eq!(
        Command::parse("create triton many"),
        Err(CommandError::InvalidGpuCount("many".into()))
    );
}

#[test]
fn newtype_rejects_out_of_range_values() {
    assert!(GpuCount::new(1).is_ok());
    assert!(GpuCount::new(64).is_ok());
    assert!(GpuCount::new(0).is_err());
    assert!(GpuCount::new(65).is_err());
    assert_eq!(u32::from(GpuCount::try_from(8).expect("valid")), 8);
    assert!(GpuCount::try_from(0).is_err());
}

#[test]
fn summarize_reports_count_and_sum() {
    assert_eq!(summarize(&[]), (0, 0));
    assert_eq!(summarize(&[1, 2, 3]), (3, 6));
    assert_eq!(summarize(&[-5, 5]), (2, 0));
}

```

```

#[test]
fn errors_render_a_reason() {
    assert_eq!(
        Command::parse("create triton 0").unwrap_err().to_string(),
        "invalid gpu count: \"0\""
    );
    assert!(
        GpuCount::new(99)
            .unwrap_err()
            .to_string()
            .contains("outside")
    );
}
}

```

Read the tests as a description of the behavior. `parses_each_command_variant` shows one valid line per variant. `rejects_bad_input_with_typed_errors` shows each failure, and compares the errors with `assert_eq!`. That comparison works because `CommandError` derives `PartialEq`. `newtype_rejects_out_of_range_values` tests both edges of the range, 1 and 64, and one step past each.

The tests run in parallel, so their order in the output can change from run to run:

```

$ cargo test --lib adt_idioms
running 5 tests
test problems::adt_idioms::tests::newtype_rejects_out_of_range_values ... ok
test problems::adt_idioms::tests::summarize_reports_count_and_sum ... ok
test problems::adt_idioms::tests::rejects_bad_input_with_typed_errors ... ok
test problems::adt_idioms::tests::parses_each_command_variant ... ok
test problems::adt_idioms::tests::errors_render_a_reason ... ok

test result: ok. 5 passed; 0 failed

```

The command enum describes what a user asks for. The next section uses an enum to describe something that changes over time.

7.6 The states of a workload

A workload is accepted (`Pending`), gets machines assigned (`Provisioning`), runs (`Running`), is shut down (`Terminating`), and ends (`Terminated`). Any of the active states can fail, and a failed workload can be tried again. A description like this, a set of states and the allowed moves between them, is called a **state machine**. Figure 7.3 draws it.

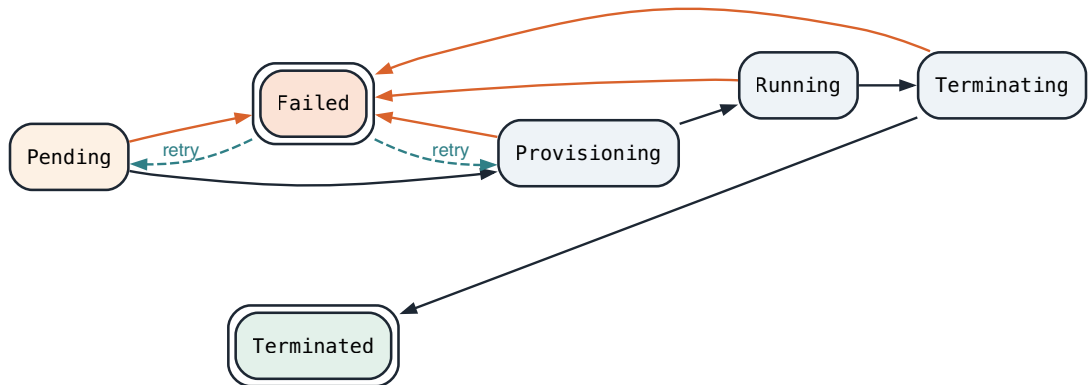


Figure 7.3 The workload's states. Double borders mark states where the workload holds no machines. Dashed arrows are retries.

Listing 7.10 The states (lines 1 to 18). `src/problems/state_machine.rs`

```

///! Algebraic data types / enum state machines.
///!
///! Cloud SDKs model workloads as finite state machines. Rust `enum` is
///! perfect for this: invalid states are unrepresentable and transitions are
///! explicit `match` arms, not nested `if` chains.

use std::time::Duration;

/// Lifecycle of a GPU cloud workload exposed through an SDK/API.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum WorkloadState {
    Pending,
    Provisioning,
    Running,
    Terminating,
    Terminated,
    Failed,
}

```

Each state is a variant with no data. Such an enum is stored as a single byte, and it derives `Copy`, so you pass states around by value.

Listing 7.11 The allowed moves (lines 20 to 56).

```

impl WorkloadState {
    /// Returns an error string for impossible transitions, or `Ok(())`.
    pub fn can_transition_to(self, next: Self) -> Result<(), String> {
        match (self, next) {
            // Forward lifecycle.
            (Self::Pending, Self::Provisioning)

```

```

    | (Self::Pending, Self::Failed)
    | (Self::Provisioning, Self::Running)
    | (Self::Provisioning, Self::Failed)
    | (Self::Running, Self::Terminating)
    | (Self::Running, Self::Failed)
    | (Self::Terminating, Self::Terminated)
    | (Self::Terminating, Self::Failed) => Ok(()),

    // Retry loops allowed from failed states.
    (Self::Failed, Self::Pending) | (Self::Failed, Self::Provisioning) => Ok(()),

    // Everything else is invalid.
    _ => Err(format!("cannot transition from {self:?} to {next:?}")),
}

pub fn is_terminal(self) -> bool {
    matches!(self, Self::Terminated | Self::Failed)
}

pub fn summary(self) -> &'static str {
    match self {
        Self::Pending => "workload accepted, waiting for scheduler",
        Self::Provisioning => "GPU resources being allocated",
        Self::Running => "workload is running",
        Self::Terminating => "cleanup in progress",
        Self::Terminated => "workload stopped and billing ended",
        Self::Failed => "workload failed; inspect logs",
    }
}
}

```

`can_transition_to` checks one move, from `self` to `next`. It matches on the pair `(self, next)`, so each pattern is one arrow in figure 7.3. Patterns joined with `|` share one arm: this is an **or-pattern**. The first arm lists the forward moves and the failure moves. The second lists the retries. The last arm, `_`, matches every other pair and returns an error.

That last arm decides what happens to moves nobody listed. With it, any move not written down is rejected. If someone adds a new state later, every move into or out of it is rejected until they list it. That is the safe default for a state machine.

`is_terminal` uses the `matches!` macro, which returns `true` when a value matches a pattern. `Failed` counts as terminal even though it can retry, because it holds no machines. `summary` returns a fixed message for each state. Its return type `&'static str` means the text lives for the whole program, because it is written into the program itself.

7.7 Errors from a server

The same file models the errors a server can send back. Here, different errors carry different information.

Listing 7.12 `ApiError` and its methods (lines 58 to 91).

```
/// API error modeled as an enum with associated data.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ApiError {
    InvalidRequest(String),
    Unauthorized,
    RateLimited { retry_after: Duration },
    NotFound { resource_id: String },
    Server { status: u16, message: String },
}

impl ApiError {
    pub fn is_retryable(&self) -> bool {
        matches!(
            self,
            Self::RateLimited { .. }
            | Self::Server {
                status: 500..=599,
                ..
            }
        )
    }

    pub fn user_message(&self) -> String {
        match self {
            Self::InvalidRequest(msg) => format!("invalid request: {msg}"),
            Self::Unauthorized => "authentication required or token expired".to_string(),
            Self::RateLimited { retry_after } => {
                format!("rate limited, retry in {}ms", retry_after.as_millis())
            },
            Self::NotFound { resource_id } => format!("resource not found: {resource_id}"),
            Self::Server { status, message } => format!("server error {status}: {message}"),
        }
    }
}
```

A rate-limit error says how long to wait before trying again. A not-found error says which resource. A server error carries the HTTP status code and a message.

`is_retryable` answers a question every client asks after a failure: could trying again help? A rate limit will pass, and a server error in the 500s may be temporary, so both are retryable. A missing resource will still be missing. The whole rule is one pattern:

```
Self::RateLimited { .. } | Self::Server { status: 500..=599, .. }
```

`status: 500..=599` inside the struct pattern matches any status from 500 to 599. `..` ignores the other fields. Chapter 19 builds a retry loop around this method.

`user_message` turns each error into text for a person. `retry_after.as_millis()` converts the waiting time to milliseconds for the message.

Listing 7.13 A small parser (lines 93 to 101).

```
/// Typical SDK helper: never expose nested `if let` soup to callers.
pub fn parse_gpu_count(raw: &str) -> Result<u32, ApiError> {
    match raw.trim() {
        "" => Err(ApiError::InvalidRequest("gpuCount is empty".into())),
        parsed => parsed.parse::<u32>().map_err(|_| {
            ApiError::InvalidRequest(format!("gpuCount must be a number, got {raw:?}"))
        }),
    }
}
```

`parse_gpu_count` is a shorter version of the gate from section 7.2. It trims the text, handles the empty case with its own message, and converts the standard library's parse error into `ApiError::InvalidRequest`. `{raw:?}` prints the text with quotes, so an empty or odd input is visible in the message.

Listing 7.14 The complete file, with its tests. `src/problems/state_machine.rs`

```
//! Algebraic data types / enum state machines.
//!
//! Cloud SDKs model workloads as finite state machines. Rust `enum` is
//! perfect for this: invalid states are unrepresentable and transitions are
//! explicit `match` arms, not nested `if` chains.

use std::time::Duration;

/// Lifecycle of a GPU cloud workload exposed through an SDK/API.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum WorkloadState {
    Pending,
    Provisioning,
    Running,
    Terminating,
    Terminated,
    Failed,
}

impl WorkloadState {
    /// Returns an error string for impossible transitions, or `Ok(())`.
    pub fn can_transition_to(self, next: Self) -> Result<(), String> {
        match (self, next) {

```

```

        // Forward lifecycle.
        (Self::Pending, Self::Provisioning)
        | (Self::Pending, Self::Failed)
        | (Self::Provisioning, Self::Running)
        | (Self::Provisioning, Self::Failed)
        | (Self::Running, Self::Terminating)
        | (Self::Running, Self::Failed)
        | (Self::Terminating, Self::Terminated)
        | (Self::Terminating, Self::Failed) => Ok(()),

        // Retry loops allowed from failed states.
        (Self::Failed, Self::Pending) | (Self::Failed, Self::Provisioning) => Ok(()),

        // Everything else is invalid.
        _ => Err(format!("cannot transition from {self:?} to {next:?}")),
    }
}

pub fn is_terminal(self) -> bool {
    matches!(self, Self::Terminated | Self::Failed)
}

pub fn summary(self) -> &'static str {
    match self {
        Self::Pending => "workload accepted, waiting for scheduler",
        Self::Provisioning => "GPU resources being allocated",
        Self::Running => "workload is running",
        Self::Terminating => "cleanup in progress",
        Self::Terminated => "workload stopped and billing ended",
        Self::Failed => "workload failed; inspect logs",
    }
}

}

/// API error modeled as an enum with associated data.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ApiError {
    InvalidRequest(String),
    Unauthorized,
    RateLimited { retry_after: Duration },
    NotFound { resource_id: String },
    Server { status: u16, message: String },
}

impl ApiError {
    pub fn is_retryable(&self) -> bool {
        matches!(
            self,
            Self::RateLimited { .. }
            | Self::Server {
                status: 500..=599,
                ..
            }
        )
    }

    pub fn user_message(&self) -> String {
        match self {
            Self::InvalidRequest(msg) => format!("invalid request: {msg}"),

```

```

        Self::Unauthorized => "authentication required or token expired".to_string(),
        Self::RateLimited { retry_after } => {
            format!("rate limited, retry in {}ms", retry_after.as_millis())
        }
        Self::NotFound { resource_id } => format!("resource not found: {resource_id}")
    },
    Self::Server { status, message } => format!("server error {status}:
{message}"),
}

}

// Typical SDK helper: never expose nested `if let` soup to callers.
pub fn parse_gpu_count(raw: &str) -> Result<u32, ApiError> {
    match raw.trim() {
        "" => Err(ApiError::InvalidRequest("gpuCount is empty".into())),
        parsed => parsed.parse::<u32>().map_err(|_| {
            ApiError::InvalidRequest(format!("gpuCount must be a number, got {raw:?}"))
        }),
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn state_machine_rejects_invalid_transitions() {
        assert!(
            WorkloadState::Pending
                .can_transition_to(WorkloadState::Provisioning)
                .is_ok()
        );
        assert!(
            WorkloadState::Running
                .can_transition_to(WorkloadState::Running)
                .is_err()
        );
        assert!(
            WorkloadState::Terminated
                .can_transition_to(WorkloadState::Running)
                .is_err()
        );
        assert!(WorkloadState::Terminating.is_terminal() == false);
        assert!(WorkloadState::Terminated.is_terminal());
    }

    #[test]
    fn api_error_retry_policy() {
        let rate_limited = ApiError::RateLimited {
            retry_after: Duration::from_millis(100),
        };
        let not_found = ApiError::NotFound {
            resource_id: "gpu-123".into(),
        };
        let server = ApiError::Server {
            status: 503,
            message: "maintenance".into(),
        };
    }
}

```

```

    assert!(rate_limited.is_retryable());
    assert!(!not_found.is_retryable());
    assert!(server.is_retryable());
}

#[test]
fn parsing_returns_typed_errors() {
    assert_eq!(parse_gpu_count("4"), Ok(4));
    assert!(parse_gpu_count("").is_err());
    assert!(parse_gpu_count("abc").is_err());
}
}

```

The tests check an allowed move and two forbidden ones, the retry rule for three errors, and the parser.

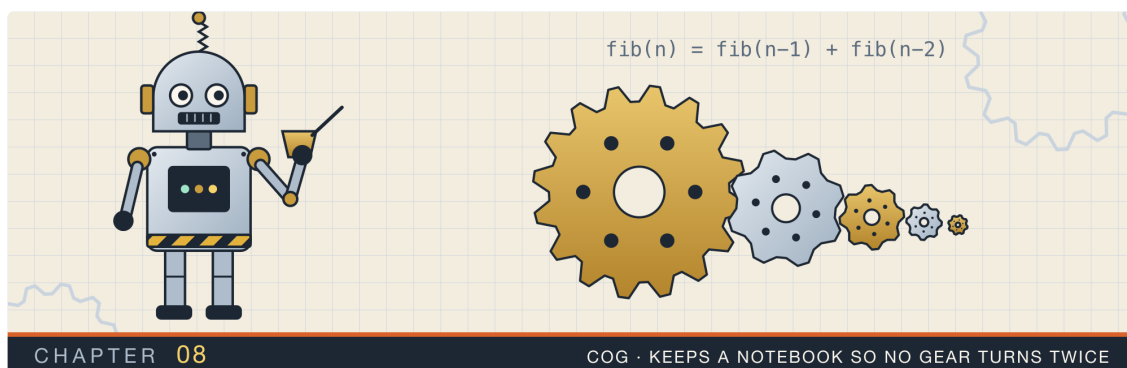
Summary

- A trait is a set of methods a type promises. You implement one with `impl Trait for Type`, and `#[derive]` generates the standard ones.
- A newtype with a private field and a checking constructor can only hold valid values, so the code that receives it needs no checks.
- An error type with `Display` and `std::error::Error` tells the caller what went wrong, and works with `?`.
- `From` converts when it cannot fail. `TryFrom` returns a `Result` when it can.
- An enum whose variants carry their own data replaces a struct full of `Option` fields, and is as large as its largest variant.
- A parser turns text into a typed value, and `map_err` turns low-level failures into your own errors.
- Matching on a pair of states describes a state machine, and a final `_` arm rejects every move not listed.
- Error enums can carry data and methods such as `is_retryable`.

Every value in this chapter had exactly one owner. Chapter 8 covers the pointer types for data that needs several owners, or that must be shared between threads.

Exercises

1. Write a `Port` newtype for network ports that rejects 0, with a `TryFrom<u16>` implementation and its own error type.
2. Implement the `FromStr` trait for `Command`, so that `"list".parse::<Command>()` works. Its method can call `Command::parse`.
3. Add a `Suspended` state to `WorkloadState` that can be entered from `Running` and left back to `Running`, and add tests for both moves.
4. Give `ApiError` a `Display` implementation that uses `user_message`, then implement `std::error::Error` for it.



Ownership and the pointer types

THIS CHAPTER COVERS

- Ownership and moves, drawn in memory
- Box : putting a value on the heap, and why recursive types need it
- Rc : one value with several owners, counted
- RefCell : changing a value that has several owners, checked while the program runs
- Weak : a pointer that does not keep its value alive, and the leak it prevents
- Arc and Mutex : sharing and changing a value across threads
- Cow : borrowing when possible and copying only when needed

In chapter 7 every value had exactly one owner. That is the normal case in Rust, and most code needs nothing more. But some structures cannot be described with one owner per value. A tree node wants to point back to its parent. A counter must be updated by several threads. An expression contains smaller expressions of its own type.

For each of these, the standard library has a **pointer type**. A pointer type holds the address of a value and adds a rule about who owns it. This chapter covers the main ones, one at a time. For each, I show where its data sits in memory and which problem it solves. The next four chapters, on linked lists, trees, and graphs, use every one of them.

I start by reviewing ownership itself, because each pointer type is a change to its rules.

8.1 Ownership and moves

Every value in Rust has one **owner**, the variable or structure that holds it. When the owner goes away, at the end of its scope, the value is dropped and its memory is freed.

Assigning a value to another variable **moves** it: ownership passes to the new variable, and the old one can no longer be used. Figure 8.1 shows a move of a `String`.

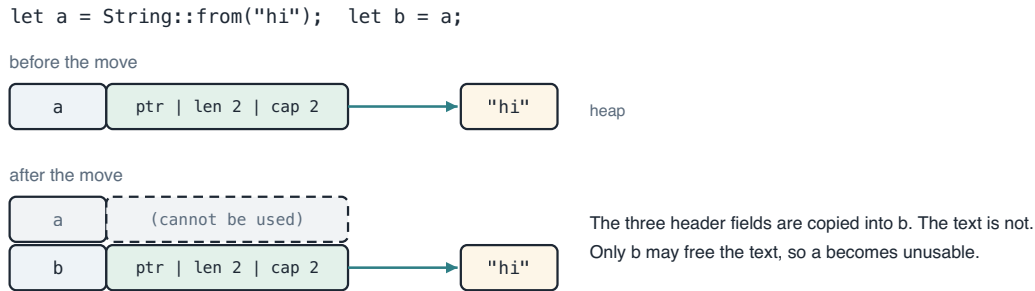


Figure 8.1 Moving a `String`. The small header is copied; the heap text is not. Only one variable may free the text, so the old one becomes unusable.

A move copies only the header on the stack, the pointer, length, and capacity. The text on the heap stays where it is. If both `a` and `b` were usable, both would try to free the same text when they went out of scope. The compiler prevents that by making `a` unusable after the move.

Instead of moving a value, you can **borrow** it with a reference. A shared reference `&T` lets you read. A mutable reference `&mut T` lets you change. The borrow rules say you may have either any number of `&T` or exactly one `&mut T`, but not both at once. And no reference may outlive its value.

These rules cover most programs. The pointer types in this chapter handle the cases they cannot.

The module for this chapter starts with a table that summarizes them:

Listing 8.1 The module comment and imports (lines 1 to 25). `src/problems/smart_pointers.rs`

```
///! Smart pointers and interior mutability.
///!
///! Each pointer type answers a different ownership question, and picking the
///! wrong one shows up as either a compile error or a runtime bug.
///!
///! | Type | Ownership | Mutation | Across threads |
```



```

///! |---|---|---|---|
///! | `Box<T>` | single | through `&mut` | if `T: Send` |
///! | `Rc<T>` | shared, non-atomic count | no | no |
///! | `Arc<T>` | shared, atomic count | no | if `T: Send + Sync` |
///! | `RefCell<T>` | single | runtime checked | no |
///! | `Mutex<T>` | single | blocking lock | yes |
///! | `Weak<T>` | non-owning | no | as `Arc` |
///! | `Cow<'a, T>` | borrow or own | no | as the borrow |
///!
///! Two rules cover most decisions. `Rc`/`RefCell` is for graphs and caches inside
///! one thread; `Arc`/`Mutex` is for state shared between threads. `Weak` exists to
///! break reference cycles, because two strong references in a cycle never reach a
///! count of zero and the memory is never freed.

use std::borrow::Cow;
use std::cell::RefCell;
use std::rc::{Rc, Weak};
use std::sync::{Arc, Mutex};
use std::thread;

```

Do not try to memorize the table yet. Each section below covers one row.

8.2 Box: one owner, on the heap

A `Box<T>` puts a value of type `T` on the heap and owns it. The `Box` itself is one pointer, 8 bytes on a 64-bit machine. When the `Box` is dropped, it drops the value and frees the heap memory. So a `Box` behaves like an owned value that happens to live on the heap.

You need a `Box` most often for a type that contains itself. Consider an arithmetic expression: it is either a number, or the sum of two smaller expressions. Written directly, that type would have no size (figure 8.2).

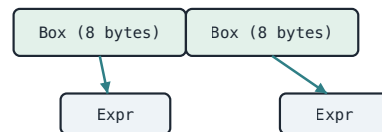
Why a recursive enum needs Box

```
enum Expr { Lit(i64), Add(Expr, Expr) }
```



an `Add` holds two `Expr`s, each of which may hold two more: no finite size

```
enum Expr { Lit(i64), Add(Box<Expr>, Box<Expr>) }
```



each child lives in its own heap block; an `Add` holds two pointers

Figure 8.2 Left: an `Add` that contains two `Expr` values directly would need infinite space. Right: with `Box`, an `Add` holds two pointers.

To lay out a type, the compiler must know its size. An `Add(Expr, Expr)` contains two `Expr` values, each of which may be another `Add`. The size has no end, so the compiler rejects the type. `Box<Expr>` is always 8 bytes, whatever it points to, so `Add(Box<Expr>, Box<Expr>)` has a fixed size.

Listing 8.2 An expression tree (lines 27 to 41).

```
/// A recursive type needs indirection, because `Expr` cannot contain itself by
/// value. `Box` provides the fixed-size indirection.
#[derive(Debug, PartialEq, Eq)]
pub enum Expr {
    Lit(i64),
    Add(Box<Expr>, Box<Expr>),
}

/// Evaluate an expression tree.
pub fn eval(expr: &Expr) -> i64 {
    match expr {
        Expr::Lit(value) => *value,
        Expr::Add(left, right) => eval(left) + eval(right),
    }
}
```

`eval` computes the value of an expression. For a number, the value is the number. For a sum, it evaluates both sides and adds them. The call `eval(left)` passes a `&Box<Expr>` where the function expects `&Expr`. Rust converts it automatically, following the box to the value inside. This automatic conversion is called **deref coercion**.

To build `1 + 2`, you write `Expr::Add(Box::new(Expr::Lit(1)), Box::new(Expr::Lit(2)))`. `Box::new` allocates on the heap and moves the value there.

8.3 `Rc`: several owners, counted

Sometimes one value really has several owners, and none of them can be picked as the one that frees it. A node in a graph with several incoming edges is an example. For this, Rust has `Rc<T>`, short for **reference counted**.

`Rc::new(value)` puts the value on the heap together with a count of how many `Rc` handles point to it. `Rc::clone(&rc)` does not copy the value. It adds 1 to the count and returns another handle to the same value. Dropping a handle subtracts 1. When the count reaches zero, the value is dropped. Figure 8.3 shows two handles.

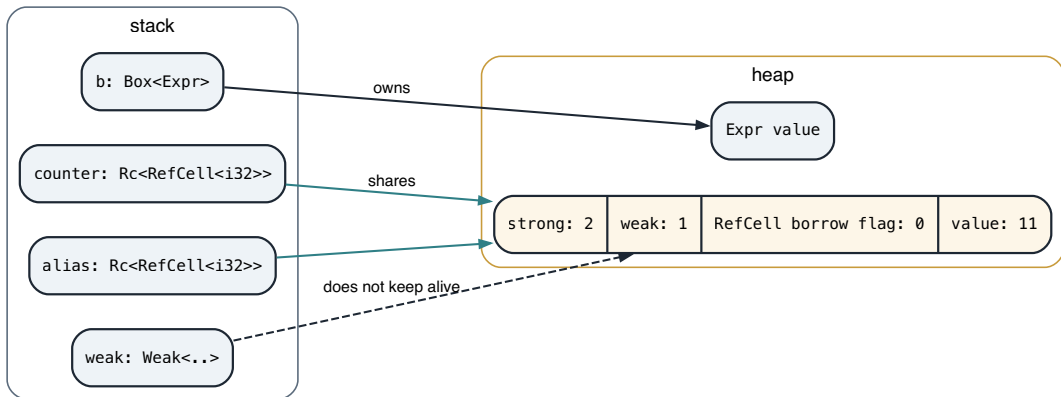


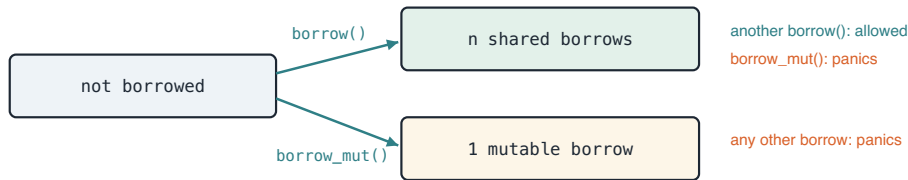
Figure 8.3 Two Rc handles share one heap block. The block holds a strong count (the number of Rc handles), a weak count (section 8.5), and the value.

An `Rc` hands out only shared references to its value. With several owners, a `&mut` through one of them would break the rule that a `&mut` is the only reference. So by itself, a value inside an `Rc` cannot be changed. The next section adds that ability.

NOTE: `Rc` updates its count with ordinary, non-atomic arithmetic. If two threads updated it at the same moment, the count could end up wrong. So the compiler does not let an `Rc` be sent to another thread. Section 8.6 covers `Arc`, the version for threads.

8.4 `RefCell`: changing a shared value, checked at run time

`RefCell<T>` holds a value and allows it to be changed through a shared reference. It keeps the borrow rules, but checks them while the program runs instead of at compile time. Figure 8.4 shows what it tracks.

What a RefCell tracks at run time

Each guard (Ref or RefMut) moves the cell back toward "not borrowed" when it is dropped.

Figure 8.4 The borrow states of a `RefCell`. Breaking the rules is not a compile error; it panics when it happens.

- `cell.borrow()` returns a guard that acts like `&T`. Many can exist at once.
- `cell.borrow_mut()` returns a guard that acts like `&mut T`. It panics if any other borrow is active.
- When a guard is dropped, its borrow ends.

`Rc` and `RefCell` are used together: `Rc<RefCell<T>>` gives several owners who can each change the value.

Listing 8.3 Two handles to one counter (lines 43 to 53).

```

/// Shared mutation inside one thread, with `Rc` for aliasing and `RefCell` for
/// the runtime borrow check.
pub fn shared_counter_with_rc_refcell() -> i32 {
    let counter = Rc::new(RefCell::new(0));
    let alias = Rc::clone(&counter);

    *alias.borrow_mut() += 1;
    *counter.borrow_mut() += 10;

    *counter.borrow()
}

```

Step through it:

1. `Rc::new(RefCell::new(0))` creates a counter holding 0, with a strong count of 1.
2. `Rc::clone(&counter)` makes a second handle, `alias`. The strong count is now 2. There is still one counter.
3. `*alias.borrow_mut() += 1;` borrows mutably through `alias` and adds 1. The guard is dropped at the end of the statement, so the borrow ends there.
4. `*counter.borrow_mut() += 10;` does the same through the other handle.

5. `*counter.borrow()` reads 11.

Each `borrow_mut` ends before the next starts, so no panic occurs. If you stored the first guard in a variable and then called `borrow_mut` again while it was still alive, the program would panic.

8.5 `Weak`: a pointer that does not own

Counting has a weakness. If two values hold `Rc` handles to each other, each keeps the other's count above zero. When you drop your own handles, both counts stay at 1, and neither value is ever freed. This is a **reference cycle**, and it leaks memory (figure 8.5, left).

Two strong pointers in a cycle never reach zero



Figure 8.5 Left: a cycle of strong pointers is never freed. Right: making the back-link `Weak` breaks the cycle.

The fix is a pointer that does not count as an owner. `Weak<T>` is made from an `Rc` with `Rc::downgrade`. It increases only the weak count, which does not keep the value alive. To use the value, you call `upgrade()`, which returns `Some(Rc)` if the value still exists and `None` if it has been dropped.

The usual pattern is that parents own children with `Rc`, and children point back to their parents with `Weak`.

Listing 8.4 A tree node with a weak parent pointer (lines 55 to 82).

```
/// A node whose parent link is weak, so a child does not keep its parent alive.
pub struct TreeNode {
    pub value: i32,
    parent: RefCell<Weak<TreeNode>>,
}

impl TreeNode {
    /// A node with no parent.
    pub fn root(value: i32) -> Rc<Self> {
```

```

        Rc::new(Self {
            value,
            parent: RefCell::new(Weak::new()),
        })
    }

    /// A node that points back weakly at `parent`.
    pub fn child_of(parent: &Rc<Self>, value: i32) -> Rc<Self> {
        Rc::new(Self {
            value,
            parent: RefCell::new(Rc::downgrade(parent)),
        })
    }

    /// The parent's value, or `None` once the parent has been dropped.
    pub fn parent_value(&self) -> Option<i32> {
        self.parent.borrow().upgrade().map(|parent| parent.value)
    }
}

```

- `parent: RefCell<Weak<TreeNode>>` stores the link to the parent. The `RefCell` allows the link to be changed after the node exists.
- `root` creates a node with `Weak::new()`, an empty weak pointer whose `upgrade` always returns `None`.
- `child_of` creates a node whose parent link is `Rc::downgrade(parent)`.
- `parent_value` borrows the link, calls `upgrade()`, and reads the parent's value if the parent still exists. `Option::map` applies the closure only when there is a value.

The test `weak_parent_link_does_not_keep_the_parent_alive` in listing 8.7 checks the behavior. It creates a root and a child, drops the root, and then checks that the child's `parent_value()` is `None`.

8.6 `Arc` and `Mutex`: sharing across threads

`Arc<T>` is `Rc<T>` with an atomic count. An **atomic** operation is one the processor completes as a single step. That stays true even when several cores run it on the same memory at the same moment. So `Arc` handles can be sent to other threads. Chapter 16 explains atomic operations in detail.

`Mutex<T>` is the thread-safe counterpart of `RefCell`. `lock()` returns a guard that gives access to the value. If another thread holds the lock, `lock()` waits until it is released, instead of panicking.

Listing 8.5 Several threads adding to one total (lines 84 to 105).

```

/// Shared mutation across threads, with `Arc` for ownership and `Mutex` for
/// exclusive access.
pub fn total_with_arc_mutex(threads: usize, per_thread: usize) -> usize {
    let total = Arc::new(Mutex::new(0usize));
    let mut handles = Vec::with_capacity(threads);

    for _ in 0..threads {
        let total = Arc::clone(&total);
        handles.push(thread::spawn(move || {
            for _ in 0..per_thread {
                let mut guard = total.lock().expect("mutex poisoned");
                *guard += 1;
            }
        }));
    }

    for handle in handles {
        handle.join().expect("worker panicked");
    }

    *total.lock().expect("mutex poisoned")
}

```

Step through it:

1. `Arc::new(Mutex::new(0usize))` creates the shared total.
2. For each thread, `Arc::clone(&total)` makes a new handle, and `thread::spawn(move || ...)` moves that handle into the new thread. `move` means the closure takes ownership of what it uses.
3. Each thread locks the mutex, adds 1, and drops the guard, `per_thread` times.
4. `handle.join()` waits for each thread to finish.
5. The last line locks once more and reads the result.

`lock()` returns a `Result`, because a thread that panics while holding the lock leaves the mutex marked as **poisoned**. `.expect("mutex poisoned")` stops the program with that message in that case. Chapter 16 covers poisoning.

This function locks once for every single increment, so the threads spend most of their time waiting for each other. It is correct but slow. Chapter 16 measures faster ways to count across threads.

8.7 Cow: borrow when you can

`Cow<'a, str>` holds either a borrowed `&'a str` or an owned `String`. Its name stands for "clone on write". It fits functions that usually return their input unchanged but sometimes need to change it.

Listing 8.6 Uppercase only when needed (lines 107 to 117).

```
/// Borrow when no change is needed, allocate only when there is one.
///
/// `Cow` is the idiomatic return type for a function that usually passes its
/// input through unchanged.
pub fn normalize(input: &str, uppercase: bool) -> Cow<'_, str> {
    if uppercase && input.bytes().any(|byte| byte.is_ascii_lowercase()) {
        Cow::Owned(input.to_uppercase())
    } else {
        Cow::Borrowed(input)
    }
}
```

`normalize` checks whether any byte is a lowercase letter. If there is nothing to change, it returns `Cow::Borrowed(input)`, which costs nothing. Only when a change is needed does it allocate a new uppercase `String` and return `Cow::Owned`. The caller can use either variant as a `&str`.

8.8 The complete file

Listing 8.7 The complete file, with its tests. `src/problems/smart_pointers.rs`

```
//! Smart pointers and interior mutability.
//!
//! Each pointer type answers a different ownership question, and picking the
//! wrong one shows up as either a compile error or a runtime bug.
//!
//! | Type | Ownership | Mutation | Across threads |
//! |---|---|---|---|
//! | `Box<T>` | single | through `&mut` | if `T: Send` |
//! | `Rc<T>` | shared, non-atomic count | no | no |
//! | `Arc<T>` | shared, atomic count | no | if `T: Send + Sync` |
//! | `RefCell<T>` | single | runtime checked | no |
//! | `Mutex<T>` | single | blocking lock | yes |
//! | `Weak<T>` | non-owning | no | as `Arc` |
//! | `Cow<'a, T>` | borrow or own | no | as the borrow |
//!
//! Two rules cover most decisions. `Rc`/`RefCell` is for graphs and caches inside
//! one thread; `Arc`/`Mutex` is for state shared between threads. `Weak` exists to
//! break reference cycles, because two strong references in a cycle never reach a
```



```

    /// count of zero and the memory is never freed.

    use std::borrow::Cow;
    use std::cell::RefCell;
    use std::rc::{Rc, Weak};
    use std::sync::{Arc, Mutex};
    use std::thread;

    /// A recursive type needs indirection, because `Expr` cannot contain itself by
    /// value. `Box` provides the fixed-size indirection.
    #[derive(Debug, PartialEq, Eq)]
    pub enum Expr {
        Lit(i64),
        Add(Box<Expr>, Box<Expr>),
    }

    /// Evaluate an expression tree.
    pub fn eval(expr: &Expr) -> i64 {
        match expr {
            Expr::Lit(value) => *value,
            Expr::Add(left, right) => eval(left) + eval(right),
        }
    }

    /// Shared mutation inside one thread, with `Rc` for aliasing and `RefCell` for
    /// the runtime borrow check.
    pub fn shared_counter_with_rc_refcell() -> i32 {
        let counter = Rc::new(RefCell::new(0));
        let alias = Rc::clone(&counter);

        *alias.borrow_mut() += 1;
        *counter.borrow_mut() += 10;

        *counter.borrow()
    }

    /// A node whose parent link is weak, so a child does not keep its parent alive.
    pub struct TreeNode {
        pub value: i32,
        parent: RefCell<Weak<TreeNode>>,
    }

    impl TreeNode {
        /// A node with no parent.
        pub fn root(value: i32) -> Rc<Self> {
            Rc::new(Self {
                value,
                parent: RefCell::new(Weak::new()),
            })
        }

        /// A node that points back weakly at `parent`.
        pub fn child_of(parent: &Rc<Self>, value: i32) -> Rc<Self> {
            Rc::new(Self {
                value,
                parent: RefCell::new(Rc::downgrade(parent)),
            })
        }
    }

```

```

    /// The parent's value, or `None` once the parent has been dropped.
    pub fn parent_value(&self) -> Option<i32> {
        self.parent.borrow().upgrade().map(|parent| parent.value)
    }
}

/// Shared mutation across threads, with `Arc` for ownership and `Mutex` for
/// exclusive access.
pub fn total_with_arc_mutex(threads: usize, per_thread: usize) -> usize {
    let total = Arc::new(Mutex::new(0usize));
    let mut handles = Vec::with_capacity(threads);

    for _ in 0..threads {
        let total = Arc::clone(&total);
        handles.push(thread::spawn(move || {
            for _ in 0..per_thread {
                let mut guard = total.lock().expect("mutex poisoned");
                *guard += 1;
            }
        }));
    }

    for handle in handles {
        handle.join().expect("worker panicked");
    }

    *total.lock().expect("mutex poisoned")
}

/// Borrow when no change is needed, allocate only when there is one.
///
/// `Cow` is the idiomatic return type for a function that usually passes its
/// input through unchanged.
pub fn normalize(input: &str, uppercase: bool) -> Cow<'_, str> {
    if uppercase && input.bytes().any(|byte| byte.is_ascii_lowercase()) {
        Cow::Owned(input.to_uppercase())
    } else {
        Cow::Borrowed(input)
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn boxed_expression_evaluates() {
        let expr = Expr::Add(Box::new(Expr::Lit(1)), Box::new(Expr::Lit(2)));
        assert_eq!(eval(&expr), 3);
    }

    #[test]
    fn rc_refcell_shares_mutation_within_a_thread() {
        assert_eq!(shared_counter_with_rc_refcell(), 11);
    }

    #[test]
    fn weak_parent_link_does_not_keep_the_parent_alive() {
        let root = TreeNode::root(1);

```

```

    let child = TreeNode::child_of(&root, 2);

    assert_eq!(child.parent_value(), Some(1));
    drop(root);
    assert_eq!(child.parent_value(), None);
}

#[test]
fn arc_mutex_sums_concurrent_updates() {
    assert_eq!(total_with_arc_mutex(8, 250), 2_000);
}

#[test]
fn cow_borrows_and_owns_as_needed() {
    assert!(matches!(normalize("A100", true), Cow::Borrowed(_)));
    assert!(matches!(normalize("a100", true), Cow::Owned(_)));
    assert_eq!(normalize("a100", true), "A100");
    assert_eq!(normalize("a100", false), "a100");
}
}

```

The tests check one behavior per pointer type. `cow_borrows_and_owns_as_needed` uses `matches!` to check which variant came back, not only the text. That is how a test shows that the unchanged case did not allocate. Run them with `cargo test --lib smart_pointers`.

Figure 8.6 turns the chapter into three questions you can ask when choosing a pointer type.

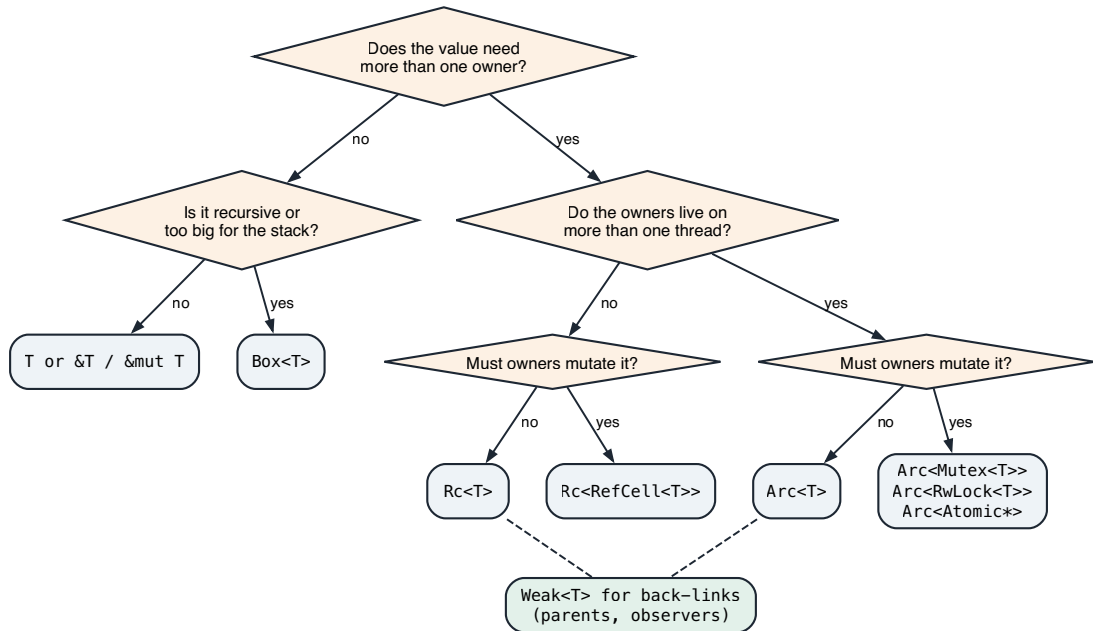


Figure 8.6 Choosing a pointer type: owners first, then threads, then whether the owners change the value.

Summary

- Every value has one owner. Assignment moves ownership, copying only the stack header, and borrowing lends access without moving.
- `Box<T>` owns a value on the heap. It gives recursive types a fixed size.
- `Rc<T>` lets several owners share a value, counted, within one thread. Cloning an `Rc` adds to the count.
- `RefCell<T>` allows changes through shared references and checks the borrow rules at run time, panicking on a violation.
- Two `Rc` handles pointing at each other leak. `Weak<T>` does not count as an owner and breaks such cycles.
- `Arc<T>` is `Rc` with an atomic count, for threads, and `Mutex<T>` makes other threads wait instead of panicking.
- `Cow` returns borrowed data unchanged and allocates only when it must change it.

Chapter 9 puts these types to work. You will build linked lists with `Box` and see how dropping a very long one can crash. Then you will build a doubly linked list with `Rc`, `RefCell`, and `Weak`.

Exercises

1. Build the expression `(1 + 2) + 3` with `Expr` and check that `eval` returns 6.
2. In `shared_counter_with_rc_refcell`, keep the first `borrow_mut` guard in a variable and call `borrow_mut` again. Run it and read the panic message.
3. Build a parent with two children that point back with `Weak`. Print `Rc::strong_count` and `Rc::weak_count` for the parent before and after creating the children.
4. Change `total_with_arc_mutex` so each thread counts in a local variable and adds its total to the mutex once. Compare the running time for 8 threads and one million increments each.



Linked lists

THIS CHAPTER COVERS

- What a linked list is, and how it differs from a `Vec` in memory
- Moving ownership of nodes with `Option::take` and `std::mem::replace`
- Reversing a list in place
- Four ways to write a list node, measured against each other
- Why dropping a long list can crash the program, and how to prevent it
- A doubly linked list built from `Rc`, `RefCell`, and `Weak`

Chapter 8 introduced the pointer types. This chapter uses them to build the classic linked structure, the linked list. Linked lists are a good first test of ownership. Each operation moves a node from one owner to another, and the compiler checks every move.

The chapter goes from simple to hard. First a singly linked list, where each node points only to the next, and an algorithm that reverses it. Then four ways to write the same list, and a benchmark that compares them. The benchmark finds a size at which the list crashes the program when it is dropped. After explaining and fixing that, the chapter ends with a doubly linked list, where each node also points back to the one before.

9.1 What a linked list is

A **linked list** stores a sequence of values in separate nodes. Each **node** holds one value and a pointer to the next node. The list itself holds a pointer to the first node, called the **head**. The last node's pointer is empty, which marks the end.

Figure 9.1 compares a linked list with a `Vec` holding the same three values.

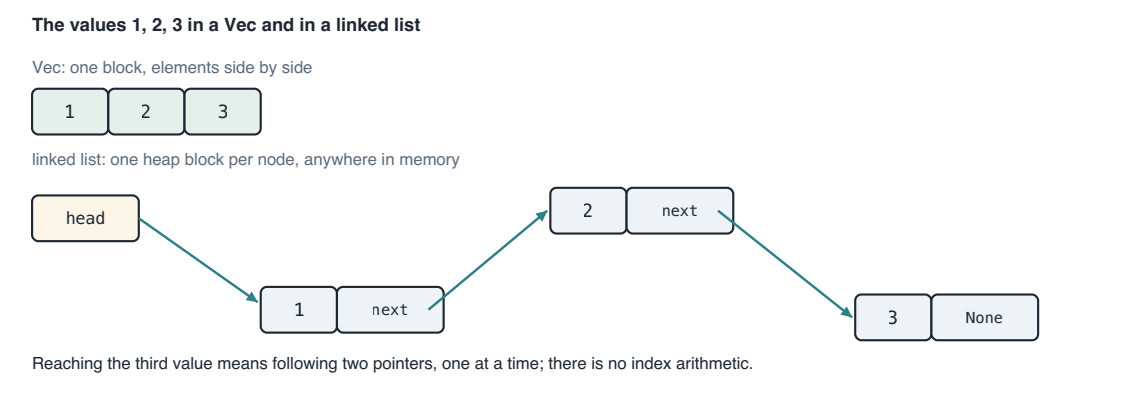


Figure 9.1 A `Vec` stores its values next to each other. A linked list stores each value in its own heap block and connects the blocks with pointers.

The two structures are good at different things:

Operation	Vec	Singly linked list
Read the value at position i	$O(1)$: compute the address	$O(i)$: follow i pointers
Add or remove at the front	$O(n)$: shift every element	$O(1)$: change the head pointer
Add at the end	amortized $O(1)$	$O(n)$, unless the list keeps a tail pointer
Memory per value	the value	the value, a pointer, and a heap block's overhead

In practice, a `Vec` is faster for most work. Its values are next to each other in memory, and the processor reads nearby memory quickly; chapter 15 measures that effect. The techniques you learn here for moving nodes between owners carry over to the trees and graphs of chapters 11 and 12.

In Rust, a node owns the next node through a `Box`, and the "empty" pointer at the end is `None`. So a node's `next` field has type `Option<Box<Node>>`.

9.2 Moving nodes: `take` and `replace`

Every list operation moves a node from one owner to another. Rust has a rule that makes this tricky. You cannot move a value out of a place that you only borrowed. That would leave the place empty, and the owner would later find garbage there.

Two standard functions solve this by putting something valid in the place as they take the value out:

- `option.take()` returns the value inside an `Option` and leaves `None` in its place.
- `std::mem::replace(&mut place, new_value)` returns the old value and puts `new_value` in its place.

Figure 9.2 shows `take` on a node's `next` field.

Option::take moves the value out and leaves None behind

before: `let rest = node.next.take();`



after:



The node stays valid (it holds `None`), and the variable `rest` now owns the rest of the list.

Figure 9.2 `node.next.take()` moves the rest of the list out of the node into a variable, and leaves `None` behind.

You will see these two calls in almost every listing of this chapter.

9.3 Reversing a list

The problem. Given the list $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$, turn it into $4 \rightarrow 3 \rightarrow 2 \rightarrow 1$. Change the pointers; do not copy the values.

The idea. Walk the list from the front. Keep two variables: `previous`, the part already reversed, and `current`, the part not yet visited. At each step, take the first node of `current`,

point it at `previous`, and make it the new front of `previous`. Figure 9.3 shows the state after two steps.

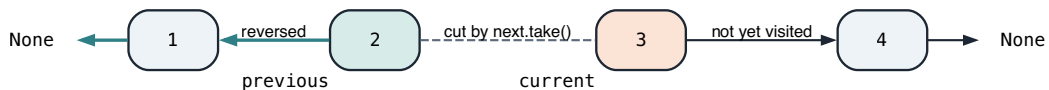
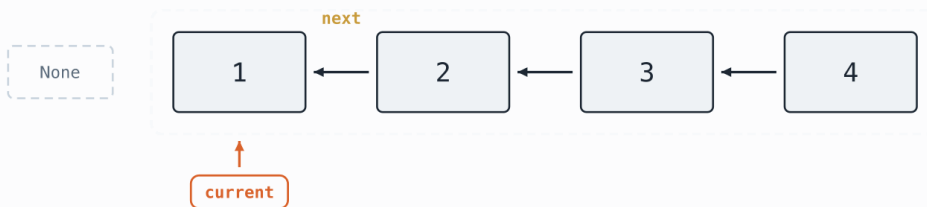


Figure 9.3 Halfway through. `previous` owns the reversed part ($2 \rightarrow 1$), and `current` owns the rest ($3 \rightarrow 4$).

Reversing a singly linked list with three bindings

`previous` and `current` walk the list. `next` holds the rest so the link is safe to cut.

☒ points back ☐ not yet reversed



Start. `previous` is `None` and `current` is the head, node 1. Save `next = 2` before the link is cut.

Teal arrows point backwards, into the part already reversed.

`previous = None`

`current = 1`

`reversed = 0`

Animation 9.3 The loop, one node per frame. Each frame saves `next`, turns the node's link around, then advances `previous` and `current`. The teal links are the ones already reversed. No node is copied and no node is allocated; the list is rearranged in place, so the extra memory is three bindings.

First, the node type and two helpers for building and reading lists:

Listing 9.1 The node and two helpers (lines 1 to 33). `src/problems/linked_list.rs`

```

/// Reverse a singly linked list.
///
/// Uses `Option<Box<ListNode>>` and a `while let` loop to move pointers in a

```

```

    //! borrow-safe way.

    #[derive(Debug, PartialEq, Eq)]
    pub struct ListNode {
        pub val: i32,
        pub next: Option<Box<ListNode>>,
    }

    impl ListNode {
        pub fn from_slice(values: &[i32]) -> Option<Box<ListNode>> {
            let mut head = None;
            for &value in values.iter().rev() {
                head = Some(Box::new(ListNode {
                    val: value,
                    next: head,
                }));
            }
            head
        }

        pub fn to_vec(head: Option<Box<ListNode>>) -> Vec<i32> {
            let mut result = Vec::new();
            let mut current = head;
            while let Some(node) = current {
                result.push(node.val);
                current = node.next;
            }
            result
        }
    }
}

```

`from_slice` builds a list from a slice. It walks the slice backward, putting each value in front of the list built so far. The last value pushed is `values[0]`, so it ends up first. Building from the back means it never needs a pointer to the end of the list.

`to_vec` reads a list into a `Vec`. It takes the list by value and consumes it. In `while let Some(node) = current`, each pass moves one box out of `current`. `current = node.next` then moves the rest of the list out of that node before the node is freed.

Now the reversal:

Listing 9.2 Reversing the list (lines 35 to 46).

```

pub fn reverse_list(head: Option<Box<ListNode>>) -> Option<Box<ListNode>> {
    let mut previous = None;
    let mut current = head;

    while let Some(mut node) = current {
        current = node.next.take();
        node.next = previous;
        previous = Some(node);
    }
}

```

```
    previous
}
```

Each pass through the loop does three moves:

1. `current = node.next.take();` takes the rest of the list out of the node, as in figure 9.2.
2. `node.next = previous;` points the node at the reversed part.
3. `previous = Some(node);` makes the node the front of the reversed part.

Trace it on $1 \rightarrow 2 \rightarrow 3$:

After pass	previous	current
start	None	$1 \rightarrow 2 \rightarrow 3$
1	1	$2 \rightarrow 3$
2	$2 \rightarrow 1$	3
3	$3 \rightarrow 2 \rightarrow 1$	None

At no moment do two variables own the same node, which is why the compiler accepts the code. The function visits each node once, so it is $O(n)$, and it uses $O(1)$ extra memory.

Listing 9.3 The complete file. `src/problems/linked_list.rs`

```
/// Reverse a singly linked list.
///
/// Uses `Option<Box<ListNode>>` and a `while let` loop to move pointers in a
/// borrow-safe way.

#[derive(Debug, PartialEq, Eq)]
pub struct ListNode {
    pub val: i32,
    pub next: Option<Box<ListNode>>,
}

impl ListNode {
    pub fn from_slice(values: &[i32]) -> Option<Box<ListNode>> {
        let mut head = None;
        for &value in values.iter().rev() {
            head = Some(Box::new(ListNode {
                val: value,
                next: head,
            }));
        }
        head
    }

    pub fn to_vec(head: Option<Box<ListNode>>) -> Vec<i32> {
```

```

        let mut result = Vec::new();
        let mut current = head;
        while let Some(node) = current {
            result.push(node.val);
            current = node.next;
        }
        result
    }
}

pub fn reverse_list(head: Option<Box<ListNode>>) -> Option<Box<ListNode>> {
    let mut previous = None;
    let mut current = head;

    while let Some(mut node) = current {
        current = node.next.take();
        node.next = previous;
        previous = Some(node);
    }

    previous
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn reverses_list() {
        let head = ListNode::from_slice(&[1, 2, 3, 4]);
        assert_eq!(ListNode::to_vec(reverse_list(head)), vec![4, 3, 2, 1]);
    }

    #[test]
    fn handles_empty_and_single() {
        assert_eq!(reverse_list(None), None);
        assert_eq!(
            ListNode::to_vec(reverse_list(ListNode::from_slice(&[42]))),
            vec![42]
        );
    }
}

```

The tests reverse a four-element list, an empty list, and a one-element list.

9.4 A list written as an enum

The first list separated the node type from the `Option` that marks the end. Another way puts both into one enum: a list is either empty, or a node holding a value and the rest of the list.

Listing 9.4 The list type (lines 1 to 13). src/bin/singly_linked_list.rs

```
// Singly linked list using an algebraic data type.
//
//     List<T> = Empty
//             | Node { value: T, next: Link<T> }
//
// `Empty` is the end of the list.
type Link<T> = Box<List<T>>;

pub enum List<T> {
    Empty,
    Node { value: T, next: Link<T> },
}
```

`List<T>` is **generic**. The `T` is a placeholder for the type of the values, so the same code works for a list of numbers or of strings. `type Link<T> = Box<List<T>>;` gives the boxed rest of the list a short name.

Listing 9.5 Creating, checking, and measuring (lines 16 to 29).

```
pub fn new() -> Self {
    List::Empty
}

pub fn is_empty(&self) -> bool {
    matches!(self, List::Empty)
}

pub fn len(&self) -> usize {
    match self {
        List::Empty => 0,
        List::Node { next, .. } => 1 + next.len(),
    }
}
```

`len` counts the nodes recursively: an empty list has length 0, and a node adds 1 to the length of the rest. Each call waits for the next, so a list of `n` nodes uses `n` stack frames. Section 9.6 shows where that ends.

Listing 9.6 Adding and removing at the front (lines 31 to 49).

```
/// O(1)
pub fn push_front(&mut self, value: T) {
    let old = std::mem::replace(self, List::Empty);
    *self = List::Node {
        value,
        next: Link::new(old),
    };
}
```

```

    /// 0(1)
    pub fn pop_front(&mut self) -> Option<T> {
        match std::mem::replace(self, List::Empty) {
            List::Empty => None,
            List::Node { value, next } => {
                *self = *next;
                Some(value)
            }
        }
    }
}

```

Both methods take `&mut self`, so they change the list in place. They use `std::mem::replace` from section 9.2. The method cannot move the old list out of `*self` and assign a new one later. Between those two steps, `*self` would be empty. `replace(self, List::Empty)` puts a temporary `Empty` in place and hands back the old list in one step.

`push_front` wraps the old list in a new node with the new value, and writes it back into `*self`.

`pop_front` matches on the old list. If it was `Empty`, there is nothing to pop, and `*self` is already `Empty` again. If it was a `Node`, the method writes the rest of the list back into `*self` with `*self = *next`, and returns the value. `*next` moves the list out of its `Box`.

Listing 9.7 The complete program. `src/bin/singly_linked_list.rs`

```

// Singly linked list using an algebraic data type.
//
//   List<T> = Empty
//           | Node { value: T, next: Link<T> }
//
// `Empty` is the end of the list.

type Link<T> = Box<List<T>>;

pub enum List<T> {
    Empty,
    Node { value: T, next: Link<T> },
}

impl<T> List<T> {
    pub fn new() -> Self {
        List::Empty
    }

    pub fn is_empty(&self) -> bool {
        matches!(self, List::Empty)
    }

    pub fn len(&self) -> usize {
        match self {
            List::Empty => 0,

```

```

        List::Node { next, .. } => 1 + next.len(),
    }
}

/// O(1)
pub fn push_front(&mut self, value: T) {
    let old = std::mem::replace(self, List::Empty);
    *self = List::Node {
        value,
        next: Link::new(old),
    };
}

/// O(1)
pub fn pop_front(&mut self) -> Option<T> {
    match std::mem::replace(self, List::Empty) {
        List::Empty => None,
        List::Node { value, next } => {
            *self = *next;
            Some(value)
        }
    }
}

}

fn main() {
    let mut list = List::new();
    list.push_front(2);
    list.push_front(1);
    list.push_front(0);

    assert_eq!(list.len(), 3);
    assert_eq!(list.pop_front(), Some(0));
    assert_eq!(list.pop_front(), Some(1));
    assert_eq!(list.pop_front(), Some(2));
    println!("ok");
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn push_pop_works() {
        let mut list = List::new();
        list.push_front(1);
        list.push_front(2);
        assert_eq!(list.pop_front(), Some(2));
        assert_eq!(list.pop_front(), Some(1));
        assert_eq!(list.pop_front(), None);
    }

    #[test]
    fn len_works() {
        let mut list = List::new();
        assert_eq!(list.len(), 0);
        list.push_front(1);
        list.push_front(2);
    }
}

```

```

    assert_eq!(list.len(), 2);
  }
}

```

`main` pushes three values and pops them back in reverse order, the last one pushed coming out first. A list that adds and removes at the same end behaves as a stack, like the `Vec` in chapter 3.

```

$ cargo run --bin singly_linked_list
ok

```

9.5 Four node layouts, measured

Section 9.3 wrote a node as a struct with an `Option<Box<Node>>`, and section 9.4 wrote the list as an enum. Which is faster? To answer, the benchmark in this section puts four versions of the same list behind one `trait`. Then one piece of code can measure them all.

9.5.1 One trait for four lists

Listing 9.8 The trait every list implements. `benchmarking_examples/lists/mod.rs`

```

///! Singly linked lists in four variants, so one benchmark loop can cover all of
///! them.
///!
///! The variants differ only in how a node is stored and in how the list frees
///! itself. `boxed` and `enum_node` free a chain of boxes recursively, which
///! overflows the stack past roughly 265,000 nodes. `boxed_drop` and `enum_drop`
///! unlink iteratively and scale.

pub mod boxed;
pub mod boxed_drop;
pub mod enum_drop;
pub mod enum_node;

/// The operations every variant supports, so the benchmark can be written once.
pub trait SinglyList<T>: Sized {
    /// A new, empty list.
    fn new() -> Self;

    /// Adds `value` to the front of the list.
    fn push_front(&mut self, value: T);

    /// Removes and returns the front value, if there is one.
    fn pop_front(&mut self) -> Option<T>;

    /// Whether the list holds nothing.
    fn is_empty(&self) -> bool;

```



```

    /// The variant's name, for benchmark output and error messages.
    fn variant() -> &'static str;
}

```

The trait lists the operations: `new`, `push_front`, `pop_front`, `is_empty`, and `variant`, which returns a name for printing. `SinglyList<T>`: `Sized` limits the trait to types whose size is known at compile time. The benchmark uses the trait only through generic functions, so the limit costs nothing. The four `pub mod` lines declare the four versions, each in its own file.

The first two versions are the struct form and the enum form:

Listing 9.9 `Option<Box<Node<T>>>. benchmarking_examples/lists/boxed.rs`

```

///! `Option<Box<Node<T>>>`: one heap allocation per node.
///!
///! Pushing takes the old head and boxes it inside the new node. Freeing the head
///! frees its `next`, which frees its `next`, so the drop recurses once per
///! element and overflows the stack on a long list.

use super::SinglyList;

struct Node<T> {
    value: T,
    next: Option<Box<Node<T>>>,
}

pub struct LinkedList<T> {
    head: Option<Box<Node<T>>>,
}

impl<T> SinglyList<T> for LinkedList<T> {
    fn new() -> Self {
        Self { head: None }
    }

    fn push_front(&mut self, value: T) {
        self.head = Some(Box::new(Node {
            value,
            // .take() leaves None in its place and extracts the old head
            next: self.head.take(),
        }));
    }

    fn pop_front(&mut self) -> Option<T> {
        self.head.take().map(|node| {
            // Point the list head to the next node in line
            self.head = node.next;
            // Return the unboxed value
            node.value
        })
    }

    fn is_empty(&self) -> bool {
        self.head.is_none()
    }
}

```

```

    }

    fn variant() -> &'static str {
        "box"
    }
}

```

Listing 9.10 `enum ListNode<T> { Empty, Next(T, Box<ListNode<T>>)} .benchmarking_examples/`
`lists/enum_node.rs`

```

//! `enum ListNode<T> { Empty, Next(T, Box<ListNode<T>>)}`.
//!
//! Same shape as the boxed variant, written with an enum instead of an
//! `Option`. The enum cannot contain itself by value, so the recursive arm holds
//! a `Box`. The drop is recursive for the same reason.

use super::SinglyList;

enum ListNode<T> {
    Empty,
    Next(T, Box<ListNode<T>>),
}

pub struct LinkedList<T> {
    head: ListNode<T>,
}

impl<T> SinglyList<T> for LinkedList<T> {
    fn new() -> Self {
        Self {
            head: ListNode::Empty,
        }
    }

    fn push_front(&mut self, value: T) {
        // Extract the old head and leave Empty in its place
        let old_head = std::mem::replace(&mut self.head, ListNode::Empty);

        // Wrap the old head inside a Box behind the new value
        self.head = ListNode::Next(value, Box::new(old_head));
    }

    fn pop_front(&mut self) -> Option<T> {
        let old_head = std::mem::replace(&mut self.head, ListNode::Empty);

        match old_head {
            ListNode::Empty => None,
            ListNode::Next(value, next_node) => {
                // Point the list head to the next node in the sequence
                self.head = *next_node;
                Some(value)
            }
        }
    }

    fn is_empty(&self) -> bool {
        matches!(self.head, ListNode::Empty)
    }
}

```

```

    }

    fn variant() -> &'static str {
        "enum"
    }
}

```

The two files implement the trait with `impl<T> SinglyList<T> for LinkedList<T>`. Each uses the tool from section 9.2 that fits its type: `take()` on the `Option` in the struct version, and `mem::replace` on the enum.

9.5.2 What each node costs in memory

Figure 9.4 draws both lists after pushing 10, 20, and 30.

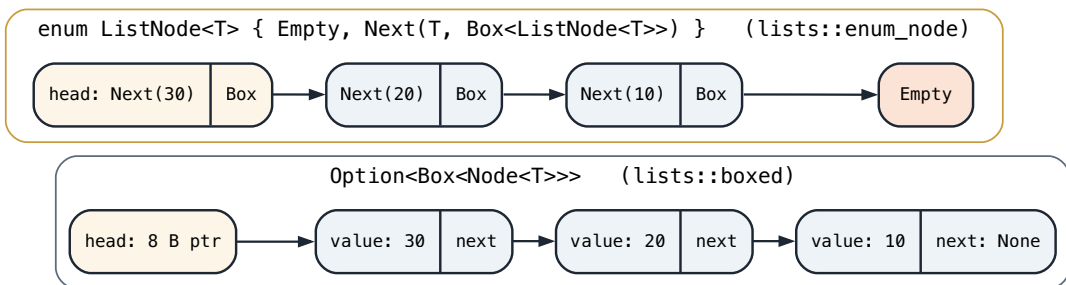


Figure 9.4 The two layouts after three pushes. The struct list's head is one pointer. The enum list keeps its first node inside the list value itself and ends with a heap-allocated `Empty`.

On a 64-bit machine, with `u64` values, each node is 16 bytes in both versions: 8 for the value and 8 for the pointer. `Option<Box<Node>>` is only 8 bytes, because a `Box` is never null, so Rust uses the null address to mean `None`. The enum gets the same treatment, using a null `Box` to mean `Empty`.

The versions still differ in two ways. The enum list keeps its first node, 16 bytes, inside the list value. So every push moves those 16 bytes into a new box, where the struct version moves one 8-byte pointer. And the enum chain ends with a boxed `Empty`, so a list of `n` values makes `n + 1` allocations instead of `n`.

Two small programs exercise the two versions:

Listing 9.11 Demos of the two versions. `benchmarking_examples/list_box.rs` and `list_enum.rs`

```

//! Demo of the `Option<Box<Node<T>>>` list in `lists::boxed`.
//!
//! Run with: cargo run --bin list_box

use systems_lab::lists::SinglyList;
use systems_lab::lists::boxed::LinkedList;

fn main() {
    let mut list = LinkedList::new();

    list.push_front(10);
    list.push_front(20);
    list.push_front(30);

    assert_eq!(list.pop_front(), Some(30));
    assert_eq!(list.pop_front(), Some(20));
    assert_eq!(list.pop_front(), Some(10));
    assert_eq!(list.pop_front(), None);
    assert!(list.is_empty());

    println!("All tests passed successfully!");
}

```

```

//! Demo of the `enum ListNode<T>` list in `lists::enum_node`.
//!
//! Run with: cargo run --bin list_enum

use systems_lab::lists::SinglyList;
use systems_lab::lists::enum_node::LinkedList;

fn main() {
    let mut list = LinkedList::new();
    list.push_front(42);
    list.push_front(100);

    assert_eq!(list.pop_front(), Some(100));
    assert_eq!(list.pop_front(), Some(42));
    assert_eq!(list.pop_front(), None);
    assert!(list.is_empty());
}

```

`use systems_lab::lists::...` imports the lists from the crate's library, where `src/lib.rs` declares them. So the four programs share one compiled copy of the list code.

9.6 When dropping a list crashes

Here is a short program built on the struct version:

```
let mut list = boxed::LinkedList::new();
for value in 0..270_000u64 {
    list.push_front(value);
}
// list goes out of scope here and is dropped
```

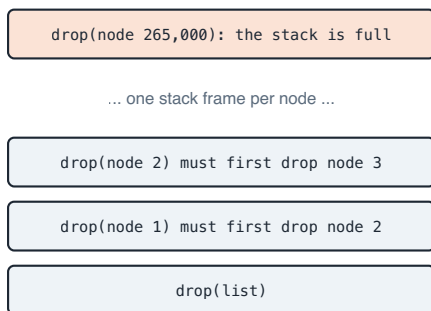
It builds the list without trouble, and then it crashes at the closing brace:

```
thread 'main' has overflowed its stack
fatal runtime error: stack overflow, aborting
```

None of the list's methods is recursive. The recursion is in the code the compiler generates to drop the list. To drop the list, Rust drops `head`. To drop a `Box<Node>`, it drops the node, which means dropping its fields, including `next`, another `Box<Node>`. That drops the next node, and so on. Each step is a function call nested inside the previous one, so each node adds one stack frame (figure 9.5, left).

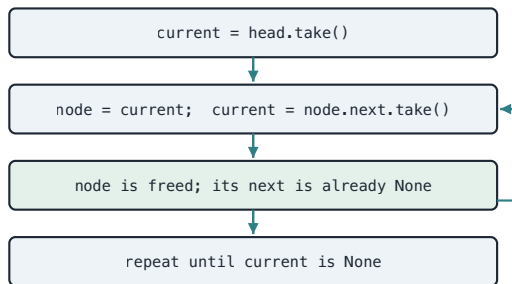
Dropping a list of 270,000 boxed nodes

Default drop: each node frees the next one first



The frames pile up until the thread's stack runs out.

Iterative Drop: detach, then free



One loop in one frame, however long the list is.

Figure 9.5 The default drop nests one call per node. A hand-written `Drop` detaches each node's successor first, so freeing one node never frees another.

The main thread's stack is 8 MiB on most systems. The crash happened between 260,000 and 270,000 nodes, so each level of nesting used about 32 bytes of stack. A thread you start with `std::thread::spawn` gets a 2 MiB stack by default. On such a thread the limit would be about a quarter as many nodes.

The fix is to write the list's `Drop` yourself, as a loop. `Drop` is a trait. Rust calls its `drop` method when a value goes out of scope. If you implement it, your code runs before the fields are dropped.

Listing 9.12 The struct version with its own `Drop`. `benchmarking_examples/lists/boxed_drop.rs`

```

//! `Option<Box<Node<T>>>` with an iterative `Drop`.
//!
//! Push and pop are identical to [`boxed`](super::boxed); only freeing changes.
//! Dropping a node would otherwise drop its `next` and recurse once per element,
//! so the list is unlinked in a loop before any node is freed.

use super::SinglyList;

struct Node<T> {
    value: T,
    next: Option<Box<Node<T>>>,
}

pub struct LinkedList<T> {
    head: Option<Box<Node<T>>>,
}

impl<T> SinglyList<T> for LinkedList<T> {
    fn new() -> Self {
        Self { head: None }
    }

    fn push_front(&mut self, value: T) {
        self.head = Some(Box::new(Node {
            value,
            next: self.head.take(),
        }));
    }

    fn pop_front(&mut self) -> Option<T> {
        self.head.take().map(|node| {
            self.head = node.next;
            node.value
        })
    }

    fn is_empty(&self) -> bool {
        self.head.is_none()
    }

    fn variant() -> &'static str {
        "box+drop"
    }
}

impl<T> Drop for LinkedList<T> {
    fn drop(&mut self) {
        // Walk the chain and unlink each node before it is freed, so no node's
        // drop has to carry the rest of the list on the stack.
        let mut current = self.head.take();

```

```

        while let Some(mut node) = current {
            current = node.next.take();
        }
    }
}

```

The operations are the same as listing 9.9. Only the `Drop` implementation at the end is new. Its loop reads:

1. `let mut current = self.head.take();` takes the whole chain out of the list.
2. `while let Some(mut node) = current` moves the first box out of `current`.
3. `current = node.next.take();` moves the rest of the chain out of that node.
4. At the end of the loop body, `node` goes out of scope and is freed. Its `next` is already `None`, so freeing it frees nothing else.

The loop runs in one stack frame, whatever the length of the list.

Listing 9.13 The enum version with its own `Drop`. [benchmarking_examples/lists/enum_drop.rs](#)

```

///! `enum ListNode<T>` with an iterative `Drop`.
///!
///! Push and pop are identical to [`enum_node`](super::enum_node); only freeing
///! changes. The recursive arm holds a `Box`, so the default drop walks the chain
///! on the stack. This one replaces the head with `Empty`, then takes the box out
///! of each node in turn.

use super::SinglyList;

enum ListNode<T> {
    Empty,
    Next(T, Box<ListNode<T>>),
}

pub struct LinkedList<T> {
    head: ListNode<T>,
}

impl<T> SinglyList<T> for LinkedList<T> {
    fn new() -> Self {
        Self {
            head: ListNode::Empty,
        }
    }

    fn push_front(&mut self, value: T) {
        let old_head = std::mem::replace(&mut self.head, ListNode::Empty);
        self.head = ListNode::Next(value, Box::new(old_head));
    }

    fn pop_front(&mut self) -> Option<T> {
        let old_head = std::mem::replace(&mut self.head, ListNode::Empty);
    }
}

```

```

        match old_head {
            ListNode::Empty => None,
            ListNode::Next(value, next_node) => {
                self.head = *next_node;
                Some(value)
            }
        }
    }

    fn is_empty(&self) -> bool {
        matches!(self.head, ListNode::Empty)
    }

    fn variant() -> &'static str {
        "enum+drop"
    }
}

impl<T> Drop for LinkedList<T> {
    fn drop(&mut self) {
        // Take the chain out of the list first, then step it forward one node at
        // a time. Each node has already lost its `next` by the time it is freed,
        // so the drop stays on the heap.
        let mut current = std::mem::replace(&mut self.head, ListNode::Empty);
        while let ListNode::Next(_, next) = current {
            current = *next;
        }
    }
}

```

The enum version does the same with `mem::replace` and a `while let ListNode::Next(_, next) = current` loop. `current = *next;` moves the rest of the chain out of the box. Assigning to `current` drops its old value, a `Next` whose box has already been emptied, so again nothing nests.

A third demo builds a list of five million nodes and drops it:

Listing 9.14 Five million nodes, dropped with a loop. `benchmarking_examples/list_drop.rs`

```

    /// Demo of the iterative `Drop` in `lists::boxed_drop`.
    ///
    /// The same program against `lists::boxed` aborts with a stack overflow at this
    /// size, because that variant frees its chain recursively.
    ///
    /// Run with: cargo run --release --bin list_drop

    use systems_lab::lists::SinglyList;
    use systems_lab::lists::boxed_drop::LinkedList;

    fn main() {
        const N: u64 = 5_000_000;

        let mut list = LinkedList::new();
        for value in 0..N {

```



```

        list.push_front(value);
    }
    println!("built a list of {N} nodes");

    list.pop_front();
    println!("dropping the remaining {} nodes", N - 1);

    drop(list);
    println!("dropped without recursing");
}

```

9.7 The benchmark

The benchmark program measures all four versions with one generic function. The full program also measures the caches of chapter 13, and is listed there. This is the part that measures the lists.

Listing 9.15 The list benchmark (lines 159 to 235). `benchmarking_examples/benchmark.rs`

```

// -----
// Lists: four variants, one loop
// -----

fn list_benchmark(variant: Option<&str>, elements: usize) {
    println!(
        "Singly linked list: push {} values, pop them back in order, then drop a full
list",
        with_thousands(elements as u64)
    );
    println!("Best of {LIST_RUNS} runs per variant, allocator warmed up.\n");
    println!(
        "  {:<10} {:>12} {:>13} {:>13}",
        "variant", "elements", "push", "drop"
    );
    println!("  {} ", "-".repeat(52));

    match variant {
        Some("box") => run_list::<boxed::LinkedList<u64>>>(elements),
        Some("enum") => run_list::<enum_node::LinkedList<u64>>>(elements),
        Some("box+drop") => run_list::<boxed_drop::LinkedList<u64>>>(elements),
        Some("enum+drop") => run_list::<enum_drop::LinkedList<u64>>>(elements),
        _ => {
            run_list::<boxed::LinkedList<u64>>>(elements);
            run_list::<enum_node::LinkedList<u64>>>(elements);
            run_list::<boxed_drop::LinkedList<u64>>>(elements);
            run_list::<enum_drop::LinkedList<u64>>>(elements);
        }
    }
}

fn run_list<L: SinglyList<u64>>>(elements: usize) {
    // Warm the allocator first: without this the first variant in a process pays
    // for fresh heap pages and looks several times slower than an identical one.

```

```

let mut warm_up = L::new();
for value in 0..elements as u64 {
    warm_up.push_front(value);
}
drop(warm_up);

let mut best_push = Duration::MAX;
let mut best_drop = Duration::MAX;

for _ in 0..LIST_RUNS {
    let mut list = L::new();

    let start = Instant::now();
    for value in 0..elements as u64 {
        list.push_front(value);
    }
    best_push = best_push.min(start.elapsed());

    // Pop every value back and check the order, so a broken list cannot post
    // a fast time.
    let mut expected = elements as u64;
    while let Some(value) = list.pop_front() {
        expected -= 1;
        assert_eq!(value, expected, "{} lost LIFO order", L::variant());
    }
    assert_eq!(expected, 0);
    black_box(expected);

    // Rebuild, then time the drop of a full list. This is where the
    // recursive variants overflow the stack.
    for value in 0..elements as u64 {
        list.push_front(value);
    }
    let start = Instant::now();
    drop(list);
    best_drop = best_drop.min(start.elapsed());
}

println!(
    " {:<10} {:>12} {best_push:>13.2?} {best_drop:>13.2?} ",
    L::variant(),
    with_thousands(elements as u64)
);
}

```

`run_list::<L: SinglyList<u64>>` is **generic** over the list type `L`. The compiler generates a separate copy of the function for each list type. So each list is measured with its own code.

The function follows three rules that apply to any small benchmark:

1. **Warm up first.** The first time a program allocates a lot of memory, the operating system has to supply fresh pages, which is slow. A warm-up pass pays that cost before timing starts.

2. **Run several times and keep the fastest.** Other programs can slow a run down but never speed it up. So the fastest run is the closest to the true cost.
3. **Check the result before trusting the time.** Every run pops all the values and checks their order. A broken list cannot report a fast time.

`std::hint::black_box(expected)` tells the compiler to treat the value as used, so it cannot delete the loop that computed it.

These results come from a release build on an 8-core aarch64 development board:

list	how it drops	260,000 nodes	270,000 nodes
<code>box</code>	recursively	completed	crashed
<code>enum</code>	recursively	completed	crashed
<code>box+drop</code>	with a loop	completed	completed
<code>enum+drop</code>	with a loop	completed	completed

variant	elements	push	drop
<code>box</code>	200,000	2.40ms	3.23ms
<code>enum</code>	200,000	3.42ms	3.20ms
<code>box+drop</code>	200,000	2.32ms	2.91ms
<code>enum+drop</code>	200,000	3.62ms	2.52ms
<code>box+drop</code>	5,000,000	58.11ms	72.32ms
<code>enum+drop</code>	5,000,000	90.98ms	67.51ms

Read the table this way:

- A push costs about 12 nanoseconds for the struct node and about 17 for the enum node. Section 9.5.2 gave two differences that could explain the gap. The benchmark does not measure them separately.
- `box` and `box+drop` share the same push code, and their push times agree, 2.40 against 2.32 ms. That agreement is a sign the measurement is stable.
- The loop drop is not much faster than the recursive one. Its purpose is to remove the size limit, not to save time.

You can reproduce the crash and the fix:

```

cargo run --release --bin benchmark list box 260000 # completes
cargo run --release --bin benchmark list box 270000 # crashes
cargo run --release --bin list_drop                # five million nodes, no crash

```

9.8 A doubly linked list

In a **doubly linked list**, each node points to the next node and also back to the previous one. That lets you add and remove at both ends in $O(1)$, and walk the list in either direction.

In Rust this creates an ownership problem. Every node in the middle is pointed to twice: by the previous node's `next` and by the following node's `prev`. A `Box` has exactly one owner, so it cannot express this.

The solution uses the three types from chapter 8. Nodes live in `Rc<RefCell<Node>>`, so they can have more than one handle and still be changed. The forward links, `next`, are strong `Rc` handles, so the list owns its nodes from front to back. The backward links, `prev`, are `Weak` handles, so they do not create reference cycles (figure 9.6).

A doubly linked list after `push_front(10)`, `push_front(20)`, `push_back(30)`, `push_back(40)`

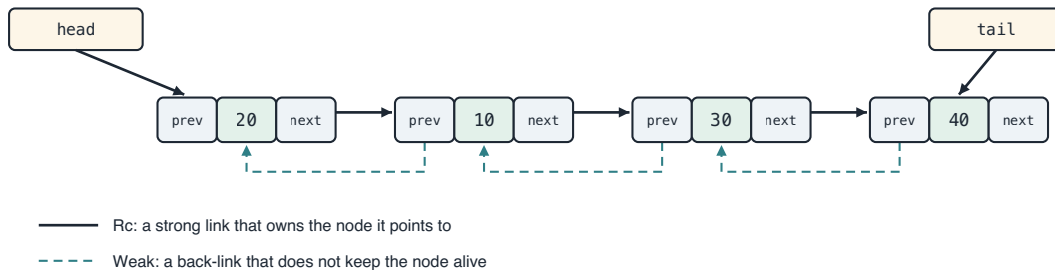


Figure 9.6 Strong links run forward and weak links run backward. If the back-links were strong, every pair of neighbors would form a cycle and no node would ever be freed.

9.8.1 The types

Listing 9.16 Type names, the node, and the list (lines 1 to 26). `src/bin/ll.rs`

```

// =====
// Type aliases for clarity.
// =====
use std::cell::{Ref, RefCell}; // Ref is needed for peek return types
use std::rc::{Rc, Weak};

```

```

type NodeRef<T> = Rc<RefCell<Node<T>>>;
type WeakNodeRef<T> = Weak<RefCell<Node<T>>>;
type OptNodeRef<T> = Option<NodeRef<T>>;
type OptWeakNodeRef<T> = Option<WeakNodeRef<T>>;

// =====
// Node and LinkedList definitions.
// =====
struct Node<T> {
    data: T,
    next: OptNodeRef<T>,
    prev: OptWeakNodeRef<T>,
}

pub struct LinkedList<T> {
    head: OptNodeRef<T>,
    tail: OptNodeRef<T>,
    len: usize,
}

```

The full type of a link is `Option<Rc<RefCell<Node<T>>>>`. It appears in almost every function, so the file names it once, `OptNodeRef<T>`, with a `type` alias. The list keeps a `head`, a `tail`, and its length.

9.8.2 Adding at either end

Listing 9.17 `new`, `len`, and `push_front` (lines 28 to 69).

```

/// Creates an empty list.
pub fn new() -> Self {
    LinkedList {
        head: None,
        tail: None,
        len: 0,
    }
}

pub fn len(&self) -> usize {
    self.len
}

pub fn is_empty(&self) -> bool {
    self.len == 0
}

// =====
// push_front
// =====
pub fn push_front(&mut self, data: T) {
    let new_node = Rc::new(RefCell::new(Node {
        data,
        next: None,
        prev: None,
    }));
}

```

```

    // take() moves the old head out of the Option, leaving None.
    match self.head.take() {
        Some(old_head) => {
            // Link old head back to new node (weakly).
            old_head.borrow_mut().prev = Some(Rc::downgrade(&new_node));
            // Link new node forward to old head.
            new_node.borrow_mut().next = Some(old_head);
        }
        None => {
            // List was empty: new node is also the tail.
            self.tail = Some(new_node.clone());
        }
    }
    self.head = Some(new_node);
    self.len += 1;
}

```

`push_front` creates the new node, then looks at the old head with `self.head.take()`:

- If there was an old head, its `prev` becomes a weak link to the new node, made with `Rc::downgrade`. The new node's `next` becomes the old head.
- If there was no old head, the list was empty, so the new node is also the tail.
`new_node.clone()` makes a second strong handle for `tail`.

Finally the new node becomes the head, and the length grows by one. `borrow_mut()` is the `RefCell` method from section 8.4. Each call is released at the end of its statement, so no two borrows of the same node overlap.

Listing 9.18 `push_back` (lines 71 to 93).

```

// -----
// push_back
// -----
pub fn push_back(&mut self, data: T) {
    let new_node = Rc::new(RefCell::new(Node {
        data,
        next: None,
        // self.tail.as_ref() gives Option<&NodeRef<T>>,
        // map(Rc::downgrade) turns it into Option<WeakNodeRef<T>>.
        prev: self.tail.as_ref().map(Rc::downgrade),
    }));

    match self.tail.take() {
        Some(old_tail) => {
            old_tail.borrow_mut().next = Some(new_node.clone());
        }
        None => {
            self.head = Some(new_node.clone());
        }
    }
    self.tail = Some(new_node);
}

```

```

        self.len += 1;
    }

```

`push_back` mirrors it at the other end. The new node's `prev` is set as it is created: `self.tail.as_ref().map(Rc::downgrade)`. `as_ref()` turns `Option<Rc<...>>` into `Option<&Rc<...>>`, so the tail is looked at without being moved. `map(Rc::downgrade)` turns that reference into a weak link.

9.8.3 Removing from either end

Listing 9.19 `pop_front` (lines 95 to 122).

```

// -----
// pop_front
// -----
pub fn pop_front(&mut self) -> Option<T> {
    self.head.take().and_then(|old_head| {
        // Take ownership of the next node (if any).
        let next = old_head.borrow_mut().next.take();

        match next {
            Some(next_node) => {
                // Disconnect the new head's prev pointer.
                next_node.borrow_mut().prev = None;
                self.head = Some(next_node);
            }
            None => {
                self.tail = None; // list becomes empty
            }
        }

        self.len -= 1;

        // Safely extract the data. At this point, old_head is the only
        // strong reference (because we removed it from the list and the
        // new head's prev doesn't point to it).
        let node = Rc::try_unwrap(old_head).ok().unwrap().into_inner();
        Some(node.data)
    })
}

```

`pop_front` must return the value by ownership, not a reference. To move the value out, the node itself must be moved out of its `Rc`, and `Rc::try_unwrap` does that. It succeeds only if the handle is the last strong one. So the method first removes every other strong link to the node:

1. `self.head.take()` removes the list's link to it.
2. The next node's `prev` is set to `None`. That link was weak, so it did not count anyway.

3. If there was no next node, the list is now empty, so `self.tail` is cleared, removing the tail's link.

Now `Rc::try_unwrap(old_head)` succeeds, and `.into_inner()` moves the `Node` out of its `RefCell`.

The call is written `.ok().unwrap()` rather than `.unwrap()`. `Result::unwrap` needs to print the error if it fails, which requires the error type to implement `Debug`. Here the error type is the `Rc` itself, and `Node` does not implement `Debug`, so `.unwrap()` would not compile. `.ok()` turns the `Result` into an `Option`, whose `unwrap` needs no `Debug`.

Listing 9.20 `pop_back` (lines 124 to 147).

```
// -----
// pop_back
// -----
pub fn pop_back(&mut self) -> Option<T> {
    self.tail.take().and_then(|old_tail| {
        // Get the previous node (upgrade the weak reference).
        let prev = old_tail.borrow_mut().prev.take().and_then(|w| w.upgrade());

        match prev {
            Some(prev_node) => {
                prev_node.borrow_mut().next = None;
                self.tail = Some(prev_node);
            }
            None => {
                self.head = None; // list becomes empty
            }
        }

        self.len -= 1;

        let node = Rc::try_unwrap(old_tail).ok().unwrap().into_inner();
        Some(node.data)
    })
}
```

`pop_back` mirrors `pop_front`. It follows the weak `prev` link with `upgrade()`, which returns `None` if the previous node is gone, then clears that node's `next`.

9.8.4 Looking without removing

Listing 9.21 `peek_front` and `peek_back` (lines 149 to 171).

```
// -----
// peek_front: returns a guarded reference to the front data.
// The returned Ref<'_, T> holds the borrow of the RefCell until it goes
// out of scope, ensuring the reference stays valid.
```



```
// -----
pub fn peek_front(&self) -> Option<Ref<'_, T>> {
    // self.head.as_ref() -> Option<&NodeRef<T>>
    self.head.as_ref().map(|node| {
        // node.borrow() returns Ref<'_, Node<T>>.
        // Ref::map projects that to a Ref<'_, T> pointing to the data field.
        Ref::map(node.borrow(), |n| &n.data)
    })
}

// -----
// peek_back: similar to peek_front.
// -----
pub fn peek_back(&self) -> Option<Ref<'_, T>> {
    self.tail
        .as_ref()
        .map(|node| Ref::map(node.borrow(), |n| &n.data))
}
```

`peek_front` returns `Option<Ref<'_, T>>`, not `Option<&T>`. The value is inside a `RefCell`, and a reference to it is only safe while the cell's borrow is active. A `Ref` is the guard that keeps the borrow active, as section 8.4 showed. `Ref::map` narrows a guard for the whole node into a guard for its `data` field. The caller reads the value with `*`, as `main` does.

9.8.5 The complete program

Listing 9.22 The complete program, with its tests. `src/bin/ll.rs`

```
// =====
// Type aliases for clarity.
// =====
use std::cell::{Ref, RefCell}; // Ref is needed for peek return types
use std::rc::{Rc, Weak};

type NodeRef<T> = Rc<RefCell<Node<T>>>;
type WeakNodeRef<T> = Weak<RefCell<Node<T>>>;
type OptNodeRef<T> = Option<NodeRef<T>>;
type OptWeakNodeRef<T> = Option<WeakNodeRef<T>>;

// =====
// Node and LinkedList definitions.
// =====
struct Node<T> {
    data: T,
    next: OptNodeRef<T>,
    prev: OptWeakNodeRef<T>,
}

pub struct LinkedList<T> {
    head: OptNodeRef<T>,
    tail: OptNodeRef<T>,
    len: usize,
}
```

```

impl<T> LinkedList<T> {
    /// Creates an empty list.
    pub fn new() -> Self {
        LinkedList {
            head: None,
            tail: None,
            len: 0,
        }
    }

    pub fn len(&self) -> usize {
        self.len
    }

    pub fn is_empty(&self) -> bool {
        self.len == 0
    }

    // -----
    // push_front
    // -----
    pub fn push_front(&mut self, data: T) {
        let new_node = Rc::new(RefCell::new(Node {
            data,
            next: None,
            prev: None,
        }));

        // take() moves the old head out of the Option, leaving None.
        match self.head.take() {
            Some(old_head) => {
                // Link old head back to new node (weakly).
                old_head.borrow_mut().prev = Some(Rc::downgrade(&new_node));
                // Link new node forward to old head.
                new_node.borrow_mut().next = Some(old_head);
            }
            None => {
                // List was empty: new node is also the tail.
                self.tail = Some(new_node.clone());
            }
        }
        self.head = Some(new_node);
        self.len += 1;
    }

    // -----
    // push_back
    // -----
    pub fn push_back(&mut self, data: T) {
        let new_node = Rc::new(RefCell::new(Node {
            data,
            next: None,
            // self.tail.as_ref() gives Option<&NodeRef<T>>,
            // map(Rc::downgrade) turns it into Option<WeakNodeRef<T>>.
            prev: self.tail.as_ref().map(Rc::downgrade),
        }));

        match self.tail.take() {
            Some(old_tail) => {

```

```

        old_tail.borrow_mut().next = Some(new_node.clone());
    }
    None => {
        self.head = Some(new_node.clone());
    }
}
self.tail = Some(new_node);
self.len += 1;
}

// -----
// pop_front
// -----
pub fn pop_front(&mut self) -> Option<T> {
    self.head.take().and_then(|old_head| {
        // Take ownership of the next node (if any).
        let next = old_head.borrow_mut().next.take();

        match next {
            Some(next_node) => {
                // Disconnect the new head's prev pointer.
                next_node.borrow_mut().prev = None;
                self.head = Some(next_node);
            }
            None => {
                self.tail = None; // list becomes empty
            }
        }

        self.len -= 1;

        // Safely extract the data. At this point, old_head is the only
        // strong reference (because we removed it from the list and the
        // new head's prev doesn't point to it).
        let node = Rc::try_unwrap(old_head).ok().unwrap().into_inner();
        Some(node.data)
    })
}

// -----
// pop_back
// -----
pub fn pop_back(&mut self) -> Option<T> {
    self.tail.take().and_then(|old_tail| {
        // Get the previous node (upgrade the weak reference).
        let prev = old_tail.borrow_mut().prev.take().and_then(|w| w.upgrade());

        match prev {
            Some(prev_node) => {
                prev_node.borrow_mut().next = None;
                self.tail = Some(prev_node);
            }
            None => {
                self.head = None; // list becomes empty
            }
        }

        self.len -= 1;
    })
}

```

```

        let node = Rc::try_unwrap(old_tail).ok().unwrap().into_inner();
        Some(node.data)
    })
}

// -----
// peek_front: returns a guarded reference to the front data.
// The returned Ref<'_, T> holds the borrow of the RefCell until it goes
// out of scope, ensuring the reference stays valid.
// -----
pub fn peek_front(&self) -> Option<Ref<'_, T>> {
    // self.head.as_ref() -> Option<&NodeRef<T>>
    self.head.as_ref().map(|node| {
        // node.borrow() returns Ref<'_, Node<T>>.
        // Ref::map projects that to a Ref<'_, T> pointing to the data field.
        Ref::map(node.borrow(), |n| &n.data)
    })
}

// -----
// peek_back: similar to peek_front.
// -----
pub fn peek_back(&self) -> Option<Ref<'_, T>> {
    self.tail
        .as_ref()
        .map(|node| Ref::map(node.borrow(), |n| &n.data))
}

// =====
// Demonstration with main().
// =====
fn main() {
    let mut list = LinkedList::new();

    list.push_front(10);
    list.push_front(20);
    list.push_back(30);
    list.push_back(40);

    println!("Length: {}", list.len());

    // Peek returns a Ref; we can dereference it with *.
    if let Some(front) = list.peek_front() {
        println!("Front: {}", *front);
    }
    if let Some(back) = list.peek_back() {
        println!("Back: {}", *back);
    }

    println!("Pop front: {:?}" , list.pop_front()); // 20
    println!("Pop back:  {:?}" , list.pop_back()); // 40

    // Drain the list.
    while let Some(val) = list.pop_front() {
        println!("Popped: {}", val);
    }
    println!("Empty? {}", list.is_empty());
}

```

```
// =====
// Tests (cargo test --bin ll)
// =====
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn new_list_is_empty() {
        let list: LinkedList<i32> = LinkedList::new();
        assert!(list.is_empty());
        assert_eq!(list.len(), 0);
        assert!(list.peek_front().is_none());
        assert!(list.peek_back().is_none());
    }

    #[test]
    fn push_front_back_and_len() {
        let mut list = LinkedList::new();
        list.push_back(1);
        list.push_back(2);
        list.push_front(0);

        assert_eq!(list.len(), 3);
        assert_eq!(*list.peek_front().unwrap(), 0);
        assert_eq!(*list.peek_back().unwrap(), 2);
    }

    #[test]
    fn pop_front_and_back_work_in_order() {
        let mut list = LinkedList::new();
        list.push_back(10);
        list.push_back(20);
        list.push_back(30);

        assert_eq!(list.pop_front(), Some(10));
        assert_eq!(list.pop_back(), Some(30));
        assert_eq!(list.pop_front(), Some(20));
        assert_eq!(list.pop_front(), None);
        assert_eq!(list.pop_back(), None);
        assert!(list.is_empty());
    }

    #[test]
    fn popping_last_item_emptyies_the_list() {
        let mut list = LinkedList::new();
        list.push_back(42);
        assert_eq!(list.peek_front().map(|x| *x), Some(42));
        assert_eq!(list.pop_back(), Some(42));
        assert!(list.is_empty());
    }

    #[test]
    fn alternating_push_pop_keeps_invariants() {
        let mut list = LinkedList::new();
        for i in 0..100 {
            list.push_back(i);
            list.push_front(-i);
        }
    }
}
```

```

    }
    assert_eq!(list.len(), 200);
    for i in (0..100).rev() {
        assert_eq!(list.pop_front(), Some(-i));
        assert_eq!(list.pop_back(), Some(i));
    }
    assert!(list.is_empty());
}

// =====
// Explanation of key methods used:
//
// - Rc::new / RefCell::new      : create a reference-counted, mutable cell.
// - borrow() / borrow_mut()    : borrow the RefCell's contents (immutably/mutably).
// - Rc::downgrade()            : turn an Rc into a Weak (no ownership).
// - Weak::upgrade()            : attempt to get an Rc from a Weak (returns Option).
// - Option::take()             : take the value out of an Option, leaving None.
// - Option::as_ref()           : convert Option<T> to Option<&T> without moving.
// - Option::map()              : apply a function to the inner value if Some.
// - Option::and_then()         : like map but returns an Option (for chaining).
// - Rc::try_unwrap()           : move the value out of the Rc if it's the only strong
ref.
// - Ref::map()                : project a Ref to a subfield, keeping the borrow
alive.
// =====

```

```

$ cargo run --bin ll
Length: 4
Front: 20
Back: 40
Pop front: Some(20)
Pop back: Some(40)
Popped: 10
Popped: 30
Empty? true

```

The test `alternating_push_pop_keeps_invariants` pushes 100 values at each end, then pops from both ends and checks every value. The comment block at the end of the file lists each standard-library method the file uses, with a one-line description.

This list costs more than the singly linked list. Each node needs a heap block, two counts, and a borrow flag, and every access is checked. It also keeps the recursive drop from section 9.6, because dropping the head drops the next `Rc`, which drops the next node. Chapter 13 avoids both costs for an LRU cache by storing its nodes in a `Vec` and linking them by index.

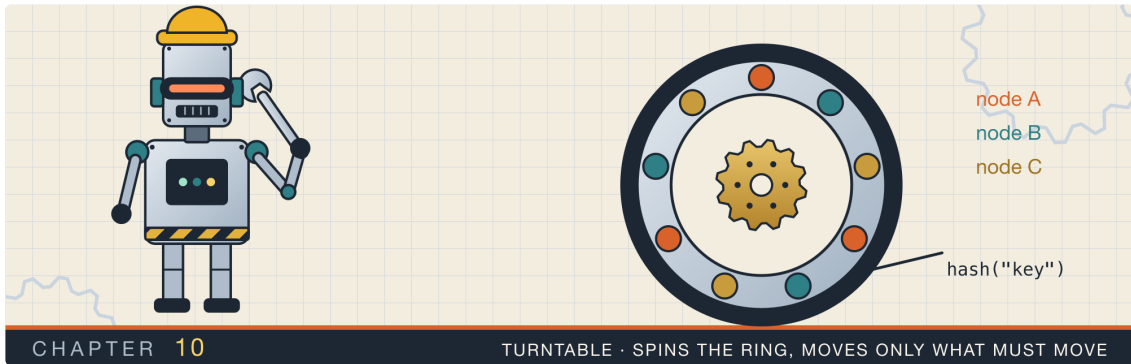
Summary

- A linked list stores each value in its own node and links the nodes with pointers. It adds and removes at the front in $O(1)$, but reaching position i takes $O(i)$.
- `Option::take` and `std::mem::replace` move a value out of a place and leave a valid value behind. List code uses them to move nodes between owners.
- Reversing a list moves three owners per node: take the rest, point the node back, make it the new front.
- A list can be a struct with `Option<Box<Node>>` or an enum. Both nodes are 16 bytes for `u64` values.
- The compiler-generated drop of a `Box` list recurses once per node and overflows the stack for long lists. A hand-written `Drop` loop removes the limit.
- A benchmark should warm up, keep the fastest of several runs, and check its results.
- A doubly linked list needs `Rc<RefCell<Node>>` with strong forward links and `Weak` back-links.

Chapter 10 uses the singly linked list from section 9.3 to merge many sorted lists into one, and compares ten ways of doing it.

Exercises

1. Write `len` for the enum list of section 9.4 with a loop instead of recursion.
2. Implement `Iterator` for the struct list of section 9.3, with an `iter()` method that yields `&i32`.
3. Implement `Drop` for the doubly linked list, and test it with one million nodes.
4. Build a doubly linked list whose nodes live in a `Vec`, with `Option<usize>` indexes as links. Compare its push and pop times with the `Rc` version.



Merging k sorted lists

THIS CHAPTER COVERS

- Merging two sorted linked lists by moving nodes, not copying values
- Three strategies for merging many lists, and why two of them are much faster than the third
- Borrowing two elements of one `Vec` at the same time, and ways to avoid needing to
- The dummy head, the tail pointer, and `Option::insert`
- Ten implementations of the same task, compared

Chapter 9 built singly linked lists and moved their nodes between owners with `take`. This chapter applies those skills to one problem. You have k linked lists, each already sorted, and you want one sorted list that contains all their nodes.

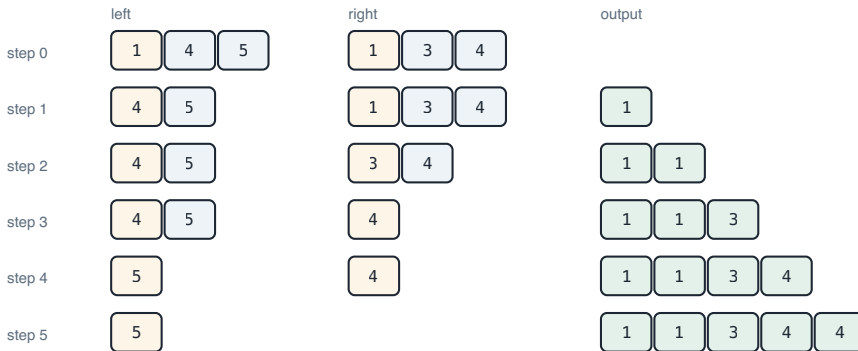
The problem is small, but it has several good solutions, and they differ in interesting ways. Some are faster than others. Some move nodes; others allocate new ones. Some fight the borrow checker, and some avoid the fight. This chapter walks through ten solutions, grouped by the strategy they use.

Two letters appear throughout. k is the number of lists. N is the total number of nodes across all of them.

10.1 Merging two sorted lists

Everything in this chapter is built on merging two sorted lists, so start there. Compare the front nodes of the two lists. Move the smaller one to the end of the output. Repeat until one list is empty, then attach the other list whole. Figure 10.1 works through an example.

Merging $1 \rightarrow 4 \rightarrow 5$ with $1 \rightarrow 3 \rightarrow 4$, one node per step



Each step compares the two front nodes (cream) and moves the smaller one to the output. When one side is empty, the other is attached whole.

Figure 10.1 Merging two sorted lists. Each step moves one node, so merging lists of lengths a and b takes $a + b$ steps.

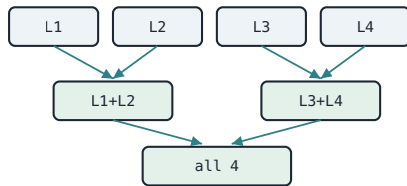
With linked lists, "move a node to the output" does not copy the value. It detaches the node from the front of its list and links it to the end of the output. No memory is allocated.

10.2 Three strategies for k lists

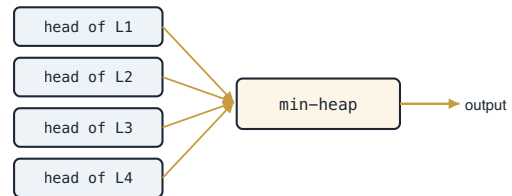
Figure 10.2 shows three ways to extend two-list merging to k lists.

Three ways to merge $k = 4$ lists1. One at a time: $O(kN)$ 

The growing result is walked again at every step.

2. Pairwise rounds: $O(N \log k)$ 

Each node takes part in one merge per round; there are $\log_2 k$ rounds.

3. A heap of the k heads: $O(N \log k)$ 

Pop the smallest head, push that list's next node.

Figure 10.2 The first strategy walks its growing result again and again. The other two touch each node only about $\log_2 k$ times.

One at a time. Merge list 2 into list 1, then list 3 into the result, and so on. Each merge walks the whole result so far, which keeps growing. The first nodes are walked k times, so the total is about $O(kN)$.

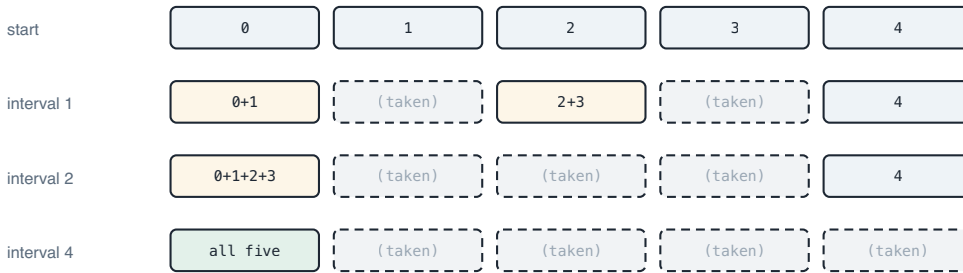
Pairwise rounds. Merge the lists in pairs, like the rounds of a tournament. After the first round there are $k / 2$ lists, after the second $k / 4$, and so on. In every round, each node takes part in exactly one merge. There are about $\log_2 k$ rounds, so the total is $O(N \log k)$.

A heap of heads. Put the front node of each list into a min-heap, the structure from chapter 5. Pop the smallest, append it to the output, and push the next node from the same list. Each node goes through one push and one pop on a heap of at most k entries. Each costs $O(\log k)$, so the total is again $O(N \log k)$.

For 100 lists of 10,000 nodes each, N is a million. $O(kN)$ is about 100 million steps. $O(N \log k)$ is about 7 million.

The pairwise strategy has a compact in-place form that most of the ten versions use. It keeps the lists in a `Vec` and merges positions that are a growing **interval** apart (figure 10.3).

Interval rounds on five lists: each round merges `lists[i]` with `lists[i + interval]`



Positions shown as (taken) were emptied by `take()` when their list moved into a merge.

Figure 10.3 Interval rounds on five lists. The interval doubles each round, and the result collects in `lists[0]`.

In each round, the list at position `i` absorbs the list at `i + interval`. The position `i` runs through `0`, `2 × interval`, `4 × interval`, and so on. The odd list out, number 4 here, waits until the interval is large enough to reach it.

10.3 First version: `split_at_mut` and a dummy head

Listing 10.1 The interval loop (lines 20 to 35). `src/bin/merge_k_sorted_lists_divide.rs`

```
pub fn merge_k_lists(mut lists: Vec<OptionalLink<ListNode>>) ->
OptionalLink<ListNode> {
    if lists.is_empty() { return None; }

    let (n, mut interval) = (lists.len(), 1);
    while interval < n {
        let mut i = 0;
        while i + interval < n {
            let (left, right) = lists.split_at_mut(i + interval);
            left[i] = Self::merge_two(&mut left[i], &mut right[0]);
            i += interval * 2;
        }
        interval *= 2;
    }

    lists[0].take()
}
```

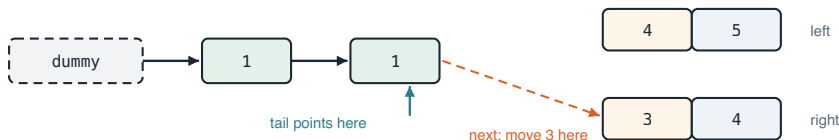
The two `while` loops follow figure 10.3. The outer one doubles `interval`. The inner one steps `i` by `interval * 2`, merging `i` with `i + interval`.

The merge needs to change two elements of the vector at once. The obvious code, `merge_two(&mut lists[i], &mut lists[i + interval])`, does not compile. Both `&mut` borrows are borrows of the whole vector `lists`, and Rust allows only one mutable borrow of a value at a time. The compiler does not reason about the two indexes being different.

`split_at_mut(i + interval)` solves this. It splits the vector into two slices that do not overlap: the part before position `i + interval` and the rest. It returns a mutable borrow of each. Now `left[i]` and `right[0]` come from different slices, and borrowing both is allowed.

The merge of two lists uses a **dummy head** and a **tail pointer** (figure 10.4).

Building the output with a dummy head and a tail pointer



dummy.next is the merged list. The dummy exists so that appending the first node needs no special case.

Figure 10.4 The dummy node gives the output a fixed starting point. `tail` always points at the last node, where the next one is attached.

Listing 10.2 Merging two lists (lines 37 to 51).

```
fn merge_two(left: &mut Optionallink<ListNode>, right: &mut Optionallink<ListNode>)
-> Optionallink<ListNode> {
    let mut dummy = Box::new(ListNode::new(0));
    let mut tail = &mut dummy;

    while let (Some(l), Some(r)) = (left.as_ref(), right.as_ref()) {
        let smaller = if l.data < r.data { &mut *left } else { &mut *right };
        let mut head = smaller.take().unwrap();
        *smaller = head.next.take();
        tail.next = Some(head);
        tail = tail.next.as_mut().unwrap();
    }

    tail.next = left.take().or(right.take());
    dummy.next
}
```

`dummy` is a throwaway node, and `tail` is a mutable reference to the last node of the output. At the start, the output is empty, so `tail` points at `dummy`. Each pass of the loop:

1. `while let (Some(l), Some(r)) = (left.as_ref(), right.as_ref())` continues while both lists have a front node, and looks at both without moving them.
2. `smaller` is a mutable reference to whichever list has the smaller front.
3. `smaller.take().unwrap()` takes that front node out, and `*smaller = head.next.take()` puts the rest of that list back.
4. `tail.next = Some(head)` attaches the node, and `tail = tail.next.as_mut().unwrap()` moves `tail` onto it.

When one list runs out, `left.take().or(right.take())` attaches whichever one is left. The merged list starts at `dummy.next`.

Listing 10.3 The complete program. `src/bin/merge_k_sorted_lists_divide.rs`

```
struct MergeKSorted { }

#[derive(Default)]
struct ListNode {
    data: u32,
    next: Option<Box<ListNode>>
}

impl ListNode {
    fn new(data: u32) -> Self {
        Self {
            data,
            next: None
        }
    }
}

type OptionalLink<T> = Option<Box<T>>;

impl MergeKSorted {
    pub fn merge_k_lists(mut lists: Vec<OptionalLink<ListNode>>) ->
OptionalLink<ListNode> {
        if lists.is_empty() { return None; }

        let (n, mut interval) = (lists.len(), 1);
        while interval < n {
            let mut i = 0;
            while i + interval < n {
                let (left, right) = lists.split_at_mut(i + interval);
                left[i] = Self::merge_two(&mut left[i], &mut right[0]);
                i += interval * 2;
            }
            interval *= 2;
        }

        lists[0].take()
    }
}
```

```

    }

    fn merge_two(left: &mut Optionallink<ListNode>, right: &mut Optionallink<ListNode>)
    -> Optionallink<ListNode> {
        let mut dummy = Box::new(ListNode::new(0));
        let mut tail = &mut dummy;

        while let (Some(l), Some(r)) = (left.as_ref(), right.as_ref()) {
            let smaller = if l.data < r.data { &mut *left } else { &mut *right };
            let mut head = smaller.take().unwrap();
            *smaller = head.next.take();
            tail.next = Some(head);
            tail = tail.next.as_mut().unwrap();
        }

        tail.next = left.take().or(right.take());
        dummy.next
    }
}

/// Build a list from a slice, front to back.
fn from_slice(values: &[u32]) -> Optionallink<ListNode> {
    let mut head = None;
    for &value in values.iter().rev() {
        let mut node = Box::new(ListNode::new(value));
        node.next = head;
        head = Some(node);
    }
    head
}

/// Collect a list's values into a vector.
fn to_vec(mut list: &Optionallink<ListNode>) -> Vec<u32> {
    let mut values = Vec::new();
    while let Some(node) = list {
        values.push(node.data);
        list = &node.next;
    }
    values
}

fn main() {
    let lists = vec![from_slice(&[1, 4, 5]), from_slice(&[1, 3, 4]), from_slice(&[2, 6])];
    let merged = MergeKSorted::merge_k_lists(lists);
    println!("{:?}", to_vec(&merged));
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn merges_three_lists() {
        let lists = vec![from_slice(&[1, 4, 5]), from_slice(&[1, 3, 4]), from_slice(&[2,
6])];
        let merged = MergeKSorted::merge_k_lists(lists);
        assert_eq!(to_vec(&merged), vec![1, 1, 2, 3, 4, 4, 5, 6]);
    }
}

```

```

#[test]
fn handles_empty_input_and_empty_lists() {
    assert!(MergeKSorted::merge_k_lists(vec![]).is_none());
    let merged = MergeKSorted::merge_k_lists(vec![None, from_slice(&[2]), None]);
    assert_eq!(to_vec(&merged), vec![2]);
}

#[test]
fn handles_an_odd_number_of_lists() {
    let lists: Vec<_> = (0..5).map(|i| from_slice(&[i, i + 10])).collect();
    let merged = MergeKSorted::merge_k_lists(lists);
    assert_eq!(to_vec(&merged), vec![0, 1, 2, 3, 4, 10, 11, 12, 13, 14]);
}
}

```

The tests check three lists, an empty input with empty lists inside it, and an odd number of lists.

```

$ cargo run --bin merge_k_sorted_lists_divide
[1, 1, 2, 3, 4, 4, 5, 6]

```

10.4 Avoiding the double borrow

`split_at_mut` works, but the next two versions show that the double borrow can be avoided altogether.

10.4.1 Take the right list out first

Listing 10.4 The loop and the merge (lines 19 to 55). `src/bin/merge_k_sorted_lists_swap.rs`

```

impl MergeKSorted {
    pub fn merge_k_lists(mut lists: Vec<OptionalLink<ListNode>>) ->
OptionalLink<ListNode> {
        if lists.is_empty() { return None; }

        let (n, mut interval) = (lists.len(), 1);
        while interval < n {
            let mut i = 0;
            while i + interval < n {
                let mut right = lists[i + interval].take();
                lists[i] = Self::merge_two(&mut lists[i], &mut right);
                i += interval * 2;
            }
            interval *= 2;
        }

        lists[0].take()
    }
}

```

```

fn merge_two(left: &mut OptionalLink<ListNode>, right: &mut OptionalLink<ListNode>)
-> OptionalLink<ListNode> {
    let mut dummy = Box::new(ListNode::new(0));
    let mut tail = &mut dummy;

    while let (Some(l), Some(r)) = (&left, &right) {
        if r.data < l.data {
            std::mem::swap(left, right);
        }
        let mut smallest = left.take().unwrap();
        *left = smallest.next.take();
        tail.next = Some(smallest);
        tail = tail.next.as_mut().unwrap();
    }

    tail.next = left.take().or(right.take());
    dummy.next
}

```

Look at the inner loop:

```

let mut right = lists[i + interval].take();
lists[i] = Self::merge_two(&mut lists[i], &mut right);

```

The right list is moved out of the vector into a local variable first. After that, the only borrow of the vector is the one for `lists[i]`, so the call compiles.

The merge changes too. Instead of choosing which list to take from, it makes sure `left` always holds the smaller front. When the right front is smaller, `std::mem::swap(left, right)` exchanges the two lists. Then the loop body always takes from `left`.

Listing 10.5 The complete program. `src/bin/merge_k_sorted_lists_swap.rs`

```

struct MergeKSorted { }

#[derive(Default)]
struct ListNode {
    data: u32,
    next: Option<Box<ListNode>>
}
impl ListNode {
    fn new(data: u32) -> Self {
        Self {
            data,
            next: None
        }
    }
}

type OptionalLink<T> = Option<Box<T>>;

```



```

impl MergeKSorted {
    pub fn merge_k_lists(mut lists: Vec<OptionalLink<ListNode>>) ->
OptionalLink<ListNode> {
        if lists.is_empty() { return None; }

        let (n, mut interval) = (lists.len(), 1);
        while interval < n {
            let mut i = 0;
            while i + interval < n {
                let mut right = lists[i + interval].take();
                lists[i] = Self::merge_two(&mut lists[i], &mut right);
                i += interval * 2;
            }
            interval *= 2;
        }

        lists[0].take()
    }

    fn merge_two(left: &mut OptionalLink<ListNode>, right: &mut OptionalLink<ListNode>)
-> OptionalLink<ListNode> {
        let mut dummy = Box::new(ListNode::new(0));
        let mut tail = &mut dummy;

        while let (Some(l), Some(r)) = (&left, &right) {
            if r.data < l.data {
                std::mem::swap(left, right);
            }
            let mut smallest = left.take().unwrap();
            *left = smallest.next.take();
            tail.next = Some(smallest);
            tail = tail.next.as_mut().unwrap();
        }

        tail.next = left.take().or(right.take());
        dummy.next
    }
}

// Build a list from a slice, front to back.
fn from_slice(values: &[u32]) -> OptionalLink<ListNode> {
    let mut head = None;
    for &value in values.iter().rev() {
        let mut node = Box::new(ListNode::new(value));
        node.next = head;
        head = Some(node);
    }
    head
}

// Collect a list's values into a vector.
fn to_vec(mut list: &OptionalLink<ListNode>) -> Vec<u32> {
    let mut values = Vec::new();
    while let Some(node) = list {
        values.push(node.data);
        list = &node.next;
    }
    values
}

```

```

fn main() {
    let lists = vec![from_slice(&[1, 4, 5]), from_slice(&[1, 3, 4]), from_slice(&[2, 6])];
    let merged = MergeKSorted::merge_k_lists(lists);
    println!("{:?}", to_vec(&merged));
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn merges_three_lists() {
        let lists = vec![from_slice(&[1, 4, 5]), from_slice(&[1, 3, 4]), from_slice(&[2,
6])];
        let merged = MergeKSorted::merge_k_lists(lists);
        assert_eq!(to_vec(&merged), vec![1, 1, 2, 3, 4, 4, 5, 6]);
    }

    #[test]
    fn handles_empty_input_and_empty_lists() {
        assert!(MergeKSorted::merge_k_lists(vec![]).is_none());
        let merged = MergeKSorted::merge_k_lists(vec![None, from_slice(&[2]), None]);
        assert_eq!(to_vec(&merged), vec![2]);
    }

    #[test]
    fn handles_an_odd_number_of_lists() {
        let lists: Vec<_> = (0..5).map(|i| from_slice(&[i, i + 10])).collect();
        let merged = MergeKSorted::merge_k_lists(lists);
        assert_eq!(to_vec(&merged), vec![0, 1, 2, 3, 4, 10, 11, 12, 13, 14]);
    }
}

```

10.4.2 Merge owned lists, with no dummy node

Listing 10.6 The loop and the merge (lines 8 to 41). src/bin/merge_k_sorted_lists_simple.rs

```

fn merge_k_lists(mut lists: Vec<Link>) -> Link {
    if lists.is_empty() { return None; }

    let (n, mut interval) = (lists.len(), 1);
    while interval < n {
        let mut i = 0;
        while i + interval < n {
            let left = lists[i].take();
            let right = lists[i + interval].take();
            lists[i] = merge_two(left, right);
            i += interval * 2;
        }
        interval *= 2;
    }

    lists[0].take()
}

```

```
fn merge_two(mut left: Link, mut right: Link) -> Link {
    let mut merged = None;
    let mut tail = &mut merged;

    while let (Some(l), Some(r)) = (&left, &right) {
        if r.data < l.data {
            std::mem::swap(&mut left, &mut right);
        }
        *tail = left;
        tail = &mut tail.as_mut().unwrap().next; // hang left list on the tail
        left = tail.take(); // step past its first node
        // cut the rest off again
    }

    *tail = left.or(right);
    merged
}
```

This version changes two things.

First, `merge_two` takes both lists by value, not by reference. The loop moves both out of the vector with `take()` before the call, so nothing is borrowed at all.

Second, there is no dummy node. `tail` is a `&mut Link`, a pointer to the empty slot where the next node belongs. It starts at `&mut merged`, the slot for the first node. The loop body is three lines, each with a comment:

```
*tail = left; // hang left list on the tail
tail = &mut tail.as_mut().unwrap().next; // step past its first node
left = tail.take(); // cut the rest off again
```

After the swap has put the smaller front in `left`, the whole left list is placed in the empty slot. `tail` then moves to the `next` slot of the list's first node. Finally `tail.take()` cuts everything after that first node back off into `left`. One node has moved to the output.

Listing 10.7 The complete program. `src/bin/merge_k_sorted_lists_simple.rs`

```
struct ListNode {
    data: u32,
    next: Link,
}

type Link = Option<Box<ListNode>>;

fn merge_k_lists(mut lists: Vec<Link>) -> Link {
    if lists.is_empty() { return None; }

    let (n, mut interval) = (lists.len(), 1);
    while interval < n {
        let mut i = 0;
        while i + interval < n {
```

```

        let left = lists[i].take();
        let right = lists[i + interval].take();
        lists[i] = merge_two(left, right);
        i += interval * 2;
    }
    interval *= 2;
}

lists[0].take()
}

fn merge_two(mut left: Link, mut right: Link) -> Link {
    let mut merged = None;
    let mut tail = &mut merged;

    while let (Some(l), Some(r)) = (&left, &right) {
        if r.data < l.data {
            std::mem::swap(&mut left, &mut right);
        }
        *tail = left;
        tail = &mut tail.as_mut().unwrap().next; // hang left list on the tail
        left = tail.take(); // step past its first node
        // cut the rest off again
    }

    *tail = left.or(right);
    merged
}

fn from_slice(values: &[u32]) -> Link {
    let mut head = None;
    for &data in values.iter().rev() {
        head = Some(Box::new(ListNode { data, next: head }));
    }
    head
}

fn to_vec(mut list: &Link) -> Vec<u32> {
    let mut values = Vec::new();
    while let Some(node) = list {
        values.push(node.data);
        list = &node.next;
    }
    values
}

fn main() {
    let lists = vec![from_slice(&[1, 4, 5]), from_slice(&[1, 3, 4]), from_slice(&[2, 6])];
    println!("{:?}", to_vec(&merge_k_lists(lists)));
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn merges_three_lists() {
        let lists = vec![from_slice(&[1, 4, 5]), from_slice(&[1, 3, 4]), from_slice(&[2, 6])];
        assert_eq!(to_vec(&merge_k_lists(lists)), vec![1, 1, 2, 3, 4, 4, 5, 6]);
    }
}

```

```

    }

    #[test]
    fn handles_empty_input_and_empty_lists() {
        assert!(merge_k_lists(vec![]).is_none());
        let merged = merge_k_lists(vec![None, from_slice(&[2]), None]);
        assert_eq!(to_vec(&merged), vec![2]);
    }

    #[test]
    fn handles_an_odd_number_of_lists() {
        let lists: Vec<_> = (0..5).map(|i| from_slice(&[i, i + 10])).collect();
        assert_eq!(to_vec(&merge_k_lists(lists)), vec![0, 1, 2, 3, 4, 10, 11, 12, 13, 14]);
    }
;
}
}

```

`to_vec` takes `&Link` and follows the nodes with `list = &node.next`, so the tests can read a list without consuming it.

10.4.3 The shortest form, with `Option::insert`

Listing 10.8 The merge and the loop (lines 8 to 41). `src/bin/merge_k_sorted_list_zero_copy.rs`

```

fn merge_two(mut left: NodeLink, mut right: NodeLink) -> NodeLink {
    let mut dummy = Box::new(Node { val: 0, next: None });
    let mut tail = &mut dummy;

    while let (Some(left_head), Some(right_head)) = (&left, &right) {
        let smaller = if left_head.val <= right_head.val { &mut left } else { &mut right };
        let mut node = smaller.take().unwrap(); // safe: checked by while let
        *smaller = node.next.take();
        tail = tail.next.insert(node);
    }

    tail.next = if left.is_some() { left } else { right };
    dummy.next
}

fn merge_k(mut lists: Vec<NodeLink>) -> NodeLink {
    if lists.is_empty() {
        return None;
    }

    let list_count = lists.len();
    let mut gap = 1;
    while gap < list_count {
        let mut index = 0;
        while index + gap < list_count {
            let left = lists[index].take();
            let right = lists[index + gap].take();
            lists[index] = merge_two(left, right);
            index += gap * 2;
        }
    }
}

```

```

        gap *= 2;
    }
    lists[0].take()
}

```

This version keeps the owned `merge_two` from listing 10.6 and brings back the dummy head. It shortens the tail bookkeeping with one method:

```
tail = tail.next.insert(node);
```

`Option::insert` stores a value in the option and returns a mutable reference to the stored value. So one call attaches the node and moves `tail` onto it. Listing 10.2 needed two lines and an `unwrap` for the same step.

Every node in the output is a box that was in the input, moved by pointer. The only allocation is the dummy node, once per `merge_two` call. The single `unwrap` has a comment saying why it cannot fail. The `while let` has already checked that both lists have a front node.

Listing 10.9 The complete program. `src/bin/merge_k_sorted_list_zero_copy.rs`

```

type NodeLink = Option<Box<Node>>;

struct Node {
    val: i32,
    next: NodeLink,
}

fn merge_two(mut left: NodeLink, mut right: NodeLink) -> NodeLink {
    let mut dummy = Box::new(Node { val: 0, next: None });
    let mut tail = &mut dummy;

    while let (Some(left_head), Some(right_head)) = (&left, &right) {
        let smaller = if left_head.val <= right_head.val { &mut left } else { &mut right };
        let mut node = smaller.take().unwrap(); // safe: checked by while let
        *smaller = node.next.take();
        tail = tail.next.insert(node);
    }

    tail.next = if left.is_some() { left } else { right };
    dummy.next
}

fn merge_k(mut lists: Vec<NodeLink>) -> NodeLink {
    if lists.is_empty() {
        return None;
    }

    let list_count = lists.len();
    let mut gap = 1;
    while gap < list_count {

```

```

        let mut index = 0;
        while index + gap < list_count {
            let left = lists[index].take();
            let right = lists[index + gap].take();
            lists[index] = merge_two(left, right);
            index += gap * 2;
        }
        gap *= 2;
    }
    lists[0].take()
}

fn build(values: &[i32]) -> NodeLink {
    let mut head = None;
    for &val in values.iter().rev() {
        head = Some(Box::new(Node { val, next: head }));
    }
    head
}

fn print(mut current: &NodeLink) {
    while let Some(node) = current {
        print!("{}", node.val);
        current = &node.next;
    }
    println();
}

fn main() {
    let lists = vec![build(&[1, 4, 7]), build(&[2, 5, 8]), build(&[0, 3, 6, 9]), None];
    print(&merge_k(lists)); // 0 1 2 3 4 5 6 7 8 9

    print(&merge_k(vec![build(&[1, 1, 5]), build(&[1, 3])])); // 1 1 1 3 5
    print(&merge_k(vec![])); // (empty line)

    let five = vec![build(&[9]), build(&[1, 5]), build(&[3]), build(&[2, 8]), build(&[4])]
    ;
    print(&merge_k(five)); // 1 2 3 4 5 8 9
}

```

`main` tries four cases: four lists including an empty one, duplicates across lists, no lists at all, and five lists.

```

$ cargo run --bin merge_k_sorted_list_zero_copy
0 1 2 3 4 5 6 7 8 9
1 1 1 3 5

1 2 3 4 5 8 9

```

The empty line is the output for no lists.

10.5 The same merge on an enum node

Chapter 9 also wrote a list as an enum. Two versions try the merge on that shape.

10.5.1 An enum with its own `take`

Listing 10.10 The node type (lines 1 to 14). `src/bin/merge_k_sorted_lists_enum.rs`

```
use ListNode::{Empty, Node};

type LinkTo<T> = Box<T>;

enum ListNode {
    Node { data: u32, next: LinkTo<ListNode> },
    Empty,
}

impl ListNode {
    fn take(&mut self) -> ListNode {
        std::mem::replace(self, Empty)
    }
}
```

`use ListNode::{Empty, Node};` lets the code write `Empty` instead of `ListNode::Empty`. The enum gets its own `take`, built on `mem::replace`, so it can be used like `Option::take`.

Listing 10.11 The merge (lines 36 to 61).

```
fn merge_two(left: &mut ListNode, right: &mut ListNode) -> ListNode {
    let mut left = left.take();
    let mut right = right.take();
    let mut merged = Empty;
    let mut tail = &mut merged;

    loop {
        let smaller = match (&left, &right) {
            (Empty, _) => { *tail = right; break; }
            (_, Empty) => { *tail = left; break; }
            (Node { data: l, .. }, Node { data: r, .. }) => {
                if l <= r { &mut left } else { &mut right }
            }
        };
        if let Node { data, next } = smaller.take() {
            *smaller = *next;
            *tail = Node { data, next: LinkTo::new(Empty) };
            if let Node { next, .. } = tail {
                tail = next;
            }
        }
    }
}
```



```
    merged
}
```

The loop picks `smaller` with a `match` on both fronts. The two `Empty` arms attach the other list and leave the loop with `break`. The third arm returns a mutable reference to the list with the smaller front.

This representation has a cost. Look at how a node is attached:

```
*tail = Node { data, next: LinkTo::new(Empty) };
```

With `Option<Box<Node>>`, a node's box moves from the input to the output. Here, `*smaller = *next` moves the rest of the input out of its box, and that box is freed. The output node is a new `Node` with a new `Box` holding a new `Empty`. So this version allocates once for every node it outputs.

Listing 10.12 The complete program. `src/bin/merge_k_sorted_lists_enum.rs`

```
use ListNode::{Empty, Node};

type LinkTo<T> = Box<T>;

enum ListNode {
    Node { data: u32, next: LinkTo<ListNode> },
    Empty,
}

impl ListNode {
    fn take(&mut self) -> ListNode {
        std::mem::replace(self, Empty)
    }
}

struct MergeKSorted {}

impl MergeKSorted {
    pub fn merge_k_lists(mut lists: Vec<ListNode>) -> ListNode {
        if lists.is_empty() { return Empty; }

        let (n, mut interval) = (lists.len(), 1);
        while interval < n {
            let mut i = 0;
            while i + interval < n {
                let (left, right) = lists.split_at_mut(i + interval);
                left[i] = Self::merge_two(&mut left[i], &mut right[0]);
                i += interval * 2;
            }
            interval *= 2;
        }
    }
}
```

```

        lists[0].take()
    }

    fn merge_two(left: &mut ListNode, right: &mut ListNode) -> ListNode {
        let mut left = left.take();
        let mut right = right.take();
        let mut merged = Empty;
        let mut tail = &mut merged;

        loop {
            let smaller = match (&left, &right) {
                (Empty, _) => { *tail = right; break; }
                (_, Empty) => { *tail = left; break; }
                (Node { data: l, .. }, Node { data: r, .. }) => {
                    if l <= r { &mut left } else { &mut right }
                }
            };

            if let Node { data, next } = smaller.take() {
                *smaller = *next;
                *tail = Node { data, next: LinkTo::new(Empty) };
                if let Node { next, .. } = tail {
                    tail = next;
                }
            }

            merged
        }
    }

    fn main() {
        let mut lists = Vec::new();
        for values in [vec![1, 4, 5], vec![1, 3, 4], vec![2, 6]] {
            let mut head = Empty;
            for data in values.into_iter().rev() {
                head = Node { data, next: LinkTo::new(head) };
            }
            lists.push(head);
        }

        let merged = MergeKSorted::merge_k_lists(lists);

        let mut current = &merged;
        while let Node { data, next } = current {
            print!("{data} ");
            current = next;
        }
        println();
    }
}

```

10.5.2 An enum with standard traits and a recursive merge

Listing 10.13 The type and its conversions (lines 1 to 20). `src/bin/merge_k_sorted_list_easy.rs`

```
pub type NodeLink = Box<ListNode>;

#[derive(Debug, PartialEq, Eq, Default)]
pub enum ListNode {
    #[default]
    Empty,
    Node(i32, NodeLink),
}

impl From<Vec<i32>> for NodeLink {
    fn from(vec: Vec<i32>) -> Self {
        vec.into_iter().rev().fold(NodeLink::default(), |acc, v| NodeLink::new(ListNode::N
ode(v, acc)))
    }
}

impl From<Vec<i32>> for ListNode {
    fn from(vec: Vec<i32>) -> Self {
        *NodeLink::from(vec)
    }
}
```

This version uses standard traits in place of hand-written helpers:

- `#[derive(Default)]` with `#[default]` on `Empty` makes the empty list the default value. So `NodeLink::default()` is a box holding `Empty`, and `std::mem::take` can be used instead of a custom `take`.
- `impl From<Vec<i32>> for NodeLink` builds a list from a vector. It walks the vector backward with `rev()` and uses `fold` to wrap each value around the list built so far. `main` calls it as `vec![1, 4, 5].into()`.

Listing 10.14 The interval loop and a recursive merge (lines 22 to 53).

```
impl ListNode {
    pub fn merge_k_lists(mut lists: Vec<NodeLink>) -> NodeLink {
        if lists.is_empty() {
            return NodeLink::default();
        }
        let mut interval = 1;
        while interval < lists.len() {
            for i in (0..lists.len()).step_by(interval * 2) {
                if i + interval < lists.len() {
                    let right = std::mem::take(&mut lists[i + interval]);
                    let left = std::mem::take(&mut lists[i]);
                    lists[i] = Self::merge(left, right);
                }
            }
            interval *= 2;
        }
    }
}
```

```

        interval *= 2;
    }
    lists.swap_remove(0)
}

fn merge(l1: NodeLink, l2: NodeLink) -> NodeLink {
    match (*l1, *l2) {
        (ListNode::Empty, l) | (l, ListNode::Empty) => NodeLink::new(l),
        (ListNode::Node(v1, n1), ListNode::Node(v2, n2)) => {
            if v1 <= v2 {
                NodeLink::new(ListNode::Node(v1, Self::merge(n1, NodeLink::new(ListNode::Node(v2, n2)))))
            } else {
                NodeLink::new(ListNode::Node(v2, Self::merge(NodeLink::new(ListNode::Node(v1, n1)), n2)))
            }
        }
    }
}

```

`std::mem::take(&mut lists[i + interval])` moves the right list out and leaves the default, an empty list, in its place. It takes the right list before the left, which avoids the double borrow as listing 10.4 did. `lists.swap_remove(0)` removes position 0 by moving the last element into it, which takes $O(1)$.

`merge` is recursive. It moves both nodes out of their boxes with `match (*l1, *l2)`. Then it builds the output node around a recursive call that merges the rest. The code reads almost like a definition of merging. It has two costs. It allocates a new `Box` for every output node. And it recurses once per output node, so a long result uses a deep stack, as chapter 9's recursive drop did.

Listing 10.15 The complete program. `src/bin/merge_k_sorted_list_easy.rs`

```

pub type NodeLink = Box<ListNode>;

#[derive(Debug, PartialEq, Eq, Default)]
pub enum ListNode {
    #[default]
    Empty,
    Node(i32, NodeLink),
}

impl From<Vec<i32>> for NodeLink {
    fn from(vec: Vec<i32>) -> Self {
        vec.into_iter().rev().fold(NodeLink::default(), |acc, v| NodeLink::new(ListNode::Node(v, acc)))
    }
}

impl From<Vec<i32>> for ListNode {
    fn from(vec: Vec<i32>) -> Self {

```

```

        *NodeLink::from(vec)
    }
}

impl ListNode {
    pub fn merge_k_lists(mut lists: Vec<NodeLink>) -> NodeLink {
        if lists.is_empty() {
            return NodeLink::default();
        }
        let mut interval = 1;
        while interval < lists.len() {
            for i in (0..lists.len()).step_by(interval * 2) {
                if i + interval < lists.len() {
                    let right = std::mem::take(&mut lists[i + interval]);
                    let left = std::mem::take(&mut lists[i]);
                    lists[i] = Self::merge(left, right);
                }
            }
            interval *= 2;
        }
        lists.swap_remove(0)
    }

    fn merge(l1: NodeLink, l2: NodeLink) -> NodeLink {
        match (*l1, *l2) {
            (ListNode::Empty, l) | (l, ListNode::Empty) => NodeLink::new(l),
            (ListNode::Node(v1, n1), ListNode::Node(v2, n2)) => {
                if v1 <= v2 {
                    NodeLink::new(ListNode::Node(v1, Self::merge(n1, NodeLink::new(ListNode::Node(v2, n2)))))
                } else {
                    NodeLink::new(ListNode::Node(v2, Self::merge(NodeLink::new(ListNode::Node(v1, n1)), n2)))
                }
            }
        }
    }
}

fn main() {
    let lists: Vec<NodeLink> = vec![
        vec![1, 4, 5].into(),
        vec![1, 3, 4].into(),
        vec![2, 6].into(),
    ];

    let merged = ListNode::merge_k_lists(lists);

    println!("{:?}", merged);
}

```

The program prints the result with `{:#?}`, which shows the nesting of the enum directly:

```

$ cargo run --bin merge_k_sorted_list_easy
Node(
  1,
  Node(

```

```
1,
Node(
  2,
  ...
```

10.6 A queue of lists

Listing 10.16 The complete program. `src/bin/merge_k_sorted_lists_pairs.rs`

```
use std::collections::VecDeque;

struct ListNode {
    data: u32,
    next: Option<Box<ListNode>>,
}

type Link = Option<Box<ListNode>>;

fn merge_k_lists(lists: Vec<Link>) -> Link {
    let mut queue: VecDeque<Link> = lists.into();
    while queue.len() > 1 {
        let a = queue.pop_front().unwrap();
        let b = queue.pop_front().unwrap();
        queue.push_back(merge_two(a, b));
    }
    queue.pop_front().flatten()
}

fn merge_two(mut a: Link, mut b: Link) -> Link {
    let mut dummy = Box::new(ListNode { data: 0, next: None });
    let mut tail = &mut dummy;

    while let (Some(x), Some(y)) = (&a, &b) {
        let pick = if x.data <= y.data { &mut a } else { &mut b };
        let mut node = pick.take().unwrap();
        *pick = node.next.take();
        tail = tail.next.insert(node);
    }
    tail.next = a.or(b);
    dummy.next
}

fn main() {
    let mut lists = Vec::new();
    for values in [vec![1, 4, 5], vec![1, 3, 4], vec![2, 6]] {
        let mut head = None;
        for data in values.into_iter().rev() {
            head = Some(Box::new(ListNode { data, next: head }));
        }
        lists.push(head);
    }

    let mut cur = merge_k_lists(lists);
    while let Some(node) = cur {
        print!("{}", node.data);
    }
}
```

```

        cur = node.next;
    }
    println!();
}

```

This version does pairwise rounds with a queue instead of intervals. A `VecDeque` is a double-ended queue: you can push and pop at both ends in $O(1)$. `merge_k_lists` puts all the lists in the queue. Then it pops two from the front and pushes their merge on the back, until one list is left.

Because merged lists go to the back, every original list is merged once before any merged list is merged again. So the rounds are the same as in figure 10.2, and the cost is $O(N \log k)$.

`queue.pop_front().flatten()` needs a word of explanation. `pop_front` returns `Option<Link>`, and a `Link` is itself an `Option<Box<ListNode>>`. So the value is an `Option<Option<...>>`. `flatten` turns it into one `Option`: `None` if the queue was empty, or the merged list.

```

$ cargo run --bin merge_k_sorted_lists_pairs
1 1 2 3 4 4 5 6

```

10.7 A heap of list heads

Listing 10.17 The complete program. `src/bin/merge_k_sorted_lists_heap.rs`

```

use std::cmp::Reverse;
use std::collections::BinaryHeap;

struct ListNode {
    data: u32,
    next: Option<Box<ListNode>>,
}

type Link = Option<Box<ListNode>>;

fn merge_k_lists(mut lists: Vec<Link>) -> Link {
    let mut heap = BinaryHeap::new();
    for (i, list) in lists.iter().enumerate() {
        if let Some(node) = list {
            heap.push(Reverse((node.data, i)));
        }
    }

    let mut dummy = Box::new(ListNode { data: 0, next: None });
    let mut tail = &mut dummy;

    while let Some(Reverse((_, i))) = heap.pop() {

```

```

        let mut node = lists[i].take().unwrap();
        lists[i] = node.next.take();
        if let Some(next) = &lists[i] {
            heap.push(Reverse((next.data, i)));
        }
        tail = tail.next.insert(node);
    }
    dummy.next
}

fn main() {
    let mut lists = Vec::new();
    for values in [vec![1, 4, 5], vec![1, 3, 4], vec![2, 6]] {
        let mut head = None;
        for data in values.into_iter().rev() {
            head = Some(Box::new(ListNode { data, next: head }));
        }
        lists.push(head);
    }

    let mut cur = merge_k_lists(lists);
    while let Some(node) = cur {
        print!("{}", node.data);
        cur = node.next;
    }
    println!();
}

```

This is the third strategy from figure 10.2. The heap does not hold nodes. It holds pairs of `(front value, list number)`, wrapped in `Reverse` to make it a min-heap, as in chapter 5. Storing the list number instead of the node keeps the nodes in their lists, and avoids needing an ordering on nodes.

Each pass of the loop:

1. Pops the smallest pair. Its second field, `i`, says which list holds that value.
2. Takes the front node out of `lists[i]` and puts the rest of that list back.
3. If the list still has a node, pushes that node's value with `i`.
4. Attaches the taken node to the output with `tail.next.insert(node)`.

When two fronts have equal values, the pairs compare by list number, so the lower-numbered list goes first.

The heap needs only the current front of each list. That makes this strategy the right one when the lists are too large to hold in memory. An example is `k` sorted files read line by line.

10.8 Recursion on halves

The last two versions do pairwise merging from the top down. They split the lists into two halves, merge each half recursively, and merge the two results.

Listing 10.18 A recursive two-list merge (lines 9 to 22). `src/bin/merrgmerge_k_sorted_recursion.rs`

```
// Merge two sorted lists
fn merge(a: List, b: List) -> List {
    match (a, b) {
        (None, rest) | (rest, None) => rest,
        (Some(mut a), Some(mut b)) => {
            if a.val > b.val {
                std::mem::swap(&mut a, &mut b); // a is now the smaller head
            }
            a.next = merge(a.next.take(), Some(b));
            Some(a)
        }
    }
}
```

The first arm, `(None, rest) | (rest, None) => rest`, handles an empty list on either side: the other list is the answer. In the second arm, `mem::swap` makes `a` the list with the smaller front. Then `a`'s node stays at the front, and its `next` becomes the merge of the rest of `a` with `b`. No node is allocated; the boxes move.

Listing 10.19 Two ways to split the lists (lines 24 to 47).

```
fn merge_k(mut lists: Vec<List>) -> List {
    match lists.len() {
        0 => None,
        1 => lists.pop().unwrap(),
        n => {
            let right = lists.split_off(n / 2); // lists = left half
            merge(merge_k(lists), merge_k(right))
        }
    }
}

// Merge k sorted lists using divide and conquer without heap allocation
fn merge_k_slice(lists: &mut [List]) -> List {
    match lists.len() {
        0 => None,
        1 => lists[0].take(),
        _ => {
            let mid = lists.len() / 2;
            let (left, right) = lists.split_at_mut(mid);
            merge(merge_k_slice(left), merge_k_slice(right))
        }
    }
}
```

```
    }
}
```

`merge_k` splits the vector with `split_off(n / 2)`, which moves the second half into a new `Vec`. That allocates a new vector at every level of the recursion.

`merge_k_slice` works on a slice, `&mut [List]`, and splits it with `split_at_mut`, the method from section 10.3. Splitting a slice only creates two views of the same memory, so this version allocates nothing to split.

Listing 10.20 Building a list front to back (lines 49 to 68).

```
// fn from_vec(v: Vec<i32>) -> List {
//     let mut head = None;
//     for val in v.into_iter().rev() {
//         head = Some(Box::new(ListNode { val, next: head }));
//     }
//     head
// }

fn from_vec(v: Vec<i32>) -> List {
    let mut head = None;
    let mut tail = &mut head; // points to the empty slot at the end

    for val in v {
        let node = Box::new(ListNode { val, next: None });
        tail = &mut tail.insert(node).next; // fill the slot, move to the new node's next
    }

    head
}
```

The commented-out `from_vec` builds the list backward, as `from_slice` did in chapter 9. The active one builds it forward, with a slot pointer like listing 10.6. `tail = &mut tail.insert(node).next` fills the empty slot and moves `tail` to the new node's `next` slot.

Listing 10.21 The complete program. `src/bin/merrgmerge_k_sorted_recursion.rs`

```
#[derive(Debug)]
struct ListNode {
    val: i32,
    next: Option<Box<ListNode>>,
}

type List = Option<Box<ListNode>>;

// Merge two sorted lists
fn merge(a: List, b: List) -> List {
    match (a, b) {
        (None, rest) | (rest, None) => rest,
```

```

        (Some(mut a), Some(mut b)) => {
            if a.val > b.val {
                std::mem::swap(&mut a, &mut b); // a is now the smaller head
            }
            a.next = merge(a.next.take(), Some(b));
            Some(a)
        }
    }
}

// Merge k sorted lists using divide and conquer
fn merge_k(mut lists: Vec<List>) -> List {
    match lists.len() {
        0 => None,
        1 => lists.pop().unwrap(),
        n => {
            let right = lists.split_off(n / 2); // lists = left half
            merge(merge_k(lists), merge_k(right))
        }
    }
}

// Merge k sorted lists using divide and conquer without heap allocation
fn merge_k_slice(lists: &mut [List]) -> List {
    match lists.len() {
        0 => None,
        1 => lists[0].take(),
        _ => {
            let mid = lists.len() / 2;
            let (left, right) = lists.split_at_mut(mid);
            merge(merge_k_slice(left), merge_k_slice(right))
        }
    }
}

// Helper: build a list from a Vec
// fn from_vec(v: Vec<i32>) -> List {
//     let mut head = None;
//     for val in v.into_iter().rev() {
//         head = Some(Box::new(ListNode { val, next: head }));
//     }
//     head
// }

fn from_vec(v: Vec<i32>) -> List {
    let mut head = None;
    let mut tail = &mut head; // points to the empty slot at the end

    for val in v {
        let node = Box::new(ListNode { val, next: None });
        tail = &mut tail.insert(node).next; // fill the slot, move to the new node's next
    }

    head
}

// Helper: turn a list back into a Vec for printing
fn to_vec(mut cur: &List) -> Vec<i32> {
    let mut out = Vec::new();

```

```

    while let Some(node) = cur {
        out.push(node.val);
        cur = &node.next;
    }
    out
}

fn main() {
    let lists = vec![
        from_vec(vec![1, 4, 5]),
        from_vec(vec![1, 3, 4]),
        from_vec(vec![2, 6]),
    ];
    let merged = merge_k(lists);
    println!("{:?}", to_vec(&merged)); // [1, 1, 2, 3, 4, 4, 5, 6]

    // Edge cases
    println!("{:?}", to_vec(&merge_k(vec![]))); // []
    println!("{:?}", to_vec(&merge_k(vec![None, None]))); // []
    println!("{:?}", to_vec(&merge_k(vec![from_vec(vec![7])]))); // [7]

    let v = [1,2,3,4,5];

    match v {
        [1, ..] => println!("it starts with one thing..."),
        [_, ..] => println!("I dunno why!"),
    }
}

```

`main` ends with a slice pattern, `[1, ..]`, which matches any array whose first element is 1.

```

$ cargo run --bin merrgmerge_k_sorted_recursion
[1, 1, 2, 3, 4, 4, 5, 6]
[]
[]
[7]
it starts with one thing...

```

The last version keeps only the slice-based recursion:

Listing 10.22 The complete program. `src/bin/mergek_sortes_list_chrush_lee.rs`

```

use std::mem;

#[derive(Debug)]
struct ListNode {
    val: i32,
    next: Option<Box<ListNode>>,
}

type List = Option<Box<ListNode>>;

fn merge(a: List, b: List) -> List {
    match (a, b) {
        (None, rest) | (rest, None) => rest,
    }
}

```

```

        (Some(mut left), Some(mut right)) => {
            if left.val > right.val {
                mem::swap(&mut left, &mut right);
            }
            left.next = merge(left.next.take(), Some(right));
            Some(left)
        }
    }
}

fn merge_k_slice(lists: &mut [List]) -> List {
    match lists.len() {
        0 => None,
        1 => lists[0].take(),
        _ => {
            let mid = lists.len() / 2;
            let (left_half, right_half) = lists.split_at_mut(mid);
            merge(merge_k_slice(left_half), merge_k_slice(right_half))
        }
    }
}

fn create_list(arr: &[i32]) -> List {
    let mut head = None;
    let mut tail = &mut head;

    for &val in arr {
        *tail = Some(Box::new(ListNode { val, next: None }));
        if let Some(node) = tail {
            tail = &mut node.next;
        }
    }

    head
}

fn print_list(mut list: &List) {
    let mut elements = Vec::new();
    while let Some(node) = list {
        elements.push(node.val.to_string());
        list = &node.next;
    }
    println!("{}", elements.join(" -> "));
}

fn main() {
    let mut lists = vec![
        create_list(&[1, 4, 5]),
        create_list(&[1, 3, 4]),
        create_list(&[2, 6]),
    ];

    println!("Input lists:");
    for list in &lists {
        print_list(list);
    }

    let merged = merge_k_slice(&mut lists);

```

```
println!("\nMerged list:");
print_list(&merged);
}
```

`create_list` builds front to back with the slot pointer, advancing it with `if let Some(node) = tail`. `print_list` collects the values as strings and joins them with `" -> "`.

```
$ cargo run --bin mergek_sortes_list_chrush_lee
Input lists:
[1 -> 4 -> 5]
[1 -> 3 -> 4]
[2 -> 6]

Merged list:
[1 -> 1 -> 2 -> 3 -> 4 -> 4 -> 5 -> 6]
```

Both recursive merges recurse once for each node of their output. For short lists that is fine. For lists with hundreds of thousands of nodes, the iterative merges of sections 10.3 and 10.4 are safer.

10.9 The ten versions side by side

Program	Strategy	Moves or allocates nodes	Borrowing technique
<code>merge_k_sorted_lists_divide</code>	interval rounds	moves	<code>split_at_mut</code>
<code>merge_k_sorted_lists_swap</code>	interval rounds	moves	take the right list first
<code>merge_k_sorted_lists_simple</code>	interval rounds	moves	owned lists, tail slot
<code>merge_k_sorted_list_zero_copy</code>	interval rounds	moves	owned lists, insert
<code>merge_k_sorted_lists_enum</code>	interval rounds	allocates per node	custom <code>take</code>
<code>merge_k_sorted_list_easy</code>	interval rounds	allocates per node	<code>mem::take</code> , recursive merge
<code>merge_k_sorted_lists_pairs</code>	queue of lists	moves	owned lists
<code>merge_k_sorted_lists_heap</code>	heap of heads	moves	list numbers in the heap
<code>merrgemerge_k_sorted_recursion</code>	recursive halves	moves	<code>split_off</code> , then <code>split_at_mut</code>
<code>mergek_sortes_list_chrush_lee</code>	recursive halves	moves	<code>split_at_mut</code>

All except the first strategy of figure 10.2 run in $O(N \log k)$. The differences are in memory. Some versions move nodes and some reallocate them. The recursive merges use stack in proportion to the output. And the extra memory ranges from $O(1)$ for the interval loop to $O(k)$ for the queue and the heap.

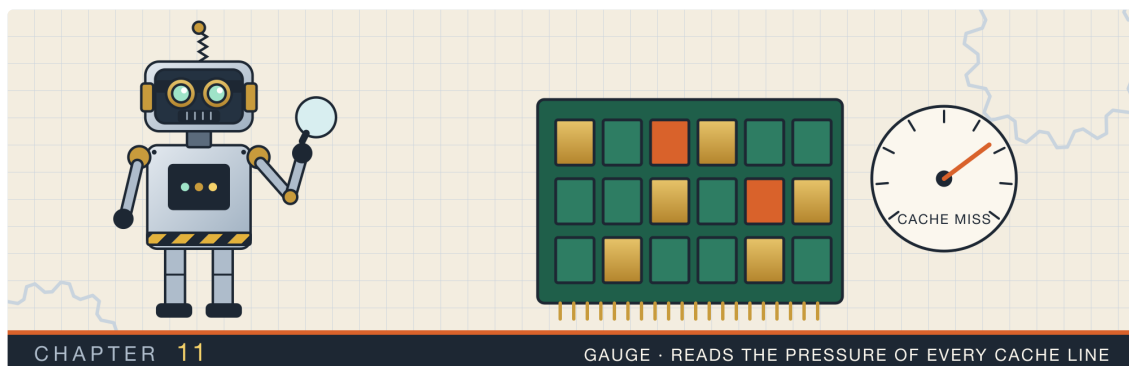
Summary

- Merging two sorted lists moves the smaller front node to the output until one list is empty, in $O(a + b)$.
- Merging k lists one at a time costs $O(kN)$. Pairwise rounds and a heap of heads both cost $O(N \log k)$.
- Two `&mut` borrows of elements of one `Vec` do not compile. `split_at_mut` splits it into two slices, or you can move one list out with `take()` first.
- A dummy head and a tail pointer attach nodes to the end of a list. `Option::insert` attaches and advances in one call.
- With `Option<Box<Node>>`, merging moves boxes and allocates nothing. The enum versions allocate a new box for every node.
- A heap of `(value, list number)` pairs needs only the front of each list, which suits lists read from disk.
- Recursive merges are short but recurse once per output node.

Chapter 11 moves from lists to trees, where each node has up to two children. It ends with tries, where each node has one child per letter.

Exercises

1. Write the "one at a time" merge and time it against listing 10.9 on 200 lists of 5,000 nodes each.
2. Make `merge_two` from listing 10.8 generic over any `T: Ord`.
3. Write a k -way merge of sorted text files with the heap strategy, keeping only one line per file in memory.
4. Count allocations in listings 10.9 and 10.12 with a custom global allocator that increments a counter, and confirm the difference described in section 10.5.1.



Trees and tries

THIS CHAPTER COVERS

- The vocabulary of trees: root, leaf, child, subtree, depth
- A binary tree as an enum, and the three depth-first visiting orders
- Binary search trees: insert, search, remove, and range queries, written two ways
- Trees with any number of children: depth-first and breadth-first visits, and grouping by level
- Saving a tree to a file and loading it back
- A trie, a tree for finding words by their first letters

In a linked list, each node points to at most one next node. In a **tree**, each node can point to several nodes, called its children. Trees describe anything organized as a hierarchy. Examples are folders on a disk, the parts of an expression, and a sorted set of numbers arranged for fast search.

Trees are easier to write in Rust than the doubly linked list of chapter 9. Each node has exactly one parent, so a parent can own its children outright, with `Box` or with a `Vec`. No `Rc` or `RefCell` is needed. The harder parts are elsewhere: replacing a node from inside a method, and choosing between recursion and an explicit stack or queue.

The chapter starts with vocabulary, then binary trees, then binary search trees, then trees with any number of children, and ends with the trie.

11.1 Tree vocabulary

Figure 11.1 names the parts of a tree.

Tree terms, on the tree used by the tests

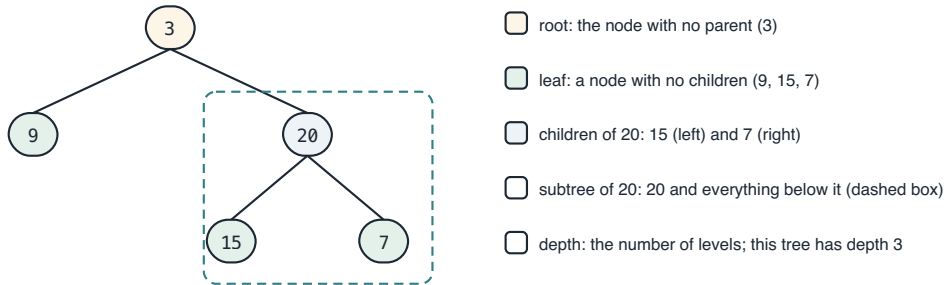


Figure 11.1 The small tree used by this chapter's tests, with its parts labeled.

- The **root** is the one node with no parent. Trees are drawn with the root at the top.
- A **leaf** is a node with no children.
- The **subtree** of a node is that node and everything below it. Every subtree is a tree itself. So a function on a tree can call itself on each subtree, which is recursion.
- The **depth** of a tree is the number of levels from the root down to its deepest leaf.

A **binary tree** is a tree in which each node has at most two children, called the left child and the right child.

11.2 A binary tree as an enum

A binary tree is either empty, or a node holding a value and two smaller binary trees. That sentence translates directly into an enum:

Listing 11.1 The tree type (lines 1 to 16). src/problems/binary_tree.rs

```

//! Binary tree as an algebraic data type.
//!
//! ```text
//! Tree = Empty
//!      | Node { value, left: Box<Tree>, right: Box<Tree> }
//! ```
```

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Tree {
    Empty,
    Node {
        value: i32,
        left: Box<Tree>,
        right: Box<Tree>,
    },
}
```

The children are `Box<Tree>` for the reason chapter 8 gave. A `Tree` cannot contain two `Tree` values directly, because its size would have no end. A `Box` is one pointer, so a node has a fixed size.

11.2.1 Depth and inversion

Listing 11.2 Depth and inversion (lines 19 to 35).

```
pub fn max_depth(&self) -> usize {
    match self {
        Tree::Empty => 0,
        Tree::Node { left, right, .. } => 1 + left.max_depth().max(right.max_depth()),
    }
}

pub fn invert(&mut self) {
    match self {
        Tree::Empty => {}
        Tree::Node { left, right, .. } => {
            left.invert();
            right.invert();
            std::mem::swap(left, right);
        }
    }
}
```

Each method is a `match` with two arms. The `Empty` arm is the **base case**, the input small enough to answer directly. The `Node` arm calls the method on the children and combines the results.

`max_depth` reads as a definition. An empty tree has depth 0. A node has depth 1 plus the larger depth of its two subtrees. For the tree in figure 11.1: the leaves have depth 1, node 20 has depth 2, and the root has depth 3.

`invert` swaps every node's left and right children, so the tree becomes its mirror image. It inverts both subtrees first and then calls `std::mem::swap(left, right)`. The pattern binds

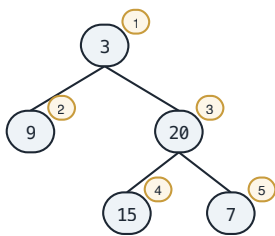
`left` and `right` as mutable references to the boxes, so the swap exchanges two pointers and moves no nodes.

11.2.2 The three depth-first orders

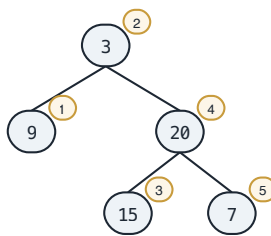
Visiting every node of a tree once is called a **traversal**. Going down one branch as far as possible before trying the next is **depth-first** traversal. There are three common depth-first orders. They differ only in when a node is visited relative to its children (figure 11.2).

The order each traversal visits the same tree (badges show the visit order)

pre-order: node, left, right



in-order: left, node, right



post-order: left, right, node

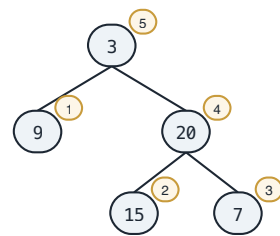


Figure 11.2 The badges show the order in which each traversal visits the nodes.

- **Pre-order** visits the node first, then its left subtree, then its right subtree.
- **In-order** visits the left subtree, then the node, then the right subtree.
- **Post-order** visits both subtrees, then the node.

Listing 11.3 The public traversal methods (lines 37 to 53).

```

pub fn preorder(&self) -> Vec<i32> {
    let mut out = Vec::new();
    self.collect_preorder(&mut out);
    out
}

pub fn inorder(&self) -> Vec<i32> {
    let mut out = Vec::new();
    self.collect_inorder(&mut out);
    out
}

pub fn postorder(&self) -> Vec<i32> {
    let mut out = Vec::new();
    self.collect_postorder(&mut out);
    out
}

```

Each public method creates an empty `Vec`, passes it to a private helper, and returns it. Creating the vector once and passing it down avoids building a new vector at every node.

Listing 11.4 The recursive helpers (lines 55 to 86).

```
fn collect_preorder(&self, out: &mut Vec<i32>) {
    match self {
        Tree::Empty => {}
        Tree::Node { value, left, right } => {
            out.push(*value);
            left.collect_preorder(out);
            right.collect_preorder(out);
        }
    }
}

fn collect_inorder(&self, out: &mut Vec<i32>) {
    match self {
        Tree::Empty => {}
        Tree::Node { value, left, right } => {
            left.collect_inorder(out);
            out.push(*value);
            right.collect_inorder(out);
        }
    }
}

fn collect_postorder(&self, out: &mut Vec<i32>) {
    match self {
        Tree::Empty => {}
        Tree::Node { value, left, right } => {
            left.collect_postorder(out);
            right.collect_postorder(out);
            out.push(*value);
        }
    }
}
```

The three helpers are identical except for one line: where `out.push(*value)` appears relative to the two recursive calls. `*value` copies the `i32` out of the reference.

Listing 11.5 The complete file, with its tests. `src/problems/binary_tree.rs`

```
///! Binary tree as an algebraic data type.
///!
///! ```text
///! Tree = Empty
///!      | Node { value, left: Box<Tree>, right: Box<Tree> }
///! ```
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Tree {
    Empty,
    Node {
```

```

        value: i32,
        left: Box<Tree>,
        right: Box<Tree>,
    },
}

impl Tree {
    pub fn max_depth(&self) -> usize {
        match self {
            Tree::Empty => 0,
            Tree::Node { left, right, .. } => 1 + left.max_depth().max(right.max_depth()),
        }
    }

    pub fn invert(&mut self) {
        match self {
            Tree::Empty => {}
            Tree::Node { left, right, .. } => {
                left.invert();
                right.invert();
                std::mem::swap(left, right);
            }
        }
    }

    pub fn preorder(&self) -> Vec<i32> {
        let mut out = Vec::new();
        self.collect_preorder(&mut out);
        out
    }

    pub fn inorder(&self) -> Vec<i32> {
        let mut out = Vec::new();
        self.collect_inorder(&mut out);
        out
    }

    pub fn postorder(&self) -> Vec<i32> {
        let mut out = Vec::new();
        self.collect_postorder(&mut out);
        out
    }

    fn collect_preorder(&self, out: &mut Vec<i32>) {
        match self {
            Tree::Empty => {}
            Tree::Node { value, left, right } => {
                out.push(*value);
                left.collect_preorder(out);
                right.collect_preorder(out);
            }
        }
    }

    fn collect_inorder(&self, out: &mut Vec<i32>) {
        match self {
            Tree::Empty => {}
            Tree::Node { value, left, right } => {
                left.collect_inorder(out);

```

```

        out.push(*value);
        right.collect_inorder(out);
    }
}

fn collect_postorder(&self, out: &mut Vec<i32>) {
    match self {
        Tree::Empty => {}
        Tree::Node { value, left, right } => {
            left.collect_postorder(out);
            right.collect_postorder(out);
            out.push(*value);
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    fn leaf(value: i32) -> Tree {
        Tree::Node {
            value,
            left: Box::new(Tree::Empty),
            right: Box::new(Tree::Empty),
        }
    }

    fn node(value: i32, left: Tree, right: Tree) -> Tree {
        Tree::Node {
            value,
            left: Box::new(left),
            right: Box::new(right),
        }
    }

    fn sample() -> Tree {
        node(3, leaf(9), node(20, leaf(15), leaf(7)))
    }

    #[test]
    fn max_depth_works() {
        assert_eq!(Tree::Empty.max_depth(), 0);
        assert_eq!(leaf(1).max_depth(), 1);
        assert_eq!(sample().max_depth(), 3);
    }

    #[test]
    fn invert_swaps_subtrees() {
        let mut tree = sample();
        tree.invert();

        match &tree {
            Tree::Node { left, right, .. } => {
                match left.as_ref() {
                    Tree::Node { value, .. } => assert_eq!(*value, 20),
                    _ => panic!("expected left node"),
                }
            }
        }
    }
}

```

```

    }
    match right.as_ref() {
      Tree::Node { value, .. } => assert_eq!(*value, 9),
      _ => panic!("expected right node"),
    }
  }
  _ => panic!("expected node"),
}
}

#[test]
fn traversals_work() {
  let tree = sample();
  assert_eq!(tree.preorder(), vec![3, 9, 20, 15, 7]);
  assert_eq!(tree.inorder(), vec![9, 3, 15, 20, 7]);
  assert_eq!(tree.postorder(), vec![9, 15, 7, 20, 3]);
}
}

```

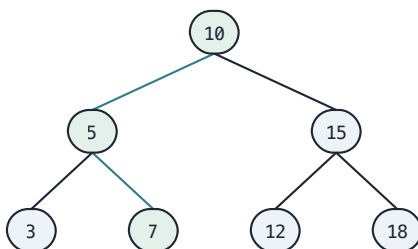
The test module defines two small helpers, `leaf` and `node`, so a test tree can be built in one readable expression. `sample()` builds the tree of figure 11.1. The traversal test checks all three orders from figure 11.2.

11.3 Binary search trees

A **binary search tree** (BST) is a binary tree with an ordering rule. For every node, all values in its left subtree are smaller than the node's value, and all values in its right subtree are larger.

The rule makes searching fast. To find a value, compare it with the root. If it is smaller, it can only be in the left subtree; if larger, only in the right. Each comparison discards one subtree (figure 11.3).

A binary search tree: searching for 7



1. $7 < 10$: go left
2. $7 > 5$: go right
3. $7 == 7$: found, after 3 comparisons

Left subtree: values smaller than the node.
Right subtree: values larger than the node.

Figure 11.3 Searching for 7 visits one node per level. In a balanced tree of n values, that is about $\log_2 n$ comparisons.

The code for this chapter has two implementations of the same BST. They agree on inserting and searching, and differ in how they remove values and answer range queries. I'll go through the first fully, then the parts where the second differs.

A binary search tree: the ordering decides which way to go

Every node is larger than the whole of its left subtree, and smaller than its right.

☒ on the path

☒ placed

☐ not yet placed



Insert 8. The tree is empty, so 8 becomes the root.

The tree keeps one value per node. A search visits one node per level.

value = 8

comparisons = 1

nodes = 1

Animation 11.5 Inserting seven values, then searching for 6. The cream path is the nodes the comparison walks through, and the grey nodes have not been inserted yet. Notice that the tree is built by nothing but comparisons: no value is ever moved after it is placed, and the shape depends on the order the values arrive. A search is the same walk, stopping when the value matches.

11.3.1 The type, insertion, and search

Listing 11.6 The type (lines 1 to 14). src/bin/bst_clean.rs

```
use std::cmp::Ordering;

type TreeLink<T> = Box<BST<T>>;

#[derive(Debug, Clone, Default)]
pub enum BST<T> {
```

```
#[default]
Empty,
Node {
    value: T,
    left: TreeLink<T>,
    right: TreeLink<T>,
},
}
```

`BST<T>` is generic over the value type. The methods require `T: Ord`, meaning values of type `T` can be ordered, which a search tree needs. `#[default]` on `Empty` makes an empty tree the default value, which the removal code uses.

Listing 11.7 Creating, inserting, and searching (lines 17 to 47).

```
pub fn new() -> Self {
    Self::Empty
}

pub fn insert(&mut self, val: T) {
    match self {
        Self::Empty => {
            *self = Self::Node {
                value: val,
                left: Box::new(Self::Empty),
                right: Box::new(Self::Empty),
            };
        }
        Self::Node { value, left, right } => match val.cmp(value) {
            Ordering::Less => left.insert(val),
            Ordering::Greater => right.insert(val),
            Ordering::Equal => {},
        },
    },
}

pub fn contains(&self, val: &T) -> bool {
    match self {
        Self::Empty => false,
        Self::Node { value, left, right } => match val.cmp(value) {
            Ordering::Less => left.contains(val),
            Ordering::Greater => right.contains(val),
            Ordering::Equal => true,
        },
    }
}
```

`insert` takes `&mut self` because it changes the tree in place. It walks down with recursive calls. When it reaches an `Empty` tree, it replaces that spot with a new leaf: `*self = Self::Node { ... }`. On the way down, `val.cmp(value)` compares the new value with the

node's and returns `Ordering::Less`, `Greater`, or `Equal`. A value equal to one already in the tree is ignored, so the tree holds each value once.

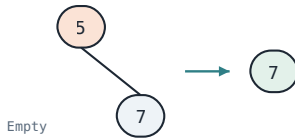
`contains` follows the same path without changing anything.

11.3.2 Removal

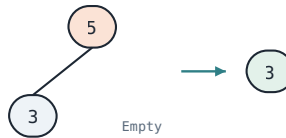
Removing a value is the hardest operation, because the tree must keep its ordering rule. There are three cases (figure 11.4).

The three cases of removing a value

1. no left child: replace the node by its right subtree



2. no right child: replace the node by its left subtree



3. two children: copy in the smallest value on the right

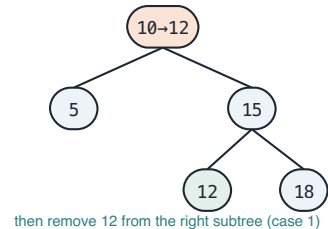


Figure 11.4 The three removal cases. In case 3, removing 10 copies 12 into the root, then removes 12 from the right subtree.

1. **No left child.** Replace the node with its right subtree.
2. **No right child.** Replace the node with its left subtree.
3. **Two children.** Keep the node, but give it a new value: the smallest value in its right subtree. That value is larger than everything on the left and smaller than everything else on the right, so the rule still holds. Then remove that smallest value from the right subtree. The smallest value has no left child, so its removal is case 1.

Listing 11.8 Removal (lines 49 to 90).

```
pub fn remove(&mut self, val: &T) -> bool {
    match self {
        Self::Empty => false,
        Self::Node { value, left, right } => match val.cmp(value) {
            Ordering::Less => left.remove(val),
            Ordering::Greater => right.remove(val),
            Ordering::Equal => {
                if left.is_empty() && right.is_empty() {
                    *self = Self::Empty;
                    return true;
                }
            }
        }
    }
}
```

```

        }
        if left.is_empty() {
            *self = *std::mem::take(right);
            return true;
        }
        if right.is_empty() {
            *self = *std::mem::take(left);
            return true;
        }
        let min_val = right.remove_min().unwrap();
        *value = min_val;
        true
    },
},
}

fn remove_min(&mut self) -> Option<T> {
    match self {
        Self::Empty => None,
        Self::Node { left, .. } if left.is_empty() => {
            let node = std::mem::take(self);
            if let Self::Node { value, right, .. } = node {
                *self = *right;
                Some(value)
            } else {
                unreachable!()
            }
        }
        Self::Node { left, .. } => left.remove_min(),
    }
}

```

The first two cases need to replace `*self` with one of its own children. That is not straightforward, because `right` is a reference into `*self`. You cannot move a subtree out through `right` and assign it to `*self` in one step. `std::mem::take(right)` solves it: it moves the subtree out and leaves the default value, an empty tree, in its place. Then `*self = *std::mem::take(right)` assigns the subtree, and the `*` moves it out of its `Box`. `take` needs `Default`, which is why the type derives it.

The code also handles a leaf, a node with no children, separately. Case 1 would already cover it.

For case 3, `remove_min` removes and returns the smallest value of the right subtree. It walks left until it reaches a node whose left child is empty, using a match guard: `Self::Node { left, .. } if left.is_empty()`. That node holds the smallest value. The method takes the whole node out with `std::mem::take(self)`, puts its right subtree in its place, and returns its value. The `if let ... else { unreachable!() }` is there because the compiler

cannot see that the value taken from a `Node` arm is a `Node`. `unreachable!()` panics if that ever turns out false.

`remove` returns `true` if the value was found and removed, and `false` otherwise, so a caller can tell the two apart.

11.3.3 Range queries

A **range query** returns every value between a low and a high bound, in order. A BST can answer it without visiting subtrees that cannot contain an answer.

Listing 11.9 Range queries (lines 92 to 114).

```
pub fn range(&self, low: &T, high: &T) -> Vec<&T> {
    let mut result = Vec::new();
    self.range_helper(low, high, &mut result);
    result
}

// Explicit lifetime 'a ties the references in the vector to the lifetime of &self.
fn range_helper<'a>(&'a self, low: &T, high: &T, acc: &mut Vec<&'a T>) {
    match self {
        Self::Empty => {}
        Self::Node { value, left, right } => {
            if value < low {
                right.range_helper(low, high, acc);
            } else if value > high {
                left.range_helper(low, high, acc);
            } else {
                left.range_helper(low, high, acc);
                acc.push(value);
                right.range_helper(low, high, acc);
            }
        }
    }
}
```

`range_helper` collects matching values into an accumulator, `acc`. Its three branches skip work:

- If the node's value is below `low`, everything in its left subtree is lower still, so only the right subtree is searched.
- If the value is above `high`, only the left subtree is searched.
- Otherwise, the helper searches the left subtree, adds the value, and searches the right subtree. That order is in-order traversal, so the results come out sorted.

The signature has an explicit lifetime: `fn range_helper<'a>(&'a self, ..., acc: &mut Vec<&'a T>)`. The references pushed into `acc` point into the tree, so they must live as long as the tree borrow `&'a self`. Without the named `'a`, the elision rules from chapter 4 would give `self` and the vector's elements separate lifetimes. The push would then not compile.

Listing 11.10 The complete program. `src/bin/bst_clean.rs`

```
use std::cmp::Ordering;

type TreeLink<T> = Box<BST<T>>;

#[derive(Debug, Clone, Default)]
pub enum BST<T> {
    #[default]
    Empty,
    Node {
        value: T,
        left: TreeLink<T>,
        right: TreeLink<T>,
    },
}

impl<T: Ord> BST<T> {
    pub fn new() -> Self {
        Self::Empty
    }

    pub fn insert(&mut self, val: T) {
        match self {
            Self::Empty => {
                *self = Self::Node {
                    value: val,
                    left: Box::new(Self::Empty),
                    right: Box::new(Self::Empty),
                };
            }
            Self::Node { value, left, right } => match val.cmp(value) {
                Ordering::Less => left.insert(val),
                Ordering::Greater => right.insert(val),
                Ordering::Equal => {},
            },
        }
    }

    pub fn contains(&self, val: &T) -> bool {
        match self {
            Self::Empty => false,
            Self::Node { value, left, right } => match val.cmp(value) {
                Ordering::Less => left.contains(val),
                Ordering::Greater => right.contains(val),
                Ordering::Equal => true,
            },
        }
    }

    pub fn remove(&mut self, val: &T) -> bool {
```

```

        match self {
            Self::Empty => false,
            Self::Node { value, left, right } => match val.cmp(value) {
                Ordering::Less => left.remove(val),
                Ordering::Greater => right.remove(val),
                Ordering::Equal => {
                    if left.is_empty() && right.is_empty() {
                        *self = Self::Empty;
                        return true;
                    }
                    if left.is_empty() {
                        *self = *std::mem::take(right);
                        return true;
                    }
                    if right.is_empty() {
                        *self = *std::mem::take(left);
                        return true;
                    }
                    let min_val = right.remove_min().unwrap();
                    *value = min_val;
                    true
                }
            },
        }
    }

fn remove_min(&mut self) -> Option<T> {
    match self {
        Self::Empty => None,
        Self::Node { left, .. } if left.is_empty() => {
            let node = std::mem::take(self);
            if let Self::Node { value, right, .. } = node {
                *self = *right;
                Some(value)
            } else {
                unreachable!()
            }
        }
        Self::Node { left, .. } => left.remove_min(),
    }
}

pub fn range(&self, low: &T, high: &T) -> Vec<&T> {
    let mut result = Vec::new();
    self.range_helper(low, high, &mut result);
    result
}

// Explicit lifetime 'a ties the references in the vector to the lifetime of &self.
fn range_helper<'a>(&'a self, low: &T, high: &T, acc: &mut Vec<&'a T>) {
    match self {
        Self::Empty => {},
        Self::Node { value, left, right } => {
            if value < low {
                right.range_helper(low, high, acc);
            } else if value > high {
                left.range_helper(low, high, acc);
            } else {
                left.range_helper(low, high, acc);
            }
        }
    }
}

```

```

        acc.push(value);
        right.range_helper(low, high, acc);
    }
}

fn is_empty(&self) -> bool {
    matches!(self, Self::Empty)
}

fn main() {
    let mut bst = BST::new();
    bst.insert(10);
    bst.insert(5);
    bst.insert(15);
    bst.insert(3);
    bst.insert(7);
    bst.insert(12);
    bst.insert(18);

    println!("Contains 7: {}", bst.contains(&7));
    println!("Contains 12: {}", bst.contains(&12));

    let range = bst.range(&4, &16);
    println!("Values in [4, 16]: {:?}", range);

    println!("Remove 5: {}", bst.remove(&5));
    println!("Remove 10: {}", bst.remove(&10));
    println!("Remove 99: {}", bst.remove(&99));

    println!("After removals: {:#?}", bst);
}

```

```

$ cargo run --bin bst_clean
Contains 7: true
Contains 12: true
Values in [4, 16]: [5, 7, 10, 12, 15]
Remove 5: true
Remove 10: true
Remove 99: false
After removals: Node {
    value: 12,
    left: Node {
        value: 7,
        left: Node {
            value: 3,
            left: Empty,
            right: Empty,
        },
        right: Empty,
    },
    right: Node {
        value: 15,
        left: Empty,
        right: Node {
            value: 18,

```



```

        left: Empty,
        right: Empty,
    },
},
}

```

Follow the two removals by hand. Node 5 has two children, 3 and 7. So it takes 7, the smallest value on its right, and the old 7 leaf is removed. Node 10 also has two children, so the root takes 12, the smallest value in its right subtree. The printed tree shows 12 at the root and 7 where 5 was.

11.3.4 A second BST, with an iterative minimum

The second implementation writes removal and range queries differently.

Listing 11.11 Removal and `pop_min` (lines 47 to 82). `src/bin/bst_easy.rs`

```

pub fn remove(&mut self, val: &T) {
    match self {
        Self::Empty => {}
        Self::Node { value, left, right } => match val.cmp(value) {
            Less => left.remove(val),
            Greater => right.remove(val),
            Equal => {
                if left.is_empty() {
                    *self = right.take();
                } else if right.is_empty() {
                    *self = left.take();
                } else {
                    *value = right.pop_min().unwrap();
                }
            }
        },
    }
}

pub fn pop_min(&mut self) -> Option<T> {
    let mut node = self;

    while node.has_left() {
        if let Self::Node { left, .. } = node {
            node = left;
        }
    }

    match node.take() {
        Self::Empty => None,
        Self::Node { value, right, .. } => {
            *node = *right;
            Some(value)
        }
    }
}

```

```
    }
}
```

This `remove` uses a helper `take`, defined at the end of the file, that swaps in `Empty` with `mem::replace`. So the type does not need `Default`. `right.take()` reaches through the `Box` automatically and returns the tree inside.

`pop_min` finds the smallest value with a loop instead of recursion. It moves a mutable reference, `node`, down the left side of the tree:

```
let mut node = self;
while node.has_left() {
    if let Self::Node { left, .. } = node {
        node = left;
    }
}
```

When the loop ends, `node` refers to the leftmost node. `match node.take()` splits it into its value and its right subtree, with no `unreachable!`. The loop uses one stack frame however deep the tree is. `pop_min` is also public, so the tree can serve as a queue that always returns the smallest value.

Listing 11.12 A range query that returns a new vector (lines 84 to 100).

```
pub fn range(&self, from: &T, to: &T) -> Vec<&T> {
    let mut result = Vec::new();

    if let Self::Node { value, left, right } = self {
        if from < value {
            result.extend(left.range(from, to));
        }
        if from <= value && value <= to {
            result.push(value);
        }
        if value < to {
            result.extend(right.range(from, to));
        }
    }

    result
}
```

This `range` returns a new `Vec` from each call and joins the children's results with `extend`. It is shorter than the accumulator version, and it allocates a vector at every node it visits, where the accumulator version allocates one.

Listing 11.13 The complete program. src/bin/bst_easy.rs

```

use std::cmp::Ordering::{Equal, Greater, Less};
use std::mem;

#[derive(Debug, Clone)]
pub enum BST<T> {
    Empty,
    Node {
        value: T,
        left: Box<BST<T>>,
        right: Box<BST<T>>,
    },
}

impl<T: Ord> BST<T> {
    pub fn new() -> Self {
        Self::Empty
    }

    pub fn insert(&mut self, val: T) {
        match self {
            Self::Empty => {
                *self = Self::Node {
                    value: val,
                    left: Box::new(Self::Empty),
                    right: Box::new(Self::Empty),
                };
            }
            Self::Node { value, left, right } => match val.cmp(value) {
                Less => left.insert(val),
                Greater => right.insert(val),
                Equal => {},
            },
        }
    }

    pub fn contains(&self, val: &T) -> bool {
        match self {
            Self::Empty => false,
            Self::Node { value, left, right } => match val.cmp(value) {
                Less => left.contains(val),
                Greater => right.contains(val),
                Equal => true,
            },
        }
    }

    pub fn remove(&mut self, val: &T) {
        match self {
            Self::Empty => {}
            Self::Node { value, left, right } => match val.cmp(value) {
                Less => left.remove(val),
                Greater => right.remove(val),
                Equal => {
                    if left.is_empty() {
                        *self = right.take();
                    } else if right.is_empty() {
                        *self = left.take();
                    }
                }
            }
        }
    }
}

```

```

        } else {
            *value = right.pop_min().unwrap();
        }
    },
}

pub fn pop_min(&mut self) -> Option<T> {
    let mut node = self;

    while node.has_left() {
        if let Self::Node { left, .. } = node {
            node = left;
        }
    }

    match node.take() {
        Self::Empty => None,
        Self::Node { value, right, .. } => {
            *node = *right;
            Some(value)
        }
    }
}

pub fn range(&self, from: &T, to: &T) -> Vec<&T> {
    let mut result = Vec::new();

    if let Self::Node { value, left, right } = self {
        if from < value {
            result.extend(left.range(from, to));
        }
        if from <= value && value <= to {
            result.push(value);
        }
        if value < to {
            result.extend(right.range(from, to));
        }
    }

    result
}

fn is_empty(&self) -> bool {
    match self {
        Self::Empty => true,
        Self::Node { .. } => false,
    }
}

fn has_left(&self) -> bool {
    match self {
        Self::Empty => false,
        Self::Node { left, .. } => !left.is_empty(),
    }
}

fn take(&mut self) -> Self {

```

```

        mem::replace(self, Self::Empty)
    }
}

fn main() {
    let mut bst = BST::new();

    bst.insert(10);
    bst.insert(5);
    bst.insert(15);
    bst.insert(3);
    bst.insert(7);
    bst.insert(12);

    println!("Contains 7: {}", bst.contains(&7));
    println!("Range 4 to 12: {:?}", bst.range(&4, &12));

    bst.remove(&10);
    println!("After removing 10: {:?}", bst.range(&0, &100));

    println!("Pop min: {:?}", bst.pop_min());
    println!("{:?}", bst);
}

```

```

$ cargo run --bin bst_easy
Contains 7: true
Range 4 to 12: [5, 7, 10, 12]
After removing 10: [3, 5, 7, 12, 15]
Pop min: Some(3)

```

WARNING: Neither tree keeps itself balanced. Suppose you insert 1, 2, 3, and so on in increasing order. Each value becomes the right child of the previous one, and the tree turns into a chain n levels deep. Searches then take $O(n)$, and every recursive method recurses n levels. The standard library's `BTreeMap` stays balanced.

11.4 Trees with any number of children

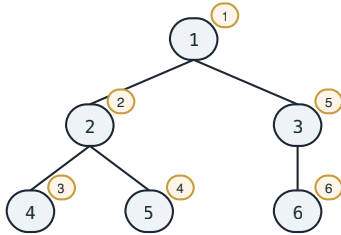
Many real trees are not binary. A folder can hold any number of files and folders. A node that owns a `Vec<TreeNode<T>>` of children describes such a tree. It needs no `Box`, because the `Vec` already keeps its elements on the heap, so the node's own size is fixed.

11.4.1 Depth first and breadth first

Figure 11.5 shows the two basic ways to visit such a tree. A **depth-first** visit goes down one branch completely before starting the next, as in section 11.2.2. A **breadth-first** visit goes level by level: the root, then all its children, then all their children.

One tree, two visiting orders

depth first (dfs): 1, 2, 4, 5, 3, 6



breadth first (bfs): 1, 2, 3, 4, 5, 6

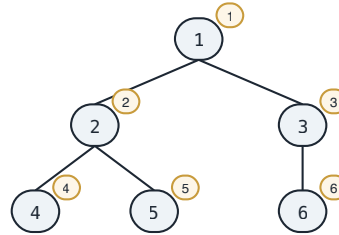


Figure 11.5 Depth first finishes node 2's branch before visiting 3. Breadth first finishes each level before the next.

Depth first is naturally recursive. Breadth first needs a **queue**: a collection where items leave in the order they arrived. You put the root in the queue. Then, repeatedly, you take the node at the front, visit it, and add its children at the back. Because children join at the back, every node of one level leaves the queue before any node of the next.

Listing 11.14 The complete program. `src/bin/test_tree.rs`

```
use std::collections::VecDeque;

struct TreeNode<T> {
    value: T,
    children: Vec<TreeNode<T>>, // Owned children directly inline
}

impl<T> TreeNode<T>
where
    T: Clone + std::fmt::Display
{
    /// Recursive DFS (Pre-order) - No callbacks
    pub fn dfs(&self) {
        println!("{}", self.value);
        for child in self.children.iter() {
            child.dfs();
        }
    }

    /// Level Order BFS - Idiomatic VecDeque::from([self])
}
```

```

pub fn bfs(&self) {
    let mut queue = VecDeque::from([self]); // Idiomatic initialization

    while let Some(node) = queue.pop_front() {
        println!("{}", node.value);

        for child in &node.children {
            queue.push_back(child);
        }
    }
}

fn main() {
    // --- Inline Construction (No variables, no push) ---

    let root = TreeNode {
        value: 1,
        children: vec![
            TreeNode {
                value: 2,
                children: vec![
                    TreeNode { value: 4, children: vec![] },
                    TreeNode { value: 5, children: Vec::new() },
                ],
            },
            TreeNode {
                value: 3,
                children: vec![TreeNode { value: 6, children: Vec::new() }],
            },
        ],
    };

    println!("=== DFS ===");
    root.dfs();
    // Output: 1, 2, 4, 5, 3, 6

    println!("\n=== BFS ===");
    root.bfs();
    // Output: 1, 2, 3, 4, 5, 6
}

```

`dfs` prints the node and then calls itself on each child, which is pre-order.

`bfs` uses a `VecDeque` as the queue. `VecDeque::from([self])` starts it with the root. The queue holds references, `&TreeNode<T>`, so no node is moved or copied. `pop_front` takes from the front, and `push_back` adds each child at the back.

The tree in `main` is written as one nested value. The shape of the code matches the shape of the tree, which makes test data easy to check.

```

$ cargo run --bin test_tree
=== DFS ===
1

```

```

2
4
5
3
6

=== BFS ===
1
2
3
4
5
6

```

11.4.2 Grouping values by level

A common variation returns the values grouped by level, as a list of lists. The first list holds the root's value, the second the values of its children, and so on. The file solves it twice.

Listing 11.15 The type and the first version (lines 1 to 18). `src/bin/tree_under_pressure.rs`

```

use std::collections::VecDeque;

#[derive(Debug)]
struct TreeNode<T> {
    value: T,
    children: Vec<TreeNode<T>>,
}

fn level_order<T>(root: &TreeNode<T>) -> Vec<Vec<&T>> {
    let mut result = Vec::new();
    let mut level = vec![root];
    while !level.is_empty() {
        result.push(level.iter().map(|n| &n.value).collect());
        level = level.iter().flat_map(|n| &n.children).collect();
    }
    result
}

```

`level_order` keeps one whole level in `level`, a `Vec` of references to nodes. Each pass of the loop does two things. It adds the values of this level to the result. Then it builds the next level from all the children of all the nodes in this level: `level.iter().flat_map(|n| &n.children).collect()`. `flat_map` turns each node into its children and joins all those children into one sequence. The loop ends when a level has no nodes.

Listing 11.16 The second version, with a queue (lines 20 to 39).

```

fn level_order_readable<T>(root: &TreeNode<T>) -> Vec<Vec<&T>> {
    let mut result = Vec::new();

```



```

    let mut queue = VecDeque::from([(root)]);

    while !queue.is_empty() {
        let current_size = queue.len();
        let mut level = Vec::with_capacity(current_size);

        for _ in 1..current_size {
            let Some(node) = queue.pop_front() else { break };

            level.push(&node.value);
            queue.extend(&node.children);
        }

        result.push(level);
    }
    result
}

```

`level_order_readable` uses one queue, as `bfs` did. The trick is `queue.len()` at the start of each round. At that moment, the queue holds exactly one level. So the loop pops exactly that many nodes, collecting their values, while their children pile up behind them for the next round.

`let Some(node) = queue.pop_front() else { break };` is a **let-else** statement. It binds `node` if the pattern matches, and runs the `else` block, which must leave the loop or the function, if it does not. `queue.extend(&node.children)` adds references to all the children at once.

Listing 11.17 The complete program. `src/bin/tree_under_pressure.rs`

```

use std::collections::VecDeque;

#[derive(Debug)]
struct TreeNode<T> {
    value: T,
    children: Vec<TreeNode<T>>,
}

fn level_order<T>(root: &TreeNode<T>) -> Vec<Vec<&T>> {
    let mut result = Vec::new();
    let mut level = vec![root];
    while !level.is_empty() {
        result.push(level.iter().map(|n| &n.value).collect());
        level = level.iter().flat_map(|n| &n.children).collect();
    }
    result
}

fn level_order_readable<T>(root: &TreeNode<T>) -> Vec<Vec<&T>> {
    let mut result = Vec::new();

```

```

    let mut queue = VecDeque::from([(root)]);

    while !queue.is_empty() {
        let current_size = queue.len();
        let mut level = Vec::with_capacity(current_size);

        for _ in 1..=current_size {
            let Some(node) = queue.pop_front() else { break };

            level.push(&node.value);
            queue.extend(&node.children);
        }

        result.push(level);
    }
    result
}

fn main() {
    let root: TreeNode<String> = TreeNode {
        value: "root".into(),
        children: vec![
            TreeNode {
                value: "branch_a".into(),
                children: vec![
                    TreeNode { value: "leaf_1".into(), children: vec![] },
                    TreeNode { value: "leaf_2".into(), children: vec![] },
                ],
            },
            TreeNode {
                value: "branch_b".into(),
                children: vec![
                    TreeNode { value: "leaf_3".into(), children: vec![] },
                ],
            },
        ],
    };

    let levels = level_order(&root);
    println!("Tree\n {:#?}", root);

    for level in levels {
        println!("{:?}", level);
    }
}

```

`main` calls the first version, `level_order`. It prints the tree with `{:#?}`, then the levels:

```

["root"]
["branch_a", "branch_b"]
["leaf_1", "leaf_2", "leaf_3"]

```

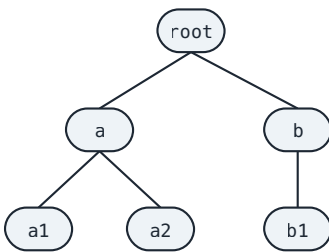
Both functions return `Vec<Vec<&T>>`, references into the tree, and both run in $O(n)$.

11.5 Saving a tree to a file and loading it back

To save a tree, you turn it into a flat sequence of text. Later, you rebuild exactly the same tree from that text. Turning a structure into a flat sequence is called **serialization**.

A list of values in pre-order is not enough, because it does not say where one node's children end. The file's format adds, after each value, the number of children that follow (figure 11.6). A reader can then rebuild the tree: read a value and a count, then read that many subtrees.

Saving a tree as value and child-count pairs, in pre-order



tree.bin, one token per line (pairs shown side by side)

root	2	root has 2 children
a	2	a has 2 children
a1	0	a1 has 0 children
a2	0	a2 has 0 children
b	1	b has 1 child
b1	0	b1 has 0 children

Figure 11.6 Each node is written as its value and its number of children, in pre-order.

Listing 11.18 Writing the pairs (lines 33 to 50). src/bin/tree_serializ_deserialize_into_file.rs

```

fn serialize(&self) -> Vec<String>
where
    T: ToString,
{
    let mut serialized = Vec::new();
    let mut stack = vec![self];

    while let Some(node) = stack.pop() {
        serialized.push(node.value.to_string());
        serialized.push(node.children.len().to_string());

        for child in node.children.iter().rev() {
            stack.push(child);
        }
    }

    serialized
}

```

`serialize` walks the tree in pre-order without recursion, using a `Vec` as a **stack**: the last item pushed is the first popped. It pushes the children in reverse order, with `iter().rev()`, so that

the first child is on top and is popped first. For each node it writes the value and the child count as strings.

The line `where T: ToString` sits on the method, not on the whole `impl`. So a tree of any type exists, and only a tree whose values can become strings gets a `serialize` method.

Listing 11.19 Reading the pairs back (lines 52 to 66).

```
fn deserialize(serialized: &[String], pos: &mut usize) -> Option<Self>
where
    T: FromStr,
{
    let value = serialized.get(*pos)?.parse().ok()?;
    let child_count: usize = serialized.get(*pos + 1)?.parse().ok()?;
    *pos += 2;

    let mut children = Vec::new();
    for _ in 0..child_count {
        children.push(Self::deserialize(serialized, pos)?);
    }

    Some(TreeNode { value, children })
}
```

`deserialize` is recursive. `pos` is a `&mut usize`, a position in the token list shared by all the calls. Each call reads a value and a count at `pos`, advances `pos` by two, and then calls itself `child_count` times to read the children.

Every step that can fail uses `?` on an `Option`. The failures are a missing token, a value that does not parse, and a count that is not a number. `.parse().ok()?` first turns the parse `Result` into an `Option`. Any failure makes the whole function return `None`.

Listing 11.20 The complete program. `src/bin/tree_serialize_deserialize_into_file.rs`

```
use std::collections::VecDeque;
use std::fs;
use std::io;
use std::str::FromStr;

#[derive(Debug)]
struct TreeNode<T> {
    value: T,
    children: Vec<TreeNode<T>>,
}

impl<T> TreeNode<T> {
    fn level_order(&self) -> Vec<Vec<&T>> {
        let mut result = Vec::new();
        let mut queue = VecDeque::from([self]);
```

```

        while !queue.is_empty() {
            let current_size = queue.len();
            let mut level = Vec::with_capacity(current_size);

            for _ in 1..current_size {
                let Some(node) = queue.pop_front() else { break };
                level.push(&node.value);
                queue.extend(&node.children);
            }

            result.push(level);
        }

        result
    }

    fn serialize(&self) -> Vec<String>
    where
        T: ToString,
    {
        let mut serialized = Vec::new();
        let mut stack = vec![self];

        while let Some(node) = stack.pop() {
            serialized.push(node.value.to_string());
            serialized.push(node.children.len().to_string());

            for child in node.children.iter().rev() {
                stack.push(child);
            }
        }

        serialized
    }

    fn deserialize(serialized: &[String], pos: &mut usize) -> Option<Self>
    where
        T: FromStr,
    {
        let value = serialized.get(*pos)?.parse().ok()?;
        let child_count: usize = serialized.get(*pos + 1)?.parse().ok()?;
        *pos += 2;

        let mut children = Vec::new();
        for _ in 0..child_count {
            children.push(Self::deserialize(serialized, pos)?);
        }

        Some(TreeNode { value, children })
    }

    fn save(&self, path: &str) -> io::Result<()>
    where
        T: ToString,
    {
        fs::write(path, self.serialize().join("\n"))
    }

    fn load(path: &str) -> Option<Self>

```

```

    where
        T: FromStr,
    {
        let text = fs::read_to_string(path).ok()?;
        let serialized: Vec<String> = text.lines().map(String::from).collect();
        Self::deserialize(&serialized, &mut 0)
    }
}

fn main() -> io::Result<()> {
    let root = TreeNode {
        value: "root".to_string(),
        children: vec![
            TreeNode {
                value: "a".to_string(),
                children: vec![
                    TreeNode { value: "a1".to_string(), children: vec![] },
                    TreeNode { value: "a2".to_string(), children: vec![] },
                ],
            },
            TreeNode {
                value: "b".to_string(),
                children: vec![TreeNode { value: "b1".to_string(), children: vec![] }],
            },
        ],
    };

    root.save("tree.bin")?;

    let Some(loaded) = TreeNode::::load("tree.bin") else {
        println!("invalid tree file");
        return Ok(());
    };

    for level in loaded.level_order() {
        println!("{:?}", level);
    }

    Ok(())
}

```

`save` joins the tokens with newlines and writes the file. `load` reads the file, splits it into lines, and starts reading at position 0 with `&mut 0`. `main` saves a tree, loads it back, and prints its levels to show the shape survived:

```

$ cargo run --bin tree_serializ_deserialize_into_file
["root"]
["a", "b"]
["a1", "a2", "b1"]

```

The format breaks if a value contains a newline, because the reader splits on newlines. For arbitrary text, write each value's length before it, or use a serialization library such as `serde`.

11.6 A trie for finding words by prefix

A **trie** (usually pronounced "try") stores a set of words by sharing their common beginnings. Each edge is one character. Walking from the root along the characters of a word leads to the node for that word. A flag on the node marks whether a stored word ends there (figure 11.7).

A trie after inserting "gpu" and "gpucloud"



Each edge is one character. Green nodes are terminal: a stored word ends there.

search("gpu") and search("gpucloud") are true. search("gp") is false: that node exists but is not terminal.

starts_with("gp") is true, because it only needs the node to exist.

Figure 11.7 A trie holding "gpu" and "gpucloud". The two words share their first three nodes.

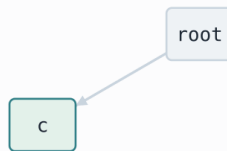
A **prefix** is the beginning of a word: "gp" is a prefix of "gpu". A trie answers "is any stored word starting with this prefix?" by walking the prefix's characters. That makes it the usual structure behind autocomplete.

A trie: every edge is one character, so prefixes are shared

A word is stored by walking its characters from the root.

 the walk

 ends a word



Insert "cat": there is no edge for 'c', so a new one is added, and this node carries the walk on.

One node per distinct prefix, not one node per word.

walk = **root/c**

word = **"cat"**

ends a word = **no**

Animation 11.6 Building a trie from "cat", "car", and "dog", then querying "ca" and "cab". Each frame adds one character, and a node appears only when its edge is created, so the sharing is visible: "cat" and "car" share the three nodes c, ca. A brass ring marks a node that ends a word. The last frames show the two different answers: "ca" reaches a node that ends no word but has children, so it is a prefix; "cab" has no edge for b, so it is absent, not even a prefix.

Listing 11.21 The complete file. `src/problems/trie.rs`

```

///! Trie / prefix tree.
///!
///! Used for autocomplete, command completion, and prefix matching.

use std::collections::HashMap;

#[derive(Default)]
pub struct Trie {
    root: Node,
}

#[derive(Default)]

```



```

struct Node {
    children: HashMap<char, Node>,
    terminal: bool,
}

impl Trie {
    pub fn new() -> Self {
        Self::default()
    }

    pub fn insert(&mut self, word: &str) {
        let mut node = &mut self.root;
        for ch in word.chars() {
            node = node.children.entry(ch).or_default();
        }
        node.terminal = true;
    }

    pub fn search(&self, word: &str) -> bool {
        self.find_node(word).is_some_and(|node| node.terminal)
    }

    pub fn starts_with(&self, prefix: &str) -> bool {
        self.find_node(prefix).is_some()
    }

    fn find_node(&self, word: &str) -> Option<&Node> {
        let mut node = &self.root;
        for ch in word.chars() {
            node = node.children.get(&ch)?;
        }
        Some(node)
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn trie_insert_search_and_prefix() {
        let mut trie = Trie::new();
        trie.insert("gpu");
        trie.insert("gpuccloud");

        assert!(trie.search("gpu"));
        assert!(trie.search("gpuccloud"));
        assert!(!trie.search("gp"));
        assert!(!trie.search("gpuccloudapi"));

        assert!(trie.starts_with("gp"));
        assert!(trie.starts_with("gpuccloud"));
        assert!(!trie.starts_with("cpu"));
    }
}

```

Each `Node` has a `HashMap` from a character to a child node, and a `terminal` flag. Both types derive `Default`, so an empty node is `Node::default()`.

`insert` walks a mutable cursor, `node`, down the word, one character per line:

```
node = node.children.entry(ch).or_default();
```

`entry(ch).or_default()` finds the child for `ch`, creating an empty one if it is missing, and returns a mutable reference to it. Assigning that to `node` moves the cursor down one level. After the last character, the node is marked terminal.

`find_node` walks the same way without changing anything. `node.children.get(&ch)?` returns `None` from the function at the first missing character. `search` then checks that the node exists and is terminal, with `is_some_and`. `starts_with` only checks that the node exists.

Searching for a word of `m` characters takes `m` steps, whatever the number of stored words. A `HashMap` per node is flexible but uses a lot of memory. For words of lowercase English letters only, an array of 26 child pointers per node is faster and smaller.

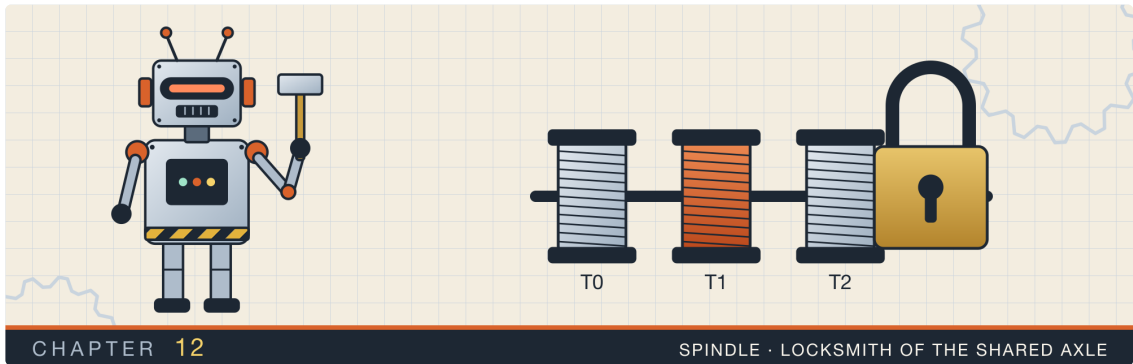
Summary

- A tree is a hierarchy of nodes. Each subtree is itself a tree, so recursion fits tree problems.
- A binary tree fits in an enum with `Box` children. Pre-order, in-order, and post-order differ only in when a node is visited relative to its subtrees.
- A binary search tree keeps smaller values on the left and larger on the right, so a search discards a subtree at each step.
- Removing from a BST has three cases. The two-children case copies in the smallest value of the right subtree.
- `std::mem::take` and `mem::replace` let a method replace `*self` with one of its own children.
- A range query skips subtrees outside the range and visits the rest in order.
- A node with `Vec` children describes a tree with any number of children. Breadth-first traversal uses a queue, and `queue.len()` at the start of a round gives the size of a level.
- Pre-order with child counts saves a tree in a form that can be rebuilt exactly.
- A trie shares prefixes between words, and finds a word in time proportional to its length.

Chapter 12 generalizes trees to graphs, where any node can connect to any other, and cycles are possible.

Exercises

1. Write an in-order traversal of the binary tree from listing 11.1 without recursion, using a `Vec` of references as a stack.
2. Add a `height` method to one of the BSTs. Insert the numbers 1 to 1,000 in order, then in shuffled order, and compare the heights.
3. Serialize a binary tree using a marker such as `#` for each empty child, and write the matching reader.
4. Add `words_with_prefix(&self, prefix: &str) -> Vec<String>` to the trie.



Graphs

THIS CHAPTER COVERS

- What a graph is, and two ways to store one in memory
- Breadth-first and depth-first search, and the queue and stack behind them
- Finding cycles with three node states
- Treating a grid of cells as a graph, to count islands
- Ordering tasks so that each comes after its prerequisites
- Shortest paths with Dijkstra's algorithm, written five ways
- Connecting every node at the lowest cost with Kruskal's algorithm and a union-find

In chapter 11, every node had exactly one parent, and no path could lead back to where it started. A **graph** drops both rules. Any node can connect to any other node, and a path can come back to its start.

Graphs describe many real systems. Roads between cities form a graph. So do links between web pages, and network links between machines. Courses and their prerequisites form a graph, and so do build steps and their inputs.

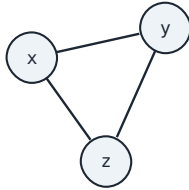
The freedom has a cost. A traversal that follows edges can reach the same node many times, or walk around a loop forever. Every algorithm in this chapter therefore keeps a record of which nodes it has already handled. The record is a `Vec<bool>`, a `HashSet`, or a `HashMap` of states.

12.1 Graph vocabulary

Figure 12.1 names the parts of a graph.

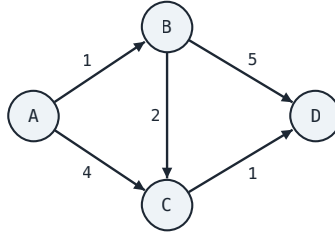
Graph terms

undirected: roads



an edge works both ways
x, y, z form a cycle

directed and weighted: the chapter's sample



an edge has a direction and a weight
no cycle: no path returns to its start

node (vertex): a thing, such as A

edge: a connection from one node to another

neighbors of B: C and D

weight: a cost on an edge, such as 5 for B to D

path: A to B to D, cost $1 + 5 = 6$

cycle: a path that returns to its start

Figure 12.1 An undirected graph, and the directed, weighted sample graph used in sections 12.2 to 12.4.

- A **node**, also called a vertex, is one of the things in the graph.
- An **edge** connects two nodes. In a **directed** graph, an edge goes one way, from one node to another. In an **undirected** graph, an edge works both ways, like a two-way road.
- The **neighbors** of a node are the nodes its edges lead to.
- A **weight** is a number on an edge, such as a distance or a cost.
- A **path** is a sequence of edges that you can follow one after another.
- A **cycle** is a path that ends at the node where it started.

A directed graph with no cycles is called a **directed acyclic graph**, or DAG. A tree is a special graph: it has no cycles, and every node except the root has exactly one parent.

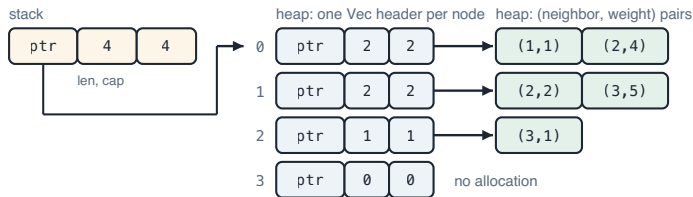
12.2 Storing a graph

The most common way to store a graph is an **adjacency list**. For every node, you keep a list of its outgoing edges. Each edge is stored as a pair: the neighbor, and the weight of the edge.

The code in this chapter uses two forms of adjacency list (figure 12.2).

Two ways to store the sample graph as an adjacency list

index form: `Vec<Vec<(usize, u32)>>, A=0, B=1, C=2, D=3`



map form: `HashMap<&str, &[(&str, u32)]>`



The index form finds a node's edges with one array index. The map form hashes the key, and allows any node type.

Figure 12.2 The sample graph in the two adjacency-list forms.

In the **index form**, the nodes are numbered 0, 1, 2, and so on. The graph is a `Vec<Vec<(usize, u32)>>`. `graph[i]` is the list of edges out of node `i`, and each edge is a `(neighbor, weight)` pair. Finding a node's edges is one array index. The record of visited nodes can be a `Vec<bool>` with one entry per node.

In the **map form**, nodes can be any type that can be hashed, such as names. The graph is a `HashMap` from a node to its edges. Finding a node's edges means hashing the key. The visited record becomes a `HashSet`.

The index form is faster, and the map form is more convenient when nodes have natural names. Several files in this chapter solve the same problem both ways, so you can compare them side by side.

The map form in `graph_bfs.rs` has a type alias:

```
/// Adjacency list keyed by node: each node maps to its `(neighbor, weight)` edges.
pub type GraphAdjList<'a, Node> = HashMap<Node, &'a [(Node, u32)]>;
```

`GraphAdjList<'a, Node>` is a `HashMap` whose values are borrowed slices, `&'a [(Node, u32)]`. The map does not own the edges. They live somewhere else for at least the lifetime `'a`, and the map points at them. The tests keep the edges in constant arrays, which live for the whole program.

12.3 Breadth-first search

Breadth-first search (BFS) visits the start node first, then every node one edge away, then every node two edges away, and so on. It works the same way as the level-order tree traversal of section 11.4: a queue holds the nodes waiting to be visited.

Unlike a tree, a graph can offer two paths to the same node. In the sample graph, A reaches C directly, and also through B. Without a record, C would enter the queue twice. So BFS marks a node as seen when it first enters the queue, and never adds it again.

The top half of figure 12.3 follows the queue step by step.

Breadth first uses a queue; depth first uses a stack

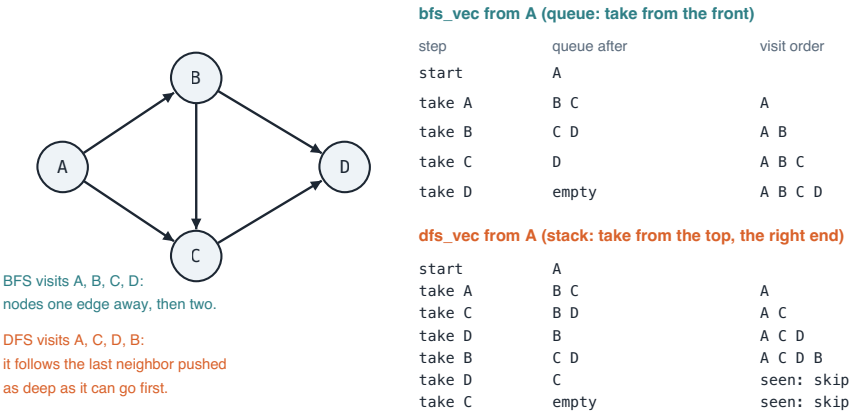
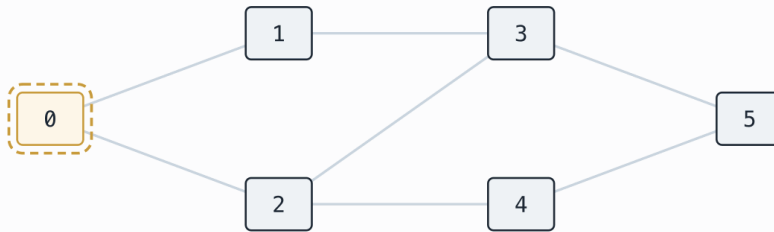


Figure 12.3 The queue in BFS and the stack in DFS, starting at A. Both visit every node once, in different orders.

Breadth-first search: a queue visits nodes in rings

Every neighbour of the current node is queued before any of their neighbours.

 visited
 in the queue
 not seen



the queue



front of the queue

Put 0 in the queue and mark it seen.

The queue is the frontier: it holds the nodes one edge away from what is visited.

visiting = -

in queue = 1

visited = 0

Animation 12.1 Breadth-first search from node 0. The rust node is the one being taken off the front of the queue now; cream nodes are waiting and green nodes are done. Because a node is marked as soon as it is queued, each node appears in the queue exactly once. The visit order is 0, 1, 2, 3, 4, 5, which is the order of distance from the start, so BFS is also how you find shortest paths in an unweighted graph.

12.3.1 BFS with numbered nodes

Listing 12.1 BFS over the index form (lines 16 to 34). src/problems/graph_bfs.rs

```
/// BFS over an index-based adjacency list. Returns nodes in visiting order.
pub fn bfs_vec(graph: &[Vec<usize, u32>], start_node: usize) -> Vec<usize> {
    let mut visited = vec![false; graph.len()];
    let mut queue = VecDeque::from([start_node]);
    let mut result = vec![];

    visited[start_node] = true;

    while let Some(current_node) = queue.pop_front() {
        result.push(current_node);
        for &(neighbor, _) in &graph[current_node] {
```



```

        if !visited[neighbor] {
            visited[neighbor] = true;
            queue.push_back(neighbor);
        }
    }
}
result
}

```

The function sets up three things. `visited` has one `false` per node. `queue` holds the start node. `result` collects the nodes in the order they are visited.

The loop takes the front node, records it, and looks at its edges. The pattern `&(neighbor, _)` takes each edge apart: it copies out the neighbor and ignores the weight with `_`. Each neighbor that has not been seen is marked and added to the back of the queue.

Every node enters the queue at most once, and every edge is looked at once. So BFS runs in $O(V + E)$ time, where V is the number of nodes and E the number of edges. Nodes that cannot be reached from the start never enter the queue, and are not in the result.

12.3.2 BFS with any node type

Listing 12.2 BFS over the map form (lines 36 to 57).

```

/// BFS over a `HashMap` adjacency list. Returns nodes in visiting order.
pub fn bfs<Node>(graph: &GraphAdjList<Node>, start_node: Node) -> Vec<Node>
where
    Node: Copy + Eq + Hash,
{
    let mut visited = HashSet::new();
    let mut queue = VecDeque::from([start_node]);
    let mut result = vec![];

    visited.insert(start_node);

    while let Some(current_node) = queue.pop_front() {
        result.push(current_node);
        // slice iter yields &(Node, u32); `&` deconstructs it
        for &(neighbor, _) in graph[&current_node] {
            if visited.insert(neighbor) {
                queue.push_back(neighbor);
            }
        }
    }
    result
}

```

The steps are the same. Three things change:

- The function is generic over `Node`. The `where` clause asks for `Copy`, so a node can be copied into the queue and the result. It also asks for `Eq + Hash`, so a node can be a key in a `HashSet`.
- `graph[¤t_node]` looks up the edges by key. Indexing a `HashMap` panics if the key is missing, so every node needs an entry, even a node with no edges.
- `visited.insert(neighbor)` does the check and the marking in one call. `insert` returns `true` if the value was not in the set before.

The tests at the end of the file build the sample graph in both forms and check that the two functions agree:

```
$ cargo test --lib problems::graph_bfs
running 5 tests
test problems::graph_bfs::tests::vec_variant_skips_unreachable_nodes ... ok
test problems::graph_bfs::tests::vec_variant_visits_level_by_level ... ok
test problems::graph_bfs::tests::vec_variant_handles_cycles ... ok
test problems::graph_bfs::tests::both_variants_agree ... ok
test problems::graph_bfs::tests::map_variant_visits_level_by_level ... ok

test result: ok. 5 passed; 0 failed; 0 ignored; 0 measured; 132 filtered out
```

Tests run in parallel, so the order of the lines can change from run to run.

12.4 Depth-first search

Depth-first search (DFS) follows one path as far as it can before it tries another. You can write it with recursion, as the tree traversals of chapter 11 were. The versions here use a `Vec` as an explicit stack instead. An explicit stack has no depth limit other than memory, while deep recursion can overflow the thread's stack.

Listing 12.3 DFS over the index form (lines 16 to 33). `src/problems/graph_dfs.rs`

```
/// DFS over an index-based adjacency list. Returns nodes in visiting order.
pub fn dfs_vec(graph: &[Vec<usize, u32>], start_node: usize) -> Vec<usize> {
    let mut visited = vec![false; graph.len()];
    let mut stack = vec![start_node];
    let mut result = vec![];

    while let Some(current_node) = stack.pop() {
        if visited[current_node] {
            continue;
        }
    }
```

```

        visited[current_node] = true;
        result.push(current_node);
        for &(neighbor, _) in &graph[current_node] {
            stack.push(neighbor);
        }
    }
    result
}

```

Compare this with listing 12.1. The queue became a stack, so `pop_front` became `pop`. The other change is when a node is marked. BFS marked a node when it was added. DFS marks a node when it is taken off the stack, and skips it if it was already visited.

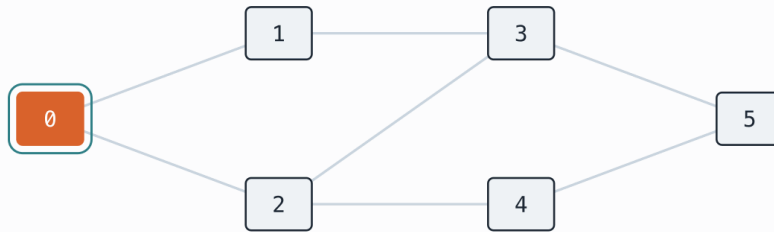
Marking late gives the true depth-first order, at a small cost. A node can sit on the stack more than once, as C and D do in the lower half of figure 12.3. The stack can hold up to one entry per edge, and the duplicates are skipped with `continue` when they come off.

The neighbors are pushed in list order, so the last neighbor pushed is the first taken. From A, the stack receives B and then C, so C is explored first. The visit order is A, C, D, B.

Depth-first search: follow one edge as far as it goes

The recursion returns only when the current node has no unseen neighbour left.

 visited being explored not seen



the stack



top of the stack

Visit 0. Go deeper into its first unseen neighbour.

The stack is the path from the start to the node being explored.

visiting = 0

stack = 0

visited = 1

Animation 12.2 Depth-first search from node 0, one visit or one edge per frame. The rust node is the one being visited now and green nodes are finished. The difference from BFS is the shape of the walk: BFS spreads out one ring at a time, while DFS goes as deep as it can before it comes back. The visit order here is 0, 1, 3, 2, 4, 5.

Listing 12.4 DFS over the map form (lines 35 to 54).

```
/// DFS over a `HashMap` adjacency list. Returns nodes in visiting order.
pub fn dfs<Node>(graph: &GraphAdjList<Node>, start_node: Node) -> Vec<Node>
where
    Node: Copy + Eq + Hash,
{
    let mut visited = HashSet::new();
    let mut stack = vec![start_node];
    let mut result = vec![];

    while let Some(current_node) = stack.pop() {
        if !visited.insert(current_node) {
            continue;
        }
        result.push(current_node);
        for &(neighbor, _) in graph[&current_node] {
```

```

        stack.push(neighbor);
    }
}
result
}

```

Here `if !visited.insert(current_node)` is the "already visited" check. If `insert` returns `false`, the node was in the set, and the loop moves on.

The file starts with `use crate::problems::graph_bfs::GraphAdjList;`, so both files share one type alias.

12.4.1 A stack of references

When the graph is a tree, DFS needs no visited record, because no node can be reached twice. The following program walks a tree with a stack of references.

Listing 12.5 The complete program. `src/bin/rust_interview_hacks_graph.rs`

```

#[derive(Debug)]
struct Node {
    value: i32,
    children: Vec<Node>,
}

fn visit_iter(root: &Node) {
    let mut stack = Vec:::<&Node>::from([root]);

    while let Some(node) = stack.pop() {
        println!("{:?}", node);
        // Idiomatic and concise way to extend stack with neighbour iterator
        stack.extend(node.children.iter());
    }
}

fn main() {
    let tree = Node {
        value: 1,
        children: vec![
            Node { value: 2, children: vec![] },
            Node {
                value: 3,
                children: vec![Node { value: 4, children: vec![] }],
            },
        ],
    };

    visit_iter(&tree);
    println!("still own it: {}", tree.value); // tree wasn't moved
}

```

The stack has type `Vec<&Node>`. It holds borrowed references into the tree, so the loop moves no nodes and copies none. `stack.extend(node.children.iter())` pushes a reference to each child. The last line of `main` uses `tree.value` after the walk, which shows that `tree` still owns its nodes.

```
$ cargo run --bin rust_interview_hacks_graph
Node { value: 1, children: [Node { value: 2, children: [] }, Node { value: 3, children:
[Node { value: 4, children: [] }]}] }
Node { value: 3, children: [Node { value: 4, children: [] }]}
Node { value: 4, children: [] }
Node { value: 2, children: [] }
still own it: 1
```

`{:?}",` prints each node with all its children, so the first line shows the whole tree. The order is 1, 3, 4, 2, because the last child pushed is the first popped.

12.5 A graph with named nodes

Programs often receive a graph as pairs of names, such as `("A", "B")`. A common way to handle names is to give each name a number the first time you see it. The algorithm then runs on numbers, with the fast index form. Assigning numbers to names this way is called **interning**.

Listing 12.6 The complete program. `src/bin/graph_bfs.rs`

```
use std::collections::{HashMap, HashSet, VecDeque};

#[derive(Default)]
struct Connection { to: String }

#[derive(Default)]
struct Graph {
    ids: HashMap<String, usize>,
    adj: Vec<Vec<usize>>,
}

impl Graph {
    fn id(&mut self, name: &str) -> usize {
        *self.ids.entry(name.to_owned()).or_insert_with(|| {
            self.adj.push(vec![]);
            self.adj.len() - 1
        })
    }

    fn add_edge(&mut self, from: &str, to: &str) {
        let (u, v) = (self.id(from), self.id(to));
        self.adj[u].push(v);
    }
}
```

```

fn bfs(&self, start: &str) -> Vec<usize> {
    let mut visited = HashSet::new();
    let mut queue = VecDeque::from([self.ids[start]]);
    let mut order = Vec::new();

    while let Some(node) = queue.pop_front() {
        if !visited.insert(node) {
            continue;
        }
        order.push(node);
        queue.extend(self.adj[node].iter());
    }
    order
}

fn main() {
    let mut g = Graph::default();
    g.add_edge("A", "B");
    g.add_edge("A", "C");
    g.add_edge("B", "D");
    g.add_edge("C", "E");
    g.add_edge("D", "F");

    println!("{:?}", g.bfs("A")); // [0, 1, 2, 3, 4, 5]
}

```

`Graph` has two fields. `ids` maps a name to its number, and `adj` is an index-form adjacency list without weights.

The `id` method does the interning in one expression. `self.ids.entry(name.to_owned())` looks up the name. `or_insert_with` runs its closure only if the name is new. The closure adds an empty edge list to `adj` and returns its index, which becomes the name's number. Either way, the expression returns a reference to the number, and the `*` at the front copies it out.

The closure changes `self.adj` while `self.ids` is borrowed by the entry. The compiler allows this because a closure borrows only the fields it uses, here `self.adj`. The two borrows touch different fields, so they do not conflict.

This `bfs` marks nodes when they leave the queue, as the DFS of section 12.4 did. A node can enter the queue more than once, and the second copy is skipped. `queue.extend(self.adj[node].iter())` adds all neighbors at once.

```

$ cargo run --bin graph_bfs
[0, 1, 2, 3, 4, 5]

```

The result is numbers, not names. A to F were numbered 0 to 5 in the order they first appeared.

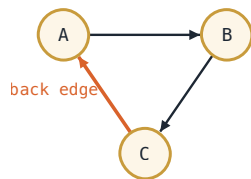
12.6 Finding cycles

Many problems need to know whether a directed graph has a cycle. A build system cannot build a file that depends on itself. A course plan cannot require a course before itself.

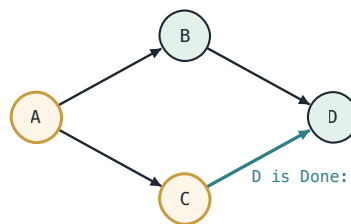
A visited flag is not enough to find cycles. Look at the diamond on the right of figure 12.4. DFS reaches D through B, and later reaches D again through C. That second visit does not mean there is a loop. It only means two paths join.

Cycle detection with three states

A to B to C to A: C finds A still Visiting



diamond: D is reached twice, but it is Done



- ☐ Unvisited: not reached yet
- ☐ Visiting: on the current path
- ☐ Done: fully explored

D is Done: no cycle

An edge to a Visiting node closes a loop. An edge to a Done node only joins a path explored earlier.

Figure 12.4 An edge to a node on the current path is a cycle. An edge to a finished node is not.

The fix is to give each node one of three states:

- **Unvisited:** DFS has not reached the node yet.
- **Visiting:** the node is on the path DFS is currently following. DFS has entered it and not yet finished it.
- **Done:** DFS has explored everything reachable from the node.

An edge that leads to a Visiting node goes back to an earlier point on the current path, so it closes a loop. Such an edge is called a **back edge**. An edge that leads to a Done node joins a path that was already fully explored, and cannot close a loop.

12.6.1 A graph that owns its node data

Listing 12.7 The graph and its states (lines 1 to 30). `src/bin/cyclic_graph.rs`

```
use std::collections::VecDeque;

type NodeId = usize;

struct Graph<T> {
    nodes: Vec<T>,
    adj: Vec<Vec<NodeId>>, // adj[i] = nodes that i points to
}

#[derive(Clone, Copy)]
enum State { Unvisited, Visiting, Done }

fn new() -> Self {
    Self { nodes: Vec::new(), adj: Vec::new() }
}

fn add_node(&mut self, data: T) -> NodeId {
    self.nodes.push(data);
    self.adj.push(Vec::new());
    self.nodes.len() - 1
}

fn add_edge(&mut self, from: NodeId, to: NodeId) {
    self.adj[from].push(to);
}

fn neighbors(&self, id: NodeId) -> impl Iterator<Item = NodeId> + '_ {
    self.adj[id].iter().copied()
}
```

This `Graph<T>` keeps the node data in `nodes`, and the edges in `adj`, as numbers. `NodeId` is a type alias for `usize`, so signatures say what a number means. `add_node` returns the new node's number, and the caller uses that number to add edges.

Storing numbers instead of references is the usual way to build graphs in Rust. A cycle of references between nodes would need `Rc` and `RefCell`, as chapter 9's doubly linked list did. Numbers into a `Vec` have no ownership problems at all.

`neighbors` returns `impl Iterator<Item = NodeId> + '_`. That return type means "some iterator of node numbers". The `+ '_` says the iterator borrows from `self`, so it cannot outlive the graph. `.copied()` turns the `&usize` items of the slice iterator into `usize` values.

Listing 12.8 BFS and DFS (lines 32 to 64).

```
fn bfs(&self, start: NodeId) -> Vec<NodeId> {
    let mut visited = vec![false; self.nodes.len()];
```

```

let mut queue = VecDeque::from([start]);
let mut order = Vec::new();
visited[start] = true;

while let Some(node) = queue.pop_front() {
    order.push(node);
    for next in self.neighbors(node) {
        if !visited[next] {
            visited[next] = true;
            queue.push_back(next);
        }
    }
}
order

}

fn dfs(&self, start: NodeId) -> Vec<NodeId> {
    let mut visited = vec![false; self.nodes.len()];
    let mut stack = vec![start];
    let mut order = Vec::new();

    while let Some(node) = stack.pop() {
        if visited[node] {
            continue;
        }
        visited[node] = true;
        order.push(node);
        stack.extend(self.neighbors(node).filter(|&n| !visited[n]));
    }
    order
}

```

These are the algorithms of sections 12.3 and 12.4, written with `neighbors`. The DFS filters out visited neighbors before pushing them, with `.filter(|&n| !visited[n])`. The filter keeps the stack smaller, but the check after `pop` is still needed. A node can be pushed by two different nodes before either copy is popped.

Listing 12.9 Cycle detection (lines 66 to 82).

```

fn has_cycle(&self) -> bool {
    let mut state = vec![State::Unvisited; self.nodes.len()];
    (0..self.nodes.len()).any(|n| self.visit(n, &mut state))
}

fn visit(&self, node: NodeId, state: &mut [State]) -> bool {
    match state[node] {
        State::Visiting => true, // back edge: cycle found
        State::Done => false,
        State::Unvisited => {
            state[node] = State::Visiting;
            let found = self.adj[node].iter().any(|&n| self.visit(n, state));
            state[node] = State::Done;
            found
        }
    }
}

```

```
    }
}
```

`visit` is a recursive DFS that returns `true` as soon as it finds a cycle. It reads the node's state:

- `Visiting` means this edge is a back edge, so the answer is `true`.
- `Done` means the node was explored before and no cycle was found through it, so the answer is `false`.
- `Unvisited` means this is the first visit. The node becomes `Visiting`, and `visit` recurses into each neighbor. `any` stops at the first neighbor that reports a cycle. Then the node becomes `Done`.

`has_cycle` calls `visit` from every node, because a cycle might not be reachable from node 0. All the calls share one `state` vector, so each node is fully explored only once, and the whole check runs in $O(V + E)$.

Listing 12.10 The complete program. `src/bin/cyclic_graph.rs`

```
use std::collections::VecDeque;

type NodeId = usize;

struct Graph<T> {
    nodes: Vec<T>,
    adj: Vec<Vec<NodeId>>, // adj[i] = nodes that i points to
}

#[derive(Clone, Copy)]
enum State { Unvisited, Visiting, Done }

impl<T> Graph<T> {
    fn new() -> Self {
        Self { nodes: Vec::new(), adj: Vec::new() }
    }

    fn add_node(&mut self, data: T) -> NodeId {
        self.nodes.push(data);
        self.adj.push(Vec::new());
        self.nodes.len() - 1
    }

    fn add_edge(&mut self, from: NodeId, to: NodeId) {
        self.adj[from].push(to);
    }

    fn neighbors(&self, id: NodeId) -> impl Iterator<Item = NodeId> + '_ {
        self.adj[id].iter().copied()
    }

    fn bfs(&self, start: NodeId) -> Vec<NodeId> {
        let mut visited = vec![false; self.nodes.len()];
```

```

    let mut queue = VecDeque::from([start]);
    let mut order = Vec::new();
    visited[start] = true;

    while let Some(node) = queue.pop_front() {
        order.push(node);
        for next in self.neighbors(node) {
            if !visited[next] {
                visited[next] = true;
                queue.push_back(next);
            }
        }
    }
    order
}

fn dfs(&self, start: NodeId) -> Vec<NodeId> {
    let mut visited = vec![false; self.nodes.len()];
    let mut stack = vec![start];
    let mut order = Vec::new();

    while let Some(node) = stack.pop() {
        if visited[node] {
            continue;
        }
        visited[node] = true;
        order.push(node);
        stack.extend(self.neighbors(node).filter(|&n| !visited[n]));
    }
    order
}

fn has_cycle(&self) -> bool {
    let mut state = vec![State::Unvisited; self.nodes.len()];
    (0..self.nodes.len()).any(|n| self.visit(n, &mut state))
}

fn visit(&self, node: NodeId, state: &mut [State]) -> bool {
    match state[node] {
        State::Visiting => true, // back edge: cycle found
        State::Done => false,
        State::Unvisited => {
            state[node] = State::Visiting;
            let found = self.adj[node].iter().any(|&n| self.visit(n, state));
            state[node] = State::Done;
            found
        }
    }
}

fn main() {
    let mut g = Graph::new();
    let a = g.add_node("A");
    let b = g.add_node("B");
    let c = g.add_node("C");

    g.add_edge(a, b);
    g.add_edge(b, c);
}

```

```

g.add_edge(c, a); // closes the cycle

let names = |ids: Vec<NodeId>| -> Vec<&str> {
    ids.into_iter().map(|i| g.nodes[i]).collect()
};

println!("BFS:  {:?}", names(g.bfs(a))); // ["A", "B", "C"]
println!("DFS:  {:?}", names(g.dfs(a))); // ["A", "B", "C"]
println!("Cycle: {}", g.has_cycle());    // true
}

```

`main` builds the loop A to B to C to A. The closure `names` converts a list of numbers back into names for printing.

```

$ cargo run --bin cyclic_graph
BFS:  ["A", "B", "C"]
DFS:  ["A", "B", "C"]
Cycle: true

```

12.6.2 The same graph with a HashMap

The second version stores the graph as a map from a node to its neighbors, with no numbering step.

Listing 12.11 The complete program. `src/bin/cyclic_graph_map.rs`

```

use std::collections::{HashMap, HashSet, VecDeque};
use std::hash::Hash;

#[derive(Clone, Copy)]
enum State { Visiting, Done }

#[derive(Default)]
struct Graph<T> {
    adj: HashMap<T, Vec<T>>,
}

impl<T: Copy + Eq + Hash> Graph<T> {
    fn add_edge(&mut self, u: T, v: T) {
        self.adj.entry(v).or_default();
        self.adj.entry(u).or_default().push(v);
    }

    fn bfs(&self, start: T) -> Vec<T> {
        let mut seen = HashSet::from([start]);
        let mut queue = VecDeque::from([start]);
        let mut order = vec![];

        while let Some(u) = queue.pop_front() {
            order.push(u);
            for &v in &self.adj[&u] {
                if seen.insert(v) {

```

```

        queue.push_back(v);
    }
}
order
}

fn has_cycle(&self) -> bool {
    fn dfs<T: Copy + Eq + Hash>(g: &Graph<T>, u: T, state: &mut HashMap<T, State>) ->
bool {
        match state.get(&u) {
            Some(State::Visiting) => return true,
            Some(State::Done) => return false,
            None => {}
        }

        state.insert(u, State::Visiting);
        let found = g.adj[&u].iter().any(|&v| dfs(g, v, state));
        state.insert(u, State::Done);
        found
    }

    let mut state = HashMap::new();
    self.adj.keys().any(|&u| dfs(self, u, &mut state))
}

fn main() {
    let mut g = Graph::default();
    g.add_edge("A", "B");
    g.add_edge("B", "C");
    g.add_edge("C", "A");

    println!("{:?}", g.bfs("A"));
    println!("{}", g.has_cycle()); // true
}

```

Three details differ from listing 12.10:

- `State` has only `Visiting` and `Done`. A node that is not in the `state` map is unvisited, so the map itself plays the role of the third state.
- `add_edge` inserts the target `v` as a key too, with `entry(v).or_default()`. Every node therefore has an entry, and `self.adj[&u]` cannot panic on a node that has no outgoing edges.
- The recursive helper is a function declared inside `has_cycle`. A nested `fn` cannot use the local variables of the function around it, so it receives the graph and the state map as arguments.

```

$ cargo run --bin cyclic_graph_map
["A", "B", "C"]
true

```

12.7 A grid is a graph

A grid of cells is a graph too, even though it is not stored as one. Each cell is a node, and each cell has an edge to the cells directly above, below, left, and right of it. The edges are computed from the coordinates when needed, not stored.

The classic grid problem counts **islands**. The grid holds land and water. An island is a group of land cells connected up, down, left, or right. Diagonal neighbors do not count.

The method is to scan every cell. When the scan finds a land cell that no earlier search has reached, that cell starts a new island. The count goes up by one, and a BFS from that cell marks every cell of the island. Filling a connected area this way is called a **flood fill** (figure 12.5).

Counting islands: each flood fill from new land adds one

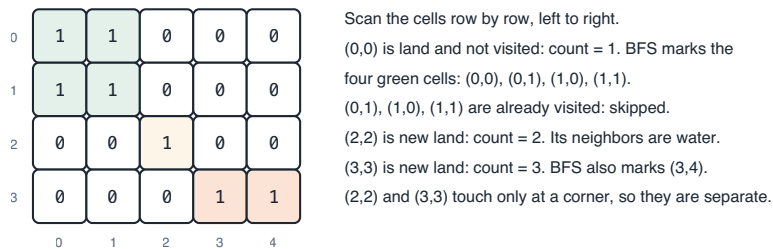
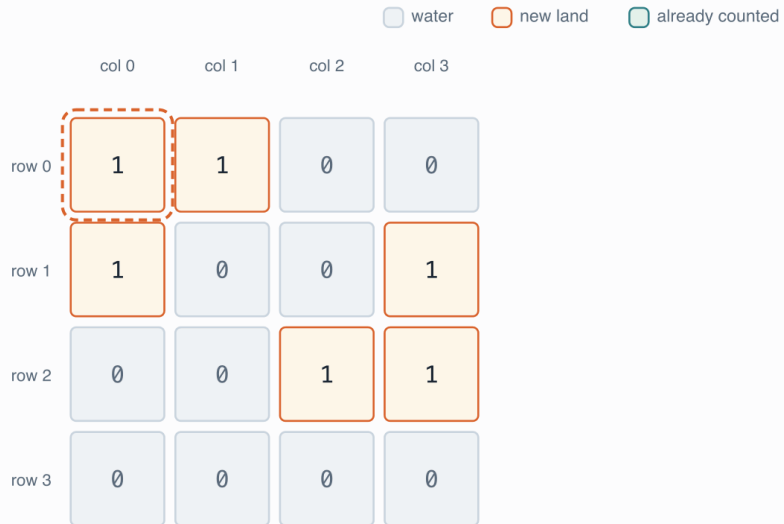


Figure 12.5 The grid from `main`. Each color is one island, found by one flood fill.

Counting islands: one flood fill per unvisited land cell

Water stops the fill, so each fill covers exactly one island.



Four rows by four columns. A 1 is land, a 0 is water, and land touches at its four sides.

The scan walks each cell once, and the fills never revisit a marked cell.

islands = 0 scan = 0,0

Animation 12.3 Counting islands on a smaller grid, so every cell fits on the page. The scan visits cells in reading order; when it reaches land that no fill has marked, that cell starts a new island and the fill spreads to every connected land cell before the scan continues. Water is grey, untouched land is cream, and marked land is green. The same code on the wider grid in `main` returns 3.

Listing 12.12 Counting islands (lines 27 to 72). `src/bin/count_islands.rs`

```
fn count_islands<T: PartialEq>(grid: &[&[T]], land: T) -> usize {
    if grid.is_empty() || grid[0].is_empty() {
        return 0;
    }

    let rows = grid.len();
    let cols = grid[0].len();
    let mut visited = HashSet::new();
    let mut count = 0;
    let dirs = [(0, 1), (1, 0), (0, -1), (-1, 0)];
```



```

// BFS closure: flood-fill one island starting from a known land cell.
// Passing `visited` explicitly avoids borrow-checker friction when calling
// the closure from the outer loop.
let bfs = |start: (usize, usize), visited: &mut HashSet<(usize, usize)>| {
    let mut queue = VecDeque::new();
    queue.push_back(start);
    visited.insert(start);

    while let Some((cr, cc)) = queue.pop_front() {
        for &(dr, dc) in &dirs {
            let nr = cr as i32 + dr;
            let nc = cc as i32 + dc;

            if (0..rows as i32).contains(&nr) && (0..cols as i32).contains(&nc) {
                let (ur, uc) = (nr as usize, nc as usize);
                if grid[ur][uc] == land && !visited.contains(&(ur, uc)) {
                    visited.insert((ur, uc));
                    queue.push_back((ur, uc));
                }
            }
        }
    }
};

for r in 0..rows {
    for c in 0..cols {
        if grid[r][c] == land && !visited.contains(&(r, c)) {
            count += 1;
            bfs((r, c), &mut visited);
        }
    }
}

count
}

```

The grid has type `&[&[T]]`, a slice of rows, where each row is a slice of cells. The function is generic, so cells can be characters, booleans, or numbers. `land: T` says which value counts as land, and `T: PartialEq` lets the code compare a cell with it.

`dirs` lists the four moves as (row change, column change) pairs. The BFS adds each move to the current cell's coordinates. The coordinates are `usize`, which cannot be negative, and a move up from row 0 would go below zero. So the code converts to `i32` first. It checks that the new position is inside the grid with `(0..rows as i32).contains(&nr)`, and only then converts back to `usize`.

The BFS is a closure stored in `bfs`. It reads `grid`, `dirs`, `rows`, `cols`, and `land` from the function around it. It does not capture `visited`, though. It takes `visited` as a parameter instead. If the closure captured `visited` mutably, that borrow would last as long as the closure.

The outer loop could then not call `visited.contains` while the closure exists. Passing it in on each call keeps the borrow short.

Each cell is looked at a constant number of times, so the function runs in $O(\text{rows} \times \text{cols})$. A `Vec<bool>` with one entry per cell would be faster than the `HashSet` of coordinates, because it avoids hashing.

Listing 12.13 The complete program, with tests. `src/bin/count_islands.rs`

```
// =====
// Count Islands
//
// Ported from the Rust playground repo.
//
// Problem: given a grid of cells, count the number of connected "land"
// regions. Two land cells are connected when they are adjacent horizontally
// or vertically (4-directional). Water cells are not traversable.
//
// Where this shape appears in practice:
// - "Islands" are connected-component counting, the same shape used to find
//   connected machines, failure domains, network partitions, or storage
//   groups in distributed systems.
// - The generic implementation accepts any land type (char, u8, bool) and
//   uses BFS with an explicit visited set, which is easy to reason about.
//
// Time complexity: O(rows * cols)
// Space complexity: O(rows * cols) for the visited set and BFS queue.
// =====

use std::collections::{HashSet, VecDeque};

/// Counts 4-directionally connected components of `land` in a rectangular grid.
///
/// The grid is generic over cell values so callers can use `'1'/'0'`,
/// `true/false`, `1u8/0u8`, or any other PartialEq value.
fn count_islands<T: PartialEq>(grid: &[&[T]], land: T) -> usize {
    if grid.is_empty() || grid[0].is_empty() {
        return 0;
    }

    let rows = grid.len();
    let cols = grid[0].len();
    let mut visited = HashSet::new();
    let mut count = 0;
    let dirs = [(0, 1), (1, 0), (0, -1), (-1, 0)];

    // BFS closure: flood-fill one island starting from a known land cell.
    // Passing `visited` explicitly avoids borrow-checker friction when calling
    // the closure from the outer loop.
    let bfs = |start: (usize, usize), visited: &mut HashSet<(usize, usize)>| {
        let mut queue = VecDeque::new();
        queue.push_back(start);
        visited.insert(start);

        while let Some((cr, cc)) = queue.pop_front() {
```

```

        for &(dr, dc) in &dirs {
            let nr = cr as i32 + dr;
            let nc = cc as i32 + dc;

            if (0..rows as i32).contains(&nr) && (0..cols as i32).contains(&nc) {
                let (ur, uc) = (nr as usize, nc as usize);
                if grid[ur][uc] == land && !visited.contains(&(ur, uc)) {
                    visited.insert((ur, uc));
                    queue.push_back((ur, uc));
                }
            }
        }
    }
};

for r in 0..rows {
    for c in 0..cols {
        if grid[r][c] == land && !visited.contains(&(r, c)) {
            count += 1;
            bfs((r, c), &mut visited);
        }
    }
}

count
}

// =====
// Runnable demonstration
// =====
fn main() {
    let grid = [
        &['1', '1', '0', '0', '0'][..],
        &['1', '1', '0', '0', '0'][..],
        &['0', '0', '1', '0', '0'][..],
        &['0', '0', '0', '1', '1'][..],
    ];

    println!("Islands: {}", count_islands(&grid, '1'));

    let boolean_grid = [
        &[true, true, false][..],
        &[false, false, false][..],
        &[false, true, true][..],
    ];
    println!("Boolean islands: {}", count_islands(&boolean_grid, true));
}

// =====
// Tests (cargo test --bin count_islands)
// =====
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn empty_grid_has_zero_islands() {
        let empty: [&[char]; 0] = [];
        assert_eq!(count_islands(&empty, '1'), 0);
    }
}

```

```

    let no_cols: [&[char]; 2] = [&[], &[]];
    assert_eq!(count_islands(&no_cols, '1'), 0);
}

#[test]
fn counts_classic_example() {
    let grid = [
        &['1', '1', '0', '0', '0'][..],
        &['1', '1', '0', '0', '0'][..],
        &['0', '0', '1', '0', '0'][..],
        &['0', '0', '0', '1', '1'][..],
    ];
    assert_eq!(count_islands(&grid, '1'), 3);
}

#[test]
fn all_water_is_zero() {
    let grid = [&['0', '0'][..], &['0', '0'][..]];
    assert_eq!(count_islands(&grid, '1'), 0);
}

#[test]
fn generic_bool_grid_works() {
    let grid = [
        &[true, false, false][..],
        &[true, false, true][..],
        &[false, false, true][..],
    ];
    assert_eq!(count_islands(&grid, true), 2);
}

#[test]
fn single_row_and_column() {
    let row = [&['1', '0', '1', '1'][..]];
    assert_eq!(count_islands(&row, '1'), 2);

    let col = [&['1'][..], &['1'][..], &['0'][..], &['1'][..]];
    assert_eq!(count_islands(&col, '1'), 2);
}
}

```

```

$ cargo run --bin count_islands
Islands: 3
Boolean islands: 2

$ cargo test --bin count_islands
running 5 tests
test tests::empty_grid_has_zero_islands ... ok
test tests::all_water_is_zero ... ok
test tests::generic_bool_grid_works ... ok
test tests::single_row_and_column ... ok
test tests::counts_classic_example ... ok

test result: ok. 5 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

```

12.8 Putting tasks in order

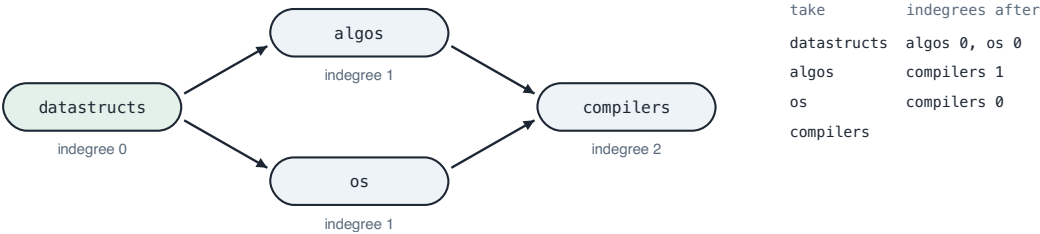
Suppose you have tasks, and some tasks must finish before others can start. A **topological order** lists every task so that each one comes after all of its prerequisites. Build tools, package managers, and course planners all compute one.

Draw an arrow from each task to the tasks that depend on it. The **indegree** of a node is the number of arrows coming into it: the number of prerequisites it is still waiting for.

Kahn's algorithm builds the order step by step (figure 12.6):

- 1. Put every node with indegree 0 in a queue. They have no prerequisites.
- 2. Take a node from the queue and add it to the order.
- 3. For each arrow out of that node, lower the target's indegree by one. The target has one fewer prerequisite to wait for. If its indegree reaches 0, add it to the queue.
- 4. Repeat until the queue is empty.

Kahn's algorithm: take a node with no remaining prerequisites, then cross it off



An arrow points from a course to a course that depends on it. The indegree counts the arrows coming in.
Order: datastructs, algos, os, compilers. If a cycle existed, its nodes would never reach indegree 0.

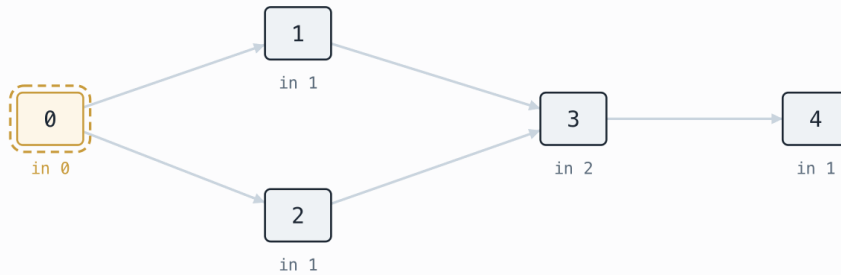
Figure 12.6 Kahn's algorithm on four courses.

If the graph has a cycle, the nodes on the cycle wait for each other forever. Their indegrees never reach 0, so they never enter the order. The order then comes out shorter than the number of nodes, and that is how the algorithm detects a cycle.

Kahn's algorithm: remove the nodes that nothing depends on

A node enters the order only when every edge into it has been removed.

 ordered ready now still blocked



Count the incoming edges of every node. A node with no incoming edge can go first.

Each edge is removed once, so the cost grows with the nodes plus the edges.

take = - ready = 1 order = -

Animation 12.4 Kahn's algorithm on a small dependency graph. Each node carries its current indegree, and a node turns cream the moment that indegree reaches zero. Taking a node removes its outgoing arrows, which is what lowers the targets. If the queue ever empties before every node is in the order, the remaining nodes form a cycle and no order exists, which is the check the chapter's code uses.

12.8.1 Kahn's algorithm on numbered nodes

Listing 12.14 The complete file. `src/problems/graph_topology.rs`

```

///! Topological sort (Kahn's algorithm).
///!
///! Common in scheduling problems: prerequisite graphs, pipeline
///! stages, and build or job dependency ordering.

use std::collections::{HashMap, VecDeque};

pub fn topological_sort(num_nodes: usize, edges: &[(usize, usize)]) -> Option<Vec<usize>>
{
    let mut graph = vec![Vec::new(); num_nodes];
    let mut indegree = vec![0usize; num_nodes];

    for &(from, to) in edges {


```

```

        graph[from].push(to);
        indegree[to] += 1;
    }

    let mut queue: VecDeque<usize> = (0..num_nodes).filter(|&n| indegree[n] == 0).collect(
);
    let mut order = Vec::with_capacity(num_nodes);

    while let Some(node) = queue.pop_front() {
        order.push(node);

        for &next in &graph[node] {
            indegree[next] -= 1;
            if indegree[next] == 0 {
                queue.push_back(next);
            }
        }
    }

    (order.len() == num_nodes).then_some(order)
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn returns_order_for_dag() {
        // 0 -> 1, 0 -> 2, 1 -> 3, 2 -> 3
        let edges = [(0, 1), (0, 2), (1, 3), (2, 3)];
        let order = topological_sort(4, &edges).expect("DAG has an order");
        assert_eq!(order.len(), 4);
        assert!(
            order.iter().position(|&n| n == 0).unwrap()
                < order.iter().position(|&n| n == 1).unwrap()
        );
        assert!(
            order.iter().position(|&n| n == 1).unwrap()
                < order.iter().position(|&n| n == 3).unwrap()
        );
    }

    #[test]
    fn detects_cycle() {
        // 0 -> 1 -> 0 is a cycle.
        let edges = [(0, 1), (1, 0)];
        assert_eq!(topological_sort(2, &edges), None);
    }

    #[test]
    fn disconnected_nodes_are_included() {
        let edges = [(0, 1)];
        let order = topological_sort(3, &edges).expect("DAG has an order");
        assert_eq!(order.len(), 3);
    }
}

```

The input is a node count and a slice of (from, to) edges. The first loop builds the adjacency list and counts each node's indegree at the same time.

The starting queue comes from one iterator chain. `(0..num_nodes)` produces every node number. `.filter(|&n| indegree[n] == 0)` keeps the nodes with no prerequisites, and `.collect()` builds the `VecDeque`.

The last line returns the answer. `bool::then_some(order)` gives `Some(order)` if the condition is true, and `None` if it is false. So the function returns `None` exactly when a cycle kept some nodes out of the order.

The tests check positions rather than one exact order. A graph usually has several valid topological orders, and the tests only require that each edge's source comes before its target.

```
$ cargo test --lib problems::graph_topology
running 3 tests
test problems::graph_topology::tests::disconnected_nodes_are_included ... ok
test problems::graph_topology::tests::returns_order_for_dag ... ok
test problems::graph_topology::tests::detects_cycle ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 134 filtered out
```

12.8.2 Kahn's algorithm on named courses

The second version takes courses with names and lists of prerequisites, the way the data might arrive from a file.

Listing 12.15 The complete program. `src/bin/dependency_resolution.rs`

```
use std::collections::{HashMap, VecDeque};

#[derive(Clone)]
struct Course {
    name: String,
    depends_on: Vec<String>,
}

fn topo_sort(courses: &[Course]) -> Vec<String> {
    let mut adj: HashMap<String, Vec<String>> = HashMap::new();
    let mut indegree: HashMap<String, usize> = HashMap::new();

    for Course { name, depends_on } in courses {
        indegree.insert(name.clone(), depends_on.len());
        adj.entry(name.clone()).or_default();
        for dep in depends_on {
            adj.entry(dep.clone()).or_default().push(name.clone());
            indegree.entry(dep.clone()).or_insert(0);
        }
    }
}
```



```

let mut queue: VecDeque<_> = indegree
    .iter()
    .filter(|&(_, deg)| *deg == 0)
    .map(|(name, _)| name.clone())
    .collect();

let mut order = Vec::new();
while let Some(node) = queue.pop_front() {
    for next in &adj[&node] {
        let d = indegree.get_mut(next).unwrap();
        *d -= 1;
        if *d == 0 {
            queue.push_back(next.clone());
        }
    }
    order.push(node);
}

if order.len() == indegree.len() { order } else { Vec::new() }
}

fn main() {
    let courses = vec![
        Course { name: "algos".into(), depends_on: vec!["datastructs".into()] },
        Course { name: "compilers".into(), depends_on: vec!["algos".into(), "os".into()] },
    ];

    Course { name: "os".into(), depends_on: vec!["datastructs".into()] },
];

let coned2 = courses.clone();

println!("{:?}", topo_sort(&courses));
}

```

The input describes edges backward from the way the algorithm needs them. Each course lists its prerequisites, but Kahn's algorithm follows arrows from a prerequisite to the courses that need it. The loop turns them around. For each prerequisite `dep` of course `name`, it adds `name` to `adj[dep]`.

A course's indegree is the length of its `depends_on` list. A prerequisite that is never listed as a course, like `datastructs` here, still needs an entry. `indegree.entry(dep.clone()).or_insert(0)` adds it with indegree 0, and `or_insert` leaves an existing entry alone.

`for Course { name, depends_on } in courses` takes each course apart in the loop header. Because `courses` is a slice reference, `name` and `depends_on` are references to the fields.

If a cycle keeps some courses out of the order, the function returns an empty `Vec`. With `Option` as in listing 12.14, the caller could tell "no courses" from "a cycle".

```
$ cargo run --bin dependency_resolution
["datastructs", "algos", "os", "compilers"]
```

The starting queue is built by iterating over a `HashMap`, and that order changes between runs. This input has only one course with no prerequisites, so the output is always the same. With several such courses, the order among them could change from run to run.

12.9 Shortest paths

In a weighted graph, the cost of a path is the sum of its edge weights. The **shortest path** from A to D is the path with the lowest cost. It is not always the one with the fewest edges.

BFS finds the path with the fewest edges, which is a different question. **Dijkstra's algorithm** finds the lowest cost from one start node to every other node, as long as no weight is negative.

12.9.1 How Dijkstra's algorithm works

The algorithm keeps two things:

- `dist`, the lowest cost found so far to each node. At the start, only the start node has a cost, 0.
- A min-heap of `(cost, node)` entries, so the cheapest entry is always taken next. Chapter 5 built min-heaps with `BinaryHeap` and `Reverse`.

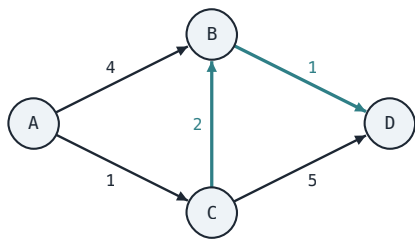
Each step takes the cheapest entry from the heap. For each edge out of that node, it computes the cost of reaching the neighbor through this node. If that cost beats the neighbor's recorded cost, the new cost is recorded and pushed onto the heap. Updating a recorded cost to a lower one is called **relaxing** the edge.

When a node is taken from the heap with its recorded cost, that cost is final. Every other entry in the heap costs at least as much, and weights are never negative. So no path found later can come back cheaper.

A node can be in the heap several times. When a cheaper path to B is found, the old entry for B stays in the heap. `BinaryHeap` has no way to find and update it. When the old entry is eventually taken, its cost is higher than B's recorded cost, and the code skips it. Such an entry is called **stale**.

Figure 12.7 follows every step on a four-node graph.

Dijkstra's algorithm on the graph in dijkstra.rs, starting at A



A to B directly costs 4.
A to C to B costs $1 + 2 = 3$, so the direct entry (4, B) becomes stale.

pop	dist after	heap after (smallest first)
start	A 0	(0, A)
(0, A)	A 0 B 4 C 1	(1, C) (4, B)
(1, C)	A 0 B 3 C 1 D 6	(3, B) (4, B) (6, D)
(3, B)	A 0 B 3 C 1 D 4	(4, B) (4, D) (6, D)
(4, B)	stale: $4 > 3$	(4, D) (6, D)
(4, D)	D has no edges	(6, D)
(6, D)	stale: $6 > 4$	empty

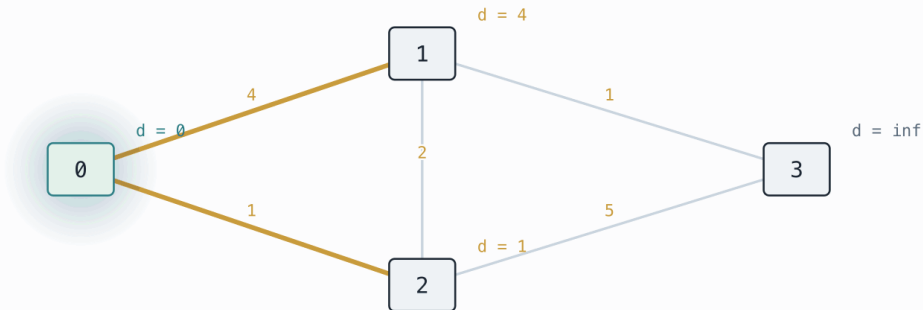
Result: A 0, C 1, B 3, D 4. Each pop takes the smallest cost in the heap, so the first time a node is popped with its recorded cost, that cost is final.

Figure 12.7 Dijkstra's algorithm from A. Two entries become stale when cheaper paths are found.

Dijkstra's algorithm: settle the closest node, then relax its edges

A node's distance is final the moment it is the closest unsettled one.

 settled
 just relaxed
 unsettled



Settle 0 at distance 0. Relax 1: $\text{inf} \rightarrow 4$, 2: $\text{inf} \rightarrow 1$.

Relaxing an edge lowers a neighbour's distance when the new route is shorter.

settle = 0

distance = 0

settled = 1

Animation 12.5 Dijkstra's algorithm on the same four-node graph as figure 12.7, one settle per frame. The number beside each node is its best known distance so far, and the rust node is the one being settled. Settling is final: the closest unsettled node can never be reached more cheaply later, because every edge weight is positive. Watch B's distance fall from 4 to 3 when C is settled, which is exactly the relaxation the table records.

Read the table row by row. Taking A records B at 4 and C at 1. Taking C finds B at $1 + 2 = 3$, which beats 4, so B is recorded at 3 and pushed again. The old (4, B) entry is now stale. Taking B at 3 finds D at 4, which beats the 6 recorded through C. Later, (4, B) and (6, D) come off the heap, and each is skipped because its cost is higher than the recorded one.

12.9.2 Two versions side by side

Listing 12.16 Dijkstra over the index form (lines 20 to 41). `src/problems/graph_dijkstra.rs`

```
/// Shortest distances from `start_node` over an index-based adjacency list.
pub fn dijkstra_vec(graph: &[Vec<usize, u32>], start_node: usize) -> Vec<Option<u32>> {
    let mut distances = vec![None; graph.len()];
```

```

let mut min_heap = BinaryHeap::from([Reverse((0, start_node))]);

distances[start_node] = Some(0);

while let Some(Reverse((current_cost, current_node))) = min_heap.pop() {
    // stale heap entry, a shorter path was already recorded
    if distances[current_node].is_some_and(|best| current_cost > best) {
        continue;
    }
    for &(neighbor, weight) in &graph[current_node] {
        let new_distance = current_cost + weight;
        if distances[neighbor].is_none_or(|best| new_distance < best) {
            distances[neighbor] = Some(new_distance);
            min_heap.push(Reverse((new_distance, neighbor)));
        }
    }
}
distances
}

```

The distances are a `Vec<Option<u32>>`, where `None` means "no path found yet". Two methods of `Option` read well here:

- `is_some_and(|best| current_cost > best)` is true only if there is a recorded cost and the popped cost is higher. That is the stale check.
- `is_none_or(|best| new_distance < best)` is true if there is no recorded cost, or the new cost is lower. That is the relax check.

The heap holds `Reverse((cost, node))`. `BinaryHeap` pops the largest item, and `Reverse` flips the comparison, so the smallest cost comes out first. Tuples compare by their first field, then by their second, so the cost decides and the node number breaks ties.

Listing 12.17 Dijkstra over the map form (lines 43 to 66).

```

/// Shortest distances from `start_node` over a `HashMap` adjacency list.
pub fn dijkstra<Node>(graph: &GraphAdjList<Node>, start_node: Node) -> HashMap<Node, u32>
where
    Node: Copy + Eq + Hash + Ord,
{
    let mut distances = HashMap::from([(start_node, 0)]);
    let mut min_heap = BinaryHeap::from([Reverse((0, start_node))]);

    while let Some(Reverse((current_cost, current_node))) = min_heap.pop() {
        // stale heap entry, a shorter path was already recorded
        if current_cost > distances[&current_node] {
            continue;
        }
        for &(neighbor, weight) in graph[&current_node] {
            let new_distance = current_cost + weight;

            if new_distance < *distances.get(&neighbor).unwrap_or(&u32::MAX) {
                distances.insert(neighbor, new_distance);
            }
        }
    }
}

```

```

        min_heap.push(Reverse((new_distance, neighbor)));
    }
}
distances
}

```

In this version, `distances` is a `HashMap` that contains only nodes that have been reached. `Node` also needs `Ord`, because nodes sit inside the tuples that the heap compares.

`distances[¤t_node]` cannot panic here. A node is pushed onto the heap only after its distance is inserted, so every popped node has an entry. `distances.get(&neighbor).unwrap_or(&u32::MAX)` treats a missing entry as the largest possible cost, so any real cost beats it.

Each edge can push one heap entry, and each heap operation takes $O(\log E)$. The whole algorithm runs in $O(E \log E)$, which is the same as $O(E \log V)$ for simple graphs.

```

$ cargo test --lib problems::graph_dijkstra
running 4 tests
test problems::graph_dijkstra::tests::vec_variant_finds_shorter_multi_hop_paths ... ok
test problems::graph_dijkstra::tests::vec_variant_marks_unreachable_nodes ... ok
test problems::graph_dijkstra::tests::map_variant_omits_unreachable_nodes ... ok
test problems::graph_dijkstra::tests::map_variant_finds_shorter_multi_hop_paths ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 133 filtered out

```

WARNING: `current_cost + weight` panics on overflow in a debug build, and wraps around silently in a release build. With `u32` weights, a long path of large weights can overflow. Listing 12.20 shows the fix with `checked_add`.

12.9.3 Edges as a struct

The next three programs use a named `Edge` struct instead of a `(neighbor, weight)` tuple, and `u64` costs. They show three steps of the same function: it first clones node names, then borrows them, and finally handles missing nodes and overflow.

Listing 12.18 The complete program. `src/bin/dijkstra.rs`

```

use std::cmp::Reverse;
use std::collections::{BinaryHeap, HashMap};
use std::hash::Hash;

#[derive(Debug)]
struct Edge<Node> {

```

```

    to: Node,
    weight: u64,
}

type AdjMap<Node> = HashMap<Node, Vec<Edge<Node>>>;

fn dijkstra_ssm_d_shortest<Node>(graph: &AdjMap<Node>, start: Node) -> HashMap<Node, u64>
where
    Node: Clone + Hash + Ord,
{
    let mut dist = HashMap::from([(start.clone(), 0)]);
    let mut heap = BinaryHeap::from([Reverse((0, start))]);

    while let Some(Reverse((cost, node))) = heap.pop() {
        if cost > dist[&node] {
            continue; // stale entry
        }
        for edge in &graph[&node] {
            let new_cost = cost + edge.weight;
            if dist.get(&edge.to).is_none_or(|&d| new_cost < d) {
                dist.insert(edge.to.clone(), new_cost);
                heap.push(Reverse((new_cost, edge.to.clone())));
            }
        }
    }
    dist
}

fn main() {
    let graph: AdjMap<str> = HashMap::from([
        ("A", vec![Edge { to: "B", weight: 4 }, Edge { to: "C", weight: 1 }]),
        ("B", vec![Edge { to: "D", weight: 1 }]),
        ("C", vec![Edge { to: "B", weight: 2 }, Edge { to: "D", weight: 5 }]),
        ("D", vec![]),
    ]);

    println!("{:#?}", graph);
    let dist = dijkstra_ssm_d_shortest(&graph, "A");
    println!("{:#?}", dist); // {"A": 0, "C": 1, "B": 3, "D": 4} (order varies)
}

```

`edge.to` and `edge.weight` read better than `.0` and `.1`. The node type needs `Clone` instead of `Copy`, so it could be a `String`. The function clones a node each time it stores one in `dist` or on the heap.

`graph[&node]` panics if a node has no entry in the map. `main` includes `"D"` with an empty list, so every node has one.

```

$ cargo run --bin dijkstra
{
  "B": [
    Edge {
      to: "D",
      weight: 1,
    },
  ],
}

```

```

    ],
    ...
  }
  {
    "B": 3,
    "C": 1,
    "A": 0,
    "D": 4,
  }
}

```

The first print shows the graph and is cut short here. A `HashMap` does not keep its entries in any order. The lines of both prints can come out in a different order on your machine.

Listing 12.19 Borrowing nodes instead of cloning them (lines 13 to 35). `src/bin/dijkstra_with_lifetime.rs`

```

fn dijkstra_ssm_shortest<'a, Node>(graph: &'a AdjMap<Node>, start: &'a Node) ->
  HashMap<&'a Node, u64>
where
  Node: Hash + Ord,
{
  let mut dist = HashMap::from([(start, 0u64)]);
  let mut heap = BinaryHeap::from([Reverse((0u64, start))]);

  while let Some(Reverse((cost, node))) = heap.pop() {
    if cost > dist[node] {
      continue; // stale entry
    }
    let Some(edges) = graph.get(node) else { continue };

    for edge in edges {
      let Some(new_cost) = cost.checked_add(edge.weight) else { continue };
      if dist.get(&edge.to).is_none_or(|&d| new_cost < d) {
        dist.insert(&edge.to, new_cost);
        heap.push(Reverse((new_cost, &edge.to)));
      }
    }
  }
  dist
}

```

This version stores references to nodes, `&'a Node`, in `dist` and in the heap, so `Node` does not need `Clone`. With `String` nodes, that saves an allocation per clone.

The lifetime `'a` ties everything together. The graph and the start node are both borrowed for `'a`, and the returned map holds `&'a Node` keys. The compiler therefore keeps the graph alive as long as the result is in use. Every reference pushed onto the heap, `&edge.to`, points into the graph, so it lives long enough.

Two `let ... else` lines make the function safer than listing 12.18:

- `let Some(edges) = graph.get(node) else { continue };` handles a node that has no entry in the map. `main` leaves `"D"` out of the map, and the function still works.
- `let Some(new_cost) = cost.checked_add(edge.weight) else { continue };` skips an edge whose total cost would overflow. `checked_add` returns `None` on overflow instead of panicking or wrapping around.

Listing 12.20 The complete program. `src/bin/dijkstra_with_lifetime.rs`

```
use std::cmp::Reverse;
use std::collections::{BinaryHeap, HashMap};
use std::hash::Hash;

#[derive(Debug)]
struct Edge<Node> {
    to: Node,
    weight: u64,
}

type AdjMap<Node> = HashMap<Node, Vec<Edge<Node>>>;

fn dijkstra_ssmd_shortest<'a, Node>(graph: &'a AdjMap<Node>, start: &'a Node) -> HashMap<&'a Node, u64>
where
    Node: Hash + Ord,
{
    let mut dist = HashMap::from([(start, 0u64)]);
    let mut heap = BinaryHeap::from([Reverse((0u64, start))]);

    while let Some(Reverse((cost, node))) = heap.pop() {
        if cost > dist[node] {
            continue; // stale entry
        }
        let Some(edges) = graph.get(node) else { continue };

        for edge in edges {
            let Some(new_cost) = cost.checked_add(edge.weight) else { continue };
            if dist.get(&edge.to).is_none_or(|&d| new_cost < d) {
                dist.insert(&edge.to, new_cost);
                heap.push(Reverse((new_cost, &edge.to)));
            }
        }
    }
    dist
}

fn main() {
    let graph: AdjMap<String> = HashMap::from([
        ("A".into(), vec![Edge { to: "B".into(), weight: 4 }, Edge { to: "C".into(), weight: 1 }]),
        ("B".into(), vec![Edge { to: "D".into(), weight: 1 }]),
        ("C".into(), vec![Edge { to: "B".into(), weight: 2 }, Edge { to: "D".into(), weight: 5 }]),
    ]),
}
```

```

    });

    println!("Graph => {:#?}", graph);

    let start = "A".into();
    println!("Shortest path {:#?}", dijkstra_ssmd_shortest(&graph, &start)); // A:0, C:1,
B:3, D:4
}

```

In `main`, `let start = "A".into();` makes a `String` in its own variable, because the function needs a reference that lives as long as the graph borrow.

```

$ cargo run --bin dijkstra_with_lifetime
Graph => {
  ...
}
Shortest path {
  "A": 0,
  "B": 3,
  "D": 4,
  "C": 1,
}

```

The last version keeps the owned, cloning style of listing 12.18 and adds the two safety checks.

Listing 12.21 The complete program. `src/bin/dijkstra_bruce_lee.rs`

```

use std::cmp::Reverse;
use std::collections::{BinaryHeap, HashMap};
use std::hash::Hash;

struct Edge<Node> {
    to: Node,
    weight: u64,
}

type AdjMap<Node> = HashMap<Node, Vec<Edge<Node>>>;

fn dijkstra_ssmd_shortest<Node>(graph: &AdjMap<Node>, start: Node) -> HashMap<Node, u64>
where
    Node: Clone + Hash + Ord,
{
    let mut dist = HashMap::from([(start.clone(), 0)]);
    let mut heap = BinaryHeap::from([Reverse((0u64, start))]);

    while let Some(Reverse((cost, node))) = heap.pop() {
        if cost > dist[&node] {
            continue; // stale entry
        }
        let Some(edges) = graph.get(&node) else { continue }; // node with no outgoing
edges

        for edge in edges {
            let Some(new_cost) = cost.checked_add(edge.weight) else { continue }; //

```

```
overflow
    if dist.get(&edge.to).is_none_or(|&d| new_cost < d) {
        dist.insert(edge.to.clone(), new_cost);
        heap.push(Reverse((new_cost, edge.to.clone())));
    }
}
dist
}

fn main() {
    let graph: AdjMap<&str> = HashMap::from([
        ("A", vec![Edge { to: "B", weight: 4 }, Edge { to: "C", weight: 1 }]),
        ("B", vec![Edge { to: "D", weight: 1 }]),
        ("C", vec![Edge { to: "B", weight: 2 }, Edge { to: "D", weight: 5 }]),
    ]); // "D" has no entry now, and it no longer panics

    println!("{:?}", dijkstra_ssmd_shortest(&graph, "A"));
}
```

```
$ cargo run --bin dijkstra_bruce_lee
{"B": 3, "C": 1, "D": 4, "A": 0}
```

Table 12.1 compares the five versions.

Table 12.1 The five Dijkstra implementations

Listing	Nodes	Edges	Missing node entry	Overflow
12.16 dijkstra_vec	usize indices	(usize, u32)	cannot happen	panics in debug
12.17 dijkstra	Copy values	(Node, u32)	panics	panics in debug
12.18 dijkstra.rs	cloned	Edge, u64	panics	panics in debug
12.20 dijkstra_with_lifetime.rs	borrowed	Edge, u64	skipped	skipped
12.21 dijkstra_bruce_lee.rs	cloned	Edge, u64	skipped	skipped

12.10 Connecting every node at the lowest cost

The last problem uses an undirected, weighted graph. Think of the nodes as buildings and the weights as the cost of laying cable between two buildings. You want to connect every building, directly or indirectly, at the lowest total cost.

A set of edges that connects all the nodes without forming a cycle is a **spanning tree**. A spanning tree with the lowest total weight is a **minimum spanning tree** (MST). With n nodes, a spanning tree has exactly $n - 1$ edges. One edge fewer would leave a node disconnected, and one edge more would close a loop.

Kruskal's algorithm builds an MST by taking edges from lightest to heaviest. It keeps an edge if the edge joins two groups of nodes that are not yet connected. It skips an edge whose two ends are already connected, because that edge would close a loop.

12.10.1 Union-find

Kruskal's algorithm needs to answer one question fast: are these two nodes already connected? A **union-find** structure, also called a disjoint set, answers it.

Union-find keeps nodes in groups. Each group is a small tree stored in a `parent` array, where `parent[i]` is the node that `i` points to. The node at the top of a group points to itself and is called the group's **root**. Two nodes are in the same group when they have the same root.

There are two operations:

- `find(x)` follows `parent` links from `x` up to the root.
- `union(x, y)` finds both roots. If the roots differ, it makes one root point to the other, which joins the two groups.

Listing 12.22 Union-find (lines 6 to 41). `src/bin/kruskals_algorithm.rs`

```
struct DisjointSet { parent: Vec<usize>, rank: Vec<usize> }

impl DisjointSet {
    fn new(n: usize) -> Self {
        Self {
            parent: (0..n).collect(),
            rank: vec![0; n]
        }
    }

    fn find(&mut self, elem: usize) -> usize {
        if self.parent[elem] != elem {
```

```

        self.parent[elem] = self.find(self.parent[elem]);
    }

    self.parent[elem]
}

fn union(&mut self, x: usize, y: usize) -> bool {
    let (x, y) = (self.find(x), self.find(y));

    if x == y { return false;}

    // Note for self : Comparator requires a reference
    match self.rank[x].cmp(&self.rank[y]) {
        Less => self.parent[x] = y,
        Greater => self.parent[y] = x,
        Equal => {
            self.parent[x] = y;
            self.rank[y] += 1;
        }
    }

    true
}
}

```

`new` makes every node its own group: `parent` is `[0, 1, 2, ...]`, built with `(0..n).collect()`.

`find` is recursive. On the way back from the root, it sets `parent[elem]` to the root directly. The next `find` on that node then takes one step instead of many. This trick is called **path compression**.

`union` returns `false` if both nodes already have the same root. Otherwise it joins the groups using `rank`, a rough measure of each group's height. The shorter group goes under the taller one, so the trees stay flat. If both have the same rank, either can go under the other, and the new root's rank grows by one. This is called **union by rank**.

With both tricks, `find` and `union` take nearly constant time in practice.

12.10.2 Kruskal's algorithm

Listing 12.23 Kruskal's algorithm (lines 43 to 58).

```

fn kruskals(n: usize, mut edges: Vec<Edge>) -> Vec<Edge> {
    // Greedy algorithm, sort the edges by weight
    edges.sort_by_key(|e| e.w);

    let mut uf = DisjointSet::new(n);
    let mut mst = Vec::new();

```

```

for edge in edges {
    if uf.union(edge.u, edge.v) {
        mst.push(edge);
    }
}

mst
}

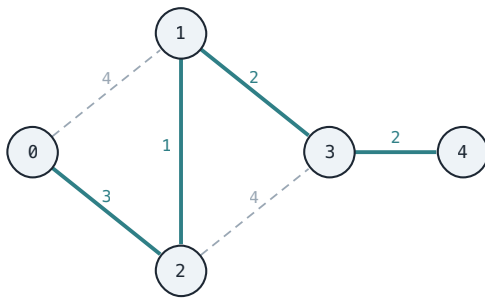
```

`edges.sort_by_key(|e| e.w)` sorts the edges by weight. Then each edge is offered to `union`. If `union` returns `true`, the edge joined two groups, and it goes into the tree. The edge is moved into `mst`, not copied, because `for edge in edges` takes ownership of each one.

Sorting dominates the running time, so the algorithm runs in $O(E \log E)$.

Figure 12.8 follows the `parent` array through every edge of the program's graph.

Kruskal's algorithm: take edges from lightest to heaviest, skip any that close a loop



edge	union	parent after	rank
1-2 1	roots 1, 2	0 2 2 3 4	rank[2] = 1
1-3 2	roots 2, 3	0 2 2 2 4	
3-4 2	roots 2, 4	0 2 2 2 2	
0-2 3	roots 0, 2	2 2 2 2 2	
0-1 4	same root 2	skip: loop	
2-3 4	same root 2	skip: loop	

`parent[i]` is the node `i` points to. A node that points to itself is the root of its group. Two nodes are in the same group when `find()` returns the same root.

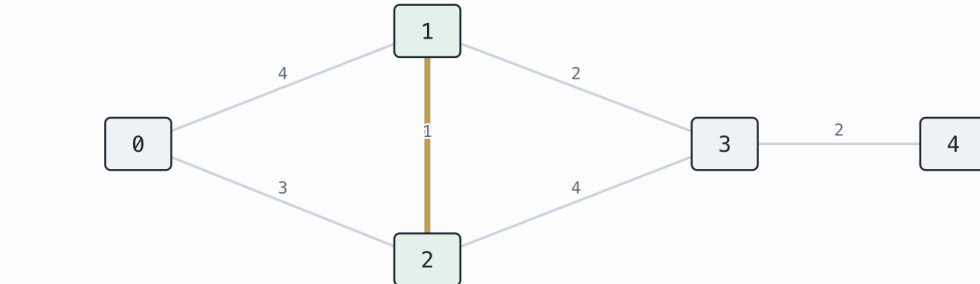
Kept edges (solid): 1-2, 1-3, 3-4, 0-2, total weight 8. They connect all five nodes with four edges.

Figure 12.8 Kruskal's algorithm with union-find. The two weight-4 edges would close loops, so they are skipped.

`sort_by_key` is a stable sort: edges with equal weights keep their input order. So 1-3 comes before 3-4, and 0-1 comes before 2-3.

Kruskal's algorithm: take the cheapest edge that closes no cycle

The edges are sorted by weight first. A union-find test decides each one.



Edge 1-2 of weight 1 joins two separate groups, so it is kept. Total weight becomes 1.

Sorting costs $O(E \log E)$. The union-find test is what keeps the result a tree.

edge = 1-2

weight = 1

total = 1

Animation 12.6 Kruskal's algorithm on the graph from figure 12.8, cheapest edge first. A kept edge is drawn thick and teal; the edge being tested is gold, or rust when it is skipped. The skip is the whole point of union-find: an edge is rejected only when its two endpoints already share a root, which means the edge would close a cycle. Four edges connect five nodes, and the accepted weights sum to 8.

Listing 12.24 The complete program. `src/bin/kruskals_algorithm.rs`

```
use std::cmp::Ordering::{Equal, Greater, Less};

#[derive(Debug)]
struct Edge { u: usize, v: usize, w: u32 }

struct DisjointSet { parent: Vec<usize>, rank: Vec<usize> }

impl DisjointSet {
    fn new(n: usize) -> Self {
        Self {
            parent: (0..n).collect(),
            rank: vec![0; n]
        }
    }
}
```

```

    }

    fn find(&mut self, elem: usize) -> usize {
        if self.parent[elem] != elem {
            self.parent[elem] = self.find(self.parent[elem]);
        }

        self.parent[elem]
    }

    fn union(&mut self, x: usize, y: usize) -> bool {
        let (x, y) = (self.find(x), self.find(y));

        if x == y { return false; }

        // Note for self : Comparator requires a reference
        match self.rank[x].cmp(&self.rank[y]) {
            Less => self.parent[x] = y,
            Greater => self.parent[y] = x,
            Equal => {
                self.parent[x] = y;
                self.rank[y] += 1;
            }
        }

        true
    }
}

fn kruskals(n: usize, mut edges: Vec<Edge>) -> Vec<Edge> {
    // Greedy algorithm, sort the edges by weight
    edges.sort_by_key(|e| e.w);

    let mut uf = DisjointSet::new(n);
    let mut mst = Vec::new();

    for edge in edges {
        if uf.union(edge.u, edge.v) {
            mst.push(edge);
        }
    }

    mst
}

fn main() {
    let edges = vec![
        Edge { u: 0, v: 1, w: 4 },
        Edge { u: 0, v: 2, w: 3 },
        Edge { u: 1, v: 2, w: 1 },
        Edge { u: 1, v: 3, w: 2 },
        Edge { u: 2, v: 3, w: 4 },
        Edge { u: 3, v: 4, w: 2 },
    ];

    let mst = kruskals(5, edges);
    let total: u32 = mst.iter().map(|e| e.w).sum();
}

```



```

    for e in &mst {
        println!("{}", e.u, e.v, e.w);
    }
    println!("total: {total}");
}

```

```

$ cargo run --bin kruskals_algorithm
1 - 2 (1)
1 - 3 (2)
3 - 4 (2)
0 - 2 (3)
total: 8

```

The tree has four edges for five nodes, as expected.

Summary

- A graph is a set of nodes and edges, with no rule against cycles. Every traversal needs a record of the nodes it has handled.
- An adjacency list stores each node's outgoing edges. The index form uses `Vec<Vec<...>>` and numbered nodes. The map form uses a `HashMap` and allows any hashable node.
- BFS uses a queue and visits nodes in order of edge count from the start. DFS uses a stack and follows one path as deep as it goes. Both run in $O(V + E)$.
- Graph nodes in Rust are usually numbers into a `Vec`, which avoids reference cycles.
- Cycle detection needs three states. An edge to a node on the current path, a Visiting node, closes a cycle.
- A grid is a graph whose edges are computed from coordinates. Counting islands runs one flood fill per island.
- Kahn's algorithm produces a topological order by repeatedly taking nodes with indegree 0. A shorter order than the node count means the graph has a cycle.
- Dijkstra's algorithm finds lowest-cost paths with a min-heap, skipping stale entries. `checked_add` and `let ... else` guard against overflow and missing nodes.
- Kruskal's algorithm builds a minimum spanning tree from the lightest edges, using union-find to skip edges that would close a loop.

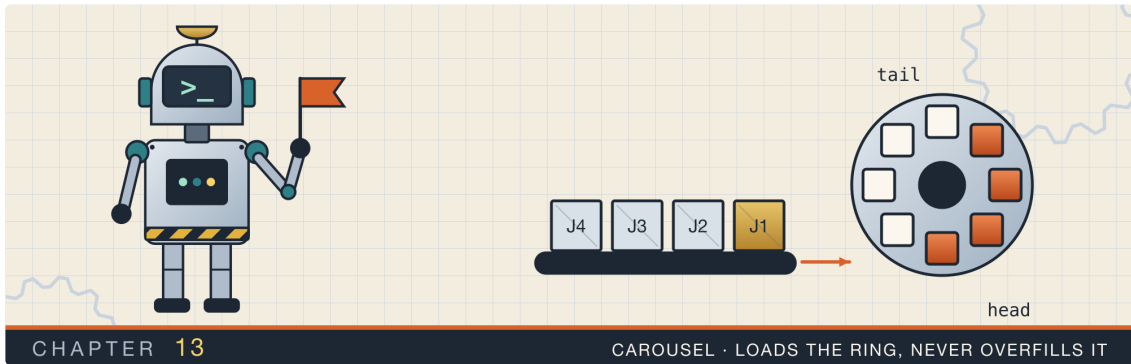
Chapter 13 builds an LRU cache, which combines a hash map with a linked list. It brings back the ownership questions of chapters 8 and 9.

Exercises

1. Change `bfs_vec` in listing 12.1 to also return, for each node, the number of edges on the shortest path from the start.
2. Extend `dijkstra_vec` to record each node's predecessor on its shortest path. Then write a function that rebuilds the path from the start to a given node.
3. Rewrite `count_islands` with a `Vec<bool>` for `visited`, and with a DFS stack instead of a BFS queue. Check that the tests still pass.
4. Change `topo_sort` in listing 12.15 to return `Option<Vec<String>>`, and add a test with a cycle.
5. Rewrite `find` in listing 12.22 with a loop instead of recursion, keeping path compression.

PART 4

Memory and the machine



An LRU cache

THIS CHAPTER COVERS

- What a cache is, and the least-recently-used rule for deciding what to throw away
- An LRU cache built from a map and a queue of stamped events
- An LRU cache built from a doubly linked list stored in a `Vec`, with indices as links
- The same list split into small operations with stated preconditions
- A benchmark of the `Vec` version against a list of `Rc<RefCell<_>>` nodes

A **cache** keeps a limited number of recent results close at hand, so a program does not have to compute or fetch them again. A web browser caches images. A database caches disk pages in memory. A CPU caches memory in small, fast storage next to its cores, which chapter 15 looks at.

A cache has a fixed size. When it is full and a new entry arrives, it must remove, or **evict**, an old entry. The rule for choosing which entry to evict is the cache's **eviction policy**.

This chapter builds a cache with the **least recently used** (LRU) policy: evict the entry that has gone longest without being used. The idea is that an entry used recently is likely to be used again soon.

You have already built both parts of an LRU cache. Chapter 3 covered `HashMap`, and chapters 8 and 9 covered linked lists and `Rc`. This chapter combines them, four ways, and then measures two of them.

13.1 What an LRU cache does

An LRU cache has two operations:

- `get(key)` returns the value for `key`, or nothing if the key is not cached. A successful `get` counts as a use, so the key becomes the most recently used.
- `put(key, value)` stores the value. The key becomes the most recently used. If the cache was full and the key is new, the least recently used entry is evicted.

Figure 13.1 follows five calls on a cache with room for two entries.

An LRU cache with room for 2 entries: the recency order after each call

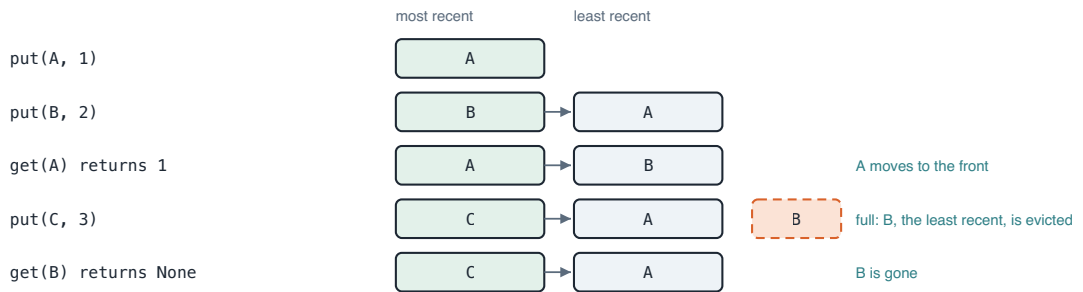


Figure 13.1 The recency order after each call. The entry at the right end is evicted next.

An LRU cache with room for two entries

The left end is the least recently used. The right end is the most recently used.



The cache is empty. Two slots are free, so `a` goes into the left one.

A hit refreshes recency without touching the other entry.

`op = put(a, 1)`

`evicted = -`

`size = 0`

Animation 13.1 The same four calls, one frame each. Two rules cover every case. A hit moves the entry to the most-recent end. A miss into a full cache evicts the entry at the least-recent end, which is the one that has gone unused longest.

The third call changes the outcome. Without `get(A)`, `A` would be the least recent entry, and `put(C, 3)` would evict `A`. Because `get(A)` counted as a use, `B` is evicted instead.

A cache needs to answer two questions fast:

1. Is this key cached, and what is its value? A `HashMap` answers that in $O(1)$.
2. Which entry is the least recently used? This needs an order that changes on every call.

Keeping the order in a `Vec` would be slow. Moving an entry from the middle to the front shifts every element between, which is $O(n)$. The four versions in this chapter keep the order in structures where each step is $O(1)$.

13.2 Version 1: stamped events in a queue

The first version avoids a linked list entirely. Its idea is to record each use as an event, and to sort out which events still count only when something must be evicted.

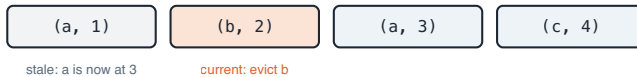
The cache has a counter called `generation` that goes up by one on every call. Each call stamps the key with the current generation in two places (figure 13.2):

- The map stores each key's value and its latest stamp: `HashMap<K, (V, u64)>`.
- A queue, `recency`, receives a `(key, stamp)` event at the back on every use.

Version 1: a map with stamps, and a queue of access events

calls: `put(a,1) put(b,2) get(a) put(c,3)`, capacity 2

`recency: VecDeque<(K, u64)>`, oldest event at the front



`put(c, 3)` makes the map hold 3 keys.

Eviction pops events from the front:

(a, 1): a's stamp is 3, not 1. Skip.

(b, 2): b's stamp is 2. Remove b.

`map: HashMap<K, (V, u64)>`, each key's value and its latest stamp



An event is current only if its stamp equals the key's stamp in the map. Older events for the same key are skipped.

Figure 13.2 The state after four calls. The event (a, 1) is stale, because a was used again at stamp 3.

The queue is in the order of use, oldest at the front. A key used several times has several events in it. Only the event whose stamp matches the map is **current**. The others are **stale**, the same idea as the stale heap entries of Dijkstra's algorithm in section 12.9.

To evict, the cache pops events from the front. It skips stale events, and removes the key of the first current event. That key is the least recently used one.

Listing 13.1 The type (lines 10 to 22). `src/problems/lru_cache.rs`

```
use std::collections::{HashMap, VecDeque};
use std::hash::Hash;

pub struct LruCache<K, V>
where
    K: Eq + Hash + Clone,
    V: Clone,
{
    capacity: usize,
    map: HashMap<K, (V, u64)>,
    recency: VecDeque<(K, u64)>,
    generation: u64,
}
```

The key must be `Clone`, because a copy of it goes into the queue with every event. The value must be `Clone`, because `get` returns a copy of it.

Listing 13.2 `get` and `put` (lines 47 to 67).

```
pub fn get(&mut self, key: &K) -> Option<V> {
    let (value, _) = self.map.get(key)?.clone();
    self.generation += 1;
    self.recency.push_back((key.clone(), self.generation));
    self.map
        .insert(key.clone(), (value.clone(), self.generation));
    Some(value)
}

pub fn put(&mut self, key: K, value: V) {
    self.generation += 1;
    let is_new = self
        .map
        .insert(key.clone(), (value, self.generation))
        .is_none();
    self.recency.push_back((key, self.generation));

    if is_new {
        self.evict_if_needed();
    }
}
```

`get` looks up the key and returns early with `None` if it is missing: `self.map.get(key)?`. Then it advances the generation, records an event, and stores the value again with the new stamp.

`put` stores the value with a new stamp, and records an event. `HashMap::insert` returns the old value if the key was already present, or `None` if it was new. Only a new key can make the map too large, so only a new key triggers eviction.

Listing 13.3 Eviction (lines 69 to 83).

```
fn evict_if_needed(&mut self) {
    while self.map.len() > self.capacity {
        let Some((key, generation)) = self.recency.pop_front() else {
            break;
        };

        match self.map.get(&key) {
            Some((_, current)) if *current == generation => {
                self.map.remove(&key);
                break;
            }
            _ => {}
        }
    }
}
```


The loop runs while the map holds more keys than the capacity. Each pass pops the oldest event. The match guard `Some((), current)) if *current == generation` accepts the event only if its stamp matches the key's stamp in the map. A match removes the key and ends the loop. A stale event matches the `_` arm and is dropped.

Each event is pushed once and popped at most once. So the total work over many calls is proportional to the number of calls, and each call costs $O(1)$ on average. A cost measured this way, averaged over a sequence of calls, is called **amortized**.

WARNING: The queue gets one new event per call, but loses events only during eviction. A workload of mostly hits evicts rarely, so the queue can grow far longer than the capacity. The memory used depends on the number of calls, not on the number of cached entries. Exercise 1 asks you to fix it.

Listing 13.4 The complete file, with tests. `src/problems/lru_cache.rs`

```

/// LRU cache without a linked list.
///
/// Design: `HashMap<K, (V, generation)>` + `VecDeque<(K, generation)>`.
///
/// Each access pushes a new `(key, generation)` event to the back. On eviction
/// we pop old events until we find the newest generation for a key, then remove
/// it. Stale events are skipped. This keeps amortized  $O(1)$  operations while
/// preserving true LRU semantics.

use std::collections::{HashMap, VecDeque};
use std::hash::Hash;

pub struct LruCache<K, V>
where
    K: Eq + Hash + Clone,
    V: Clone,
{
    capacity: usize,
    map: HashMap<K, (V, u64)>,
    recency: VecDeque<(K, u64)>,
    generation: u64,
}

impl<K, V> LruCache<K, V>
where
    K: Eq + Hash + Clone,
    V: Clone,
{
    pub fn new(capacity: usize) -> Self {
        assert!(capacity > 0, "LRU cache capacity must be > 0");
        Self {
            capacity,
            map: HashMap::new(),
            recency: VecDeque::new(),
        }
    }
}

```

```

        generation: 0,
    }
}

pub fn len(&self) -> usize {
    self.map.len()
}

pub fn is_empty(&self) -> bool {
    self.map.is_empty()
}

pub fn get(&mut self, key: &K) -> Option<V> {
    let (value, _) = self.map.get(key)?.clone();
    self.generation += 1;
    self.recency.push_back((key.clone(), self.generation));
    self.map
        .insert(key.clone(), (value.clone(), self.generation));
    Some(value)
}

pub fn put(&mut self, key: K, value: V) {
    self.generation += 1;
    let is_new = self
        .map
        .insert(key.clone(), (value, self.generation))
        .is_none();
    self.recency.push_back((key, self.generation));

    if is_new {
        self.evict_if_needed();
    }
}

fn evict_if_needed(&mut self) {
    while self.map.len() > self.capacity {
        let Some((key, generation)) = self.recency.pop_front() else {
            break;
        };

        match self.map.get(&key) {
            Some(_, current) if *current == generation => {
                self.map.remove(&key);
                break;
            }
            _ => {}
        }
    }
}

}

}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn put_get_and_len() {

```

```

    let mut cache = LruCache::new(2);
    cache.put("a".to_string(), 1);
    cache.put("b".to_string(), 2);

    assert_eq!(cache.len(), 2);
    assert_eq!(cache.get(&"a".to_string()), Some(1));
    assert_eq!(cache.get(&"missing".to_string()), None);
}

#[test]
fn evicts_least_recently_used() {
    let mut cache = LruCache::new(2);
    cache.put("a".to_string(), 1);
    cache.put("b".to_string(), 2);
    cache.put("c".to_string(), 3);

    assert_eq!(cache.get(&"a".to_string()), None);
    assert_eq!(cache.get(&"b".to_string()), Some(2));
    assert_eq!(cache.get(&"c".to_string()), Some(3));
}

#[test]
fn get_updates_recency() {
    let mut cache = LruCache::new(2);
    cache.put("a".to_string(), 1);
    cache.put("b".to_string(), 2);

    // Access "a" so it becomes most recently used.
    assert_eq!(cache.get(&"a".to_string()), Some(1));

    cache.put("c".to_string(), 3); // evicts "b"
    assert_eq!(cache.get(&"b".to_string()), None);
    assert_eq!(cache.get(&"a".to_string()), Some(1));
    assert_eq!(cache.get(&"c".to_string()), Some(3));
}

#[test]
fn updating_existing_key_moves_it_to_mru() {
    let mut cache = LruCache::new(2);
    cache.put("a".to_string(), 1);
    cache.put("b".to_string(), 2);

    // Update "a"; it becomes MRU, so "b" should be evicted next.
    cache.put("a".to_string(), 10);
    cache.put("c".to_string(), 3);

    assert_eq!(cache.get(&"a".to_string()), Some(10));
    assert_eq!(cache.get(&"b".to_string()), None);
    assert_eq!(cache.get(&"c".to_string()), Some(3));
}
}

```

The tests cover the cases of figure 13.1. They check plain eviction, a `get` that saves an entry, and a `put` to an existing key, which also counts as a use.

13.3 Version 2: a linked list inside a `Vec`

The classic LRU design keeps the entries in a doubly linked list, ordered from most to least recently used. A doubly linked list can move an entry to the front in $O(1)$. Removing the last entry and adding one at the front are $O(1)$ too. The map stores, for each key, where that key's node is.

Chapter 9 showed that a doubly linked list of `Rc<RefCell<_>>` nodes is awkward in Rust. Each node is owned jointly, every change goes through runtime borrow checks, and the back links must be `Weak` to avoid leaks.

This version uses a different approach. All nodes live in one `Vec`, and a link is the index of another node in that `Vec`. A `Vec` used as a pool of nodes this way is called an **arena**. Figure 13.3 shows one.

Version 2: nodes in a `Vec`, linked by index

after `put(A,10)` `put(B,20)` `put(C,30)` `get(A)`, capacity 3

lookup_table: `HashMap<K, usize>`

A	0
B	1
C	2

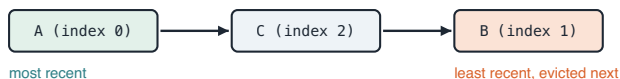
nodes: `Vec<Node<K, V>>`

index	key	value	prev	next
0	A	10	None	Some(2)
1	B	20	Some(2)	None
2	C	30	Some(0)	Some(1)

`head = Some(0)`

`tail = Some(1)`

Following next from head gives the recency order:



An index is a pointer the borrow checker does not track.

Figure 13.3 Three entries in the arena. The order in the `Vec` is the order of insertion. The `prev` and `next` indices give the order of use.

An index is a plain `usize`. The borrow checker does not track it, so a node can be "pointed to" by several other nodes with no `Rc` and no `RefCell`. The cost is that the compiler cannot catch a wrong index. The code must keep the links correct by itself.

Listing 13.5 The node and the cache (lines 20 to 46). `src/problems/lru_cache_easy.rs`

```

use std::collections::HashMap;
use std::hash::Hash;

/// A single cache entry, linked into the recency list by index.

```

```

struct Node<K, V> {
    /// Retained so we can delete the corresponding map entry when evicting.
    key: K,
    value: V,
    /// Neighbour closer to the MRU end (or `None` if this is `head`).
    prev: Option<usize>,
    /// Neighbour closer to the LRU end (or `None` if this is `tail`).
    next: Option<usize>,
}

/// Fixed-capacity LRU cache.
pub struct LruCache<K, V> {
    /// Maximum number of entries; `>= 1` (enforced by `new`).
    cap: usize,
    /// Key → arena index. The single source of truth for "is this key cached?".
    lookup_table: HashMap<K, usize>,
    /// Arena of nodes; indices are the "pointers" of the linked list.
    nodes: Vec<Node<K, V>>,
    /// Arena index of the most recently used entry.
    head: Option<usize>,
    /// Arena index of the least recently used entry (next to be evicted).
    tail: Option<usize>,
}

```

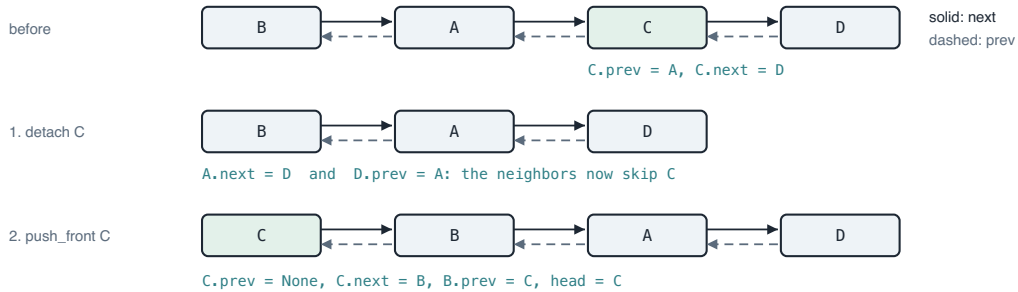
Each node stores its key as well as its value. When a node is evicted, the cache reads the key from the node to remove the key from `lookup_table`.

`prev` and `next` are `Option<usize>`, where `None` means "no neighbor on that side". `head` is the index of the most recently used node, and `tail` is the index of the least recently used one.

13.3.1 Moving a node to the front

Every `get` and `put` ends by moving one node to the front of the list. The method `touch` does it in two steps (figure 13.4):

1. **Detach** the node: make its two neighbors point at each other, so the list skips it.
2. **Push it to the front**: make it point at the old head, and make it the new head.

Moving node C to the front: detach it, then link it at the head**Figure 13.4** Moving C to the front. Only the links around C and at the head change.**Listing 13.6** `new` and `touch` (lines 52 to 104).

```

/// Create a cache holding at most `cap` entries.
///
/// # Panics
/// Panics if `cap == 0`, since a zero-capacity cache cannot store anything.
pub fn new(cap: usize) -> Self {
    assert!(cap > 0, "capacity must be > 0");
    Self {
        cap,
        lookup_table: HashMap::new(),
        nodes: Vec::new(),
        head: None,
        tail: None,
    }
}

/// Promote node `i` to `head` (MRU).
///
/// Handles both cases in one pass:
/// * **Already linked** – detach from its current neighbours first.
/// * **Brand new** – the node has `prev = next = None` and is not yet
///   referenced by `head`/`tail`, so the detach step is skipped.
fn touch(&mut self, i: usize) {
    let (prev, next) = (self.nodes[i].prev, self.nodes[i].next);

    // A node is "linked" if any of its four references exist.
    let linked = prev.is_some()
        || next.is_some()
        || self.head == Some(i)
        || self.tail == Some(i);

    if linked {
        // Splice `i` out of the list.
        match prev {
            Some(p) => self.nodes[p].next = next,
            None => self.head = next,
        }
        match next {

```

```

        Some(n) => self.nodes[n].prev = prev,
        None => self.tail = prev,
    }
}

// Splice `i` in at the front.
self.nodes[i].prev = None;
self.nodes[i].next = self.head;
if let Some(h) = self.head {
    self.nodes[h].prev = Some(i);
}
self.head = Some(i);
if self.tail.is_none() {
    self.tail = Some(i);
}
}

```

`touch` first copies the node's `prev` and `next` into local variables. `Option<usize>` is `Copy`, so this reads the values without keeping a borrow of `self.nodes`.

A brand-new node is not in the list yet, so there is nothing to detach. `touch` tells the two cases apart with `linked`. A node is in the list if it has a neighbor, or if it is the head or the tail. A list with one node has no neighbors, so the head and tail checks are needed for that case.

The detach step handles the ends of the list. If the node has no `prev`, it was the head, so the head moves to its `next`. If it has no `next`, it was the tail, so the tail moves to its `prev`.

The push step sets the node's links, points the old head back at it, and makes it the head. If the list was empty, the node is also the tail.

13.3.2 get and put

Listing 13.7 `get` and `put` (lines 106 to 149).

```

/// Look up `key`, promoting it to MRU on a hit.
///
/// Returns `None` if the key is not cached.
pub fn get(&mut self, key: &K) -> Option<&V> {
    let i = *self.lookup_table.get(key)?;
    self.touch(i);
    Some(&self.nodes[i].value)
}

/// Insert or update `key`. Promotes it to MRU. Evicts the LRU entry if
/// the cache is full.
pub fn put(&mut self, key: K, value: V) {
    // Case 1: key already present → update value, promote.
    if let Some(&i) = self.lookup_table.get(&key) {
        self.nodes[i].value = value;
        self.touch(i);
        return;
    }
}

```

```

    }

    // Case 2: room to grow → append a fresh node.
    if self.nodes.len() < self.cap {
        let i = self.nodes.len();
        self.nodes.push(Node {
            key: key.clone(),
            value,
            prev: None,
            next: None,
        });
        self.lookup_table.insert(key, i);
        self.touch(i);
        return;
    }

    // Case 3: at capacity → reuse the LRU slot.
    // `tail` is guaranteed to be `Some` here because `cap >= 1` and the
    // arena is full, but we still handle `None` gracefully.
    if let Some(i) = self.tail {
        let old_key = std::mem::replace(&mut self.nodes[i].key, key);
        self.nodes[i].value = value;
        self.lookup_table.remove(&old_key);
        self.lookup_table.insert(self.nodes[i].key.clone(), i);
        self.touch(i);
    }
}

```

`get` finds the index, touches the node, and returns a reference to the value. The return type `Option<&V>` borrows from the cache. Because `get` takes `&mut self`, the cache stays mutably borrowed while you hold that reference.

`put` has three cases:

1. **The key is present.** Replace the value and touch the node.
2. **There is room.** Push a new node onto the end of the `Vec`, record its index, and touch it.
3. **The cache is full.** Reuse the tail node's slot for the new entry.

The third case is the arena's advantage. Eviction does not free anything, and insertion does not allocate. The tail slot receives the new key and value, and `touch` moves it to the front.

`std::mem::replace(&mut self.nodes[i].key, key)` puts the new key into the node and returns the old key, in one step. The old key is then removed from `lookup_table`. The new key was moved into the node, so the map receives a clone of it.

The file `src/bin/lru_cache_arena.rs` has the same cache with a `main` function:

Listing 13.8 The `main` function (lines 152 to 167). `src/bin/lru_cache_arena.rs`

```
fn main() {
    let mut cache = LruCache::new(2);

    cache.put("A", 10);
    cache.put("B", 20);
    println!("get A = {:?}", cache.get(&"A")); // Some(10), A now MRU

    cache.put("C", 30); // evicts B (LRU)

    println!("get B = {:?}", cache.get(&"B")); // None
    println!("get A = {:?}", cache.get(&"A")); // Some(10)
    println!("get C = {:?}", cache.get(&"C")); // Some(30)

    cache.put("A", 99);
    println!("get A = {:?}", cache.get(&"A")); // Some(99)
}
```

```
$ cargo run --bin lru_cache_arena
get A = Some(10)
get B = None
get A = Some(10)
get C = Some(30)
get A = Some(99)
```

The complete library file follows, with its tests. One test uses a capacity of 1, where the single node is the head and the tail at once. Another checks that capacity 0 panics.

Listing 13.9 The complete file. `src/problems/lru_cache_easy.rs`

```
/// A fixed-capacity LRU (Least Recently Used) cache.
///
/// ## Design
///
/// Two structures cooperate to give  $O(1)$  `get` and `put`:
///
/// * `map: HashMap<K, usize>` – key  $\rightarrow$  index into the arena.
/// * `nodes: Vec<Node<K, V>>` – an arena of nodes; indices play the role
///   of pointers, sidestepping `Rc<RefCell<...>>` and its borrow-checker pain.
///
/// The arena is stitched into a doubly linked list through `head`/`tail`:
///
/// ```text
/// MRU  $\rightarrow$  head  $\rightarrow$  ...  $\rightarrow$  tail  $\rightarrow$  LRU
/// ```
///
/// On eviction we reuse the tail slot instead of removing it, so no node
/// is ever deallocated mid-life. That keeps the arena dense and index-stable.

use std::collections::HashMap;
use std::hash::Hash;

/// A single cache entry, linked into the recency list by index.
```

```

struct Node<K, V> {
    /// Retained so we can delete the corresponding map entry when evicting.
    key: K,
    value: V,
    /// Neighbour closer to the MRU end (or `None` if this is `head`).
    prev: Option<usize>,
    /// Neighbour closer to the LRU end (or `None` if this is `tail`).
    next: Option<usize>,
}

/// Fixed-capacity LRU cache.
pub struct LruCache<K, V> {
    /// Maximum number of entries; `>= 1` (enforced by `new`).
    cap: usize,
    /// Key → arena index. The single source of truth for "is this key cached?".
    lookup_table: HashMap<K, usize>,
    /// Arena of nodes; indices are the "pointers" of the linked list.
    nodes: Vec<Node<K, V>>,
    /// Arena index of the most recently used entry.
    head: Option<usize>,
    /// Arena index of the least recently used entry (next to be evicted).
    tail: Option<usize>,
}

impl<K, V> LruCache<K, V>
where
    K: Hash + Eq + Clone,
{
    /// Create a cache holding at most `cap` entries.
    ///
    /// # Panics
    /// Panics if `cap == 0`, since a zero-capacity cache cannot store anything.
    pub fn new(cap: usize) -> Self {
        assert!(cap > 0, "capacity must be > 0");
        Self {
            cap,
            lookup_table: HashMap::new(),
            nodes: Vec::new(),
            head: None,
            tail: None,
        }
    }

    /// Promote node `i` to `head` (MRU).
    ///
    /// Handles both cases in one pass:
    /// * **Already linked** – detach from its current neighbours first.
    /// * **Brand new** – the node has `prev = next = None` and is not yet
    ///   referenced by `head`/`tail`, so the detach step is skipped.
    fn touch(&mut self, i: usize) {
        let (prev, next) = (self.nodes[i].prev, self.nodes[i].next);

        // A node is "linked" if any of its four references exist.
        let linked = prev.is_some()
            || next.is_some()
            || self.head == Some(i)
            || self.tail == Some(i);

        if linked {

```

```

        // Splice `i` out of the list.
        match prev {
            Some(p) => self.nodes[p].next = next,
            None => self.head = next,
        }
        match next {
            Some(n) => self.nodes[n].prev = prev,
            None => self.tail = prev,
        }
    }

    // Splice `i` in at the front.
    self.nodes[i].prev = None;
    self.nodes[i].next = self.head;
    if let Some(h) = self.head {
        self.nodes[h].prev = Some(i);
    }
    self.head = Some(i);
    if self.tail.is_none() {
        self.tail = Some(i);
    }
}

/// Look up `key`, promoting it to MRU on a hit.
///
/// Returns `None` if the key is not cached.
pub fn get(&mut self, key: &K) -> Option<&V> {
    let i = *self.lookup_table.get(key)?;
    self.touch(i);
    Some(&self.nodes[i].value)
}

/// Insert or update `key`. Promotes it to MRU. Evicts the LRU entry if
/// the cache is full.
pub fn put(&mut self, key: K, value: V) {
    // Case 1: key already present → update value, promote.
    if let Some(&i) = self.lookup_table.get(&key) {
        self.nodes[i].value = value;
        self.touch(i);
        return;
    }

    // Case 2: room to grow → append a fresh node.
    if self.nodes.len() < self.cap {
        let i = self.nodes.len();
        self.nodes.push(Node {
            key: key.clone(),
            value,
            prev: None,
            next: None,
        });
        self.lookup_table.insert(key, i);
        self.touch(i);
        return;
    }

    // Case 3: at capacity → reuse the LRU slot.
    // `tail` is guaranteed to be `Some` here because `cap >= 1` and the
    // arena is full, but we still handle `None` gracefully.

```

```

        if let Some(i) = self.tail {
            let old_key = std::mem::replace(&mut self.nodes[i].key, key);
            self.nodes[i].value = value;
            self.lookup_table.remove(&old_key);
            self.lookup_table.insert(self.nodes[i].key.clone(), i);
            self.touch(i);
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn basic_eviction() {
        let mut c = LruCache::new(2);
        c.put("A", 1);
        c.put("B", 2);

        assert_eq!(c.get(&"A"), Some(&1)); // A is now MRU
        c.put("C", 3); // evicts B

        assert_eq!(c.get(&"B"), None);
        assert_eq!(c.get(&"A"), Some(&1));
        assert_eq!(c.get(&"C"), Some(&3));
    }

    #[test]
    fn updating_existing_key_keeps_it() {
        let mut c = LruCache::new(2);
        c.put("A", 1);
        c.put("B", 2);

        c.put("A", 100); // update A, A becomes MRU
        c.put("C", 3); // evicts B

        assert_eq!(c.get(&"A"), Some(&100));
        assert_eq!(c.get(&"B"), None);
        assert_eq!(c.get(&"C"), Some(&3));
    }

    #[test]
    fn get_refreshes_recency() {
        let mut c = LruCache::new(2);
        c.put("A", 1);
        c.put("B", 2);

        assert_eq!(c.get(&"A"), Some(&1)); // A refreshed
        c.put("C", 3); // evicts B

        assert_eq!(c.get(&"B"), None);
        assert_eq!(c.get(&"A"), Some(&1));
    }

    #[test]
    fn capacity_one() {
        let mut c = LruCache::new(1);
        c.put("A", 1);
    }
}

```

```

        c.put("B", 2); // evicts A

        assert_eq!(c.get(&"A"), None);
        assert_eq!(c.get(&"B"), Some(&2));
    }

    #[test]
    fn missing_key_returns_none() {
        let mut c: LruCache<&str, i32> = LruCache::new(2);
        assert_eq!(c.get(&"nope"), None);
    }

    #[test]
    #[should_panic(expected = "capacity must be > 0")]
    fn zero_capacity_panics() {
        let _: LruCache<&str, i32> = LruCache::new(0);
    }
}

```

13.4 Version 3: one operation per method

`touch` in version 2 does two jobs, and must first work out which situation it is in. Version 3 splits it into three methods, each with a stated precondition. A **precondition** is something that must be true when a function is called. The compiler cannot check these preconditions, so the doc comments state them.

This version also fixes the key and value types to `i32`. `i32` is `Copy`, so the code never clones a key.

Listing 13.10 The link operations (lines 63 to 94). `src/bin/lru_cache_modular.rs`

```

/// Unlink node `i` from the list. `i` must currently be linked.
fn detach(&mut self, i: usize) {
    let (prev, next) = (self.nodes[i].prev, self.nodes[i].next);
    match prev {
        Some(p) => self.nodes[p].next = next,
        None => self.head = next,
    }
    match next {
        Some(n) => self.nodes[n].prev = prev,
        None => self.tail = prev,
    }
}

/// Link node `i` in at the MRU end. `i` must currently be unlinked.
fn push_front(&mut self, i: usize) {
    self.nodes[i].prev = None;
    self.nodes[i].next = self.head;
    match self.head {
        Some(h) => self.nodes[h].prev = Some(i),
        None => self.tail = Some(i), // list was empty
    }
}

```

```

    }
    self.head = Some(i);
}

/// Promote an already-linked node to MRU.
fn promote_to_front(&mut self, i: usize) {
    if self.head == Some(i) {
        return; // already MRU
    }
    self.detach(i);
    self.push_front(i);
}

```

- `detach` unlinks a node. Its precondition is that the node is in the list.
- `push_front` links a node at the head. Its precondition is that the node is not in the list.
- `promote_to_front` combines them for a node that is in the list. If the node is already the head, it does nothing.

Each method now does one thing, with no `linked` check. `put` calls `push_front` for a new node and `promote_to_front` for an existing one, so each call satisfies its precondition.

Listing 13.11 The complete program. `src/bin/lru_cache_modular.rs`

```

///! A fixed-capacity LRU (Least Recently Used) cache.
///!
///! ## Design
///!
///! Two structures cooperate to give O(1) `get` and `put`:
///!
///! * `lookup_table: HashMap<i32, usize>` – key → index into the arena.
///! * `nodes: Vec<Node>` – an arena of nodes; indices play the role
///!   of pointers, sidestepping `Rc<RefCell<...>>` and its borrow-checker pain.
///!
///! The arena is stitched into a doubly linked list through `head`/`tail`:
///!
///! ```text
///!   MRU → head → ... → tail → LRU
///! ```
///!
///! On eviction we reuse the tail slot instead of removing it, so no node
///! is ever deallocated mid-life. That keeps the arena dense and index-stable.

use std::collections::HashMap;

///! A single cache entry, linked into the recency list by index.
struct Node {
    /// Retained so we can delete the corresponding map entry when evicting.
    key: i32,
    value: i32,
    /// Neighbour closer to the MRU end (or `None` if this is `head`).
    prev: Option<usize>,
    /// Neighbour closer to the LRU end (or `None` if this is `tail`).
    next: Option<usize>,
}

```

```

}

/// Fixed-capacity LRU cache.
pub struct LruCache {
    /// Maximum number of entries; `>= 1` (enforced by `new`).
    cap: usize,
    /// Key → arena index. The single source of truth for "is this key cached?".
    lookup_table: HashMap<i32, usize>,
    /// Arena of nodes; indices are the "pointers" of the linked list.
    nodes: Vec<Node>,
    /// Arena index of the most recently used entry.
    head: Option<usize>,
    /// Arena index of the least recently used entry (next to be evicted).
    tail: Option<usize>,
}

impl LruCache {
    /// Create a cache holding at most `cap` entries.
    ///
    /// # Panics
    /// Panics if `cap == 0`, since a zero-capacity cache cannot store anything.
    pub fn new(cap: usize) -> Self {
        assert!(cap > 0, "capacity must be > 0");
        Self {
            cap,
            lookup_table: HashMap::new(),
            nodes: Vec::new(),
            head: None,
            tail: None,
        }
    }

    /// Unlink node `i` from the list. `i` must currently be linked.
    fn detach(&mut self, i: usize) {
        let (prev, next) = (self.nodes[i].prev, self.nodes[i].next);
        match prev {
            Some(p) => self.nodes[p].next = next,
            None => self.head = next,
        }
        match next {
            Some(n) => self.nodes[n].prev = prev,
            None => self.tail = prev,
        }
    }

    /// Link node `i` in at the MRU end. `i` must currently be unlinked.
    fn push_front(&mut self, i: usize) {
        self.nodes[i].prev = None;
        self.nodes[i].next = self.head;
        match self.head {
            Some(h) => self.nodes[h].prev = Some(i),
            None => self.tail = Some(i), // list was empty
        }
        self.head = Some(i);
    }

    /// Promote an already-linked node to MRU.
    fn promote_to_front(&mut self, i: usize) {
        if self.head == Some(i) {

```

```

        return; // already MRU
    }
    self.detach(i);
    self.push_front(i);
}

/// Look up `key`, promoting it to MRU on a hit.
///
/// Returns `None` if the key is not cached.
pub fn get(&mut self, key: i32) -> Option<i32> {
    let i = *self.lookup_table.get(&key)?;
    self.promote_to_front(i);
    Some(self.nodes[i].value)
}

/// Insert or update `key`. Promotes it to MRU. Evicts the LRU entry if
/// the cache is full.
pub fn put(&mut self, key: i32, value: i32) {
    // Case 1: key already present → update value, promote.
    if let Some(&i) = self.lookup_table.get(&key) {
        self.nodes[i].value = value;
        self.promote_to_front(i);
        return;
    }

    // Case 2: room to grow → append a fresh (unlinked) node.
    if self.nodes.len() < self.cap {
        let i = self.nodes.len();
        self.nodes.push(Node {
            key,
            value,
            prev: None,
            next: None,
        });
        self.lookup_table.insert(key, i);
        self.push_front(i);
        return;
    }

    // Case 3: at capacity → reuse the LRU slot.
    // `tail` is guaranteed to be `Some` here because `cap >= 1` and the
    // arena is full, but we still handle `None` gracefully.
    if let Some(i) = self.tail {
        // Swap the new key into the node and hand back the old one it held.
        let old_key = std::mem::replace(&mut self.nodes[i].key, key);
        self.nodes[i].value = value;
        self.lookup_table.remove(&old_key);
        self.lookup_table.insert(key, i);
        self.promote_to_front(i);
    }
}

fn main() {
    let mut cache = LruCache::new(2);

    cache.put(1, 10);
    cache.put(2, 20);
    println!("get 1 = {:?}", cache.get(1)); // Some(10), 1 now MRU
}

```



```

    cache.put(3, 30); // evicts 2 (LRU)

    println!("get 2 = {:?}", cache.get(2)); // None
    println!("get 1 = {:?}", cache.get(1)); // Some(10)
    println!("get 3 = {:?}", cache.get(3)); // Some(30)

    cache.put(1, 99);
    println!("get 1 = {:?}", cache.get(1)); // Some(99)
}

#[cfg(test)]
mod tests_for_lru {
    use super::*;

    #[test]
    fn basic_eviction() {
        let mut c = LruCache::new(2);
        c.put(1, 10);
        c.put(2, 20);

        assert_eq!(c.get(1), Some(10)); // 1 is now MRU
        c.put(3, 30); // evicts 2

        assert_eq!(c.get(2), None);
        assert_eq!(c.get(1), Some(10));
        assert_eq!(c.get(3), Some(30));
    }

    #[test]
    fn updating_existing_key_keeps_it() {
        let mut c = LruCache::new(2);
        c.put(1, 10);
        c.put(2, 20);

        c.put(1, 100); // update 1, 1 becomes MRU
        c.put(3, 30); // evicts 2

        assert_eq!(c.get(1), Some(100));
        assert_eq!(c.get(2), None);
        assert_eq!(c.get(3), Some(30));
    }

    #[test]
    fn get_refreshes_recency() {
        let mut c = LruCache::new(2);
        c.put(1, 10);
        c.put(2, 20);

        assert_eq!(c.get(1), Some(10)); // 1 refreshed
        c.put(3, 30); // evicts 2

        assert_eq!(c.get(2), None);
        assert_eq!(c.get(1), Some(10));
    }

    #[test]
    fn capacity_one() {
        let mut c = LruCache::new(1);

```

```

        c.put(1, 10);
        c.put(2, 20); // evicts 1

        assert_eq!(c.get(1), None);
        assert_eq!(c.get(2), Some(20));
    }

    #[test]
    fn missing_key_returns_none() {
        let mut c = LruCache::new(2);
        assert_eq!(c.get(42), None);
    }

    #[test]
    #[should_panic(expected = "capacity must be > 0")]
    fn zero_capacity_panics() {
        let _ = LruCache::new(0);
    }
}

```

```

$ cargo run --bin lru_cache_modular
get 1 = Some(10)
get 2 = None
get 1 = Some(10)
get 3 = Some(30)
get 1 = Some(99)

```

13.5 Version 4: measuring the arena against `Rc<RefCell<_>>`

Is the arena faster than a list of `Rc<RefCell<_>>` nodes, and by how much? The only reliable answer is to measure. This section builds both caches behind one trait, and runs the same requests through each.

13.5.1 One trait, two caches

Listing 13.12 The trait. `benchmarking_examples/cache/mod.rs`

```

///! Two LRU caches of the same design, stored differently, so one benchmark loop
///! can cover both.
///!
///! [arena] keeps nodes in a Vec and links them by usize index.
///! [rc_list] allocates an Rc<RefCell<_>> per node and links them by pointer.

pub mod arena;
pub mod rc_list;

use std::hash::Hash;

/// The operations every cache supports.
pub trait Cache<K, V>

```

```

where
    K: Hash + Eq + Clone,
    V: Copy,
{
    /// A cache that holds at most `capacity` entries.
    ///
    /// # Panics
    /// Panics if `capacity == 0`, since a zero-capacity cache cannot store
    /// anything.
    fn new(capacity: usize) -> Self;

    /// Looks up `key`, promoting it to most recently used on a hit.
    fn get(&mut self, key: &K) -> Option<V>;

    /// Inserts or updates `key`, evicting the least recently used entry when
    /// the cache is full.
    fn put(&mut self, key: K, value: V);

    /// How many entries are cached.
    fn len(&self) -> usize;

    /// The variant's name, for benchmark output.
    fn variant() -> &'static str;
}

```

The trait `Cache<K, V>` lists what the benchmark needs. `new` is part of the trait, so generic code can create a cache with `C::new(capacity)` without knowing which type `C` is. `variant` has no `self` parameter. It is called on the type, and gives each cache a name for the output.

`V: Copy` lets `get` return the value itself, `Option<V>`, instead of a reference. The benchmark stores `u64` values, which are `Copy`.

Listing 13.13 The arena cache behind the trait. `benchmarking_examples/cache/arena.rs`

```

/// Fixed-capacity LRU cache backed by an arena.
///
/// ## Design
///
/// Two structures cooperate to give O(1) `get` and `put`:
///
/// * `lookup_table: HashMap<K, usize>` – key to index into the arena.
/// * `nodes: Vec<Node<K, V>>` – an arena of nodes; indices play the role of
///   pointers, sidestepping `Rc<RefCell<...>>` and its borrow-checker pain.
///
/// The arena is stitched into a doubly linked list through `head`/`tail`:
///
/// ```text
/// MRU -> head -> ... -> tail -> LRU
/// ```
///
/// On eviction the tail slot is reused instead of removed, so no node is
/// deallocated mid-life and the arena stays dense and index-stable.

use std::collections::HashMap;

```

```

use std::hash::Hash;

use super::Cache;

/// A single cache entry, linked into the recency list by index.
struct Node<K, V> {
    /// Retained so the corresponding map entry can be removed when evicting.
    key: K,
    value: V,
    /// Neighbour closer to the MRU end (`None` if this is `head`).
    prev: Option<usize>,
    /// Neighbour closer to the LRU end (`None` if this is `tail`).
    next: Option<usize>,
}

pub struct LruCache<K, V> {
    /// Maximum number of entries; `>= 1`.
    cap: usize,
    /// Key to arena index. The single source of truth for "is this cached?".
    lookup_table: HashMap<K, usize>,
    /// Arena of nodes; indices are the "pointers" of the linked list.
    nodes: Vec<Node<K, V>>,
    /// Arena index of the most recently used entry.
    head: Option<usize>,
    /// Arena index of the least recently used entry (next to evict).
    tail: Option<usize>,
}

impl<K, V> LruCache<K, V>
where
    K: Hash + Eq + Clone,
    V: Copy,
{
    /// Promotes node `i` to `head` (MRU).
    ///
    /// Handles both cases in one pass:
    /// * **Already linked** - detach from its current neighbours first.
    /// * **Brand new** - the node has `prev = next = None` and is not yet
    ///   referenced by `head`/`tail`, so the detach step is skipped.
    fn touch(&mut self, i: usize) {
        let (prev, next) = (self.nodes[i].prev, self.nodes[i].next);

        // A node is "linked" if any of its four references exist.
        let linked =
            prev.is_some() || next.is_some() || self.head == Some(i) || self.tail ==
Some(i);

        if linked {
            // Splice `i` out of the list.
            match prev {
                Some(p) => self.nodes[p].next = next,
                None => self.head = next,
            }
            match next {
                Some(n) => self.nodes[n].prev = prev,
                None => self.tail = prev,
            }
        }
    }
}

```

```

        // Splice `i` in at the front.
        self.nodes[i].prev = None;
        self.nodes[i].next = self.head;
        if let Some(h) = self.head {
            self.nodes[h].prev = Some(i);
        }
        self.head = Some(i);
        if self.tail.is_none() {
            self.tail = Some(i);
        }
    }
}

impl<K, V> Cache<K, V> for LruCache<K, V>
where
    K: Hash + Eq + Clone,
    V: Copy,
{
    fn new(capacity: usize) -> Self {
        assert!(capacity > 0, "capacity must be > 0");
        Self {
            cap: capacity,
            lookup_table: HashMap::new(),
            nodes: Vec::new(),
            head: None,
            tail: None,
        }
    }

    fn get(&mut self, key: &K) -> Option<V> {
        let i = *self.lookup_table.get(key)?;
        self.touch(i);
        Some(self.nodes[i].value)
    }

    fn put(&mut self, key: K, value: V) {
        // Case 1: key already present, update the value and promote.
        if let Some(&i) = self.lookup_table.get(&key) {
            self.nodes[i].value = value;
            self.touch(i);
            return;
        }

        // Case 2: room to grow, append a fresh node.
        if self.nodes.len() < self.cap {
            let i = self.nodes.len();
            self.nodes.push(Node {
                key: key.clone(),
                value,
                prev: None,
                next: None,
            });
            self.lookup_table.insert(key, i);
            self.touch(i);
            return;
        }

        // Case 3: at capacity, reuse the LRU slot. `tail` is `Some` here because
        // `cap >= 1` and the arena is full, but handle `None` anyway.
    }
}

```

```

        if let Some(i) = self.tail {
            let old_key = std::mem::replace(&mut self.nodes[i].key, key);
            self.nodes[i].value = value;
            self.lookup_table.remove(&old_key);
            self.lookup_table.insert(self.nodes[i].key.clone(), i);
            self.touch(i);
        }
    }

    fn len(&self) -> usize {
        self.lookup_table.len()
    }

    fn variant() -> &'static str {
        "arena (Vec + indices)"
    }
}

```

This is version 2, split into two `impl` blocks. `touch` stays in an ordinary `impl`, because it is not part of the trait. The trait methods go in `impl<K, V> Cache<K, V> for LruCache<K, V>.`

13.5.2 The `Rc<RefCell<_>>` list

Listing 13.14 The node and the cache (lines 10 to 31). `benchmarking_examples/cache/rc_list.rs`

```

use std::cell::RefCell;
use std::collections::HashMap;
use std::hash::Hash;
use std::rc::{Rc, Weak};

use super::Cache;

type Link<K, V> = Option<Rc<RefCell<Node<K, V>>>>;

struct Node<K, V> {
    key: K,
    value: V,
    prev: Weak<RefCell<Node<K, V>>>,
    next: Link<K, V>,
}

pub struct LruCache<K, V> {
    cap: usize,
    map: HashMap<K, Rc<RefCell<Node<K, V>>>>,
    head: Link<K, V>,
    tail: Link<K, V>,
}

```

This is the doubly linked list of chapter 9. A node is `Rc<RefCell<Node>>`: `Rc` so several owners can hold it, and `RefCell` so it can be changed through those shared handles. `next` is a

strong link, and `prev` is a `Weak` link, so two neighbors do not keep each other alive forever. The map holds one more `Rc` to every node.

Listing 13.15 `detach` and `push_front` (lines 33 to 67).

```
impl<K, V> LruCache<K, V>
where
  K: Hash + Eq + Clone,
  V: Copy,
{
  fn detach(&mut self, node: &Rc<RefCell<Node<K, V>>>) {
    let (prev, next) = {
      let entry = node.borrow();
      (entry.prev.clone(), entry.next.clone())
    };

    match prev.upgrade() {
      Some(previous) => previous.borrow_mut().next = next.clone(),
      None => self.head = next.clone(),
    }
    match &next {
      Some(following) => following.borrow_mut().prev = prev,
      None => self.tail = prev.upgrade(),
    }
  }

  fn push_front(&mut self, node: Rc<RefCell<Node<K, V>>>) {
    let head = self.head.clone();
    {
      let mut entry = node.borrow_mut();
      entry.prev = Weak::new();
      entry.next = head.clone();
    }
    match &head {
      Some(previous) => previous.borrow_mut().prev = Rc::downgrade(&node),
      None => self.tail = Some(node.clone()),
    }
    self.head = Some(node);
  }
}
```

The logic matches listing 13.10, with pointers instead of indices. The difference is in the ceremony around each step:

- `detach` reads the node's links inside a small block. `node.borrow()` returns a guard that holds a shared borrow of the `RefCell`. The block ends before any neighbor is changed, so the guard is dropped early. A `RefCell` panics if a `borrow_mut` happens while another borrow of the same cell is alive.
- `prev.upgrade()` turns the `Weak` link into an `Rc`, if the node still exists.

- Each `.clone()` of an `Rc` or a `Weak` increases a reference count, and each drop decreases one.

Listing 13.16 The trait methods and `Drop` (lines 69 to 137).

```
impl<K, V> Cache<K, V> for LruCache<K, V>
where
    K: Hash + Eq + Clone,
    V: Copy,
{
    fn new(capacity: usize) -> Self {
        assert!(capacity > 0, "capacity must be > 0");
        Self {
            cap: capacity,
            map: HashMap::new(),
            head: None,
            tail: None,
        }
    }

    fn get(&mut self, key: &K) -> Option<V> {
        let node = self.map.get(key)?.clone();
        self.detach(&node);
        self.push_front(node.clone());
        Some(node.borrow().value)
    }

    fn put(&mut self, key: K, value: V) {
        if let Some(node) = self.map.get(&key).cloned() {
            node.borrow_mut().value = value;
            self.detach(&node);
            self.push_front(node);
            return;
        }

        if self.map.len() >= self.cap {
            if let Some(victim) = self.tail.clone() {
                let old_key = victim.borrow().key.clone();
                self.map.remove(&old_key);
                self.detach(&victim);
            }
        }

        let node = Rc::new(RefCell::new(Node {
            key: key.clone(),
            value,
            prev: Weak::new(),
            next: None,
        }));
        self.map.insert(key, node.clone());
        self.push_front(node);
    }

    fn len(&self) -> usize {
        self.map.len()
    }
}
```



```

    fn variant() -> &'static str {
        "Rc<RefCell> list"
    }
}

impl<K, V> Drop for LruCache<K, V> {
    fn drop(&mut self) {
        // Unlink iteratively: dropping a million-node chain recursively overflows
        // the stack.
        let mut current = self.head.take();
        while let Some(node) = current {
            let next = node.borrow_mut().next.take();
            node.borrow_mut().prev = Weak::new();
            current = next;
        }
    }
}

```

On a miss with a full cache, `put` removes the tail node from the map and detaches it. When `victim` goes out of scope, the last `Rc` to that node is dropped, and its memory is freed. Then `Rc::new` allocates a new node for the new key. So every eviction frees one heap block and allocates another.

The `Drop` implementation is the fix from chapter 9. Dropping a long chain of `next` links recursively uses one stack frame per node, and a large cache would overflow the stack. The loop breaks the links one node at a time. The map drops its `Rc` handles afterward, and each drop then frees a single node.

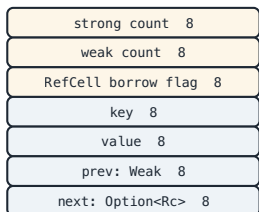
Figure 13.5 compares where the two caches keep their nodes.

Where the nodes live: one block for the arena, one allocation per node for the Rc list

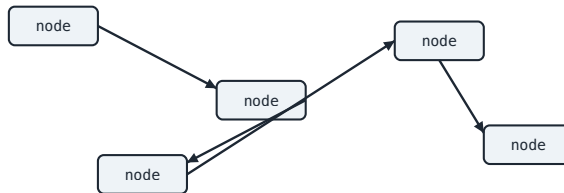
arena: `Vec<Node<u64, u64>>`, 48 bytes per node, side by side



Rc list: each node is a separate heap block of 56 bytes



Rc and RefCell
bookkeeping



Blocks sit wherever the allocator put them. A miss frees one and allocates another.

Figure 13.5 Node storage for `u64` keys and values. The sizes were measured with `std::mem::size_of`.

The arena's node is 48 bytes: the key and value take 8 bytes each, and each `Option<usize>` takes 16. The `Rc` node is 32 bytes. The heap block around it adds a strong count, a weak count, and the `RefCell` flag, for 56 bytes. The memory sizes are close. What differs is the number of allocations and where the nodes sit in memory.

13.5.3 The benchmark

Listing 13.17 The request stream and the timed loop (lines 51 to 105). `benchmarking_examples/benchmark.rs`

```
// -----
// Cache: arena vs Rc<RefCell> linked list
// -----

/// Request stream with realistic locality: four requests in five come from a hot
/// set of 1% of the keys, the rest from the whole space.
fn key_at(state: u64) -> u64 {
    let mixed = state >> 17;
    if (state >> 33) % 100 < 80 {
        mixed % (KEY_SPACE / 100)
    } else {
        mixed % KEY_SPACE
    }
}

/// The fastest of `CACHE_RUNS` timed passes, with the hit count and final size
/// from the last one. Each pass starts from an empty cache, so the stream is the
/// same every time.
fn best_cache_run<C: Cache<u64, u64>>() -> (Duration, u64, usize) {
    let mut best = Duration::MAX;
    let mut hits = 0;
    let mut len = 0;

    for _ in 0..CACHE_RUNS {
        let mut cache = C::new(CACHE_CAPACITY);
        let (elapsed, run_hits) = cache_workload(&mut cache);
        best = best.min(elapsed);
        hits = run_hits;
        len = cache.len();
    }
    (best, hits, len)
}

/// The same request stream against any cache.
fn cache_workload<C: Cache<u64, u64>>(cache: &mut C) -> (Duration, u64) {
    let mut state = 0x9e37_79b9_7f4a_7c15u64;
    let mut hits = 0u64;

    let start = Instant::now();
    for _ in 0..OPERATIONS {
        state = state
            .wrapping_mul(6_364_136_223_846_793_005)
            .wrapping_add(1_442_695_040_888_963_407);
        let key = key_at(state);
```

```

        match cache.get(&key) {
            Some(value) => {
                hits += 1;
                black_box(value);
            }
            None => cache.put(key, key.rotate_left(17)),
        }
    }
    (start.elapsed(), hits)
}

```

A fair comparison needs both caches to see exactly the same requests. The program makes its own request stream from a number called `state`. Each step multiplies and adds two fixed constants, so the same starting `state` always produces the same sequence. A generator of this kind is called a **linear congruential generator**.

`key_at` shapes the stream to look like real traffic. Real caches see some keys far more often than others. Here, 80% of requests pick from a hot set of 1% of the keys, and the rest pick from all 131,072 keys. The capacity, 65,536, is half the key space.

Each request calls `get`. On a miss, it calls `put`, the way a program fills a cache after it computes a missing value. `black_box` tells the compiler to treat the value as used, so the optimizer cannot remove the work.

Listing 13.18 Running and checking both caches (lines 107 to 157).

```

fn cache_benchmark() {
    println!(
        "LRU cache: {} entries, {} requests over {} keys",
        with_thousands(CACHE_CAPACITY as u64),
        with_thousands(OPERATIONS),
        with_thousands(KEY_SPACE)
    );
    println!("A miss inserts and evicts the least recently used entry.");
    println!("Best of {CACHE_RUNS} runs per cache.\n");

    let (arena_time, arena_hits, arena_len) = best_cache_run::<ArenaCache<u64, u64>>();
    let (list_time, list_hits, list_len) = best_cache_run::<RcCache<u64, u64>>();

    // Same population and same hit count, or this compares nothing.
    assert_eq!(arena_len, list_len);
    assert_eq!(
        arena_hits, list_hits,
        "the two runs saw different request streams"
    );

    println!(
        " hit rate: {:.1}% ({} hits, {} inserts)\n",
        arena_hits as f64 * 100.0 / OPERATIONS as f64,
        with_thousands(arena_hits),
    );
}

```

```

        with_thousands(OPERATIONS - arena_hits)
    );

    let per_request = |elapsed: Duration| elapsed.as_nanos() as f64 / OPERATIONS as f64;

    // `{:2?}` uses Duration's own Debug format, rounded to two decimals.
    println!("  {:<22} {:>12} {:>13}", "storage", "total", "per request");
    println!("  {}", "-".repeat(48));
    for (name, elapsed) in [
        (
            <ArenaCache<u64, u64> as Cache<u64, u64>>::variant(),
            arena_time,
        ),
        (<RcCache<u64, u64> as Cache<u64, u64>>::variant(), list_time),
    ] {
        println!(
            "  {name:<22} {elapsed:>12.2?} {:>8.0} ns",
            per_request(elapsed)
        );
    }

    let speedup = list_time.as_secs_f64() / arena_time.as_secs_f64();
    println!(
        "\n  the arena is {speedup:.2}x faster on the same requests: {arena_time:.2?}
    against {list_time:.2?}"
    );
    println!("  the arena reuses the evicted slot; the list frees a node and allocates
    another");
}

```

`best_cache_run::<ArenaCache<u64, u64>>()` runs the generic function with the arena cache, and the next line runs it with the `Rc` cache. The `::<...>` syntax, called the *turbofish*, names the type parameter explicitly.

Before printing any times, the function checks that both caches finished with the same size and the same number of hits. Two correct LRU caches given the same requests make the same decisions. If the counts differed, one cache would have a bug, and its time would mean nothing.

`<ArenaCache<u64, u64> as Cache<u64, u64>>::variant()` is the fully qualified way to call a trait function that has no `self`. It names both the type and the trait.

Listing 13.19 The complete benchmark program. `benchmarking_examples/benchmark.rs`

```

//! One main for every benchmark in the lab.
//!
//! Usage:
//!   cargo run --release --bin benchmark           # cache, then lists
//!   cargo run --release --bin benchmark cache     # cache only
//!   cargo run --release --bin benchmark list      # all four variants
//!   cargo run --release --bin benchmark list enum 500000 # one variant, one size
//!
//! The recursive list variants abort with a stack overflow past roughly 265,000
//! nodes, so run them below that size or on their own.

```

```

use std::hint::black_box;
use std::time::{Duration, Instant};

use systems_lab::cache::Cache;
use systems_lab::cache::arena::LruCache as ArenaCache;
use systems_lab::cache::rc_list::LruCache as RcCache;
use systems_lab::lists::SinglyList;
use systems_lab::lists::{boxed, boxed_drop, enum_drop, enum_node};

const CACHE_CAPACITY: usize = 65_536;
const KEY_SPACE: u64 = 131_072;
const OPERATIONS: u64 = 10_000_000;
const DEFAULT_LIST_ELEMENTS: usize = 200_000;
/// Timed passes per cache; the fastest is reported, since a slow pass is noise.
const CACHE_RUNS: usize = 3;
/// Timed passes per list variant.
const LIST_RUNS: usize = 3;

fn main() {
    let arguments: Vec<String> = std::env::args().skip(1).collect();

    match arguments.first().map(String::as_str) {
        Some("cache") => cache_benchmark(),
        Some("list") => {
            let variant = arguments.get(1).map(String::as_str);
            let elements = arguments
                .get(2)
                .and_then(|value| value.parse().ok())
                .unwrap_or(DEFAULT_LIST_ELEMENTS);
            list_benchmark(variant, elements);
        }
        _ => {
            cache_benchmark();
            println!();
            list_benchmark(None, DEFAULT_LIST_ELEMENTS);
        }
    }
}

// -----
// Cache: arena vs Rc<RefCell> linked list
// -----

/// Request stream with realistic locality: four requests in five come from a hot
/// set of 1% of the keys, the rest from the whole space.
fn key_at(state: u64) -> u64 {
    let mixed = state >> 17;
    if (state >> 33) % 100 < 80 {
        mixed % (KEY_SPACE / 100)
    } else {
        mixed % KEY_SPACE
    }
}

/// The fastest of `CACHE_RUNS` timed passes, with the hit count and final size
/// from the last one. Each pass starts from an empty cache, so the stream is the
/// same every time.
fn best_cache_run<C: Cache<u64, u64>>() -> (Duration, u64, usize) {

```

```

let mut best = Duration::MAX;
let mut hits = 0;
let mut len = 0;

for _ in 0..CACHE_RUNS {
    let mut cache = C::new(CACHE_CAPACITY);
    let (elapsed, run_hits) = cache_workload(&mut cache);
    best = best.min(elapsed);
    hits = run_hits;
    len = cache.len();
}
(best, hits, len)
}

/// The same request stream against any cache.
fn cache_workload<C: Cache<u64, u64>>(cache: &mut C) -> (Duration, u64) {
    let mut state = 0x9e37_79b9_7f4a_7c15u64;
    let mut hits = 0u64;

    let start = Instant::now();
    for _ in 0..OPERATIONS {
        state = state
            .wrapping_mul(6_364_136_223_846_793_005)
            .wrapping_add(1_442_695_040_888_963_407);
        let key = key_at(state);

        match cache.get(&key) {
            Some(value) => {
                hits += 1;
                black_box(value);
            }
            None => cache.put(key, key.rotate_left(17)),
        }
    }
    (start.elapsed(), hits)
}

fn cache_benchmark() {
    println!(
        "LRU cache: {} entries, {} requests over {} keys",
        with_thousands(CACHE_CAPACITY as u64),
        with_thousands(OPERATIONS),
        with_thousands(KEY_SPACE)
    );
    println!("A miss inserts and evicts the least recently used entry.");
    println!("Best of {CACHE_RUNS} runs per cache.\n");

    let (arena_time, arena_hits, arena_len) = best_cache_run::<ArenaCache<u64, u64>>();
    let (list_time, list_hits, list_len) = best_cache_run::<RcCache<u64, u64>>();

    // Same population and same hit count, or this compares nothing.
    assert_eq!(arena_len, list_len);
    assert_eq!(
        arena_hits, list_hits,
        "the two runs saw different request streams"
    );

    println!(
        " hit rate: {:.1}% ({} hits, {} inserts)\n",

```

```

        arena_hits as f64 * 100.0 / OPERATIONS as f64,
        with_thousands(arena_hits),
        with_thousands(OPERATIONS - arena_hits)
    );

    let per_request = |elapsed: Duration| elapsed.as_nanos() as f64 / OPERATIONS as f64;

    // `{:2?}` uses Duration's own Debug format, rounded to two decimals.
    println!("  {:<22} {:>12} {:>13}", "storage", "total", "per request");
    println!("  {}", "-".repeat(48));
    for (name, elapsed) in [
        (
            <ArenaCache<u64, u64> as Cache<u64, u64>>::variant(),
            arena_time,
        ),
        (<RcCache<u64, u64> as Cache<u64, u64>>::variant(), list_time),
    ] {
        println!(
            "    {name:<22} {elapsed:>12.2?} {:>8.0} ns",
            per_request(elapsed)
        );
    }

    let speedup = list_time.as_secs_f64() / arena_time.as_secs_f64();
    println!(
        "\n  the arena is {speedup:.2}x faster on the same requests: {arena_time:.2?}
    against {list_time:.2?}"
    );
    println!("  the arena reuses the evicted slot; the list frees a node and allocates
    another");
}

// -----
// Lists: four variants, one loop
// -----

fn list_benchmark(variant: Option<&str>, elements: usize) {
    println!(
        "Singly linked list: push {} values, pop them back in order, then drop a full
    list",
        with_thousands(elements as u64)
    );
    println!("Best of {LIST_RUNS} runs per variant, allocator warmed up.\n");
    println!(
        "  {:<10} {:>12} {:>13} {:>13}",
        "variant", "elements", "push", "drop"
    );
    println!("  {}", "-".repeat(52));

    match variant {
        Some("box") => run_list::<boxed::LinkedList<u64>>(elements),
        Some("enum") => run_list::<enum_node::LinkedList<u64>>(elements),
        Some("box+drop") => run_list::<boxed_drop::LinkedList<u64>>(elements),
        Some("enum+drop") => run_list::<enum_drop::LinkedList<u64>>(elements),
        _ => {
            run_list::<boxed::LinkedList<u64>>(elements);
            run_list::<enum_node::LinkedList<u64>>(elements);
            run_list::<boxed_drop::LinkedList<u64>>(elements);
            run_list::<enum_drop::LinkedList<u64>>(elements);
        }
    }
}

```

```

    }
}

fn run_list<L: SinglyList<u64>>(elements: usize) {
    // Warm the allocator first: without this the first variant in a process pays
    // for fresh heap pages and looks several times slower than an identical one.
    let mut warm_up = L::new();
    for value in 0..elements as u64 {
        warm_up.push_front(value);
    }
    drop(warm_up);

    let mut best_push = Duration::MAX;
    let mut best_drop = Duration::MAX;

    for _ in 0..LIST_RUNS {
        let mut list = L::new();

        let start = Instant::now();
        for value in 0..elements as u64 {
            list.push_front(value);
        }
        best_push = best_push.min(start.elapsed());

        // Pop every value back and check the order, so a broken list cannot post
        // a fast time.
        let mut expected = elements as u64;
        while let Some(value) = list.pop_front() {
            expected -= 1;
            assert_eq!(value, expected, "{} lost LIFO order", L::variant());
        }
        assert_eq!(expected, 0);
        black_box(expected);

        // Rebuild, then time the drop of a full list. This is where the
        // recursive variants overflow the stack.
        for value in 0..elements as u64 {
            list.push_front(value);
        }
        let start = Instant::now();
        drop(list);
        best_drop = best_drop.min(start.elapsed());
    }

    println!(
        " {:<10} {:>12} {best_push:>13.2?} {best_drop:>13.2?}",
        L::variant(),
        with_thousands(elements as u64)
    );
}

fn with_thousands(value: u64) -> String {
    let digits = value.to_string();
    let mut out = String::with_capacity(digits.len() + digits.len() / 3);
    for (index, digit) in digits.chars().enumerate() {
        if index > 0 && (digits.len() - index) % 3 == 0 {
            out.push(',');
        }
    }
}

```



```

        out.push(digit);
    }
    out
}

```

The list half of this file was explained in section 9.7. Run only the cache half with the `cache` argument, and always with `--release`. A debug build is not optimized, and its times do not show how the code performs.

```

$ cargo run --release --bin benchmark cache
LRU cache: 65,536 entries, 10,000,000 requests over 131,072 keys
A miss inserts and evicts the least recently used entry.
Best of 3 runs per cache.

```

```

hit rate: 89.8% (8,979,314 hits, 1,020,686 inserts)

```

storage	total	per request
arena (Vec + indices)	587.49ms	59 ns
Rc<RefCell> list	1.86s	186 ns

```

the arena is 3.16x faster on the same requests: 587.49ms against 1.86s
the arena reuses the evicted slot; the list frees a node and allocates another

```

10,000,000 requests, 89.8% hits, capacity 65,536, i5-1038NG7, best of 3

arena (Vec + indices)



59 ns per request

Rc<RefCell> list



186 ns per request

Measured with cargo run --release --bin benchmark cache. Times vary between runs and machines.

Figure 13.6 Time per request on an Intel Core i5-1038NG7 laptop.

On this machine, the arena handled a request in 59 ns on average, and the `Rc` list took 186 ns. Your numbers will differ, but the ratio should be similar on most machines.

The two caches do the same $O(1)$ work per request. Three differences in how they do it add up:

- **Allocation.** About one request in ten is a miss, and each miss in the `Rc` list frees a node and allocates another. The arena reuses the tail slot.
- **Bookkeeping.** The `Rc` list updates reference counts on every clone and drop, and checks the `RefCell` borrow flag on every access. The arena reads and writes plain integers.

- **Location.** The arena's nodes sit side by side in one block. The `Rc` nodes sit wherever the allocator put them. Chapter 15 explains why nearby memory is faster to reach.

This benchmark measures the total, and does not separate the three costs. Exercise 4 suggests a way to separate them.

Summary

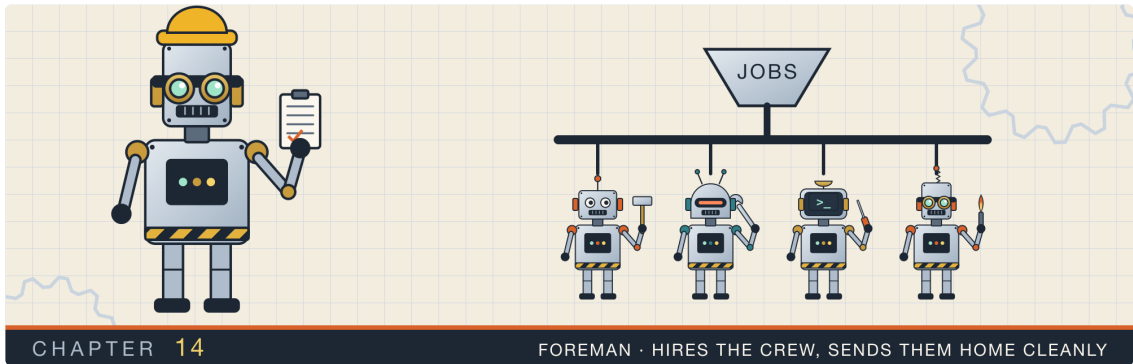
- An LRU cache evicts the entry that has gone longest without use. It needs fast lookup by key and a recency order that changes on every call.
- Version 1 records each use as a stamped event in a queue, and skips stale events during eviction. It is amortized $O(1)$, but its queue grows with the number of calls.
- Version 2 keeps a doubly linked list in a `Vec`, with indices as links. Eviction reuses the tail slot, so the cache stops allocating once it is full.
- Version 3 splits the list operations into `detach`, `push_front`, and `promote_to_front`, each with one job and a stated precondition.
- Behind a common trait, the arena cache handled requests about three times faster than an `Rc<RefCell<_>>` list in this benchmark.
- A fair benchmark feeds both versions the same input, checks that their results agree, and runs in release mode.

Chapter 14 spreads a cache across several machines, and asks how to decide which machine holds which key.

Exercises

1. Bound the queue in version 1. When `recency` grows longer than twice the capacity, rebuild it with only the current events. Check that the cost stays amortized $O(1)$.
2. Add `peek(&self, key: &K) -> Option<&V>` to version 2, which returns a value without changing the order.
3. Add `remove(&mut self, key: &K) -> Option<V>` to version 3. The removed slot must be reused by a later `put`. Keep a list of free slots.

4. Change the benchmark's `CACHE_CAPACITY` to the full key space, so no request evicts anything. Compare the ratio with the one above, and explain what the change tells you about the cost of allocation.



Shards and consistent hashing

THIS CHAPTER COVERS

- Why one lock around a whole map makes threads wait, and splitting the map into shards
- Choosing a shard from a hash with a bit mask
- Reading a value under a lock without cloning it, by passing a closure
- Why `hash % N` moves almost every key when `N` changes
- Consistent hashing: a ring of virtual nodes, searched with `partition_point`

The caches of chapter 13 serve one thread at a time. This chapter asks what happens when many threads, or many machines, share one set of keys.

Both halves of the chapter use the same idea. You split the keys into groups by their hash, and each group is handled separately. A group of keys handled separately is called a **shard**.

Section 14.1 splits a map inside one program, so threads that use different keys do not wait for each other. Section 14.2 splits keys across machines, so that adding or removing a machine moves as few keys as possible.

14.1 A map split into shards

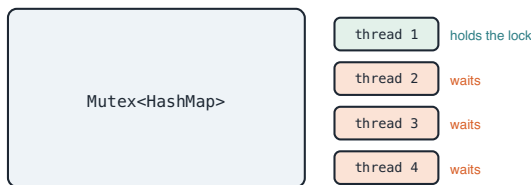
Section 8.6 showed how to share a value between threads: wrap it in a `Mutex`, and share the `Mutex` with an `Arc`. A thread calls `lock()` before it touches the value. While one thread holds the lock, every other thread that calls `lock()` waits.

A single `Mutex<HashMap<K, V>>` is safe, but it has a cost. If four threads use the map at once, three of them wait, even when each thread wants a different key. Waiting for a lock that another thread holds is called **lock contention**.

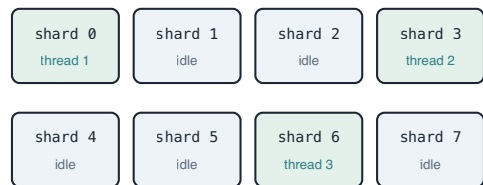
The fix is to give the map several locks. Split it into, say, eight smaller maps, each with its own `Mutex` (figure 14.1). A key always goes to the same small map. Two threads then wait for each other only when their keys land in the same shard.

One lock for the whole map, or one lock per shard

`Mutex<HashMap>`: every thread waits for the one lock



`ShardedCache`: threads on different shards run together



Two threads wait for each other only when their keys hash to the same shard.

Figure 14.1 One lock for everything, against one lock per shard.

14.1.1 The type

Listing 14.1 The type and the constructor (lines 16 to 42). `src/problems/sharded_cache.rs`

```
/// A fixed-size map split across independently locked shards.
///
/// The shard count is rounded up to a power of two so the index is a mask instead
/// of a modulo.
pub struct ShardedCache<K, V> {
    shards: Vec<Mutex<HashMap<K, V>>>,
    hasher: RandomState,
    mask: usize,
}

/// Create a cache with at least `shard_count` shards (rounded to a power of two).
pub fn new(shard_count: usize) -> Self {
    let shard_count = shard_count.max(1).next_power_of_two();
```

```

let shards = (0..shard_count)
    .map(|_| Mutex::new(HashMap::new()))
    .collect();
Self {
    shards,
    hasher: RandomState::new(),
    mask: shard_count - 1,
}
}

```

`ShardedCache` has three fields:

- `shards` is a `Vec` of maps, each inside its own `Mutex`.
- `hasher` produces the hash that picks a shard for a key.
- `mask` turns a hash into a shard number, as the next section explains.

`new` builds the `Vec` with an iterator. `(0..shard_count).map(|_| Mutex::new(HashMap::new()))` makes one empty, locked map per shard, and `collect()` gathers them. The `_` in the closure ignores the shard number, which is not needed.

The first line of `new` adjusts the requested count. `shard_count.max(1)` turns a request for 0 shards into 1. `.next_power_of_two()` rounds up to the next power of two, so a request for 6 shards gives 8.

14.1.2 Picking a shard with a mask

A shard count that is a power of two lets the code pick a shard cheaply. For 8 shards, the mask is $8 - 1 = 7$, which is `0b111` in binary. The operation `hash & mask` keeps the lowest three bits of the hash and clears the rest. The result is a number from 0 to 7 (figure 14.2).

Picking a shard: keep the low bits of the hash



With a power-of-two count, `hash & (count - 1)` equals `hash % count`, computed with one AND instruction.

Figure 14.2 `hash & mask` keeps the low bits. For a power-of-two count, it gives the same result as `hash % count`.

`hash % 8` would give the same answer. The `&` is one simple CPU instruction, while `%` is a division, which takes more time. The difference is small, but this code runs on every call.

Listing 14.2 Picking a shard (lines 44 to 49).

```
fn shard(&self, key: &K) -> &Mutex<HashMap<K, V>> {
    let mut hasher = self.hasher.build_hasher();
    key.hash(&mut hasher);
    let index = (hasher.finish() as usize) & self.mask;
    &self.shards[index]
}
```

The method creates a hasher, feeds the key into it with `key.hash(&mut hasher)`, and reads the result with `finish()`. It returns a reference to the chosen shard's `Mutex`.

The cache keeps one `RandomState` for its whole life. `RandomState::new()` picks random secret keys for the hash function when the cache is created. Every lookup in the same cache uses the same keys, so a key always maps to the same shard. A different cache, or the same program run again, would place the keys differently.

`HashMap` uses `RandomState` for the same reason. Random keys stop an attacker from choosing inputs that all hash to the same place and slow the map down.

14.1.3 Each method locks one shard

Listing 14.3 `insert`, `get`, and `with` (lines 51 to 78).

```
/// Insert a value, returning the previous one if the key was present.
pub fn insert(&self, key: K, value: V) -> Option<V> {
    self.shard(&key)
        .lock()
        .expect("shard mutex poisoned")
        .insert(key, value)
}

/// Read a value by cloning it out of the shard.
pub fn get(&self, key: &K) -> Option<V>
where
    V: Clone,
{
    self.shard(key)
        .lock()
        .expect("shard mutex poisoned")
        .get(key)
        .cloned()
}

/// Run `inspect` against the value under the shard lock, without cloning.
///
```

```

/// Use this for read-modify-write on a single key; the closure holds the lock
/// for its duration, so keep it short and do not call back into the cache.
pub fn with<R>(&self, key: &K, inspect: impl FnOnce(Option<&V>) -> R) -> R {
    let guard = self.shard(key).lock().expect("shard mutex poisoned");
    inspect(guard.get(key))
}

```

Each method follows the same three steps: pick the shard, lock it, and call the matching `HashMap` method. The methods take `&self`, not `&mut self`. The `Mutex` provides the permission to change the map, so a shared reference to the cache is enough. Several threads can then share one cache through an `Arc`.

`lock()` returns a `Result`. It is an error only if another thread panicked while it held the same lock. Such a lock is called **poisoned**, and chapter 16 covers it. Here `.expect(...)` turns that error into a panic with a clear message.

The lock stays held until the guard returned by `lock()` is dropped. In `insert`, the guard is a temporary value in one expression, so it is dropped at the end of the statement.

`get` must return a clone of the value, which is why it asks for `V: Clone`. A reference to the value would point into the map, and the map is only reachable through the guard. When `get` returns, the guard is dropped and the lock released. A reference could then be read while another thread changes the map, so the compiler does not allow it to escape.

14.1.4 Reading under the lock with a closure

Cloning a large value only to read one field of it wastes work. `with` avoids the clone. You pass it a closure, and `with` runs the closure while it holds the lock:

```
pub fn with<R>(&self, key: &K, inspect: impl FnOnce(Option<&V>) -> R) -> R
```

- `inspect: impl FnOnce(Option<&V>) -> R` accepts any closure that takes an `Option<&V>` and returns some type `R`. `FnOnce` means `with` calls it at most once.
- The closure receives a reference into the map, valid while the lock is held.
- Only the closure's result, of type `R`, leaves the function. `R` cannot borrow from the map.

The test `with_exposes_the_value_without_cloning` uses it to read the length of a `String`:

```
let observed = cache.with(&"node", |value| value.map(|v| v.len()));
```


The doc comment of `with` gives two rules. Keep the closure short, because every other thread that needs the same shard waits while it runs. And do not call the cache from inside the closure. A call that lands on the same shard would try to lock a `Mutex` that this thread already holds. The standard `Mutex` does not allow that: the thread blocks forever or panics. A thread waiting forever for a lock is called a **deadlock**.

14.1.5 Operations on every shard

Listing 14.4 `len` and `clear` (lines 94 to 109).

```
pub fn len(&self) -> usize {
    self.shards
        .iter()
        .map(|shard| shard.lock().expect("shard mutex poisoned").len())
        .sum()
}

pub fn is_empty(&self) -> bool {
    self.len() == 0
}

pub fn clear(&self) {
    for shard in &self.shards {
        shard.lock().expect("shard mutex poisoned").clear();
    }
}
```

`len` locks each shard in turn, reads its size, releases the lock, and adds up the sizes. `clear` also visits the shards one at a time.

This is the price of sharding, and the comment at the top of the file states it. No moment exists when all shards are locked together. While `len` runs, other threads can insert into shards it has already counted. The total is correct if no other thread is writing, and approximate if one is.

14.1.6 The complete file

Listing 14.5 The complete file, with tests. `src/problems/sharded_cache.rs`

```
/// Sharded concurrent map to reduce lock contention.
///
/// A single `Mutex<HashMap>` serializes every access. Splitting the map into N
/// shards, each with its own lock, lets independent keys proceed in parallel. The
/// cost is that `len` and `clear` must visit every shard, and there is no global
/// atomic snapshot.
///
/// Typical uses: per-account token caches, metadata caches, and request
```

```

//! de-duplication tables.

use std::collections::HashMap;
use std::collections::hash_map::RandomState;
use std::hash::{BuildHasher, Hash, Hasher};
use std::sync::Mutex;

/// A fixed-size map split across independently locked shards.
///
/// The shard count is rounded up to a power of two so the index is a mask instead
/// of a modulo.
pub struct ShardedCache<K, V> {
    shards: Vec<Mutex<HashMap<K, V>>>,
    hasher: RandomState,
    mask: usize,
}

impl<K, V> ShardedCache<K, V>
where
    K: Hash + Eq,
{
    /// Create a cache with at least `shard_count` shards (rounded to a power of two).
    pub fn new(shard_count: usize) -> Self {
        let shard_count = shard_count.max(1).next_power_of_two();

        let shards = (0..shard_count)
            .map(|_| Mutex::new(HashMap::new()))
            .collect();

        Self {
            shards,
            hasher: RandomState::new(),
            mask: shard_count - 1,
        }
    }

    fn shard(&self, key: &K) -> &Mutex<HashMap<K, V>> {
        let mut hasher = self.hasher.build_hasher();
        key.hash(&mut hasher);
        let index = (hasher.finish() as usize) & self.mask;
        &self.shards[index]
    }

    /// Insert a value, returning the previous one if the key was present.
    pub fn insert(&self, key: K, value: V) -> Option<V> {
        self.shard(&key)
            .lock()
            .expect("shard mutex poisoned")
            .insert(key, value)
    }

    /// Read a value by cloning it out of the shard.
    pub fn get(&self, key: &K) -> Option<V>
    where
        V: Clone,
    {
        self.shard(key)
            .lock()
            .expect("shard mutex poisoned")
            .get(key)
    }
}

```

```

        .cloned()
    }

    /// Run `inspect` against the value under the shard lock, without cloning.
    ///
    /// Use this for read-modify-write on a single key; the closure holds the lock
    /// for its duration, so keep it short and do not call back into the cache.
    pub fn with<R>(&self, key: &K, inspect: impl FnOnce(Option<&V>) -> R) -> R {
        let guard = self.shard(key).lock().expect("shard mutex poisoned");
        inspect(guard.get(key))
    }

    pub fn remove(&self, key: &K) -> Option<V> {
        self.shard(key)
            .lock()
            .expect("shard mutex poisoned")
            .remove(key)
    }

    pub fn contains_key(&self, key: &K) -> bool {
        self.shard(key)
            .lock()
            .expect("shard mutex poisoned")
            .contains_key(key)
    }

    pub fn len(&self) -> usize {
        self.shards
            .iter()
            .map(|shard| shard.lock().expect("shard mutex poisoned").len())
            .sum()
    }

    pub fn is_empty(&self) -> bool {
        self.len() == 0
    }

    pub fn clear(&self) {
        for shard in &self.shards {
            shard.lock().expect("shard mutex poisoned").clear();
        }
    }

    pub fn shard_count(&self) -> usize {
        self.shards.len()
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::Arc;
    use std::thread;

    #[test]
    fn insert_get_remove() {
        let cache: ShardedCache<String, u32> = ShardedCache::new(4);
        assert_eq!(cache.insert("server-1".into(), 8), None);
        assert_eq!(cache.get(&"server-1".into()), Some(8));
    }
}

```

```

    assert_eq!(cache.insert("server-1".into(), 16), Some(8));
    assert_eq!(cache.get(&"server-1".into()), Some(16));
    assert_eq!(cache.remove(&"server-1".into()), Some(16));
    assert!(!cache.contains_key(&"server-1".into()));
    assert!(cache.is_empty());
}

#[test]
fn with_exposes_the_value_without_cloning() {
    let cache: ShardedCache<str, String> = ShardedCache::new(2);
    cache.insert("node", "ABCD".to_string());
    let observed = cache.with(&"node", |value| value.map(|v| v.len()));
    assert_eq!(observed, Some(4));
}

#[test]
fn concurrent_writers_all_land() {
    let cache = Arc::new(ShardedCache::new(16));
    let mut handles = Vec::new();

    for writer in 0..8 {
        let cache = Arc::clone(&cache);
        handles.push(thread::spawn(move || {
            for key in 0..500 {
                cache.insert(format!("{writer}-{key}"), key);
            }
        }));
    }

    for handle in handles {
        handle.join().expect("writer panicked");
    }

    assert_eq!(cache.len(), 8 * 500);
}

#[test]
fn keys_spread_across_shards() {
    let cache: ShardedCache<u32, u32> = ShardedCache::new(8);
    for key in 0..4_096 {
        cache.insert(key, key);
    }
    // Every shard should hold roughly 1/8 of the keys; assert none is starved.
    let smallest = cache
        .shards
        .iter()
        .map(|shard| shard.lock().expect("shard mutex poisoned").len())
        .min()
        .expect("at least one shard");
    assert!(smallest > 0, "at least one shard was empty");
}
}

```

The test `concurrent_writers_all_land` is the one that uses threads. It wraps the cache in an `Arc`, and starts eight threads with `thread::spawn`. Each thread gets its own clone of the `Arc` and inserts 500 keys. `join()` waits for each thread to finish. The final `len` must be $8 \times 500 = 4,000$, which shows that no insert was lost.

The test `keys_spread_across_shards` inserts 4,096 keys into 8 shards and checks that no shard is empty. It reaches the private `shards` field directly. A test module inside the file can see private items of its parent module.

```
$ cargo test --lib problems::sharded_cache
running 4 tests
test problems::sharded_cache::tests::with_exposes_the_value_without_cloning ... ok
test problems::sharded_cache::tests::insert_get_remove ... ok
test problems::sharded_cache::tests::concurrent_writers_all_land ... ok
test problems::sharded_cache::tests::keys_spread_across_shards ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 133 filtered out
```

14.2 Spreading keys across machines

Now move from threads to machines. Suppose a cache is spread over several servers, and each key must live on exactly one of them. Every client must compute the same server for the same key, with no central directory to ask.

14.2.1 The problem with `hash % N`

The obvious rule is `hash(key) % N`, where `N` is the number of servers. It spreads keys evenly and needs no storage. It breaks down when `N` changes.

Figure 14.3 shows keys with hashes 0 to 11 when a fourth server joins three others. A key stays in place only if `hash % 3` equals `hash % 4`.

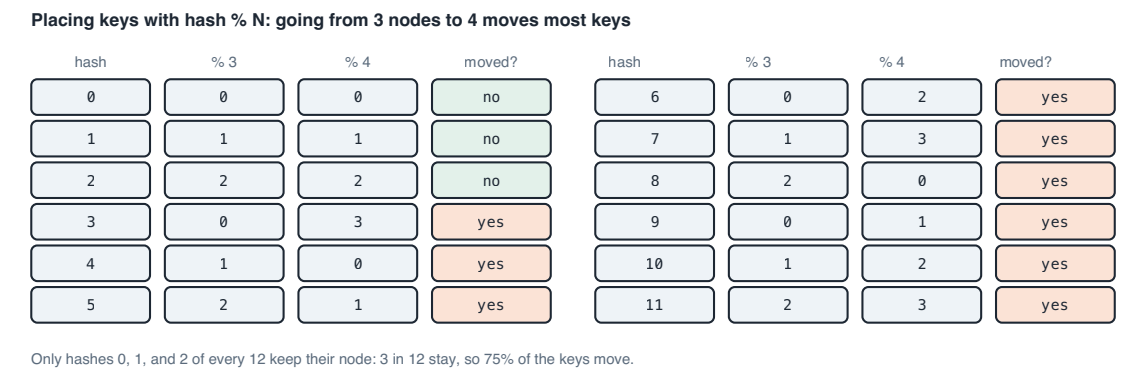


Figure 14.3 From 3 servers to 4 with `hash % N`. Three keys in twelve stay, and the rest move.

The pattern repeats every 12 hashes, so 75% of all keys change servers. For a cache, a moved key is a miss: the new server does not have it. Adding one server would empty most of the cache at once.

14.2.2 The ring

Consistent hashing places both servers and keys on a circle of hash values, called the **ring**. The ring covers every `u64` value. After the largest value, it wraps around to 0.

Each server is hashed to a point on the ring. A key is hashed too, and it belongs to the first server point clockwise from it. In practice, "clockwise" means "the next larger hash".

When a server joins, it takes over only the keys between its point and the point before it. Every other key keeps its server. When a server leaves, only its own keys move, to the next point clockwise.

With one point per server, the arcs between points are uneven, so some servers get much more of the ring than others. The fix is to place each server at many points, called **virtual nodes**. With many small arcs per server, the shares even out (figure 14.4).

A hash ring with 3 nodes and 4 virtual nodes each

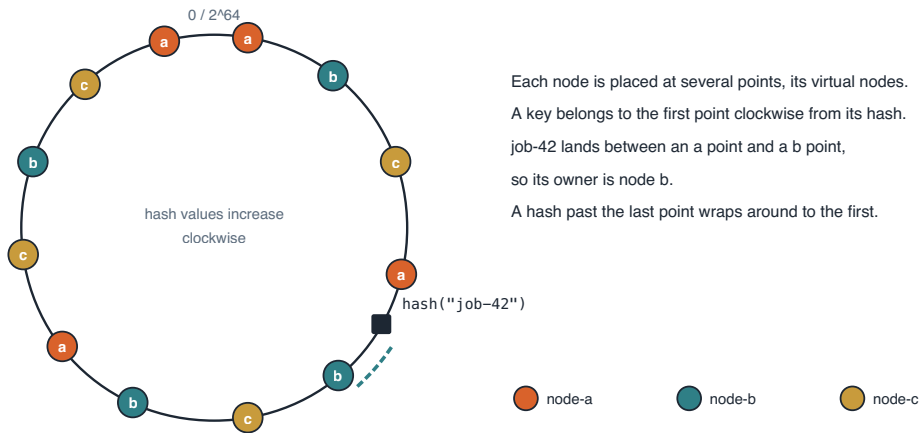
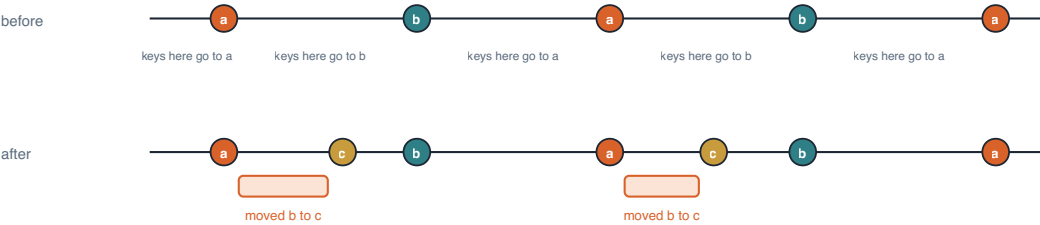


Figure 14.4 Three nodes with four virtual nodes each. The key belongs to the next point clockwise, which belongs to node b.

Figure 14.5 shows the ring cut open and laid flat, before and after a third node joins.

Adding node c: only keys in the arcs that c takes over change owner



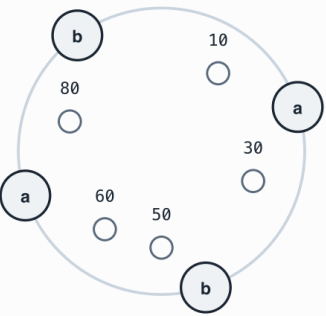
Every other key keeps its owner. With n nodes, adding one moves about $1/(n+1)$ of the keys.

Figure 14.5 Adding node c. The keys that move all move to c. None moves between a and b.

Consistent hashing: a new node takes only its own arc of keys

Nodes and keys share one ring. A key belongs to the first node clockwise from it.

☐ a node ☐ the new node



Four virtual nodes sit on the ring. Each key belongs to the first node clockwise from it.

Moving one key means one cache miss, not a rebuild of the whole table.

nodes = 4 moved = 0 keys = 5

Animation 14.1 The ring cut open and laid flat. Each key belongs to the first node to its right. Adding c takes over only the arc that reaches back to the previous node, so the keys that move are the ones that now find c first. With a thousand nodes and a thousand keys each, $\text{hash } \% N$ moves almost every key when N changes. This moves about $1/(n+1)$ of them.

With n servers, a new server takes over about $1/(n + 1)$ of the keys. Going from 3 servers to 4 moves about 25% of the keys, where `hash % N` moved 75%.

14.2.3 The ring as a sorted Vec

Listing 14.6 The type and `add_node` (lines 17 to 55). `src/problems/consistent_hash.rs`

```
/// A hash ring mapping keys to node names.
pub struct ConsistentHash {
    replicas: usize,
    /// Sorted `(hash, node)` pairs. The node name is an `Arc<str>` so that the
    /// many virtual nodes for one physical node share a single allocation.
    ring: Vec<(u64, Arc<str>)>,
}

/// Create an empty ring with `replicas` virtual nodes per physical node.
/// Panics if `replicas` is zero.
pub fn new(replicas: usize) -> Self {
    assert!(replicas > 0, "replicas must be > 0");
    Self {
        replicas,
        ring: Vec::new(),
    }
}

/// Add a node and its virtual nodes. Adding a node that already exists is a
/// no-op.
pub fn add_node(&mut self, node: &str) {
    if self.ring.iter().any(|(_, name)| name.as_ref() == node) {
        return;
    }

    // One allocation for the name, then a refcount bump per virtual node.
    let name: Arc<str> = Arc::from(node);
    for replica in 0..self.replicas {
        let hash = virtual_node_hash(node, replica);
        self.ring.push((hash, Arc::clone(&name)));
    }
    self.ring.sort_by_key(|(hash, _)| *hash);
}

/// Remove a node and all of its virtual nodes.
pub fn remove_node(&mut self, node: &str) {
    self.ring.retain(|(_, name)| name.as_ref() != node);
}
```

The ring is a `Vec<(u64, Arc<str>)>`, kept sorted by hash. Each entry is one virtual node: its position on the ring, and the name of the server it belongs to.

With 128 virtual nodes per server, each name would be stored 128 times as a `String`. The code stores it as `Arc<str>` instead. `Arc::from(node)` copies the name into one heap block

once. Each `Arc::clone(&name)` then adds a pointer to that same block and increases its reference count. No text is copied.

`add_node` does three things:

1. It checks whether the server is already on the ring, by scanning the entries. If so, it returns.
2. It pushes one entry per virtual node, with the hash from `virtual_node_hash`.
3. It sorts the ring by hash again.

`remove_node` keeps every entry that belongs to another server, with `retain`. Removing entries does not change the order of the rest, so the ring stays sorted.

Listing 14.7 The hash functions (lines 85 to 100).

```
fn hash_of(input: &str) -> u64 {
    let mut hasher = DefaultHasher::new();
    input.hash(&mut hasher);
    hasher.finish()
}

/// Hash of the `replica`-th virtual node for `node`.
///
/// Hashing the two fields separately avoids allocating a "{node}#{replica}"
/// string for every virtual node on the ring.
fn virtual_node_hash(node: &str, replica: usize) -> u64 {
    let mut hasher = DefaultHasher::new();
    node.hash(&mut hasher);
    replica.hash(&mut hasher);
    hasher.finish()
}
```

`virtual_node_hash` feeds both the server name and the replica number into one hasher. Each replica number gives a different point for the same server. Feeding the two values separately avoids building a string such as `"node-a#17"` for every virtual node.

WARNING: `DefaultHasher::new()` gives the same result every time within one program, which the ring needs. The standard library does not promise that it stays the same across Rust versions. Servers and clients built with different compilers could then disagree about who owns a key. A real deployment should use a hash function with a fixed, documented algorithm.

14.2.4 Finding a key's server

Listing 14.8 `get` (lines 57 to 68).

```
/// Return the node responsible for `key`, or `None` when the ring is empty.
pub fn get(&self, key: &str) -> Option<&str> {
    if self.ring.is_empty() {
        return None;
    }

    let hash = hash_of(key);
    // First virtual node strictly clockwise from the key.
    let index = self.ring.partition_point(|(point, _)| *point <= hash);
    let index = if index == self.ring.len() { 0 } else { index };
    Some(self.ring[index].1.as_ref())
}
```

`get` must find the first ring entry with a hash larger than the key's hash. The ring is sorted, so a binary search can find it in $O(\log n)$ steps, as in section 3.10.

`partition_point` performs that binary search. It takes a condition that is true for a first part of the slice and false for the rest. It returns the index where the condition turns false (figure 14.6). Here the condition is `point <= hash`, so the result is the index of the first point larger than the key's hash.

Finding the owner: `partition_point` on the sorted `Vec`



key hash 50: `point <= 50` is true for indexes 0 to 2 and false from index 3, so `partition_point` returns 3.

The owner is `ring[3]`, node a. A hash above 90 would return 6, the length, which wraps to index 0.

The predicate must be true for a prefix and false after it. A sorted ring guarantees that.

Figure 14.6 `partition_point` returns the first index where the condition is false.

If the key's hash is larger than every point, `partition_point` returns the length of the ring. The ring wraps around there, so the code uses index 0 instead.

`get` returns `Option<&str>`, a reference into the ring. `.as_ref()` turns the `Arc<str>` into a `&str`. An empty ring has no servers, so the function returns `None` for it.

14.2.5 The complete file

Listing 14.9 The complete file, with tests. src/problems/consistent_hash.rs

```

///! Consistent hashing ring.
///!
///! When a distributed gateway shards work across N backends, a plain
///! `hash(key) % N` remaps nearly every key the moment N changes. Consistent
///! hashing places each node at several points on a hash ring, so adding or
///! removing one node only moves the keys that node owns. That property is the
///! reason it shows up in caches, sharded queues, and load balancers.
///!
///! Each physical node is placed at `replicas` points (virtual nodes) to smooth the
///! key distribution; more replicas means a less lumpy ring.

use std::collections::HashSet;
use std::collections::hash_map::DefaultHasher;
use std::hash::{Hash, Hasher};
use std::sync::Arc;

/// A hash ring mapping keys to node names.
pub struct ConsistentHash {
    replicas: usize,
    /// Sorted `(hash, node)` pairs. The node name is an `Arc<str>` so that the
    /// many virtual nodes for one physical node share a single allocation.
    ring: Vec<(u64, Arc<str>)>,
}

impl ConsistentHash {
    /// Create an empty ring with `replicas` virtual nodes per physical node.
    /// Panics if `replicas` is zero.
    pub fn new(replicas: usize) -> Self {
        assert!(replicas > 0, "replicas must be > 0");
        Self {
            replicas,
            ring: Vec::new(),
        }
    }

    /// Add a node and its virtual nodes. Adding a node that already exists is a
    /// no-op.
    pub fn add_node(&mut self, node: &str) {
        if self.ring.iter().any(|(_, name)| name.as_ref() == node) {
            return;
        }

        // One allocation for the name, then a refcount bump per virtual node.
        let name: Arc<str> = Arc::from(node);
        for replica in 0..self.replicas {
            let hash = virtual_node_hash(node, replica);
            self.ring.push((hash, Arc::clone(&name)));
        }
        self.ring.sort_by_key(|(hash, _)| *hash);
    }

    /// Remove a node and all of its virtual nodes.
    pub fn remove_node(&mut self, node: &str) {

```

```

        self.ring.retain(|(_, name)| name.as_ref() != node);
    }

    /// Return the node responsible for `key`, or `None` when the ring is empty.
    pub fn get(&self, key: &str) -> Option<&str> {
        if self.ring.is_empty() {
            return None;
        }

        let hash = hash_of(key);
        // First virtual node strictly clockwise from the key.
        let index = self.ring.partition_point(|(point, _)| *point <= hash);
        let index = if index == self.ring.len() { 0 } else { index };
        Some(self.ring[index].1.as_ref())
    }

    /// Number of distinct physical nodes.
    pub fn node_count(&self) -> usize {
        self.ring
            .iter()
            .map(|(_, name)| name.as_ref())
            .collect:::<HashSet<_>>()
            .len()
    }

    /// Number of virtual nodes on the ring.
    pub fn ring_len(&self) -> usize {
        self.ring.len()
    }
}

fn hash_of(input: &str) -> u64 {
    let mut hasher = DefaultHasher::new();
    input.hash(&mut hasher);
    hasher.finish()
}

/// Hash of the `replica`-th virtual node for `node`.
///
/// Hashing the two fields separately avoids allocating a `"{node}#{replica}"`
/// string for every virtual node on the ring.
fn virtual_node_hash(node: &str, replica: usize) -> u64 {
    let mut hasher = DefaultHasher::new();
    node.hash(&mut hasher);
    replica.hash(&mut hasher);
    hasher.finish()
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn empty_ring_has_no_owner() {
        let ring = ConsistentHash::new(64);
        assert_eq!(ring.get("workload-1"), None);
        assert_eq!(ring.node_count(), 0);
    }
}

```

```

#[test]
fn lookup_is_stable_for_a_fixed_ring() {
    let mut ring = ConsistentHash::new(64);
    ring.add_node("node-a");
    ring.add_node("node-b");

    let first = ring.get("job-42").map(str::to_string);
    for _ in 0..100 {
        assert_eq!(ring.get("job-42").map(str::to_string), first);
    }
    assert!(first.is_some());
}

#[test]
fn keys_spread_across_nodes() {
    let mut ring = ConsistentHash::new(128);
    for node in ["node-a", "node-b", "node-c"] {
        ring.add_node(node);
    }

    let mut counts = std::collections::HashMap::new();
    for key in 0..3_000 {
        let owner = ring.get(&format!("key-{}", key)).expect("ring is populated");
        *counts.entry(owner.to_string()).or_insert(0) += 1;
    }

    assert_eq!(counts.len(), 3, "every node should own some keys");
    let smallest = *counts.values().min().expect("non-empty");
    assert!(smallest > 300, "distribution is too lumpy: {counts:?}");
}

#[test]
fn adding_a_node_only_moves_keys_to_the_new_node() {
    let keys: Vec<String> = (0..2_000).map(|i| format!("workload-{}", i)).collect();

    let mut ring = ConsistentHash::new(128);
    ring.add_node("node-a");
    ring.add_node("node-b");
    let before: Vec<String> = keys
        .iter()
        .map(|key| ring.get(key).expect("ring is populated").to_string())
        .collect();

    ring.add_node("node-c");
    let after: Vec<String> = keys
        .iter()
        .map(|key| ring.get(key).expect("ring is populated").to_string())
        .collect();

    for (old, new) in before.iter().zip(after.iter()) {
        assert!(
            old == new || new == "node-c",
            "adding a node moved a key from {old} to {new}"
        );
    }
}

#[test]
fn removing_a_node_drops_it_from_the_ring() {

```

```

    let mut ring = ConsistentHash::new(32);
    ring.add_node("node-a");
    ring.add_node("node-b");
    assert_eq!(ring.node_count(), 2);
    assert_eq!(ring.ring_len(), 64);

    ring.remove_node("node-a");
    assert_eq!(ring.node_count(), 1);
    for _ in 0..500 {
        assert!(matches!(ring.get("any-key"), Some("node-b")));
    }
}

```

`node_count` counts distinct servers by collecting the names into a `HashSet`, which keeps one copy of each.

The tests check the properties this section described:

- `keys_spread_across_nodes` places 3,000 keys on three servers with 128 virtual nodes each. Each server must own more than 300 keys. An even split would give each one 1,000.
- `adding_a_node_only_moves_keys_to_the_new_node` records the owner of 2,000 keys, adds a third server, and looks again. Every key must either keep its owner or move to the new server.
- `removing_a_node_drops_it_from_the_ring` checks that 2 servers with 32 virtual nodes give 64 ring entries, and that removing one leaves every key with the other.

```

$ cargo test --lib problems::consistent_hash
running 5 tests
test problems::consistent_hash::tests::empty_ring_has_no_owner ... ok
test problems::consistent_hash::tests::lookup_is_stable_for_a_fixed_ring ... ok
test problems::consistent_hash::tests::removing_a_node_drops_it_from_the_ring ... ok
test problems::consistent_hash::tests::adding_a_node_only_moves_keys_to_the_new_node ...
ok
test problems::consistent_hash::tests::keys_spread_across_nodes ... ok

test result: ok. 5 passed; 0 failed; 0 ignored; 0 measured; 132 filtered out

```

Summary

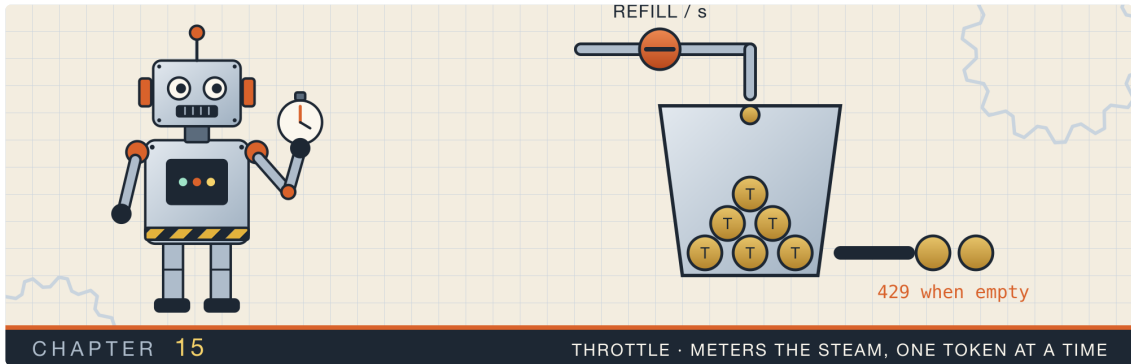
- A single `Mutex` around a map makes every thread wait for every other. Sharding gives each group of keys its own lock, so threads wait only when their keys share a shard.
- With a power-of-two shard count, `hash & (count - 1)` picks the shard with one AND instruction.

- A value inside a `Mutex` can leave the lock only as a clone. A closure passed to `with` can read the value in place while the lock is held.
- Operations over all shards, such as `len`, lock one shard at a time and are not a single snapshot.
- `hash % N` moves most keys when `N` changes. Consistent hashing moves only the keys the new server takes over, about $1/(n + 1)$ of them.
- Virtual nodes even out each server's share of the ring. `Arc<str>` lets all of a server's entries share one copy of its name.
- The ring is a sorted `Vec`, and `partition_point` finds a key's server by binary search.

Chapter 15 looks below the data structures, at how the CPU caches memory and how the operating system hands it out.

Exercises

1. Add `update(&self, key: &K, change: impl FnOnce(&mut V))` to `ShardedCache`, which changes a value in place under the lock.
2. Write a test that counts how many keys change owner when a fourth server joins a ring of three. Compare the result with 25%.
3. Measure how even the ring is. For 10, 50, and 200 virtual nodes per server, place 100,000 keys and report the largest and smallest share.
4. `add_node` sorts the whole ring after every call. Rewrite it to insert each new entry at its sorted position with `partition_point` and `Vec::insert`, and compare the two for 100 servers.



Memory, CPU caches, and the operating system

THIS CHAPTER COVERS

- How long it takes the CPU to read memory, measured at five data sizes
- A bump allocator, and what alignment means
- Why the order in which you read an array changes the time, and a benchmark that measured something else
- False sharing: two threads that slow each other down without sharing any data
- Pages, page tables, and the kinds of page fault
- Round-robin scheduling, and what a system call costs

The earlier chapters counted steps: $O(1)$, $O(\log n)$, $O(n)$. Step counts predict how a program grows, but not how long one step takes. On a real machine, one memory read can take 2 ns or 150 ns, depending on where the data is. Chapter 13 already saw a factor of three between two caches with the same step count.

This chapter looks at the machine under the data structures. Section 15.1 measures the memory system. Sections 15.2 to 15.4 show three programs whose speed depends on how they lay out and touch memory. Sections 15.5 to 15.7 look at three services of the operating system: virtual memory, scheduling, and system calls.

Most programs in this chapter print times. Run them with `--release`, and expect your numbers to differ from mine. All the numbers here come from one laptop with an Intel Core i5-1038NG7 CPU, running macOS.

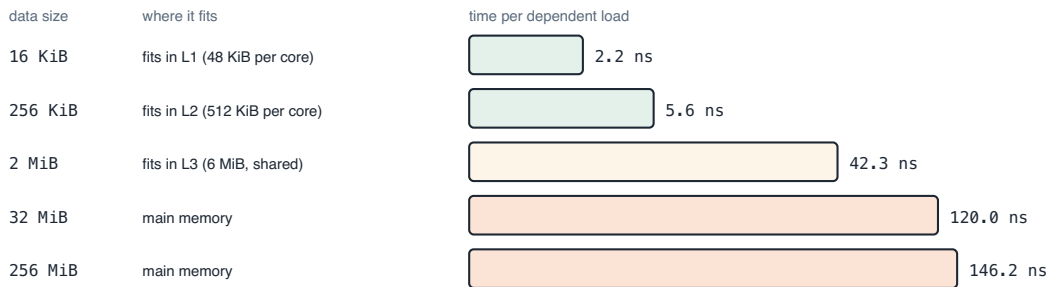
15.1 How far away is memory?

Main memory, the RAM, is large and slow compared with the CPU. To hide the difference, the CPU keeps copies of recently used memory in small, fast memories called **caches**. There are usually three levels:

- **L1** is the smallest and fastest, and each core has its own. On this CPU it holds 48 KiB of data per core.
- **L2** is larger and slower, also per core: 512 KiB here.
- **L3** is shared by all cores: 6 MiB here.

Caches do not copy single bytes. They copy blocks of 64 bytes, called **cache lines**. Reading one `u64` brings in the line around it, so the next seven `u64` values in memory arrive at no extra cost.

To see the cost of each level, I measured a **pointer chase**. The program fills an array with the indexes of a random cycle, so each element holds the index of the next one to read. The loop then runs `i = next[i]` many times. Each read needs the result of the previous one, so the reads cannot overlap. The time per step is the time of one read. Figure 15.1 shows the result for five array sizes.

Memory latency on an Intel Core i5-1038NG7, measured with a dependent pointer chase

Each load's address comes from the previous load, so the loads cannot overlap. The bar length is logarithmic. The larger sizes also pay for address translation misses (section 15.5). A cache line is 64 bytes on this CPU.

Figure 15.1 Time per dependent read, by data size. Data that fits in L1 is about 65 times faster to reach than data in main memory.

A read from L1 took 2.2 ns. A read that had to go to main memory took 120 to 146 ns. In 120 ns, this CPU can run several hundred simple instructions. The larger sizes also include the cost of translating addresses, which section 15.5 explains.

Those five numbers describe one hierarchy, not five separate machines. Data in L1 answers a dependent read in 2.2 ns. The same read against 256 KiB costs 5.6 ns. That array no longer fits in a 48 KiB L1, so the line comes from L2. At 2 MiB the cost is 42.3 ns, having left L2 as well. From 32 MiB upward it is 120 ns or more, which is main memory. A larger level holds several times more than the one above it, and answers several times more slowly. The trade is forced. A store that answers in 2 ns has to be small, because reaching a location costs more as the store grows.

A cache line is 64 bytes, and it arrives as one unit. A program that reads one `u64` and then the next seven pays for a single line and uses all of it. A program that jumps to a new random line on every read pays the same 120 ns. It uses one eighth of what it fetched.

The rest of the chapter comes back to one fact. Data that is close together in memory, and read in order, is much cheaper than data scattered around. Keeping related data close together is called **locality**.

15.2 A bump allocator

Programs get heap memory from an **allocator**. Every `Box::new` and every growing `Vec` asks the allocator for a block, and every `drop` gives one back. A general-purpose allocator must handle blocks of any size, freed in any order, from any thread. That takes bookkeeping.

A **bump allocator** is the simplest allocator. It owns one large buffer and an offset, the position of the first free byte. To allocate, it hands out the bytes at the offset and moves the offset forward. It never frees a single block. Instead, `reset` frees everything at once by setting the offset back to 0.

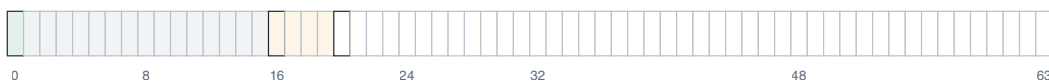
This fits work that allocates many small pieces and throws them all away together. A parser can allocate its syntax tree in an arena and drop it after compiling. A server can allocate per request and reset after sending the response.

15.2.1 Alignment

Most types must start at an address that is a multiple of some number, called their **alignment**. A `u64` has alignment 8, so it must start at an address divisible by 8. The CPU reads aligned values in one step, and some instructions refuse unaligned ones.

So an allocator must sometimes skip bytes before a block, to reach an address with the right alignment. The skipped bytes are called **padding**. Figure 15.2 shows two allocations and the padding between them.

A 64-byte arena after `alloc(1, 8)` and `alloc(4, 16)`



byte 0: the first block (1 byte, aligned to 8)

bytes 1 to 15: padding, so the next block starts at a multiple of 16

bytes 16 to 19: the second block (4 bytes)

`reset()` sets the offset back to 0 and frees every block at once. No single block can be freed.

offset after both calls: 20

`used() = 20`, `remaining() = 44`

Figure 15.2 The state checked by the test `allocations_are_aligned`. Aligning the second block to 16 costs 15 bytes of padding.

15.2.2 The code

Listing 15.1 `alloc` (lines 27 to 42) and `align_up` (lines 65 to 68). `src/problems/bump_allocator.rs`

```

/// Allocate `size` bytes aligned to `align`.
///
/// Returns `None` when the request does not fit. `align` must be a power of
/// two.
pub fn alloc(&mut self, size: usize, align: usize) -> Option<&mut [u8]> {
    assert!(align.is_power_of_two(), "align must be a power of two");

    let start = align_up(self.offset, align)?;
    let end = start.checked_add(size)?;
    if end > self.buffer.len() {
        return None;
    }

    self.offset = end;
    Some(&mut self.buffer[start..end])
}

fn align_up(offset: usize, align: usize) -> Option<usize> {
    let mask = align - 1;
    offset.checked_add(mask).map(|value| value & !mask)
}

```

`align_up` rounds the offset up to the next multiple of `align`. The alignment must be a power of two, which the `assert!` in `alloc` checks. A power of two has exactly one bit set, so `align - 1` is a mask with all the bits below it set. For `align = 16`, the mask is `0b1111`.

The rounding happens in two steps. Adding the mask pushes the offset past the next multiple, unless it is already on one. Then `& !mask` clears the low bits, which rounds down to a multiple. Work through the second allocation of figure 15.2 by hand:

1. The offset is 1, and the alignment is 16, so the mask is 15.
2. $1 + 15 = 16$.
3. 16 with the low four bits cleared is 16. The block starts at 16.

An offset of 16 would give $16 + 15 = 31$, which rounds down to 16 again. An offset already aligned stays where it is.

`alloc` uses `checked_add` twice, and the `?` operator returns `None` if either addition overflows. It also returns `None` if the block would end past the buffer. Only after every check passes does it move the offset. So a failed allocation leaves the arena unchanged, which the test `exhaustion_returns_none_and_keeps_state` checks.

The block is returned as `&mut [u8]`, a slice of the buffer. That reference borrows the arena mutably, so you cannot call `alloc` again while you hold it. A production arena allocator hands out many live references at once, and needs `unsafe` code or interior mutability to do so.

WARNING: `align_up` aligns the offset, not the memory address. The address of a block is the buffer's address plus the offset. So the block is aligned only if the buffer itself starts at a multiple of `align`. The buffer is a `Vec<u8>`, which only promises alignment 1. The tests pass because the macOS allocator returns 16-byte aligned blocks. A request for alignment 64 could return a misaligned address.

Listing 15.2 The complete file, with tests. `src/problems/bump_allocator.rs`

```

///! A bump (arena) allocator over a fixed byte buffer.
///!
///! Bump allocation is the simplest allocator: hand out bytes from a cursor and
///! never free an individual block. It is O(1) per allocation, has no fragmentation
///! bookkeeping, and is freed all at once with [BumpArena::reset]. Parsers,
///! request handlers, and per-frame buffers in games use exactly this shape.
///!
///! This version does not try to beat the system allocator. It shows alignment,
///! the memory layout you hand to hardware, and why "free everything at once" is
///! both the strength and the limitation.

///! A fixed-capacity bump allocator.
pub struct BumpArena {
    buffer: Vec<u8>,
    offset: usize,
}

impl BumpArena {
    /// Create an arena backed by `capacity` zeroed bytes.
    pub fn with_capacity(capacity: usize) -> Self {
        Self {
            buffer: vec![0; capacity],
            offset: 0,
        }
    }

    /// Allocate `size` bytes aligned to `align`.
    ///
    /// Returns `None` when the request does not fit. `align` must be a power of
    /// two.
    pub fn alloc(&mut self, size: usize, align: usize) -> Option<&mut [u8]> {
        assert!(align.is_power_of_two(), "align must be a power of two");

        let start = align_up(self.offset, align)?;
        let end = start.checked_add(size)?;
        if end > self.buffer.len() {
            return None;
        }

        self.offset = end;
    }
}

```

```

        Some(&mut self.buffer[start..end])
    }

    /// Bytes handed out so far, including alignment padding.
    pub fn used(&self) -> usize {
        self.offset
    }

    /// Bytes still available before the next allocation.
    pub fn remaining(&self) -> usize {
        self.buffer.len() - self.offset
    }

    /// Total capacity.
    pub fn capacity(&self) -> usize {
        self.buffer.len()
    }

    /// Free everything at once, as an arena does.
    pub fn reset(&mut self) {
        self.offset = 0;
    }
}

fn align_up(offset: usize, align: usize) -> Option<usize> {
    let mask = align - 1;
    offset.checked_add(mask).map(|value| value & !mask)
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn allocations_are_aligned() {
        let mut arena = BumpArena::with_capacity(64);

        let first = arena.alloc(1, 8).expect("fits");
        assert_eq!(first.as_ptr() as usize % 8, 0);
        assert_eq!(first.len(), 1);

        let second = arena.alloc(4, 16).expect("fits");
        assert_eq!(second.as_ptr() as usize % 16, 0);

        // The first block occupies 1 byte. Aligning the next block to 16 adds 15
        // bytes of padding, so the arena has consumed 16 + 4 = 20 bytes.
        assert_eq!(arena.used(), 20);
        assert_eq!(arena.remaining(), 44);
    }

    #[test]
    fn exhaustion_returns_none_and_keeps_state() {
        let mut arena = BumpArena::with_capacity(16);
        assert!(arena.alloc(16, 1).is_some());

        let used_before = arena.used();
        assert!(arena.alloc(1, 1).is_none());
        assert_eq!(arena.used(), used_before, "a failed alloc must not advance");
    }
}

```

```

#[test]
fn reset_reclaims_everything() {
    let mut arena = BumpArena::with_capacity(32);
    assert!(arena.alloc(20, 4).is_some());
    assert_eq!(arena.remaining(), 12);

    arena.reset();
    assert_eq!(arena.used(), 0);
    assert_eq!(arena.remaining(), arena.capacity());
    assert!(arena.alloc(32, 1).is_some());
}

#[test]
fn zero_sized_allocation_is_allowed() {
    let mut arena = BumpArena::with_capacity(8);
    let empty = arena.alloc(0, 8).expect("fits");
    assert!(empty.is_empty());
    assert_eq!(arena.used(), 0);
}

#[test]
fn writes_are_isolated_between_allocations() {
    let mut arena = BumpArena::with_capacity(16);
    arena
        .alloc(4, 4)
        .expect("fits")
        .copy_from_slice(&[1, 2, 3, 4]);
    let second = arena.alloc(4, 4).expect("fits");
    assert!(second.iter().all(|&byte| byte == 0));
    second.copy_from_slice(&[9, 9, 9, 9]);

    let view = arena.buffer.as_slice();
    assert_eq!(&view[0..4], &[1, 2, 3, 4]);
    assert_eq!(&view[4..8], &[9, 9, 9, 9]);
}
}

```

```

$ cargo test --lib problems::bump_allocator
running 5 tests
test problems::bump_allocator::tests::allocations_are_aligned ... ok
test problems::bump_allocator::tests::exhaustion_returns_none_and_keeps_state ... ok
test problems::bump_allocator::tests::reset_reclaims_everything ... ok
test problems::bump_allocator::tests::zero_sized_allocation_is_allowed ... ok
test problems::bump_allocator::tests::writes_are_isolated_between_allocations ... ok

test result: ok. 5 passed; 0 failed; 0 ignored; 0 measured; 132 filtered out

```

15.3 Reading in order, and reading scattered

Figure 15.3 shows how the order of reads changes the work. Reading a `u64` array in order uses every value of each cache line it loads. The CPU also notices the pattern and fetches the

next lines before they are needed, which is called **prefetching**. Reading scattered values loads a whole line for each value and uses one eighth of it.

A cache line holds 8 u64 values. The order of access decides how many lines each load brings in

in order: one line serves 8 loads, and the prefetcher fetches the next line early



scattered: each load needs a different line, and 7 of its 8 values go unused



Both patterns read 6 values. The first reads 1 line of memory; the second reads 6.

Figure 15.3 Six reads in order touch one line. Six scattered reads touch six lines.

15.3.1 The program

Listing 15.3 The two loops (lines 9 to 30). src/bin/cs_locality.rs

```

// A power of two, so a stride must be odd to visit every index.
const ELEMENTS: usize = 4 * 1024 * 1024;

// Odd, and therefore coprime with `ELEMENTS`: the walk visits every slot once.
const STRIDE: usize = 1_000_003;

fn sequential_sum(data: &[u64]) -> (u64, Duration) {
    let start = Instant::now();
    let sum = data.iter().sum();
    (sum, start.elapsed())
}

fn strided_sum(data: &[u64]) -> (u64, Duration) {
    let start = Instant::now();
    let mut index = 0usize;
    let mut sum = 0u64;
    for _ in 0..data.len() {
        index = (index + STRIDE) % data.len();
        sum = sum.wrapping_add(data[index]);
    }
    (sum, start.elapsed())
}

```

Both functions add up every element of the same 32 MiB array, which is too large for any cache. `sequential_sum` reads it in order with `data.iter().sum()`. `strided_sum` jumps 1,000,003 elements forward at each step, wrapping around with `%`.

The jump must eventually visit every element exactly once. The array has 4,194,304 elements, which is 2 to the power 22. The stride is odd, so it shares no factor with a power of two. A walk whose step shares no factor with the length visits every position once before it returns to the start.

Listing 15.4 The complete program. `src/bin/cs_locality.rs`

```

///! Cache locality: the same data is much faster to walk in order than to jump
///! around, because memory moves in cache lines and the prefetcher helps only the
///! sequential case.
///!
///! Run with: cargo run --bin cs_locality

use std::time::{Duration, Instant};

/// A power of two, so a stride must be odd to visit every index.
const ELEMENTS: usize = 4 * 1024 * 1024;

/// Odd, and therefore coprime with `ELEMENTS`: the walk visits every slot once.
const STRIDE: usize = 1_000_003;

fn sequential_sum(data: &[u64]) -> (u64, Duration) {
    let start = Instant::now();
    let sum = data.iter().sum();
    (sum, start.elapsed())
}

fn strided_sum(data: &[u64]) -> (u64, Duration) {
    let start = Instant::now();
    let mut index = 0usize;
    let mut sum = 0u64;
    for _ in 0..data.len() {
        index = (index + STRIDE) % data.len();
        sum = sum.wrapping_add(data[index]);
    }
    (sum, start.elapsed())
}

fn main() {
    let data: Vec<u64> = (0..ELEMENTS as u64).collect();
    println!(
        "{} elements, {} MiB",
        data.len(),
        data.len() * 8 / (1024 * 1024)
    );

    let (sequential, sequential_time) = sequential_sum(&data);
    let (strided, strided_time) = strided_sum(&data);

    println!("\n sequential: {sequential} in {sequential_time:?}");
    println!("  strided:    {strided} in {strided_time:?}");

    // Both touch every element exactly once, so the sums must match; only the
    // access pattern differs.
    assert_eq!(sequential, strided);
    assert_eq!(sequential, ELEMENTS as u64 * (ELEMENTS as u64 - 1) / 2);
}

```

```
println!("\nequal work, different pattern; timings vary by machine");
}
```

`main` checks that both sums are equal, and that they match the formula for $0 + 1 + \dots + (n - 1)$. The two loops did the same additions. Only the order of reading differs.

```
$ cargo run --release --bin cs_locality
4194304 elements, 32 MiB

sequential: 8796090925056 in 3.127311ms
strided:    8796090925056 in 37.375274ms

equal work, different pattern; timings vary by machine
```

15.3.2 What the strided loop measured

The strided loop took twelve times as long. It is tempting to credit all of that to the cache, but the two loops differ in more than their reading order.

`strided_sum` computes `(index + STRIDE) % data.len()` at every step. `data.len()` is not known at compile time, so `%` is compiled as a real division instruction, one of the slowest integer instructions. Each step's index depends on the previous one, so the divisions run one after another.

To separate the two costs, I ran four variants of the loop:

Table 15.1 Variants of the 32 MiB sum, release build

Loop	Time
<code>data.iter().sum()</code> , in order	3.1 ms
<code>strided_sum</code> as written, stride 1,000,003	37.4 ms
<code>strided_sum</code> with stride 1, reading in order	36 ms
sum through a list of indexes, in order	4.4 ms
sum through a list of indexes, in random order	31 ms
pointer chase in random order, each read waits for the last	478 ms

The third row settles it. With stride 1, the loop reads memory in perfect order, and it still takes 36 ms. The division sets the pace of `strided_sum`, not the memory.

The last three rows remove the division and show the real cost of scattered reads. Reading through a list of indexes in random order took about seven times as long as reading through the same list in order. The reads are independent of each other, so the CPU keeps several of them in flight at once. In the pointer chase of section 15.1, each read must wait for the previous one. There, the same 4 million reads take 478 ms, about 114 ns each.

Two lessons come out of this. Scattered memory access is expensive, from a few times slower to a hundred times, depending on whether reads can overlap. And a benchmark measures everything in its loop. Before you explain a number, change one thing at a time and check that the number moves.

15.4 False sharing

Chapter 16 covers threads in depth. This section needs only two facts about them:

- `thread::scope(|scope| { ... })` creates a scope. Threads started inside it with `scope.spawn` may borrow local variables, because the scope waits for all of them to finish before it returns.
- An atomic integer such as `AtomicUsize` can be changed by several threads at once without a lock. `fetch_add(1, Ordering::Relaxed)` adds 1 in one indivisible step. The `Ordering` argument is explained in chapter 16. `Relaxed` is enough for a counter that nothing else depends on.

Now the problem. Each core has its own L1 cache. Before a core writes to a cache line, it must have the only copy of that line. Every other core's copy is thrown away. If two cores write to the same line in turn, the line moves back and forth between them on almost every write.

That is expected when two threads write the same variable. **False sharing** is the same slowdown when the two threads write different variables that happen to sit in the same cache line (figure 15.4). The threads share no data, but the hardware moves memory in whole lines, so they share a line.

False sharing: two cores write different counters that sit on the same cache line

Each write needs the line in its own core's cache, in exclusive state.
The line moves back and forth between the cores on almost every increment.



With `#[repr(align(64))]`, each counter has its own line, and each core keeps its line.
The program is the same. Only the placement of the counters in memory differs.

Figure 15.4 Two counters in one line, and two counters in separate lines.

The fix is to place each counter in its own cache line. `#[repr(align(64))]` on a struct makes its alignment 64, so every value of the type starts on a new line. The size rounds up to a multiple of the alignment, so the value fills the line.

15.4.1 Four counters

Listing 15.5 Two counter types (lines 14 to 39). `src/bin/concurrency_false_sharing.rs`

```
/// Packed by default: consecutive counters share a cache line.
struct Unpadded(AtomicUsize);

impl Unpadded {
    fn new() -> Self {
        Self(AtomicUsize::new(0))
    }

    fn increment(&self) {
        self.0.fetch_add(1, Ordering::Relaxed);
    }
}

/// Padded to a cache line, so no two counters share one.
#[repr(align(64))]
struct Padded(AtomicUsize);

impl Padded {
    fn new() -> Self {
        Self(AtomicUsize::new(0))
    }

    fn increment(&self) {
        self.0.fetch_add(1, Ordering::Relaxed);
    }
}
```

```
    }
}
```

`Unpadded` is 8 bytes, so a `Vec` of four of them fits in 32 bytes, inside one or two cache lines. `Padded` is 64 bytes, and each one gets a line.

Listing 15.6 The timed run (lines 41 to 55).

```
/// Scoped threads borrow the vector directly, so there is no `Arc` and no clone.
fn run_unpadded() -> Duration {
    let counters: Vec<Unpadded> = (0..THREADS).map(|_| Unpadded::new()).collect();
    let start = Instant::now();
    thread::scope(|scope| {
        for counter in &counters {
            scope.spawn(move || {
                for _ in 0..ITERATIONS {
                    counter.increment();
                }
            });
        }
    });
    start.elapsed()
}
```

The loop `for counter in &counters` gives each thread a reference to its own counter. `move ||` moves that reference into the thread's closure. Each thread increments only its own counter, two million times.

Listing 15.7 The complete program. `src/bin/concurrency_false_sharing.rs`

```
///! False sharing: two independent counters that land on the same cache line make
///! each other slow, because every write moves the line between cores.
///!
///! Run with: cargo run --bin concurrency_false_sharing

use std::mem::size_of;
use std::sync::atomic::{AtomicUsize, Ordering};
use std::thread;
use std::time::{Duration, Instant};

const THREADS: usize = 4;
const ITERATIONS: usize = 2_000_000;

/// Packed by default: consecutive counters share a cache line.
struct Unpadded(AtomicUsize);

impl Unpadded {
    fn new() -> Self {
        Self(AtomicUsize::new(0))
    }

    fn increment(&self) {
```

```

        self.0.fetch_add(1, Ordering::Relaxed);
    }
}

/// Padded to a cache line, so no two counters share one.
#[repr(align(64))]
struct Padded(AtomicUsize);

impl Padded {
    fn new() -> Self {
        Self(AtomicUsize::new(0))
    }

    fn increment(&self) {
        self.0.fetch_add(1, Ordering::Relaxed);
    }
}

/// Scoped threads borrow the vector directly, so there is no `Arc` and no clone.
fn run_unpadded() -> Duration {
    let counters: Vec<Unpadded> = (0..THREADS).map(|_| Unpadded::new()).collect();
    let start = Instant::now();
    thread::scope(|scope| {
        for counter in &counters {
            scope.spawn(move || {
                for _ in 0..ITERATIONS {
                    counter.increment();
                }
            });
        }
    });
    start.elapsed()
}

fn run_padded() -> Duration {
    let counters: Vec<Padded> = (0..THREADS).map(|_| Padded::new()).collect();
    let start = Instant::now();
    thread::scope(|scope| {
        for counter in &counters {
            scope.spawn(move || {
                for _ in 0..ITERATIONS {
                    counter.increment();
                }
            });
        }
    });
    start.elapsed()
}

fn main() {
    println!("atomic:   {} bytes", size_of::<AtomicUsize>());
    println!(
        "unpadded: {} bytes (several counters per 64 byte line)",
        size_of::<Unpadded>()
    );
    println!(
        "padded:   {} bytes (one counter per 64 byte line)",
        size_of::<Padded>()
    );
}

```

```

let unpadded = run_unpadded();
let padded = run_padded();

println!("\n{THREADS} threads x {ITERATIONS} increments, each on its own counter");
println!("  unpadded (shared lines): {unpadded:?}");
println!("  padded   (own line):      {padded:?}");

assert!(size_of::<Padded>() >= 64, "padding must fill a cache line");
assert!(unpadded.as_nanos() > 0 && padded.as_nanos() > 0);

println!("\ntimings vary by machine; the padded run is normally the faster one");
}

```

```

$ cargo run --release --bin concurrency_false_sharing
atomic: 8 bytes
unpadded: 8 bytes (several counters per 64 byte line)
padded: 64 bytes (one counter per 64 byte line)

4 threads x 2000000 increments, each on its own counter
unpadded (shared lines): 205.466825ms
padded   (own line):      20.667095ms

timings vary by machine; the padded run is normally the faster one

```

Adding 56 bytes of padding to each counter made the same work ten times faster.

15.4.2 Two counters

The second program makes the sharing certain instead of likely. `SameLine` holds both counters and is aligned to 64, so both are guaranteed to be in one line.

Listing 15.8 The complete program. `src/bin/false_sharing.rs`

```

use std::sync::atomic::{AtomicU64, Ordering};
use std::thread;
use std::time::{Duration, Instant};

const ITERS: u64 = 20_000_000;

/// Both counters are guaranteed to live inside one 64-byte cache line.
#[repr(align(64))]
struct SameLine {
    a: AtomicU64,
    b: AtomicU64,
}

/// Each of these occupies its own 64-byte block.
#[repr(align(64))]
struct Padded(AtomicU64);

fn time<F: FnOnce()>(f: F) -> Duration {
    let t = Instant::now();

```

```

    f();
    t.elapsed()
}

fn main() {
    // ---- 1. False sharing: two counters on the same cache line ----
    let same = Sameline { a: AtomicU64::new(0), b: AtomicU64::new(0) };

    let d1 = time(|| {
        thread::scope(|s| {
            s.spawn(|| for _ in 0..ITERS { same.a.fetch_add(1, Ordering::Relaxed); });
            s.spawn(|| for _ in 0..ITERS { same.b.fetch_add(1, Ordering::Relaxed); });
        });
    });

    // ---- 2. Padded: same work, but counters are on separate cache lines ----
    let p0 = Padded(AtomicU64::new(0));
    let p1 = Padded(AtomicU64::new(0));

    let d2 = time(|| {
        thread::scope(|s| {
            s.spawn(|| for _ in 0..ITERS { p0.0.fetch_add(1, Ordering::Relaxed); });
            s.spawn(|| for _ in 0..ITERS { p1.0.fetch_add(1, Ordering::Relaxed); });
        });
    });

    println!("same cache line (false sharing): {:?}", d1);
    println!("padded (separate lines):          {:?}", d2);
}

```

`time` is a small generic helper. It takes any closure that can be called once, `F: FnOnce()`, runs it, and returns how long it took.

The spawned closures have no `move`. They borrow `same`, `p0`, and `p1` directly, which `thread::scope` allows.

```

$ cargo run --release --bin false_sharing
same cache line (false sharing): 838.152232ms
padded (separate lines):          170.243616ms

```

Here the difference is about five times.

NOTE: A cache line is 64 bytes on this Intel CPU. Apple's M-series CPUs use 128-byte lines. Some Intel CPUs also fetch lines in adjacent pairs, so some libraries pad shared counters to 128 bytes to be safe on both.

15.5 Pages and page faults

Every address your program uses is a **virtual address**. The operating system gives each process its own private range of addresses. The CPU translates each virtual address into a **physical address**, a location in the actual RAM.

Translation works on fixed-size blocks called **pages**. On this machine a page is 4,096 bytes. Apple's ARM Macs use 16 KiB pages. The operating system keeps a **page table** for each process, which maps each virtual page to a physical block of RAM, called a **frame**.

An address splits into two parts (figure 15.5). The high part is the page number, which the page table translates. The low part is the offset inside the page, which stays the same. With 4,096-byte pages, the offset is the low 12 bits, because 2 to the power 12 is 4,096.

A virtual address is a page number and an offset. The page table maps pages to physical frames

address 0x1234 with 4096-byte pages (12 offset bits)



page table of one process

page 0	frame 7	present, read-only
page 1	frame 3	present, writable
page 2	on disk	not present
page 3	no entry	unmapped

Accessing a page raises a page fault when:

- the page has no mapping
invalid: the program crashes (SIGSEGV)
- the access breaks the page's permission
protection: usually a crash
- the page is mapped but not in memory
the kernel loads it, then retries
- the kernel must fix the entry first
minor: e.g. copy-on-write, first touch

A major fault reads from disk and costs milliseconds. A minor fault only updates tables and costs microseconds.

Figure 15.5 Splitting an address, and the four cases in which accessing a page causes a fault.

Looking up the page table on every access would be slow. The CPU keeps recent translations in a small cache called the **TLB** (translation lookaside buffer). Scattered reads over a large array need many different pages, miss the TLB, and pay for a table lookup. That cost is part of the larger numbers in figure 15.1.

When the translation cannot complete, the CPU stops the program and hands control to the kernel. This is a **page fault**. The kernel decides what kind it is:

- **Invalid:** the address is not mapped at all. The program is stopped, usually with a segmentation fault.

- **Protection:** the page is mapped, but the access is not allowed, such as a write to a read-only page.
- **Major:** the page is valid, but its contents are on disk. The kernel reads it in and restarts the access. This takes milliseconds.
- **Minor:** the kernel can fix the mapping without reading the disk. Examples are the first write to a freshly allocated page, or a write to a page shared after `fork` (copy-on-write). This takes microseconds.

Listing 15.9 The address arithmetic and the fault model (lines 6 to 35). `src/bin/os_paging.rs`

```
const PAGE_SIZE: u64 = 4096;

fn split_page(address: u64) -> (u64, u64) {
    (address / PAGE_SIZE, address % PAGE_SIZE)
}

fn pages_needed(bytes: u64) -> u64 {
    bytes.div_ceil(PAGE_SIZE)
}

#[derive(Debug, PartialEq, Eq)]
enum Fault {
    /// The page is resident; a copy-on-write write or permission upgrade.
    Minor,
    /// The mapping is valid but the page must be read from backing store.
    Major,
    /// The mapping is valid and resident, but the access permission is wrong.
    Protection,
    /// No valid mapping at this address.
    Invalid,
}

fn classify_fault(mapped: bool, present: bool, writable: bool) -> Fault {
    match (mapped, present, writable) {
        (false, _, _) => Fault::Invalid,
        (true, false, _) => Fault::Major,
        (true, true, false) => Fault::Protection,
        (true, true, true) => Fault::Minor,
    }
}
```

`split_page` divides by the page size for the page number, and takes the remainder for the offset. `pages_needed` rounds up with `div_ceil`, because 4,097 bytes need two pages, not one.

`classify_fault` matches on three facts at once, written as a tuple. They are: is the address mapped, is the page present in memory, and does the mapping allow the access. The arms are tried in order, so the first arm that fits wins. `(false, _, _)` catches every unmapped address, whatever the other two values are.

The model is a simplification. It treats every page that is not in memory as a major fault. A real kernel also counts some of those as minor, because no disk read is needed. Examples are a first touch of new memory, and a file page already in the kernel's file cache.

Listing 15.10 The complete program. `src/bin/os_paging.rs`

```

///! Virtual memory arithmetic: address to page and offset, page counts, and how a
///! page fault is classified.
///!
///! Run with: cargo run --bin os_paging

const PAGE_SIZE: u64 = 4096;

fn split_page(address: u64) -> (u64, u64) {
    (address / PAGE_SIZE, address % PAGE_SIZE)
}

fn pages_needed(bytes: u64) -> u64 {
    bytes.div_ceil(PAGE_SIZE)
}

#[derive(Debug, PartialEq, Eq)]
enum Fault {
    /// The page is resident; a copy-on-write write or permission upgrade.
    Minor,
    /// The mapping is valid but the page must be read from backing store.
    Major,
    /// The mapping is valid and resident, but the access permission is wrong.
    Protection,
    /// No valid mapping at this address.
    Invalid,
}

fn classify_fault(mapped: bool, present: bool, writable: bool) -> Fault {
    match (mapped, present, writable) {
        (false, _, _) => Fault::Invalid,
        (true, false, _) => Fault::Major,
        (true, true, false) => Fault::Protection,
        (true, true, true) => Fault::Minor,
    }
}

fn main() {
    println!("page size: {PAGE_SIZE} bytes");

    println!("\naddress -> page, offset");
    for address in [0u64, 0x1234, 0x1fff, 0x2000] {
        let (page, offset) = split_page(address);
        println!("  0x{address:04x} -> page {page:>2}, offset 0x{offset:03x}");
    }

    println!("\npages needed");
    for bytes in [0u64, 1, PAGE_SIZE, PAGE_SIZE + 1] {
        println!("  {bytes:>5} bytes -> {}", pages_needed(bytes));
    }
}

```

```

println!("\nfault classification");
println!(
    "    unmapped          -> {:?}" ,
    classify_fault(false, false, false)
);
println!(
    "    mapped, not in ram -> {:?}" ,
    classify_fault(true, false, true)
);
println!(
    "    resident, read-only-> {:?}" ,
    classify_fault(true, true, false)
);
println!(
    "    resident, writable -> {:?}" ,
    classify_fault(true, true, true)
);

assert_eq!(split_page(0x1234), (1, 0x234));
assert_eq!(pages_needed(0), 0);
assert_eq!(pages_needed(PAGE_SIZE + 1), 2);
assert_eq!(classify_fault(true, false, true), Fault::Major);
assert_eq!(classify_fault(true, true, false), Fault::Protection);
assert_eq!(classify_fault(false, true, true), Fault::Invalid);

println!("\nall checks passed");
}

```

```

$ cargo run --bin os_paging
page size: 4096 bytes

address -> page, offset
0x0000 -> page 0, offset 0x000
0x1234 -> page 1, offset 0x234
0x1fff -> page 1, offset 0xfff
0x2000 -> page 2, offset 0x000

pages needed
0 bytes -> 0
1 bytes -> 1
4096 bytes -> 1
4097 bytes -> 2

fault classification
unmapped          -> Invalid
mapped, not in ram -> Major
resident, read-only-> Protection
resident, writable -> Minor

all checks passed

```

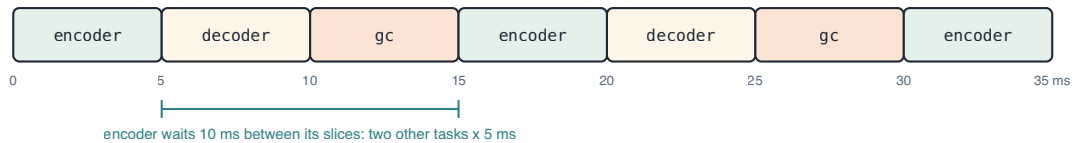
`0x1fff` is the last byte of page 1, and `0x2000` is the first byte of page 2.

15.6 Taking turns on the CPU

A machine runs more threads than it has cores. The operating system's **scheduler** decides which thread runs on each core, and switches between them many times per second. A **context switch** saves one thread's registers and loads another's.

Round robin is the simplest fair policy. The runnable tasks wait in a line. The first runs for a fixed time, called the **quantum** or time slice, and then goes to the back of the line (figure 15.6). No task can hold a core forever, so none starves.

Round robin with a 5 ms quantum: each runnable task gets one slice in turn



A task that still has work is stopped at the end of its slice and goes to the back of the line.

Figure 15.6 Three tasks sharing one core in 5 ms slices.

Listing 15.11 The complete program. src/bin/os_scheduler.rs

```

//! Round-robin scheduling: each runnable task gets one fixed time slice in turn,
//! so no task starves. This is the simplest fair policy a scheduler can use.
//!
//! Run with: cargo run --bin os_scheduler

const QUANTUM_MS: u64 = 5;

/// The task chosen on each of `ticks` time slices.
fn schedule<'a>(tasks: &[&'a str], ticks: usize) -> Vec<&'a str> {
    if tasks.is_empty() {
        return Vec::new();
    }
    (0..ticks).map(|tick| tasks[tick % tasks.len()]).collect()
}

fn main() {
    let tasks = ["encoder", "decoder", "gc"];

    println!("{}", runnable tasks, {QUANTUM_MS} ms quantum", tasks.len());
    for (tick, task) in schedule(&tasks, 7).iter().enumerate() {
        let start = tick as u64 * QUANTUM_MS;
        println!(
            "  slice {tick}: {start:>2}-{start+2:>2} ms {task}",
            start + QUANTUM_MS
        );
    }
}

```

```

    }

    let order = schedule(&tasks, 7);
    assert_eq!(
        order,
        [
            "encoder", "decoder", "gc", "encoder", "decoder", "gc", "encoder"
        ]
    );
    assert!(schedule(&[], 3).is_empty());

    // A long task is preempted, not run to completion, which is what bounds its
    // worst-case latency to one quantum times the number of tasks.
    println!(
        "\nworst-case wait for one slice: {} ms",
        QUANTUM_MS * tasks.len() as u64
    );
    println!("all checks passed");
}

```

`schedule` returns the task for each slice: `slice tick` goes to `task tick % tasks.len()`. The early return avoids a division by zero when there are no tasks.

The signature `fn schedule<'a>(tasks: &[&'a str], ticks: usize) -> Vec<&'a str>` has one named lifetime. It says that the returned names borrow from the same strings as the names in `tasks`, not from the slice that holds them. The caller can drop the slice and keep the result.

```

$ cargo run --bin os_scheduler
3 runnable tasks, 5 ms quantum
slice 0: 0- 5 ms  encoder
slice 1: 5-10 ms decoder
slice 2: 10-15 ms gc
slice 3: 15-20 ms encoder
slice 4: 20-25 ms decoder
slice 5: 25-30 ms gc
slice 6: 30-35 ms encoder

worst-case wait for one slice: 15 ms
all checks passed

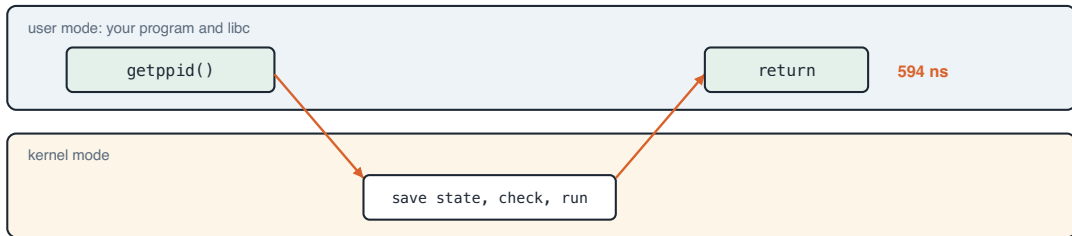
```

Between two of its own slices, a task waits 10 ms, while the other two tasks run. From an arbitrary moment, it waits at most one full round of 15 ms, which is the number the program prints. A shorter quantum makes tasks wait less, but switches more often, and each switch costs time.

15.7 The cost of a system call

A program cannot touch hardware or other processes directly. It asks the kernel, with a **system call**: opening a file, reading a socket, starting a thread. The CPU switches from user mode into kernel mode, the kernel does the work, and the CPU switches back (figure 15.7). The switch itself costs time, even when the work is tiny.

A system call crosses from user mode into the kernel and back



macOS libc answers getpid() from a value it stored in the process, so that call never enters the kernel: 3 ns.

Figure 15.7 A system call's round trip on this machine.

15.7.1 Calling C from Rust

The program calls `getpid`, which returns the process ID. `getpid` is a C function from the system's C library. The `libc` crate declares it for Rust.

Calling a function written in another language is called a **foreign function interface** (FFI) call. Rust cannot check what C code does, so every call to it must be inside an `unsafe` block. Each block here carries a `SAFETY` comment explaining why the call is sound. `getpid` takes no pointers and changes no memory.

Listing 15.12 The timed loop (lines 27 to 47). `src/bin/syscall_overhead.rs`

```

/// Calls `getpid()` `iterations` times and returns the average cost in
/// nanoseconds per call.
fn measure_getpid_ns_per_call(iterations: u64) -> f64 {
    let start = Instant::now();

    for _ in 0..iterations {
        // SAFETY: `libc::getpid` is a pure read-only system call with no
        // pointer arguments and no memory side effects. The FFI declaration in
        // the `libc` crate matches the Linux kernel ABI.
        //
        // `black_box` prevents the optimizer from assuming the loop has no
        // observable effect, which matters because the syscall result is
  
```

```

        // otherwise unused.
        unsafe {
            libc::getpid();
        }
    }

    let elapsed = start.elapsed();
    elapsed.as_nanos() as f64 / iterations as f64
}

```

The comment inside the loop mentions `black_box`, but the loop does not use it, and does not need it. The compiler cannot see inside an external C function, so it must assume each call has an effect, and it cannot remove the loop.

Listing 15.13 The complete program. `src/bin/syscall_overhead.rs`

```

// =====
// Syscall overhead measurement
//
// Ported from the Rust playground repo (`mkdir_syscall.rs`).
//
// What this measures:
// - Every syscall crosses the user/kernel boundary: argument validation,
//   kernel work, scheduling checks, and return to user space.
// - This program calls `getpid()` in a tight loop and estimates the average
//   per-call cost. `getpid()` is a tiny syscall, so the measured cost is
//   close to the irreducible syscall overhead on this kernel/hardware.
//
// Where this appears in practice:
// - High-performance data planes care about per-packet or per-I/O syscall
//   cost. The difference between one syscall per event and batching via
//   io_uring/recvmmsg can be enormous.
// - It also demonstrates unsafe FFI, safety comments, warm-up loops, and
//   benchmark methodology: warm the code path, run enough iterations, and
//   report average cost rather than one lucky sample.
//
// Run:
//   cargo run --bin syscall_overhead
// =====

use std::time::Instant;

/// Calls `getpid()` `iterations` times and returns the average cost in
/// nanoseconds per call.
fn measure_getpid_ns_per_call(iterations: u64) -> f64 {
    let start = Instant::now();

    for _ in 0..iterations {
        // SAFETY: `libc::getpid` is a pure read-only system call with no
        // pointer arguments and no memory side effects. The FFI declaration in
        // the `libc` crate matches the Linux kernel ABI.
        //
        // `black_box` prevents the optimizer from assuming the loop has no
        // observable effect, which matters because the syscall result is
        // otherwise unused.
        unsafe {

```



```

        libc::getpid());
    }
}

let elapsed = start.elapsed();
elapsed.as_nanos() as f64 / iterations as f64
}

fn main() {
    const ITERATIONS: u64 = 10_000_000;

    // Warm up the code path before timing. The first calls may fault in the
    // libc/PLT, cache lines, or CPU frequency may still be ramping.
    for _ in 0..100_000 {
        // SAFETY: same reasoning as above; read-only syscall.
        unsafe { libc::getpid() };
    }

    let ns_per_call = measure_getpid_ns_per_call(ITERATIONS);

    println!("Iterations:      {}", ITERATIONS);
    println!(
        "Total time:          {:.3}s",
        ns_per_call * ITERATIONS as f64 / 1e9
    );
    println!("Per syscall:         {:.1} ns", ns_per_call);

    // Example framing: if a service processed 10M packets/s and used one
    // getpid-style syscall per packet, this fraction of each second would be
    // spent crossing into the kernel just for the syscall overhead.
    let cpu_seconds_per_second = ns_per_call * 10_000_000.0 / 1_000_000_000.0;
    println!(
        "At 10M calls/s, syscall overhead would consume: {:.2} CPU-seconds per second",
        cpu_seconds_per_second
    );
}

// =====
// Tests (cargo test --bin syscall_overhead)
// =====
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn measurement_returns_a_positive_finite_number() {
        let ns = measure_getpid_ns_per_call(10_000);
        assert!(ns.is_finite(), "nanoseconds per call must be finite");
        assert!(ns > 0.0, "a syscall cannot cost zero time");
        // On any real kernel this is microseconds or tens/hundreds of ns, not
        // seconds per call.
        assert!(
            ns < 1_000_000.0,
            "suspiciously slow syscall measurement: {ns} ns"
        );
    }

    #[test]
    fn getpid_returns_nonzero_pid() {

```

```

    // SAFETY: read-only syscall, no pointers, no side effects.
    let pid = unsafe { libc::getpid() };
    assert!(pid > 0);
  }
}

```

`main` warms up with 100,000 untimed calls, then times ten million. Warming up lets the first-call costs, such as loading the function's code into the cache, happen outside the measurement.

```

$ cargo run --release --bin syscall_overhead
Iterations:      10000000
Total time:      0.031s
Per syscall:     3.1 ns
At 10M calls/s, syscall overhead would consume: 0.03 CPU-seconds per second

```

15.7.2 What the 3.1 ns measured

3.1 ns is far too fast for a trip into the kernel. It is about the time of two L1 reads. The program measured something other than its title says.

The C library on macOS stores the process ID the first time it is asked, and answers later `getpid` calls from that stored value. None of the ten million calls entered the kernel.

To measure a real round trip, the call must be one the library cannot answer by itself. On the same machine, I timed `getppid`, which returns the parent's process ID, and a raw `syscall(SYS_getpid)`. Both took about 594 ns per call, nearly 200 times the cached `getpid`.

On Linux, the answer is different again. The GNU C library stopped caching `getpid` in version 2.25, so there the same program does measure a real system call.

The lesson matches section 15.3. Before you trust a number, check that the code did the work you meant to time. The comment block at the top of the file names the practical point. A program that handles millions of events per second cannot afford one system call per event. Interfaces such as Linux's `recvmsg` and `io_uring` exist to handle many events per system call.

Summary

- Reading memory on this laptop took 2.2 ns from L1 and up to 146 ns from main memory. Caches move 64-byte lines.

- A bump allocator hands out bytes from an offset and frees everything at once. `(offset + align - 1) & !(align - 1)` rounds an offset up to a power-of-two alignment.
- Reading in order uses whole cache lines and lets the CPU prefetch. Scattered reads were about seven times slower when independent, and about 114 ns each when each depended on the last.
- The `cs_locality` benchmark mostly measured a division instruction. Change one thing at a time before explaining a number.
- False sharing slows threads that write different variables in one cache line. `#[repr(align(64))]` gave each counter its own line and made the loop up to ten times faster.
- A virtual address is a page number and an offset. The page table maps pages to frames, the TLB caches the mappings, and a page fault hands control to the kernel.
- Round robin gives each task one quantum in turn. A shorter quantum lowers waiting but adds switches.
- A system call costs hundreds of nanoseconds here. macOS answers `getpid` without one, so the benchmark measured a cached value.

Chapter 16 starts the concurrency part of the book: threads, atomics, and locks, and the rules that keep them correct.

Exercises

1. Change `strided_sum` to use `& (data.len() - 1)` instead of `% data.len()`, and compare its time with stride 1 and stride 1,000,003.
2. Write the pointer chase of section 15.1, and measure it on your machine for sizes from 4 KiB to 512 MiB. Find where the time jumps, and compare with your CPU's cache sizes.
3. Fix the warning in section 15.2. Make `BumpArena` allocate its buffer with `std::alloc::alloc` and a `Layout` aligned to 64. Add a test that requests alignment 64.
4. Change `concurrency_false_sharing.rs` to use `#[repr(align(128))]`, and check whether the time changes on your machine.
5. Time `getppid` in a loop on your system, and compare it with `getpid`.

PART 5

Concurrency



Threads, atomics, and locks

THIS CHAPTER COVERS

- Starting threads with `thread::spawn` and `thread::scope`, and collecting their results
- The `Send` and `Sync` traits, which let the compiler reject data races
- Atomic integers, and why `count += 1` from two threads can lose an update
- Splitting a sum across threads, measuring it, and Amdahl's law
- A spin lock built from one atomic flag, with acquire and release ordering
- A counting semaphore built from a `Mutex` and a `Condvar`
- What happens to a `Mutex` when its holder panics, and how to detect a deadlock

Chapter 15 showed that a laptop CPU has several cores. A program that runs on one thread uses one of them. To use the others, the program must split its work across several **threads**: independent sequences of instructions that the operating system schedules onto cores.

Threads share the process's memory, so they can all read the same data. That sharing also makes them dangerous. When two threads change the same memory at the same time, the result can be wrong in ways that are hard to reproduce.

Rust's promise here is narrow and specific. Safe Rust code cannot contain a **data race**. In a data race, two threads access the same memory at the same time, and at least one of them writes. Nothing orders the two accesses. The compiler checks this with two traits, `Send` and `Sync`. The chapter starts there.

16.1 Starting threads

16.1.1 `spawn` and `join`

`std::thread::spawn` starts a new thread that runs a closure. It returns a `JoinHandle`. Calling `join()` on the handle waits for the thread to finish and returns the closure's result:

```
let handle = thread::spawn(|| 6 * 7);
assert_eq!(handle.join().expect("worker panicked"), 42);
```

`join` returns a `Result`, because the thread might have panicked. `Ok` carries the closure's return value.

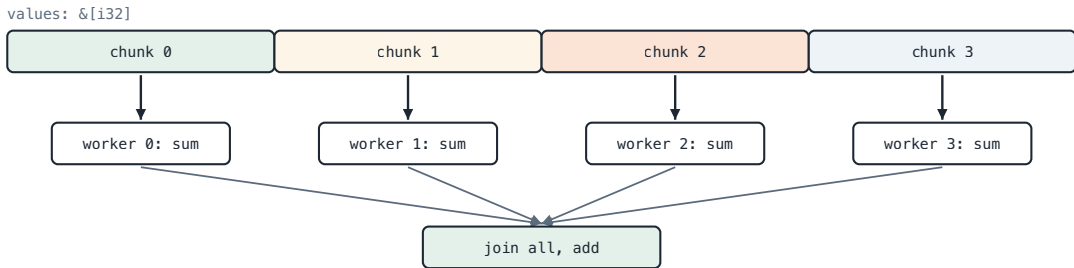
A spawned thread may outlive the function that started it. So the closure cannot borrow local variables, which might be gone by the time the thread uses them. The compiler requires the closure to own everything it uses, or to borrow only data that lives for the whole program. That requirement is written `'static`. To share data with a spawned thread, you move an `Arc` into the closure, as section 8.6 did.

16.1.2 Scoped threads

`thread::scope` removes that restriction for work that finishes before the function continues. It takes a closure that receives a `scope` value. Threads started with `scope.spawn` may borrow local data. The scope waits for every one of them before `thread::scope` returns, so the borrowed data is still alive while they run.

The file `threads.rs` uses a scope to sum a slice on several threads (figure 16.1).

thread::scope: workers borrow chunks of the slice, and the scope waits for all of them



Each worker holds a &[i32] into the caller's data. No Arc is needed, because the scope ends before the data can be dropped.

Figure 16.1 Each worker sums one chunk, borrowing it from the caller's slice.

Listing 16.1 `available_workers` and `parallel_sum` (lines 27 to 60). `src/problems/threads.rs`

```

/// Number of hardware threads the runtime can use, defaulting to 1.
pub fn available_workers() -> usize {
    thread::available_parallelism()
        .map(|count| count.get())
        .unwrap_or(1)
}

/// Sum a slice by splitting it into chunks that run on scoped threads.
///
/// `thread::scope` is what makes this ergonomic: the workers borrow `values`
/// directly, so there is no `Arc` and no `static` bound. The scope waits for
/// every thread, which removes the forgotten-join bug.
pub fn parallel_sum(values: &[i32]) -> i64 {
    if values.is_empty() {
        return 0;
    }

    let workers = available_workers().min(values.len());
    let chunk_size = values.len().div_ceil(workers);

    thread::scope(|scope| {
        let handles: Vec<_> = values
            .chunks(chunk_size)
            .map(|chunk| {
                scope.spawn(move || chunk.iter().map(|value|
i64::from(*value)).sum::<i64>())
            })
            .collect();

        handles
            .into_iter()
            .map(|handle| handle.join().expect("worker panicked"))
            .sum()
    })
}

```

`thread::available_parallelism()` asks the operating system how many threads can run at the same time. On this laptop it returns 8: four cores, each running two hardware threads. It returns a `Result`, so the code falls back to 1 if the question cannot be answered.

The function never starts more workers than there are values. `values.len().div_ceil(workers)` computes a chunk size that rounds up, so the chunks cover every value. `values.chunks(chunk_size)` then yields sub-slices of that size, with a shorter last one.

Each worker converts its values to `i64` with `i64::from(*value)` before adding. The sum of many `i32` values can exceed the `i32` range, and an `i64` has room for it.

The order of two steps decides whether the threads run in parallel. The workers are all started, and their handles collected into a `Vec`, before any is joined. Iterators are lazy. A single chain like `.map(spawn).map(join)` would start one thread, wait for it, then start the next. The work would run one chunk at a time. Collecting first starts every thread before waiting on any of them.

The closure passed to `scope.spawn` is a `move` closure. It moves `chunk`, a `&[i32]`, into the thread. Moving a reference copies the reference, not the data.

16.1.3 Send and Sync

How does the compiler know which values may be used from another thread? Two marker traits describe it:

- A type is `Send` if a value of it can be moved to another thread.
- A type is `Sync` if a reference to it, `&T`, can be shared with other threads. Put another way, `T` is `Sync` exactly when `&T` is `Send`.

These traits have no methods. They are **auto traits**: the compiler implements them for a type automatically when all of the type's fields have them. `i32`, `String`, `Vec<i32>`, and `Arc<Mutex<T>>` are both `Send` and `Sync`.

`Rc` is neither. Its reference count is a plain integer, so two threads cloning the same `Rc` could lose a count, as section 16.2 shows. `RefCell` is `Send` but not `Sync`: its borrow flag is not safe to change from two threads. `thread::spawn` and `scope.spawn` require their closures to be `Send`. A closure that captures an `Rc` is therefore rejected at compile time.

Listing 16.2 A compile-time check (lines 110 to 114).

```
/// A compile-time assertion that `T` may cross thread boundaries.
///
/// Calling `assert_send_sync::<Client>()` fails to compile if `Client` is not
/// thread-safe, which turns a concurrency assumption into a build error.
pub fn assert_send_sync<T: Send + Sync>() {}
```

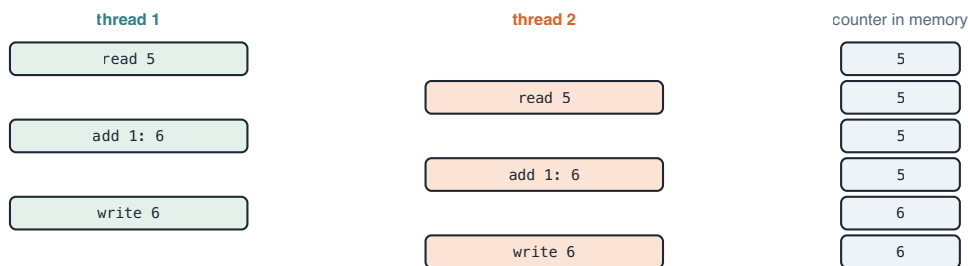
`assert_send_sync` has an empty body. Calling `assert_send_sync::<AtomicCounter>()` compiles only if `AtomicCounter` is `Send + Sync`. If a later change added an `Rc` field to it, the call would stop compiling. The check costs nothing at run time.

16.2 Atomic integers

16.2.1 The lost update

`count += 1` looks like one step, but the CPU performs three: read the value, add one, and write the result back. If two threads do this at the same time, their steps can interleave (figure 16.2).

Two threads add 1 to the same plain counter. One increment is lost



Both threads read 5 before either wrote. The final value is 6, not 7. `fetch_add` does the read, add, and write as one step.

Figure 16.2 A lost update. Both threads read 5, so one of the two increments disappears.

Safe Rust does not compile this: sharing a plain `usize` for writing between threads needs `&mut` in two places at once. Other languages compile it, and the program loses counts only under load.

An **atomic** type performs the read, the change, and the write as one indivisible operation. No other thread can see or change the value in between. `AtomicUsize::fetch_add(1, ordering)` adds 1 and returns the old value.

16.2.2 A counter without a lock

Listing 16.3 `AtomicCounter` and `scoped_increment` (lines 62 to 99).

```
/// A counter that several threads may increment without a lock.
///
/// `Relaxed` is correct here because the counter is the only value being
/// communicated; there is no other data whose visibility depends on it.
#[derive(Default)]
pub struct AtomicCounter {
    value: AtomicUsize,
}

impl AtomicCounter {
    pub const fn new() -> Self {
        Self {
            value: AtomicUsize::new(0),
        }
    }

    /// Increment and return the new value.
    pub fn increment(&self) -> usize {
        self.value.fetch_add(1, Ordering::Relaxed) + 1
    }

    pub fn get(&self) -> usize {
        self.value.load(Ordering::Relaxed)
    }
}

/// Increment a counter from several scoped threads that share it by reference.
pub fn scoped_increment(counter: &AtomicCounter, threads: usize, per_thread: usize) {
    thread::scope(|scope| {
        for _ in 0..threads {
            scope.spawn(|| {
                for _ in 0..per_thread {
                    counter.increment();
                }
            });
        }
    });
}
```

`increment` takes `&self`, not `&mut self`. An atomic can be changed through a shared reference, the same way a `RefCell` can, but safely across threads. So many threads can hold `&AtomicCounter` and increment it at once.

`fetch_add` returns the value before the addition, so `increment` adds 1 to report the new value.

`scoped_increment` starts `threads` threads that each increment the counter `per_thread` times. The closure has no `move`, so it borrows `counter`, which the scope allows. The test runs $8 \text{ threads} \times 500 \text{ increments}$ and checks that the total is exactly 4,000. No increment is lost.

16.2.3 Memory ordering

Every atomic operation takes an `Ordering` argument. The ordering answers a question beyond the atomic value itself. When this thread changes the atomic, what may other threads assume about the *other* memory this thread changed?

Compilers and CPUs reorder memory operations to run faster. Within one thread, the reordering is invisible. Another thread watching the memory can see writes in a different order from the one in the source code. The ordering argument limits that reordering around the atomic operation.

- `Relaxed` makes the operation itself atomic, and promises nothing about other memory. That is enough for a counter whose value no other data depends on, which is the case here.
- `Release`, used when writing, and `Acquire`, used when reading, work as a pair. Section 16.4 uses them to build a lock.
- `SeqCst` is the strongest. All `SeqCst` operations appear in one order that every thread agrees on. It is the simplest to reason about, and sometimes slower.

16.2.4 Initializing a value once

Listing 16.4 `OnceLock` (lines 101 to 108).

```
/// A value initialized once and shared by every caller.
///
/// `OnceLock` is the primitive behind lazy process-wide configuration, and it is
/// safe because initialization happens at most once even under concurrent calls.
pub fn build_channel() -> &'static str {
    static CHANNEL: OnceLock<&'static str> = OnceLock::new();
    CHANNEL.get_or_init(|| "stable")
}
```

A `static` is a single value shared by the whole program. `OnceLock` holds a value that starts empty and is set at most once. `get_or_init` runs the closure the first time, and returns the

stored value after that. If several threads call it at the same moment, one runs the closure and the others wait for its result.

Listing 16.5 The complete file, with tests. `src/problems/threads.rs`

```

///! Threads and fearless concurrency.
///!
///! Rust's claim about concurrency is narrow and checkable: safe code cannot
///! produce a data race, because `Send` and `Sync` describe what may cross a
///! thread boundary and the compiler refuses programs that violate the rules.
///!
///! - `Send` means a value may be moved to another thread.
///! - `Sync` means `&T` may be shared across threads.
///!
///! Both are auto traits: a type is `Send`/`Sync` when its fields are. `Rc` and
///! `RefCell` are the standard counter-examples, which is precisely why sharing
///! them across threads does not compile. The compile error is the feature, not an
///! obstacle.
///!
///! Three ways to get work onto other threads, in increasing order of ceremony:
///!
///! 1. `thread::scope` for work that borrows local data and finishes with the
///!    scope. No `static`, no `Arc`.
///! 2. `thread::spawn` for work that must own its data and outlive the caller.
///! 3. A thread pool when the number of units of work is large and threads should
///!    be reused.

use std::sync::OnceLock;
use std::sync::atomic::{AtomicUsize, Ordering};
use std::thread;

///! Number of hardware threads the runtime can use, defaulting to 1.
pub fn available_workers() -> usize {
    thread::available_parallelism()
        .map(|count| count.get())
        .unwrap_or(1)
}

///! Sum a slice by splitting it into chunks that run on scoped threads.
///!
///! `thread::scope` is what makes this ergonomic: the workers borrow `values`
///! directly, so there is no `Arc` and no `static` bound. The scope waits for
///! every thread, which removes the forgotten-join bug.
pub fn parallel_sum(values: &[i32]) -> i64 {
    if values.is_empty() {
        return 0;
    }

    let workers = available_workers().min(values.len());
    let chunk_size = values.len().div_ceil(workers);

    thread::scope(|scope| {
        let handles: Vec<_> = values
            .chunks(chunk_size)
            .map(|chunk| {
                scope.spawn(move || chunk.iter().map(|value|
i64::from(*value)).sum:<i64>())

```

```

        })
        .collect();

    handles
        .into_iter()
        .map(|handle| handle.join().expect("worker panicked"))
        .sum()
    })
}

/// A counter that several threads may increment without a lock.
///
/// `Relaxed` is correct here because the counter is the only value being
/// communicated; there is no other data whose visibility depends on it.
#[derive(Default)]
pub struct AtomicCounter {
    value: AtomicUsize,
}

impl AtomicCounter {
    pub const fn new() -> Self {
        Self {
            value: AtomicUsize::new(0),
        }
    }

    /// Increment and return the new value.
    pub fn increment(&self) -> usize {
        self.value.fetch_add(1, Ordering::Relaxed) + 1
    }

    pub fn get(&self) -> usize {
        self.value.load(Ordering::Relaxed)
    }
}

/// Increment a counter from several scoped threads that share it by reference.
pub fn scoped_increment(counter: &AtomicCounter, threads: usize, per_thread: usize) {
    thread::scope(|scope| {
        for _ in 0..threads {
            scope.spawn(|| {
                for _ in 0..per_thread {
                    counter.increment();
                }
            });
        }
    });
}

/// A value initialized once and shared by every caller.
///
/// `OnceLock` is the primitive behind lazy process-wide configuration, and it is
/// safe because initialization happens at most once even under concurrent calls.
pub fn build_channel() -> &'static str {
    static CHANNEL: OnceLock<&'static str> = OnceLock::new();
    CHANNEL.get_or_init(|| "stable")
}

/// A compile-time assertion that `T` may cross thread boundaries.

```

```

///
/// Calling `assert_send_sync::<Client>()` fails to compile if `Client` is not
/// thread-safe, which turns a concurrency assumption into a build error.
pub fn assert_send_sync<T: Send + Sync>() {}

#[cfg(test)]
mod tests {
    use super::*;
    use std::thread;

    #[test]
    fn parallel_sum_matches_the_sequential_sum() {
        let values: Vec<i32> = (1..=10_000).collect();
        let expected: i64 = values.iter().map(|value| i64::from(*value)).sum();
        assert_eq!(parallel_sum(&values), expected);
    }

    #[test]
    fn parallel_sum_handles_small_and_empty_inputs() {
        assert_eq!(parallel_sum(&[]), 0);
        assert_eq!(parallel_sum(&[7]), 7);
        assert_eq!(parallel_sum(&[-1, 1, -2, 2]), 0);
    }

    #[test]
    fn scoped_threads_share_a_counter_without_arc() {
        let counter = AtomicCounter::new();
        scoped_increment(&counter, 8, 500);
        assert_eq!(counter.get(), 4_000);
    }

    #[test]
    fn spawn_returns_a_join_handle_that_carries_the_result() {
        let handle = thread::spawn(|| 6 * 7);
        assert_eq!(handle.join().expect("worker panicked"), 42);
    }

    #[test]
    fn once_lock_initializes_a_single_value() {
        assert_eq!(build_channel(), "stable");
        assert_eq!(build_channel(), build_channel());
    }

    #[test]
    fn counter_is_thread_safe_by_construction() {
        assert_send_sync::<AtomicCounter>();
        assert!(available_workers() >= 1);
    }
}

```

```

$ cargo test --lib problems::threads
running 6 tests
test problems::threads::tests::counter_is_thread_safe_by_construction ... ok
test problems::threads::tests::once_lock_initializes_a_single_value ... ok
test problems::threads::tests::spawn_returns_a_join_handle_that_carries_the_result ... ok
test problems::threads::tests::scoped_threads_share_a_counter_without_arc ... ok
test problems::threads::tests::parallel_sum_handles_small_and_empty_inputs ... ok
test problems::threads::tests::parallel_sum_matches_the_sequential_sum ... ok

```

```
test result: ok. 6 passed; 0 failed; 0 ignored; 0 measured; 131 filtered out
```

16.3 How much faster do threads make it?

16.3.1 A billion elements

`parallel_sum.rs` times the same idea at a large scale. It sums a billion elements three times, as `i32`, `i64`, and `i128`, once on one thread and once on several.

Listing 16.6 The parallel sum for `i32` (lines 13 to 29). `src/bin/parallel_sum.rs`

```
fn par_i32(data: &[i32], n: usize) -> (i32, Duration) {
    let t = Instant::now();
    let sum = thread::scope(|s| {
        let handles: Vec<_> = (0..n)
            .map(|i| {
                s.spawn(move || {
                    let chunk = data.len() / n;
                    let start = i * chunk;
                    let end = if i == n - 1 { data.len() } else { start + chunk };
                    data[start..end].iter().sum::<i32>()
                })
            })
            .collect();
        handles.into_iter().map(|h| h.join().unwrap()).sum::<i32>()
    });
    (sum, t.elapsed())
}
```

This version divides the work by index instead of with `chunks`. Worker `i` sums the range from `i * chunk` to the next boundary. The last worker takes everything to the end, so the leftover elements from the integer division are not lost. The handles are collected before joining, for the reason in section 16.1.2.

Listing 16.7 The complete program. `src/bin/parallel_sum.rs`

```
use std::env;
use std::thread;
use std::time::{Duration, Instant};

const SIZE: usize = 1_000_000_000;

fn seq_i32(data: &[i32]) -> (i32, Duration) {
    let t = Instant::now();
    let sum = data.iter().sum::<i32>();
    (sum, t.elapsed())
}
```

```

fn par_i32(data: &[i32], n: usize) -> (i32, Duration) {
    let t = Instant::now();
    let sum = thread::scope(|s| {
        let handles: Vec<_> = (0..n)
            .map(|i| {
                s.spawn(move || {
                    let chunk = data.len() / n;
                    let start = i * chunk;
                    let end = if i == n - 1 { data.len() } else { start + chunk };
                    data[start..end].iter().sum::<i32>()
                })
            })
        .collect();
        handles.into_iter().map(|h| h.join().unwrap()).sum::<i32>()
    });
    (sum, t.elapsed())
}

fn seq_i64(data: &[i64]) -> (i64, Duration) {
    let t = Instant::now();
    let sum = data.iter().sum::<i64>();
    (sum, t.elapsed())
}

fn par_i64(data: &[i64], n: usize) -> (i64, Duration) {
    let t = Instant::now();
    let sum = thread::scope(|s| {
        let handles: Vec<_> = (0..n)
            .map(|i| {
                s.spawn(move || {
                    let chunk = data.len() / n;
                    let start = i * chunk;
                    let end = if i == n - 1 { data.len() } else { start + chunk };
                    data[start..end].iter().sum::<i64>()
                })
            })
        .collect();
        handles.into_iter().map(|h| h.join().unwrap()).sum::<i64>()
    });
    (sum, t.elapsed())
}

fn seq_i128(data: &[i128]) -> (i128, Duration) {
    let t = Instant::now();
    let sum = data.iter().sum::<i128>();
    (sum, t.elapsed())
}

fn par_i128(data: &[i128], n: usize) -> (i128, Duration) {
    let t = Instant::now();
    let sum = thread::scope(|s| {
        let handles: Vec<_> = (0..n)
            .map(|i| {
                s.spawn(move || {
                    let chunk = data.len() / n;
                    let start = i * chunk;
                    let end = if i == n - 1 { data.len() } else { start + chunk };
                    data[start..end].iter().sum::<i128>()
                })
            })
        .collect();
        handles.into_iter().map(|h| h.join().unwrap()).sum::<i128>()
    });
    (sum, t.elapsed())
}

```



```

        })
    })
    .collect();
    handles.into_iter().map(|h| h.join().unwrap()).sum::<i128>()
});
(sum, t.elapsed())
}

fn main() {
    let n: usize = env::args()
        .nth(1)
        .and_then(|s| s.parse().ok())
        .unwrap_or(8);

    {
        let data: Vec<i32> = vec![1; SIZE];
        let (s1, t1) = seq_i32(&data);
        let (s2, t2) = par_i32(&data, n);
        assert_eq!(s1, s2);
        println!("i32    seq sum = {s1}, elapsed = {t1:?}");
        println!("i32    par sum = {s2}, elapsed = {t2:?}  (n = {n})");
    }

    {
        let data: Vec<i64> = vec![1; SIZE];
        let (s1, t1) = seq_i64(&data);
        let (s2, t2) = par_i64(&data, n);
        assert_eq!(s1, s2);
        println!("i64    seq sum = {s1}, elapsed = {t1:?}");
        println!("i64    par sum = {s2}, elapsed = {t2:?}  (n = {n})");
    }

    {
        let data: Vec<i128> = vec![1; SIZE];
        let (s1, t1) = seq_i128(&data);
        let (s2, t2) = par_i128(&data, n);
        assert_eq!(s1, s2);
        println!("i128  seq sum = {s1}, elapsed = {t1:?}");
        println!("i128  par sum = {s2}, elapsed = {t2:?}  (n = {n})");
    }
}

```

The thread count comes from the first command-line argument, and defaults to 8. Each block creates its vector, times both sums, and drops the vector at the closing brace before the next block starts.

WARNING: The three vectors take 4 GB, 8 GB, and 16 GB of memory. On a machine with 16 GB, the larger two do not fit next to everything else in memory. The operating system then compresses pages or swaps them to disk. Lower `SIZE` before you run this on a small machine.

Here is one run on the 16 GB laptop:

```
$ cargo run --release --bin parallel_sum
i32  seq sum = 1000000000, elapsed = 522.646188ms
i32  par sum = 1000000000, elapsed = 142.427408ms (n = 8)
i64  seq sum = 1000000000, elapsed = 7.750109265s
i64  par sum = 1000000000, elapsed = 2.591278253s (n = 8)
i128 seq sum = 1000000000, elapsed = 16.173010504s
i128 par sum = 1000000000, elapsed = 10.198263426s (n = 8)
```

Read these numbers with care. The `i32` rows are the only ones that measure the sum. Eight threads made it about 3.7 times faster, not 8 times. The `i64` and `i128` rows are dominated by the memory pressure from the warning. Reading 8 GB should take well under a second, so 7.7 seconds means pages were being brought back from compression or disk.

The runs were not stable, either. In an earlier run, the sequential `i32` sum took 4.6 seconds instead of 0.52. To check the `i32` result, I wrote a separate test that fills the vector once and times each sum several times. The sequential sum took 0.25 to 0.29 seconds, and eight threads took 0.08 seconds, about 3.2 times faster.

Why not 8 times? Summing is almost no work per byte, so the loop is limited by how fast memory delivers bytes, not by how fast cores add. One core cannot use all of the memory bandwidth, so several cores help, but only until the memory bus is full. The laptop also has four cores, not eight. Its eight hardware threads share those cores in pairs.

16.3.2 Amdahl's law

Most programs also contain work that cannot be split: reading input, starting threads, combining results. **Amdahl's law** states how much that serial part limits the speedup.

Suppose a fraction `s` of the run time is serial, and `p` processors share the rest perfectly. The run time becomes `s + (1 - s) / p` of the original, and the speedup is one divided by that:

```
speedup(s, p) = 1 / (s + (1 - s) / p)
```

Work through one case by hand: `s = 0.5` and `p = 4`. The run time is `0.5 + 0.5 / 4 = 0.625` of the original, so the speedup is `1 / 0.625 = 1.6`.

Listing 16.8 The complete program. `src/bin/concurrency_amdahl.rs`

```
//! Amdahl's law: the serial fraction of the work caps the speedup from adding
//! processors. A program that is 50 percent serial cannot go faster than 2x, no
//! matter how many cores it gets.
//!
//! Run with: cargo run --bin concurrency_amdahl
```

```

/// Speedup with `processors` when `serial` is the non-parallel fraction.
fn speedup(serial: f64, processors: f64) -> f64 {
    1.0 / (serial + (1.0 - serial) / processors)
}

fn main() {
    let processor_counts = [1.0, 2.0, 4.0, 8.0, 16.0];
    let serial_fractions = [0.0, 0.05, 0.10, 0.25, 0.50];

    print!("{:>8} |", "serial");
    for processors in processor_counts {
        print!(" {processors:>6.0}");
    }
    println!("    <- processors");
    println!("{:~>9}+{~}", "", "-".repeat(7 * processor_counts.len()));

    for serial in serial_fractions {
        print!("{serial:>8.2} |");
        for processors in processor_counts {
            print!(" {:>6.2}", speedup(serial, processors));
        }
        println();
    }

    println!("\nlimit as processors grow: 1 / serial");
    for serial in serial_fractions {
        let limit = if serial == 0.0 {
            f64::INFINITY
        } else {
            1.0 / serial
        };
        println!(" {serial:>4.2} -> {limit:>6.1}");
    }

    assert!((speedup(0.0, 4.0) - 4.0).abs() < 1e-9);
    assert!((speedup(0.5, 4.0) - 1.6).abs() < 1e-9);
    assert!(speedup(0.5, 16.0) < 1.9, "2x is the ceiling for 50% serial");

    println!("\nall checks passed");
}

```

```

$ cargo run --bin concurrency_amdahl
serial |      1      2      4      8     16    <- processors
-----+-----
0.00 | 1.00  2.00  4.00  8.00 16.00
0.05 | 1.00  1.90  3.48  5.93  9.14
0.10 | 1.00  1.82  3.08  4.71  6.40
0.25 | 1.00  1.60  2.29  2.91  3.37
0.50 | 1.00  1.33  1.60  1.78  1.88

limit as processors grow: 1 / serial
0.00 -> inf
0.05 -> 20.0
0.10 -> 10.0
0.25 -> 4.0
0.50 -> 2.0

```

all checks passed

Amdahl's law: $\text{speedup} = 1 / (s + (1 - s) / p)$

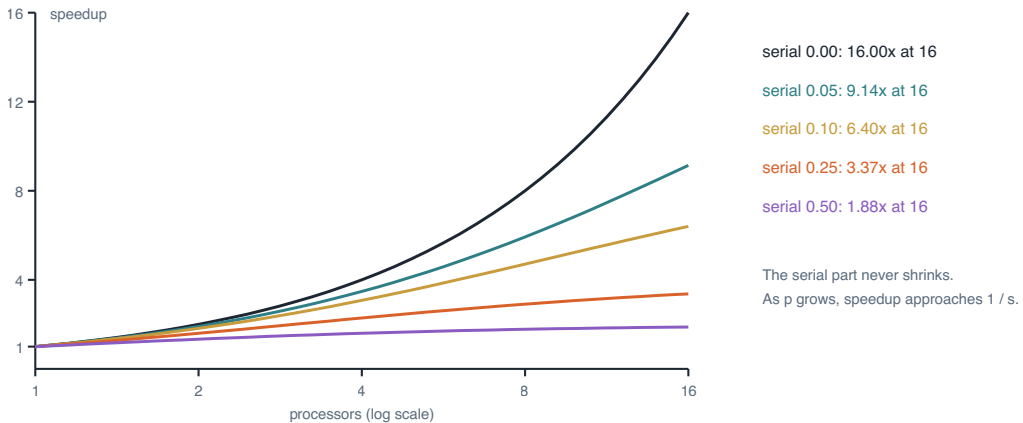


Figure 16.3 The table as curves. With 5% serial work, 16 processors give a speedup of 9.14.

As p grows, $(1 - s) / p$ shrinks toward zero, and the speedup approaches $1 / s$. A program that is 5% serial can never run more than 20 times faster, however many cores it gets.

16.4 A spin lock

A `Mutex` from the standard library lets one thread at a time use a value. This section builds a simpler lock from one atomic `bool`, to show what a lock must do.

A **spin lock** waits by looping. A thread that wants the lock checks it over and over until it is free. This works well when the lock is held for a very short time. When it is held longer, the waiting threads waste whole cores doing nothing useful. `std::sync::Mutex` instead asks the operating system to put a waiting thread to sleep.

16.4.1 The type

Listing 16.9 The lock, the guard, and `new` (lines 24 to 48). `src/problems/spin_lock.rs`

```
/// A mutual-exclusion lock that busy-waits instead of parking.
pub struct SpinLock<T> {
    locked: AtomicBool,
    value: UnsafeCell<T>,
}
```

```

// SAFETY: `value` is only accessed while `locked` is held, and the acquire/release
// ordering on `locked` makes a hand-off between threads well defined. Sharing
// `&SpinLock` across threads therefore only requires the protected value to be
// `Send`, which lets one thread move data in and another move it out.
unsafe impl<T: Send> Sync for SpinLock<T> {}

/// Grants access to the protected value and releases the lock when dropped.
pub struct SpinGuard<'a, T> {
    lock: &'a SpinLock<T>,
}

/// Create an unlocked spin lock.
pub const fn new(value: T) -> Self {
    Self {
        locked: AtomicBool::new(false),
        value: UnsafeCell::new(value),
    }
}

```

`SpinLock<T>` holds a flag, `locked`, and the protected value inside an `UnsafeCell<T>`. `UnsafeCell` is the one type through which Rust allows changing data behind a shared reference. `RefCell`, `Mutex`, and the atomic types are all built on it. Its `get()` method returns a raw pointer, `*mut T`. Using that pointer requires `unsafe`, and the code must make sure no two threads use it at once.

`UnsafeCell` is not `Sync`, so `SpinLock` would not be `Sync` either. The line `unsafe impl<T: Send> Sync for SpinLock<T> {}` declares that it is. The compiler cannot check this claim, which is why the implementation needs `unsafe`. The `SAFETY` comment above it gives the reason the claim holds: the value is used only while the lock is held.

The bound is `T: Send`, not `T: Sync`. Only one thread uses the value at a time, but different threads use it one after another, so the value effectively moves between threads.

`SpinGuard` is returned by `lock`. It holds a reference to the lock, gives access to the value, and releases the lock when it is dropped.

16.4.2 Taking and releasing the lock

Listing 16.10 `lock`, `try_lock`, and `unlock` (lines 50 to 84).

```

/// Acquire the lock, spinning until it is free.
pub fn lock(&self) -> SpinGuard<'_, T> {
    loop {
        if self
            .locked
            .compare_exchange_weak(false, true, Ordering::Acquire, Ordering::Relaxed)
            .is_ok()

```

```

    {
        return SpinGuard { lock: self };
    }

    // Spin on a relaxed load so the contention loop does not keep
    // bouncing the cache line with exclusive accesses.
    while self.locked.load(Ordering::Relaxed) {
        spin_loop();
    }
}

/// Acquire the lock only if it is free right now.
pub fn try_lock(&self) -> Option<SpinGuard<'_, T>> {
    self.locked
        .compare_exchange(false, true, Ordering::Acquire, Ordering::Relaxed)
        .ok()
        .map(|_| SpinGuard { lock: self })
}

/// Whether the lock is currently held.
pub fn is_locked(&self) -> bool {
    self.locked.load(Ordering::Relaxed)
}

fn unlock(&self) {
    self.locked.store(false, Ordering::Release);
}

```

Taking the lock must be one atomic step: check that it is free and mark it taken, with no other thread slipping in between. `compare_exchange(false, true, ...)` does that. If the flag is `false`, it sets it to `true` and returns `Ok`. If it is already `true`, it changes nothing and returns `Err`.

`lock` uses `compare_exchange_weak`, which may fail even when the flag is `false`. The weak form is cheaper on some CPUs, and inside a loop a false failure costs one more pass.

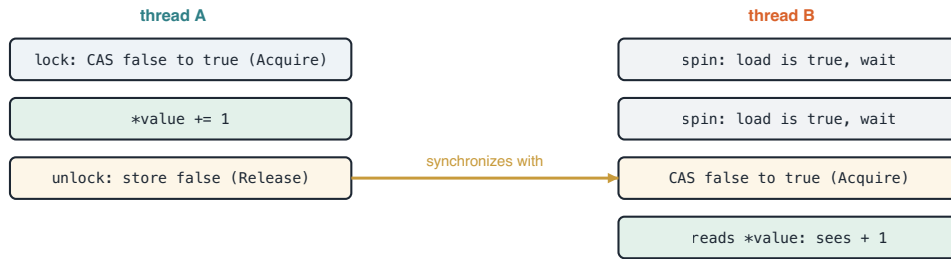
When the exchange fails, the thread waits in an inner loop that only reads the flag with a `Relaxed` load. A read lets the cache line stay shared between the waiting cores. A compare-exchange must take the line for writing, as section 15.4 described, so repeating it would bounce the line between cores. When the read sees `false`, the outer loop tries the exchange again. `spin_loop()` tells the CPU that this is a wait loop. The CPU can then save power and let the other hardware thread on the core run.

`try_lock` makes one attempt and returns `None` if the lock is taken.

16.4.3 Acquire and release

The orderings in `lock` and `unlock` make the lock correct (figure 16.4).

Release and acquire: what one thread wrote before releasing, the next sees after acquiring



Release keeps A's earlier writes from moving after the unlock. Acquire keeps B's later reads from moving before the lock. Together they guarantee that B sees the increment.

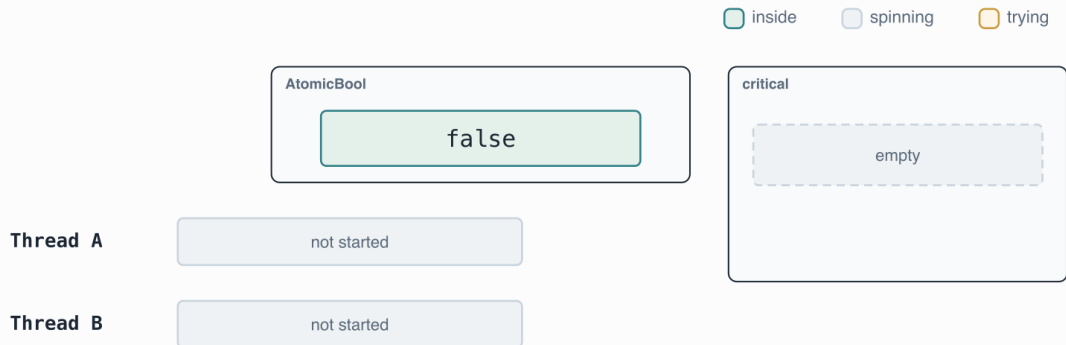
Figure 16.4 The release store in `unlock` pairs with the acquire exchange in the next `lock`.

- `unlock` stores `false` with `Release`. Every write the thread made before the store happens before the store, as far as other threads can tell. That includes its changes to the value.
- `lock` exchanges with `Acquire`. Every read the thread makes after the exchange happens after it.

When B's acquire reads the `false` that A's release wrote, the two operations **synchronize**. Everything A wrote before unlocking is then visible to B after locking. With `Relaxed` in both places, B could take the lock and still read an old copy of the value.

A spin lock: one atomic word, two threads

`compare_exchange(false, true)` takes the lock. A plain store releases it.



The lock is one `AtomicBool` holding `false`, and nothing owns it.

A spin lock is worth it only when the critical section is shorter than a thread wake-up.

lock = **free** owner = -

Animation 16.1 Two threads contending for one atomic word. The box shows the value the atomic holds, and the lanes show what each thread is doing. B never blocks in the kernel: it fails the exchange, loops, and tries again, which is why this is called a spin lock. The handoff happens when A's release store of `false` is read by B's acquire exchange, which is the `synchronize-with` pair from figure 16.4.

The second ordering argument to `compare_exchange`, `Relaxed`, applies when the exchange fails. A failed attempt does not take the lock, so it needs no ordering.

16.4.4 The guard

Listing 16.11 The guard's traits (lines 93 to 113).

```
impl<T> Drop for SpinGuard<'_, T> {
    fn drop(&mut self) {
        self.lock.unlock();
    }
}

impl<T> Deref for SpinGuard<'_, T> {
    type Target = T;
```



```

    fn deref(&self) -> &T {
        // SAFETY: holding the guard means this thread has exclusive access.
        unsafe { &*self.lock.value.get() }
    }
}

impl<T> DerefMut for SpinGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T {
        // SAFETY: holding the guard means this thread has exclusive access.
        unsafe { &mut *self.lock.value.get() }
    }
}

```

The guard implements three traits:

- `Drop` releases the lock. The lock is released when the guard goes out of scope, even if the code between panics. This pattern, tying a resource's release to a value's drop, is called **RAII**.
- `Deref` lets `*guard` and method calls on the guard reach the value, as `&T`.
- `DerefMut` gives `&mut T`, so `*lock.lock() += 1` changes the value.

Each method turns the raw pointer from `UnsafeCell::get` into a reference. That is sound only because holding the guard means holding the lock.

WARNING: `SpinGuard` has a gap. Its auto traits come from its field `&SpinLock<T>`, so the guard is `Sync` whenever `T` is `Send`. Take a type such as `Cell<u64>`, which is `Send` but not `Sync`. Safe code can share `&SpinGuard<Cell<u64>>` between two scoped threads, and both can call `Cell::set` at once. That is a data race. I compiled such a program to confirm it. `std::sync::MutexGuard` avoids this by being `Sync` only when `T` is `Sync`. Exercise 3 asks you to close the gap.

Listing 16.12 The complete file, with tests. `src/problems/spin_lock.rs`

```

///! A spin lock built from a single `AtomicBool`.
///!
///! The loop is short. The memory ordering and the ownership argument are what
///! make the unsafe block sound.
///!
///! - `lock`/`try_lock` use a compare-exchange with `Acquire` ordering, so the
///!   critical section observes every write the previous holder made before it
///!   released.
///! - `unlock` uses a `Release` store, so writes inside the critical section are
///!   visible to whoever acquires next.
///! - The guard derefs to `&T`/`&mut T`. Mutual exclusion is what makes handing out
///!   `&mut T` sound, and that is exactly what the lock guarantees.
///!

```

```

///! A spin lock is the wrong choice when the critical section can block or is
///! long: every waiting thread burns a core. Use it for very short sections on
///! dedicated cores, and prefer `std::sync::Mutex` (which parks the thread) in
///! general code.

use std::cell::UnsafeCell;
use std::hint::spin_loop;
use std::ops::{Deref, DerefMut};
use std::sync::atomic::{AtomicBool, Ordering};

/// A mutual-exclusion lock that busy-waits instead of parking.
pub struct SpinLock<T> {
    locked: AtomicBool,
    value: UnsafeCell<T>,
}

// SAFETY: `value` is only accessed while `locked` is held, and the acquire/release
// ordering on `locked` makes a hand-off between threads well defined. Sharing
// `&SpinLock` across threads therefore only requires the protected value to be
// `Send`, which lets one thread move data in and another move it out.
unsafe impl<T: Send> Sync for SpinLock<T> {}

/// Grants access to the protected value and releases the lock when dropped.
pub struct SpinGuard<'a, T> {
    lock: &'a SpinLock<T>,
}

impl<T> SpinLock<T> {
    /// Create an unlocked spin lock.
    pub const fn new(value: T) -> Self {
        Self {
            locked: AtomicBool::new(false),
            value: UnsafeCell::new(value),
        }
    }

    /// Acquire the lock, spinning until it is free.
    pub fn lock(&self) -> SpinGuard<'_, T> {
        loop {
            if self
                .locked
                .compare_exchange_weak(false, true, Ordering::Acquire, Ordering::Relaxed)
                .is_ok()
            {
                return SpinGuard { lock: self };
            }

            // Spin on a relaxed load so the contention loop does not keep
            // bouncing the cache line with exclusive accesses.
            while self.locked.load(Ordering::Relaxed) {
                spin_loop();
            }
        }
    }

    /// Acquire the lock only if it is free right now.
    pub fn try_lock(&self) -> Option<SpinGuard<'_, T>> {
        self.locked
            .compare_exchange(false, true, Ordering::Acquire, Ordering::Relaxed)

```

```

        .ok()
        .map(|_| SpinGuard { lock: self })
    }

    /// Whether the lock is currently held.
    pub fn is_locked(&self) -> bool {
        self.locked.load(Ordering::Relaxed)
    }

    fn unlock(&self) {
        self.locked.store(false, Ordering::Release);
    }
}

impl<T: Default> Default for SpinLock<T> {
    fn default() -> Self {
        Self::new(T::default())
    }
}

impl<T> Drop for SpinGuard<'_, T> {
    fn drop(&mut self) {
        self.lock.unlock();
    }
}

impl<T> Deref for SpinGuard<'_, T> {
    type Target = T;

    fn deref(&self) -> &T {
        // SAFETY: holding the guard means this thread has exclusive access.
        unsafe { &*self.lock.value.get() }
    }
}

impl<T> DerefMut for SpinGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T {
        // SAFETY: holding the guard means this thread has exclusive access.
        unsafe { &mut *self.lock.value.get() }
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::Arc;
    use std::thread;

    #[test]
    fn serializes_concurrent_increments() {
        let lock = Arc::new(SpinLock::new(0usize));
        let mut handles = Vec::new();

        for _ in 0..8 {
            let lock = Arc::clone(&lock);
            handles.push(thread::spawn(move || {
                for _ in 0..1_000 {
                    *lock.lock() += 1;
                }
            }));
        }
    }
}

```

```

    }));
}
for handle in handles {
    handle.join().expect("worker panicked");
}

assert_eq!(*lock.lock(), 8_000);
}

#[test]
fn try_lock_reports_contention() {
    let lock = SpinLock::new(1u32);
    let guard = lock.lock();
    assert!(lock.try_lock().is_none());
    assert!(lock.is_locked());
    drop(guard);
    assert!(lock.try_lock().is_some());
    assert!(!lock.is_locked());
}

#[test]
fn guard_gives_mutable_access() {
    let lock = SpinLock::new(vec![1, 2, 3]);
    {
        let mut guard = lock.lock();
        guard.push(4);
    }
    assert_eq!(*lock.lock(), vec![1, 2, 3, 4]);
}
}

```

The first test shares the lock between 8 threads with an `Arc`, and each adds 1,000. The total must be 8,000.

```

$ cargo test --lib problems::spin_lock
running 3 tests
test problems::spin_lock::tests::try_lock_reports_contention ... ok
test problems::spin_lock::tests::guard_gives_mutable_access ... ok
test problems::spin_lock::tests::serializes_concurrent_increments ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 134 filtered out

```

16.5 A counting semaphore

A lock lets one thread at a time proceed. A **semaphore** lets up to N threads proceed at once. It holds a count of **permits**. A thread takes a permit before it starts, and returns it when it finishes. When no permit is left, the next thread waits (figure 16.5).

Semaphores limit how much of something runs at once. Examples are open connections to a database, requests to a server, and open files.

A semaphore with 2 permits: at most two threads run the guarded section at once



Dropping a SemaphoreGuard adds 1 to available and wakes one waiter, which then takes the permit.

Figure 16.5 A semaphore with two permits, both taken.

16.5.1 Waiting without spinning

A waiting thread should sleep, not spin, because the wait may be long. A **condition variable**, `Condvar`, lets a thread sleep until another thread signals it. It always works together with a `Mutex`:

1. The waiting thread locks the mutex and checks the condition.
2. If the condition is false, it calls `condvar.wait(guard)`. That call releases the mutex and puts the thread to sleep, in one step.
3. Another thread locks the mutex, changes the state, and calls `notify_one()`.
4. The sleeping thread wakes up, holding the mutex again, and checks the condition again.

The check in step 4 is needed for two reasons. Another thread may have taken the permit first. And a thread can sometimes wake up with no notification at all, which is called a **spurious wakeup**. So the wait always sits in a `while` loop, not an `if`.

Listing 16.13 The state and `acquire_guard` (lines 13 to 45). `src/problems/semaphore.rs`

```

struct State {
    available: usize,
}

/// A counting semaphore. Cloning is not provided; share one behind `Arc`.
pub struct Semaphore {
    state: Mutex<State>,
    released: Condvar,
}

/// Create a semaphore with `permits` permits available.
pub fn new(permits: usize) -> Self {
    Self {
        state: Mutex::new(State { available: permits }),
        released: Condvar::new(),
    }
}

```

```

    }
}

/// Block until a permit is available.
pub fn acquire(&self) {
    let _guard = self.acquire_guard();
}

/// Block until a permit is available, returning a guard that releases it.
pub fn acquire_guard(&self) -> SemaphoreGuard<'_> {
    let mut state = self.state.lock().expect("semaphore mutex poisoned");
    while state.available == 0 {
        state = self.released.wait(state).expect("semaphore mutex poisoned");
    }
    state.available -= 1;
    SemaphoreGuard { semaphore: self }
}

```

`acquire_guard` follows the four steps. `self.released.wait(state)` takes the guard, sleeps, and returns a new guard when the thread wakes. When the loop ends, a permit is free and the thread holds the mutex, so it takes the permit by subtracting 1.

The permit is returned by a guard, the same RAI pattern as in section 16.4.

Listing 16.14 The other ways to acquire, and `release` (lines 47 to 103).

```

/// Take a permit only if one is free right now.
pub fn try_acquire(&self) -> Option<SemaphoreGuard<'_>> {
    let mut state = self.state.lock().expect("semaphore mutex poisoned");
    if state.available == 0 {
        None
    } else {
        state.available -= 1;
        Some(SemaphoreGuard { semaphore: self })
    }
}

/// Take a permit, waiting at most `timeout`.
pub fn try_acquire_timeout(&self, timeout: Duration) -> Option<SemaphoreGuard<'_>> {
    let mut state = self.state.lock().expect("semaphore mutex poisoned");
    if state.available > 0 {
        state.available -= 1;
        return Some(SemaphoreGuard { semaphore: self });
    }

    let (mut state, _timed_out) = self
        .released
        .wait_timeout(state, timeout)
        .expect("semaphore mutex poisoned");

    if state.available == 0 {
        None
    } else {
        state.available -= 1;
        Some(SemaphoreGuard { semaphore: self })
    }
}

```

```

    }
}

/// Permits currently free. Useful for metrics, not for control flow.
pub fn available_permits(&self) -> usize {
    self.state
        .lock()
        .expect("semaphore mutex poisoned")
        .available
}

fn release(&self) {
    let mut state = self.state.lock().expect("semaphore mutex poisoned");
    state.available += 1;
    self.released.notify_one();
}

/// Releases one permit when dropped.
pub struct SemaphoreGuard<'a> {
    semaphore: &'a Semaphore,
}

impl Drop for SemaphoreGuard<'_> {
    fn drop(&mut self) {
        self.semaphore.release();
    }
}

```

`release` adds a permit and wakes one sleeping thread with `notify_one`. It is private, and only the guard's `Drop` calls it. A caller therefore cannot release a permit it never took.

Two methods have problems:

- `acquire` calls `acquire_guard` and stores the guard in `_guard`. The guard is dropped when `acquire` returns, so the permit is released at once. A caller of `acquire` never actually holds a permit. `acquire_guard` is the method to use.
- `try_acquire_timeout` waits once. If it wakes early, from a spurious wakeup or because another thread took the permit first, it returns `None` before the timeout has passed. A correct version loops, and waits again for the time remaining. Exercise 4 asks you to write it.

Listing 16.15 The complete file, with tests. `src/problems/semaphore.rs`

```

///! Counting semaphore built from `Mutex` + `Condvar`.
///!
///! A semaphore bounds how many callers may proceed at once without protecting any
///! data. It is the primitive behind connection pools, per-account limits, and
///! bounded fan-out in a client library or command-line tool.
///!
///! The RAII guard returns the permit on drop, so a panicking worker cannot leak a
///! permit and slowly deadlock the pool.

use std::sync::{Condvar, Mutex};

```

```

use std::time::Duration;

struct State {
    available: usize,
}

/// A counting semaphore. Cloning is not provided; share one behind `Arc`.
pub struct Semaphore {
    state: Mutex<State>,
    released: Condvar,
}

impl Semaphore {
    /// Create a semaphore with `permits` permits available.
    pub fn new(permits: usize) -> Self {
        Self {
            state: Mutex::new(State { available: permits }),
            released: Condvar::new(),
        }
    }

    /// Block until a permit is available.
    pub fn acquire(&self) {
        let _guard = self.acquire_guard();
    }

    /// Block until a permit is available, returning a guard that releases it.
    pub fn acquire_guard(&self) -> SemaphoreGuard<'_> {
        let mut state = self.state.lock().expect("semaphore mutex poisoned");
        while state.available == 0 {
            state = self.released.wait(state).expect("semaphore mutex poisoned");
        }
        state.available -= 1;
        SemaphoreGuard { semaphore: self }
    }

    /// Take a permit only if one is free right now.
    pub fn try_acquire(&self) -> Option<SemaphoreGuard<'_>> {
        let mut state = self.state.lock().expect("semaphore mutex poisoned");
        if state.available == 0 {
            None
        } else {
            state.available -= 1;
            Some(SemaphoreGuard { semaphore: self })
        }
    }

    /// Take a permit, waiting at most `timeout`.
    pub fn try_acquire_timeout(&self, timeout: Duration) -> Option<SemaphoreGuard<'_>> {
        let mut state = self.state.lock().expect("semaphore mutex poisoned");
        if state.available > 0 {
            state.available -= 1;
            return Some(SemaphoreGuard { semaphore: self });
        }

        let (mut state, _timed_out) = self
            .released
            .wait_timeout(state, timeout)
            .expect("semaphore mutex poisoned");
    }
}

```



```

        if state.available == 0 {
            None
        } else {
            state.available -= 1;
            Some(SemaphoreGuard { semaphore: self })
        }
    }

    /// Permits currently free. Useful for metrics, not for control flow.
    pub fn available_permits(&self) -> usize {
        self.state
            .lock()
            .expect("semaphore mutex poisoned")
            .available
    }

    fn release(&self) {
        let mut state = self.state.lock().expect("semaphore mutex poisoned");
        state.available += 1;
        self.released.notify_one();
    }
}

/// Releases one permit when dropped.
pub struct SemaphoreGuard<'a> {
    semaphore: &'a Semaphore,
}

impl Drop for SemaphoreGuard<'_> {
    fn drop(&mut self) {
        self.semaphore.release();
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::Arc;
    use std::sync::atomic::{AtomicUsize, Ordering};
    use std::thread;

    #[test]
    fn never_exceeds_the_permit_count() {
        let permits = 4;
        let semaphore = Arc::new(Semaphore::new(permits));
        let active = Arc::new(AtomicUsize::new(0));
        let peak = Arc::new(AtomicUsize::new(0));

        let mut handles = Vec::new();
        for _ in 0..32 {
            let semaphore = Arc::clone(&semaphore);
            let active = Arc::clone(&active);
            let peak = Arc::clone(&peak);
            handles.push(thread::spawn(move || {
                let _guard = semaphore.acquire_guard();
                let now = active.fetch_add(1, Ordering::SeqCst) + 1;
                peak.fetch_max(now, Ordering::SeqCst);
                thread::sleep(Duration::from_millis(1));
            }));
        }
    }
}

```

```

        active.fetch_sub(1, Ordering::SeqCst);
    }));
}
for handle in handles {
    handle.join().expect("worker panicked");
}

assert!(peak.load(Ordering::SeqCst) <= permits);
assert_eq!(semaphore.available_permits(), permits);
}

#[test]
fn guard_returns_the_permit_on_drop() {
    let semaphore = Semaphore::new(1);
    {
        let _guard = semaphore.acquire_guard();
        assert_eq!(semaphore.available_permits(), 0);
    }
    assert_eq!(semaphore.available_permits(), 1);
}

#[test]
fn try_acquire_reports_contention() {
    let semaphore = Semaphore::new(1);
    let first = semaphore.try_acquire().expect("first permit is free");
    assert!(semaphore.try_acquire().is_none());
    drop(first);
    assert!(semaphore.try_acquire().is_some());
}

#[test]
fn timeout_gives_up_when_no_permit_arrives() {
    let semaphore = Semaphore::new(0);
    assert!(
        semaphore
            .try_acquire_timeout(Duration::from_millis(5))
            .is_none()
    );
}
}

```

The test `never_exceeds_the_permit_count` starts 32 threads on a semaphore with 4 permits. Each thread counts itself in `active` while it holds a permit, and records the highest count ever seen in `peak` with `fetch_max`. The peak must not exceed 4.

```

$ cargo test --lib problems::semaphore
running 4 tests
test problems::semaphore::tests::guard_returns_the_permit_on_drop ... ok
test problems::semaphore::tests::try_acquire_reports_contention ... ok
test problems::semaphore::tests::timeout_gives_up_when_no_permit_arrives ... ok
test problems::semaphore::tests::never_exceeds_the_permit_count ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 133 filtered out

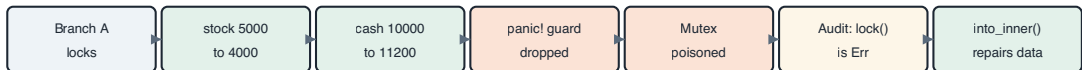
```

16.6 When a lock holder panics

A lock protects data that must stay consistent. A bank transfer, for example, takes money from one account and adds it to another. Between those two steps, the data is briefly inconsistent. The lock hides that moment from other threads.

What if the thread panics between the two steps? The guard is dropped during the panic, so the lock is released. The data, though, stays half-changed. The standard `Mutex` records that this happened, and the mutex becomes **poisoned** (figure 16.6). Every later `lock()` returns `Err(PoisonError)` instead of `Ok(guard)`.

A panic while holding a `MutexGuard` poisons the `Mutex`



The transaction stopped halfway: stock went down and cash went up, but it was never marked settled.

Poisoning makes every later `lock()` return `Err(PoisonError)`, so no thread reads the half-done state by accident.

`PoisonError::into_inner()` hands over the guard anyway, for code that knows how to repair the data.

Repairing the data does not clear the flag: later `lock()` calls still return `Err`.

Figure 16.6 The sequence in `mutex_poisoning.rs`.

The error does not hide the data. `PoisonError::into_inner()` returns the guard anyway, for code that can check and repair the data.

Listing 16.16 The recovery function (lines 52 to 80). `src/bin/mutex_poisoning.rs`

```

/// Recovers from a poisoned mutex by taking ownership of the guard that the
/// panicked thread left behind.
///
/// `PoisonError::into_inner()` gives us the `MutexGuard`, which means we still
/// hold the lock while repairing state. The guard is dropped when this function
/// returns.
fn recover_ledger(poisoned: PoisonError<MutexGuard<'_, Ledger>>) -> Ledger {
    // into_inner() returns the MutexGuard. We own the lock exclusively even
    // though the previous holder panicked.
    let mut guard = poisoned.into_inner();

    println!("[Reconciliation] Physical audit reveals a cash discrepancy.");
    println!(
        "[Reconciliation] Digital cash: {} | Corrected cash: {}",
        guard.cash_on_hand,
        guard.cash_on_hand - 200
    );
}

```

```

    // Correct the data while we still hold the guard.
    guard.cash_on_hand -= 200;
    guard.is_settled = true;
    guard.transaction_id += 1;

    println!("[Reconciliation] Ledger corrected. Transaction marked failed and closed.");

    // Clone the inner data to return an owned value. The guard drops here and
    // releases the lock.
    guard.clone()
}

```

`recover_ledger` receives the `PoisonError` and takes the guard out of it. It holds the lock while it corrects the ledger. It returns a clone of the corrected data, and the guard is dropped at the end of the function.

Listing 16.17 The complete program. `src/bin/mutex_poisoning.rs`

```

// =====
// Mutex poisoning: failed transaction and audit recovery
//
// Ported from the Rust playground repo (`scam_transaction.rs`).
//
// Story:
//   A branch executes a financial transaction that mutates shared state under
//   a `Mutex`. Mid-transaction the thread panics. In Rust, a `Mutex` whose
//   guard was held during a panic becomes **poisoned**. Future `lock()` calls
//   return `Err(PoisonError)` instead of silently letting another thread
//   observe an inconsistent partial update.
//
// Where this appears in practice:
//   - Controllers/reconcilers mutate shared state; a panic while holding a lock
//     is a real failure mode in worker pools, caches, and rate limiters.
//   - Rust chooses fail-fast over data corruption: a poisoned mutex tells
//     callers that the protected invariant may be broken. The caller must
//     decide whether to abort, recover, or reset state.
//   - `PoisonError::into_inner()` returns the `MutexGuard`, so a recovery path
//     can inspect/correct the last known state without waiting for the lock to
//     be released by the panicked thread.
//
// Run:
//   cargo run --bin mutex_poisoning
// =====

use std::error::Error;
use std::sync::{Arc, Mutex, MutexGuard, PoisonError};
use std::thread;

/// Shared ledger state. In a real system this would be behind a transactional
/// store; here it is plain in-memory state to illustrate lock poisoning.
#[derive(Debug, Clone)]
struct Ledger {
    cash_on_hand: i64,
    stock_value: i64,
    transaction_id: u32,
}

```

```

    is_settled: bool,
}

impl Ledger {
    fn new() -> Self {
        Ledger {
            cash_on_hand: 10_000,
            stock_value: 5_000,
            transaction_id: 0,
            is_settled: false,
        }
    }
}

/// Recovers from a poisoned mutex by taking ownership of the guard that the
/// panicked thread left behind.
///
/// `PoisonError::into_inner()` gives us the `MutexGuard`, which means we still
/// hold the lock while repairing state. The guard is dropped when this function
/// returns.
fn recover_ledger(poisoned: PoisonError<MutexGuard<'_, Ledger>>) -> Ledger {
    // into_inner() returns the MutexGuard. We own the lock exclusively even
    // though the previous holder panicked.
    let mut guard = poisoned.into_inner();

    println!("[Reconciliation] Physical audit reveals a cash discrepancy.");
    println!(
        "[Reconciliation] Digital cash: {} | Corrected cash: {}",
        guard.cash_on_hand,
        guard.cash_on_hand - 200
    );

    // Correct the data while we still hold the guard.
    guard.cash_on_hand -= 200;
    guard.is_settled = true;
    guard.transaction_id += 1;

    println!("[Reconciliation] Ledger corrected. Transaction marked failed and closed.");

    // Clone the inner data to return an owned value. The guard drops here and
    // releases the lock.
    guard.clone()
}

fn main() -> Result<(), Box<dyn Error>> {
    let ledger = Arc::new(Mutex::new(Ledger::new()));

    // Thread A: the faulty transaction. It mutates one side of the ledger and
    // then panics before marking the transaction settled.
    let ledger_a = Arc::clone(&ledger);
    let handle_a = thread::spawn(move || {
        // SAFETY-free normal code: if the mutex is already poisoned, unwrap()
        // panics; in a controlled demo this is the first lock so it is fine.
        let mut guard = ledger_a.lock().unwrap();

        println!(
            "[Branch A] Transaction {} commenced.",
            guard.transaction_id + 1
        );
    });
}

```

```

        guard.stock_value -= 1_000;
        println!(
            "[Branch A] Stock deducted. New value: {}",
            guard.stock_value
        );
        guard.cash_on_hand += 1_200;
        println!(
            "[Branch A] Cash updated. New balance: {}",
            guard.cash_on_hand
        );

        // Panic while `guard` is still alive -> the Mutex becomes poisoned.
        panic!("[Branch A] Settlement failure: arithmetic overflow in reconciliation.");
    });

    // A panic in a spawned thread does not abort the process unless the panic
    // strategy is set to abort. `join()` returns Err, which is expected here.
    let _ = handle_a.join();

    // Thread B: the forensic audit. It sees Err(PoisonError), inspects the
    // recovered state, and repairs the ledger.
    let ledger_b = Arc::clone(&ledger);
    let handle_b = thread::spawn(move || {
        println!("\n[Audit] Attempting to access the ledger...");

        match ledger_b.lock() {
            Ok(guard) => {
                println!("[Audit] Ledger is not poisoned: {:?}", *guard);
            }
            Err(poisoned) => {
                println!("[Audit] Mutex is poisoned. Recovered state before correction:");
                println!("[Audit] {:?}", *poisoned.get_ref());

                let corrected = recover_ledger(poisoned);
                println!("[Audit] Corrected ledger: {:?}", corrected);
            }
        }
    });

    handle_b.join().unwrap();

    // Important: recovering the data does NOT clear the poison bit on the
    // Mutex itself. Subsequent lock() calls still return Err(PoisonError) so
    // callers can decide whether the repaired state is trustworthy.
    // Uncommenting the next lines would panic:
    //
    // let _guard = ledger.lock().unwrap();

    Ok(())
}

// =====
// Tests (cargo test --bin mutex_poisoning)
// =====
#[cfg(test)]
mod tests {
    use super::*;

    #[test]

```

```

fn ledger_starts_in_expected_state() {
    let ledger = Ledger::new();
    assert_eq!(ledger.cash_on_hand, 10_000);
    assert_eq!(ledger.stock_value, 5_000);
    assert!(!ledger.is_settled);
}

#[test]
fn recovery_repairs_a_poisoned_ledger() {
    let ledger = Arc::new(Mutex::new(Ledger::new()));

    // Poison the mutex from a helper thread, exactly like the demo.
    let ledger_a = Arc::clone(&ledger);
    let handle = thread::spawn(move || {
        let _guard = ledger_a.lock().unwrap();
        panic!("boom");
    });
    let _ = handle.join();

    let lock_result = ledger.lock();
    assert!(lock_result.is_err(), "mutex should be poisoned");

    let recovered = recover_ledger(lock_result.unwrap_err());
    assert!(recovered.is_settled);
    assert_eq!(recovered.transaction_id, 1);
    assert_eq!(recovered.cash_on_hand, 10_000 - 200);
}

```

In `main`, thread A changes the stock and the cash and then panics before marking the transaction settled. The panic does not stop the process. It ends only thread A, and `handle_a.join()` returns an `Err`, which `main` ignores on purpose with `let _ =`.

Thread B then locks the ledger. `match` on the result separates the two cases. `poisoned.get_ref()` shows the data without taking the guard, and `recover_ledger` repairs it.

```

$ cargo run --bin mutex_poisoning
[Branch A] Transaction 1 commenced.
[Branch A] Stock deducted. New value: 4000
[Branch A] Cash updated. New balance: 11200

thread '<unnamed>' (5795284) panicked at src/bin/mutex_poisoning.rs:109:9:
[Branch A] Settlement failure: arithmetic overflow in reconciliation.
note: run with 'RUST_BACKTRACE=1' environment variable to display a backtrace

[Audit] Attempting to access the ledger...
[Audit] Mutex is poisoned. Recovered state before correction:
[Audit] Ledger { cash_on_hand: 11200, stock_value: 4000, transaction_id: 0, is_settled:
false }
[Reconciliation] Physical audit reveals a cash discrepancy.
[Reconciliation] Digital cash: 11200 | Corrected cash: 11000
[Reconciliation] Ledger corrected. Transaction marked failed and closed.

```

```
[Audit] Corrected ledger: Ledger { cash_on_hand: 11000, stock_value: 4000, transaction_id: 1, is_settled: true }
```

The panic message goes to standard error, so in a terminal it can appear between the other lines. The number in parentheses is the thread's ID and changes on every run.

Recovering the data does not clear the poison flag. A later `lock()` on the same mutex still returns `Err`, as the comment at the end of `main` notes. Rust 1.77 added `Mutex::clear_poison` for code that has repaired the data and wants to say so.

16.7 Deadlock

A **deadlock** happens when threads wait for each other in a circle, so none can continue. The classic case needs two locks. Thread 1 holds lock A and waits for lock B. Thread 2 holds lock B and waits for lock A. Neither will ever release what it holds.

To analyze a deadlock, draw a **wait-for graph**. Each thread is a node. An edge from T0 to T1 means T0 is waiting for a lock that T1 holds. The threads are deadlocked exactly when the graph has a cycle (figure 16.7). Finding a cycle is the three-state DFS from section 12.6.

Wait-for graphs: an edge from T0 to T1 means T0 waits for a lock that T1 holds



Figure 16.7 The two graphs from the program. Only the right one is deadlocked.

Listing 16.18 The complete program. `src/bin/concurrency_deadlock.rs`

```
//! Deadlock detection: a deadlock exists exactly when the wait-for graph has a
//! cycle. Each edge `(waiter, holder)` means the first task is blocked on the
//! second.
//!
//! Run with: cargo run --bin concurrency_deadlock

/// Iterative-friendly depth-first search over a small adjacency list.
fn has_deadlock(waits_for: &[(usize, usize)]) -> bool {
```



```

    let Some(highest) = waits_for.iter().flat_map(|(a, b)| [*a, *b]).max() else {
        return false;
    };

    let mut edges = vec![Vec::new(); highest + 1];
    for (waiter, holder) in waits_for {
        edges[*waiter].push(*holder);
    }

    // 1 = on the current path, 2 = fully explored.
    fn visit(node: usize, edges: &[Vec<usize>], state: &mut [u8]) -> bool {
        match state[node] {
            1 => return true,
            2 => return false,
            _ => state[node] = 1,
        }
        for &next in &edges[node] {
            if visit(next, edges, state) {
                return true;
            }
        }
        state[node] = 2;
        false
    }

    let mut state = vec![0u8; edges.len()];
    (0..edges.len()).any(|node| visit(node, &edges, &mut state))
}

fn main() {
    let acyclic = [(0, 1), (1, 2), (0, 2)];
    let cycle = [(0, 1), (1, 2), (2, 0)];
    let self_wait = [(0, 0)];

    for (label, graph) in [
        ("0->1, 1->2, 0->2", &acyclic[..]),
        ("0->1, 1->2, 2->0", &cycle[..]),
        ("0->0", &self_wait[..]),
    ] {
        println!("{label:<20} deadlock: {}", has_deadlock(graph));
    }

    // The four Coffman conditions must all hold for a deadlock; removing any one
    // of them prevents it.
    println!("\nCoffman conditions:");
    for condition in [
        "mutual exclusion",
        "hold and wait",
        "no preemption",
        "circular wait",
    ] {
        println!(" - {condition}");
    }

    assert!(!has_deadlock(&acyclic));
    assert!(has_deadlock(&cycle));
    assert!(has_deadlock(&self_wait));
    assert!(!has_deadlock(&[]));
}

```

```
println!("\nall checks passed");
}
```

`has_deadlock` takes the edges as `(waiter, holder)` pairs. It finds the largest thread number with a `let ... else`, so an empty list returns `false` at once. It then builds an adjacency list and runs the DFS.

The states are plain numbers here: 0 for unvisited, 1 for on the current path, and 2 for fully explored. An edge back to a node in state 1 is a cycle. The third test graph, `(0, 0)`, is a thread waiting for itself, as happens when a thread tries to lock a `Mutex` it already holds.

```
$ cargo run --bin concurrency_deadlock
0->1, 1->2, 0->2    deadlock: false
0->1, 1->2, 2->0    deadlock: true
0->0                deadlock: true

Coffman conditions:
- mutual exclusion
- hold and wait
- no preemption
- circular wait

all checks passed
```

The program ends with the four **Coffman conditions**. A deadlock needs all four:

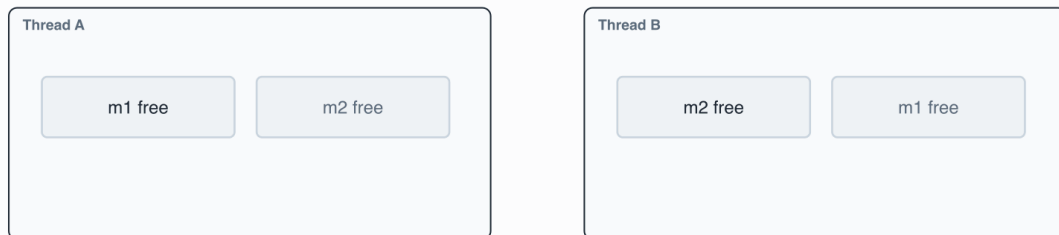
1. **Mutual exclusion:** a lock can be held by only one thread.
2. **Hold and wait:** a thread holds one lock while waiting for another.
3. **No preemption:** a lock cannot be taken away from its holder.
4. **Circular wait:** the waiting forms a cycle.

Removing any one prevents deadlock. The most practical rule removes the fourth: give every lock a fixed order, and always take locks in that order. If every thread takes lock A before lock B, no thread can hold B while waiting for A, and no cycle can form.

Deadlock: two threads each hold one lock and want the other

A waits-for cycle is the signature. Breaking the order breaks the cycle.

■ held ■ wanted ■ free



Two threads, two mutexes. Both mutexes start free.

Both locks are healthy. It is the order of the two requests that jams the pair.

m1 = **free** m2 = **free** waiting = -

Animation 16.2 The circular wait being built, one lock at a time. Each thread's panel shows what it holds and what it wants. Nothing here is a bug in either lock: each acquisition is individually correct. The deadlock comes only from the order, which is why the fix is a rule about order and not about the locks. If both threads took m1 before m2, the last two frames could not happen.

Summary

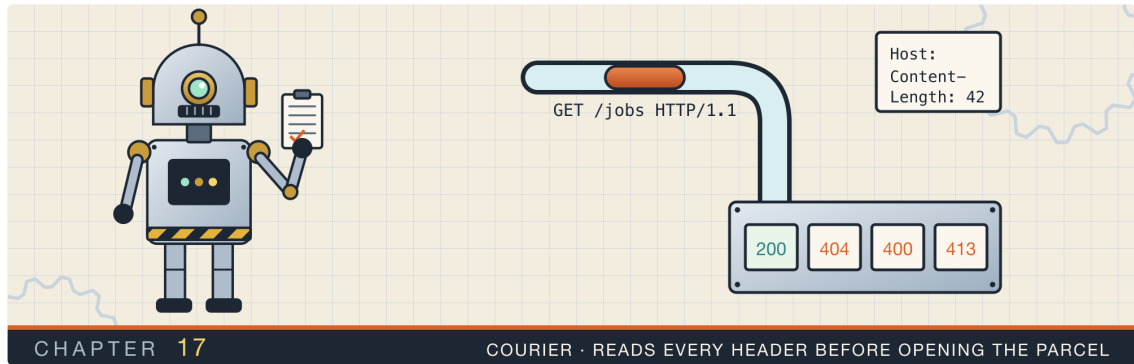
- `thread::spawn` needs closures that own their data. `thread::scope` lets threads borrow local data, and waits for them.
- Collect the join handles before joining, or lazy iterators run the threads one at a time.
- `Send` and `Sync` are auto traits. `Rc` and `RefCell` lack them, so the compiler rejects sharing them across threads.
- An atomic performs read, change, and write as one step. `Relaxed` suffices for a standalone counter. `Release` and `Acquire` pair up to pass other data between threads.

- Eight threads summed a large array about 3 to 4 times faster, limited by memory bandwidth and four physical cores. Amdahl's law caps speedup at $1 / s$ for a serial fraction s .
- A spin lock is an atomic flag, a compare-exchange loop, and a guard that releases on drop. Its `unsafe impl Sync` is a promise the code must keep.
- A semaphore built from a `Mutex` and a `Condvar` waits in a `while` loop, because wakeups can be spurious.
- A panic while holding a `MutexGuard` poisons the mutex, and `into_inner` gives the data to recovery code.
- Threads are deadlocked when their wait-for graph has a cycle. A global lock order prevents it.

Chapter 17 uses these tools to build queues that pass work between threads, and makes producers wait when the queue is full.

Exercises

1. Write a version of `parallel_sum` from listing 16.1 as a single lazy chain, `.map(spawn).map(join).sum()`, and time both on a large slice. Explain the difference.
2. Make the three pairs of functions in `parallel_sum.rs` one generic function over `T`: `Sum<T> + Copy + Send + Sync`. Which extra bound does `sum::<T>()` over `&T` items need?
3. Close the gap in `SpinGuard`. Add a field `_marker: PhantomData<&'a mut T>`, and explain why this makes the guard `Sync` only when `T: Sync`.
4. Rewrite `try_acquire_timeout` to loop until a permit is free or the whole timeout has passed. Use `Instant::now()` to compute the time left for each wait.
5. Write a program that deadlocks with two `Mutex` values and two threads. Then fix it by taking the locks in a fixed order.



Queues and bounded buffers

THIS CHAPTER COVERS

- Passing work between threads through a shared queue
- Putting a waiting thread to sleep with a Condvar , and waking it again
- A bounded buffer, where producers wait when the queue is full
- Three ways to handle a poisoned lock inside a queue
- Closing a queue so that every waiting thread wakes up and every loop ends
- A ring buffer for one producer and one consumer, with no lock at all

Chapter 16 split one job across threads. Many programs have a different shape. Some threads produce work: they read requests, parse files, or receive messages. Other threads consume it: they process each item. The two groups run at different speeds, so they need a place to leave work for each other.

That place is a **queue**. Producers add items at the back, and consumers take them from the front. Items come out in the order they went in, which is called **FIFO**, for first in, first out.

This chapter builds the queue six times. Each version fixes a limitation of the one before, and the last one removes the lock.

17.1 Why a queue needs a limit

Suppose producers add items faster than consumers can handle them. If the queue can grow without limit, it grows until the machine runs out of memory, and the program is killed (figure 17.1).

A fast producer and a slow consumer

unbounded: the queue grows until memory runs out



bounded (capacity 3): a full queue makes the producer wait



Making the producer wait is called backpressure. The system slows down instead of failing.

Figure 17.1 An unbounded queue grows without end. A bounded queue makes the producer wait.

A **bounded** queue has a fixed capacity. When it is full, `push` waits until a consumer makes room. The producer slows down to the consumer's speed. Slowing down the source of work when the rest of the system cannot keep up is called **backpressure**. A slow system is better than a crashed one.

A bounded queue with blocking `push` and `pop` is also called a **bounded buffer**.

17.2 A shared queue

The first version is unbounded. It shows the two tools every version uses: a `Mutex` around a `VecDeque`, and a `Condvar` for waiting.

`VecDeque` from chapter 3 adds at the back and removes from the front in $O(1)$. The `Mutex` makes sure one thread at a time touches it.

The `Condvar` is for consumers that find the queue empty. Section 16.5 introduced condition variables. A thread waits on one while holding a mutex, and the wait releases the mutex and sleeps in one step. When another thread calls `notify_one`, one sleeping thread wakes up with the mutex locked again.

Listing 17.1 The queue (lines 7 to 31). src/bin/thread_safe_queue.rs

```

#[derive(Default)]
struct SyncQueue {
    items: Mutex<VecDeque<i32>>,
    not_empty: Condvar,
}

impl SyncQueue {
    fn push(&self, item: i32) {
        if let Ok(mut q) = self.items.lock() {
            q.push_back(item);
            self.not_empty.notify_one();
        }
    }

    fn pop(&self) -> Option<i32> {
        let q = self.items.lock().ok()?;
        let mut q = self
            .not_empty
            .wait_while(q, |q| q.is_empty())
            .ok()?;
        q.pop_front()
    }
}

```

`push` locks the queue, adds the item, and wakes one waiting consumer. `if let Ok(mut q)` skips the push if the lock is poisoned, so in that case the item is silently lost. Section 17.4 shows better ways to handle that.

`pop` uses `wait_while`. It takes the guard and a condition, and waits for as long as the condition is true:

```

let mut q = self.not_empty.wait_while(q, |q| q.is_empty()).ok()?;

```

`wait_while` contains the `while` loop that section 16.5 wrote by hand. It checks the condition, waits if it is true, and checks again after each wakeup. It returns the guard when the queue is not empty. The `.ok()?` turns a poisoned lock into `None`.

Listing 17.2 The complete program. src/bin/thread_safe_queue.rs

```

use std::{
    collections::VecDeque,
    sync::{Arc, Condvar, Mutex},
    thread
};

#[derive(Default)]

```

```

struct SyncQueue {
    items: Mutex<VecDeque<i32>>,
    not_empty: Condvar,
}

impl SyncQueue {

    fn push(&self, item: i32) {
        if let Ok(mut q) = self.items.lock() {
            q.push_back(item);
            self.not_empty.notify_one();
        }
    }

    fn pop(&self) -> Option<i32> {
        let q = self.items.lock().ok()?;
        let mut q = self
            .not_empty
            .wait_while(q, |q| q.is_empty())
            .ok()?;
        q.pop_front()
    }
}

fn main() {
    let q = Arc::new(SyncQueue::default());

    // producer
    let producer = {
        let q = Arc::clone(&q);
        thread::spawn(move || {
            for i in 0..5 {
                q.push(i);
                println!("Producer pushed {i}");
            }
        })
    };

    let consumer = {
        let q = Arc::clone(&q);
        thread::spawn(move || {
            for _ in 0..5 {
                println!("Consumer popped: {:?}", q.pop());
            }
        })
    };

    producer.join().unwrap();
    consumer.join().unwrap();
}

```

`main` shares the queue through an `Arc`. Each thread is created inside a block that makes its own clone of the `Arc` first:


```
let producer = {
    let q = Arc::clone(&q);
    thread::spawn(move || { ... })
};
```

The block keeps the clone's name short, and the outer `q` stays available for the next thread.

```
$ cargo run --bin thread_safe_queue
Producer pushed 0
Producer pushed 1
Producer pushed 2
Producer pushed 3
Producer pushed 4
Consumer popped: Some(0)
Consumer popped: Some(1)
Consumer popped: Some(2)
Consumer popped: Some(3)
Consumer popped: Some(4)
```

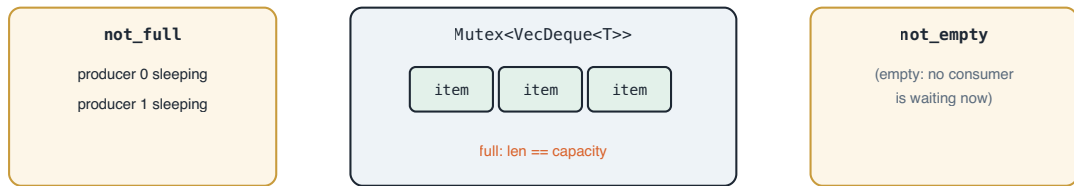
In this run the producer finished before the consumer started. On another run the lines can interleave. The values always come out in the order 0 to 4, because the queue is FIFO.

17.3 A bounded buffer

The bounded version adds a capacity, and a second reason to wait: a producer must wait while the queue is full. So it has two condition variables, one for each kind of waiting thread (figure 17.2).

- `not_empty` is where consumers wait. A producer notifies it after adding an item.
- `not_full` is where producers wait. A consumer notifies it after removing an item.

One Mutex, two Condvars: producers wait in `not_full`, consumers wait in `not_empty`



`pop()` takes an item, then calls `not_full.notify_one()`: one slot opened, so one producer wakes.

`push()` adds an item, then calls `not_empty.notify_one()`: one item arrived, so one consumer wakes.

A woken thread relocks the mutex and checks its condition again before it continues.

Figure 17.2 Two condition variables share one mutex. Each wakes only the kind of thread that can make progress.

With a single condition variable and `notify_one`, a producer could wake another producer instead of a consumer. The woken producer would find the queue still full and sleep again, while the consumer that could make progress never woke. Every thread could end up asleep. `notify_all` on one condition variable avoids that, but wakes every thread for each change. Two condition variables wake the right kind of thread directly.

Listing 17.3 The type and `new` (lines 19 to 38). `src/bin/bounded_buffer.rs`

```
struct BoundedQueue<T> {
    inner: Mutex<VecDeque<T>>,
    capacity: usize,
    not_empty: Condvar,
    not_full: Condvar,
}

fn new(capacity: usize) -> Self {
    // Capacity 0 makes no sense: no producer could ever push,
    // and every pop would block forever.
    assert!(capacity > 0, "capacity must be > 0");

    Self {
        inner: Mutex::new(VecDeque::with_capacity(capacity)),
        capacity,
        not_empty: Condvar::new(),
        not_full: Condvar::new(),
    }
}
```

`VecDeque::with_capacity(capacity)` allocates room for every item at the start, so the deque never grows later. A capacity of 0 is rejected, because nothing could ever be pushed.

Listing 17.4 `push` (lines 40 to 74).

```

fn push(&self, item: T) {
    // Acquire the shelf lock. `guard` is our exclusive handle to the deque.
    let mut guard = self.inner.lock().unwrap();

    // Wait while the shelf is full.
    //
    // Why `while` and not `if`:
    // 1. Condvars are allowed to wake up for no reason (spurious wakeup).
    // 2. Even on a real wakeup, another producer may have raced us and
    //    taken the free slot before we reacquired the lock.
    // So we must re-check the condition every time we wake.
    //
    // Why `guard = ...wait(guard)`?
    // `wait` consumes the guard (it releases the lock on our behalf),
    // puts us to sleep, then re-locks and hands us a NEW guard.
    // The reassignment just means "here's my fresh key to the shelf."
    // The VecDeque inside is the same one – we're not replacing the queue.
    while guard.len() == self.capacity {
        guard = self.not_full.wait(guard).unwrap();
    }

    // We have room. Add the item to the back of the deque.
    guard.push_back(item);

    // Release the lock BEFORE ringing the bell.
    // This is an optimization, not a correctness requirement.
    // If we notify while still holding the lock, the thread we wake will
    // immediately block on `lock()` until we release it – a wasted wakeup.
    drop(guard);

    // Ring the "not empty" bell: one consumer can now make progress.
    // `notify_one` (not `notify_all`) because exactly ONE item was added,
    // so only ONE consumer can succeed. Waking all of them = stampede.
    self.not_empty.notify_one();
}

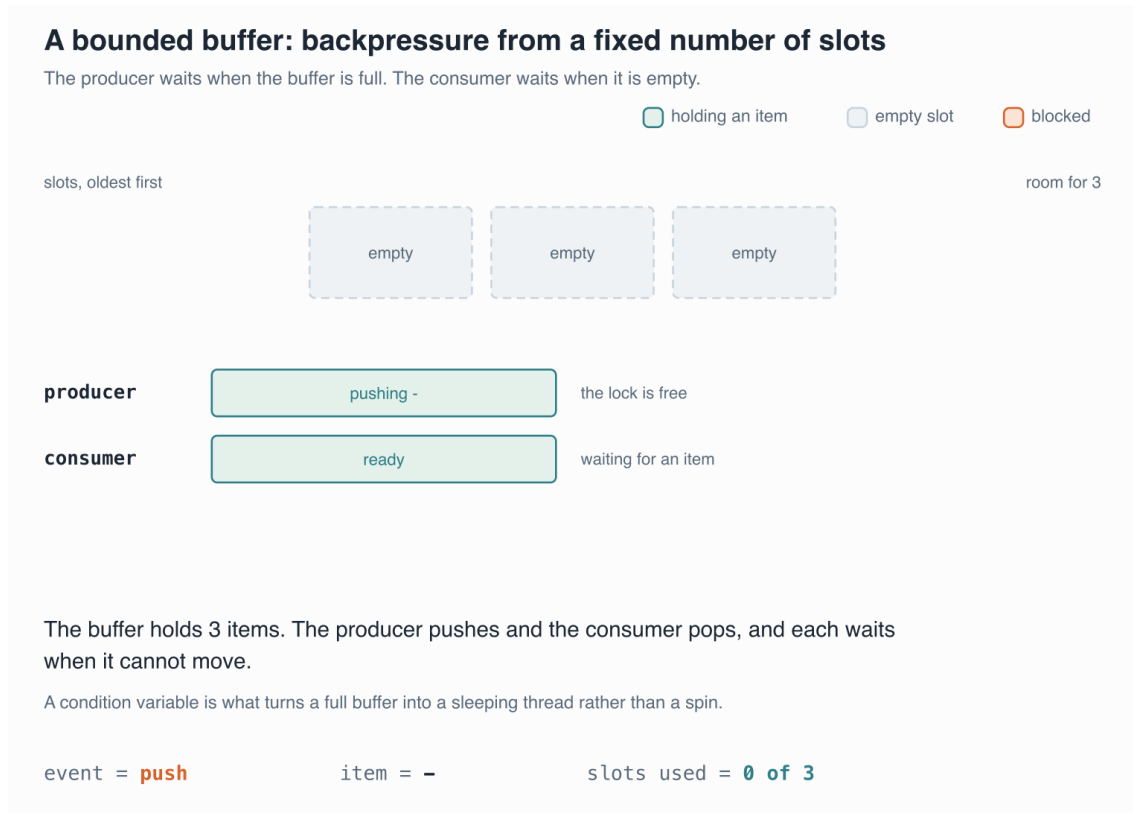
```

The comments in `push` explain each decision. Figure 17.3 puts the steps of a blocked `push` in order.

What happens inside a blocked push

- | | | |
|---|--|--|
| 1 | lock the mutex | producer holds the guard |
| 2 | len == capacity: call not_full.wait(guard) | wait unlocks the mutex and sleeps, in one step |
| 3 | a consumer locks, pops, unlocks | then calls not_full.notify_one() |
| 4 | wait returns a new guard | the producer holds the mutex again |
| 5 | while loop checks len again | another producer may have taken the slot |
| 6 | push_back, unlock, notify not_empty | |

Figure 17.3 A producer that finds the queue full.



Animation 17.1 A producer and a consumer sharing three slots. The buffer row shows what is stored, and the two lanes show what each thread is doing right now. The producer is blocked exactly once, on the frame where the buffer is full, and the consumer's pop is what wakes it. That is backpressure: a fast producer cannot outrun a slow consumer, because the buffer's capacity is the limit.

The line `guard = self.not_full.wait(guard).unwrap();` looks odd at first. `wait` takes the guard by value, because it must release the mutex. When the thread wakes, `wait` locks the mutex again and returns a new guard. The assignment stores that new guard in the same variable.

After the push, the code drops the guard before calling `notify_one`. If it notified first, the woken consumer would try to lock a mutex that is still held, and wait again at once. Unlocking first avoids that extra wait. The program would be correct either way.

Listing 17.5 `pop` (lines 76 to 100).

```
fn pop(&self) -> T {
    // Acquire the shelf lock.
    let mut guard = self.inner.lock().unwrap();

    // Wait while the shelf is empty.
    // Same `while` reasoning as push: spurious wakeups + races with other
    // consumers who may have grabbed the item before we reacquired the lock.
    while guard.is_empty() {
        guard = self.not_empty.wait(guard).unwrap();
    }

    // Safe to unwrap: the `while` loop guarantees the deque is non-empty,
    // and we hold the lock so nobody can change that between the check
    // and this line.
    let item = guard.pop_front().unwrap();

    // Release before notifying (same optimization as in push).
    drop(guard);

    // Ring the "not full" bell: one producer can now make progress.
    // Exactly one slot opened, so wake exactly one producer.
    self.not_full.notify_one();

    item
}
```

`pop` mirrors `push`. It waits on `not_empty` while the queue is empty, removes the front item, unlocks, and wakes one producer through `not_full`. The `unwrap()` on `pop_front()` cannot fail. The loop ended because the queue was not empty, and the thread has held the lock since that check.

Listing 17.6 The complete program. `src/bin/bounded_buffer.rs`

```
use std::collections::VecDeque;
use std::sync::{Arc, Condvar, Mutex};
use std::thread;
use std::time::Duration;
```

```
// A thread-safe bounded buffer (a "shelf" that holds at most `capacity` items).
//
// Three fields do all the work:
// - `inner`      : the actual VecDeque, wrapped in a Mutex so only one thread
//                  touches it at a time.
// - `not_empty`  : waiting room for CONSUMERS. A consumer sleeps here when the
//                  shelf is empty. Producers ring this bell after pushing.
// - `not_full`   : waiting room for PRODUCERS. A producer sleeps here when the
//                  shelf is full. Consumers ring this bell after popping.
//
// Two separate condvars so we always wake the RIGHT kind of thread.
// One shared condvar would cause livelock: consumers waking consumers,
// producers waking producers, everyone re-checking and going back to sleep.
struct BoundedQueue<T> {
    inner: Mutex<VecDeque<T>>,
    capacity: usize,
    not_empty: Condvar,
    not_full: Condvar,
}

impl<T> BoundedQueue<T> {
    fn new(capacity: usize) -> Self {
        // Capacity 0 makes no sense: no producer could ever push,
        // and every pop would block forever.
        assert!(capacity > 0, "capacity must be > 0");

        Self {
            inner: Mutex::new(VecDeque::with_capacity(capacity)),
            capacity,
            not_empty: Condvar::new(),
            not_full: Condvar::new(),
        }
    }

    fn push(&self, item: T) {
        // Acquire the shelf lock. `guard` is our exclusive handle to the deque.
        let mut guard = self.inner.lock().unwrap();

        // Wait while the shelf is full.
        //
        // Why `while` and not `if`:
        // 1. Condvars are allowed to wake up for no reason (spurious wakeup).
        // 2. Even on a real wakeup, another producer may have raced us and
        //    taken the free slot before we reacquired the lock.
        // So we must re-check the condition every time we wake.
        //
        // Why `guard = ...wait(guard)`?
        // `wait` consumes the guard (it releases the lock on our behalf),
        // puts us to sleep, then re-locks and hands us a NEW guard.
        // The reassignment just means "here's my fresh key to the shelf."
        // The VecDeque inside is the same one - we're not replacing the queue.
        while guard.len() == self.capacity {
            guard = self.not_full.wait(guard).unwrap();
        }

        // We have room. Add the item to the back of the deque.
        guard.push_back(item);

        // Release the lock BEFORE ringing the bell.
    }
}
```

```

    // This is an optimization, not a correctness requirement.
    // If we notify while still holding the lock, the thread we wake will
    // immediately block on `lock()` until we release it – a wasted wakeup.
    drop(guard);

    // Ring the "not empty" bell: one consumer can now make progress.
    // `notify_one` (not `notify_all`) because exactly ONE item was added,
    // so only ONE consumer can succeed. Waking all of them = stampede.
    self.not_empty.notify_one();
}

fn pop(&self) -> T {
    // Acquire the shelf lock.
    let mut guard = self.inner.lock().unwrap();

    // Wait while the shelf is empty.
    // Same `while` reasoning as push: spurious wakeups + races with other
    // consumers who may have grabbed the item before we reacquired the lock.
    while guard.is_empty() {
        guard = self.not_empty.wait(guard).unwrap();
    }

    // Safe to unwrap: the `while` loop guarantees the deque is non-empty,
    // and we hold the lock so nobody can change that between the check
    // and this line.
    let item = guard.pop_front().unwrap();

    // Release before notifying (same optimization as in push).
    drop(guard);

    // Ring the "not full" bell: one producer can now make progress.
    // Exactly one slot opened, so wake exactly one producer.
    self.not_full.notify_one();

    item
}

fn main() {
    // Small capacity (3) so that with two slow consumers you can actually
    // SEE producers block: the queue fills up and push() sleeps in `not_full`.
    let queue = Arc::new(BoundedQueue::new(3));

    let mut handles = vec![];

    // ---- Producers ----
    // Two producers, each pushes 5 items. `Arc::clone` bumps the refcount so
    // each thread shares the same queue; ownership of the clone moves in.
    for p in 0..2 {
        let q = Arc::clone(&queue);
        handles.push(thread::spawn(move || {
            for i in 0..5 {
                let item = format!("p{:}:item{:}", p, i);
                println!("[producer {}] pushing {}", p, item);
                q.push(item);
                // Slow down so we don't blast through the whole buffer instantly.
                thread::sleep(Duration::from_millis(30));
            }
        })));
    }
}

```

```

    }

    // ---- Consumers ----
    // Two consumers, each pops 5 items. Consumers are SLOWER than producers
    // here (60ms vs 30ms), so the queue fills up and you'll see producers
    // stall on `not_full.wait(...)`
    for c in 0..2 {
        let q = Arc::clone(&queue);
        handles.push(thread::spawn(move || {
            for _ in 0..5 {
                let item = q.pop();
                println!("[consumer {}] got {}", c, item);
                thread::sleep(Duration::from_millis(60));
            }
        })));
    }

    // Join everything. If our synchronization is wrong, this is where you'd
    // hang (a thread sleeping forever) or panic (unwrap on empty / on a
    // poisoned mutex after another thread panicked while holding it).
    for h in handles {
        h.join().unwrap();
    }
    println!("done");
}

```

`main` starts two producers and two consumers on a queue with room for 3 items. Producers sleep 30 ms after each push, and consumers sleep 60 ms after each pop, so the producers are faster.

```

$ cargo run --bin bounded_buffer
[producer 0] pushing p0:item0
[producer 1] pushing p1:item0
[consumer 1] got p0:item0
[consumer 0] got p1:item0
[producer 0] pushing p0:item1
[producer 1] pushing p1:item1
[producer 0] pushing p0:item2
[producer 1] pushing p1:item2
[consumer 1] got p0:item1
[consumer 0] got p1:item1
[producer 0] pushing p0:item3
[producer 1] pushing p1:item3
[consumer 1] got p0:item2
[consumer 0] got p1:item2
[producer 0] pushing p0:item4
[producer 1] pushing p1:item4
[consumer 1] got p1:item3
[consumer 0] got p0:item3
[consumer 1] got p0:item4
[consumer 0] got p1:item4
done

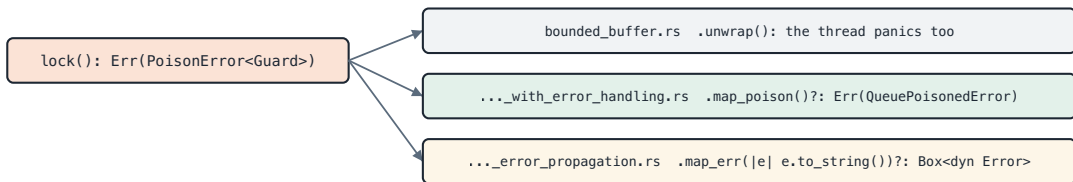
```


The order changes from run to run. Items from one producer always come out in that producer's order. The "pushing" line is printed before `push` is called, so the output does not show how long a push waited. Exercise 1 adds that.

17.4 When the lock is poisoned

`bounded_buffer.rs` calls `.unwrap()` on every `lock()` and `wait()`. If a thread panics while holding the lock, the mutex is poisoned, as section 16.6 showed. Every other thread's `unwrap()` then panics too. Two more versions return an error instead (figure 17.4).

Three ways the queues handle a poisoned mutex



The `PoisonError` holds a guard that borrows the mutex, so it cannot leave the function in a `Box<dyn Error>`. Both error-returning versions convert it into a value that owns nothing: a unit struct, or a `String`.

Figure 17.4 Three responses to a poisoned mutex.

17.4.1 A custom error type

Listing 17.7 The error and an extension trait (lines 7 to 26). `src/bin/bounded_buffer_with_error_handling.rs`

```

#[derive(Debug)]
pub struct QueuePoisonedError;

impl fmt::Display for QueuePoisonedError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "The queue's internal mutex was poisoned by a panicking thread")
    }
}

impl std::error::Error for QueuePoisonedError {}

trait PoisonMap<T> {
    fn map_poison(self) -> Result<T, QueuePoisonedError>;
}

impl<T> PoisonMap<T> for Result<T, PoisonError<T>> {
    fn map_poison(self) -> Result<T, QueuePoisonedError> {
        self.map_err(|_| QueuePoisonedError)
    }
}
  
```

```
    }
}
```

`QueuePoisonedError` is a unit struct: it carries no data. It implements `Display` for a readable message, and `std::error::Error` so it works with `?` and `Box<dyn Error>`.

`PoisonMap` adds a method to a type the code does not own. A trait defined in your crate can be implemented for any type, including the standard library's `Result`. A trait used this way is called an **extension trait**. `impl<T> PoisonMap<T> for Result<T, PoisonError<T>>` gives every `Result<T, PoisonError<T>>` a `map_poison` method, which replaces the `PoisonError` with `QueuePoisonedError`.

Listing 17.8 `push` and `pop` (lines 46 to 66).

```
fn push(&self, item: T) -> Result<(), QueuePoisonedError> {
    let mut guard = self.inner.lock().map_poison()?;
    while guard.len() == self.capacity {
        guard = self.not_full.wait(guard).map_poison()?;
    }
    guard.push_back(item);
    drop(guard);
    self.not_empty.notify_one();
    Ok(())
}

fn pop(&self) -> Result<T, QueuePoisonedError> {
    let mut guard = self.inner.lock().map_poison()?;
    while guard.is_empty() {
        guard = self.not_empty.wait(guard).map_poison()?;
    }
    let item = guard.pop_front().unwrap();
    drop(guard);
    self.not_full.notify_one();
    Ok(item)
}
```

Each `lock()` and `wait()` call ends in `.map_poison()?`. The logic is the same as listing 17.4, and each method now returns a `Result`.

Listing 17.9 The complete program. `src/bin/bounded_buffer_with_error_handling.rs`

```
use std::collections::VecDeque;
use std::fmt;
use std::sync::{Arc, Condvar, Mutex, PoisonError};
use std::thread;
use std::time::Duration;

#[derive(Debug)]
pub struct QueuePoisonedError;
```

```

impl fmt::Display for QueuePoisonedError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "The queue's internal mutex was poisoned by a panicking thread")
    }
}

impl std::error::Error for QueuePoisonedError {}

trait PoisonMap<T> {
    fn map_poison(self) -> Result<T, QueuePoisonedError>;
}

impl<T> PoisonMap<T> for Result<T, PoisonError<T>> {
    fn map_poison(self) -> Result<T, QueuePoisonedError> {
        self.map_err(|_| QueuePoisonedError)
    }
}

struct BoundedQueue<T> {
    inner: Mutex<VecDeque<T>>,
    capacity: usize,
    not_empty: Condvar,
    not_full: Condvar,
}

impl<T> BoundedQueue<T> {
    fn new(capacity: usize) -> Self {
        assert!(capacity > 0, "capacity must be > 0");
        Self {
            inner: Mutex::new(VecDeque::with_capacity(capacity)),
            capacity,
            not_empty: Condvar::new(),
            not_full: Condvar::new(),
        }
    }

    fn push(&self, item: T) -> Result<(), QueuePoisonedError> {
        let mut guard = self.inner.lock().map_poison()?;
        while guard.len() == self.capacity {
            guard = self.not_full.wait(guard).map_poison()?;
        }
        guard.push_back(item);
        drop(guard);
        self.not_empty.notify_one();
        Ok(())
    }

    fn pop(&self) -> Result<T, QueuePoisonedError> {
        let mut guard = self.inner.lock().map_poison()?;
        while guard.is_empty() {
            guard = self.not_empty.wait(guard).map_poison()?;
        }
        let item = guard.pop_front().unwrap();
        drop(guard);
        self.not_full.notify_one();
        Ok(item)
    }
}

```

```

fn main() {
    let queue = Arc::new(BoundedQueue::new(3));
    let mut handles = vec![];

    // ---- Producers ----
    for p in 0..2 {
        let q = Arc::clone(&queue);
        handles.push(thread::spawn(move || {
            for i in 0..5 {
                let item = format!("p{}:item{}", p, i);
                println!("[producer {}] pushing {}", p, item);

                // Uses match, avoids naming an error variable, uses dbg!
                match q.push(item) {
                    Ok(_) => {}
                    Err(err) => {
                        dbg!(err);
                        break;
                    }
                }
                thread::sleep(Duration::from_millis(30));
            }
        }));
    }

    // ---- Consumers ----
    for c in 0..2 {
        let q = Arc::clone(&queue);
        handles.push(thread::spawn(move || {
            for _ in 0..5 {
                // Uses match, avoids naming an error variable, uses dbg!
                match q.pop() {
                    Ok(item) => {
                        println!("[consumer {}] got {}", c, item);
                    }
                    Err(err) => {
                        dbg!(err);
                        break;
                    }
                }
                thread::sleep(Duration::from_millis(60));
            }
        }));
    }

    // Join everything.
    for h in handles {
        match h.join() {
            Ok(_) => {}
            Err(err) => {
                dbg!(err);
            }
        }
    }
    println!("done");
}

#[cfg(test)]
mod tests {

```

```

use super::*;

// 1. Mark the test as one that MUST panic to pass
// 2. Optional: use 'expected' to verify it panics with the correct error message
#[test]
#[should_panic(expected = "capacity must be > 0")]
fn test_zero_capacity_panics() {
    // This line invokes the assert!(capacity > 0) inside new()
    let _queue: BoundedQueue<i32> = BoundedQueue::new(0);
}

#[test]
fn test_queue_poisoning_on_thread_panic() {
    // 1. Create a queue shared via Arc so multiple threads can touch it
    let queue = Arc::new(BoundedQueue::new(5));
    let queue_clone = Arc::clone(&queue);

    // 2. Spawn a thread designed to crash while holding the internal lock
    let handle = thread::spawn(move || {
        // Manually lock the inner mutex so this thread owns the lock guard
        let _guard = queue_clone.inner.lock().unwrap();

        // Trigger a serious runtime panic while holding the lock guard!
        panic!("Intentional worker thread crash!");
    });

    // 3. Wait for the thread to die. It will return an Err because it panicked.
    let join_result = handle.join();
    assert!(join_result.is_err(), "The thread was supposed to panic");

    // 4. Now try to interact with the queue.
    // Because the thread panicked while holding the guard, the Mutex is poisoned.
    // Our push and pop methods should cleanly catch this and return our custom error.
    match queue.push("test_item") {
        Err(QueuePoisonedError) => {
            // Success! The queue correctly caught the poisoned state
            // instead of panicking the main thread.
        }
        Ok(_) => {
            panic!("Expected QueuePoisonedError, but push succeeded!");
        }
    }

    // Verify pop behaves the same way
    match queue.pop() {
        Err(QueuePoisonedError) => {}
        Ok(_) => panic!("Expected QueuePoisonedError, but pop succeeded!"),
    }
}

```

The threads in `main` handle an error by printing it with `dbg!` and leaving their loop with `break`.

The second test poisons the queue on purpose. It spawns a thread that locks the inner mutex and panics while holding it. A test in the same file can reach the private `inner` field. After that, both `push` and `pop` must return `Err(QueuePoisonedError)`.

```
$ cargo test --bin bounded_buffer_with_error_handling
running 2 tests
test tests::test_zero_capacity_panics - should panic ... ok
test tests::test_queue_poisoning_on_thread_panic ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

17.4.2 A boxed error

The third version returns a general error type:

```
type RuntimeError = Box<dyn Error + Send + Sync>;
```

`Box<dyn Error + Send + Sync>` can hold any error type. `+ Send + Sync` is needed because the error travels between threads: each thread returns a `Result`, and `join` hands it to the main thread.

Listing 17.10 `push` (lines 27 to 40). `src/bin/bounded_buffer_error_propagation.rs`

```
fn push(&self, item: T) -> Result<(), RuntimeError> {
    let mut guard = self.inner
        .lock()
        .map_err(|e| e.to_string())?;

    while guard.len() == self.capacity {
        guard = self.not_full.wait(guard).map_err(|e| e.to_string())?;
    }

    guard.push_back(item);
    drop(guard);
    self.not_empty.notify_one();
    Ok(())
}
```

Why `.map_err(|e| e.to_string())` instead of `?` directly? A `PoisonError` contains the guard, and the guard borrows the mutex. A `Box<dyn Error>` must not borrow anything local, so the poison error cannot go into one. `to_string()` turns it into a `String`, which owns its text. Then `?` converts the `String` into the boxed error, through a `From` implementation in the standard library.

Listing 17.11 The complete program. src/bin/bounded_buffer_error_propagation.rs

```

use std::collections::VecDeque;
use std::error::Error;
use std::sync::{Arc, Condvar, Mutex};
use std::thread;
use std::time::Duration;

type RuntimeError = Box<dyn Error + Send + Sync>;

struct BoundedQueue<T> {
    inner: Mutex<VecDeque<T>>,
    capacity: usize,
    not_empty: Condvar,
    not_full: Condvar,
}

impl<T> BoundedQueue<T> {
    fn new(capacity: usize) -> Self {
        assert!(capacity > 0, "capacity must be > 0");
        Self {
            inner: Mutex::new(VecDeque::with_capacity(capacity)),
            capacity,
            not_empty: Condvar::new(),
            not_full: Condvar::new(),
        }
    }

    fn push(&self, item: T) -> Result<(), RuntimeError> {
        let mut guard = self.inner
            .lock()
            .map_err(|e| e.to_string())?;

        while guard.len() == self.capacity {
            guard = self.not_full.wait(guard).map_err(|e| e.to_string())?;
        }

        guard.push_back(item);
        drop(guard);
        self.not_empty.notify_one();
        Ok(())
    }

    fn pop(&self) -> Result<T, RuntimeError> {
        let mut guard = self.inner
            .lock()
            .map_err(|e| e.to_string())?;

        while guard.is_empty() {
            guard = self.not_empty
                .wait(guard)
                .map_err(|e| e.to_string())?;
        }

        let item = match guard.pop_front() {
            Some(item) => item,
            None => return Err("deque is non-empty by loop invariant".into()),
        };
    }
}

```

```

        drop(guard);
        self.not_full.notify_one();
        Ok(item)
    }
}

fn main() -> Result<(), RuntimeError> {
    let queue = Arc::new(BoundedQueue::new(3));

    let producers = (0..2).map(|p| {
        let q = Arc::clone(&queue);
        thread::spawn(move || -> Result<(), RuntimeError> {
            for i in 0..5 {
                let item = format!("p{}:item{}", p, i);
                println!("[producer {}] pushing {}", p, item);
                q.push(item)?;
                thread::sleep(Duration::from_millis(30));
            }
            Ok(())
        })
    });

    let consumers = (0..2).map(|c| {
        let q = Arc::clone(&queue);
        thread::spawn(move || -> Result<(), RuntimeError> {
            for _ in 0..5 {
                let item = q.pop()?;
                println!("[consumer {}] got {}", c, item);
                thread::sleep(Duration::from_millis(60));
            }
            Ok(())
        })
    });

    let handles: Vec<_> = producers.chain(consumers).collect();

    for h in handles {
        match h.join() {
            Ok(r) => println!("Thread joined : {:#?}", r),
            Err(e) => eprintln!("Error occurred : {:#?}", e),
        }
    }

    println!("done");
    Ok(())
}

```

`main` builds the threads with two iterator chains, `producers` and `consumers`. These are lazy: no thread starts until something consumes them. `producers.chain(consumers).collect()` starts all four threads while building the `Vec`, and only then does the loop join them.

Each thread's closure is written `move || -> Result<(), RuntimeError> { ... }`, with its return type, so the `?` inside it knows what error type to convert to. `join()` returns `Ok(r)`, where `r` is the thread's own `Result`.


```

$ cargo run --bin bounded_buffer_error_propagation
[producer 1] pushing p1:item0
[producer 0] pushing p0:item0
[consumer 1] got p1:item0
[consumer 0] got p0:item0
[producer 0] pushing p0:item1
[producer 1] pushing p1:item1
[consumer 1] got p0:item1
[consumer 0] got p1:item1
[producer 0] pushing p0:item2
[producer 1] pushing p1:item2
[producer 0] pushing p0:item3
[producer 1] pushing p1:item3
[consumer 0] got p1:item2
[consumer 1] got p0:item2
[producer 0] pushing p0:item4
[producer 1] pushing p1:item4
Thread joined : Ok(
    (),
)
[consumer 0] got p1:item3
[consumer 1] got p0:item3
Thread joined : Ok(
    (),
)
[consumer 1] got p0:item4
[consumer 0] got p1:item4
Thread joined : Ok(
    (),
)
Thread joined : Ok(
    (),
)
done

```

The join messages appear between the consumer lines, because `main` joins the producers first, while the consumers are still running.

17.5 Many producers and consumers with `wait_while`

The next version is the bounded buffer written more compactly, with `wait_while` for both waits. Its `main` uses scoped threads, so no `Arc` is needed.

Listing 17.12 The complete program. `src/bin/mpmc_bounded_buffer.rs`

```

use std::thread;

use std::collections::VecDeque;
use std::sync::{Condvar, Mutex};

pub struct BoundedBuffer<T> {

```

```

    queue: Mutex<VecDeque<T>>,
    capacity: usize,
    not_empty: Condvar,
    not_full: Condvar,
}

impl<T> BoundedBuffer<T> {
    pub fn new(capacity: usize) -> Self {
        assert!(capacity > 0, "capacity must be > 0");

        Self {
            queue: Mutex::new(VecDeque::with_capacity(capacity)),
            capacity,
            not_empty: Condvar::new(),
            not_full: Condvar::new(),
        }
    }

    /// Blocks while the buffer is full.
    pub fn push(&self, item: T) {
        let cap = self.capacity;

        let mut queue = self.queue.lock().unwrap();
        queue = self
            .not_full
            .wait_while(queue, |q| q.len() >= cap)
            .unwrap();

        queue.push_back(item);
        drop(queue);

        self.not_empty.notify_one();
    }

    /// Blocks while the buffer is empty.
    pub fn pop(&self) -> T {
        let mut queue = self.queue.lock().unwrap();
        queue = self
            .not_empty
            .wait_while(queue, |q| q.is_empty())
            .unwrap();

        let item = queue.pop_front().unwrap();
        self.not_full.notify_one();
        item
    }
}

fn main() {
    let buffer = BoundedBuffer::<(usize, usize)>::new(8);
    let buffer = &buffer;

    const PRODUCERS: usize = 4;
    const CONSUMERS: usize = 2;
    const PER_PRODUCER: usize = 10;
    const PER_CONSUMER: usize = PRODUCERS * PER_PRODUCER / CONSUMERS;

    thread::scope(|s| {
        for p in 0..PRODUCERS {

```

```

        s.spawn(move || {
            for i in 0..PER_PRODUCER {
                buffer.push((p, i));
            }
        });
    }

    for c in 0..CONSUMERS {
        s.spawn(move || {
            for _ in 0..PER_CONSUMER {
                let item = buffer.pop();
                println!("consumer {c}: {item:?}");
            }
        });
    }
}

```

MPMC stands for multiple producers, multiple consumers. Here four producers push 10 items each, and two consumers pop 20 each. `PER_CONSUMER` is computed from the other constants, so every pushed item is popped and every thread finishes.

`push` reads `self.capacity` into a local `cap` before the closure `|q| q.len() >= cap`. The closure then captures a `usize` instead of borrowing `self`.

`let buffer = &buffer;` replaces the buffer with a reference to it. Each `move` closure then copies the reference into its thread. The scope guarantees that the buffer outlives all the threads.

```

$ cargo run --bin mpmc_bounded_buffer
consumer 1: (3, 0)
consumer 1: (2, 0)
consumer 0: (1, 0)
consumer 0: (3, 2)
consumer 0: (3, 3)
...
consumer 0: (3, 8)
consumer 0: (0, 9)
consumer 0: (2, 9)

```

The full output has 40 lines. Each item is a `(producer, index)` pair. Across the whole output, each producer's indexes appear in increasing order, but the producers are mixed together. Which consumer prints an item depends on which one the operating system happened to wake.

17.6 Closing a queue

Every version so far has one problem. A consumer loop needs to know when to stop. The demos stop after a fixed count, but a real consumer does not know how many items will come. It calls `pop` in a loop, and when the producers finish, `pop` waits forever.

The library version adds a way to **close** the queue (figure 17.5).

Closing a queue: everyone waiting wakes up, and the loops end

<code>close()</code>	<code>closed = true</code> , then <code>notify_all</code> on both <code>condvars</code>
<code>push(item)</code> after <code>close</code>	returns <code>Err(item)</code> : the caller gets the item back
<code>pop()</code> with items left	returns <code>Some(item)</code> : the queue drains first
<code>pop()</code> when empty and closed	returns <code>None</code> : a while let <code>Some(..)</code> loop ends

Figure 17.5 What each operation does after `close`.

Listing 17.13 The state (lines 14 to 25). `src/problems/bounded_queue.rs`

```
struct State<T> {
    items: VecDeque<T>,
    capacity: usize,
    closed: bool,
}

// A bounded MPMC queue. Share it behind `Arc`; all methods take `&self`.
pub struct BoundedQueue<T> {
    state: Mutex<State<T>>,
    not_empty: Condvar,
    not_full: Condvar,
}
```

The mutex now protects a small struct: the items, the capacity, and a `closed` flag. The flag must be inside the mutex, because threads check it together with the items.

Listing 17.14 `push` and `pop` (lines 45 to 80).

```
// Push an item, blocking while the queue is full.
//
// Returns `Err(item)` if the queue is closed, so the caller keeps ownership
// instead of losing the value.
pub fn push(&self, item: T) -> Result<(), T> {
    let mut state = self.state.lock().expect("queue mutex poisoned");
```

```

        loop {
            if state.closed {
                return Err(item);
            }
            if state.items.len() < state.capacity {
                state.items.push_back(item);
                self.not_empty.notify_one();
                return Ok(());
            }
            state = self.not_full.wait(state).expect("queue mutex poisoned");
        }
    }

    /// Pop an item, blocking while the queue is empty.
    ///
    /// Returns `None` once the queue is closed and drained, which is how a
    /// consumer loop terminates cleanly.
    pub fn pop(&self) -> Option<T> {
        let mut state = self.state.lock().expect("queue mutex poisoned");
        loop {
            if let Some(item) = state.items.pop_front() {
                self.not_full.notify_one();
                return Some(item);
            }
            if state.closed {
                return None;
            }
            state = self.not_empty.wait(state).expect("queue mutex poisoned");
        }
    }
}

```

Both methods use a `loop` that checks every condition after each wakeup.

`push` returns `Result<(), T>`. If the queue is closed, it returns `Err(item)`, handing the item back. The value is not lost, and the caller decides what to do with it.

`pop` returns `Option<T>`. It checks for an item before it checks `closed`, so a closed queue still gives up the items already in it. Only a closed queue that is empty returns `None`. A consumer can therefore loop with `while let Some(item) = queue.pop()`, and the loop ends after the last item.

Listing 17.15 `close` (lines 92 to 102).

```

/// Close the queue.
///
/// Pending and future `pop` calls return `None` after the queue drains, and
/// future `push` calls fail. Idempotent.
pub fn close(&self) {
    let mut state = self.state.lock().expect("queue mutex poisoned");
    state.closed = true;
    drop(state);
    self.not_empty.notify_all();
}

```

```

        self.not_full.notify_all();
    }

```

`close` sets the flag and wakes every waiting thread on both condition variables with `notify_all`. Each woken thread sees `closed` in its loop and returns. Calling `close` twice does no harm. An operation that gives the same result when repeated is called **idempotent**.

Listing 17.16 The complete file, with tests. `src/problems/bounded_queue.rs`

```

/// Bounded blocking queue with backpressure.
///
/// Unbounded queues are a classic production failure: a fast producer and a slow
/// consumer turn into an OOM kill instead of a slow response. A bounded queue makes
/// the producer wait, which is backpressure.
///
/// This is an MPMC queue (many producers, many consumers) built from one `Mutex`
/// and two condition variables: `not_empty` for waiting consumers and `not_full`
/// for waiting producers. `close` unblocks everyone so shutdown cannot hang.

use std::collections::VecDeque;
use std::sync::{Condvar, Mutex};

struct State<T> {
    items: VecDeque<T>,
    capacity: usize,
    closed: bool,
}

/// A bounded MPMC queue. Share it behind `Arc`; all methods take `&self`.
pub struct BoundedQueue<T> {
    state: Mutex<State<T>>,
    not_empty: Condvar,
    not_full: Condvar,
}

impl<T> BoundedQueue<T> {
    /// Create a queue that holds at most `capacity` items.
    ///
    /// Panics if `capacity` is zero, because a zero-capacity queue can never make
    /// progress.
    pub fn new(capacity: usize) -> Self {
        assert!(capacity > 0, "capacity must be > 0");
        Self {
            state: Mutex::new(State {
                items: VecDeque::with_capacity(capacity),
                capacity,
                closed: false,
            }),
            not_empty: Condvar::new(),
            not_full: Condvar::new(),
        }
    }

    /// Push an item, blocking while the queue is full.
    ///
    /// Returns `Err(item)` if the queue is closed, so the caller keeps ownership

```

```

/// instead of losing the value.
pub fn push(&self, item: T) -> Result<(), T> {
    let mut state = self.state.lock().expect("queue mutex poisoned");
    loop {
        if state.closed {
            return Err(item);
        }
        if state.items.len() < state.capacity {
            state.items.push_back(item);
            self.not_empty.notify_one();
            return Ok(());
        }
        state = self.not_full.wait(state).expect("queue mutex poisoned");
    }
}

/// Pop an item, blocking while the queue is empty.
///
/// Returns `None` once the queue is closed and drained, which is how a
/// consumer loop terminates cleanly.
pub fn pop(&self) -> Option<T> {
    let mut state = self.state.lock().expect("queue mutex poisoned");
    loop {
        if let Some(item) = state.items.pop_front() {
            self.not_full.notify_one();
            return Some(item);
        }
        if state.closed {
            return None;
        }
        state = self.not_empty.wait(state).expect("queue mutex poisoned");
    }
}

/// Pop without blocking.
pub fn try_pop(&self) -> Option<T> {
    let mut state = self.state.lock().expect("queue mutex poisoned");
    let item = state.items.pop_front();
    if item.is_some() {
        self.not_full.notify_one();
    }
    item
}

/// Close the queue.
///
/// Pending and future `pop` calls return `None` after the queue drains, and
/// future `push` calls fail. Idempotent.
pub fn close(&self) {
    let mut state = self.state.lock().expect("queue mutex poisoned");
    state.closed = true;
    drop(state);
    self.not_empty.notify_all();
    self.not_full.notify_all();
}

pub fn is_closed(&self) -> bool {
    self.state.lock().expect("queue mutex poisoned").closed
}

```

```

pub fn capacity(&self) -> usize {
    self.state.lock().expect("queue mutex poisoned").capacity
}

pub fn len(&self) -> usize {
    self.state.lock().expect("queue mutex poisoned").items.len()
}

pub fn is_empty(&self) -> bool {
    self.state
        .lock()
        .expect("queue mutex poisoned")
        .items
        .is_empty()
}
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::Arc;
    use std::sync::atomic::{AtomicUsize, Ordering};
    use std::thread;
    use std::time::Duration;

    #[test]
    fn preserves_order_across_threads() {
        let queue = Arc::new(BoundedQueue::new(8));
        let producer = {
            let queue = Arc::clone(&queue);
            thread::spawn(move || {
                for value in 0..1_000 {
                    queue.push(value).expect("queue stays open");
                }
                queue.close();
            })
        };
        let mut received = Vec::new();
        while let Some(value) = queue.pop() {
            received.push(value);
        }
        producer.join().expect("producer panicked");

        assert_eq!(received, (0..1_000).collect::<Vec<_>>());
    }

    #[test]
    fn never_exceeds_capacity() {
        let queue = Arc::new(BoundedQueue::new(4));
        let observed_peak = Arc::new(AtomicUsize::new(0));
        let producer = {
            let queue = Arc::clone(&queue);
            let observed_peak = Arc::clone(&observed_peak);
            thread::spawn(move || {
                for value in 0..200u32 {
                    queue.push(value).expect("queue stays open");
                    observed_peak.fetch_max(queue.len(), Ordering::SeqCst);
                }
            })
        };
        producer.join().expect("producer panicked");
        let peak = observed_peak.load(Ordering::SeqCst);
        assert!(peak <= queue.capacity());
    }
}

```



```

        }
        queue.close();
    })
};

while queue.pop().is_some() {
    thread::sleep(Duration::from_micros(50));
}
producer.join().expect("producer panicked");

assert!(observed_peak.load(Ordering::SeqCst) <= 4);
}

#[test]
fn close_wakes_a_blocked_consumer() {
    let queue = Arc::new(BoundedQueue::new(1));
    let consumer = {
        let queue = Arc::clone(&queue);
        thread::spawn(move || queue.pop())
    };

    thread::sleep(Duration::from_millis(20));
    queue.close();

    assert_eq!(consumer.join().expect("consumer panicked"), None);
}

#[test]
fn push_after_close_returns_the_item() {
    let queue = BoundedQueue::new(1);
    queue.close();
    assert_eq!(
        queue.push("workload".to_string()),
        Err("workload".to_string())
    );
}
}

```

The tests check the four promises:

- `preserves_order_across_threads` pushes 0 to 999 from one thread and pops them on another. They must arrive in order, and the consumer's `while let` loop must end after `close`.
- `never_exceeds_capacity` records the largest length seen, with a slow consumer. It must never pass 4.
- `close_wakes_a_blocked_consumer` starts a consumer on an empty queue, waits 20 ms so it is asleep, and closes the queue. The consumer must return `None` instead of sleeping forever.
- `push_after_close_returns_the_item` checks that `push` hands the item back.

```
$ cargo test --lib problems::bounded_queue
running 4 tests
test problems::bounded_queue::tests::push_after_close_returns_the_item ... ok
test problems::bounded_queue::tests::preserves_order_across_threads ... ok
test problems::bounded_queue::tests::never_exceeds_capacity ... ok
test problems::bounded_queue::tests::close_wakes_a_blocked_consumer ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 133 filtered out
```

17.7 A ring buffer without a lock

Every queue so far takes a lock for each operation. With exactly one producer thread and one consumer thread, the lock can go. This case is called **SPSC**, for single producer, single consumer.

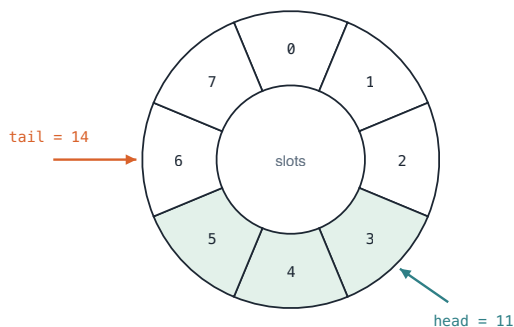
17.7.1 The idea

A **ring buffer** is a fixed array used in a circle (figure 17.6). Two counters track it:

- **head** counts the items taken so far. The consumer reads the next item at **head % N**.
- **tail** counts the items added so far. The producer writes the next item at **tail % N**.

The number of items in the ring is **tail - head**. The ring is empty when the two are equal, and full when the difference is **N**.

An SPSC ring with **N = 8** slots: head and tail only grow, and **index = counter % N**



The consumer reads at **head % 8 = 3**. Only it changes head.

The producer writes at **tail % 8 = 6**. Only it changes tail.

len = tail - head = 3 items, in slots 3, 4, and 5.

full when tail - head == 8; empty when **head == tail**.

push: write the slot, then store **tail + 1** with Release.

pop: load tail with Acquire, read the slot, store **head + 1**.

Figure 17.6 A ring with 8 slots holding 3 items. The counters keep growing, and **% 8** maps them to slots.

No lock is needed because each counter has one writer. Only the producer changes `tail`, and only the consumer changes `head`. Each side only reads the other's counter.

The ordering of each store makes the handoff safe. The producer writes the item into its slot first, and then stores the new `tail` with `Release`. The consumer loads `tail` with `Acquire` before it reads the slot. This is the release and acquire pairing from section 16.4. If the consumer sees the new `tail`, it also sees the item written before it.

17.7.2 The type

Listing 17.17 The ring (lines 23 to 36). `src/problems/ring_buffer.rs`

```
/// Ring buffer with capacity `N` (the maximum number of queued items).
pub struct SpscRing<T, const N: usize> {
    slots: [UnsafeCell<MaybeUninit<T>>; N],
    /// Next index to read. Written only by the consumer.
    head: AtomicUsize,
    /// Next index to write. Written only by the producer.
    tail: AtomicUsize,
}

// SAFETY: at most one thread mutates `head` and one mutates `tail`. A slot is
// initialized before `tail` is published with a release store, and the consumer
// performs an acquire load of `tail` before reading that slot, so there is no
// data race and no read of uninitialized memory under the SPSC discipline.
unsafe impl<T: Send, const N: usize> Sync for SpscRing<T, N> {}
```

Three new pieces of Rust appear here:

- `const N: usize` is a **const generic**: a type parameter that is a number instead of a type. `SpscRing<u64, 256>` is a ring of 256 slots, and the array has that length at compile time.
- `MaybeUninit<T>` is memory that may or may not hold a valid `T`. An empty slot holds no value, and Rust does not allow a variable of type `T` to hold garbage. `MaybeUninit` makes the uninitialized state explicit. Reading it as a `T` is `unsafe`, because the code must know that the slot holds a value.
- `UnsafeCell` lets the producer write a slot through `&self`, as in the spin lock of section 16.4.

`new` builds the array with `std::array::from_fn`, which calls a closure once per index.

17.7.3 Push and pop

Listing 17.18 `push` and `pop` (lines 69 to 103).

```

/// Enqueue a value, returning `Err(value)` when the ring is full.
pub fn push(&self, value: T) -> Result<(), T> {
    let tail = self.tail.load(Ordering::Relaxed);
    let head = self.head.load(Ordering::Acquire);
    if tail.wrapping_sub(head) >= N {
        return Err(value);
    }

    let index = tail % N;
    // SAFETY: the slot at `index` is free because the ring is not full, and
    // only the producer writes it.
    unsafe { (*self.slots[index].get()).write(value) };

    // Publish the slot to the consumer.
    self.tail.store(tail.wrapping_add(1), Ordering::Release);
    Ok(())
}

/// Dequeue a value, or `None` when the ring is empty.
pub fn pop(&self) -> Option<T> {
    let head = self.head.load(Ordering::Relaxed);
    let tail = self.tail.load(Ordering::Acquire);
    if head == tail {
        return None;
    }

    let index = head % N;
    // SAFETY: `head != tail`, so the producer has published this slot and
    // only the consumer reads it.
    let value = unsafe { (*self.slots[index].get()).assume_init_read() };

    // Publish the freed slot to the producer.
    self.head.store(head.wrapping_add(1), Ordering::Release);
    Some(value)
}

```

`push` loads its own counter, `tail`, with `Relaxed`, because only this thread writes it. It loads the other side's counter, `head`, with `Acquire`. That load pairs with the consumer's `Release` store of `head`, so the consumer has finished reading a slot before the producer writes it again.

If the ring is full, `push` returns `Err(value)`, handing the value back. Otherwise it writes the value into the slot with `MaybeUninit::write`, and publishes it by storing `tail + 1` with `Release`.

`pop` is the mirror image. It loads `tail` with `Acquire`, reads the value out of the slot with `assume_init_read`, and publishes the free slot by storing `head + 1` with `Release`.

The counters use `wrapping_add` and `wrapping_sub`. After 2 to the power 64 operations a counter would wrap around to 0 , and wrapping arithmetic keeps `tail - head` correct across that point.

Listing 17.19 Drop (lines 112 to 117).

```
impl<T, const N: usize> Drop for SpscRing<T, N> {
    fn drop(&mut self) {
        // Drop any items that were never popped.
        while self.pop().is_some() {}
    }
}
```

`MaybeUninit` never drops its contents, because it cannot know whether a value is there. So when the ring is dropped, its `Drop` pops every remaining item. Each popped value is a normal `T`, and it is dropped at the end of the loop body. The test `dropped_ring_drops_queued_items` counts those drops.

WARNING: The safety of `SpscRing` depends on a rule the compiler does not check: one thread calls `push`, and one thread calls `pop`. Both methods take `&self`, and the type is `Sync`. So safe code can share the ring with two producer threads, and both can write the same slot at once. That is a data race. A sound design splits the ring into a `Producer` handle and a `Consumer` handle that cannot be cloned. Exercise 5 asks you to build it.

Listing 17.20 The complete file, with tests. `src/problems/ring_buffer.rs`

```
/// Lock-free single-producer / single-consumer (SPSC) ring buffer.
///
/// The classic producer/consumer exercise, but with no mutex: the producer owns
/// the `tail` index and the consumer owns the `head` index, and each publishes its
/// write with a release store so the other side sees the slot only after it is
/// initialized. This is the shape inside channel implementations, log shippers,
/// and kernel ring buffers.
///
/// Two rules keep this sound:
///
/// - exactly one thread calls [SpscRing::push] and exactly one calls
///   [SpscRing::pop];
/// - a slot is written before `tail` is published, and read only after `head`
///   reaches it.
///
/// If you need many producers or consumers, use a mutex-backed queue such as
/// [crate::problems::bounded_queue::BoundedQueue].

use std::cell::UnsafeCell;
use std::mem::MaybeUninit;
use std::sync::atomic::{AtomicUsize, Ordering};
```

```

/// Ring buffer with capacity `N` (the maximum number of queued items).
pub struct SpscRing<T, const N: usize> {
    slots: [UnsafeCell<MaybeUninit<T>>; N],
    /// Next index to read. Written only by the consumer.
    head: AtomicUsize,
    /// Next index to write. Written only by the producer.
    tail: AtomicUsize,
}

// SAFETY: at most one thread mutates `head` and one mutates `tail`. A slot is
// initialized before `tail` is published with a release store, and the consumer
// performs an acquire load of `tail` before reading that slot, so there is no
// data race and no read of uninitialized memory under the SPSC discipline.
unsafe impl<T: Send, const N: usize> Sync for SpscRing<T, N> {}

impl<T, const N: usize> SpscRing<T, N> {
    /// Create an empty ring. Panics if `N` is zero.
    pub fn new() -> Self {
        assert!(N > 0, "ring capacity must be > 0");
        Self {
            slots: std::array::from_fn(|_| UnsafeCell::new(MaybeUninit::uninit())),
            head: AtomicUsize::new(0),
            tail: AtomicUsize::new(0),
        }
    }

    /// Maximum number of items the ring can hold.
    pub const fn capacity(&self) -> usize {
        N
    }

    /// Number of queued items. Only meaningful to the owning side in a race.
    pub fn len(&self) -> usize {
        self.tail
            .load(Ordering::Acquire)
            .wrapping_sub(self.head.load(Ordering::Acquire))
    }

    pub fn is_empty(&self) -> bool {
        self.len() == 0
    }

    pub fn is_full(&self) -> bool {
        self.len() >= N
    }

    /// Enqueue a value, returning `Err(value)` when the ring is full.
    pub fn push(&self, value: T) -> Result<(), T> {
        let tail = self.tail.load(Ordering::Relaxed);
        let head = self.head.load(Ordering::Acquire);
        if tail.wrapping_sub(head) >= N {
            return Err(value);
        }

        let index = tail % N;
        // SAFETY: the slot at `index` is free because the ring is not full, and
        // only the producer writes it.
        unsafe { (*self.slots[index].get()).write(value) };
    }
}

```

```

        // Publish the slot to the consumer.
        self.tail.store(tail.wrapping_add(1), Ordering::Release);
        Ok(())
    }

    /// Dequeue a value, or `None` when the ring is empty.
    pub fn pop(&self) -> Option<T> {
        let head = self.head.load(Ordering::Relaxed);
        let tail = self.tail.load(Ordering::Acquire);
        if head == tail {
            return None;
        }

        let index = head % N;
        // SAFETY: `head != tail`, so the producer has published this slot and
        // only the consumer reads it.
        let value = unsafe { (*self.slots[index].get()).assume_init_read() };

        // Publish the freed slot to the producer.
        self.head.store(head.wrapping_add(1), Ordering::Release);
        Some(value)
    }
}

impl<T, const N: usize> Default for SpscRing<T, N> {
    fn default() -> Self {
        Self::new()
    }
}

impl<T, const N: usize> Drop for SpscRing<T, N> {
    fn drop(&mut self) {
        // Drop any items that were never popped.
        while self.pop().is_some() {}
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::Arc;
    use std::sync::atomic::AtomicUsize;
    use std::thread;

    #[test]
    fn is_fifo_within_capacity() {
        let ring = SpscRing::<u32, 4>::new();
        assert!(ring.push(1).is_ok());
        assert!(ring.push(2).is_ok());
        assert!(ring.push(3).is_ok());
        assert_eq!(ring.pop(), Some(1));
        assert_eq!(ring.pop(), Some(2));
        assert_eq!(ring.pop(), Some(3));
        assert_eq!(ring.pop(), None);
    }

    #[test]
    fn full_ring_returns_the_value() {

```

```

    let ring = SpscRing::<u32, 2>::new();
    assert!(ring.push(1).is_ok());
    assert!(ring.push(2).is_ok());
    assert!(ring.is_full());
    assert_eq!(ring.push(3), Err(3));
}

#[test]
fn producer_and_consumer_run_concurrently_in_order() {
    const ITEMS: u64 = 50_000;
    const SENTINEL: u64 = u64::MAX;

    let ring = Arc::new(SpscRing::<u64, 256>::new());

    let producer = {
        let ring = Arc::clone(&ring);
        thread::spawn(move || {
            for value in 0..ITEMS {
                let mut pending = value;
                loop {
                    match ring.push(pending) {
                        Ok(()) => break,
                        Err(returned) => {
                            pending = returned;
                            thread::yield_now();
                        }
                    }
                }
            }
        })
    };

    let mut pending = SENTINEL;
    loop {
        match ring.push(pending) {
            Ok(()) => break,
            Err(returned) => {
                pending = returned;
                thread::yield_now();
            }
        }
    }

    };

    let consumer = {
        let ring = Arc::clone(&ring);
        thread::spawn(move || {
            let mut received = Vec::with_capacity(ITEMS as usize);
            loop {
                match ring.pop() {
                    Some(SENTINEL) => break,
                    Some(value) => received.push(value),
                    None => thread::yield_now(),
                }
            }
            received
        })
    };

    producer.join().expect("producer panicked");
    let received = consumer.join().expect("consumer panicked");
}

```



```

    assert_eq!(received, (0..ITEMS).collect::<Vec<_>>());
}

#[test]
fn dropped_ring_drops_queued_items() {
    struct Tracker(Arc<AtomicUsize>);
    impl Drop for Tracker {
        fn drop(&mut self) {
            self.0.fetch_add(1, Ordering::SeqCst);
        }
    }

    let dropped = Arc::new(AtomicUsize::new(0));
    {
        let ring = SpscRing::<Tracker, 4>::new();
        assert!(ring.push(Tracker(Arc::clone(&dropped))).is_ok());
        assert!(ring.push(Tracker(Arc::clone(&dropped))).is_ok());
        assert!(ring.push(Tracker(Arc::clone(&dropped))).is_ok());
    }
    assert_eq!(dropped.load(Ordering::SeqCst), 3);
}
}

```

The concurrent test sends 50,000 values through a ring of 256 slots. When the ring is full, the producer calls `thread::yield_now()` and tries again. When it is empty, the consumer does the same. The producer ends with a sentinel value, `u64::MAX`, which tells the consumer to stop. The consumer must receive every value, in order.

```

$ cargo test --lib problems::ring_buffer
running 4 tests
test problems::ring_buffer::tests::is_fifo_within_capacity ... ok
test problems::ring_buffer::tests::full_ring_returns_the_value ... ok
test problems::ring_buffer::tests::dropped_ring_drops_queued_items ... ok
test problems::ring_buffer::tests::producer_and_consumer_run_concurrently_in_order ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 133 filtered out

```

Summary

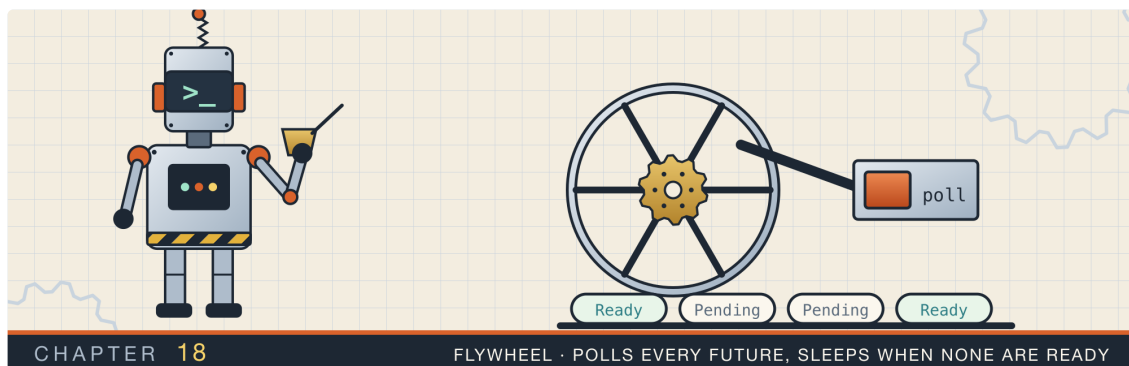
- A queue connects threads that produce work with threads that consume it. A bounded queue makes producers wait when it is full, which is backpressure.
- A `Condvar` puts a thread to sleep until another thread notifies it. The wait releases the mutex, and the thread holds it again when it wakes.
- Always check the condition again after waking, with a `while` loop or `wait_while`. Wakeups can be spurious, and another thread may have acted first.

- Two condition variables, `not_empty` and `not_full`, wake only the kind of thread that can make progress.
- A poisoned lock can panic the caller, become a custom error through an extension trait, or become a `String` inside a boxed error.
- A `closed` flag, `notify_all`, and `pop` returning `None` after the queue drains let every consumer loop end.
- An SPSC ring buffer needs no lock: each counter has one writer, and release and acquire publish each slot. Its one-producer rule must be enforced by the API, or it is unsound.

Chapter 18 puts a queue at the center of a thread pool: a fixed set of threads that take jobs from a shared queue.

Exercises

1. In `bounded_buffer.rs`, measure how long each `push` waits and print it. Change the sleep times until the producers visibly block.
2. Add `try_push(&self, item: T) -> Result<(), T>` to the library `BoundedQueue`. It should return at once when the queue is full.
3. Add `pop_timeout(&self, timeout: Duration) -> Option<T>` using `Condvar::wait_timeout_while`.
4. Replace the two condition variables in `bounded_buffer.rs` with one, using `notify_all`. Check that it still works, and count the wakeups with an atomic counter.
5. Split `SpSCRing` into `Producer<T, N>` and `Consumer<T, N>` handles that share the ring through an `Arc`. Give `push` and `pop` a `&mut self` receiver, so that each handle can be used by only one thread at a time.



Thread pools

THIS CHAPTER COVERS

- A pool of worker threads that take jobs from a shared channel
- A measured bug: a lock held for too long turned four parallel jobs into a sequence
- Returning errors instead of panicking, and shutting down in `Drop`
- Keeping workers alive when a job panics, with `catch_unwind`
- A bounded job queue, results sent back through a channel, and many producers
- A parallel map that returns results in input order

Starting a thread costs time and memory. The operating system must create it, give it a stack, and schedule it. For a program that runs thousands of small jobs, starting a thread per job wastes most of its time on starting threads.

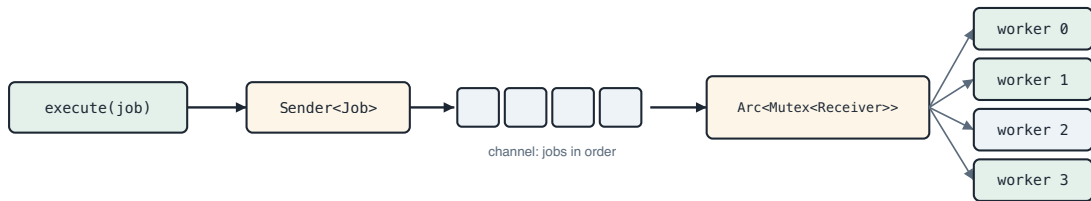
A **thread pool** starts a fixed number of threads once, and reuses them. The threads are called **workers**. Each worker waits for a job, runs it, and waits for the next. Jobs wait in a queue until a worker is free. This is the queue of chapter 17, with the consumers built in.

This chapter builds the pool several times. Each version fixes a problem found in the one before, so read them in order.

18.1 The parts of a pool

Every pool here has the same parts (figure 18.1).

The parts of every pool in this chapter



Job = Box<dyn FnOnce() + Send + 'static>: any closure, moved to the heap, runnable once on another thread.
 The channel has one Receiver, so the workers share it behind an Arc<Mutex<...>>. A worker locks it to take a job.
 Dropping the Sender closes the channel: recv() then returns Err, and each worker leaves its loop.

Figure 18.1 A job travels from `execute`, through a channel, to whichever worker takes it first.

The job type. A job is any closure that the pool can run once on another thread:

```
type Job = Box<dyn FnOnce() + Send + 'static>;
```

Each closure has its own unnamed type, so a queue cannot hold them directly. Box<dyn FnOnce()> puts each closure on the heap behind a pointer of one common type. dyn FnOnce() means "some type that can be called once with no arguments". Send lets the job move to a worker thread. 'static means the job borrows nothing that could be dropped before the job runs.

The channel. std::sync::mpsc::channel() returns a Sender and a Receiver. Values sent through the sender come out of the receiver in the same order. mpsc stands for multiple producer, single consumer: the Sender can be cloned, but there is only one Receiver.

The shared receiver. Every worker needs to take jobs from the one receiver. So the pool wraps it in a Mutex, and shares the mutex through an Arc. A worker locks the mutex, calls recv() to take the next job, and runs it.

Shutdown. When every Sender is dropped, the channel closes. recv() then returns Err once the remaining jobs are taken. A worker that sees the Err leaves its loop, and the thread ends.

18.2 Version 1, and a bug you can measure

Listing 18.1 The complete program. src/bin/thread_pool.rs

```
use std::sync::{mpsc, Arc, Mutex};
use std::thread;

type Job = Box<dyn FnOnce() + Send + 'static>;

pub struct ThreadPool {
    sender: mpsc::Sender<Job>,
    workers: Vec<thread::JoinHandle<>>,
}

impl ThreadPool {
    pub fn new(size: usize) -> Self {
        let (sender, receiver) = mpsc::channel:::<Job>();
        let receiver = Arc::new(Mutex::new(receiver));

        let workers = (0..size)
            .map(|_| {
                let rx = Arc::clone(&receiver);
                thread::spawn(move || {
                    while let Ok(job) = rx.lock().unwrap().recv() {
                        job();
                    }
                })
            })
            .collect();

        Self { sender, workers }
    }

    pub fn execute(&self, job: Job) {
        self.sender.send(job).unwrap();
    }

    pub fn join(self) {
        drop(self.sender);
        for w in self.workers {
            let _ = w.join();
        }
    }
}

fn main() {
    let pool = ThreadPool::new(4);
    for i in 0..10_000 {
        pool.execute(Box::new(move || println!("task {i}")));
    }

    pool.join();
}
```

`new` creates the channel and starts `size` workers. Each gets its own clone of the `Arc` around the receiver. `execute` sends a boxed job. `join` takes `self` by value, drops the sender to close the channel, and waits for every worker. `main` queues 10,000 jobs that each print a line.

The program prints 10,000 lines and looks correct. It has one bug, in the worker loop:

```
while let Ok(job) = rx.lock().unwrap().recv() {
    job();
}
```

`rx.lock().unwrap()` creates a `MutexGuard`. The guard is a temporary value inside the `while let` condition. A temporary in that position lives until the end of the loop body, so the guard is still alive while `job()` runs. The worker holds the lock for the whole job, and no other worker can take a job in the meantime.

To see the effect, I ran four jobs that each sleep for 200 ms on four workers. I ran them once with this loop and once with the loop of version 2. Figure 18.2 shows the result.

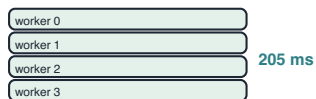
Four jobs that sleep 200 ms on four workers, measured with each loop shape

```
while let Ok(job) = rx.lock().unwrap().recv() { job(); }
```



The guard is a temporary of the while-let condition, so the lock stays held until the job ends.

```
let job = rx.lock().unwrap().recv(); match job { ... }
```



The guard is a temporary of the let statement, so the lock is released before the job runs.

Figure 18.2 The same four jobs with each loop shape, release build.

With `while let`, the four jobs ran one after another and took 814 ms. With the lock released before the job, they ran at the same time and took 205 ms. The first pool had four threads but used one at a time.

The same bug has a second effect. If a job panics, the panic happens while the guard is alive, so it poisons the mutex. When I piped this program's output into `head`, which closes the pipe after five lines, `println!` panicked inside a job. The other three workers then panicked on `.unwrap()` of the poisoned lock, and `main` panicked when `send` found no worker left.

18.3 Version 2: release the lock before the job

Listing 18.2 The worker loop and `execute` (lines 21 to 62). `src/problems/thread_pool_v2.rs`

```
/// Start `size` workers. Panics if `size` is zero.
pub fn new(size: usize) -> Self {
    assert!(size > 0, "a thread pool needs at least one worker");

    let (sender, receiver) = mpsc::channel:::<Job>();
    let receiver = Arc::new(Mutex::new(receiver));

    let workers = (0..size)
        .map(|_| {
            let receiver = Arc::clone(&receiver);
            thread::spawn(move || {
                loop {
                    // The guard is a temporary of this `let` statement, so the
                    // lock is released here, before the job runs.
                    let job = receiver.lock().unwrap().recv();
                    match job {
                        Ok(job) => job(),
                        Err(_) => break, // every sender is gone: shut down
                    }
                }
            })
        })
        .collect();

    Self { sender, workers }
}

/// Queue a closure to run on the next free worker.
pub fn execute<F>(&self, job: F)
where
    F: FnOnce() + Send + 'static,
{
    self.sender.send(Box::new(job)).unwrap();
}

/// Stop accepting jobs, let the workers drain the queue, and wait for them.
pub fn join(self) {
    drop(self.sender);
    for worker in self.workers {
        let _ = worker.join();
    }
}
```

The fix is one line. `let job = receiver.lock().unwrap().recv();` is a separate statement, and the temporary guard is dropped at the end of that statement. The lock is released before `match` runs the job.

Version 2 also changes `execute`. It is generic over `F: FnOnce() + Send + 'static`, and does the boxing itself. Callers write `pool.execute(|| ...)` instead of `pool.execute(Box::new(|| ...))`.

Listing 18.3 The complete file, with tests. `src/problems/thread_pool_v2.rs`

```

///! Thread pool, version 2: workers really run in parallel.
///!
///! Version 1 (`src/bin/thread_pool.rs`) holds the receiver's lock while a job runs,
///! because the `MutexGuard` in `while let Ok(job) = rx.lock().unwrap().recv()` lives
///! until the end of the loop body. This version takes the job in its own `let`
///! statement, so the guard is dropped before the job starts.
///!
///! It also accepts any closure in `execute`, so callers no longer write `Box::new`.

use std::sync::{Arc, Mutex, mpsc};
use std::thread;

type Job = Box<dyn FnOnce() + Send + 'static>;

pub struct ThreadPool {
    sender: mpsc::Sender<Job>,
    workers: Vec<thread::JoinHandle<()>>,
}

impl ThreadPool {
    /// Start `size` workers. Panics if `size` is zero.
    pub fn new(size: usize) -> Self {
        assert!(size > 0, "a thread pool needs at least one worker");

        let (sender, receiver) = mpsc::channel::<Job>();
        let receiver = Arc::new(Mutex::new(receiver));

        let workers = (0..size)
            .map(|_| {
                let receiver = Arc::clone(&receiver);
                thread::spawn(move || {
                    loop {
                        // The guard is a temporary of this `let` statement, so the
                        // lock is released here, before the job runs.
                        let job = receiver.lock().unwrap().recv();
                        match job {
                            Ok(job) => job(),
                            Err(_) => break, // every sender is gone: shut down
                        }
                    }
                })
            })
            .collect();

        Self { sender, workers }
    }

    /// Queue a closure to run on the next free worker.
    pub fn execute<F>(&self, job: F)
    where

```



```

        F: FnOnce() + Send + 'static,
    {
        self.sender.send(Box::new(job)).unwrap();
    }

    /// Stop accepting jobs, let the workers drain the queue, and wait for them.
    pub fn join(self) {
        drop(self.sender);
        for worker in self.workers {
            let _ = worker.join();
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::Barrier;
    use std::sync::atomic::{AtomicUsize, Ordering};
    use std::time::Duration;

    #[test]
    fn runs_every_job() {
        let pool = ThreadPool::new(4);
        let counter = Arc::new(AtomicUsize::new(0));
        for _ in 0..100 {
            let counter = Arc::clone(&counter);
            pool.execute(move || {
                counter.fetch_add(1, Ordering::Relaxed);
            });
        }
        pool.join();
        assert_eq!(counter.load(Ordering::Relaxed), 100);
    }

    #[test]
    fn jobs_run_at_the_same_time() {
        // Four jobs wait on a barrier for four parties. They can only all get
        // through if four workers run them at once; version 1 would never finish.
        let pool = ThreadPool::new(4);
        let barrier = Arc::new(Barrier::new(4));
        let (done_tx, done_rx) = mpsc::channel();

        for _ in 0..4 {
            let barrier = Arc::clone(&barrier);
            let done_tx = done_tx.clone();
            pool.execute(move || {
                barrier.wait();
                done_tx.send(()).unwrap();
            });
        }

        for _ in 0..4 {
            done_rx
                .recv_timeout(Duration::from_secs(5))
                .expect("jobs did not run in parallel");
        }
        pool.join();
    }
}

```

```

#[test]
#[should_panic(expected = "at least one worker")]
fn zero_workers_is_rejected() {
    let _ = ThreadPool::new(0);
}
}

```

The test `jobs_run_at_the_same_time` checks the fix without measuring time. It uses a `Barrier` for four threads. A barrier makes each thread that calls `wait()` sleep until four threads have called it, then releases them all. Four jobs wait on the barrier. They can only get through if four workers run them at the same moment. With version 1's loop, the first job would hold the lock and wait on the barrier forever.

The test waits for the results with `recv_timeout(Duration::from_secs(5))`, so a broken pool fails the test after five seconds instead of hanging it.

18.4 Version 3: errors, and shutdown on drop

Version 2 still panics in three places: for a size of zero, when `send` fails, and when the lock is poisoned. And if the caller forgets to call `join`, the pool is dropped without waiting for its jobs. Version 3 fixes both.

Listing 18.4 The error type (lines 14 to 30). `src/problems/thread_pool_v3.rs`

```

/// Why the pool refused a request.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum PoolError {
    NoWorkers,
    ShutDown,
}

impl fmt::Display for PoolError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            Self::NoWorkers => write!(f, "a thread pool needs at least one worker"),
            Self::ShutDown => write!(f, "the thread pool has shut down"),
        }
    }
}

impl std::error::Error for PoolError {}

```

`PoolError` names the two ways a request can fail. It implements `Display` for a message, and `Error` so callers can use `?` and `Box<dyn Error>` with it.

Listing 18.5 `execute`, `shutdown`, and `Drop` (lines 61 to 87).

```

/// Queue a closure to run on the next free worker.
pub fn execute<F>(&self, job: F) -> Result<(), PoolError>
where
    F: FnOnce() + Send + 'static,
{
    self.sender
        .as_ref()
        .ok_or(PoolError::ShutDown)?
        .send(Box::new(job))
        .map_err(|_| PoolError::ShutDown)
}

/// Stop accepting jobs, drain the queue, and wait for every worker.
/// Calling it again does nothing.
pub fn shutdown(&mut self) {
    drop(self.sender.take());
    for worker in self.workers.drain(..) {
        let _ = worker.join();
    }
}

impl Drop for ThreadPool {
    fn drop(&mut self) {
        self.shutdown();
    }
}

```

The sender is now an `Option<Sender<Job>>`. `shutdown` takes it out with `self.sender.take()`, which leaves `None` behind, and drops it. `execute` checks for `None` with `.ok_or(PoolError::ShutDown)?`.

`shutdown` takes `&mut self`, not `self`, so that `Drop` can call it. `self.workers.drain(..)` removes the handles from the vector one at a time and gives each to the loop. A second call finds no sender and no workers, so it does nothing.

`impl Drop for ThreadPool` calls `shutdown`. When a pool goes out of scope, it closes its channel and waits for every queued job to finish.

Listing 18.6 The worker loop (lines 89 to 102).

```

fn worker_loop(receiver: &Mutex<mpsc::Receiver<Job>>) {
    loop {
        // A poisoned lock means another worker panicked while holding it;
        // stop this worker instead of panicking a second time.
        let Ok(guard) = receiver.lock() else { return };
        let job = guard.recv();
        drop(guard);

        match job {
            Ok(job) => job(),

```

```

        Err(_) => return,
    }
}
}

```

The worker loop is now a named function. It uses `let ... else` to leave quietly if the lock is poisoned, and drops the guard explicitly before running the job.

Listing 18.7 The complete file, with tests. `src/problems/thread_pool_v3.rs`

```

//! Thread pool, version 3: errors instead of panics, and shutdown on drop.
//!
//! - `new` returns `Err(PoolError::NoWorkers)` for a size of zero.
//! - `execute` returns `Err(PoolError::ShutDown)` once the pool no longer accepts work.
//! - Dropping the pool closes the queue and joins every worker, so a pool that goes
//! out of scope still finishes the jobs it was given.

use std::fmt;
use std::sync::{Arc, Mutex, mpsc};
use std::thread;

type Job = Box<dyn FnOnce() + Send + 'static>;

/// Why the pool refused a request.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum PoolError {
    NoWorkers,
    ShutDown,
}

impl fmt::Display for PoolError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            Self::NoWorkers => write!(f, "a thread pool needs at least one worker"),
            Self::ShutDown => write!(f, "the thread pool has shut down"),
        }
    }
}

impl std::error::Error for PoolError {}

pub struct ThreadPool {
    // `None` after shutdown: dropping the sender is what tells workers to stop.
    sender: Option<mpsc::Sender<Job>>,
    workers: Vec<thread::JoinHandle<()>>,
}

impl ThreadPool {
    /// Start `size` workers.
    pub fn new(size: usize) -> Result<Self, PoolError> {
        if size == 0 {
            return Err(PoolError::NoWorkers);
        }

        let (sender, receiver) = mpsc::channel:::<Job>();
        let receiver = Arc::new(Mutex::new(receiver));

```

```

        let workers = (0..size)
            .map(|_| {
                let receiver = Arc::clone(&receiver);
                thread::spawn(move || worker_loop(&receiver))
            })
            .collect();

        Ok(Self {
            sender: Some(sender),
            workers,
        })
    }

    /// Queue a closure to run on the next free worker.
    pub fn execute<F>(&self, job: F) -> Result<(), PoolError>
    where
        F: FnOnce() + Send + 'static,
    {
        self.sender
            .as_ref()
            .ok_or(PoolError::ShutDown)?
            .send(Box::new(job))
            .map_err(|_| PoolError::ShutDown)
    }

    /// Stop accepting jobs, drain the queue, and wait for every worker.
    /// Calling it again does nothing.
    pub fn shutdown(&mut self) {
        drop(self.sender.take());
        for worker in self.workers.drain(..) {
            let _ = worker.join();
        }
    }
}

impl Drop for ThreadPool {
    fn drop(&mut self) {
        self.shutdown();
    }
}

fn worker_loop(receiver: &Mutex<mpsc::Receiver<Job>>) {
    loop {
        // A poisoned lock means another worker panicked while holding it;
        // stop this worker instead of panicking a second time.
        let Ok(guard) = receiver.lock() else { return };
        let job = guard.recv();
        drop(guard);

        match job {
            Ok(job) => job(),
            Err(_) => return,
        }
    }
}

#[cfg(test)]
mod tests {

```

```

use super::*;
use std::sync::atomic::{AtomicUsize, Ordering};

#[test]
fn zero_workers_is_an_error() {
    assert_eq!(ThreadPool::new(0).err(), Some(PoolError::NoWorkers));
}

#[test]
fn dropping_the_pool_finishes_queued_jobs() {
    let counter = Arc::new(AtomicUsize::new(0));
    {
        let pool = ThreadPool::new(2).unwrap();
        for _ in 0..50 {
            let counter = Arc::clone(&counter);
            pool.execute(move || {
                counter.fetch_add(1, Ordering::Relaxed);
            })
            .unwrap();
        }
    } // drop: close the queue and join
    assert_eq!(counter.load(Ordering::Relaxed), 50);
}

#[test]
fn execute_after_shutdown_is_an_error() {
    let mut pool = ThreadPool::new(1).unwrap();
    pool.shutdown();
    assert_eq!(pool.execute(|| {}), Err(PoolError::ShutDown));
    pool.shutdown(); // idempotent
}

#[test]
fn errors_render_a_message() {
    assert_eq!(
        PoolError::NoWorkers.to_string(),
        "a thread pool needs at least one worker"
    );
    assert_eq!(
        PoolError::ShutDown.to_string(),
        "the thread pool has shut down"
    );
}
}

```

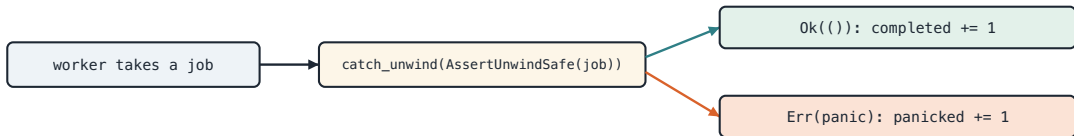
The test `dropping_the_pool_finishes_queued_jobs` creates a pool inside a block, queues 50 jobs, and lets the pool go out of scope at the closing brace. After the block, all 50 jobs must have run.

18.5 Version 4: surviving a panicking job

Version 3 has one remaining weakness. If a job panics, the panic unwinds out of `worker_loop` and ends that worker's thread. The pool now has one worker fewer, and nothing reports it. After enough panics, no worker is left, and jobs queue up forever.

Version 4 catches the panic (figure 18.3).

Version 4 runs each job inside `catch_unwind`



The panic stops at `catch_unwind`. The worker thread survives and takes the next job.

Without it, a panicking job unwinds out of the worker's closure and ends the thread: the pool silently shrinks.

The default panic hook still prints the panic message to standard error.

Figure 18.3 A job's panic stops at `catch_unwind`, and the worker lives on.

Listing 18.8 The worker loop (lines 138 to 153). `src/problems/thread_pool_v4.rs`

```

fn worker_loop(receiver: &Mutex<mpsc::Receiver<Job>>, counters: &Counters) {
    loop {
        let Ok(guard) = receiver.lock() else { return };
        let job = guard.recv();
        drop(guard);

        let Ok(job) = job else { return };

        // `AssertUnwindSafe` is sound here: the job owns everything it touches,
        // and a panicking job's partial state is never observed by this loop.
        match panic::catch_unwind(AssertUnwindSafe(job)) {
            Ok(()) => counters.completed.fetch_add(1, Ordering::Relaxed),
            Err(_) => counters.panicked.fetch_add(1, Ordering::Relaxed),
        };
    }
}
  
```

`panic::catch_unwind(f)` runs `f`. If `f` panics, the unwinding stops at `catch_unwind`, which returns `Err`. Otherwise it returns `Ok` with `f`'s result.

`catch_unwind` requires the closure to be `UnwindSafe`. The trait marks types that cannot leave broken data visible after a panic. A boxed `dyn FnOnce` does not have it, because the compiler cannot see what the closure captured. `AssertUnwindSafe(job)` is a wrapper that declares the

closure safe anyway. The comment gives the reason: the job owns everything it touches, and this loop never looks at its partial state.

Each outcome increments one of two atomic counters, `completed` or `panicked`.

Listing 18.9 Named workers (lines 67 to 99).

```
/// Start `size` named workers.
pub fn new(size: usize) -> Result<Self, PoolError> {
    if size == 0 {
        return Err(PoolError::NoWorkers);
    }

    let (sender, receiver) = mpsc::channel::<Job>();
    let receiver = Arc::new(Mutex::new(receiver));
    let counters = Arc::new(Counters::default());

    let mut pool = Self {
        sender: Some(sender),
        workers: Vec::with_capacity(size),
        counters,
    };

    for id in 0..size {
        let receiver = Arc::clone(&receiver);
        let counters = Arc::clone(&pool.counters);
        let spawned = thread::Builder::new()
            .name(format!("pool-worker-{}", id))
            .spawn(move || worker_loop(&receiver, &counters));

        match spawned {
            Ok(handle) => pool.workers.push(handle),
            // Dropping `pool` here closes the queue and joins the workers
            // that did start, so none of them is leaked.
            Err(error) => return Err(PoolError::Spawn(error)),
        }
    }

    Ok(pool)
}
```

`thread::Builder` configures a thread before starting it. Here it gives each worker a name, `pool-worker-0` and so on, which appears in panic messages and debuggers. `Builder::spawn` returns a `Result`, because the operating system can refuse to create a thread. `thread::spawn` panics in that case.

The pool is built before the workers are started. If starting the third worker fails, `return Err(...)` drops the half-built `pool`. Its `Drop` closes the channel and joins the two workers that did start, so no thread is left running.

Listing 18.10 shutdown and the report (lines 113 to 136).

```

    /// Close the queue, wait for every queued job, and report the outcome.
    pub fn shutdown(mut self) -> Report {
        self.close_and_join();
        Report {
            completed: self.counters.completed.load(Ordering::Relaxed),
            panicked: self.counters.panicked.load(Ordering::Relaxed),
        }
    }

    fn close_and_join(&mut self) {
        drop(self.sender.take());
        for worker in self.workers.drain(..) {
            // Jobs cannot unwind out of `worker_loop`, so a join error would mean
            // a bug in the loop itself; there is nothing further to clean up.
            let _ = worker.join();
        }
    }

    impl Drop for ThreadPool {
        fn drop(&mut self) {
            self.close_and_join();
        }
    }
}

```

shutdown takes self by value and returns a Report with the two counts. The work is in close_and_join, which Drop also calls. When shutdown returns, self is dropped, and Drop runs close_and_join a second time. The second call finds nothing left to do.

Listing 18.11 The complete file, with tests. src/problems/thread_pool_v4.rs

```

    ///! Thread pool, version 4: survives panicking jobs and reports what happened.
    ///!
    ///! - Workers are named threads created with `thread::Builder`, and a failure to
    ///!   spawn one is returned as `PoolError::Spawn` after the already started workers
    ///!   are shut down.
    ///! - Each job runs inside `catch_unwind`, so a panicking job no longer kills its
    ///!   worker and silently shrinks the pool.
    ///! - `shutdown` consumes the pool and returns a `Report` with the number of jobs
    ///!   that completed and the number that panicked.

    use std::fmt;
    use std::io;
    use std::panic::{self, AssertUnwindSafe};
    use std::sync::atomic::{AtomicUsize, Ordering};
    use std::sync::{Arc, Mutex, mpsc};
    use std::thread;

    type Job = Box<dyn FnOnce() + Send + 'static>;

    /// Why the pool could not be created or refused a job.
    #[derive(Debug)]
    pub enum PoolError {
        NoWorkers,

```

```

        Spawn(io::Error),
        ShutDown,
    }

    impl fmt::Display for PoolError {
        fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
            match self {
                Self::NoWorkers => write!(f, "a thread pool needs at least one worker"),
                Self::Spawn(error) => write!(f, "failed to spawn a worker thread: {error}"),
                Self::ShutDown => write!(f, "the thread pool has shut down"),
            }
        }
    }

    impl std::error::Error for PoolError {
        fn source(&self) -> Option<&(dyn std::error::Error + 'static)> {
            match self {
                Self::Spawn(error) => Some(error),
                _ => None,
            }
        }
    }

    /// What the pool did over its lifetime.
    #[derive(Debug, Clone, Copy, PartialEq, Eq)]
    pub struct Report {
        pub completed: usize,
        pub panicked: usize,
    }

    #[derive(Default)]
    struct Counters {
        completed: AtomicUsize,
        panicked: AtomicUsize,
    }

    pub struct ThreadPool {
        sender: Option<mpsc::Sender<Job>>,
        workers: Vec<thread::JoinHandle<>>,
        counters: Arc<Counters>,
    }

    impl ThreadPool {
        /// Start `size` named workers.
        pub fn new(size: usize) -> Result<Self, PoolError> {
            if size == 0 {
                return Err(PoolError::NoWorkers);
            }

            let (sender, receiver) = mpsc::channel::<Job>();
            let receiver = Arc::new(Mutex::new(receiver));
            let counters = Arc::new(Counters::default());

            let mut pool = Self {
                sender: Some(sender),
                workers: Vec::with_capacity(size),
                counters,
            };

```

```

        for id in 0..size {
            let receiver = Arc::clone(&receiver);
            let counters = Arc::clone(&pool.counters);
            let spawned = thread::Builder::new()
                .name(format!("pool-worker-{}", id))
                .spawn(move || worker_loop(&receiver, &counters));

            match spawned {
                Ok(handle) => pool.workers.push(handle),
                // Dropping `pool` here closes the queue and joins the workers
                // that did start, so none of them is leaked.
                Err(error) => return Err(PoolError::Spawn(error)),
            }
        }

        Ok(pool)
    }

    /// Queue a closure to run on the next free worker.
    pub fn execute<F>(&self, job: F) -> Result<(), PoolError>
    where
        F: FnOnce() + Send + 'static,
    {
        self.sender
            .as_ref()
            .ok_or(PoolError::ShutDown)?
            .send(Box::new(job))
            .map_err(|_| PoolError::ShutDown)
    }

    /// Close the queue, wait for every queued job, and report the outcome.
    pub fn shutdown(mut self) -> Report {
        self.close_and_join();
        Report {
            completed: self.counters.completed.load(Ordering::Relaxed),
            panicked: self.counters.panicked.load(Ordering::Relaxed),
        }
    }

    fn close_and_join(&mut self) {
        drop(self.sender.take());
        for worker in self.workers.drain(..) {
            // Jobs cannot unwind out of `worker_loop`, so a join error would mean
            // a bug in the loop itself; there is nothing further to clean up.
            let _ = worker.join();
        }
    }
}

impl Drop for ThreadPool {
    fn drop(&mut self) {
        self.close_and_join();
    }
}

fn worker_loop(receiver: &Mutex<mpsc::Receiver<Job>>, counters: &Counters) {
    loop {
        let Ok(guard) = receiver.lock() else { return };
        let job = guard.recv();
    }
}

```

```

        drop(guard);

        let Ok(job) = job else { return };

        // `AssertUnwindSafe` is sound here: the job owns everything it touches,
        // and a panicking job's partial state is never observed by this loop.
        match panic::catch_unwind(AssertUnwindSafe(job)) {
            Ok(()) => counters.completed.fetch_add(1, Ordering::Relaxed),
            Err(_) => counters.panicked.fetch_add(1, Ordering::Relaxed),
        };
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::error::Error;
    use std::sync::mpsc::channel;
    use std::time::Duration;

    #[test]
    fn reports_completed_jobs() {
        let pool = ThreadPool::new(4).unwrap();
        for _ in 0..20 {
            pool.execute(|| {}).unwrap();
        }
        assert_eq!(
            pool.shutdown(),
            Report {
                completed: 20,
                panicked: 0
            }
        );
    }

    #[test]
    fn a_panicking_job_does_not_kill_its_worker() {
        // One worker: if the panic killed it, the second job would never run.
        let pool = ThreadPool::new(1).unwrap();
        let (done_tx, done_rx) = channel();

        pool.execute(|| panic!("job failed on purpose")).unwrap();
        pool.execute(move || done_tx.send(()).unwrap()).unwrap();

        done_rx
            .recv_timeout(Duration::from_secs(5))
            .expect("the worker died with the panicking job");
        assert_eq!(
            pool.shutdown(),
            Report {
                completed: 1,
                panicked: 1
            }
        );
    }

    #[test]
    fn workers_are_named() {
        let pool = ThreadPool::new(1).unwrap();

```

```

        let (name_tx, name_rx) = channel();
        pool.execute(move || {
            let name = thread::current().name().map(str::to_string);
            name_tx.send(name).unwrap();
        })
        .unwrap();
        assert_eq!(name_rx.recv().unwrap().as_deref(), Some("pool-worker-0"));
    }

    #[test]
    fn zero_workers_is_an_error() {
        assert!(matches!(ThreadPool::new(0), Err(PoolError::NoWorkers)));
    }

    #[test]
    fn spawn_errors_expose_their_source() {
        let error = PoolError::Spawn(io::Error::other("no threads left"));
        assert!(error.to_string().contains("no threads left"));
        assert!(error.source().is_some());
        assert!(PoolError::Shutdown.source().is_none());
    }
}

```

The test `a_panicking_job_does_not_kill_its_worker` uses a pool with one worker. The first job panics. If that killed the worker, the second job would never run, and `recv_timeout` would fail after five seconds.

The test `spawn_errors_expose_their_source` checks the `source` method. `Error::source` returns the lower-level error that caused this one, so a caller can print the whole chain.

```

$ cargo test --lib problems::thread_pool
running 12 tests
test problems::thread_pool_v3::tests::zero_workers_is_an_error ... ok
test problems::thread_pool_v3::tests::errors_render_a_message ... ok
test problems::thread_pool_v2::tests::zero_workers_is_rejected - should panic ... ok
test problems::thread_pool_v3::tests::execute_after_shutdown_is_an_error ... ok
test problems::thread_pool_v3::tests::dropping_the_pool_finishes_queued_jobs ... ok
test problems::thread_pool_v2::tests::runs_every_job ... ok
test problems::thread_pool_v4::tests::spawn_errors_expose_their_source ... ok
test problems::thread_pool_v2::tests::jobs_run_at_the_same_time ... ok
test problems::thread_pool_v4::tests::zero_workers_is_an_error ... ok
test problems::thread_pool_v4::tests::reports_completed_jobs ... ok
test problems::thread_pool_v4::tests::a_panicking_job_does_not_kill_its_worker ... ok
test problems::thread_pool_v4::tests::workers_are_named ... ok

test result: ok. 12 passed; 0 failed; 0 ignored; 0 measured; 125 filtered out

```

The filter `problems::thread_pool` matches all three versions. `cargo test` hides what passing tests print, including the message of the job that panics on purpose. Add `-- --nocapture` to see it. The message names the thread, `pool-worker-0`, which shows the thread names at work.

18.6 Shorter pools

Three more pools take the lessons of versions 2 and 3 in a more compact form, and each adds one feature.

18.6.1 Shutdown in Drop, compactly

Listing 18.12 The complete program. `src/bin/thread_pool_bruce_lee.rs`

```
use std::sync::{mpsc, Arc, Mutex};
use std::thread::{self, JoinHandle};

type Job = Box<dyn FnOnce() + Send + 'static>;

pub struct ThreadPool {
    workers: Vec<JoinHandle<()>>,
    job_sender: Option<mpsc::Sender<Job>>,
}

impl ThreadPool {
    pub fn new(size: usize) -> Self {
        assert!(size > 0, "pool size must be > 0");
        let (job_sender, job_receiver) = mpsc::channel();
        let job_receiver = Arc::new(Mutex::new(job_receiver));

        let workers = (0..size)
            .map(|_| Self::spawn_worker(Arc::clone(&job_receiver)))
            .collect();

        Self { workers, job_sender: Some(job_sender) }
    }

    fn spawn_worker(job_receiver: Arc<Mutex<mpsc::Receiver<Job>>>) -> JoinHandle<()> {
        thread::spawn(move || loop {
            let message = job_receiver.lock().unwrap().recv();
            match message {
                Ok(job) => job(),
                Err(_) => break,
            }
        })
    }

    pub fn execute(&self, task: impl FnOnce() + Send + 'static) {
        if let Some(job_sender) = &self.job_sender {
            job_sender.send(Box::new(task)).unwrap();
        }
    }
}

impl Drop for ThreadPool {
    fn drop(&mut self) {
        self.job_sender = None; // close channel so workers stop
        for worker in self.workers.drain(..) {
            worker.join().unwrap(); // Wait for workers to finish... This will ensure
```

```

    graceful shutdown..
    }
}

fn main() {
    let pool = ThreadPool::new(4);
    for i in 0..8 {
        pool.execute(move || println!("task {i} on {:?}", thread::current().id()));
    }
}

```

This pool keeps version 2's worker loop and version 3's `Option` sender and `Drop`. `self.job_sender = None` drops the sender by overwriting it. `thread::spawn(move || loop { ... })` passes a closure whose whole body is the loop.

`main` has no explicit shutdown. The pool is dropped at the end of `main`, and `Drop` waits for all eight jobs.

```

$ cargo run --bin thread_pool_bruce_lee
task 0 on ThreadId(4)
task 4 on ThreadId(4)
task 5 on ThreadId(4)
task 6 on ThreadId(4)
task 7 on ThreadId(4)
task 1 on ThreadId(3)
task 2 on ThreadId(2)
task 3 on ThreadId(5)

```

In this run, one worker took five of the eight jobs. Printing a line is so quick that the first worker to wake could take the next job before the others were scheduled. The pool does not promise an even split.

`Drop` calls `worker.join().unwrap()`. If a job panicked, its worker's `join` returns `Err`, and the `unwrap` panics inside `Drop`. The later versions use `let _ = worker.join();` instead.

18.6.2 A bounded queue and results

Listing 18.13 The complete program. `src/bin/thread_pool_with_back_pressure.rs`

```

use std::sync::{mpsc, Arc, Mutex};
use std::thread::{self, JoinHandle};

type Job = Box<dyn FnOnce() + Send + 'static>;

pub struct ThreadPool {
    workers: Vec<JoinHandle<()>>,
    job_sender: Option<mpsc::SyncSender<Job>>, // changed: bounded sender
}

```

```

}

impl ThreadPool {
    pub fn new(size: usize, queue_size: usize) -> Self { // changed: queue_size
        assert!(size > 0, "pool size must be > 0");
        let (job_sender, job_receiver) = mpsc::sync_channel(queue_size); // changed:
        bounded queue
        let job_receiver = Arc::new(Mutex::new(job_receiver));

        let workers = (0..size)
            .map(|_| Self::spawn_worker(Arc::clone(&job_receiver)))
            .collect();

        Self { workers, job_sender: Some(job_sender) }
    }

    fn spawn_worker(job_receiver: Arc<Mutex<mpsc::Receiver<Job>>>) -> JoinHandle<()> {
        thread::spawn(move || loop {
            let message = job_receiver.lock().unwrap().recv();
            match message {
                Ok(job) => job(),
                Err(_) => break,
            }
        })
    }

    pub fn execute(&self, task: impl FnOnce() + Send + 'static) {
        if let Some(job_sender) = &self.job_sender {
            job_sender.send(Box::new(task)).unwrap(); // blocks when queue is full
        }
    }

    // new: run a task and get its result back
    pub fn submit<T: Send + 'static>(
        &self,
        task: impl FnOnce() -> T + Send + 'static,
    ) -> mpsc::Receiver<T> {
        let (result_sender, result_receiver) = mpsc::channel();
        self.execute(move || {
            let _ = result_sender.send(task());
        });
        result_receiver
    }
}

impl Drop for ThreadPool {
    fn drop(&mut self) {
        self.job_sender = None;
        for worker in self.workers.drain(..) {
            let _ = worker.join();
        }
    }
}

fn main() {
    let pool = ThreadPool::new(4, 10); // 4 workers, at most 10 jobs waiting

    let results: Vec<_> = (0..8).map(|i| pool.submit(move || i * i)).collect();

```



```

    for r in results {
        println!("{}", r.recv().unwrap());
    }
}

```

The comments marked `changed` and `new` show the two additions.

`mpsc::sync_channel(queue_size)` creates a channel that holds at most `queue_size` messages. `SyncSender::send` blocks while the channel is full. This is the backpressure of section 17.1, built into the standard library's channel. A caller that queues jobs faster than the workers run them has to wait.

`submit` returns a result. It creates a new channel for each job, and the job sends its return value into it. The caller gets the `Receiver` back at once, and calls `recv()` when it needs the value. A value that will be ready later is often called a **future**, and chapter 22 builds on that idea.

```

$ cargo run --bin thread_pool_with_back_pressure
0
1
4
9
16
25
36
49

```

The results print in order because `main` calls `recv()` on the receivers in order, even though the jobs may finish in any order.

18.6.3 Many producers

Listing 18.14 The complete program. `src/bin/mpmc_thread_pool_engine.rs`

```

use std::sync::mpsc::{self, Receiver, SendError, SyncSender};
use std::sync::{Arc, Mutex};
use std::thread::{self, JoinHandle};

type Job = Box<dyn FnOnce() + Send + 'static>;
type SharedLockedChannel = Arc<Mutex<Receiver<Job>>>;

pub struct ThreadPool {
    tx: Option<SyncSender<Job>>,
    workers: Vec<JoinHandle<()>>,
}

impl ThreadPool {
    pub fn new(num_workers: usize, queue_size: usize) -> Self {

```

```

    let (tx, rx) = mpsc::sync_channel::<Job>(queue_size);
    let rx = SharedLockedChannel::new(Mutex::new(rx));

    let workers = (0..num_workers)
        .map(|_| Self::spawn_worker(Arc::clone(&rx)))
        .collect();

    Self { tx: Some(tx), workers }
}

fn spawn_worker(rx: SharedLockedChannel) -> JoinHandle<()> {
    thread::spawn(move || loop {
        let Ok(receiver) = rx.lock() else { break }; // mutex poisoned
        let Ok(job) = receiver.recv() else { break }; // channel closed
        drop(receiver); // release lock before running the job
        job();
    })
}

pub fn execute<F>(&self, f: F) -> Result<(), SendError<Job>>
where
    F: FnOnce() + Send + 'static,
{
    let job: Job = Box::new(f);
    match &self.tx {
        Some(tx) => tx.send(job), // blocks when queue is full
        None => Err(SendError(job)),
    }
}

pub fn join_all<I>(&self, handles: I)
where
    I: IntoIterator<Item = JoinHandle<()>>,
{
    for handle in handles {
        let _ = handle.join();
    }
}

impl Drop for ThreadPool {
    fn drop(&mut self) {
        drop(self.tx.take()); // close channel so workers exit
        for worker in self.workers.drain(..) {
            let _ = worker.join();
        }
    }
}

fn main() {
    let pool = Arc::new(ThreadPool::new(4, 16));

    let producers: Vec<_> = (0..3)
        .map(|p| {
            let pool = Arc::clone(&pool);
            thread::spawn(move || {
                for i in 0..5 {
                    let _ = pool.execute(move || println!("producer {p}, job {i}"));
                }
            })
        })
}

```

```

        })
    })
    .collect();

    pool.join_all(producers);
} // last Arc dropped here: Drop waits for all queued jobs

```

This pool's worker loop is the clearest of the chapter:

```

let Ok(receiver) = rx.lock() else { break }; // mutex poisoned
let Ok(job) = receiver.recv() else { break }; // channel closed
drop(receiver); // release lock before running the job
job();

```

Each `let ... else` line handles one failure and says which. The lock is released explicitly before the job runs.

`execute` returns `Result<(), SendError<Job>>`. When the pool is shut down, it returns the job inside the error, so the caller can still run it or report it.

`main` shares the pool between three producer threads with an `Arc`. Each producer queues five jobs. When the last `Arc` is dropped at the end of `main`, `Drop` runs and waits for every queued job.

```

$ cargo run --bin mpmc_thread_pool_engine
producer 0, job 0
producer 0, job 3
producer 0, job 4
producer 1, job 1
producer 1, job 2
producer 0, job 1
producer 1, job 4
producer 2, job 0
producer 2, job 1
producer 2, job 2
producer 0, job 2
producer 2, job 4
producer 2, job 3
producer 1, job 3
producer 1, job 0

```

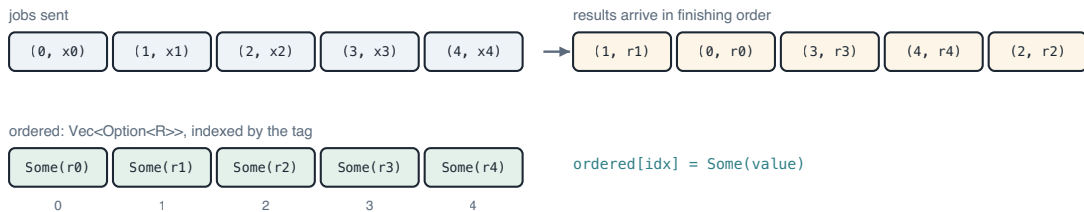
All 15 jobs ran. The order is mixed, even within one producer, because four workers run the jobs at the same time.

18.7 A parallel map that keeps order

The last pool is a single function. `map_order` applies a function to every input on several threads, and returns the results in the same order as the inputs.

The difficulty is that jobs finish in any order. The fix is to tag each input with its position, and use the tag to put each result back in place (figure 18.4).

map_order: tag each input with its index, and put each result back in its slot



The result channel ends when every worker has dropped its Sender clone, so `iter()` stops after the last result.

Figure 18.4 The index travels with the job, and puts the result back in its slot.

Listing 18.15 The complete file, with tests. `src/problems/worker_pool.rs`

```

//! Bounded worker pool with ordered results.
//!
//! Programs often fan out network calls across a bounded number of workers.
//! This example maps `Vec<T>` to `Vec<R>` with N threads and preserves input
//! order in the output. It demonstrates `Arc<Mutex<_>>`, channels, scoped
//! ownership, and panic-free dispatch.

use std::sync::{mpsc, Arc, Mutex};
use std::thread;

pub fn map_order<T, R, F>(worker_count: usize, inputs: Vec<T>, mapper: F) -> Vec<R>
where
    T: Send + 'static,
    R: Send + 'static,
    F: Fn(T) -> R + Send + Sync + 'static,
{
    assert!(worker_count > 0, "worker_count must be > 0");
    assert!(!inputs.is_empty(), "inputs must not be empty");

    let expected = inputs.len();
    let mapper = Arc::new(mapper);
    let (job_tx, job_rx) = mpsc::channel::<(usize, T)>();
    let job_rx = Arc::new(Mutex::new(job_rx));
    let (result_tx, result_rx) = mpsc::channel::<(usize, R)>();

```

```

let mut handles = Vec::with_capacity(worker_count);
for _worker_id in 0..worker_count {
    let mapper = Arc::clone(&mapper);
    let job_rx = Arc::clone(&job_rx);
    let result_tx = result_tx.clone();
    handles.push(thread::spawn(move || {
        loop {
            let job = {
                let guard = job_rx.lock().expect("job_rx poisoned");
                guard.recv()
            };
            match job {
                Ok((idx, input)) => {
                    let output = mapper(input);
                    if result_tx.send((idx, output)).is_err() {
                        break;
                    }
                }
                Err(_) => break, // all senders dropped
            }
        }
    }));
}

// Send all jobs, then close the job channel so workers finish after draining it.
for (idx, input) in inputs.into_iter().enumerate() {
    job_tx
        .send((idx, input))
        .expect("bounded by memory; channel only blocks until a worker receives");
}
drop(job_tx);

// Drop our result sender. Only the worker clones remain, so `iter()` ends
// after every worker exits.
drop(result_tx);

// Reassemble results in input order.
let mut ordered: Vec<Option<R>> = Vec::new();
for (idx, value) in result_rx.iter() {
    if ordered.len() <= idx {
        ordered.resize_with(idx + 1, || None);
    }
    ordered[idx] = Some(value);
}

for handle in handles {
    let _ = handle.join();
}

assert_eq!(
    ordered.len(),
    expected,
    "every job must produce exactly one result"
);
ordered
    .into_iter()
    .map(|value| value.expect("all results present"))
    .collect()
}

```

```

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn maps_all_inputs_and_preserves_order() {
        let inputs: Vec<i32> = (0..100).collect();
        let results = map_order(4, inputs, |x| x * x);
        assert_eq!(results.len(), 100);
        for (i, value) in results.iter().enumerate() {
            assert_eq!(*value, (i as i32) * (i as i32));
        }
    }

    #[test]
    fn one_worker_works() {
        let results = map_order(1, vec![1, 2, 3], |x| x + 10);
        assert_eq!(results, vec![11, 12, 13]);
    }
}

```

The function uses two channels. Jobs go out as `(usize, T)` pairs, and results come back as `(usize, R)` pairs.

`mapper` is shared by all workers, so it goes in an `Arc`, and it must be `Sync` as well as `Send`. It has type `F: Fn(T) -> R`, not `FnOnce`, because each worker calls it many times.

The worker loop takes the job inside a block: `let job = { let guard = ...; guard.recv() }`; . The guard is dropped at the end of the block, so the lock is released before `mapper` runs. This is the third way in this chapter to write the same fix.

After sending every job, the function drops its own `job_tx` so that the workers stop when the queue is empty. It also drops its own `result_tx`. Only the workers' clones remain, and `result_rx.iter()` ends when the last worker exits and drops its clone.

Each result is stored at its index in `ordered`, a `Vec<Option<R>>`, which grows as needed with `resize_with`. At the end, every slot must hold `Some`, and the options are unwrapped into the final `Vec<R>`.

```

$ cargo test --lib problems::worker_pool
running 2 tests
test problems::worker_pool::tests::one_worker_works ... ok
test problems::worker_pool::tests::maps_all_inputs_and_preserves_order ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 135 filtered out

```

18.8 The versions compared

Table 18.1 What each pool handles

Pool	Jobs run in parallel	Shutdown	Poisoned lock	Panicking job	Queue
v1 <code>thread_pool.rs</code>	no: lock held during job	<code>join(self)</code>	panics	poisons the lock	unbounded
v2 <code>thread_pool_v2.rs</code>	yes	<code>join(self)</code>	panics	kills the worker	unbounded
v3 <code>thread_pool_v3.rs</code>	yes	<code>Drop</code>	worker exits	kills the worker	unbounded
v4 <code>thread_pool_v4.rs</code>	yes	<code>Drop, Report</code>	worker exits	caught and counted	unbounded
<code>thread_pool_bruce_lee.rs</code>	yes	<code>Drop</code>	panics	kills the worker	unbounded
<code>thread_pool_with_back_pressure.rs</code>	yes	<code>Drop</code>	panics	kills the worker	bounded
<code>mpmc_thread_pool_engine.rs</code>	yes	<code>Drop</code>	worker exits	kills the worker	bounded

Summary

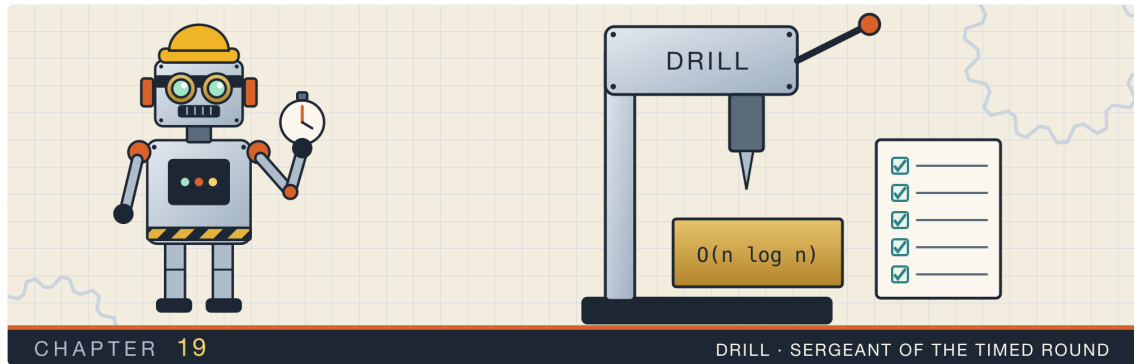
- A thread pool reuses a fixed set of worker threads. Jobs are boxed closures, `Box<dyn FnOnce() + Send + 'static>`, sent through a channel whose receiver the workers share behind `Arc<Mutex<_>>`.

- In `while let Ok(job) = rx.lock().unwrap().recv()`, the guard lives through the loop body. The workers ran one at a time: 814 ms instead of 205 ms for four 200 ms jobs.
- Release the lock before running the job: take it in its own `let` statement, in a block, or with `drop(guard)`.
- Dropping the sender closes the channel. Workers drain the queue, see `Err` from `recv`, and exit. Doing this in `Drop` makes a pool finish its jobs when it goes out of scope.
- `catch_unwind` with `AssertUnwindSafe` keeps a worker alive when its job panics.
- `sync_channel` bounds the queue and adds backpressure. A per-job channel carries a result back to the caller.
- Tagging each job with its index returns parallel results in input order.

Chapter 19 turns to failures between programs: limiting request rates, retrying failed calls, and making repeated requests safe.

Exercises

1. Add a test to `thread_pool_v2.rs` that measures four jobs sleeping 200 ms each, and asserts that they finish in under 400 ms.
2. Add `submit` from listing 18.13 to version 4, returning the job's result through a channel.
3. Make version 4's queue bounded with `sync_channel`, and add a `try_execute` method that returns the job instead of blocking when the queue is full.
4. Write `map_order` so that it returns `Result<Vec<R>, E>` for a mapper of type `Fn(T) -> Result<R, E>`, and stops sending jobs after the first error.



Rate limits, retries, and idempotency

THIS CHAPTER COVERS

- A token bucket that limits how often something may happen, with a lock and with atomics
- A measured flaw in the atomic version, and where it comes from
- Retrying failed calls with exponential backoff, jitter, and the server's `Retry-After`
- Deciding which errors to retry, with a trait
- Making a repeated request safe with an idempotency key

Programs that talk to other programs over a network see failures that local code never sees. A server is busy and refuses a request. A connection drops halfway. A reply takes so long that the client gives up, without knowing whether the server did the work.

This chapter builds three tools for those situations:

- A **rate limiter** protects a service from too many requests, by refusing requests above a set rate.
- A **retry** policy tries a failed call again, waiting longer after each failure.
- An **idempotency key** makes it safe to send the same request twice, by making the second one do nothing new.

The three work together. A client that retries must not overload the server, so it backs off. And a retried request must not repeat a side effect, such as charging a card twice.

19.1 A token bucket

Suppose a service allows each client 5 requests per second. A simple counter that resets every second allows 5 requests at 0.99 s and 5 more at 1.01 s: 10 requests in 20 ms. A **token bucket** smooths this out.

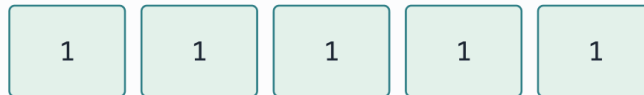
Picture a bucket that holds up to 5 tokens. Every request takes one token out. If the bucket is empty, the request is refused. Tokens flow back in at a steady rate, say 1 per second, until the bucket is full again (figure 19.1).

A token bucket: refill from elapsed time, then spend

Capacity 5. A call adds the tokens earned since the last call before it checks.

☒ a token ☐ spent or missing

tokens in the bucket



elapsed time is the only input: $\text{tokens} = \min(\text{capacity}, \text{tokens} + \text{elapsed} * \text{rate})$

The bucket holds 5 tokens, which is its capacity, so a burst can spend all of them at once.

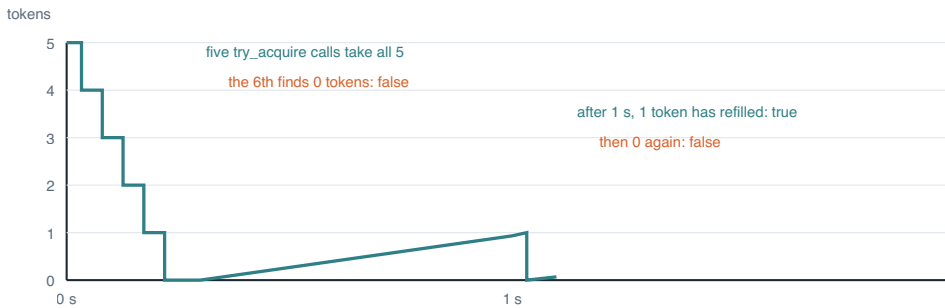
The bucket never fills past its capacity, however long the caller waits.

tokens = 5

event = **full**

capacity = 5

Animation 19.1 The bucket at capacity 5 with a refill of 1 token per second. A full cell is a token in the bucket. A burst spends tokens faster than they arrive, which is allowed up to the capacity; after that a call is refused. When a second passes, one token appears with no timer and no background thread, because the count is worked out from the clock at the start of each call.

A token bucket with capacity 5 and a refill of 1 token per second (the test `it_works`)

Each call first adds elapsed seconds x rate, capped at the capacity, then takes one token if at least one is there.

Figure 19.1 The bucket in the test `it_works`: capacity 5, refill 1 token per second.

Two numbers describe the bucket. The **capacity** is the largest burst allowed at once. The **rate** is the long-run number of requests per second.

The bucket does not need a timer that adds tokens. Each request computes how many tokens arrived since the last request, from the time that has passed. It adds them before it checks.

19.1.1 The bucket behind a lock

Listing 19.1 The bucket (lines 6 to 31). `src/problems/rate_limiter.rs`

```
pub struct TokenBucket {
    /// Max tokens.
    capacity: f64,
    /// Tokens per second.
    rate: f64,
    state: Mutex<BucketState>,
}

struct BucketState {
    tokens: f64,
    last_refill: Instant,
}

pub fn new(capacity: usize, rate_per_sec: f64) -> Self {
    assert!(capacity > 0);
    assert!(rate_per_sec > 0.0);
    Self {
        capacity: capacity as f64,
        rate: rate_per_sec,
        state: Mutex::new(BucketState {
            tokens: capacity as f64, // Start full
            last_refill: Instant::now(),
        }),
    }
}
```

```
    }
}
```

The capacity and the rate never change, so they are plain fields. The token count and the time of the last refill change on every call, so they live in `BucketState` behind a `Mutex`. The token count is an `f64`, because a fraction of a token can arrive between two requests. The bucket starts full.

`Instant` is the standard library's clock for measuring durations. It only moves forward, unlike the wall-clock time, which can jump when the system clock is corrected.

Listing 19.2 `try_acquire` (lines 33 to 50).

```
pub fn try_acquire(&self) -> bool {
    let mut s = self.state.lock().unwrap();
    let now = Instant::now();

    // Refill tokens based on elapsed time
    let elapsed = now - s.last_refill;
    let new_tokens = elapsed.as_secs_f64() * self.rate;
    s.tokens = (s.tokens + new_tokens).min(self.capacity);
    s.last_refill = now;

    // Try to take one
    if s.tokens >= 1.0 {
        s.tokens -= 1.0;
        true
    } else {
        false
    }
}
```

`try_acquire` does all its work under the lock:

1. It computes the time since the last refill, `now - s.last_refill`, a `Duration`.
2. It converts that to seconds and multiplies by the rate: the number of new tokens.
3. It adds them and caps the total at the capacity with `.min(self.capacity)`.
4. It records `now` as the last refill.
5. If at least one token is there, it takes it and returns `true`. Otherwise it returns `false`.

Work through the test by hand. The bucket starts with 5 tokens. Five calls happen within microseconds, so almost no tokens refill, and each call takes one. The sixth finds about 0.00001 tokens and returns `false`. After a one-second sleep, about 1.00001 tokens are there, so one call succeeds, and the next finds about 0.00001 again.

Listing 19.3 The complete file, with its test. `src/problems/rate_limiter.rs`

```

    /// Thread-safe token bucket rate limiter.

    use std::sync::Mutex;
    use std::time::Instant;

    pub struct TokenBucket {
        /// Max tokens.
        capacity: f64,
        /// Tokens per second.
        rate: f64,
        state: Mutex<BucketState>,
    }

    struct BucketState {
        tokens: f64,
        last_refill: Instant,
    }

    impl TokenBucket {
        pub fn new(capacity: usize, rate_per_sec: f64) -> Self {
            assert!(capacity > 0);
            assert!(rate_per_sec > 0.0);
            Self {
                capacity: capacity as f64,
                rate: rate_per_sec,
                state: Mutex::new(BucketState {
                    tokens: capacity as f64, // Start full
                    last_refill: Instant::now(),
                }),
            }
        }

        pub fn try_acquire(&self) -> bool {
            let mut s = self.state.lock().unwrap();
            let now = Instant::now();

            // Refill tokens based on elapsed time
            let elapsed = now - s.last_refill;
            let new_tokens = elapsed.as_secs_f64() * self.rate;
            s.tokens = (s.tokens + new_tokens).min(self.capacity);
            s.last_refill = now;

            // Try to take one
            if s.tokens >= 1.0 {
                s.tokens -= 1.0;
                true
            } else {
                false
            }
        }
    }

    #[cfg(test)]
    mod tests {
        use super::*;
        use std::thread;
        use std::time::Duration;

```

```

#[test]
fn it_works() {
    let bucket = TokenBucket::new(5, 1.0); // 5 tokens, refill 1 per sec

    // Use all 5
    for _ in 0..5 {
        assert!(bucket.try_acquire());
    }
    // 6th fails
    assert!(!bucket.try_acquire());

    // Wait 1 second
    thread::sleep(Duration::from_secs(1));
    // Should have 1 token back
    assert!(bucket.try_acquire());
    // And none left
    assert!(!bucket.try_acquire());
}
}

```

```

$ cargo test --lib problems::rate_limiter
running 1 test
test problems::rate_limiter::tests::it_works ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 136 filtered out; finished in
1.00s

```

The test takes one second, because it sleeps to let a token refill. A test that depends on real time can also fail on a heavily loaded machine. Exercise 1 removes the dependency by passing the time in.

19.1.2 The bucket with atomics

Every call to the locked bucket takes the mutex. Under heavy load, threads wait for each other there, as section 14.1 described for maps. The second version replaces the lock with atomic integers.

Listing 19.4 The limiter (lines 5 to 37). `src/bin/rate_limiter_atomic_token_bucket.rs`

```

#[repr(align(64))]
pub struct RateLimiter {
    tokens: AtomicI64,
    last_refilled_at: AtomicU64, // secs since created_at
    capacity: i64,
    tokens_per_sec: i64,
    created_at: Instant,
}

pub fn new(capacity: i64, tokens_per_sec: i64) -> Self {
    Self {

```

```

        tokens: AtomicI64::new(capacity),
        last_refilled_at: AtomicU64::new(0),
        capacity,
        tokens_per_sec,
        created_at: Instant::now(),
    }
}

pub fn try_acquire(&self) -> bool {
    let now = self.created_at.elapsed().as_secs();
    let last = self.last_refilled_at.fetch_max(now, Relaxed);
    if now > last {
        self.tokens.fetch_add((now - last) as i64 * self.tokens_per_sec, Relaxed);
        self.tokens.fetch_min(self.capacity, Relaxed);
    }

    if self.tokens.load(Relaxed) <= 0 {
        return false;
    }
    self.tokens.fetch_sub(1, Relaxed) > 0
}

```

The design differs from the locked version in three ways:

- Tokens are whole numbers in an `AtomicI64`, and time is whole seconds since the limiter was created. Tokens arrive in steps, once per second, instead of continuously.
- `last_refilled_at.fetch_max(now, Relaxed)` records the current second and returns the previous value, in one atomic step. If several threads start a new second at once, only the first sees an older value, so only that thread adds tokens.
- `#[repr(align(64))]` gives the limiter its own cache lines, so it does not share a line with a neighbor, as section 15.4 described.

Then comes the check. If the count is 0 or less, the call returns `false`. Otherwise it takes a token with `fetch_sub(1)`, and returns `true` if the count before the subtraction was positive.

Listing 19.5 The complete program. `src/bin/rate_limiter_atomic_token_bucket.rs`

```

use std::sync::atomic::{AtomicI64, AtomicU64, Ordering::Relaxed};
use std::time::{Instant, Duration};
use std::thread;

#[repr(align(64))]
pub struct RateLimiter {
    tokens: AtomicI64,
    last_refilled_at: AtomicU64, // secs since created_at
    capacity: i64,
    tokens_per_sec: i64,
    created_at: Instant,
}

impl RateLimiter {

```

```

pub fn new(capacity: i64, tokens_per_sec: i64) -> Self {
    Self {
        tokens: AtomicI64::new(capacity),
        last_refilled_at: AtomicU64::new(0),
        capacity,
        tokens_per_sec,
        created_at: Instant::now(),
    }
}

pub fn try_acquire(&self) -> bool {
    let now = self.created_at.elapsed().as_secs();
    let last = self.last_refilled_at.fetch_max(now, Relaxed);
    if now > last {
        self.tokens.fetch_add((now - last) as i64 * self.tokens_per_sec, Relaxed);
        self.tokens.fetch_min(self.capacity, Relaxed);
    }

    if self.tokens.load(Relaxed) <= 0 {
        return false;
    }
    self.tokens.fetch_sub(1, Relaxed) > 0
}

fn main() {
    let limiter = RateLimiter::new(5, 5);

    for i in 1..=6 {
        println!("request {i}: {}", limiter.try_acquire());
    }

    thread::sleep(Duration::from_secs(1));

    for i in 1..=6 {
        println!("request {i}: {}", limiter.try_acquire());
    }
}

```

```

$ cargo run --bin rate_limiter_atomic_token_bucket
request 1: true
request 2: true
request 3: true
request 4: true
request 5: true
request 6: false
request 1: true
request 2: true
request 3: true
request 4: true
request 5: true
request 6: false

```


With one thread, the output is exactly right: 5 requests pass, the sixth fails, and after a second 5 more pass.

19.1.3 What happens with many threads

Each atomic operation is safe on its own. The problem is that `try_acquire` makes several of them, and other threads can act between them. Figure 19.2 shows one interleaving with one token left.

The atomic version: a check and a take that are two separate steps

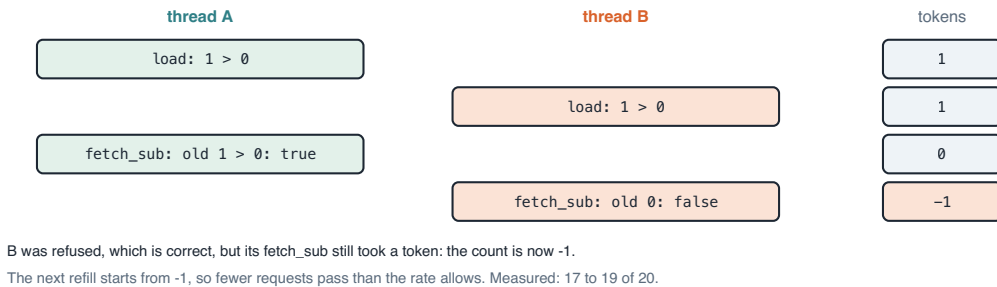


Figure 19.2 Both threads pass the check before either takes the token.

B is correctly refused, but its `fetch_sub` still ran. The count drops below zero, and the next refill starts from there, so the limiter lets through fewer requests than its rate. To measure this, I ran 8 threads calling `try_acquire` in a loop for 3.5 seconds, on a limiter with capacity 5 and rate 5. A correct limiter admits 20 requests: 5 at the start and 5 in each of the next three seconds. Three runs admitted 18, 19, and 17, and ended with the count at 0, -1, and -1.

A second gap goes the other way. The refill is a `fetch_add` followed by a separate `fetch_min`. Between the two, the count can be above the capacity, and other threads can take the extra tokens before the cap is applied. That would let through more than the capacity in one burst. My runs did not catch this window, but nothing in the code prevents it.

Both gaps close if the check, the take, and the refill happen as one atomic step. `compare_exchange` in a loop does that. It reads the count, computes the new count, and writes it only if no other thread changed the count in between. Exercise 2 asks you to write it.

19.2 Retrying failed calls

Many failures are temporary. A server restarts, a network link drops a packet, or a service is briefly overloaded. Trying again a moment later often works.

Retrying well takes three decisions:

1. **Which errors to retry.** A "not found" error will fail again. A "server busy" error may not.
2. **How long to wait.** Waiting a little longer after each failure gives a struggling server time to recover.
3. **How many times.** At some point, the caller should give up and report the error.

19.2.1 Which errors to retry

Listing 19.6 The `Retryable` trait (lines 21 to 43). `src/problems/retry.rs`

```
/// An error that knows whether retrying could help.
pub trait Retryable {
    /// `true` if the operation is worth retrying.
    fn is_retryable(&self) -> bool;

    /// A server-provided minimum delay, such as HTTP `Retry-After`.
    fn retry_after(&self) -> Option<Duration> {
        None
    }
}

impl Retryable for ApiError {
    fn is_retryable(&self) -> bool {
        ApiError::is_retryable(self)
    }

    fn retry_after(&self) -> Option<Duration> {
        match self {
            ApiError::RateLimited { retry_after } => Some(*retry_after),
            _ => None,
        }
    }
}
```

`Retryable` is a trait that an error type implements to answer the first question. `is_retryable` has no default, so each error type must decide. `retry_after` has a default body that returns `None`, so an error type only needs to implement it if the server can send a hint.

The trait is implemented for `ApiError`, the error enum from section 7.7. Its `is_retryable` method returns `true` for a rate limit and for server errors with a status from 500 to 599. `retry_after` returns the delay carried by a `RateLimited` error. That delay usually comes from the HTTP header `Retry-After`, which a server sends to say when to try again.

The line `ApiError::is_retryable(self)` calls the enum's own method, not the trait's. Both have the same name. The path `ApiError::is_retryable` finds the inherent method first, so the trait method does not call itself forever.

19.2.2 How long to wait

A fixed delay has a problem. If a server fails, all its clients fail at the same moment. With a fixed delay they all retry at the same moment too. The server is hit by a wave of retries at the moment it recovers. This is called a **thundering herd**.

Two techniques spread the retries out:

- **Exponential backoff** doubles the delay after each failure: 100 ms, 200 ms, 400 ms, and so on, up to a cap (figure 19.3). The longer the outage, the less often each client tries.
- **Jitter** adds randomness to the delay, so clients that failed together do not retry together (figure 19.4). **Full jitter** picks a random delay between 0 and the backoff.

Exponential backoff: base 100 ms, doubling, capped at 1,000 ms

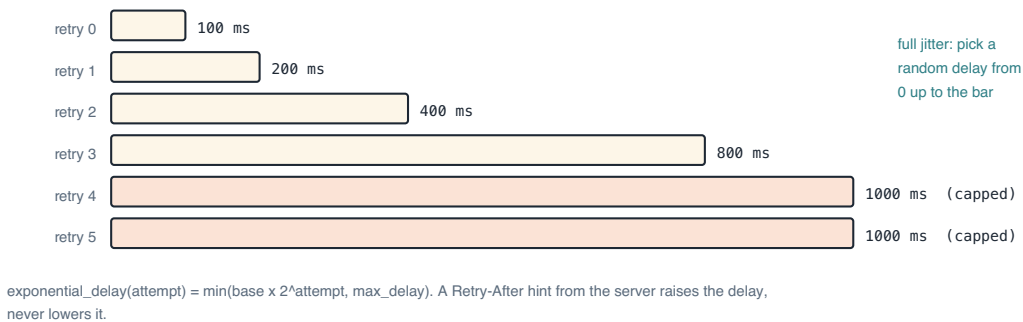
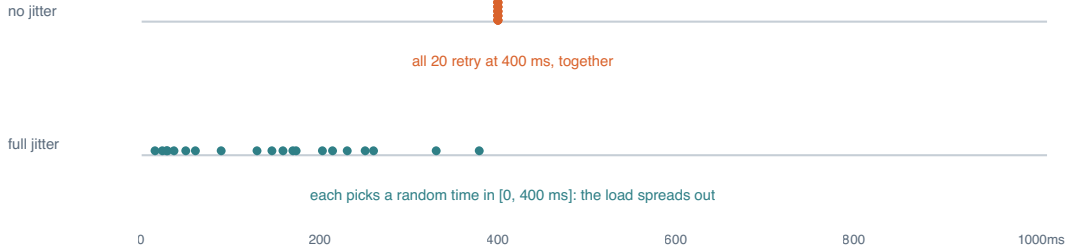


Figure 19.3 The delays from the test `exponential_backoff_doubles_until_capped`.

Twenty clients retry after a failure at time 0**Figure 19.4** The same twenty retries, without and with full jitter.**Listing 19.7** The policy (lines 45 to 99).

```

/// How randomness is applied to the backoff.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Jitter {
    /// Delay is exactly the capped exponential backoff.
    None,
    /// Delay is `unit_random * capped_backoff` (AWS "full jitter").
    Full,
}

/// Backoff configuration.
#[derive(Debug, Clone, Copy)]
pub struct RetryPolicy {
    /// Total attempts, including the first one. Must be at least 1.
    pub max_attempts: u32,
    /// Delay before the first retry.
    pub base_delay: Duration,
    /// Upper bound on a single delay.
    pub max_delay: Duration,
    pub jitter: Jitter,
}

impl RetryPolicy {
    pub fn new(
        max_attempts: u32,
        base_delay: Duration,
        max_delay: Duration,
        jitter: Jitter,
    ) -> Self {
        assert!(max_attempts > 0, "max_attempts must be > 0");
        Self {
            max_attempts,
            base_delay,
            max_delay,
            jitter,
        }
    }
}

/// Capped exponential backoff for a zero-based retry attempt:
/// `min(base * 2^attempt, max_delay)`.

```

```

pub fn exponential_delay(&self, attempt: u32) -> Duration {
    let shift = attempt.min(31);
    let factor = 1u32 << shift;
    self.base_delay.saturating_mul(factor).min(self.max_delay)
}

/// Backoff for `attempt` with `unit_random` in `[0, 1)` applied when jitter
/// is enabled.
pub fn delay_for(&self, attempt: u32, unit_random: f64) -> Duration {
    let capped = self.exponential_delay(attempt);
    match self.jitter {
        Jitter::None => capped,
        Jitter::Full => capped.mul_f64(unit_random.clamp(0.0, 1.0)),
    }
}
}

```

`RetryPolicy` holds the four settings. `max_attempts` counts the first try as well, so 3 means one try and two retries.

`exponential_delay` computes $\text{base} * 2^{\text{attempt}}$, capped at `max_delay`. Three details keep it from overflowing:

- `1u32 << shift` computes 2 to the power `shift` by shifting the bit 1 left. `attempt.min(31)` keeps the shift within a `u32`, because shifting by 32 or more is an error.
- `Duration::saturating_mul` returns the largest `Duration` instead of overflowing.
- `.min(self.max_delay)` applies the cap.

`delay_for` applies jitter. It takes a random number in `[0, 1)` as an argument, and multiplies the delay by it with `Duration::mul_f64`. `clamp(0.0, 1.0)` guards against a random source that returns something out of range.

19.2.3 The retry loop

Listing 19.8 `retry` (lines 101 to 134).

```

/// Retry `operation` until it succeeds, fails permanently, or the policy is
/// exhausted.
///
/// `unit_random` must return values in `[0, 1)`, pass `|| 0.0` for deterministic
/// tests or a real RNG in production.
pub fn retry<T, E, F, R>(policy: &RetryPolicy, mut operation: F, mut unit_random: R) -> Result<T, E>
where
    F: FnMut(u32) -> Result<T, E>,
    E: Retryable,
    R: FnMut() -> f64,
{
    let mut attempt = 0;

```

```

loop {
  match operation(attempt) {
    Ok(value) => return Ok(value),
    Err(error) => {
      let exhausted = attempt + 1 >= policy.max_attempts;
      if exhausted || !error.is_retryable() {
        return Err(error);
      }

      let backoff = policy.delay_for(attempt, unit_random());
      // Honor the server's hint, but never retry sooner than our own
      // backoff says is healthy.
      let delay = error
        .retry_after()
        .map_or(backoff, |hint| hint.max(backoff));

      std::thread::sleep(delay);
      attempt += 1;
    }
  }
}

```

`retry` is generic over four types. `T` and `E` are the success and error types. `F` is the operation, a closure that receives the attempt number. `R` produces random numbers.

The operation is `FnMut`, not `FnOnce`, because the loop calls it several times, and it may change captured state, like the test's attempt counter.

On each error, the loop checks two reasons to stop: the attempts are used up, or the error is not retryable. Either way it returns the last error. Otherwise it computes its own backoff, and then applies the server's hint:

```

let delay = error.retry_after().map_or(backoff, |hint| hint.max(backoff));

```

`map_or(default, f)` returns `default` for `None`, and `f(hint)` for `Some(hint)`. So without a hint, the delay is the backoff. With a hint, it is the larger of the two. The client never retries sooner than the server asked, and never sooner than its own backoff.

`std::thread::sleep(delay)` blocks the thread for the delay. In async code, chapter 22, the wait would be an async timer instead, so the thread can do other work.

Injecting randomness. The random number comes from the `unit_random` closure the caller passes in. Production code passes a real random number generator. Tests pass `|| 0.0`, which makes every delay predictable. Passing in a dependency this way, instead of creating it inside the function, is called **dependency injection**. It makes the function testable without a real clock or a real random source.

Listing 19.9 The complete file, with tests. src/problems/retry.rs

```

///! Retry with exponential backoff, full jitter, and `Retry-After`.
///!
///! Every network client retries. Done badly it makes an outage worse: synchronized
///! retries from thousands of clients form a thundering herd, and retrying a
///! non-idempotent `POST` can create duplicate workloads. This module models the
///! policy explicitly.
///!
///! Two details:
///!
///! - **Jitter.** Full jitter picks a random delay in `[0, capped_backoff]` so
///!   clients spread out instead of retrying in lockstep.
///! - **`Retry-After`.** When the server tells you when to come back, wait at
///!   least that long; never retry sooner than the server asked.
///!
///! The random source is injected as a closure so the delay math is deterministic
///! under test.

use crate::problems::state_machine::ApiError;
use std::time::Duration;

/// An error that knows whether retrying could help.
pub trait Retryable {
    /// `true` if the operation is worth retrying.
    fn is_retryable(&self) -> bool;

    /// A server-provided minimum delay, such as HTTP `Retry-After`.
    fn retry_after(&self) -> Option<Duration> {
        None
    }
}

impl Retryable for ApiError {
    fn is_retryable(&self) -> bool {
        ApiError::is_retryable(self)
    }

    fn retry_after(&self) -> Option<Duration> {
        match self {
            ApiError::RateLimited { retry_after } => Some(*retry_after),
            _ => None,
        }
    }
}

/// How randomness is applied to the backoff.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Jitter {
    /// Delay is exactly the capped exponential backoff.
    None,
    /// Delay is `unit_random * capped_backoff` (AWS "full jitter").
    Full,
}

/// Backoff configuration.
#[derive(Debug, Clone, Copy)]
pub struct RetryPolicy {
    /// Total attempts, including the first one. Must be at least 1.

```

```

    pub max_attempts: u32,
    /// Delay before the first retry.
    pub base_delay: Duration,
    /// Upper bound on a single delay.
    pub max_delay: Duration,
    pub jitter: Jitter,
}

impl RetryPolicy {
    pub fn new(
        max_attempts: u32,
        base_delay: Duration,
        max_delay: Duration,
        jitter: Jitter,
    ) -> Self {
        assert!(max_attempts > 0, "max_attempts must be > 0");
        Self {
            max_attempts,
            base_delay,
            max_delay,
            jitter,
        }
    }

    /// Capped exponential backoff for a zero-based retry attempt:
    /// `min(base * 2^attempt, max_delay)`.
    pub fn exponential_delay(&self, attempt: u32) -> Duration {
        let shift = attempt.min(31);
        let factor = 1u32 << shift;
        self.base_delay.saturating_mul(factor).min(self.max_delay)
    }

    /// Backoff for `attempt` with `unit_random` in `[0, 1)` applied when jitter
    /// is enabled.
    pub fn delay_for(&self, attempt: u32, unit_random: f64) -> Duration {
        let capped = self.exponential_delay(attempt);
        match self.jitter {
            Jitter::None => capped,
            Jitter::Full => capped.mul_f64(unit_random.clamp(0.0, 1.0)),
        }
    }
}

/// Retry `operation` until it succeeds, fails permanently, or the policy is
/// exhausted.
///
/// `unit_random` must return values in `[0, 1)`, pass `|| 0.0` for deterministic
/// tests or a real RNG in production.
pub fn retry<T, E, F, R>(policy: &RetryPolicy, mut operation: F, mut unit_random: R) -> Result<T, E>
where
    F: FnMut(u32) -> Result<T, E>,
    E: Retryable,
    R: FnMut() -> f64,
{
    let mut attempt = 0;
    loop {
        match operation(attempt) {
            Ok(value) => return Ok(value),

```



```

        Err(error) => {
            let exhausted = attempt + 1 >= policy.max_attempts;
            if exhausted || !error.is_retryable() {
                return Err(error);
            }

            let backoff = policy.delay_for(attempt, unit_random());
            // Honor the server's hint, but never retry sooner than our own
            // backoff says is healthy.
            let delay = error
                .retry_after()
                .map_or(backoff, |hint| hint.max(backoff));

            std::thread::sleep(delay);
            attempt += 1;
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    fn policy(max_attempts: u32, jitter: Jitter) -> RetryPolicy {
        RetryPolicy::new(
            max_attempts,
            Duration::from_millis(100),
            Duration::from_millis(1_000),
            jitter,
        )
    }

    #[test]
    fn exponential_backoff_doubles_until_capped() {
        let policy = policy(6, Jitter::None);
        assert_eq!(policy.exponential_delay(0), Duration::from_millis(100));
        assert_eq!(policy.exponential_delay(1), Duration::from_millis(200));
        assert_eq!(policy.exponential_delay(2), Duration::from_millis(400));
        assert_eq!(policy.exponential_delay(3), Duration::from_millis(800));
        // Capped from here on.
        assert_eq!(policy.exponential_delay(4), Duration::from_millis(1_000));
        assert_eq!(policy.exponential_delay(50), Duration::from_millis(1_000));
    }

    #[test]
    fn full_jitter_scales_the_delay() {
        let policy = policy(6, Jitter::Full);
        assert_eq!(policy.delay_for(2, 0.0), Duration::ZERO);
        assert_eq!(policy.delay_for(2, 0.5), Duration::from_millis(200));
        // Values above 1.0 are clamped rather than overflowing.
        assert_eq!(policy.delay_for(2, 9.9), Duration::from_millis(400));
    }

    #[test]
    fn retries_then_succeeds() {
        let policy = RetryPolicy::new(5, Duration::ZERO, Duration::ZERO, Jitter::None);
        let mut attempts = 0;
        let result: Result<u32, ApiError> = retry(

```

```

        &policy,
        |_| {
            attempts += 1;
            if attempts < 3 {
                Err(ApiError::Server {
                    status: 503,
                    message: "temporarily unavailable".into(),
                })
            } else {
                Ok(42)
            }
        },
        || 0.0,
    );

    assert_eq!(result, Ok(42));
    assert_eq!(attempts, 3);
}

#[test]
fn non_retryable_errors_return_immediately() {
    let policy = RetryPolicy::new(5, Duration::ZERO, Duration::ZERO, Jitter::None);
    let mut attempts = 0;
    let result: Result<(), ApiError> = retry(
        &policy,
        |_| {
            attempts += 1;
            Err(ApiError::NotFound {
                resource_id: "item-0".into(),
            })
        },
        || 0.0,
    );

    assert!(result.is_err());
    assert_eq!(attempts, 1);
}

#[test]
fn exhausted_policy_returns_the_last_error() {
    let policy = RetryPolicy::new(3, Duration::ZERO, Duration::ZERO, Jitter::None);
    let mut attempts = 0;
    let result: Result<(), ApiError> = retry(
        &policy,
        |_| {
            attempts += 1;
            Err(ApiError::Server {
                status: 500,
                message: "boom".into(),
            })
        },
        || 0.0,
    );

    assert!(result.is_err());
    assert_eq!(attempts, 3);
}

#[test]

```

```

fn retry_after_is_honored_as_a_floor() {
    let policy = policy(2, Jitter::None);
    let error = ApiError::RateLimited {
        retry_after: Duration::from_secs(5),
    };
    let backoff = policy.delay_for(0, 0.0);
    let chosen = error
        .retry_after()
        .map_or(backoff, |hint| hint.max(backoff));
    assert_eq!(chosen, Duration::from_secs(5));
}
}

```

The tests use a policy with zero delays, so they run instantly. `retries_then_succeeds` fails twice with a 503 and then succeeds: three attempts in total. `non_retryable_errors_return_immediately` returns `NotFound`, and must stop after one attempt.

```

$ cargo test --lib problems::retry
running 6 tests
test problems::retry::tests::full_jitter_scales_the_delay ... ok
test problems::retry::tests::exponential_backoff_doubles_until_capped ... ok
test problems::retry::tests::retry_after_is_honored_as_a_floor ... ok
test problems::retry::tests::non_retryable_errors_return_immediately ... ok
test problems::retry::tests::exhausted_policy_returns_the_last_error ... ok
test problems::retry::tests::retries_then_succeeds ... ok

test result: ok. 6 passed; 0 failed; 0 ignored; 0 measured; 131 filtered out

```

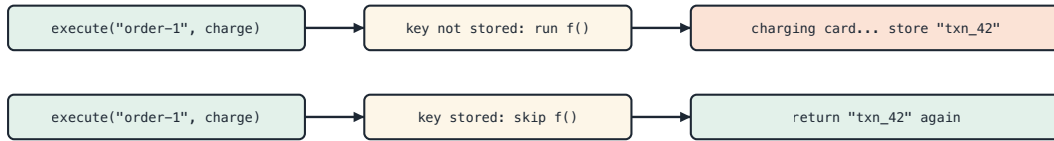
19.3 Idempotency keys

Retries have a danger. Suppose a client asks a server to charge a card, and the reply is lost. The client cannot tell whether the charge happened. If it retries, the card may be charged twice.

An operation is **idempotent** if doing it twice has the same effect as doing it once. Reading a record is idempotent. Charging a card is not, unless the server makes it so.

The usual way is an **idempotency key**. The client picks a unique key for each logical request, such as an order number, and sends it with every attempt. The server records the result under that key. When the same key arrives again, the server returns the recorded result and does not repeat the work (figure 19.5).

An idempotency key: the second call with the same key returns the stored result



A client that retries after a timeout sends the same key. The card is charged once, and both calls get the same answer.

A failed `f()` stores nothing, so the next call with that key tries again.

Figure 19.5 Two calls with one key. The work runs once.

Listing 19.10 The complete program. `src/bin/idempotent_operation.rs`

```

use std::collections::HashMap;
use std::hash::Hash;
use std::sync::Mutex;

pub struct Idempotent<K, V> {
    store: Mutex<HashMap<K, V>>,
}

impl<K, V> Idempotent<K, V>
where
    K: Eq + Hash + Clone,
    V: Clone,
{
    pub fn new() -> Self {
        Self { store: Mutex::new(HashMap::new()) }
    }

    pub fn execute<F>(&self, key: K, f: F) -> Result<V, Box<dyn std::error::Error>>
    where
        F: FnOnce() -> Result<V, Box<dyn std::error::Error>>,
    {
        let mut store = self.store.lock().map_err(|_| "lock poisoned")?;
        if let Some(v) = store.get(&key) {
            return Ok(v.clone());
        }
        let v = f()?;
        store.insert(key, v.clone());
        Ok(v)
    }
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let idem = Idempotent::new();

    let charge = || -> Result<String, Box<dyn std::error::Error>> {
        println!("charging card...");
        Ok("txn_42".into())
    };
};
  
```

```

    assert_eq!(idem.execute("order-1", charge)?, "txn_42");
    assert_eq!(idem.execute("order-1", charge)?, "txn_42");
    Ok(())
}

```

`Idempotent<K, V>` stores results in a `HashMap` behind a `Mutex`. `execute` takes a key and a closure `f` that does the work:

1. It locks the store. A poisoned lock becomes an error: `map_err(|_| "lock poisoned")?` turns the `PoisonError` into a `&str`, and `?` converts that into a `Box<dyn Error>`.
2. If the key is present, it returns a clone of the stored result.
3. Otherwise it runs `f()`. If `f` fails, `?` returns the error, and nothing is stored, so a later call can try again.
4. It stores a clone of the result and returns it.

In `main`, `charge` is called with the same key twice. The closure captures nothing, so it is `Copy`, and passing it to `execute` twice copies it. The line "charging card..." appears once:

```

$ cargo run --bin idempotent_operation
charging card...

```

The second file is the same program with a type alias, `type RuntimeError = Box<dyn std::error::Error>;`, which shortens the signatures:

Listing 19.11 The complete program. `src/bin/idempotent_operation_with_error_propagation.rs`

```

use std::collections::HashMap;
use std::hash::Hash;
use std::sync::Mutex;

type RuntimeError = Box<dyn std::error::Error>;

pub struct Idempotent<K, V> {
    store: Mutex<HashMap<K, V>>,
}

impl<K, V> Idempotent<K, V>
where
    K: Eq + Hash + Clone,
    V: Clone,
{
    pub fn new() -> Self {
        Self { store: Mutex::new(HashMap::new()) }
    }

    pub fn execute<F>(&self, key: K, f: F) -> Result<V, RuntimeError>
    where
        F: FnOnce() -> Result<V, RuntimeError>,

```

```

    {
        let mut store = self.store.lock().map_err(|_| "lock poisoned"?;
        if let Some(v) = store.get(&key) {
            return Ok(v.clone());
        }
        let v = f()?;
        store.insert(key, v.clone());
        Ok(v)
    }
}

fn main() -> Result<(), RuntimeError> {
    let idem = Idempotent::new();

    let charge = || -> Result<String, RuntimeError> {
        println!("charging card...");
        Ok("txn_42".into())
    };

    assert_eq!(idem.execute("order-1", charge)?, "txn_42");
    assert_eq!(idem.execute("order-1", charge)?, "txn_42");
    Ok(())
}

```

```

$ cargo run --bin idempotent_operation_with_error_propagation
charging card...

```

Holding the lock while `f()` runs makes the store correct. Two calls with the same key cannot both find it missing and both run the work. It is also the store's limit. Every call waits for the one lock, even calls with different keys, so one slow operation delays all others. A production store locks per key instead. It records "in progress" for a key and releases the lock while the work runs. Later callers with that key wait for the result. A real service also keeps the results in a database, so they survive a restart.

Summary

- A token bucket allows bursts up to its capacity and a long-run rate. Each call adds the tokens earned since the last call before it checks.
- The locked bucket is correct and simple. The atomic bucket avoids the lock, but its separate check, take, and refill steps let the count go negative. With 8 threads it admitted 17 to 19 requests where 20 were due.
- Retry only errors that can succeed later. A `Retryable` trait lets each error type decide, and carry a server hint.

- Exponential backoff with full jitter spreads retries out and prevents a thundering herd. The server's `Retry-After` is a floor on the delay.
- Passing randomness in as a closure makes the retry logic testable.
- An idempotency key makes a retried request safe: the server stores the result under the key and returns it for every repeat.

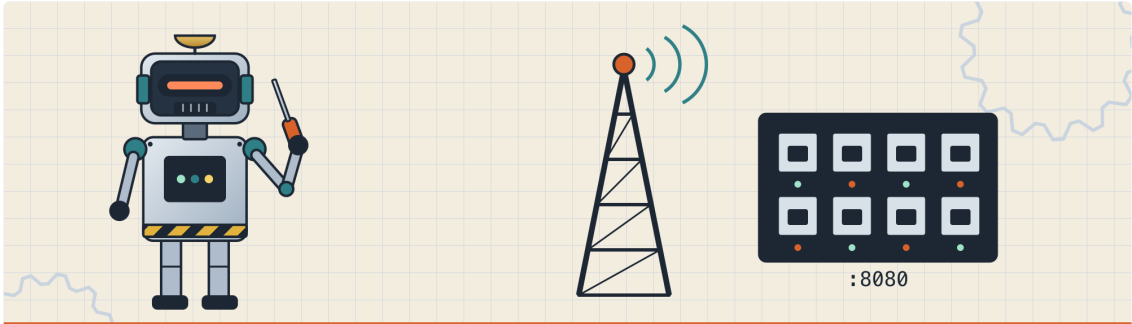
Chapter 20 moves down to the network itself: IP addresses, sockets, and a server that echoes back what it receives.

Exercises

1. Change `TokenBucket::try_acquire` to take the current time as an argument, `try_acquire_at(&self, now: Instant)`. Rewrite the test without `thread::sleep`.
2. Rewrite the atomic limiter's `try_acquire` with a `compare_exchange` loop that refills, checks, and takes a token in one atomic step. Run the 8-thread test from section 19.1.3 on it.
3. Add a `max_elapsed: Duration` to `RetryPolicy`, and make `retry` stop when the total time spent would pass it.
4. Change `Idempotent` so that `execute` does not hold the lock while `f` runs. Two calls with the same key must still run `f` only once. Hint: store an enum with `InProgress` and `Done(V)` states, and a `Condvar`.

PART 6

The network



Addresses, windows, and echo servers

THIS CHAPTER COVERS

- Parsing IPv4 addresses with `??`, matching private ranges with slice patterns, and byte order
- The effective TCP window and the bandwidth-delay product
- A thread-per-connection echo server and the TCP lifecycle it walks through
- An epoll echo server over raw `libc` calls: non-blocking sockets, `EAGAIN`, and write buffers
- Where each model breaks, and what a production server adds

Networking questions start at the bottom of the stack and work up. What is an IPv4 address as a number? Why does a fast link with high latency deliver less than its bandwidth? What happens between `bind` and the first byte echoed back? How does one thread serve ten thousand connections? The four programs in this chapter answer those in order. The last one uses no runtime and no crate beyond `libc`, so every system call is visible.

16.1 IPv4 addresses

Listing 16.1 Parse, classify, and convert IPv4 addresses. `src/bin/net_ipv4.rs`

[illegible]

```

/// Parse `a.b.c.d`, rejecting anything that is not exactly four octets.
fn parse(input: &str) -> Option<[u8; 4]> {
    let mut parts = input.split('.').map(|part| part.parse::<u8>().ok());
    let address = [
        parts.next()?,
        parts.next()?,
        parts.next()?,
        parts.next()?,
    ];
    parts.next().is_none().then_some(address)
}

/// `10.0.0.0/8`, `172.16.0.0/12`, and `192.168.0.0/16` are not routable on the
/// public internet.
fn is_private(address: [u8; 4]) -> bool {
    matches!(address, [10, ..] | [172, 16..=31, ..] | [192, 168, ..])
}

/// Addresses travel big-endian, so the first octet is the most significant byte.
fn to_network_order(address: [u8; 4]) -> u32 {
    u32::from_be_bytes(address)
}

fn from_network_order(value: u32) -> [u8; 4] {
    value.to_be_bytes()
}

fn main() {
    println!(
        "{:<16} {:<18} {:>8} {:>12}",
        "input", "parsed", "private", "as u32"
    );

    for text in [
        "10.0.0.1",
        "172.16.5.4",
        "192.168.1.10",
        "8.8.8.8",
        "256.1.1.1",
        "1.2.3",
    ] {
        match parse(text) {
            Some(address) => println!(
                "{text:<16} {:<18} {:>8} {:>12}",
                format!("{address:?}"),
                is_private(address),
                to_network_order(address)
            ),
            None => println!("{text:<16} invalid"),
        }
    }

    assert_eq!(parse("10.0.0.1"), Some([10, 0, 0, 1]));
    assert_eq!(parse("1.2.3"), None);
    assert_eq!(parse("1.2.3.4.5"), None);
    assert_eq!(parse("256.0.0.1"), None);

    assert!(is_private([10, 1, 2, 3]));

```

```

assert!(is_private([172, 31, 0, 1]));
assert!(!is_private([172, 32, 0, 1]));
assert!(!is_private([8, 8, 8, 8]));

assert_eq!(to_network_order([127, 0, 0, 1]), 0x7f00_0001);
assert_eq!(from_network_order(0x7f00_0001), [127, 0, 0, 1]);

println!("\nall checks passed");
}

```

`parse` is four `part.next()??` expressions and a check that nothing is left over. The iterator yields `Option<u8>` (the result of `part.parse::<u8>().ok()`), so `next()` returns `Option<Option<u8>>`. The first `?` returns `None` if there is no fourth part. The second returns `None` if a part is not a valid `u8`, which is how `"256.1.1.1"` is rejected. `parts.next().is_none().then_some(address)` rejects a fifth part.

`is_private` is one `matches!` over array patterns. `[10, ..]` matches any address starting with 10. `[172, 16..=31, ..]` uses a range pattern for the second octet: exactly the /12 block from 172.16 to 172.31. Rust's patterns are shorter here than the bit masks they replace.

Network byte order is big-endian: the first octet is the most significant byte. `u32::from_be_bytes` turns the four octets into that number on any machine, whatever its native order (figure 16.1).

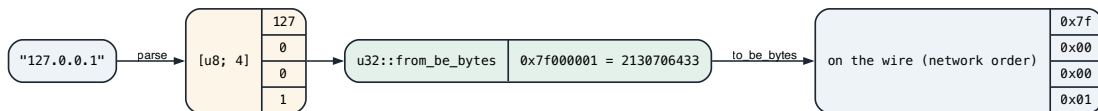


Figure 16.1 An address as text, as octets, as a `u32`, and on the wire.

input	parsed	private	as u32
10.0.0.1	[10, 0, 0, 1]	true	167772161
172.16.5.4	[172, 16, 5, 4]	true	2886731012
8.8.8.8	[8, 8, 8, 8]	false	134744072
256.1.1.1	invalid		
1.2.3	invalid		

`str::parse::<u8>` accepts a leading `+` and leading zeros, so `"1.02.3.4"` parses. The standard library's `"1.2.3.4".parse::<std::net::Ipv4Addr>()` rejects both, and `Ipv4Addr::is_private` implements the same three ranges. Write your own version; name the standard one.

16.2 How much data can be in flight

Listing 16.2 The effective window and the bandwidth-delay product. `src/bin/net_window.rs`

```

//! TCP window math: the effective window is the smaller of the receiver and
//! congestion windows, and the bandwidth-delay product is how many bytes must be
//! in flight to fill a link.
//!
//! Run with: cargo run --bin net_window

//! The sender may have this much unacknowledged data in flight.
fn effective_window(receive: u32, congestion: u32) -> u32 {
    receive.min(congestion)
}

//! Bytes in flight needed to keep a link busy: bandwidth times round-trip time.
fn bandwidth_delay_product(bytes_per_second: u64, rtt_millis: u64) -> u64 {
    bytes_per_second * rtt_millis / 1_000
}

fn main() {
    println!("effective window = min(receiver window, congestion window)");
    for (receive, congestion) in [(64_000, 16_000), (16_000, 64_000), (32_000, 32_000)] {
        println!(
            "    receiver {receive:>7} B, congestion {congestion:>7} B -> {:>7} B",
            effective_window(receive, congestion)
        );
    }

    println!("\nbandwidth-delay product (bytes needed in flight)");
    for (rate, rtt) in [(1_000_000, 1), (10_000_000, 50), (1_000_000_000, 100)] {
        println!(
            "    {:>12} B/s x {rtt:>3} ms -> {:>12} B",
            rate,
            bandwidth_delay_product(rate, rtt)
        );
    }

    // A window smaller than the product leaves the link idle, no matter how much
    // bandwidth is available. This is why latency, not only bandwidth, matters.
    let product = bandwidth_delay_product(1_000_000_000, 100);
    let window = u64::from(effective_window(64_000, 1_000_000));
    println!(
        "\nwindow {window} B vs product {product} B: link is {} the window too small",
        if window < product {
            "starved,"
        } else {
            "usable,"
        }
    );

    assert_eq!(effective_window(64_000, 16_000), 16_000);
    assert_eq!(effective_window(16_000, 64_000), 16_000);
    assert_eq!(bandwidth_delay_product(10_000_000, 50), 500_000);
    assert!(window < product);
}

```

```
println!("\nall checks passed");
}
```

A TCP sender may have at most `min(receive window, congestion window)` bytes unacknowledged. The receive window is flow control: how much the receiver can buffer. The congestion window is the sender's estimate of what the network can carry. Once that many bytes are in flight, the sender stops and waits for acknowledgments.

The bandwidth-delay product is how many bytes a link holds while they travel: bandwidth times round-trip time. If the window is smaller than the product, the sender runs out of window before the first acknowledgment returns. The link then sits idle for the rest of each round trip. The program's last case makes it concrete (figure 16.2).

1 GB/s link, 100 ms round trip: 100,000,000 bytes must be in flight



64,000 B window: 0.064% of the pipe is full, the sender idles until ACKs return

throughput = window / RTT = 64,000 B / 0.1 s = 640 KB/s on a 1 GB/s link

fix: window scaling (RFC 7323) lets the window reach the bandwidth-delay product

Figure 16.2 A 64,000-byte window on a link whose bandwidth-delay product is 100,000,000 bytes.

```
window 64000 B vs product 100000000 B: link is starved, the window too small
```

Latency costs as much as bandwidth for bulk transfers. That is why TCP's window-scaling option exists: the original 16-bit window field caps at 65,535 bytes. On Linux, `net.ipv4.tcp_rmem` and `tcp_wmem` bound how large the kernel lets the windows grow.

`bandwidth_delay_product` multiplies before it divides, `bytes_per_second * rtt_millis / 1_000`, which keeps precision in integer arithmetic. The intermediate product overflows `u64` only when bytes per second times milliseconds passes about 1.8×10^{19} , far beyond any real link.

16.3 A thread-per-connection echo server

Listing 16.3 The canonical socket exercise with `std::net`. `src/bin/tcp_echo_server.rs`

```

///! TCP echo server: the canonical socket exercise.
///!
///! Demonstrates the server side of the TCP lifecycle: `bind` -> `listen` ->
///! `accept` -> read/write loop -> peer `close` (a `read` of 0). It uses a
///! thread-per-connection model, which is the right default for bounded concurrency
///! and blocking handlers; the event-loop alternative is `src/bin/epoll_echo.rs`.
///!
///! `TCP_NODELAY` is set because an echo server must not sit on Nagle's algorithm
///! waiting to coalesce a reply that will never be followed by more data.
///!
///! ```bash
///! cargo run --bin tcp_echo_server          # binds 127.0.0.1:0, prints the port
///! cargo run --bin tcp_echo_server 0.0.0.0:9000
///! ```

use std::io::{Read, Write};
use std::net::{TcpListener, TcpStream};
use std::thread;

///! Accept connections forever, handling each on its own thread.
pub fn serve(listener: TcpListener) {
    for incoming in listener.incoming() {
        match incoming {
            Ok(stream) => {
                thread::spawn(move || {
                    if let Err(error) = handle_connection(stream) {
                        eprintln!("connection error: {error}");
                    }
                });
            }
            Err(error) => eprintln!("accept error: {error}"),
        }
    }
}

///! Echo every byte back until the peer closes the connection.
fn handle_connection(mut stream: TcpStream) -> std::io::Result<> {
    stream.set_nodelay(true)?;
    if let Ok(peer) = stream.peer_addr() {
        println!("connection from {peer}");
    }

    let mut buffer = [0u8; 4096];
    loop {
        let read = stream.read(&mut buffer)?;
        if read == 0 {
            // A zero-byte read is the peer's FIN: an orderly close.
            return Ok(());
        }
        stream.write_all(&buffer[..read])?;
    }
}

```

```

fn main() {
    let address = std::env::args()
        .nth(1)
        .unwrap_or_else(|| "127.0.0.1:0".to_string());

    let listener = TcpListener::bind(&address).expect("failed to bind");
    let local = listener.local_addr().expect("failed to read local address");
    println!("tcp echo server listening on {local}");

    serve(listener);
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::net::Shutdown;

    #[test]
    fn echoes_bytes_until_the_client_closes() {
        let listener = TcpListener::bind("127.0.0.1:0").expect("bind ephemeral port");
        let address = listener.local_addr().expect("local address");

        // Accept exactly one connection, echo it, then return so the test can join.
        let server = thread::spawn(move || {
            let (stream, _) = listener.accept().expect("accept one connection");
            handle_connection(stream).expect("echo");
        });

        let mut client = TcpStream::connect(address).expect("connect");
        client.write_all(b"hello echo").expect("write");
        client.shutdown(Shutdown::Write).expect("half close");

        let mut echoed = Vec::new();
        client.read_to_end(&mut echoed).expect("read echo");

        server.join().expect("server thread panicked");
        assert_eq!(echoed, b"hello echo");
    }
}

```

`TcpListener::bind` performs three system calls: `socket`, `bind`, and `listen`. Binding to port 0 asks the kernel for any free port. `local_addr()` reports which one it chose, which is how both `main` and the test avoid colliding with other programs. `listener.incoming()` is an endless iterator over `accept`, and each accepted stream moves into its own thread.

The TCP three-way handshake and the close

Sequence numbers count bytes. Each side must hear the other at least once.



A client and a server. The client knows the server's address and port.

The handshake is what makes both sides agree on where the byte stream starts.

messages = 0 state = **closed**

Animation 16.3 What happens between `accept` returning and the first byte echoed back. Each arrow crosses the two lifelines, and the labels carry the sequence numbers. Three arrows are needed before either side can send data: the client proposes a number, the server acknowledges it and proposes its own, and the client acknowledges that. The close at the end is a matching exchange, which is why a socket that has sent FIN can still read until its peer sends one.

`handle_connection` is the whole protocol:

- `set_nodelay(true)` disables Nagle's algorithm. Nagle delays small writes to coalesce them. That is wrong for an echo server whose reply is never followed by more data, and the module comment says so.
- `read` returns how many bytes arrived, which can be anything from 1 to the buffer size, no matter how the client wrote them. TCP is a byte stream with no message boundaries.
- `read` returning 0 means the peer sent FIN: an orderly close. The function returns `Ok(())` and the stream is closed when it is dropped.

- `write_all` loops over partial writes until every byte is sent.

The test is a model of how to test a server. It binds an ephemeral port, runs `handle_connection` for exactly one connection in a thread, writes "hello gpu", and then half-closes with `shutdown(Shutdown::Write)`. The half-close sends FIN, so the server's `read` returns 0 and the server finishes. The client's read side stays open, so `read_to_end` can collect the echo.

Thread per connection is the right default when connections are few or handlers block on other I/O. Each thread costs a kernel stack and scheduling work. At thousands of mostly idle connections, the model spends its memory on stacks that wait. That is the case for the next program.

16.4 An epoll echo server

Listing 16.4 One thread, many non-blocking sockets, over `libc`.src/bin/epoll_echo.rs

```

//! epoll echo server: one thread, many non-blocking sockets.
//!
//! Reads run until `EAGAIN`, and a reply that does not fit the socket buffer is
//! queued and finished under `EPOLLOUT`, so one slow client cannot stall the loop.
//! Linux only; kqueue and IOCP are the portable equivalents.

#[cfg(target_os = "linux")]
mod linux {
    use std::collections::HashMap;
    use std::io;
    use std::sync::atomic::{AtomicBool, Ordering};

    const MAX_EVENTS: usize = 64;
    const EPOLL_TIMEOUT_MS: i32 = 100;

    /// Per-connection state. `out`/`out_pos` form a write buffer that survives a
    /// partial write.
    #[derive(Default)]
    struct Connection {
        out: Vec<u8>,
        out_pos: usize,
    }

    impl Connection {
        fn has_pending_output(&self) -> bool {
            self.out_pos < self.out.len()
        }
    }

    /// Bind a non-blocking listener on `127.0.0.1:port`; port 0 asks the kernel for
    /// a free port, which is returned alongside the fd.
    pub fn bind_listener(port: u16) -> io::Result<(i32, u16)> {

```

```

// SAFETY: all pointers are to stack values that outlive the call.
unsafe {
    let fd = libc::socket(libc::AF_INET, libc::SOCK_STREAM | libc::SOCK_NONBLOCK,
0);

    if fd < 0 {
        return Err(io::Error::last_os_error());
    }

    let enable: libc::c_int = 1;
    if libc::setsockopt(
        fd,
        libc::SOL_SOCKET,
        libc::SO_REUSEADDR,
        &enable as *const _ as *const libc::c_void,
        std::mem::size_of::<libc::c_int>() as libc::socklen_t,
    ) < 0
    {
        let error = io::Error::last_os_error();
        libc::close(fd);
        return Err(error);
    }

    let mut address: libc::sockaddr_in = std::mem::zeroed();
    address.sin_family = libc::AF_INET as libc::sa_family_t;
    address.sin_port = port.to_be();
    address.sin_addr.s_addr = libc::INADDR_LOOPBACK.to_be();

    if libc::bind(
        fd,
        &address as *const _ as *const libc::sockaddr,
        std::mem::size_of::<libc::sockaddr_in>() as libc::socklen_t,
    ) < 0
    {
        let error = io::Error::last_os_error();
        libc::close(fd);
        return Err(error);
    }

    if libc::listen(fd, 128) < 0 {
        let error = io::Error::last_os_error();
        libc::close(fd);
        return Err(error);
    }

    let mut bound: libc::sockaddr_in = std::mem::zeroed();
    let mut length = std::mem::size_of::<libc::sockaddr_in>() as libc::socklen_t;
    if libc::getsockname(fd, &mut bound as *mut _ as *mut libc::sockaddr, &mut len
gth) < 0 {
        let error = io::Error::last_os_error();
        libc::close(fd);
        return Err(error);
    }

    Ok((fd, u16::from_be(bound.sin_port)))
}

}

/// Run the event loop until `shutdown` becomes true.
pub fn event_loop(epfd: i32, listen_fd: i32, shutdown: &AtomicBool) -> io::Result<>

```

```

{
    epoll_add(epfd, listen_fd, libc::EPOLLIN as u32)?;

    let mut connections: HashMap<i32, Connection> = HashMap::new();
    let mut events = vec![libc::epoll_event { events: 0, u64: 0 }; MAX_EVENTS];

    while !shutdown.load(Ordering::Relaxed) {
        // SAFETY: `events` is a valid, writable buffer of MAX_EVENTS entries.
        let ready = unsafe {
            libc::epoll_wait(
                epfd,
                events.as_mut_ptr(),
                events.len() as i32,
                EPOLL_TIMEOUT_MS,
            )
        };

        if ready < 0 {
            let error = io::Error::last_os_error();
            if error.raw_os_error() == Some(libc::EINTR) {
                continue;
            }
            return Err(error);
        }

        for event in &events[..ready as usize] {
            let fd = event.u64 as i32;
            if fd == listen_fd {
                accept_ready(epfd, listen_fd, &mut connections)?;
                continue;
            }

            let readable = event.events & (libc::EPOLLIN as u32) != 0;
            let writable = event.events & (libc::EPOLLOUT as u32) != 0;

            if readable && !read_into(fd, &mut connections)? {
                continue; // connection closed
            }
            if writable && connections.contains_key(&fd) {
                flush(fd, &mut connections)?;
            }
            if connections.contains_key(&fd) {
                update_interest(epfd, fd, &mut connections)?;
            }
        }
    }

    Ok(())
}

/// Accept every pending connection, then return; `accept4` with
/// `SOCK_NONBLOCK` means a client socket can never block the loop.
fn accept_ready(
    epfd: i32,
    listen_fd: i32,
    connections: &mut HashMap<i32, Connection>,
) -> io::Result<()> {
    loop {
        // SAFETY: a null sockaddr is explicitly allowed for accept.

```

```

        let client = unsafe {
            libc::accept4(
                listen_fd,
                std::ptr::null_mut(),
                std::ptr::null_mut(),
                libc::SOCK_NONBLOCK,
            )
        };

        if client < 0 {
            if would_block() {
                return Ok(());
            }
            return Err(io::Error::last_os_error());
        }

        epoll_add(epfd, client, libc::EPOLLIN as u32)?;
        connections.insert(client, Connection::default());
    }
}

/// Drain readable bytes into the connection's output buffer.
///
/// Returns `false` when the peer closed the connection.
fn read_into(fd: i32, connections: &mut HashMap<i32, Connection>) -> io::Result<bool>
{
    let mut buffer = [0u8; 4096];
    loop {
        // SAFETY: `buffer` is a valid writable region of `buffer.len()` bytes.
        let read =
            unsafe { libc::read(fd, buffer.as_mut_ptr() as *mut libc::c_void, buffer.l
en()) };

        if read == 0 {
            close_connection(fd, connections);
            return Ok(false);
        }
        if read < 0 {
            if would_block() {
                break;
            }
            close_connection(fd, connections);
            return Err(io::Error::last_os_error());
        }

        if let Some(connection) = connections.get_mut(&fd) {
            connection.out.extend_from_slice(&buffer[..read as usize]);
        }
    }

    flush(fd, connections)?;
    Ok(true)
}

/// Write as much of the buffered reply as the socket accepts.
fn flush(fd: i32, connections: &mut HashMap<i32, Connection>) -> io::Result<()> {
    let Some(connection) = connections.get_mut(&fd) else {
        return Ok(());
    };
};

```

```

while connection.has_pending_output() {
    let remaining = connection.out.len() - connection.out_pos;
    // SAFETY: the slice is valid for `remaining` bytes.
    let written = unsafe {
        libc::write(
            fd,
            connection.out[connection.out_pos..].as_ptr() as *const libc::c_void,
            remaining,
        )
    };

    if written < 0 {
        if would_block() {
            // Keep the rest buffered; EPOLLOUT will drive the next attempt.
            return Ok(());
        }
        return Err(io::Error::last_os_error());
    }
    connection.out_pos += written as usize;
}

connection.out.clear();
connection.out_pos = 0;
Ok(())
}

/// Ask for `EPOLLOUT` only while a reply is still buffered.
fn update_interest(
    epfd: i32,
    fd: i32,
    connections: &mut HashMap<i32, Connection>,
) -> io::Result<()> {
    let pending = connections
        .get(&fd)
        .is_some_and(Connection::has_pending_output);
    let events = if pending {
        (libc::EPOLLIN | libc::EPOLLOUT) as u32
    } else {
        libc::EPOLLIN as u32
    };
    epoll_modify(epfd, fd, events)
}

fn close_connection(fd: i32, connections: &mut HashMap<i32, Connection>) {
    connections.remove(&fd);
    // SAFETY: `fd` is owned by us and open at this point.
    unsafe {
        libc::close(fd);
    }
}

fn epoll_add(epfd: i32, fd: i32, events: u32) -> io::Result<()> {
    let mut event = libc::epoll_event {
        events,
        u64: fd as u64,
    };
    // SAFETY: `event` outlives the call.
    if unsafe { libc::epoll_ctl(epfd, libc::EPOLL_CTL_ADD, fd, &mut event) } < 0 {

```

```

        Err(io::Error::last_os_error())
    } else {
        Ok(())
    }
}

fn epoll_modify(epfd: i32, fd: i32, events: u32) -> io::Result<()> {
    let mut event = libc::epoll_event {
        events,
        u64: fd as u64,
    };
    // SAFETY: `event` outlives the call.
    if unsafe { libc::epoll_ctl(epfd, libc::EPOLL_CTL_MOD, fd, &mut event) } < 0 {
        Err(io::Error::last_os_error())
    } else {
        Ok(())
    }
}

fn would_block() -> bool {
    matches!(
        io::Error::last_os_error().raw_os_error(),
        Some(libc::EAGAIN)
    )
}

}

#[cfg(target_os = "linux")]
fn main() {
    use std::sync::atomic::AtomicBool;

    let port: u16 = std::env::args()
        .nth(1)
        .and_then(|argument| argument.parse().ok())
        .unwrap_or(9000);

    let (listen_fd, bound_port) = linux::bind_listener(port).expect("failed to bind");
    // SAFETY: epoll_create1 takes flags and returns an fd or -1.
    let epfd = unsafe { libc::epoll_create1(0) };
    assert!(epfd >= 0, "epoll_create1 failed");

    // A write to a peer that vanished raises SIGPIPE, which by default kills the
    // process. Servers ignore it and handle EPIPE from write instead.
    // SAFETY: installing SIG_IGN is a single-threaded startup action.
    unsafe {
        libc::signal(libc::SIGPIPE, libc::SIG_IGN);
    }

    println!("epoll echo server listening on 127.0.0.1:{bound_port} (Ctrl-C to stop)");

    let shutdown = AtomicBool::new(false);
    let result = linux::event_loop(epfd, listen_fd, &shutdown);
    // SAFETY: both fds are owned by this process.
    unsafe {
        libc::close(epfd);
        libc::close(listen_fd);
    }

    if let Err(error) = result {

```

```

        eprintln!("event loop error: {error}");
        std::process::exit(1);
    }
}

#[cfg(not(target_os = "linux"))]
fn main() {
    eprintln!("epoll_echo requires Linux; use kqueue or IOCP on this platform.");
}

#[cfg(all(test, target_os = "linux"))]
mod tests {
    use super::linux;
    use std::io::{Read, Write};
    use std::net::TcpStream;
    use std::sync::Arc;
    use std::sync::atomic::{AtomicBool, Ordering};
    use std::thread;

    #[test]
    fn echoes_a_message_through_the_event_loop() {
        let (listen_fd, port) = linux::bind_listener(0).expect("bind");
        // SAFETY: epoll_create1 takes flags and returns an fd or -1.
        let epfd = unsafe { libc::epoll_create1(0) };
        assert!(epfd >= 0, "epoll_create1 failed");

        let shutdown = Arc::new(AtomicBool::new(false));
        let server_shutdown = Arc::clone(&shutdown);
        let server = thread::spawn(move || {
            let result = linux::event_loop(epfd, listen_fd, &server_shutdown);
            // SAFETY: the test owns both fds.
            unsafe {
                libc::close(epfd);
                libc::close(listen_fd);
            }
            result
        });

        let mut client = TcpStream::connect(("127.0.0.1", port)).expect("connect");
        client.write_all(b"epoll echo").expect("write");
        let mut echoed = [0u8; 10];
        client.read_exact(&mut echoed).expect("read echo");
        assert_eq!(&echoed, b"epoll echo");

        shutdown.store(true, Ordering::Relaxed);
        drop(client);
        let result = server.join().expect("server thread panicked");
        assert!(result.is_ok(), "event loop returned {result:?}");
    }
}

```

The whole Linux implementation lives in `mod linux` behind `#[cfg(target_os = "linux")]`, and a second `main` for every other platform prints a message instead. The program compiles everywhere and runs where `epoll` exists. Figure 16.3 contrasts the two server models.

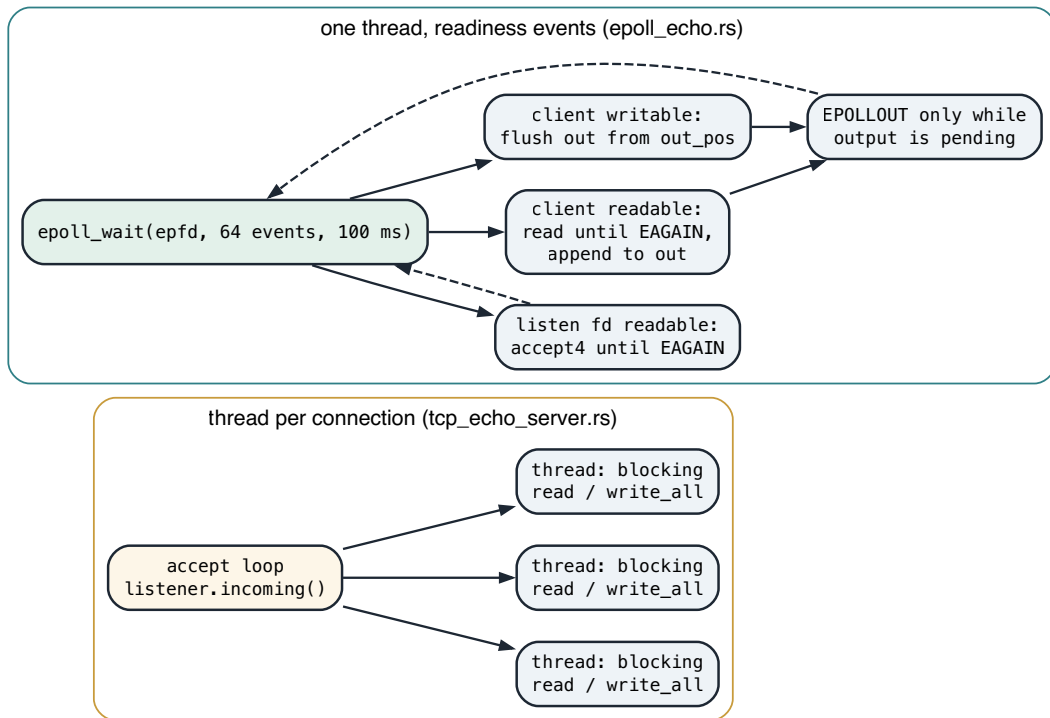


Figure 16.3 Readiness events against threads. The epoll server's only blocking call is `epoll_wait`.

16.4.1 Setting up the listener by hand

`bind_listener` is what `TcpListener::bind` does inside, spelled out. `socket(AF_INET, SOCK_STREAM | SOCK_NONBLOCK, 0)` creates a non-blocking socket in one call. `SO_REUSEADDR` lets a restarted server bind while old connections sit in `TIME_WAIT`. The `sockaddr_in` is built with `std::mem::zeroed()`, and its fields are converted to network order: `port.to_be()` and `INADDR_LOOPBACK.to_be()`. That is the byte-order lesson of section 16.1 in its natural habitat. Every failure path closes the fd before returning the error, so nothing leaks, and the `SAFETY` comment explains why the raw pointers are valid.

16.4.2 The loop

`event_loop` registers the listening socket for `EPOLLIN`, then waits, at most 64 events at a time. The wait has a 100 ms timeout, so the loop re-checks the `shutdown` flag regularly.

`EINTR` (a signal arrived during the wait) is not an error; the loop continues. Each event carries the `fd` in its `u64` field, put there at registration.

- **The listener is readable:** `accept_ready` calls `accept4` with `SOCK_NONBLOCK` in a loop until it returns `EAGAIN`, registering each new client for `EPOLLIN`. Accepting until `EAGAIN` drains a burst of connections in one wake-up.
- **A client is readable:** `read_into` reads until `EAGAIN`, appending everything to the connection's `out` buffer, then tries to `flush` it. A read of 0 closes the connection.
- **A client is writable:** `flush` writes from `out_pos` onward. When `write` returns `EAGAIN` the socket's send buffer is full; the rest stays queued.
- **After either:** `update_interest` asks for `EPOLLOUT` only while output is pending. A socket with room in its send buffer is almost always writable. Leaving `EPOLLOUT` on permanently would wake the loop continuously for nothing.

`Connection` is two fields, `out` and `out_pos`, and `has_pending_output` compares them. That buffer keeps one slow client harmless. Its reply waits in memory, and the loop moves on to other sockets instead of blocking in `write`.

`main` ignores `SIGPIPE`. By default, writing to a socket whose peer has gone away raises `SIGPIPE`, which kills the process. With the signal ignored, `write` returns `EPIPE` instead. Rust's standard library does this for you at startup on Unix, but a program that makes raw `libc` calls should not rely on it.

16.4.3 What a production server would change

The program is a complete, tested event loop, and reading it critically is good practice.

- **One client's error stops the server.** `read_into` and `flush` return `Err` on any failure other than `EAGAIN`, and `event_loop` propagates it with `?`. A client that resets its connection (`ECONNRESET`) makes the whole loop return. A per-connection error should close that connection and continue.
- **The output buffer is unbounded.** A client that sends continuously and never reads grows `out` without limit. Backpressure here means turning off `EPOLLIN` for that client while its buffer is over a threshold.
- **`epoll_ctl` runs after every event.** `update_interest` issues a `EPOLL_CTL_MOD` system call even when the interest set has not changed. Tracking the current set per connection removes most of them.

- **Level-triggered mode.** The loop registers without `EPOLLET`, so `epoll` keeps reporting a socket while it stays ready. Reading until `EAGAIN` is required in edge-triggered mode and harmless here.

Frameworks such as `mio` (under Tokio) wrap exactly this loop, portably over `epoll`, `kqueue`, and `IOCP`. Chapter 18 builds the other half of an async runtime, the part that turns readiness into resumed tasks.

16.5 Questions that come up

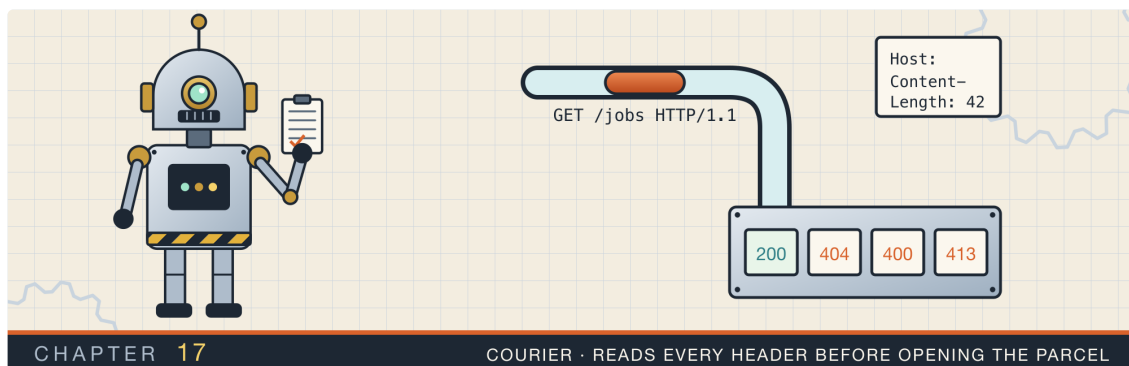
"What does a `read` of 0 bytes mean on a TCP socket?" The peer closed its sending side (FIN). No more data will arrive. It is not an error.

"Why set `TCP_NODELAY`?" To stop Nagle's algorithm from holding back small writes. Request-response protocols that write a small reply and wait want it off.

"Thread per connection or an event loop?" Threads for a modest number of connections and simple blocking code. An event loop, or an async runtime on top of one, for many mostly idle connections, where per-thread stacks and context switches dominate.

"Level-triggered or edge-triggered `epoll`?" Level-triggered reports a ready fd until it is no longer ready. Edge-triggered reports only transitions, so the handler must drain the fd until `EAGAIN`. Otherwise it may never hear about the remaining data.

"A 10 Gb/s link between regions delivers 50 MB/s. Why?" Check the bandwidth-delay product against the window: at 80 ms RTT, 10 Gb/s needs about 100 MB in flight. A small socket buffer, or window scaling disabled somewhere on the path, caps throughput at $\text{window} / \text{RTT}$.



Parsing and serving HTTP/1.1

THIS CHAPTER COVERS

- Parsing an HTTP/1.1 request from bytes: request line, headers, and body framing
- Why Content-Length plus Transfer-Encoding must be rejected, and what request smuggling is
- Decoding chunked bodies, enforcing size limits, and reporting "incomplete" as a normal result
- A small REST server: a pure routing function, keep-alive, and status codes
- Reading the server against the RFC, and the three things it gets wrong

A web server is a common exercise for SDK and platform roles. It is also a trap for anyone who treats HTTP as text with line breaks. The protocol is a byte stream whose message boundaries are decided by headers. The security bugs in real servers come from disagreeing about where one request ends and the next begins. The parser is built around that point, and the server on top of it keeps protocol handling separate from routing. Each can then be tested alone.

17.1 The parser

Listing 17.1 An HTTP/1.1 request parser with framing checks and limits. src/problems/http_request.rs

```
/// HTTP/1.1 request parsing: request line, headers, and body framing.
///
/// The framing checks are the point. `Content-Length` together with
/// `Transfer-Encoding` is rejected as ambiguous (request smuggling), as are
```

```

    //! obsolete line folding and a missing `Host`. Header and body sizes are bounded
    //! so a slow client cannot hold a connection open.

    use std::fmt;

    #[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
    pub enum Method {
        Get,
        Head,
        Post,
        Put,
        Patch,
        Delete,
        Options,
    }

    impl Method {
        pub fn parse(token: &str) -> Option<Self> {
            Some(match token {
                "GET" => Self::Get,
                "HEAD" => Self::Head,
                "POST" => Self::Post,
                "PUT" => Self::Put,
                "PATCH" => Self::Patch,
                "DELETE" => Self::Delete,
                "OPTIONS" => Self::Options,
                _ => return None,
            })
        }

        pub fn is_safe(self) -> bool {
            matches!(self, Self::Get | Self::Head | Self::Options)
        }

        pub fn is_idempotent(self) -> bool {
            matches!(
                self,
                Self::Get | Self::Head | Self::Options | Self::Put | Self::Delete
            )
        }
    }

    #[derive(Debug, Clone, Copy, PartialEq, Eq)]
    pub enum Version {
        Http10,
        Http11,
    }

    #[derive(Debug, Clone, Copy)]
    pub struct Limits {
        pub max_header_bytes: usize,
        pub max_body_bytes: usize,
    }

    impl Default for Limits {
        fn default() -> Self {
            Self {
                max_header_bytes: 16 * 1024,
                max_body_bytes: 8 * 1024 * 1024,
            }
        }
    }

```

```

    }
  }
}

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Request {
  pub method: Method,
  pub target: String,
  pub version: Version,
  pub headers: Vec<(String, String)>,
  pub body: Vec<u8>,
}

impl Request {
  /// First header value matching `name`, case-insensitively.
  pub fn header(&self, name: &str) -> Option<&str> {
    self.headers
      .iter()
      .find(|(key, _)| key.eq_ignore_ascii_case(name))
      .map(|(_, value)| value.as_str())
  }

  /// HTTP/1.1 reuses the connection unless `Connection: close`; HTTP/1.0 only
  /// reuses it with `Connection: keep-alive`.
  pub fn should_keep_alive(&self) -> bool {
    let connection = self.header("connection").unwrap_or("");
    let has = |token: &str| {
      connection
        .split(',')
        .any(|part| part.trim().eq_ignore_ascii_case(token))
    };
    match self.version {
      Version::Http11 => !has("close"),
      Version::Http10 => has("keep-alive"),
    }
  }
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum ParseError {
  /// Not a complete request yet; read more and retry.
  Incomplete,
  Malformed(&'static str),
  MissingHost,
  /// `Content-Length` and `Transfer-Encoding` were both present.
  AmbiguousBodyLength,
  UnsupportedTransferEncoding,
  PayloadTooLarge,
}

impl fmt::Display for ParseError {
  fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
    match self {
      Self::Incomplete => write!(f, "incomplete request"),
      Self::Malformed(reason) => write!(f, "malformed request: {reason}"),
      Self::MissingHost => write!(f, "HTTP/1.1 requires exactly one Host header"),
      Self::AmbiguousBodyLength => {
        write!(
          f,

```

```

        "Content-Length and Transfer-Encoding are mutually exclusive"
    )
}
Self::UnsupportedTransferEncoding => write!(f, "unsupported Transfer-
Encoding"),
Self::PayloadTooLarge => write!(f, "request exceeds configured limits"),
}
}
}

impl std::error::Error for ParseError {}

/// Parse one request from the start of `input`.
pub fn parse_request(input: &[u8], limits: Limits) -> Result<Request, ParseError> {
    let Some(header_end) = input.windows(4).position(|window| window == b"\r\n\r\n") else
    {
        return Err(if input.len() > limits.max_header_bytes {
            ParseError::PayloadTooLarge
        } else {
            ParseError::Incomplete
        });
    };
    if header_end + 4 > limits.max_header_bytes {
        return Err(ParseError::PayloadTooLarge);
    }

    let head = &input[..header_end];
    let body_start = header_end + 4;

    let mut lines = head
        .split(|byte| *byte == b'\n')
        .map(|line| line.strip_suffix(b"\r").unwrap_or(line));

    let request_line = std::str::from_utf8(lines.next().ok_or(ParseError::Malformed("empty
")))?
        .map_err(|_| ParseError::Malformed("request line"))?;
    let mut parts = request_line.split(' ');
    let method = Method::parse(parts.next().ok_or(ParseError::Malformed("method"))?)
        .ok_or(ParseError::Malformed("unknown method"))?;
    let target = parts.next().ok_or(ParseError::Malformed("target"))?;
    let version = match parts.next() {
        Some("HTTP/1.1") => Version::Http11,
        Some("HTTP/1.0") => Version::Http10,
        _ => return Err(ParseError::Malformed("version")),
    };
    if parts.next().is_some() || target.is_empty() {
        return Err(ParseError::Malformed("request line"));
    }

    let mut headers = Vec::new();
    for line in lines {
        if line.is_empty() {
            continue;
        }
        if matches!(line.first(), Some(b' ') | Some(b'\t')) {
            return Err(ParseError::Malformed("obsolete line folding"));
        }
        let colon = line
            .iter()

```

```

        .position(|byte| *byte == b':')
        .ok_or(ParseError::Malformed("header without colon"))?;
    let name = std::str::from_utf8(&line[..colon]);
    .map_err(|_| ParseError::Malformed("header name"))?;
    if name.is_empty() || name.bytes().any(|byte| byte.is_ascii_whitespace()) {
        return Err(ParseError::Malformed("invalid header name"));
    }
    let value = std::str::from_utf8(&line[colon + 1..]);
    .map_err(|_| ParseError::Malformed("header value"))?;
    headers.push((name.to_string(), value.trim().to_string()));
}

if version == Version::Http11
    && headers
        .iter()
        .filter(|(name, _)| name.eq_ignore_ascii_case("host"))
        .count()
        != 1
{
    return Err(ParseError::MissingHost);
}

let body = parse_body(input, body_start, &headers, limits)?;

Ok(Request {
    method,
    target: target.to_string(),
    version,
    headers,
    body,
})
}

fn parse_body(
    input: &[u8],
    body_start: usize,
    headers: &[(String, String)],
    limits: Limits,
) -> Result<Vec<u8>, ParseError> {
    let collect = |name: &str| -> Vec<&str> {
        headers
            .iter()
            .filter(|(key, _)| key.eq_ignore_ascii_case(name))
            .map(|(_, value)| value.as_str())
            .collect()
    };
    let transfer_encoding = collect("transfer-encoding");
    let content_length = collect("content-length");

    // Two framing headers is the classic request-smuggling setup.
    if !transfer_encoding.is_empty() && !content_length.is_empty() {
        return Err(ParseError::AmbiguousBodyLength);
    }

    if !transfer_encoding.is_empty() {
        let joined = transfer_encoding.join(",");
        let codings: Vec<&str> = joined
            .split(',')
            .map(str::trim)

```

```

        .filter(|token| !token.is_empty())
        .collect();
    let all_chunked = codings
        .iter()
        .all(|token| token.eq_ignore_ascii_case("chunked"));
    if !all_chunked
        || !codings
            .last()
            .is_some_and(|t| t.eq_ignore_ascii_case("chunked"))
    {
        return Err(ParseError::UnsupportedTransferEncoding);
    }
    return decode_chunked(&input[body_start..], limits.max_body_bytes);
}

if content_length.is_empty() {
    return Ok(Vec::new());
}

let mut lengths = content_length.iter().map(|value| {
    value
        .trim()
        .parse::<usize>()
        .map_err(|_| ParseError::Malformed("invalid Content-Length"))
});
let first = lengths.next().expect("non-empty");
if lengths.any(|length| length != Ok(first)) {
    return Err(ParseError::Malformed("conflicting Content-Length"));
}
if first > limits.max_body_bytes {
    return Err(ParseError::PayloadTooLarge);
}
let end = body_start + first;
if input.len() < end {
    return Err(ParseError::Incomplete);
}
Ok(input[body_start..end].to_vec())
}

fn decode_chunked(data: &[u8], max_body: usize) -> Result<Vec<u8>, ParseError> {
    let mut cursor = 0;
    let mut body = Vec::new();

    loop {
        let line_end = find_crlf(data, cursor).ok_or(ParseError::Incomplete)?;
        let size_token = data[cursor..line_end]
            .split(|byte| *byte == b';')
            .next()
            .unwrap_or(&[]);
        let size = usize::from_str_radix(
            std::str::from_utf8(size_token)
                .map_err(|_| ParseError::Malformed("chunk size"))?
                .trim(),
            16,
        )
        .map_err(|_| ParseError::Malformed("chunk size"))?;
        cursor = line_end + 2;

        if size == 0 {

```



```

        // Consume trailers up to the final empty line.
        loop {
            let trailer_end = find_crlf(data, cursor).ok_or(ParseError::Incomplete)?;
            let is_empty = trailer_end == cursor;
            cursor = trailer_end + 2;
            if is_empty {
                return Ok(body);
            }
        }

    }

    if body.len() + size > max_body {
        return Err(ParseError::PayloadTooLarge);
    }
    let chunk_end = cursor
        .checked_add(size)
        .ok_or(ParseError::Malformed("chunk overflow"))?;
    if chunk_end + 2 > data.len() {
        return Err(ParseError::Incomplete);
    }
    body.extend_from_slice(&data[cursor..chunk_end]);
    if &data[chunk_end..chunk_end + 2] != b"\r\n" {
        return Err(ParseError::Malformed("chunk not CRLF-terminated"));
    }
    cursor = chunk_end + 2;
}

}

fn find_crlf(data: &[u8], from: usize) -> Option<usize> {
    data.get(from..)?
        .windows(2)
        .position(|window| window == b"\r\n")
        .map(|offset| offset + from)
}

#[cfg(test)]
mod tests {
    use super::*;

    fn parse(raw: &[u8]) -> Result<Request, ParseError> {
        parse_request(raw, Limits::default())
    }

    #[test]
    fn parses_a_get_and_a_post_with_content_length() {
        let get = parse(b"GET /v1/gpu-workloads HTTP/1.1\r\nHost: api\r\nAccept: */*\r\n\r\n")
            .expect("valid");
        assert_eq!(get.method, Method::Get);
        assert_eq!(get.header("HOST"), Some("api"));
        assert!(get.body.is_empty());
        assert!(get.should_keep_alive());

        let post = parse(b"POST / HTTP/1.1\r\nHost: api\r\nContent-Length: 5\r\n\r\nhello")
            .expect("valid");
        assert_eq!(post.body, b"hello");
        assert!(!post.method.is_idempotent());
    }
}

```

```

#[test]
fn decodes_chunked_bodies() {
    let raw = b"POST / HTTP/1.1\r\nHost: api\r\nTransfer-Encoding: chunked\r\n\r\n5\r\nhello\r\n6\r\n world\r\n0\r\n\r\n";
    assert_eq!(parse(raw).expect("valid").body, b"hello world");
}

#[test]
fn rejects_ambiguous_framing_and_bad_framing() {
    let smuggling = b"POST / HTTP/1.1\r\nHost: api\r\nContent-Length: 5\r\nTransfer-Encoding: chunked\r\n\r\nhello";
    assert_eq!(parse(smuggling), Err(ParseError::AmbiguousBodyLength));

    let gzip = b"POST / HTTP/1.1\r\nHost: api\r\nTransfer-Encoding: gzip\r\n\r\n";
    assert_eq!(parse(gzip), Err(ParseError::UnsupportedTransferEncoding));
}

#[test]
fn rejects_missing_host_and_line_folding() {
    assert_eq!(
        parse(b"GET / HTTP/1.1\r\nAccept: */*\r\n\r\n"),
        Err(ParseError::MissingHost)
    );
    let folded = b"GET / HTTP/1.1\r\nHost: api\r\nX-A: 1\r\n continued\r\n\r\n";
    assert!(matches!(parse(folded), Err(ParseError::Malformed(_))));
}

#[test]
fn reports_incomplete_and_enforces_limits() {
    assert_eq!(
        parse(b"GET / HTTP/1.1\r\nHost: api\r\n"),
        Err(ParseError::Incomplete)
    );
    let limits = Limits {
        max_header_bytes: 1024,
        max_body_bytes: 10,
    };
    let raw = b"POST / HTTP/1.1\r\nHost: api\r\nContent-Length: 999\r\n\r\n";
    assert_eq!(parse_request(raw, limits), Err(ParseError::PayloadTooLarge));
}

#[test]
fn http_1_0_needs_keep_alive_opt_in() {
    let raw = b"GET / HTTP/1.0\r\n\r\n";
    assert!(!parse(raw).expect("valid").should_keep_alive());
    let raw = b"GET / HTTP/1.0\r\nConnection: keep-alive\r\n\r\n";
    assert!(parse(raw).expect("valid").should_keep_alive());
}
}

```

17.1.1 Types first

The file opens with its vocabulary. `Method` is an enum with a `parse` that returns `None` for an unknown token, and two predicates from RFC 9110. `is_safe` is true for GET, HEAD, and OPTIONS, which do not change state. `is_idempotent` adds PUT and DELETE to those. Chapter

15's retry loop is exactly the consumer that needs `is_idempotent`. `Version` has two variants because the parser accepts two. `Limits` carries the header and body caps, with defaults of 16 KiB and 8 MiB.

`ParseError` distinguishes the cases a server must treat differently. `Incomplete` is not a failure: it means "read more bytes and try again", and the server's loop in section 17.2 depends on it. `PayloadTooLarge` maps naturally to status 413, `MissingHost` and `Malformed` to 400.

`Request::header` finds a header case-insensitively, because header names are case-insensitive. Then `should_keep_alive` encodes the one rule that differs by version. HTTP/1.1 keeps the connection open unless the client sends `Connection: close`. HTTP/1.0 closes it unless the client sends `Connection: keep-alive`. The `has` closure splits the header on commas because `Connection` carries a list.

17.1.2 The head of the request

`parse_request` works on `&[u8]`, not `&str`, because an HTTP body is arbitrary bytes and only the head is text. It first looks for the blank line that ends the head, `\r\n\r\n`, with `windows(4).position(...)`. If it is missing, the request is `Incomplete`. The exception is a buffer already larger than `max_header_bytes`: then the client is sending an endless header and gets `PayloadTooLarge`. That check stops a slow client from holding memory with a header that never ends. Figure 17.1 shows the order of the remaining checks.

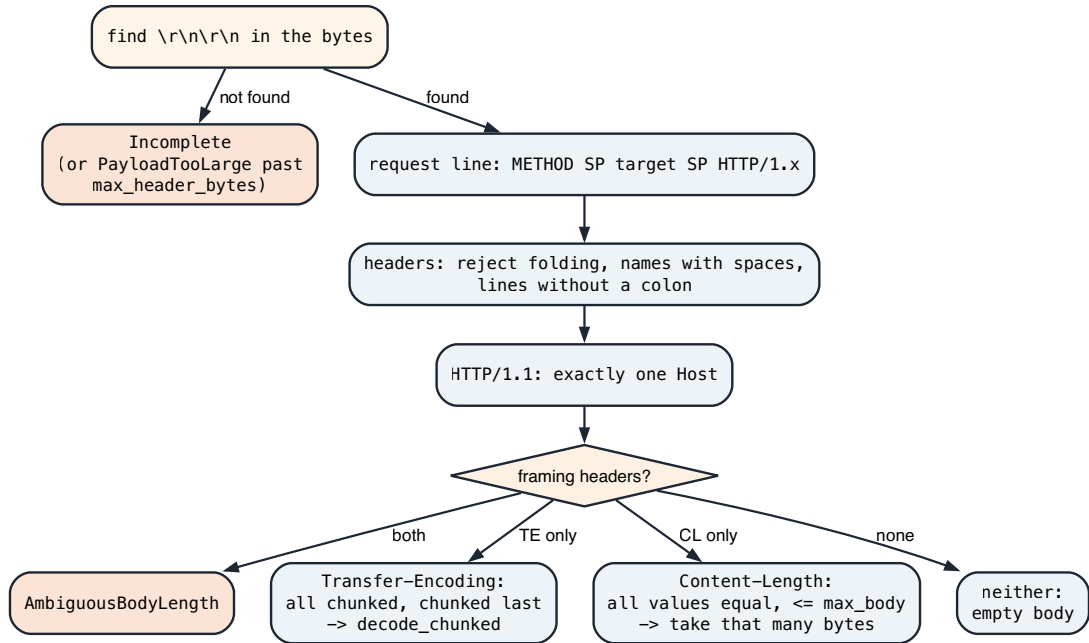


Figure 17.1 The order of checks in `parse_request` and `parse_body`.

The request line must be exactly three space-separated parts: a known method, a non-empty target, and HTTP/1.1 or HTTP/1.0. A fourth part is an error. Header lines are split at the first colon. A name with whitespace is rejected, because RFC 9112 forbids whitespace between the name and the colon. A line that starts with a space or tab is *obsolete line folding*, a continuation syntax the RFC deprecates. Since different servers interpret it differently, the parser rejects it.

HTTP/1.1 requires exactly one `Host` header. The count uses `filter(...).count() != 1`, which rejects both zero and two.

17.1.3 Body framing and request smuggling

`parse_body` decides how long the body is, which is what the module's first comment is about. A request with both `Content-Length` and `Transfer-Encoding` is rejected as `AmbiguousBodyLength`.

The reason is request smuggling (figure 17.2). A front-end proxy may frame a request by `Content-Length`, while the back-end server frames the same bytes by `Transfer-Encoding: chunked`. The two then disagree about where the request ends. The attacker places a second

request in the part the back-end treats as "after the body". The back-end runs it as though it arrived on its own, past whatever checks the proxy applied. Refusing ambiguous framing outright, and closing the connection, removes the disagreement.

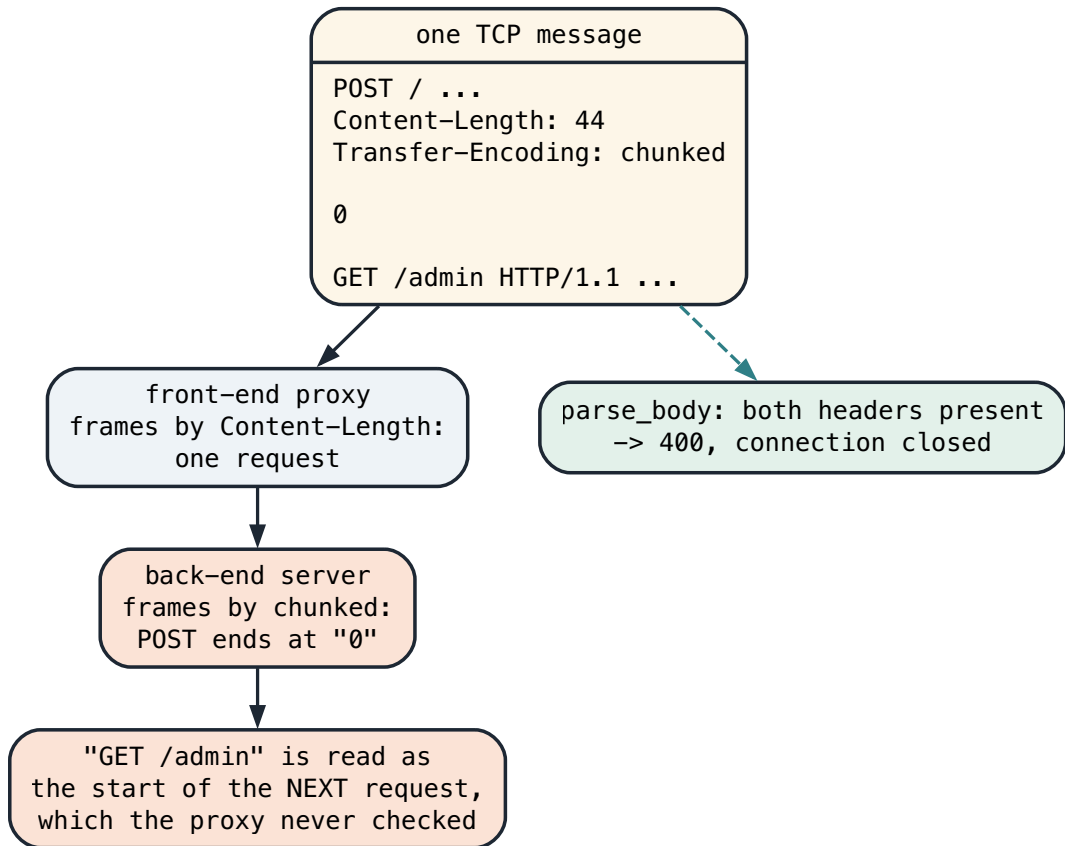
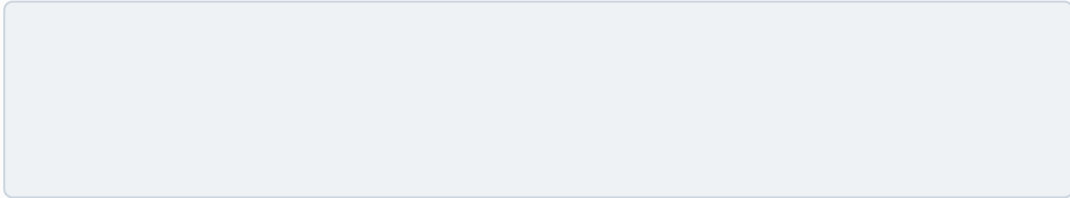


Figure 17.2 A CLTE desynchronization. The parser answers such a request with 400 and closes the connection.

HTTP/1.1 body framing: the headers decide where the message ends

When two rules disagree, one request can be read as two. The parser refuses that.

☒ one reading ☐ two readings



body 5 bytes

The head ends at the first blank line. Content-Length says the body is exactly 5 bytes: hello.

The boundary of a message is a decision the headers make, not something the bytes carry.

framing = **Content-Length**

Animation 17.3 The three framing rules on the same request shape. The head always ends at the first blank line; what differs is how the body's length is decided. Content-Length gives an exact byte count, a chunked body carries its own sizes and ends at a zero-size chunk, and a message with both is refused outright. The last frame is the disagreement removed: a connection that closes cannot have its leftover bytes read as a second request.

The remaining rules close smaller gaps:

- **Transfer-Encoding** values from every such header are joined and split into codings. The parser supports only **chunked**, and requires it to be last, because a body whose final coding is not **chunked** has no defined end. Anything else is **UnsupportedTransferEncoding**.
- Multiple **Content-Length** headers are allowed only if they all agree. The check parses each value and compares with `lengths.any(|length| length != 0k(first))`, comparing **Results** directly.
- A length above `max_body_bytes` is rejected *before* the server waits for the bytes. A client cannot claim a huge body and make the server buffer it.

- If fewer bytes than the length have arrived, the result is `Incomplete`.

17.1.4 Chunked bodies

`decode_chunked` walks a cursor through `size-in-hex` CRLF data CRLF records until a size of 0, then consumes optional trailer lines until an empty line. Chunk extensions after a `;` are ignored, as the RFC permits. Each chunk checks the running total against the body limit. It computes the chunk's end with `checked_add`, returns `Incomplete` if the data has not all arrived, and insists on the CRLF after the data. `find_crlf` uses `data.get(from..)?` so that a cursor past the end is `None` rather than a panic.

One arithmetic detail needs tightening. `body.len() + size > max_body` runs before the `checked_add`, and a chunk size near `usize::MAX` overflows that addition. `from_str_radix` will happily parse such a size from sixteen fs. The form `size > max_body - body.len()` cannot overflow.

`parse_request` returns the request but not how many bytes it consumed. A caller therefore cannot find the start of a second, pipelined request in the same buffer. The server below accepts that limitation and says so.

17.2 A small REST server

Listing 17.2 Routes, responses, and a keep-alive loop over `std::net::src/bin/http_server.rs`

```

// Minimal HTTP/1.1 server with REST-style routes.
//
// It uses only `std::net` plus the request parser in
// [`systems_lab::problems::http_request`], so the interesting parts
// stay visible: the accept loop, the keep-alive loop, status codes, and the
// routing decision.
//
// Routes:
//
// | Method | Path | Response |
// |---|---|---|
// | `GET` | `/healthz` | `200` plain text |
// | `GET` | `/v1/gpu-workloads` | `200` JSON list |
// | `POST` | `/v1/gpu-workloads` | `201` created |
// | `GET` | `/v1/gpu-workloads/{id}` | `200` one workload |
// | `DELETE` | `/v1/gpu-workloads/{id}` | `204` no content |
//
// A known path with the wrong method returns `405`, and an unknown path returns
// `404`. Pipelined requests are not supported: the loop handles one request per
// read buffer, which matches how most servers treat pipelining in practice.
//
// ```bash

```

```

/// cargo run --bin http_server          # binds 127.0.0.1:8080
/// curl -i http://127.0.0.1:8080/healthz
/// curl -i -X POST -d '{"gpuCount":1}' http://127.0.0.1:8080/v1/gpu-workloads
/// ```

use systems_lab::problems::http_request::{
    Limits, Method, ParseError, Request, parse_request,
};
use std::io::{Read, Write};
use std::net::{TcpListener, TcpStream};
use std::thread;

/// Map a request to `(status, content_type, body)`.
///
/// Pure, so the routing rules are unit-testable without a socket.
pub fn route(request: &Request) -> (u16, &'static str, Vec<u8>) {
    let target = request.target.as_str();

    if target == "/healthz" {
        return match request.method {
            Method::Get | Method::Head => (200, "text/plain", b"ok\n".to_vec()),
            _ => method_not_allowed(),
        };
    }

    if target == "/v1/gpu-workloads" {
        return match request.method {
            Method::Get | Method::Head => (
                200,
                "application/json",
                br#"{"id":"wl-a100-1","gpuCount":1,"state":"Running"}"#.to_vec(),
            ),
            Method::Post => (
                201,
                "application/json",
                br#"{"id":"wl-a100-2","gpuCount":1,"state":"Pending"}"#.to_vec(),
            ),
            _ => method_not_allowed(),
        };
    }

    if let Some(id) = target.strip_prefix("/v1/gpu-workloads/") {
        if id.is_empty() || id.contains('/') {
            return (404, "text/plain", b"not found\n".to_vec());
        }
        return match request.method {
            Method::Get | Method::Head => (
                200,
                "application/json",
                format!(r#"{"id":"{id}","state":"Running"}"#).into_bytes(),
            ),
            Method::Delete => (204, "", Vec::new()),
            _ => method_not_allowed(),
        };
    }

    (404, "text/plain", b"not found\n".to_vec())
}

```



```

fn method_not_allowed() -> (u16, &'static str, Vec<u8>) {
    (405, "text/plain", b"method not allowed\n".to_vec())
}

fn status_reason(status: u16) -> &'static str {
    match status {
        200 => "OK",
        201 => "Created",
        204 => "No Content",
        400 => "Bad Request",
        404 => "Not Found",
        405 => "Method Not Allowed",
        413 => "Payload Too Large",
        _ => "Internal Server Error",
    }
}

fn build_response(status: u16, content_type: &str, body: &[u8], keep_alive: bool) -> Vec<u8> {
    let mut response = format!("HTTP/1.1 {} {} \r\n", status, status_reason(status));
    response.push_str(&format!("Content-Length: {} \r\n", body.len()));
    if !content_type.is_empty() {
        response.push_str(&format!("Content-Type: {} \r\n", content_type));
    }
    response.push_str(if keep_alive {
        "Connection: keep-alive \r\n"
    } else {
        "Connection: close \r\n"
    });
    response.push_str("\r\n");

    let mut bytes = response.into_bytes();
    bytes.extend_from_slice(body);
    bytes
}

/// Serve connections forever, one thread per connection.
pub fn serve(listener: TcpListener) {
    for incoming in listener.incoming() {
        match incoming {
            Ok(stream) => {
                thread::spawn(move || {
                    if let Err(error) = handle_connection(stream) {
                        eprintln!("connection error: {error}");
                    }
                });
            }
            Err(error) => eprintln!("accept error: {error}"),
        }
    }
}

fn handle_connection(mut stream: TcpStream) -> std::io::Result<()> {
    stream.set_nodelay(true)?;
    let limits = Limits::default();
    let mut buffer = Vec::new();
    let mut chunk = [0u8; 4096];

    loop {

```

```

        match parse_request(&buffer, limits) {
            Ok(request) => {
                let keep_alive = request.should_keep_alive();
                let (status, content_type, body) = route(&request);
                let response = build_response(status, content_type, &body, keep_alive);
                stream.write_all(&response)?;

                if !keep_alive {
                    return Ok(());
                }
                buffer.clear();
            }
            Err(ParseError::Incomplete) => {
                let read = stream.read(&mut chunk)?;
                if read == 0 {
                    // Peer closed mid-request.
                    return Ok(());
                }
                buffer.extend_from_slice(&chunk[..read]);
            }
            Err(error) => {
                let response =
                    build_response(400, "text/plain", error.to_string().as_bytes(), false)
;
                stream.write_all(&response)?;
                return Ok(());
            }
        }
    }
}

fn main() {
    let address = std::env::args()
        .nth(1)
        .unwrap_or_else(|| "127.0.0.1:8080".to_string());

    let listener = TcpListener::bind(&address).expect("failed to bind");
    let local = listener.local_addr().expect("failed to read local address");
    println!("http server listening on http://{local}");

    serve(listener);
}

#[cfg(test)]
mod tests {
    use super::*;
    use systems_lab::problems::http_request::Version;

    fn request(method: Method, target: &str) -> Request {
        Request {
            method,
            target: target.to_string(),
            version: Version::Http11,
            headers: vec![("host".to_string(), "localhost".to_string())],
            body: Vec::new(),
        }
    }
}

#[test]

```

```

fn healthz_is_ok() {
    let (status, content_type, body) = route(&request(Method::Get, "/healthz"));
    assert_eq!(status, 200);
    assert_eq!(content_type, "text/plain");
    assert_eq!(body, b"ok\n");
}

#[test]
fn lists_and_creates_workloads() {
    let (status, _, _) = route(&request(Method::Get, "/v1/gpu-workloads"));
    assert_eq!(status, 200);

    let (status, content_type, body) = route(&request(Method::Post, "/v1/gpu-
workloads"));
    assert_eq!(status, 201);
    assert_eq!(content_type, "application/json");
    assert!(String::from_utf8(body).expect("json").contains("Pending"));
}

#[test]
fn gets_and_deletes_one_workload() {
    let (status, _, body) = route(&request(Method::Get, "/v1/gpu-workloads/wl-7"));
    assert_eq!(status, 200);
    assert!(String::from_utf8(body).expect("json").contains("wl-7"));

    let (status, _, body) = route(&request(Method::Delete, "/v1/gpu-workloads/wl-7"));
    assert_eq!(status, 204);
    assert!(body.is_empty());
}

#[test]
fn wrong_method_on_known_path_is_405() {
    let (status, _, _) = route(&request(Method::Put, "/healthz"));
    assert_eq!(status, 405);
    let (status, _, _) = route(&request(Method::Put, "/v1/gpu-workloads"));
    assert_eq!(status, 405);
}

#[test]
fn unknown_path_is_404() {
    let (status, _, _) = route(&request(Method::Get, "/nope"));
    assert_eq!(status, 404);
}
}

```

17.2.1 Routing as a pure function

`route(&Request) -> (u16, &'static str, Vec<u8>)` decides the status, content type, and body without touching a socket. Its doc comment gives the reason: it is unit-testable, and the tests at the bottom exercise every route by building a `Request` value directly. A known path with an unsupported method returns 405, an unknown path 404. For `/v1/gpu-workloads/{id}`, `strip_prefix` extracts the id, and an empty id or one containing `/` is a 404.

`build_response` writes the status line, `Content-Length`, an optional `Content-Type`, and a `Connection` header that tells the client what the server will do.

17.2.2 The connection loop

`handle_connection` accumulates bytes in `buffer` and asks the parser what it has (figure 17.3). On `Ok`, it routes, writes the response, and either closes or clears the buffer for the next request. On `Incomplete`, it reads more; a read of 0 means the client left mid-request. Any other error gets a 400 whose body is the error's `Display` text, and the connection closes. After a framing error the server cannot know where the next request starts.

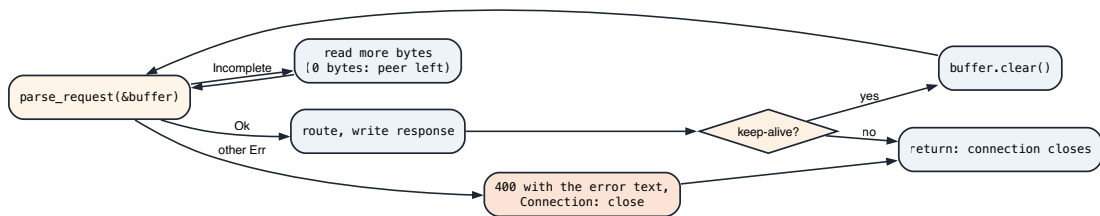


Figure 17.3 The keep-alive loop in `handle_connection`.

A session with `curl` and `nc` against the server, started with `cargo run --bin http_server 127.0.0.1:18080`:

```

$ curl -si http://127.0.0.1:18080/healthz
HTTP/1.1 200 OK
Content-Length: 3
Content-Type: text/plain
Connection: keep-alive

ok
$ curl -si -X POST -d '{"gpuCount":1}' http://127.0.0.1:18080/v1/gpu-workloads
HTTP/1.1 201 Created
Content-Length: 49
Content-Type: application/json
Connection: keep-alive

{"id":"wl-a100-2","gpuCount":1,"state":"Pending"}
$ printf 'POST / HTTP/1.1\r\nHost: x\r\nContent-Length: 5\r\nTransfer-Encoding:
chunked\r\n\r\nhello' | nc 127.0.0.1 18080
HTTP/1.1 400 Bad Request
Content-Length: 59
Content-Type: text/plain
Connection: close

```

Content-Length and Transfer-Encoding are mutually exclusive

17.2.3 Reading the server against the RFC

The same session shows three places where the server departs from RFC 9110. None of them is hard to fix, and finding them is good practice.

```
$ curl -si -X DELETE http://127.0.0.1:18080/v1/gpu-workloads/wl-7
HTTP/1.1 204 No Content
Content-Length: 0
Connection: keep-alive

$ printf 'HEAD /healthz HTTP/1.1\r\nHost: x\r\nConnection: close\r\n\r\n' | nc 127.0.0.1
18080
HTTP/1.1 200 OK
Content-Length: 3
Content-Type: text/plain
Connection: close

ok
```

- **HEAD must not carry a body.** A HEAD response repeats the GET headers, including the `Content-Length` of a body it does not send. The server sends `ok`. A client that trusts the spec reads those three bytes as the start of the *next* response. The fix is one line in `handle_connection: skip body when request.method == Method::Head`.
- **204 must not carry Content-Length.** `build_response` always writes it. Skipping it for 204 (and 1xx) fixes that.
- **Errors all become 400.** `PayloadTooLarge` deserves 413, and `status_reason` already has its text. Mapping `ParseError` to a status in one `match` fixes it. A 405 should also carry an `Allow` header listing the methods that path supports.

Two more points concern safety rather than the RFC. The `id` is interpolated into JSON with `format!`, so an `id` containing `"` produces invalid JSON; a real server serializes with `serde_json`. And `buffer.clear()` after each request discards any pipelined bytes that arrived with it, which the module comment acknowledges.

17.3 Questions that come up

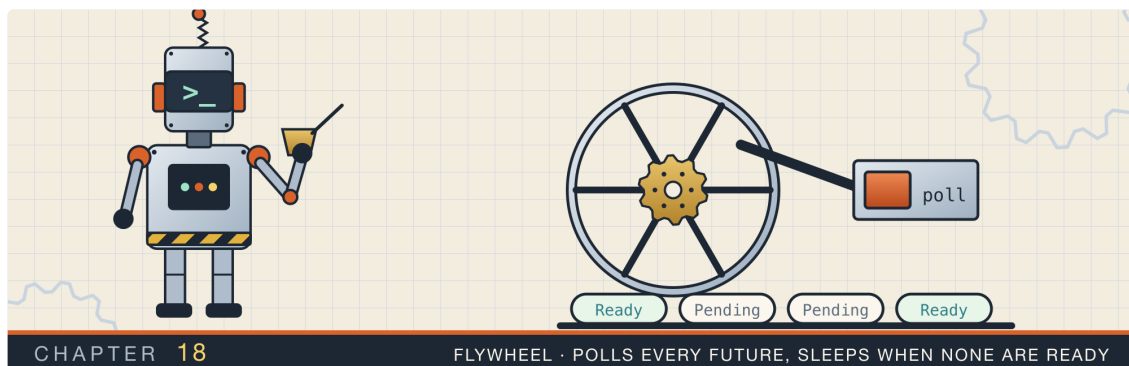
"How does a server know where an HTTP/1.1 request body ends?" `Transfer-Encoding: chunked` means read chunks until a zero-size chunk. Else `Content-Length` gives the count. With neither header there is no body. For responses, closing the connection can also end the body.

"What is request smuggling, and how do you prevent it?" Two servers frame the same bytes differently, so each sees a different message. Reject requests with both framing headers. Reject conflicting `Content-Length`s. Close the connection after any framing error.

"How do you stop a client from exhausting your server's memory?" Limit header size and body size. Check the declared length before reading the body. Time out idle and slow connections.

"Why is routing a separate pure function?" So the rules are testable without sockets. The protocol code (framing, keep-alive) can then be changed without touching the application logic.

"What does HTTP/2 change?" Binary framing with explicit lengths, many concurrent streams on one connection, and header compression. Framing ambiguity of the HTTP/1.1 kind disappears inside HTTP/2, although it can return where a proxy downgrades to HTTP/1.1.



Async Rust from the executor up

THIS CHAPTER COVERS

- What `Future::poll`, `Poll::Pending`, and a `Waker` promise each other
- `block_on`: an executor in fifteen lines that parks the thread between polls
- Two hand-written futures: one that yields, one that waits on a timer thread
- A task queue where waking a task re-enqueues it
- An `async` block as a state machine, and why `poll` takes `Pin<&mut Self>`
- Two HTTP clients with caching, retries, and idempotency keys: one blocking over `TcpStream`, one on Tokio and `request`

Any serious use of Tokio raises the question of what `.await` does. The short answer: it calls `poll` on a future. If the future is not ready, control returns to whoever called it, after arranging to be called again. The long answer is a runtime, and this chapter builds a small one so that the short answer has something concrete behind it. It then uses a real runtime for the job runtimes exist for: talking to a server over the network while retrying failures safely.

18.1 The contract

A `Future` has one method:

```
fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
```

It returns `Poll::Ready(value)` when done or `Poll::Pending` when not. `Pending` comes with an obligation: before returning it, the future must make sure that `cx.waker()` will be called when progress is possible. The executor's side of the contract is to poll again after a wake, and not to busy-loop otherwise. Everything in this chapter is one side or the other of that agreement.

A runtime exists to avoid one thread per task. A thread is given a stack, and a default stack is measured in megabytes. Ten thousand idle connections held as threads would reserve gigabytes before doing any work. A suspended future is a state machine, and it holds only the variables alive at its suspension points. Ten thousand parked connections cost kilobytes each instead. The work must then be written as a state machine, and `async` is the notation for writing one that still looks like ordinary code.

WARNING The two halves of the contract fail in different ways, and only one is easy to debug. Polling a future that is not ready is always safe. It returns `Pending` again, so a spurious wake and an early return from `park` do no harm. Forgetting to arrange a wake is not safe. The task returns `Pending`, nothing calls its waker, and the task never runs again. There is no crash and no error message. That one task stops making progress. When an `async` program hangs with no visible reason, look first for a waker that was dropped instead of called.

Listing 18.1 `block_on`, two futures, and a task executor, with no runtime dependency. `src/problems/async_mini.rs`

```

    ///! A minimal async runtime, built from `Future`, `Waker`, and `Pin`.
    ///!
    ///! `.await` is not magic. A `Future` is a state machine with one method, `poll`,
    ///! that returns `Ready(value)` or `Pending`; when it returns `Pending` it must
    ///! arrange for the `Waker` in its `Context` to be called later. An executor is a
    ///! loop that polls tasks and parks until a waker fires.
    ///!
    ///! This module implements that loop twice, from scratch and without a runtime
    ///! dependency:
    ///!
    ///! - [block_on] drives one future on the current thread, parking on `Pending`;
    ///! - [MiniExecutor] queues independent tasks and re-enqueues them when woken.
    ///!
    ///! It is deliberately small. It makes three things concrete: why `poll` takes `Pin<&mut
    Self>`, what `Pending` promises, and where
    ///! `Send` matters for a multi-threaded runtime.

    use std::collections::VecDeque;
    use std::future::Future;
    use std::pin::Pin;
    use std::sync::atomic::{AtomicBool, Ordering};
    use std::sync::{Arc, Mutex};
    use std::task::{Context, Poll, Wake, Waker};

```



```

use std::thread;
use std::time::Duration;

/// Block the current thread until `future` completes.
///
/// The future is pinned on the heap so its address is stable across `await`
/// points, and the thread parks whenever the future reports `Pending`. The waker
/// unparks it, so the loop polls again only when there is a reason to.
pub fn block_on<F: Future>(future: F) -> F::Output {
    let mut future = Box::pin(future);
    let waker = Waker::from(Arc::new(ThreadWaker(thread::current())));
    let mut context = Context::from_waker(&waker);

    loop {
        match future.as_mut().poll(&mut context) {
            Poll::Ready(value) => return value,
            // `park` may return spuriously, so the loop re-polls either way.
            Poll::Pending => thread::park(),
        }
    }
}

/// Waker that unparks the thread which is driving the future.
struct ThreadWaker(thread::Thread);

impl Wake for ThreadWaker {
    fn wake(self: Arc<Self>) {
        self.0.unpark();
    }

    fn wake_by_ref(self: &Arc<Self>) {
        self.0.unpark();
    }
}

/// A future that yields to the executor `times` times before completing.
///
/// Completes with the number of times it yielded, which makes it easy to assert
/// that a runtime actually suspended and resumed a task.
pub struct YieldTimes {
    remaining: usize,
    yielded: usize,
}

impl YieldTimes {
    pub fn new(times: usize) -> Self {
        Self {
            remaining: times,
            yielded: 0,
        }
    }
}

impl Future for YieldTimes {
    type Output = usize;

    fn poll(self: Pin<&mut Self>, context: &mut Context<'_>) -> Poll<usize> {
        // `YieldTimes` is `Unpin`, so a mutable reference is safe to take.
        let this = self.get_mut();

```

```

        if this.remaining == 0 {
            return Poll::Ready(this.yielded);
        }

        this.remaining -= 1;
        this.yielded += 1;
        // Ask to be polled again; an executor that ignores this hangs.
        context.waker().wake_by_ref();
        Poll::Pending
    }
}

struct DelayState {
    ready: bool,
    waker: Option<Waker>,
}

/// A future that completes after a wall-clock duration.
///
/// A background thread sleeps and then wakes the task. Storing and waking the
/// [Waker] – rather than spinning in `poll` – is the whole point: `poll` never
/// blocks the executor thread.
pub struct Delay {
    state: Arc<Mutex<DelayState>>,
}

impl Delay {
    pub fn new(duration: Duration) -> Self {
        let state = Arc::new(Mutex::new(DelayState {
            ready: false,
            waker: None,
        }));

        let thread_state = Arc::clone(&state);
        thread::spawn(move || {
            thread::sleep(duration);
            let waker = {
                let mut guard = thread_state.lock().expect("delay mutex poisoned");
                guard.ready = true;
                guard.waker.take()
            };
            if let Some(waker) = waker {
                waker.wake();
            }
        });

        Self { state }
    }
}

impl Future for Delay {
    type Output = ();

    fn poll(self: Pin<&mut Self>, context: &mut Context<'_>) -> Poll<()> {
        let mut guard = self.state.lock().expect("delay mutex poisoned");
        if guard.ready {
            Poll::Ready(())
        } else {
            guard.waker = Some(context.waker().clone());
        }
    }
}

```

```

        Poll::Pending
    }
}

struct Task {
    future: Mutex<Pin<Box<dyn Future<Output = ()> + Send>>>,
    queue: Arc<Mutex<VecDeque<Arc<Task>>>>,
    completed: AtomicBool,
}

impl Wake for Task {
    fn wake(self: Arc<Self>) {
        // Re-enqueue the task. Cloning the `Arc` is a refcount bump, not a deep copy.
        self.queue
            .lock()
            .expect("executor queue poisoned")
            .push_back(Arc::clone(&self));
    }

    fn wake_by_ref(self: &Arc<Self>) {
        self.queue
            .lock()
            .expect("executor queue poisoned")
            .push_back(Arc::clone(self));
    }
}

/// A single-threaded, `Send`-only executor.
///
/// Real runtimes add work stealing, timers, and I/O readiness. This keeps only the
/// part that matters for explaining how `.await` makes progress: a queue of tasks
/// where waking a task puts it back on the queue.
#[derive(Default)]
pub struct MiniExecutor {
    queue: Arc<Mutex<VecDeque<Arc<Task>>>>,
}

impl MiniExecutor {
    pub fn new() -> Self {
        Self::default()
    }

    /// Add a task to the executor.
    pub fn spawn<F>(&self, future: F)
    where
        F: Future<Output = ()> + Send + 'static,
    {
        let task = Arc::new(Task {
            future: Mutex::new(Box::pin(future)),
            queue: Arc::clone(&self.queue),
            completed: AtomicBool::new(false),
        });
        self.queue
            .lock()
            .expect("executor queue poisoned")
            .push_back(task);
    }
}

```

```

    /// Run until the queue is empty.
    ///
    /// A task that returns `Pending` without waking stays suspended, which is why
    /// this returns rather than blocks: a runtime with timers or I/O would also
    /// park on an event source.
    pub fn run(&self) {
        loop {
            let Some(task) = self
                .queue
                .lock()
                .expect("executor queue poisoned")
                .pop_front()
            else {
                return;
            };

            if task.completed.load(Ordering::Acquire) {
                continue;
            }

            let waker = Waker::from(Arc::clone(&task));
            let mut context = Context::from_waker(&waker);
            let mut future = task.future.lock().expect("task mutex poisoned");
            if future.as_mut().poll(&mut context).is_ready() {
                task.completed.store(true, Ordering::Release);
            }
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::atomic::AtomicUsize;
    use std::time::Instant;

    #[test]
    fn block_on_drives_a_hand_written_future() {
        assert_eq!(block_on(YieldTimes::new(3)), 3);
    }

    #[test]
    fn block_on_polls_an_async_block() {
        let total = block_on(async {
            let a = YieldTimes::new(1).await;
            let b = YieldTimes::new(2).await;
            a + b
        });
        assert_eq!(total, 3);
    }

    #[test]
    fn block_on_waits_for_a_delay() {
        let start = Instant::now();
        block_on(Delay::new(Duration::from_millis(30)));
        assert!(start.elapsed() >= Duration::from_millis(20));
    }

    #[test]

```

```

fn executor_runs_spawned_tasks_to_completion() {
    let executor = MiniExecutor::new();
    let counter = Arc::new(AtomicUsize::new(0));

    for _ in 0..4 {
        let counter = Arc::clone(&counter);
        executor.spawn(async move {
            YieldTimes::new(2).await;
            counter.fetch_add(1, Ordering::SeqCst);
        });
    }

    executor.run();
    assert_eq!(counter.load(Ordering::SeqCst), 4);
}

#[test]
fn executor_polls_an_immediately_ready_task() {
    let executor = MiniExecutor::new();
    let flag = Arc::new(AtomicBool::new(false));

    let task_flag = Arc::clone(&flag);
    executor.spawn(async move {
        task_flag.store(true, Ordering::SeqCst);
    });

    executor.run();
    assert!(flag.load(Ordering::SeqCst));
}
}

```

18.2 block_on

`block_on` drives one future to completion on the calling thread:

1. `Box::pin(future)` moves the future to the heap and pins it there, so its address will not change between polls (section 18.5 explains why).
2. The waker is built from a `ThreadWaker` holding a handle to the current thread. `Waker::from(Arc<W>)` works for any `W: Wake`, the safe way to build a waker without writing a raw vtable.
3. The loop polls. On `Ready` it returns the value. On `Pending` it calls `thread::park()`, which sleeps until some other code calls `unpark` on this thread, which is exactly what `ThreadWaker::wake` does.

The comment on the `Pending` arm notes that `park` may return spuriously. The loop tolerates it by polling again: an extra poll of a future that is not ready returns `Pending` a second time. The contract makes spurious polls safe and missing wakes fatal. `park/unpark` also has a

property the loop needs when the wake arrives *before* the park. `unpark` leaves a token, and the next `park` returns at once.

18.3 Two futures by hand

`YieldTimes` returns `Pending` a set number of times before it completes, and each time it calls `context.waker().wake_by_ref()` first. It asks to be polled again immediately, which is what `tokio::task::yield_now` does. Its value is the number of times it yielded, so a test can check that the executor really suspended and resumed it. The comment in `poll` spells out the contract: an executor that ignored the wake would hang.

`YieldTimes` contains two `usizes`, so it is `Unpin`, and `self.get_mut()` turns `Pin<&mut Self>` into `&mut Self` without `unsafe`.

`Delay` completes after a wall-clock duration, and it shows the real pattern for I/O. Its state is `ready` and an optional stored `Waker`, shared with a background thread through `Arc<Mutex<_>>`. `poll` checks `ready`; if not, it stores a clone of the current waker and returns `Pending`. The thread sleeps, sets `ready`, takes the waker, releases the lock, and wakes it. Figure 18.1 shows the exchange.

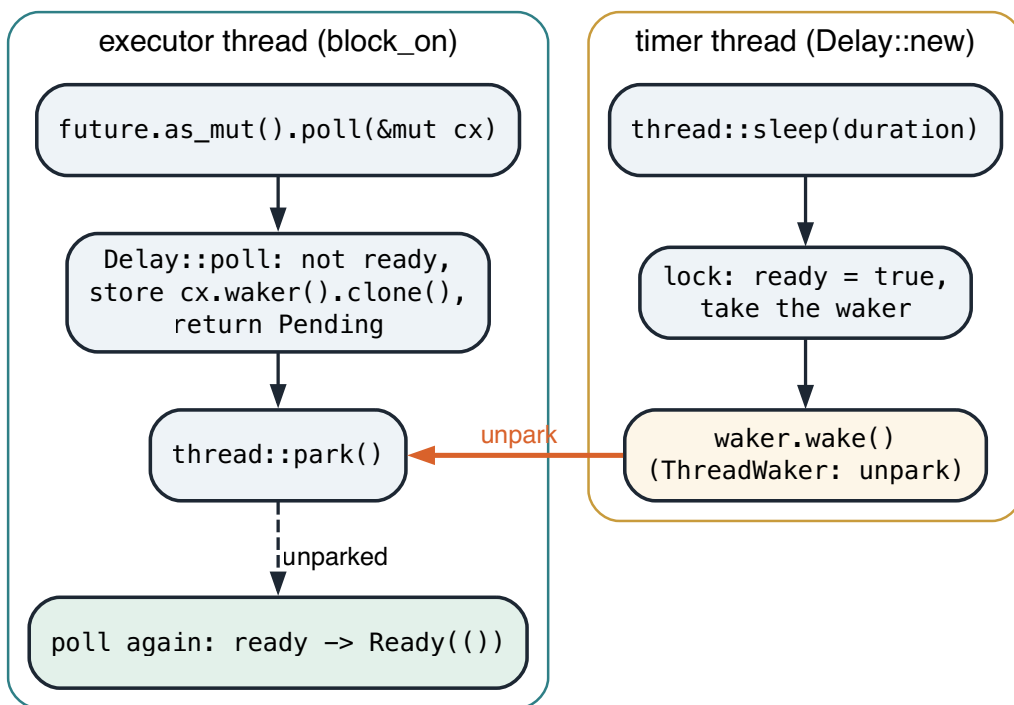


Figure 18.1 One `Delay` from start to finish. The executor thread sleeps in `park`, not in `poll`.

Two details make it correct. `poll` stores the waker on *every* `Pending`, not only the first. A future may be polled with a different waker each time, as when a runtime moves the task between threads. Only the latest waker is guaranteed to reach the current owner. The second detail is the order in the timer thread: it calls `wake()` *after* dropping the lock. The woken executor therefore does not immediately block on the mutex the waker is still holding.

A real runtime replaces "a thread per timer" with one timer wheel. It replaces the thread for I/O with the `epoll` loop from chapter 16. The reactor stores wakers by file descriptor, and wakes them when `epoll_wait` reports the descriptor ready.

A future is polled, parks the thread, and is woken

A poll returns Pending until the work is done, and the waker schedules the next poll.

□ pending □ ready □ parked



The executor polls the future. The work is not finished, so the future stores the waker and returns Pending.

The waker is the handoff: without it the executor would have to poll in a loop.

state = **Pending**

phase = **poll**

queue = **1 task**

Animation 18.1 One future from first poll to Ready, matching the exchange in figure 18.1. Polling returns Pending as long as the work is not done, and each Pending stores the waker. The executor then parks, which costs no CPU, and the waker is the only thing that can schedule the next poll. This is why an executor that ignored the waker would hang forever rather than spin.

18.4 An executor with a task queue

`block_on` runs one future. `MiniExecutor` runs many, and it shows how waking and scheduling are the same operation.

A `Task` holds its future as `Mutex<Pin<Box<dyn Future<Output = ()> + Send>>>`, a handle to the executor's queue, and a `completed` flag. `Task` implements `Wake`, and waking it pushes an `Arc` of the task back on the queue: to wake a task is to schedule it. `spawn` wraps a future in a task and enqueues it. `run` pops tasks one at a time, builds a waker from the task itself,

and polls it. A `Ready` result marks the task completed, so a stale wake that arrives later is skipped by the `completed` check.

`run` returns when the queue is empty. Its doc comment says why that is the right behavior for this executor. A task that returned `Pending` without waking itself is waiting on something outside the executor. A runtime with timers or I/O would park on its event source at that point instead of returning.

The `+ Send` bound on spawned futures is there because the task, and so the future inside it, is reachable from a `Waker`. Wakers may be sent to other threads, as `Delay` does. This is the same bound that `tokio::spawn` imposes. It is also why holding a `std::sync::MutexGuard` or an `Rc` across an `.await` in a spawned task is a compile error.

`async_demo` exercises all of it:

Listing 18.2 The runtime in use. `src/bin/async_demo.rs`

```

    ///! Runs the hand-written async runtime in ['systems_lab::problems::async_mini'].
    ///!
    ///! There is no Tokio here on purpose: the point is to show that 'Future', 'Waker',
    ///! 'Poll', and 'Pin' are enough to build an executor, and that '.await' is just
    ///! sugar over a state machine that is polled.
    ///!
    ///! ```bash
    ///! cargo run --bin async_demo
    ///! ```

    use systems_lab::problems::async_mini::{Delay, MiniExecutor, YieldTimes, block_on};
    use std::sync::Arc;
    use std::sync::atomic::{AtomicUsize, Ordering};
    use std::time::{Duration, Instant};

    fn main() {
        println!("== block_on drives a hand-written future ==");
        let yields = block_on(YieldTimes::new(3));
        println!("YieldTimes suspended and resumed {yields} time(s)");

        println!("\n== block_on drives an async block ==");
        let total = block_on(async {
            let first = YieldTimes::new(1).await;
            let second = YieldTimes::new(2).await;
            first + second
        });
        println!("async block finished with {total}");

        println!("\n== block_on parks the thread until the waker fires ==");
        let start = Instant::now();
        block_on(Delay::new(Duration::from_millis(50)));
        println!("Delay completed after {:?}", start.elapsed());

        println!("\n== MiniExecutor runs independent tasks ==");
        let executor = MiniExecutor::new();

```

```

    let completed = Arc::new(AtomicUsize::new(0));

    for id in 1..=3 {
        let completed = Arc::clone(&completed);
        executor.spawn(async move {
            YieldTimes::new(id).await;
            completed.fetch_add(1, Ordering::SeqCst);
            println!("task {id} completed after {id} yield(s)");
        });
    }

    executor.run();
    println!("{}", task(s) completed", completed.load(Ordering::SeqCst));
}

```

```

== block_on drives a hand-written future ==
YieldTimes suspended and resumed 3 time(s)

== block_on drives an async block ==
async block finished with 3

== block_on parks the thread until the waker fires ==
Delay completed after 53.370994ms

== MiniExecutor runs independent tasks ==
task 1 completed after 1 yield(s)
task 2 completed after 2 yield(s)
task 3 completed after 3 yield(s)
3 task(s) completed

```

The tasks finish in order 1, 2, 3 because each yield sends a task to the back of the queue. The tasks take turns: a round-robin scheduler, as in chapter 11, with a yield as the end of a time slice.

18.5 What `async` compiles to, and why `Pin`

The `async` block in `main` is:

```

async {
    let first = YieldTimes::new(1).await;
    let second = YieldTimes::new(2).await;
    first + second
}

```

The compiler turns it into an anonymous type that implements `Future`. That type is an enum with one variant per suspension point. Each variant holds the variables that are alive across

that point (figure 18.2). Polling it runs the code up to the next `.await` whose inner future returns `Pending`, saves the state, and returns `Pending` too.

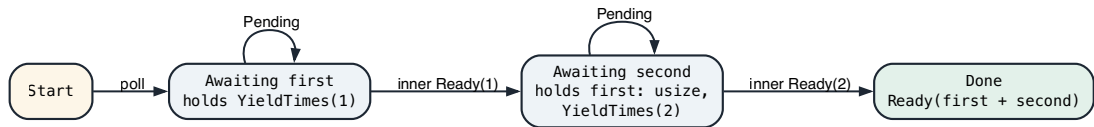


Figure 18.2 The state machine behind the `async` block. The value `first` survives the second `.await`, so it is stored in the second state.

A state can hold a reference to another field of the same state machine. One example is a borrow of a local buffer that is still in use across an `.await`. If the state machine moved in memory after such a borrow was created, the reference would point at the old location. `Pin<&mut Self>` is the promise that the value will not move again after the first poll. That is why `block_on` pins the future before polling it, and why `MiniExecutor` stores `Pin<Box<...>>`. Types without self-references, such as `YieldTimes`, are `Unpin`, and for them pinning imposes nothing.

18.6 Two HTTP clients with retries and caching

The last three programs apply chapter 15's reliability patterns to real HTTP calls. One uses nothing but the standard library, and one uses Tokio and `reqwest`. Both send `POST /post` with a JSON body to `httpbin.org`, a public echo service, under the key `create-demo-001`. Figure 18.3 compares their layering.

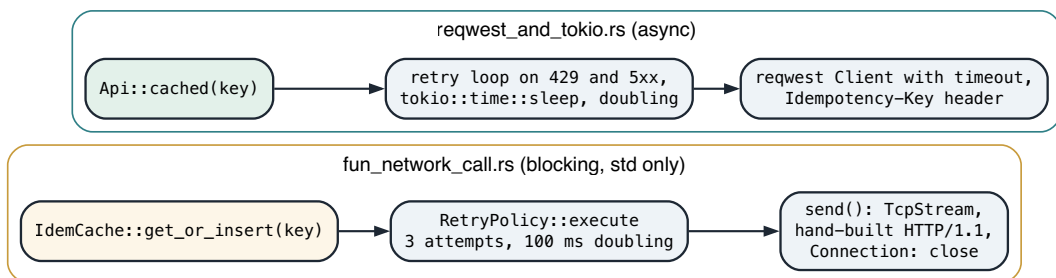


Figure 18.3 The same three layers in both clients: cache, retry, transport.

18.6.1 A blocking client over TcpStream

Listing 18.3 Cache, retry, and a hand-written HTTP/1.1 request. src/bin/fun_network_call.rs

```
use std::collections::HashMap;
use std::io::{Read, Write};
use std::net::TcpStream;
use std::sync::Mutex;
use std::time::Duration;

type RuntimeError = Box<dyn std::error::Error>;

/// header represents vec![(key, value)];
type RequestHeader = Vec<(String, String)>;
enum RequestType {
    GET    { path: String, headers: Vec<(String, String)> },
    OPTION { path: String, headers: Vec<(String, String)> },
    POST   { path: String, headers: Vec<(String, String)>, body: String },
    PUT    { path: String, headers: Vec<(String, String)>, body: String },
}

/// An idempotency cache store to store requests if it's succeeded!
struct IdemCache {
    store: Mutex<HashMap<String, String>>,
}

impl IdemCache {
    fn new() -> Self {
        Self { store: Mutex::new(HashMap::new()) }
    }

    fn get_or_insert<F>(&self, key: &str, f: F) -> Result<String, RuntimeError>
    where
        F: FnOnce() -> Result<String, RuntimeError>,
    {
        let mut store = self.store.lock().map_err(|_| "lock poisoned"?);
        if let Some(v) = store.get(key) {
            return Ok(v.clone());
        }
        let v = f()?;
        store.insert(key.to_string(), v.clone());
        Ok(v)
    }
}

struct RetryPolicy {
    max_attempts: u32,
    base_delay: Duration,
    max_delay: Duration,
}

impl RetryPolicy {
    fn new() -> Self {
        Self {
            max_attempts: 3,
            base_delay: Duration::from_millis(100),
            max_delay: Duration::from_secs(5),
        }
    }
}
```

```

    }
}

fn execute<F, T>(&self, mut f: F) -> Result<T, RuntimeError>
where
    F: FnMut() -> Result<T, RuntimeError>,
{
    let mut attempt = 0;
    loop {
        attempt += 1;
        match f() {
            Ok(v) => return Ok(v),
            Err(e) if attempt >= self.max_attempts => return Err(e),
            Err(_) => {
                let delay = (self.base_delay * 2u32.pow(attempt - 1)).min(self.max_delay);
                std::thread::sleep(delay);
            }
        }
    }
}

/// -- Build a HTTP client with those strategies

struct HttpClient {
    host: String,
    port: u16,
    cache: IdemCache,
    retry: RetryPolicy,
}

impl HttpClient {
    fn new(host: &str, port: u16) -> Self {
        Self {
            host: host.to_string(),
            port,
            cache: IdemCache::new(),
            retry: RetryPolicy::new(),
        }
    }

    fn execute(&self, key: &str, req: &RequestType) -> Result<String, RuntimeError> {
        self.cache.get_or_insert(key, || self.retry.execute(|| self.send(req)))
    }

    fn send(&self, req: &RequestType) -> Result<String, RuntimeError> {
        let (method, path, headers, body) = match req {
            RequestType::GET { path, headers } => ("GET", path, headers,
None),
            RequestType::OPTION { path, headers } => ("OPTIONS", path, headers,
None),
            RequestType::POST { path, headers, body } => ("POST", path, headers,
Some(body)),
            RequestType::PUT { path, headers, body } => ("PUT", path, headers,
Some(body)),
        };

        let mut raw = format!(

```

```

        "{method} {path} HTTP/1.1\r\nHost: {}\r\nConnection: close\r\n",
        self.host
    );
    for (k, v) in headers { raw += &format!("{k}: {v}\r\n"); }
    if let Some(b) = body { raw += &format!("Content-Length: {}\r\n", b.len()); }
    raw += "\r\n";
    if let Some(b) = body { raw += b; }

    let mut stream = TcpStream::connect((self.host.as_str(), self.port))?;
    stream.write_all(raw.as_bytes())?;
    let mut resp = String::new();
    stream.read_to_string(&mut resp)?;
    Ok(resp.split_once("\r\n\r\n").map(|(_, b)| b).unwrap_or("").to_string())
}

// --- The actual REST call ---

fn create_item(client: &HttpClient, item_name: &str) -> Result<String, RuntimeError> {
    client.execute("create-demo-001", &RequestType::POST {
        path: "/post".into(),
        headers: vec![
            ("Content-Type".into(), "application/json".into()),
            ("Accept".into(), "application/json".into()),
        ],
        body: format!(r#"{{"name":"{item_name}"}}"#),
    })
}

fn main() -> Result<(), RuntimeError> {
    let client = HttpClient::new("httpbin.org", 80);
    println!("created:\n{}", create_item(&client, "demo")?);
    Ok(())
}

```

The design is three layers composed in one line:

```
self.cache.get_or_insert(key, || self.retry.execute(|| self.send(req)))
```

`IdemCache` is chapter 15's idempotency store under another name. `RetryPolicy::execute` retries any error with a doubling delay, `base × 2(attempt - 1)`, capped at `max_delay`. `RequestType` is an enum whose variants carry exactly the fields each method needs. Then `send` flattens it with one `match` into a tuple of method, path, headers, and optional body.

`send` writes the request by hand: request line, `Host`, `Connection: close`, the caller's headers, `Content-Length` when there is a body, a blank line, the body. `Connection: close` lets `read_to_string` work as the response reader. The server closes the connection after its response, so reading to end-of-file reads exactly one response. Chapter 17's parser is the other side of this conversation.

Two gaps separate it from a client you would ship, and both are visible in the code:

- **The status code is never read.** `send` returns whatever follows the first blank line. A 500 with an error page is returned as success, cached under the idempotency key, and never retried. Only transport errors (connect, write, read) reach the retry loop.
- **Everything is retried.** A retry policy should consult the error, as `Retryable` does in chapter 15. Here a DNS failure for a misspelled host is retried three times.

The `RequestHeader` alias and the `OPTION` variant are declared but not used.

18.6.2 An async client with `request`

Listing 18.4 The same layers on Tokio. `src/bin/request_and_tokio.rs`

```
use std::{collections::HashMap, sync::Mutex, time::Duration};
use request::{Client, Method, StatusCode};
use serde_json::Value;

type Error = Box<dyn std::error::Error + Send + Sync>;

const RETRY: [StatusCode; 5] = [
    StatusCode::TOO_MANY_REQUESTS,
    StatusCode::INTERNAL_SERVER_ERROR,
    StatusCode::BAD_GATEWAY,
    StatusCode::SERVICE_UNAVAILABLE,
    StatusCode::GATEWAY_TIMEOUT,
];

struct Config {
    base: String,
    max_attempts: u32,
    base_delay: Duration,
    max_delay: Duration,
    timeout: Duration,
}

struct Api {
    http: Client,
    cache: Mutex<HashMap<String, String>>,
    cfg: Config,
}

impl Api {
    fn new(cfg: Config) -> Result<Self, Error> {
        Ok(Self {
            http: Client::builder().timeout(cfg.timeout).build()?,
            cache: Mutex::new(HashMap::new()),
            cfg,
        })
    }

    fn cached(&self, key: &str) -> Option<String> {
```

```

        self.cache
            .lock()
            .unwrap_or_else(|e| e.into_inner())
            .get(key)
            .cloned()
    }

    fn store(&self, key: &str, text: &str) {
        self.cache
            .lock()
            .unwrap_or_else(|e| e.into_inner())
            .insert(key.into(), text.into());
    }

    async fn call(
        &self,
        key: &str,
        method: Method,
        path: &str,
        body: Option<&Value>,
    ) -> Result<String, Error> {
        match self.cached(key) {
            Some(hit) => return Ok(hit),
            None => {}
        }

        let url = format!("{}", self.cfg.base, path);
        let mut delay = self.cfg.base_delay;
        let mut attempt = 0;

        loop {
            attempt += 1;

            let req = self
                .http
                .request(method.clone(), &url)
                .header("Idempotency-Key", key)
                .header("Accept", "application/json");
            let req = match body {
                Some(b) => req.json(b),
                None => req,
            };

            let resp = req.send().await?;
            let status = resp.status();
            let text = resp.text().await?;

            match status {
                s if s.is_success() => {
                    self.store(key, &text);
                    return Ok(text);
                }
                s if RETRY.contains(&s) && attempt < self.cfg.max_attempts => {
                    tokio::time::sleep(delay).await;
                    delay = (delay * 2).min(self.cfg.max_delay);
                }
                s => return Err(format!("HTTP {s} after {attempt} attempts: {text}").into())
            }
        }
    },
}

```



```

    }
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let api = Api::new(Config {
        base: "https://httpbin.org".into(),
        max_attempts: 3,
        base_delay: Duration::from_millis(100),
        max_delay: Duration::from_secs(5),
        timeout: Duration::from_secs(15),
    })?;

    let body = serde_json::json!({ "name": "demo" });
    let out = api
        .call("create-demo-001", Method::POST, "/post", Some(&body))
        .await?;
    println!("created:\n{out}");
    Ok(())
}

```

This client reads the status code and makes a decision about it. `RETRY` lists the statuses to retry: 429 and four 5xx codes. The `match` on `status` then has three arms with guards. Success stores and returns. A retryable status with attempts left sleeps with `tokio::time::sleep(delay).await` and doubles the delay. Anything else returns an error that includes the status, the attempt count, and the response body.

`tokio::time::sleep` does not block the thread. It returns a future that registers with Tokio's timer and returns `Pending`, which is the `Delay` of section 18.3 done properly. Other tasks then run on the same thread while this one waits.

Several choices in this client are deliberate:

- `Client::builder().timeout(cfg.timeout)` bounds every request. A client without a timeout can wait forever on a server that accepted the connection and never answered.
- The request carries `Idempotency-Key: create-demo-001`, so a server that honors the header can safely deduplicate the retried `POST`. The client-side cache gives the same guarantee within one process.
- The cache is a `std::sync::Mutex`, which is fine in async code because the guard is never held across an `.await`. `cached` and `store` lock, act, and return. The lock is taken with `unwrap_or_else(|e| e.into_inner())`, chapter 12's "ignore the poison" choice. That is safe here because a `HashMap` insert cannot leave the map half-updated.
- `type Error = Box<dyn std::error::Error + Send + Sync>` so errors can cross task boundaries.

Compared with chapter 15's `retry`, this loop has no jitter and ignores `Retry-After`. Transport errors from `send().await?` are not retried at all, which is the opposite gap from the blocking client. Merging the two means retrying transport errors and retryable statuses with a policy that asks the error. That is the exercise the two files set up.

18.6.3 A new project, started

Listing 18.5 A fresh Tokio project, not yet written. `rust-async-http-examples/`

```
[package]
name = "rust-async-http-examples"
version = "0.1.0"
edition = "2024"

[dependencies]
tokio = { version = "1.53.1", features = ["full"] }
```

```
fn main() {
    println!("Hello, world!");
}
```

The repository's newest crate is `cargo new` output with Tokio added as a dependency. A natural first program for it is the client from section 18.6.2 as a library. The retry and cache layers go in `lib.rs`. Its tests can then run against a local server such as chapter 17's, instead of a public one.

18.7 Questions that come up

"What happens when a future returns `Pending`?" It has arranged for its waker to be called when it can make progress. The executor stops polling it and runs other tasks. When the waker fires, the executor polls it again.

"Why does `poll` take `Pin<&mut Self>?`" Async state machines can hold references into themselves across `.await` points. Pinning guarantees the value will not move after it has been polled, so those references stay valid.

"Why must spawned futures be `Send` in Tokio?" The multi-threaded runtime may resume a task on a different worker thread after any `.await`. Everything the task holds across an `.await` must therefore be safe to move between threads.

"Can you call blocking code inside `async`?" Only briefly. A blocking call holds the worker thread, and every task scheduled on it waits. Use `tokio::task::spawn_blocking` for blocking work, and `async` versions of I/O and sleep.

"Is a `std::sync::Mutex` allowed in `async` code?" Yes, if the guard is dropped before the next `.await`. Holding it across an `.await` can deadlock or block the worker; use `tokio::sync::Mutex` when the lock must be held across one.

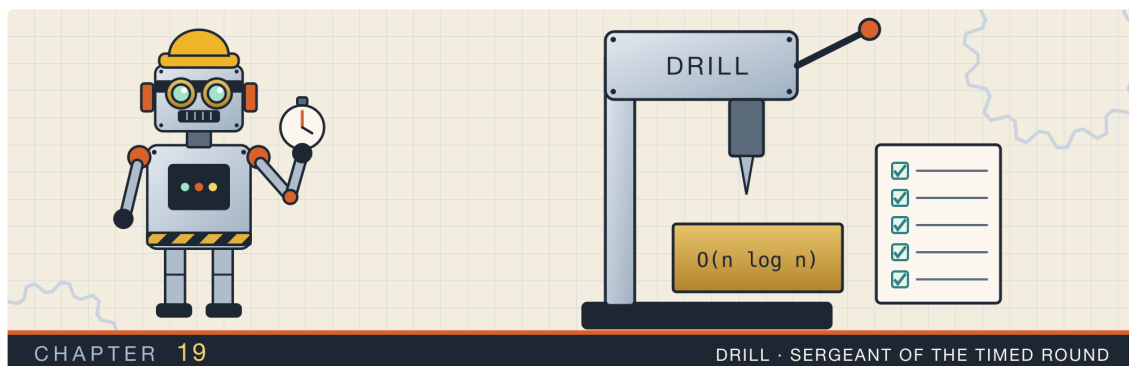
Summary

- `poll` returns `Ready` or `Pending`. `Pending` is a promise that the waker will be called when progress becomes possible.
- `block_on` is an executor in fifteen lines: `poll`, park on `Pending`, then `poll` again when the wake arrives. Spurious wakes are harmless; missing wakes are not.
- `YieldTimes` and `Delay` are the two shapes of a future. One re-arms itself and asks to be polled again. The other waits on an event outside the executor, and keeps the waker of whoever is waiting for it.
- Waking a task and scheduling it are the same operation. `Task` implements `Wake`, and `wake` puts the task back on the queue.
- An `async` block compiles to an enum with one variant per suspension point. Each variant holds the variables alive across that point. A variant may borrow from another field of the same enum, so `poll` takes `Pin<&mut Self>`. Pinning guarantees the value cannot move after the first `poll`, so those references stay valid.
- A spawned future must be `Send`. A runtime may resume the task on a different worker thread, so nothing that is not `Send` may be held across an `.await`.
- The two HTTP clients are the same three layers: cache, retry, and transport. One is blocking over `TcpStream`, the other runs on Tokio. Comparing them shows both gaps: the blocking client never reads the status code, and the `async` one never retries a transport error.

Chapter 19 collects compact implementations of the structures from parts 2 to 6. Each one is the shortest program that still shows the structure working, and each fits on one screen.

PART 7

Compact implementations



Drills against the clock

THIS CHAPTER COVERS

- Ten short implementations, each sized to type in a live round
- Where each drill's full version lives in this book
- Two drills that differ from their reference versions, one of them in a way that changes the answer
- A plan for a 45-minute round, and a way to practice with this repository

Every chapter so far has shown the long form: the reference implementation with its explanations, its tests, and its history. In a live round you will not have that. You will have a blank editor, a problem stated in two sentences, and about forty minutes. `drills.rs` is the short form: ten answers written at the length you can type under that pressure. Each group carries one test, so you know it runs.

This chapter is a guide to using them. It maps each drill to its chapter, reads the places where the short form cuts a corner, and ends with a way to rehearse.

19.1 The drills

Listing 19.1 Ten answers sized for a live round. `src/problems/drills.rs`

```
/// Compact implementations of the common primitives.  
///  
/// Each item is a short standalone implementation. The modules next to this one
```

```
//! are the full versions with long explanations and complete test suites.
```

```
use std::cell::UnsafeCell;
use std::collections::VecDeque;
use std::collections::hash_map::DefaultHasher;
use std::hash::Hasher;
use std::ops::{Deref, DerefMut};
use std::sync::atomic::{AtomicBool, Ordering};
use std::sync::{Arc, Condvar, Mutex, mpsc};
use std::thread::{self, JoinHandle};
use std::time::Duration;

// --- Thread-safe counter (Arc + Mutex) ---

pub fn count_parallel(threads: usize, per_thread: usize) -> usize {
    let counter = Arc::new(Mutex::new(0usize));
    let handles: Vec<_> = (0..threads)
        .map(|_| {
            let counter = Arc::clone(&counter);
            thread::spawn(move || {
                for _ in 0..per_thread {
                    *counter.lock().expect("poisoned") += 1;
                }
            })
        })
        .collect();

    for handle in handles {
        handle.join().expect("worker panicked");
    }
    *counter.lock().expect("poisoned")
}

// --- Scoped threads (borrow instead of Arc) ---

pub fn parallel_sum(values: &[i32]) -> i64 {
    let chunk_size = values.len().div_ceil(4).max(1);
    thread::scope(|scope| {
        values
            .chunks(chunk_size)
            .map(|chunk| {
                scope.spawn(move || chunk.iter().map(|value|
i64::from(*value)).sum::<i64>())
            })
            .map(|handle| handle.join().expect("worker panicked"))
            .sum()
    })
}

// --- Spin lock from an atomic ---

pub struct SpinLock<T> {
    locked: AtomicBool,
    value: UnsafeCell<T>,
}

unsafe impl<T: Send> Sync for SpinLock<T> {}

impl<T> SpinLock<T> {
```

```

pub const fn new(value: T) -> Self {
    Self {
        locked: AtomicBool::new(false),
        value: UnsafeCell::new(value),
    }
}

pub fn lock(&self) -> SpinGuard<'_, T> {
    while self.locked.swap(true, Ordering::Acquire) {
        std::hint::spin_loop();
    }
    SpinGuard { lock: self }
}

pub struct SpinGuard<'a, T> {
    lock: &'a SpinLock<T>,
}

impl<T> Drop for SpinGuard<'_, T> {
    fn drop(&mut self) {
        self.lock.locked.store(false, Ordering::Release);
    }
}

impl<T> Deref for SpinGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T {
        unsafe { &*self.lock.value.get() }
    }
}

impl<T> DerefMut for SpinGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T {
        unsafe { &mut *self.lock.value.get() }
    }
}

// --- Semaphore (acquire blocks) ---

pub struct Semaphore {
    permits: Mutex<usize>,
    released: Condvar,
}

impl Semaphore {
    pub fn new(permits: usize) -> Self {
        Self {
            permits: Mutex::new(permits),
            released: Condvar::new(),
        }
    }

    pub fn acquire(&self) {
        let mut permits = self.permits.lock().expect("poisoned");
        while *permits == 0 {
            permits = self.released.wait(permits).expect("poisoned");
        }
        *permits -= 1;
    }
}

```

```

    }

    pub fn try_acquire(&self) -> bool {
        let mut permits = self.permits.lock().expect("poisoned");
        if *permits == 0 {
            false
        } else {
            *permits -= 1;
            true
        }
    }

    pub fn release(&self) {
        *self.permits.lock().expect("poisoned") += 1;
        self.released.notify_one();
    }
}

// --- Bounded blocking queue (backpressure + close) ---

pub struct BlockingQueue<T> {
    inner: Mutex<Inner<T>>,
    not_empty: Condvar,
    not_full: Condvar,
}

struct Inner<T> {
    items: VecDeque<T>,
    capacity: usize,
    closed: bool,
}

impl<T> BlockingQueue<T> {
    pub fn new(capacity: usize) -> Self {
        Self {
            inner: Mutex::new(Inner {
                items: VecDeque::new(),
                capacity,
                closed: false,
            }),
            not_empty: Condvar::new(),
            not_full: Condvar::new(),
        }
    }

    pub fn push(&self, item: T) -> Result<(), T> {
        let mut inner = self.inner.lock().expect("poisoned");
        loop {
            if inner.closed {
                return Err(item);
            }
            if inner.items.len() < inner.capacity {
                inner.items.push_back(item);
                self.not_empty.notify_one();
                return Ok(());
            }
            inner = self.not_full.wait(inner).expect("poisoned");
        }
    }
}

```



```

pub fn pop(&self) -> Option<T> {
    let mut inner = self.inner.lock().expect("poisoned");
    loop {
        if let Some(item) = inner.items.pop_front() {
            self.not_full.notify_one();
            return Some(item);
        }
        if inner.closed {
            return None;
        }
        inner = self.not_empty.wait(inner).expect("poisoned");
    }
}

pub fn close(&self) {
    let mut inner = self.inner.lock().expect("poisoned");
    inner.closed = true;
    drop(inner);
    self.not_empty.notify_all();
    self.not_full.notify_all();
}

// --- Thread pool (fixed workers + channel) ---

pub struct ThreadPool {
    sender: Option<mpsc::Sender<Box<dyn FnOnce() + Send + 'static>>>,
    workers: Vec<JoinHandle<>>,
}

impl ThreadPool {
    pub fn new(size: usize) -> Self {
        let (sender, receiver) = mpsc::channel::<Box<dyn FnOnce() + Send + 'static>>();
        let receiver = Arc::new(Mutex::new(receiver));
        let workers = (0..size)
            .map(|_| {
                let receiver = Arc::clone(&receiver);
                thread::spawn(move || {
                    loop {
                        let Ok(job) = receiver.lock().expect("poisoned").recv() else {
                            return;
                        };
                        job();
                    }
                })
            })
            .collect();
        Self {
            sender: Some(sender),
            workers,
        }
    }

    pub fn execute(&self, job: impl FnOnce() + Send + 'static) {
        self.sender
            .as_ref()
            .expect("pool dropped")
            .send(Box::new(job))
    }
}

```

```

        .expect("worker gone");
    }
}

impl Drop for ThreadPool {
    fn drop(&mut self) {
        drop(self.sender.take());
        for worker in self.workers.drain(..) {
            let _ = worker.join();
        }
    }
}

// --- Retry with exponential backoff ---

pub fn retry<T, E>(
    max_attempts: u32,
    mut operation: impl FnMut() -> Result<T, E>,
    retryable: impl Fn(&E) -> bool,
) -> Result<T, E> {
    let mut delay = Duration::from_millis(25);
    let mut attempt = 0;
    loop {
        match operation() {
            Ok(value) => return Ok(value),
            Err(error) => {
                attempt += 1;
                if attempt >= max_attempts || !retryable(&error) {
                    return Err(error);
                }
                thread::sleep(delay);
                delay = (delay * 2).min(Duration::from_secs(1));
            }
        }
    }
}

// --- Consistent hashing ring ---

#[derive(Default)]
pub struct HashRing {
    points: Vec<(u64, String)>,
}

impl HashRing {
    pub fn new() -> Self {
        Self::default()
    }

    pub fn add(&mut self, node: &str, replicas: usize) {
        for replica in 0..replicas {
            self.points.push((hash(node, replica), node.to_string()));
        }
        self.points.sort_unstable_by_key(|(point, _)| *point);
    }

    pub fn get(&self, key: &str) -> Option<&str> {
        if self.points.is_empty() {
            return None;
        }
    }
}

```

```

    }
    let hash = hash(key, 0);
    let index = self.points.partition_point(|(point, _)| *point <= hash) %
self.points.len();
    Some(self.points[index].1.as_str())
  }
}

fn hash(input: &str, replica: usize) -> u64 {
  let mut hasher = DefaultHasher::new();
  hasher.write(input.as_bytes());
  hasher.write_usize(replica);
  hasher.finish()
}

// --- HTTP request head ---

pub struct RequestHead {
  pub method: String,
  pub target: String,
  pub headers: Vec<(String, String)>,
}

pub fn parse_request_head(input: &str) -> Option<RequestHead> {
  let mut lines = input.split("\r\n");
  let mut request_line = lines.next()?.split(' ');
  let method = request_line.next()?.to_string();
  let target = request_line.next()?.to_string();
  if request_line.next()? != "HTTP/1.1" {
    return None;
  }

  let mut headers = Vec::new();
  for line in lines {
    if line.is_empty() {
      break;
    }
    let (name, value) = line.split_once(':');
    headers.push((name.trim().to_ascii_lowercase(), value.trim().to_string()));
  }
  Some(RequestHead {
    method,
    target,
    headers,
  })
}

// --- Newtype and enum ADT ---

pub struct Port(u16);

impl Port {
  pub fn new(value: u16) -> Result<Self, &'static str> {
    (1..=u16::MAX)
      .contains(&value)
      .then_some(Self(value))
      .ok_or("port must be non-zero")
  }
}

```

```

    pub fn get(self) -> u16 {
        self.0
    }
}

pub enum Route {
    Health,
    List,
    ById(String),
}

impl Route {
    pub fn parse(path: &str) -> Option<Self> {
        match path {
            "/healthz" => Some(Self::Health),
            "/v1/gpu-workloads" => Some(Self::List),
            _ => path
                .strip_prefix("/v1/gpu-workloads/")
                .map(|id| Self::ById(id.to_string())),
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::atomic::AtomicUsize;

    #[test]
    fn counter_scoped_sum_and_spin_lock() {
        assert_eq!(count_parallel(4, 100), 400);
        assert_eq!(parallel_sum(&[1, 2, 3, 4]), 10);

        let lock = Arc::new(SpinLock::new(0usize));
        let handles: Vec<_> = (0..4)
            .map(|_| {
                let lock = Arc::clone(&lock);
                thread::spawn(move || {
                    for _ in 0..100 {
                        *lock.lock() += 1;
                    }
                })
            })
            .collect();
        for handle in handles {
            handle.join().expect("worker panicked");
        }
        assert_eq!(*lock.lock(), 400);
    }

    #[test]
    fn semaphore_queue_and_pool() {
        let semaphore = Semaphore::new(1);
        assert!(semaphore.try_acquire());
        assert!(semaphore.try_acquire());
        semaphore.release();
        assert!(semaphore.try_acquire());

        let queue = BlockingQueue::new(1);
    }
}

```

```

    assert_eq!(queue.push(1), Ok(()));
    queue.close();
    assert_eq!(queue.pop(), Some(1));
    assert_eq!(queue.pop(), None);
    assert_eq!(queue.push(2), Err(2));

    let pool = ThreadPool::new(3);
    let counter = Arc::new(AtomicUsize::new(0));
    for _ in 0..100 {
        let counter = Arc::clone(&counter);
        pool.execute(move || {
            counter.fetch_add(1, Ordering::SeqCst);
        });
    }
    drop(pool);
    assert_eq!(counter.load(Ordering::SeqCst), 100);
}

#[test]
fn retry_ring_http_and_adt() {
    let mut attempts = 0;
    let ok = retry(
        3,
        || {
            attempts += 1;
            if attempts < 3 { Err("transient") } else { Ok(7) }
        },
        |_| true,
    );
    assert_eq!(ok, Ok(7));

    let mut attempts = 0;
    let bad: Result<(), &str> = retry(
        3,
        || {
            attempts += 1;
            Err("permanent")
        },
        |_| false,
    );
    assert!(bad.is_err());
    assert_eq!(attempts, 1);

    let mut ring = HashRing::new();
    ring.add("a", 64);
    ring.add("b", 64);
    let before = ring.get("job-1").map(str::to_string);
    ring.add("c", 64);
    let after = ring.get("job-1").map(str::to_string);
    assert!(after == before || after.as_deref() == Some("c"));

    let head =
        parse_request_head("GET /v1/gpu-workloads HTTP/1.1\r\nHost: api\r\n\r\n").expect("valid");
    assert_eq!(
        (head.method.as_str(), head.target.as_str()),
        ("GET", "/v1/gpu-workloads")
    );
    assert_eq!(head.headers[0], ("host".to_string(), "api".to_string()));
}

```

```

    assert!(parse_request_head("GET / HTTP/2\r\n\r\n").is_none());

    assert!(Port::new(8080).is_ok());
    assert!(Port::new(0).is_err());
    assert!(matches!(Route::parse("/healthz"), Some(Route::Health)));
    assert!(matches!(
        Route::parse("/v1/gpu-workloads/wl-1"),
        Some(Route::ById(id)) if id == "wl-1"
    ));
}
}

```

The module comment states the file's role in one line: the neighbouring modules are the reference versions, and this file is what you type. Table 19.1 pairs each drill with the chapter that explains its full form.

Drill	Lines	Full version
<code>count_parallel: Arc<Mutex<usize>></code> counter	18	chapter 1 (<code>total_with_arc_mutex</code>), chapter 12
<code>parallel_sum: scoped threads</code>	12	chapter 12, <code>threads::parallel_sum</code>
<code>SpinLock</code> and <code>SpinGuard</code>	45	chapter 12, listing 12.4
Semaphore	36	chapter 12, listing 12.5
<code>BlockingQueue</code> with <code>close</code>	62	chapter 13, listing 13.6
<code>ThreadPool</code> with <code>Drop</code>	45	chapter 14, listing 14.5
<code>retry</code> with backoff	21	chapter 15, listing 15.4
HashRing	33	chapter 10, listing 10.2
<code>parse_request_head</code>	29	chapter 17, listing 17.1
Port newtype and Route enum	32	chapter 1, listing 1.3; chapter 17, <code>route</code>

19.2 What the short forms leave out

A drill is allowed to be simpler than its reference. Knowing *what* it left out is part of the answer, because the follow-up question is usually aimed at exactly that.

The spin lock uses `swap. while self.locked.swap(true, Acquire) { spin_loop() }` is a test-and-set lock: every waiting thread writes the lock's cache line on every iteration. The reference version in chapter 12 uses `compare_exchange_weak` and spins on a *load*, so waiters

share the line read-only until it is released. Both are correct; the reference scales better under contention. The drill also omits the `SAFETY` comments, which in a real review you would add back.

The semaphore has no guard. `acquire` and `release` are separate calls, so a caller that panics between them leaks a permit. The reference returns a RAIL guard. In exchange, the drill's `acquire` really does hold the permit it takes, which section 12.4 found the reference's `acquire` does not.

The thread pool receives in a let-else. `let Ok(job) = receiver.lock().expect(..).recv() else { return; };` is a `let` statement, so the `MutexGuard` temporary is dropped at its end, before `job()` runs. The pool is fully parallel, which a timing confirms: four 200 ms jobs on four workers take 205 ms. The same receive written as `while let` would serialize the pool (chapter 14, figure 14.2), so this is the form to memorize.

The hash ring wraps with %. `partition_point(..) % self.points.len()` maps an index one past the end back to 0 in one expression. The reference uses an `if` instead. The drill also hashes in a plainer way: it calls `hasher.write(input.as_bytes())` and `write_usize(replica)` directly. It stores a `String` per virtual node where the reference shares an `Arc<str>`.

The request parser checks the version and little else. `parse_request_head` accepts only `HTTP/1.1`, splits headers at the first colon, and lowercases the names. It has no size limits, no `Host` check, and no body framing. That is the right scope for a ten-minute question about a request head. For a question about serving requests, chapter 17's checks are the answer.

Port::new is a one-expression validator. `(1..=u16::MAX).contains(&value).then_some(Self(value)).ok_or(..)` reads left to right as the rule. `Route::parse` matches the two fixed paths and falls back to `strip_prefix` for the id route, returning `None` for everything else.

19.3 The drill that is slower than it looks

`parallel_sum` changed behaviour in the short form. Compare it with the reference in chapter 12:

```
// drills.rs
values
```

```

        .chunks(chunk_size)
        .map(|chunk| scope.spawn(move || ...))
        .map(|handle| handle.join().expect("worker panicked"))
        .sum()

// threads.rs
let handles: Vec<_> = values.chunks(chunk_size).map(|chunk| scope.spawn(...)).collect();
handles.into_iter().map(|handle| handle.join().expect("worker panicked")).sum()

```

Iterators are lazy. In the drill, `sum` pulls one item through both `maps` before it asks for the next. The first chunk's thread is therefore spawned and then *joined* before the second chunk's thread is spawned. The four workers run one after another. The reference collects every `JoinHandle` into a `Vec` first, which spawns all four, and only then joins them.

The result is the same number either way, so the drill's test passes. Timing four workers that each sleep 200 ms shows the difference:

```

lazy spawn/join chain, four 200 ms workers: 809.402756ms (sum 4)
collect first, then join:                  203.482125ms (sum 4)

```

The fix is one `collect()` between the two `maps`. Chapter 14 drew the same lesson from the thread pool. A concurrency test that checks only the answer cannot tell parallel from serial. Put a barrier in it, as `thread_pool_v2`'s `jobs_run_at_the_same_time` does, or time it.

19.4 Running a round

A 45-minute coding round has a shape. Planning it keeps the last ten minutes, where most of the signal is, from being spent debugging (figure 19.1).

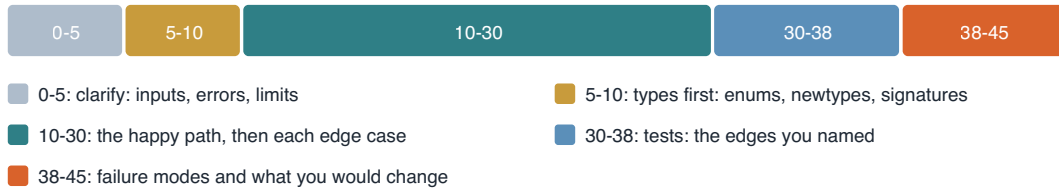
A 45-minute coding round, planned (minutes)

Figure 19.1 One way to spend a 45-minute round. The proportions matter more than the exact minutes.

- **Clarify.** Ask what the inputs are, what should happen on bad input, and what the limits are (size, concurrency, latency). Each answer becomes a type or a check.
- **Types first.** Write the enums, newtypes, and signatures before the bodies. A `Result<T, ParseError>` return type decided in minute six saves a rewrite in minute twenty-five.
- **The happy path, then each edge.** Get one case working end to end, then add the edges you listed.
- **Tests.** Test the edges you named, not only the example given.
- **Failure modes.** Say what breaks at scale and what you would change: the reference versions in this book are that list, written out.

19.5 Practicing with this repository

The repository is set up for the drill loop that the problem index in appendix C describes. Write a minimal correct version in 30 to 45 minutes, then explain its failure modes and how you would test them.

1. Pick a drill. Read its chapter once.
2. Close the book and the file. Write it from an empty buffer, in a scratch binary under `src/bin/`.
3. Run `cargo test` on it, then run the reference tests: `cargo test thread_pool`, `cargo test bounded_queue`, `cargo test --bin http_server`.
4. Compare with the reference. Write down what you left out, and whether it was a choice.
5. Say the failure modes out loud, as if to a reviewer.

The chapters' "Questions that come up" sections are the follow-ups to rehearse. The history this book has followed, ten merges, nine pools, six buffers, is what that loop looks like when you keep the drafts.

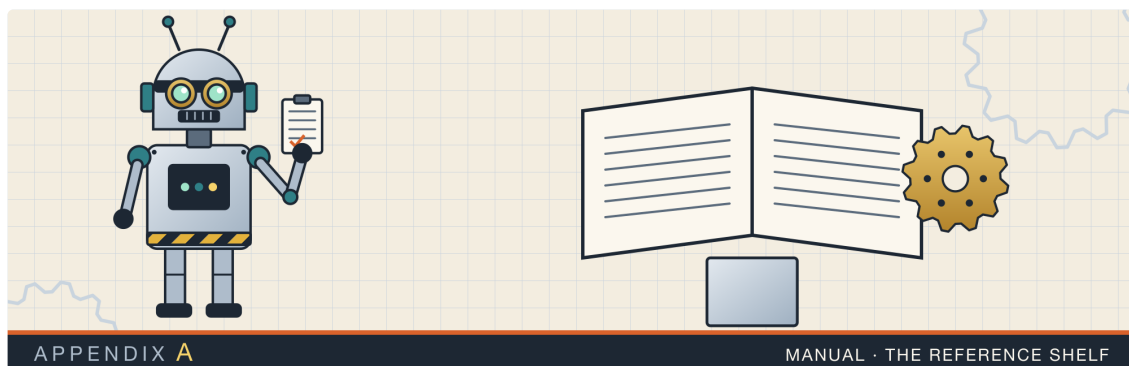
19.6 Questions that come up

"Your parallel sum gives the right answer. Is it parallel?" Check that all threads are spawned before any is joined. A lazy iterator that spawns and joins in one chain is serial.

"What would you add to this spin lock for production?" Spin on a relaxed load before retrying the exchange. Back off or yield under contention. Add `SAFETY` comments, and prefer `std::sync::Mutex` unless measurements say otherwise.

"Your semaphore's caller panicked. What happened to the permit?" It leaked. Return a guard whose `Drop` releases it.

"How did you decide what to leave out?" By the question's scope. Name what a full version adds, and where it would change the answer.



Appendix A. The Rust cheat sheet

This appendix reproduces `rust-interview-lab/src/bin/RUST_CHEATSHEET.md`, the page of compile errors and idioms that cost the most time while the Rust lab was being written. Its own instructions are the right way to use it: read a block, close the file, and type it from memory.

Its sections map onto the chapters of this book:

- reference levels and iterator closures: chapters 1 and 3
- `Option` and `Result` combinators: chapter 1
- collections and heaps: chapter 3
- locks and guards: chapters 12 to 14
- strings and match ergonomics: chapters 1 to 3

Rust cheat sheet – the gotchas that cost you minutes

Every snippet below was compiled with `rustc 1.96`, edition 2024. Comments in the code are the explanation.

Read a block, close the file, type it from memory. If you need to look, you don't have it yet.

0. The errors I hit while writing this sheet

These are real, not hypothetical. All three are the kind of thing that eats 5 minutes.

```
// 1. filter does not change the iterator's Item, even when you destructure it.
let d: Vec<i32> = v.iter().filter(|&&x| x > 2).collect();
// error[E0277]: a value of type `Vec<i32>` cannot be built from an iterator
//           over elements of type `i32`
let d: Vec<i32> = v.iter().filter(|&&x| x > 2).collect();           // ok
let d: Vec<i32> = v.iter().filter(|&&x| x > 2).copied().collect(); // ok

// 2. An unannotated float literal can stay ambiguous until a method needs the type.
let mut ys = vec![2.5, 1.0, 3.5];
ys.sort_by(|a, b| b.total_cmp(a));
// error[E0599]: no method named `total_cmp` found for reference `&{float}`
let mut ys: Vec<f64> = vec![2.5, 1.0, 3.5];           // ok

// 3. Option/Result combinators take `self`, so they move. A second call fails.
let r: Result<i32, String> = Ok(1);
let m = r.map(|v| v + 1);
let e = r.map_err(|err| err.len());
// error[E0382]: use of moved value: `r`
let m = r.as_ref().map(|v| v + 1);           // borrow instead
let e = r.clone().map_err(|err| err.len()); // or clone
```

1. Which reference level do I have?

The single most common source of noise. Memorise this table.

You have	You want	Write
<code>&Option<T></code>	<code>Option<&T></code>	<code>.as_ref()</code>
<code>&mut Option<T></code>	<code>Option<&mut T></code>	<code>.as_mut()</code>
<code>Option<String></code>	<code>Option<&str></code>	<code>.as_deref()</code>
<code>Option<Box<T>></code>	<code>Option<&T></code>	<code>.as_deref()</code>
<code>Option<Vec<T>></code>	<code>Option<&[T]></code>	<code>.as_deref()</code>
<code>Option<&T>, T: Copy</code>	<code>Option<T></code>	<code>.copied()</code>
<code>Option<&T>, T: Clone</code>	<code>Option<T></code>	<code>.cloned()</code>
<code>Option<T></code>	<code>Option<T> copy</code>	<code>.clone()</code>

```

let a: Option<String> = Some("x".into());

let r: Option<&String> = a.as_ref();      // &Option<T> -> Option<&T>
let s: Option<&str>    = a.as_deref();    // Option<String> -> Option<&str>
let c: Option<String> = a.clone();       // deep copy

let n: Option<i32> = Some(4);
let nr: Option<&i32> = n.as_ref();
let copied: Option<i32> = nr.copied();   // Option<&i32> -> Option<i32>
let cloned: Option<String> = r.cloned();

// Most common real signature: you receive a reference, you return a borrow.
fn takes_ref_option(o: &Option<String>) -> Option<&str> {
    o.as_deref()
}

```

Why it matters: `&Option<T>` is a reference to the whole *Option*. You usually want the *Option* of a reference, which is `Option<&T>`. `as_ref` is the bridge, `as_deref` does the `String -> str` step at the same time.

2. Option combinators

```

let n: Option<i32> = Some(4);

// Transform the value. Closure gets T by value.
let doubled = n.map(|v| v * 2);           // Some(8)

// Transform and possibly fail. Closure returns Option.
let positive = n.and_then(|v| (v > 0).then_some(v)); // Some(4)
let lazy     = n.and_then(|v| if v > 100 { Some(v) } else { None });

// Replace None. map_or is eager, map_or_else is lazy. unwrap_or is for a plain value.
let defaulted = n.map_or(0, |v| v * 2);    // 8
let lazy_default = n.map_or_else(|| 0, |v| v * 2); // 8
let plain      = n.unwrap_or(0);          // 4
let plain_lazy = n.unwrap_or_else(|| 0);    // 4
let default_trait = n.unwrap_or_default(); // needs i32: Default

// filter takes &T, so deref the closure argument.
let filtered = n.filter(|v| *v > 10);      // None

// Combining two Options.
let chained = n.and(Some(9));              // Some(9): n is Some, so take the
arg
let alternative = n.or(Some(9));           // Some(4): n is Some, keep it
let paired     = n.zip(Some("a"));        // Some((4, "a"))

// Predicate by value.
let checked = n.is_some_and(|v| v > 3);   // true

// Mutating a slot in place.

```

```

let mut slot: Option<i32> = None;
let inserted: &mut i32 = slot.get_or_insert(7);    // inserts, returns &mut T
*inserted += 1;
let taken: Option<i32> = slot.take();              // moves out, leaves None
let old: Option<i32> = slot.replace(3);            // swaps in 3, returns the old value

```

Which one to say out loud: `map` transforms, `and_then` chains a second fallible step, `unwrap_or` is the default, `ok_or` converts to a `Result` so `?` can take over.

3. Result and ?

```

fn option_to_result() -> Result<i32, String> {
    let o: Option<i32> = Some(1);
    let v = o.ok_or_else(|| "missing".to_string())?; // Option -> Result -> ``
    Ok(v)
}

fn result_combinators() -> Result<i32, String> {
    let r: Result<i32, String> = Ok(1);
    let m = r.as_ref().map(|v| v + 1);           // borrow, do not move
    let e = r.clone().map_err(|err| err.len());
    let chain: Result<i32, String> = Ok(2).and_then(|v| Ok(v * 2));
    let o: Option<i32> = r.ok();                  // discards the error
    Ok(1)
}

// `` in main: no unwrap needed, and the error type is erased.
fn main() -> Result<(), Box<dyn std::error::Error>> {
    let n: i32 = "42".parse::<i32>()?;
    Ok(())
}

```

4. Iterator closures: `|x|` vs `&x` vs `*x`

The rule: some adapters take `Self::Item`, some take `&Self::Item`. The ones taking a reference hand you one extra `&`.

For `v.iter()` where `Item = &i32`:

Adapter	Closure argument
<code>map</code> , <code>filter_map</code> , <code>for_each</code> , <code>flat_map</code>	<code>&i32</code>
<code>any</code> , <code>all</code> , <code>position</code> , <code>fold</code> 's item	<code>&i32</code>
<code>filter</code> , <code>find</code> , <code>take_while</code> , <code>skip_while</code> , <code>partition</code>	<code>&&i32</code>
<code>max_by_key</code> , <code>min_by_key</code>	<code>&&i32</code>
<code>max_by</code> , <code>min_by</code>	<code>&&i32</code> , <code>&&i32</code>
<code>slice</code> <code>sort_by</code>	<code>&T</code> , <code>&T</code>
<code>slice</code> <code>sort_by_key</code> , <code>binary_search_by</code>	<code>&T</code>

The escape hatch: call `.copied()` or `.cloned()` first and every downstream closure works by value. That removes the whole problem.

```
let v = vec![3, 1, 4, 1, 5];

// Item is &i32. `x * 2` works because i32: Copy and auto-deref handles the rest.
let a: Vec<i32> = v.iter().map(|x| x * 2).collect();

// Three ways to filter, all valid. Pick one and stay consistent.
let b: Vec<i32> = v.iter().copied().filter(|x| *x > 2).collect(); // Item = i32
let cc: Vec<i32> = v.iter().filter(|x| **x > 2).copied().collect(); // Item = &i32
let d: Vec<i32> = v.iter().filter(|&x| x > 2).collect(); // destructure
let e: Vec<i32> = v.clone().into_iter().filter(|x| *x > 2).collect(); // owned

let found: Option<&i32> = v.iter().find(|x| **x == 4); // && i32
let idx: Option<usize> = v.iter().position(|x| *x == 4); // &i32
let any: bool = v.iter().any(|x| *x == 4); // &i32
let all: bool = v.iter().all(|x| *x > 0); // &i32
let mx: Option<&i32> = v.iter().max_by_key(|x| **x); // && i32
let taken: Vec<&i32> = v.iter().take_while(|x| **x > 1).collect();

let total: i32 = v.iter().fold(0, |acc, x| acc + x); // acc: i32, x: &i32
let summed: i32 = v.iter().sum();
let count = v.iter().count();
let mapped: Vec<String> = v.iter().map(|x| x.to_string()).collect();

let (small, big): (Vec<i32>, Vec<i32>) = v.iter().copied().partition(|x| *x > 2);
let zipped: Vec<(i32, i32)> = v.iter().copied().zip(v.iter().copied()).collect();

// Option inside a collection: flatten drops the Nones.
let flat: Vec<i32> = vec![Some(1), None, Some(2)].into_iter().flatten().collect();
let transposed: Option<i32> = Some(Some(1)).flatten();
```

`filter` does not change the Item type, even with `|&&x|`. That is error 1 above.

5. HashMap, HashSet, BTreeMap

```
use std::collections::{BTreeMap, HashMap, HashSet};

// The counting idiom. Memorise this exact line.
let words = ["a", "b", "a"];
let mut counts: HashMap<&str, u32> = HashMap::new();
for w in words {
    *counts.entry(w).or_insert(0) += 1;
}

// entry(): insert default, then modify in one lookup.
counts.entry("c").or_insert(0);
counts.entry("a").and_modify(|c| *c += 10).or_insert(1);

// Grouping: or_default() needs V: Default. Vec<u32> is.
let mut groups: HashMap<u32, Vec<u32>> = HashMap::new();
for n in [1u32, 2, 3, 4] {
    groups.entry(n % 2).or_default().push(n);
}

// get returns Option<&V>, never V. That is why you keep needing .copied().
let map: HashMap<i32, i32> = vec![(1, 2)].into_iter().collect();
let lookup: Option<&i32> = map.get(&1);
let value: Option<i32> = map.get(&1).copied();
let exists: bool = map.contains_key(&1);

let set: HashSet<i32> = vec![1, 1, 2].into_iter().collect();

// Ordered iteration needs BTreeMap. HashMap order is arbitrary.
let mut bt: BTreeMap<i32, &str> = BTreeMap::new();
bt.insert(1, "a");
let first = bt.first_key_value();
let in_range = bt.range(0..2).count();
```

Key requirements: `HashMap/HashSet` need `Hash + Eq`. `BTreeMap/BTreeSet` need `Ord`. `f64` has neither, so it cannot be a key in any of them.

6. Sorting, searching, dequeues

```
let mut v = vec![(1, "b"), (2, "a"), (1, "a")];

v.sort_by_key(|p| p.0); // stable, key: &T -> K
v.sort_by_key(|p| std::cmp::Reverse(p.0)); // descending without a custom comparator
v.sort_by(|a, b| a.0.cmp(&b.0).then(a.1.cmp(b.1))); // tie-break: Ordering::then
v.sort_unstable(); // fastest, order not preserved
v.sort_unstable_by(|a, b| b.0.cmp(&a.0)); // descending

let mut xs = vec![3, 1, 4, 1, 5];
xs.sort_unstable();
```



```

xs.dedup(); // only removes CONSECUTIVE duplicates

let idx: Result<usize, usize> = xs.binary_search(&4); // Ok(found) |
Err(insert_at)
let idx2: Result<usize, usize> = v.binary_search_by_key(&2, |p| p.0); // slice must be
sorted
let has = xs.contains(&4); // linear scan, no sort needed

// Floats: sort with total_cmp, never partial_cmp().unwrap(). NaN makes the latter panic.
let mut ys: Vec<f64> = vec![2.5, 1.0, 3.5];
ys.sort_by(|a, b| b.total_cmp(a)); // descending, NaN-safe, total order

// VecDeque is the queue and stack with O(1) at both ends.
let mut q = std::collections::VecDeque::new();
q.push_back(1);
q.push_front(0);
let front = q.pop_front();
let back = q.pop_back();

```

`sort_by_key` cannot return a borrowed value — use `sort_by` for that. `dedup` without a preceding sort does almost nothing.

7. BinaryHeap, Ord, and f64

`BinaryHeap<T>` requires `T: Ord`, which is why `f64` does not work: it only implements `PartialOrd`, because NaN has no ordering.

```

use std::cmp::{Ordering, Reverse};
use std::collections::BinaryHeap;

// Ord types are fine. This is a MAX-heap.
let mut max_heap: BinaryHeap<i32> = BinaryHeap::new();
max_heap.push(3);
max_heap.push(1);
let top: Option<i32> = max_heap.pop(); // Some(3) - largest first
let peek: Option<&i32> = max_heap.peek(); // borrow, does not remove
let from_vec: BinaryHeap<i32> = BinaryHeap::from(vec![1, 2, 3]);

// Min-heap: wrap in Reverse. pop() then gives Reverse<T>, so unwrap the field.
let mut min_heap: BinaryHeap<Reverse<i32>> = BinaryHeap::new();
min_heap.push(Reverse(5));
min_heap.push(Reverse(2));
let smallest = min_heap.pop().map(|r| r.0); // Some(2)

// Tuples are Ord when all elements are. This is a tie-broken key.
let mut pairs: BinaryHeap<(i32, usize)> = BinaryHeap::new();
pairs.push((1, 0));
pairs.push((2, 1));

// A heap of f64 needs a wrapper. All five items below are required:
// derive PartialEq, then impl Eq, PartialOrd, Ord.

```

```
#[derive(PartialEq)]
struct F(f64);

impl Eq for F {}

impl PartialOrd for F {
    fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
        Some(self.cmp(other))           // delegate: total_cmp is already a total order
    }
}

impl Ord for F {
    fn cmp(&self, other: &Self) -> Ordering {
        self.0.total_cmp(&other.0)     // never panics, handles NaN and -0.0
    }
}

let mut float_heap: BinaryHeap<F> = BinaryHeap::new();
float_heap.push(F(1.5));
float_heap.push(F(0.5));
let smallest_float = float_heap.pop().map(|f| f.0);
```

If the interviewer only wants a sorted list, skip the wrapper: `sort_by(|a, b| b.total_cmp(a))`. Reach for the heap when you need streaming top-k, and say why.

8. Locks, guards, and the deadlock I wrote

`Mutex::lock()` returns `LockResult<MutexGuard<T>>`, so it needs `unwrap()`, and the guard is what derefs to `T`.

```
use std::sync::{Arc, Mutex, RwLock};
use std::sync::atomic::{AtomicUsize, Ordering};

let shared = Arc::new(Mutex::new(vec![1]));

let mut handles = Vec::new();
for i in 0..4 {
    let shared = Arc::clone(&shared);           // clone the handle, move it in
    handles.push(std::thread::spawn(move || {
        let mut guard = shared.lock().unwrap(); // MutexGuard<Vec<i32>>
        guard.push(i);                          // method call auto-derefs
        *guard = vec![i];                      // explicit deref to replace the value
    }));                                       // guard drops here -> unlock
}
for h in handles { h.join().unwrap(); }

// Poisoning: lock() is a Result. Propagate, or take the data back.
let ignoring = shared.lock().unwrap_or_else(|e| e.into_inner());

// THE TRAP. These two look identical and are not.
let _ = shared.lock();                      // guard dropped at the end of THIS statement: not locked
```

```
let _g = shared.lock();    // guard held until end of scope: intended

// THE OTHER TRAP. `lock()` returns a guard, so binding it holds the lock for the
// rest of the body. This deadlocks on the final line, same thread, no warning:
// let again = shared.lock().unwrap();
// let g2 = shared.lock().unwrap();    // hangs forever
// Bind the DATA instead, or scope the guard:
let len = { let g = shared.lock().unwrap(); g.len() };
```

```
// RwLock: many readers OR one writer.
let rw = Arc::new(RwLock::new(0));
{
    let r1 = rw.read().unwrap();
    let r2 = rw.read().unwrap();    // two readers coexist
}
{
    let mut w = rw.write().unwrap();
    *w += 1;
}

// Atomics: every operation takes an Ordering. Forgetting it is a compile error.
let counter = AtomicUsize::new(0);
counter.fetch_add(1, Ordering::Relaxed);
let loaded = counter.load(Ordering::SeqCst);
let swapped = counter.swap(5, Ordering::AcqRel);
let cas: Result<usize, usize> =
    counter.compare_exchange(5, 6, Ordering::AcqRel, Ordering::Relaxed);
```

Which to reach for:

Need	Type	Notes
Counter across threads	<code>AtomicUsize</code>	needs an <code>Ordering</code> every call
Counter, single thread	<code>Cell<T></code>	T: Copy only, get/set
Shared mutable state, single thread	<code>RefCell<T></code>	<code>borrow/borrow_mut</code> , panics on conflict
Shared mutable state, threads	<code>Mutex<T></code>	blocking, poisons on panic
Read-mostly across threads	<code>RwLock<T></code>	many readers or one writer
Single-thread shared ownership	<code>Rc<T></code>	not <code>Send</code>
Cross-thread shared ownership	<code>Arc<T></code>	clone the handle, not the data

Lock guard held across `.await` is not `Send` — that needs `tokio::sync::Mutex`.

9. Strings

```
let s: String = "hi there".to_string();
let s2: String = String::from("hi");
let s3: String = "hi".to_owned();

let r: &str = &s;           // deref coercion: String -> &str
let r2: &str = s.as_str();  // the explicit form
let own: String = r.to_string(); // &str -> String

// parse returns Result, so it needs `?` or an unwrap in a function.
let n: i32 = "42".parse::<i32>()?;
let with_default = "42".parse::<i32>().unwrap_or(0);
let as_option     = "42".parse::<i32>().ok();

let parts: Vec<&str> = s.split_whitespace().collect();
let joined = parts.join(",");
let upper  = s.to_uppercase();
let first: Option<char> = s.chars().next();    // bytes vs chars are different
let nth:   Option<char> = s.chars().nth(1);
let reversed: String = s.chars().rev().collect();
let formatted = format!("{s} {n}");

let mut m = String::new();
m.push('a');
m.push_str("bc");
```

`s[0]` gives a *byte* and panics on a non-ASCII boundary. Use `s.chars().nth(i)`. Function parameters should be `&str`, struct fields should be `String`.

10. Match ergonomics

Matching a reference makes the bindings references automatically. You rarely need `&` in the pattern.

```
match &opt {
    Some(x) => { let _: &String = x; } // x is &String, no `&` written
    None => {}
}

match &mut mopt {
    Some(x) => x.push('!'), // x is &mut String
    None => {}
}

match opt {
    Some(x) => { let _: String = x; } // x is String: this one MOVES
    None => {}
}
```

```

if let Some(x) = &mopt {}           // x: &String
while let Some(x) = stack.pop() {} // x: i32, owned
let Some(x) = mopt else { return }; // let-else: bind or leave

if let Some(Ok(v)) = Some(Ok::<i32, String>(1)) {} // nested patterns in one line

let s = String::from("quit");
match &s[..] {                      // &str patterns need a slice, not &String
    "quit" => {}
    _ => {}
}

match n {
    x @ 1..=5 => { let _ = x; }     // bind and test at once
    _ => {}
}

```

When the compiler says `expected &String, found String`, add `&` to the scrutinee (`match &mopt`) rather than to the pattern.

11. Escaping the borrow checker

```

let mut v = vec![1, 2, 3, 4];

// Two mutable borrows into one slice, without cloning.
let (left, right) = v.split_at_mut(2);
left[0] = 9;
right[0] = 8;

// Read and write in one expression does not compile. Copy the index out first.
let i = 1usize;
let value = v[i];
v[i] = value + 1;

for x in &mut v { *x += 1; } // &mut i32, deref to assign
for x in v.iter().copied() { } // by value, no borrow held

// Move a value out of a slot that is borrowed elsewhere.
let mut owned = vec![1, 2, 3];
let taken = std::mem::take(&mut owned); // leaves Default (empty)
let replaced = std::mem::replace(&mut owned, vec![7]); // leaves vec![7]

let mut opt = Some(vec![1]);
let moved =
opt.take(); // Option<T> -> Option<T>, leaves None

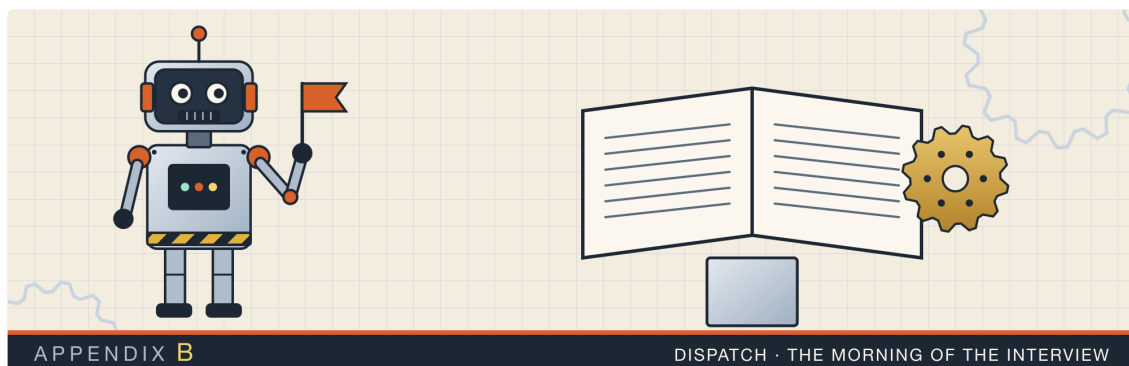
let snapshot = v.len(); // copy the value out
let copy = v.clone(); // the pragmatic unblock when you are short on time

```

Order of attack when the borrow checker fights you: copy the value out, then `split_at_mut`, then `mem::take`, then `clone`. Save `clone` for when the clock matters and say out loud that you know it is a clone.

12. Final checklist of syntax slips

- `Vec::pop`, `HashMap::get`, `stack.pop()`, `BinaryHeap::pop` all return `Option`. Handle it.
- `.iter()` gives `&T`; add `.copied()` / `.cloned()` before the closures.
- `filter`, `find`, `take_while`, `max_by_key` give `&&T`.
- The binding needs `mut`, not the value: `let mut v = ...`.
- `.collect()` needs a target type: `let v: Vec<_> = ...` or `collect::<Vec<_>>()`.
- Integer literals default to `i32`; annotate `0usize`, `vec![0u32; n]` when indexing.
- Indexing needs `usize`: `v[x as usize]`.
- Unannotated float literals can stay ambiguous until a method pins the type.
- `Option/Result` combinators take `self` and `move`. Use `.as_ref()` to keep the value.
- Locks return guards. Bind the data, not the guard. `let _ = lock()` unlocks instantly.
- `#[derive(Debug, Clone, PartialEq)]` on any struct you plan to print or assert on.
- `assert_eq!` needs `Debug` and `PartialEq` on both sides.
- `.to_string()` needs `Display`; `format!("{x:?}")` needs `Debug`.
- Closure needs `move` only to cross a thread boundary or to return it.
- `Arc::clone(&x)` inside the loop, then `move` the clone into the spawn.



Appendix B. The benchmark report

Chapters 4 and 9 quote measurements from the Rust lab's benchmark program. This appendix reproduces the full report written from those runs: `rust-interview-lab/benchmarking_examples/benchmarking-report.md`. It presents them as a short paper, with method, results, discussion, and threats to validity. The recorded program output it relies on is in `rust-interview-lab/BENCHMARKS.md`.

A correction to sections 2.2 and 5.2 of the report. They attribute the enum list's slower push to a discriminant stored alongside the boxed continuation. On a 64-bit target, `std::mem::size_of::<ListNode<u64>>()` is 16, the same as `Node<u64>`: the `Box` field's non-null niche encodes `Empty`, so there is no separate discriminant. Chapter 4, section 4.3, gives the two structural differences that remain (the inline head moved on every push, and the extra allocation for the final `Empty`). The measured timings are unaffected.

The report is built as its own PDF by this script and stylesheet:

Listing B.1 The report's build script. `benchmarking_examples/build_report_pdf.py`

```
#!/usr/bin/env python3
"""Render `benchmarking-report.md` as a PDF.

    python3 benchmarking_examples/build_report_pdf.py

Markdown is rendered by Python-Markdown with Pygments for the code blocks, the
result is wrapped in one HTML document, and WeasyPrint paginates it using
`report.css`. Requires `markdown`, `pygments`, and `weasyprint`.
"""

from __future__ import annotations
import pathlib
```

```

import sys

import markdown
from pygments.formatters import HtmlFormatter
from weasyprint import CSS, HTML

HERE = pathlib.Path(__file__).resolve().parent
SOURCE = HERE / "benchmarking-report.md"
STYLESHEET = HERE / "report.css"
OUTPUT = HERE / "benchmarking-report.pdf"
TITLE = "A Comparative Study of LRU Cache Storage Strategies and Singly Linked List
Deallocation"

EXTENSIONS = ["extra", "codehilite", "sane_lists"]
CONFIG = {"codehilite": {"guess_lang": False, "css_class": "highlight"}}

# The title block is written as ordinary Markdown paragraphs. These identifiers
# let the stylesheet centre them and distinguish them from body text.
TITLE_BLOCK = (
    ('<p><strong>Arpan Pathak</strong></p>', '<p id="title-author"><strong>Arpan Pathak</
strong></p>'),
    ('<p>11 September 2026</p>', '<p id="title-date">11 September 2026</p>'),
    (
        '<p><strong>Keywords:</strong>',
        '<p id="title-keywords"><strong>Keywords:</strong>',
    ),
)

def build_document() -> str:
    """The whole report as one HTML document."""
    body = markdown.markdown(
        SOURCE.read_text(encoding="utf-8"), extensions=EXTENSIONS, extension_configs=CONFI
G
    )

    for old, new in TITLE_BLOCK:
        body = body.replace(old, new, 1)

    style = HtmlFormatter(style="friendly").get_style_defs(".highlight")
    return "\n".join(
        [
            "<!doctype html>",
            '<html lang="en">',
            "<head>",
            '<meta charset="utf-8">',
            f"<title>{TITLE}</title>",
            '<meta name="author" content="Arpan Pathak">',
            f"<style>{style}</style>",
            "</head>",
            "<body>",
            body,
            "</body>",
            "</html>",
        ]
    )

def main() -> int:

```



```

if not SOURCE.exists():
    print(f"missing {SOURCE}", file=sys.stderr)
    return 1

document = build_document()
stylesheet = CSS(filename=str(STYLESHEET))
rendered = HTML(string=document, base_url=str(HERE))
page_count = len(rendered.render(stylesheets=[stylesheet]).pages)
rendered.write_pdf(target=str(OUTPUT), stylesheets=[stylesheet])

print(f"wrote {OUTPUT.name} ({page_count} pages)")
return 0

if __name__ == "__main__":
    sys.exit(main())

```

Listing B.2 The report's stylesheet. `benchmarking_examples/report.css`

```

/* Print stylesheet for the benchmarking report.
   Rendered by WeasyPrint, which implements the paged-media parts of CSS. */

@page {
    size: A4;
    margin: 22mm 20mm 20mm 20mm;

    @bottom-center {
        content: counter(page);
        font-family: "DejaVu Sans", Helvetica, sans-serif;
        font-size: 8.5pt;
        color: #666;
    }
}

html {
    font-size: 10.5pt;
}

body {
    font-family: "DejaVu Serif", Georgia, serif;
    line-height: 1.5;
    color: #111;
    margin: 0;
    text-align: left;
}

h1,
h2,
h3 {
    font-family: "DejaVu Sans", Helvetica, sans-serif;
    color: #000;
    line-height: 1.25;
}

/* Title block: centered, in the manner of a journal article. */
h1 {
    font-size: 16pt;
    text-align: center;
}

```

```

    margin: 0 0 0.7em 0;
}

#title-author {
    text-align: center;
    font-size: 12pt;
    margin: 0.5em 0 0 0;
}

#title-date {
    text-align: center;
    font-size: 10pt;
    color: #444;
    margin: 0.15em 0 0 0;
}

#title-keywords {
    font-size: 9.5pt;
    color: #333;
    margin: 0.9em 0 0.4em 0;
}

h2 {
    font-size: 12.5pt;
    margin: 1.7em 0 0.5em 0;
    padding-bottom: 0.12em;
    border-bottom: 0.6pt solid #bbb;
    break-after: avoid;
}

h3 {
    font-size: 11pt;
    margin: 1.1em 0 0.4em 0;
    break-after: avoid;
}

p {
    margin: 0 0 0.7em 0;
    text-align: justify;
}

table {
    width: 100%;
    border-collapse: collapse;
    margin: 0.7em 0 1.1em 0;
    font-size: 9.5pt;
    break-inside: avoid;
}

th {
    text-align: left;
    font-family: "DejaVu Sans", Helvetica, sans-serif;
    font-size: 8.5pt;
    letter-spacing: 0.03em;
    text-transform: uppercase;
    color: #333;
    border-bottom: 1pt solid #333;
    padding: 0.35em 0.5em 0.3em 0.5em;
}

```

```

td {
  border-bottom: 0.4pt solid #ddd;
  padding: 0.35em 0.5em;
  vertical-align: top;
}

pre {
  background: #f6f6f6;
  border: 0.4pt solid #ddd;
  padding: 0.6em 0.7em;
  font-size: 8.6pt;
  line-height: 1.35;
  white-space: pre-wrap;
  break-inside: avoid;
}

code {
  font-family: "DejaVu Sans Mono", monospace;
  font-size: 0.92em;
}

p code,
li code {
  background: #f2f2f2;
  padding: 0 0.15em;
  /* An identifier split across two lines is a defect, and the longest token in
     this report is short enough to fit on a line of the text block. */
  white-space: nowrap;
}

td code,
th code {
  background: #f2f2f2;
  padding: 0 0.15em;
}

h1 code {
  white-space: nowrap;
}

hr {
  border: none;
  border-top: 0.6pt solid #bbb;
  margin: 1.6em 0;
}

ol,
ul {
  margin: 0 0 0.8em 1.3em;
  padding: 0;
}

li {
  margin-bottom: 0.3em;
  text-align: justify;
}

h2 + p,

```

```
h2 + table,  
h2 + pre,  
h3 + p {  
    margin-top: 0;  
}
```

A Comparative Study of LRU Cache Storage Strategies and Singly Linked List Deallocation

Arpan Pathak

11 September 2026

Abstract

Two experiments are reported. The first compares two storage strategies for a fixed-capacity least-recently-used cache: an index arena, in which nodes are held in a contiguous vector and linked by integer offsets, and a reference-counted doubly linked list, in which each node is a separate heap allocation linked by pointers. On a workload of ten million requests with a measured hit rate of 89.8 percent, the arena completed in 1.00 s against 1.86 s for the reference-counted list, a ratio of 1.87. The magnitude of the difference is bounded by the hash table lookup that both implementations perform on every request; across access patterns the ratio ranged from 1.1 to 1.9.

The second experiment compares four singly linked list variants that combine two node representations with two deallocation strategies. The variants that rely on compiler-generated deallocation cannot free more than approximately 265,000 nodes on an 8 MiB stack, because freeing a node frees its successor and the recursion consumes one stack frame per element; beyond that size the process terminates with a stack overflow. The iterative variants free five million nodes without incident. The iterative strategy is not substantially faster, saving about 0.3 ms on a 3.2 ms deallocation at 200,000 nodes; its principal contribution is the removal of the size limit.

The measurement procedure, including the treatment of allocation warm-up, and the threats to the validity of both results are discussed.

Keywords: cache replacement, index arena, reference counting, linked list, stack overflow, benchmarking methodology

1. Introduction

A cache that must bound memory occupancy needs a replacement policy, and the least-recently-used policy requires a recency order over the stored entries. The order is conventionally maintained by a doubly linked list, and the two decisions that determine the cost of that list are how nodes are allocated and how links between them are represented.

The first decision concerns ownership. A node may be owned individually, as one heap allocation reached through a pointer, or collectively, as one element of a contiguous array. Individually owned nodes support stable addresses and permit removal without disturbing the rest of the structure, at the cost of one allocation per node and a reference count or equivalent mechanism to express ownership. Collectively owned nodes require the allocator to be invoked once, and express the identity of a node by its index, at the cost of a level of indirection.

The second decision concerns deallocation. A recursively defined structure releases its storage through a recursive destructor unless the recursion is replaced by an iterative walk. This choice is frequently treated as a matter of performance, and the present study finds that the performance difference is small. Its consequences are otherwise: the recursive form has a maximum size beyond which the program terminates.

This document reports measurements of both decisions. Section 2 describes the structures. Section 3 gives the experimental method. Section 4 presents the results, Section 5 discusses them and states the threats to their validity, and Section 6 concludes.

2. Background

2.1 Storage for a fixed-capacity LRU cache

Both cache implementations under study maintain a hash table that maps a key to a node, and a doubly linked list that orders the nodes by recency. Insertion of a new entry when the cache

is full removes the least recently used node. A lookup that succeeds moves the located node to the most recently used end.

In the index arena implementation, the nodes are elements of a `Vec`, and the links are values of type `Option<usize>`. The hash table maps a key to the index of its node. Removal of the least recently used entry reuses that entry's slot rather than deallocating it, so the arena is allocated once, grows to its capacity, and is never resized during operation. The implementation is `benchmarking_examples/cache/arena.rs`.

In the second implementation, each node is allocated separately and reached through a reference-counted cell. Forward links hold strong references and backward links hold weak references, which prevents ownership cycles. Removal of the least recently used entry removes the tail node from the hash table and detaches it, after which the new entry is allocated as a fresh node. The implementation is `benchmarking_examples/cache/rc_list.rs`.

Both implementations satisfy the same trait, `cache::Cache`, and are therefore exercised by one workload.

2.2 Node representation and deallocation in a singly linked list

Four variants of a singly linked list are compared. They share push and pop operations and differ in two independent properties.

Variant	Node representation	Deallocation
<code>box</code>	<code>Option<Box<Node<T>>></code>	compiler-generated, recursive
<code>enum</code>	<code>enum ListNode<T></code>	compiler-generated, recursive
<code>box+drop</code>	<code>Option<Box<Node<T>>></code>	explicit, iterative
<code>enum+drop</code>	<code>enum ListNode<T></code>	explicit, iterative

The two representations have different sizes. `Option<Box<Node<T>>>` occupies one machine word on a 64-bit target, because a boxed value is never null and the all-zero address is therefore available to encode the empty case. The enum representation carries a discriminant in addition to the boxed continuation, and is consequently larger. The variant definitions are in `benchmarking_examples/lists/`.

The deallocation strategies differ in their recursion. The compiler-generated destructor for a node releases the node, whose destructor releases its successor, and so on, consuming one

stack frame per element. The iterative strategy replaces the head of the list with the empty state and then walks the chain, detaching each node before it is freed, so no destructor is called with the remainder of the list still attached.

3. Method

3.1 Environment

Property	Value
Processor	AArch64, 8 cores, development board
Compiler	<code>rustc 1.96.0-nightly</code>
Build profile	<code>--release</code>
Concurrency	one thread per measurement
Default stack size	8 MiB

Absolute times are properties of this machine and this toolchain. The ratios and the maximum list size are the quantities intended to generalise, the latter only within the constraint of the stack size in force.

3.2 Workloads

Cache workload. A fixed-capacity cache of 65,536 entries receives ten million requests over a key space of 131,072 keys. Four requests in five are drawn from a hot set comprising one percent of the key space, and the remainder are drawn uniformly over the whole key space. The request sequence is generated from a fixed seed, so both implementations receive an identical sequence. A request that misses performs an insertion, which evicts the least recently used entry once the cache has reached capacity. The measured hit rate was 89.8 percent, comprising 8,979,314 hits and 1,020,686 insertions.

Linked list workload. Each variant is measured at 200,000 elements, a size at which all four complete. The two iterative variants are additionally measured at 5,000,000 elements. The recursive variants are measured at incrementally increasing sizes until termination occurs, to establish the size at which deallocation fails.

3.3 Measurement procedure

An initial observation established that a single timing pass is not reproducible on this platform. Timings of one push loop of fixed size varied between 2.3 ms and 11.4 ms depending on whether the measurement was the first large allocation performed by the process. The variance is attributable to the allocator and the operating system rather than to the structure under test: a first pass over a fresh heap requires pages to be mapped, and that cost is charged to the structure being measured.

Accordingly, each measurement is preceded by a warm-up pass of the same magnitude as the measured pass, and then repeated three times, with the shortest of the three reported. The shortest pass is preferred to the mean because the quantity of interest is the cost of the work, while a lengthened pass reflects interference from outside the process. This procedure follows the practice recommended for benchmarking where the number of repetitions is constrained [3, 4]. The residual variation between runs of the cache experiment, computed over the reported configuration, is approximately three percent.

Measurements are normalised per request for the cache experiment and per element for the list experiment, so that the two experiments can be reported in comparable units.

3.4 Verification and controls

Each measurement is accompanied by a check that the intended work was performed.

In the list experiment, every value inserted is removed and compared with the expected value, so that a structure which loses or reorders elements cannot report a time. The verification loop is not included in any timed interval.

In the cache experiment, the two implementations process the same request sequence and their hit counts are compared before any timing is reported. Agreement establishes that both implementations were given the same problem.

The variants `box` and `box+drop` share an identical push implementation and differ only in deallocation. Their push times therefore serve as a control: a discrepancy beyond the run-to-run variation indicates a fault in the measurement rather than a difference between the structures.

4. Results

4.1 Cache storage strategies

LRU cache: 65,536 entries, 10,000,000 requests over 131,072 keys

hit rate: 89.8% (8,979,314 hits, 1,020,686 inserts)

storage	total	per request
arena (Vec + indices)	1.00s	100 ns
Rc<RefCell> list	1.86s	186 ns

The arena required 1.00 s and the reference-counted list 1.86 s, giving a ratio of 1.87 in favour of the arena. Measured per request, the costs are 100 ns and 186 ns respectively. Repeated runs of the same configuration produced ratios between 1.81 and 1.87.

4.2 Deallocation in singly linked lists

All four variants completed at 260,000 elements. At 270,000 elements the two recursive variants terminated with a stack overflow and the two iterative variants completed.

Variant	260,000 elements	270,000 elements
box	completed	stack overflow
enum	completed	stack overflow
box+drop	completed	completed
enum+drop	completed	completed

The termination message recorded for the recursive variants is:

```
thread 'main' has overflowed its stack
fatal runtime error: stack overflow, aborting
```

On an 8 MiB stack, the limit therefore lies between 260,000 and 270,000 elements, corresponding to approximately 32 bytes of stack per element.

Timings at 200,000 elements, where all four variants complete:

variant	elements	push	drop
---------	----------	------	------

box	200,000	2.40ms	3.23ms
enum	200,000	3.42ms	3.20ms
box+drop	200,000	2.32ms	2.91ms
enum+drop	200,000	3.62ms	2.52ms

Timings at 5,000,000 elements, where only the iterative variants complete:

box+drop	5,000,000	58.11ms	72.32ms
enum+drop	5,000,000	90.98ms	67.51ms

Normalised per element at 200,000 elements, insertion costs 12.0 ns for the boxed node and 17.1 ns for the enum node, and deallocation costs 16.2 ns, 16.0 ns, 14.6 ns and 12.6 ns for the four variants in the order listed.

5. Discussion

5.1 Interpretation of the cache result

The arena was faster by a factor of 1.87 on the reported workload, and the mechanism is visible in the operations each implementation performs when a lookup succeeds. The arena writes two indices into vector elements and reads two more; no memory is allocated or released and no reference count is modified. The reference-counted list clones a handle, upgrades a weak handle, borrows a `RefCell`, writes through the borrow, and drops the clone. Each step is small, but all of them occur on every successful lookup, and the node they operate on must be reached through a pointer to a separate allocation.

Eviction differs in the same direction. The arena reuses the least recently used slot. The list frees the tail node and allocates a node for the incoming key, an allocation and a deallocation that the arena does not perform.

The ratio is nevertheless smaller than the difference in list manipulation alone would suggest, because every request performs a hash table lookup and that cost is identical in both implementations. The measured ratio therefore depends on the proportion of each request that is spent on list maintenance. Across access patterns, the ratio ranged from 1.1 to 1.9; a stream with no reuse tends towards the lower end, since such a stream is dominated by lookups that miss and by evictions.

5.2 Interpretation of the list result

The difference in insertion cost between the two node representations, 12.0 ns against 17.1 ns per element, is consistent with the difference in node size. Both representations perform one allocation per element; the enum node additionally carries a discriminant, so each insertion moves more memory.

The difference in deallocation cost between the recursive and the iterative strategies is small, 0.32 ms and 0.68 ms at 200,000 elements, against totals of 3.23 ms and 3.20 ms. The iterative strategy is therefore not a performance measure. Its contribution is that it removes the maximum size: the recursive variants terminate above approximately 265,000 elements on an 8 MiB stack, and the iterative variants complete at 5,000,000, the largest size tested.

5.3 Threats to validity

Internal validity. The measurements are taken on a single machine, with one implementation measured after another in the same process. Allocation warm-up was identified as a source of variation and is controlled by the procedure described in Section 3.3, and the two list variants that share an insertion implementation provide a control on that stage. The deallocation measurements remain the least stable part of the results, since they follow a sequence of allocations and deallocations of the same memory.

External validity. Absolute times are specific to this processor, compiler and allocator. The maximum list size is a property of the stack size in addition to the code; a program running on a thread with a smaller stack would fail at a smaller size, and one with a larger stack at a larger size. The value of 5,000,000 reported for the iterative variants is the largest size tested and not an observed limit.

Construct validity. The cache fn wrapped experiment measures request latency for both a lookup and an insertion, but reports only the aggregate. It does not separate the cost of the hash table lookup from the cost of list maintenance, and it does not report peak memory, although the two node representations differ in size. The list experiment measures insertion, removal and deallocation, and does not measure iteration, search, or insertion at an arbitrary position.

Generality. Both experiments are single-threaded and therefore characterise neither contention nor synchronisation cost. A shared arena would require sharding or locking, and the cost of that mechanism is not represented here.

6. Conclusion

For a fixed-capacity LRU cache, an index arena was faster than a reference-counted linked list by a factor of 1.87 on the reported workload, with the difference attributable to the per-node bookkeeping and the allocation performed on eviction. The ratio is bounded by the hash table lookup common to both implementations, and ranged from 1.1 to 1.9 across access patterns.

For a singly linked list, an iterative deallocation strategy is worth adopting for robustness rather than for speed. It saved about ten percent of the deallocation time at the size measured, and it raised the maximum list size from approximately 265,000 elements to beyond 5,000,000, the largest size tested.

Further work would separate lookup cost from list maintenance cost by timing the two phases independently, report peak memory for each variant, and extend the cache experiment to a multi-threaded configuration in which the synchronisation cost of the arena becomes measurable.

References

1. The Rust Reference. Type layout. On size, alignment, and the null-pointer optimisation.
2. The Rust Standard Library. `std::mem::size_of`, `std::mem::align_of`, and `std::thread::Builder::stack_size`.
3. T. Kalibera and R. Jones. Rigorous Benchmarking in Reasonable Time. *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*, 2013.
4. A. Georges, D. Buytaert, and L. Eeckhout. Statistically Rigorous Java Performance Evaluation. *Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2007.
5. T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney. Producing Wrong Data Without Doing Anything Obviously Wrong! *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2009.

Appendix A. Source files

File	Contents
<code>benchmarking_examples/cache/mod.rs</code>	The <code>Cache</code> trait
<code>benchmarking_examples/cache/arena.rs</code>	Index arena implementation
<code>benchmarking_examples/cache/rc_list.rs</code>	Reference-counted implementation
<code>benchmarking_examples/lists/mod.rs</code>	The <code>SinglyList</code> trait
<code>benchmarking_examples/lists/boxed.rs</code>	Boxed node, recursive deallocation
<code>benchmarking_examples/lists/enum_node.rs</code>	Enum node, recursive deallocation
<code>benchmarking_examples/lists/boxed_drop.rs</code>	Boxed node, iterative deallocation
<code>benchmarking_examples/lists/enum_drop.rs</code>	Enum node, iterative deallocation
<code>benchmarking_examples/benchmark.rs</code>	The measurement program
<code>rust-interview-lab/BENCHMARKS.md</code>	Captured program output